

Mixed-Signal and Digital IC Design

A Graduate Theory Volume

A graduate-level theory volume for mixed-signal and digital integrated-circuit design. It carries the analysis a working designer must be able to do from first principles — device operating regions, delay, metastability, noise, sampling, data conversion, clocking, links, matching and reliability — with every number computed at build time rather than quoted.

Theory only. The practice volume (design-house workflow, deliverables, pricing) and the source-code appendix are in the companion book, *Semiconductor IP Design*. Nothing here depends on them.

How to read. Green boxes recall undergraduate material; purple boxes carry graduate detail; red boxes mark places where real designs have failed.

Contents

Z1. How a Chip Is Described: Levels of Abstraction	7
Z2. Boolean Algebra and Combinational Logic	9
Z3. Sequential Logic and State.....	11
Z4. Circuit Analysis: the Minimum Needed	13
A1. Signals, Systems and Linearity	14
A2. Transforms and Spectral Representation	17
A3. Sampling, Reconstruction and Quantisation	19
A4. Probability, Random Processes and Noise	21
A5. Linear Algebra and Numerical Methods for Hardware	23
B1. The MOSFET — What a Digital Designer Must Know	25
B2. CMOS Logic, Delay and Power	27
B3. Timing, Clocking and Static Timing Analysis.....	29
B4. Interconnect, Transmission Lines and Signal Integrity.....	31
B5. Analogue Building Blocks.....	34
C1. Arithmetic Units — The Core of Every Datapath.....	36
C2. Control: State Machines, Pipelines and Flow Control.....	39
C3. Clock Domain Crossing, Reset and Low Power	41
D1. Instruction Set Architecture	44
D2. Pipelining, Hazards and Speculation	46
D3. Memory Hierarchy and Coherence.....	48
E1. Digital Filters as Hardware	50
E2. Transforms in Hardware: FFT and Friends	53
F1. The Wireline Link, End to End.....	55
F2. Error Control Coding.....	58
G1. Electromagnetics for the Digital Engineer	61
G2. Radar: Signal Processing as an IP Block Family.....	63
H1. Verification: Theory and Organisation	66
H2. The Implementation Flow	69
H3. Hardware Security.....	71

I1. Information Theory	73
I2. Algebraic Structures for Hardware	75
I3. Optimisation and Numerical Methods	77
I4. Discrete Mathematics and Graphs in Hardware	78
J1. On-Chip Memory	79
J2. Accelerators: Systolic Arrays and the Roofline.....	80
J3. On-Chip Interconnect and Networks	83
J4. Image and Video Coding	85
K1. Wireless Physical Layers	86
K2. Protocol Stacks as Hardware.....	88
K3. Reliability, Safety and Yield.....	90
K4. Current Research Directions	92
L1. Semiconductor Physics: What Survives into Design	93
L2. Power Delivery and Conversion.....	95
L3. Thermal and Packaging	96
L4. Feedback and Control.....	97
M1. The Hardware–Software Interface.....	98
M2. Machine Learning Hardware.....	100
M3. Image Signal Processing	102
M4. Measurement and Characterisation.....	103
N1. Asynchronous and Globally Asynchronous Design	104
N2. Physical Design in More Depth.....	105
N3. Statistics for Silicon.....	106
N4. Optical Links and Photonics.....	107
Q1. Performance Modelling and Queueing	108
Q2. Compilers and High-Level Synthesis Internals.....	110
Q3. Embedded Software Around the Hardware.....	112
Q4. Electromagnetic Compatibility	113
R1. Cryptographic Mathematics.....	114
R2. Manufacturing Test	116
R3. Quantum Computing: What a Hardware Engineer Should Know.....	117

P1. MIPI CSI-2 and DSI — Camera and Display Interfaces	118
P2. PCI Express, Generations 3 to 6	121
P3. The Gigabit Ethernet MAC and Its Transceiver Interfaces	124
S1. Storage Interfaces	127
S2. Display and Audio Interfaces	128
S3. Clock, Reset and Power Architecture of an IP Block.....	129
S4. Packaging an IP for Delivery	131
X1. Fixed-Point Arithmetic, Worked End to End	133
X2. Timing Closure Arithmetic, Worked	140
X3. Metastability and Clock-Domain Crossing, Worked	146
X4. The Wireline Link Budget, Worked	151
X5. Reed–Solomon Decoding, Traced and Measured	157
X6. The FFT as a Hardware Block, Worked	162
X7. Power and Energy, Worked	166
X8. The Mathematics of Verification, Worked	170
X9. Computer Arithmetic Beyond the Adder, Worked	175
X10. Memory: The Block That Decides the Floorplan	180
X11. Interconnect: Arbitration, Flow Control and Deadlock	184
X12. Data Converters and PLLs, Worked	189
X13. Machine-Learning Hardware, Worked	193
X14. Physical Design as the RTL Designer Sees It	197
X15. Hardware Security, Worked	200
X16. Reliability, Yield and Functional Safety, Worked	203
X17. Reading the Current Literature	207
X18. Performance Modelling: Queues, Bursts and Tails	210
X19. Processors as IP, Worked.....	213
X20. Image and Video Coding as a Hardware Block	216
X21. Feedback and Control Inside Digital Blocks	219
X22. Resampling and Timing Recovery, Worked.....	222
X23. Complex Analysis and Transforms, for People Who Build Them.....	225
X24. Numerical Linear Algebra in Finite Precision	230

X25. Detection and Estimation, the Receiver's Mathematics	234
X26. Optimisation, the Tool Behind Every EDA Algorithm	238
X27. Design for Test: Scan, BIST and ATPG, Worked	242
X28. Caches and Coherence, Worked	247
X29. LDPC and Polar Codes, Worked	251
X30. OFDM and MIMO, Worked	255
X31. Analogue Building Blocks, Budgeted	259
X32. The Power Delivery Network, Worked	263
X33. Boolean Reasoning: BDDs, SAT and Equivalence Checking	266
X34. Logic Synthesis: What the Tool Is Actually Doing	269
X35. Hardware Data Structures: CAM, Hashing and Bloom Filters	272
X36. Cryptographic Engines, Sized	276
X37. Bus Protocols: Ordering, Outstanding Transactions and Exclusives	280
X38. Colour, the ISP Pipeline and Display Processing	284
X39. Clock Generation, Distribution and Time Synchronisation	288
X40. The Coverage Map: Which Theory an IP Block Uses	291
X41. DRAM: The Timing That Shapes Every Memory Controller	294
X42. Reset, Initialisation and the States Nobody Simulates	297
X43. Power Intent: Domains, Retention and Isolation	300
Z17. From One Transistor to Our Datapath	303
T1. The MOSFET as Four Different Machines	311
T2. Delay — From One RC to a Whole Timing Path	321
T3. Metastability — the One Failure You Cannot Design Away	333
T4. Noise — and Why It Sets the Size of Everything	342
T5. Sampling — Where a Continuous Signal Becomes a Number	348
T6. Data Converters — Where Each Architecture Spends the Same Limits	353
T7. The Phase-Locked Loop — a Feedback System Whose Output Is Time	357
T8. The Serial Link — Measured, Including Where It Does Not Work	363
T9. The Channel — Where the Loss Comes From, Term by Term	368
T10. Switched Capacitors — Accuracy Bought with Time	373
T11. Bias and References — Making a Number That Does Not Move	377

T12. Matching — Two Identical Devices Are Not Identical.....	383
T13. Power Integrity — the Network Everything Else Sits On.....	387
T14. Reliability — What the Circuit Looks Like After Ten Years.....	391
T15. Layout — Where a Correct Schematic Can Still Fail.....	396
T16. ESD — the Failure That Happens Before the Chip Is Powered.....	400
T17. The Implementation Flow — What Each Stage Decides.....	404
T18. Floorplan and Power Plan — The Geometry Everything Sits On	410
T19. Placement, Clock Tree and Routing	418
T20. Signoff — The Checks That Decide Whether It May Be Built.....	428
T21. The Verification Environment — What It Proves	435
T22. From DFT to the Tester — Time, Cost and What Escapes.....	442
T23. The Flow, Run End to End — One Block, Nine Stages, Real Numbers.....	448

Z1. How a Chip Is Described: Levels of Abstraction

This chapter is the gentlest in the book and the one that everything else refers back to. A modern chip contains on the order of 10^{10} transistors; no person reasons about them individually. The field copes by stacking abstractions, each hiding the one beneath. Knowing the stack — and knowing where it leaks — is the organising skill of the discipline.

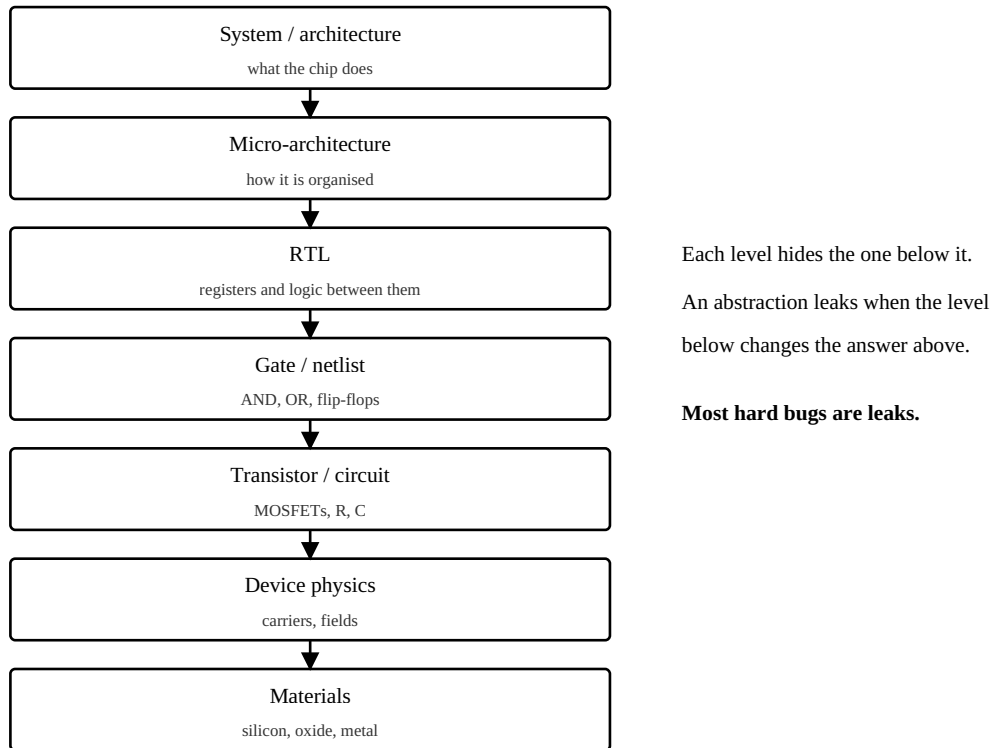


Figure 1. The abstraction stack. Work at one level, but know which level below will eventually intrude.

Table 1. What each level is, and who works there

Level	Unit of thought	Typical artefact	Role
System	Throughput, latency, power budget	Architecture document, performance model	System architect
Micro-architecture	Pipelines, buffers, state machines	Block diagrams, cycle-accurate model	Micro-architect, modelling engineer
RTL	Registers and the logic between them	SystemVerilog / VHDL	Design engineer
Gate	Boolean functions, standard cells	Netlist	Synthesis tool (not usually a person)
Circuit	Transistors, resistance, capacitance	SPICE netlist	Analogue / custom designer
Device	Carrier transport, fields	Compact models (BSIM)	Device engineer
Materials	Doping, oxides, metals	Process recipe	Process engineer

Undergraduate review · The middle of this stack is where digital IP work happens. A digital designer writes RTL and trusts the synthesis tool to produce gates, the library to characterise them, and the process to build them. The trust is well-founded almost all of the time.

Graduate · **Almost all of the time is the important qualifier, and the exceptions form a checklist worth memorising.** Abstractions leak in a small number of recurring ways, each of which appears later in this book:

- **Timing** — RTL describes behaviour without delay, but a real gate takes time, so a functionally correct design can fail (Chapter B3).
- **Metastability** — a flip-flop is modelled as producing 0 or 1, but a real one can do neither for a while (Chapter C3).
- **Power** — RTL has no notion of energy, yet switching activity and glitching are decided by RTL structure (Chapter B2).
- **Physical size** — RTL has no geometry, but wire delay depends on it (Chapter B4).
- **Side channels** — two circuits computing the same function can be distinguished by their power consumption (Chapter H3).
- **Radiation and ageing** — a stored bit is assumed permanent; physically it is not (Chapter K3).

A useful way to read the rest of this book is as a systematic tour of these leaks, because they are exactly the places where an engineer's judgement is still required and a tool cannot be trusted alone.

Z1.2 The design flow in one page

Table 2. From idea to silicon

Phase	Input	Output	Principal risk
Specification	Requirements	Written specification	Ambiguity
Modelling	Specification	Reference model	Misreading the specification
RTL design	Specification	Synthesisable RTL	Functional error
Verification	RTL + model	Coverage, bug reports	Missing a case
Synthesis	RTL + constraints + library	Gate netlist	Wrong constraints
Place and route	Netlist + floorplan	Layout	Congestion, timing
Sign-off	Layout	GDSII	An unchecked corner
Fabrication	GDSII	Wafers	Yield
Bring-up	Packaged parts	Working system	No visibility
Production test	—	Sorted devices	Test escape

The cost of fixing a defect rises by roughly an order of magnitude at each phase boundary. A specification ambiguity caught while writing the specification costs an hour. Caught in verification it costs a week. Caught at bring-up it costs a mask set — hundreds of thousands to millions of dollars and a three-month schedule slip. This asymmetry, not perfectionism, is why the industry invests so heavily in front-end rigour, and it is the economic reason a modelling team exists at all.

Z2. Boolean Algebra and Combinational Logic

Undergraduate review · Digital logic rests on a two-element Boolean algebra with operations AND ($\cdot$), OR ($+$) and NOT ($'$). Its axioms give the identities used to simplify expressions, and every combinational circuit is the realisation of some Boolean function.

Table 3. Identities used constantly

Name	Statement
Identity	$A \cdot 1 = A, \quad A + 0 = A$
Null	$A \cdot 0 = 0, \quad A + 1 = 1$
Idempotence	$A \cdot A = A$
Complement	$A \cdot A' = 0, \quad A + A' = 1$
De Morgan	$(A \cdot B)' = A' + B', \quad (A + B)' = A' \cdot B'$
Absorption	$A + A \cdot B = A$
Consensus	$AB + A'C + BC = AB + A'C$
Shannon expansion	$f = x \cdot f _{x=1} + x' \cdot f _{x=0}$

Graduate · **Two of these are more than algebra.** De Morgan's laws are the reason CMOS is built from NAND and NOR rather than AND and OR: a CMOS gate is naturally inverting, so *every* logic function must be expressible in inverting gates, which De Morgan guarantees. The **consensus term** is the redundant product that can be removed without changing the function — but removing it can create a *static hazard*, a momentary glitch when inputs change. In synchronous logic a glitch before the next clock edge is harmless and the term is removed for area; in asynchronous logic or in a clock-gating enable it is not, and the redundant term must be kept. **The same simplification is correct in one context and a bug in another, which is why hazard-free synthesis is a distinct option in the tools.**

Table 4. Representations of a Boolean function

Form	Size	Canonical?	Use
Truth table	2^n rows	Yes	Definition; small functions
Karnaugh map	2^n	No	Hand minimisation up to ~5 variables
Sum of products	Varies	No (unless canonical)	PLA, two-level synthesis
Quine–McCluskey	—	—	Exact minimisation; exponential
Espresso	—	—	Heuristic; what tools actually use
BDD	Order-dependent	Yes	Equivalence checking
AIG	Compact	No	Modern synthesis and formal engines

Z2.2 Standard combinational blocks

Table 5. The building blocks every design contains

Block	Function	Cost	Note
Multiplexer	Select one of n inputs	$n-1$ two-input muxes, or a tree	A mux is functionally complete — any function can be built from muxes

Block	Function	Cost	Note
Decoder	$n \rightarrow 2^n$ one-hot	2^n gates	Address decoding, instruction decode
Encoder / priority encoder	one-hot $\rightarrow$ binary	—	Interrupt arbitration, leading-zero detection
Comparator	Equality or magnitude	XOR tree / subtract	Magnitude comparison is an adder in disguise
Shifter / barrel shifter	Shift by a variable amount	$\log_2 n$ mux stages	Used in floating point, in gearboxes
Adder	See Chapter C1	—	The most studied block in the field
Parity / population count	XOR tree / adder tree	—	ECC, binary neural networks

Graduate · The observation that a multiplexer is functionally complete is more than a curiosity: it is why FPGAs work.

An FPGA's logic element is a small lookup table, which is physically a multiplexer tree driven by configuration bits. Any four-input function is realised by loading sixteen bits. The entire programmable-logic industry rests on the Shannon expansion identity in the table above. **Recognising that a familiar identity is the foundation of a product category is the kind of connection this book is trying to make routine.**

Z3. Sequential Logic and State

Undergraduate review · A combinational circuit's output depends only on its present inputs. Adding memory — elements that hold a value — produces sequential logic, whose output depends on the history of inputs. That history is summarised in the **state**.

Table 6. Storage elements

Element	Behaviour	Use	Hazard
SR latch	Set/reset, level sensitive	Building block	Forbidden input combination
D latch	Transparent when enabled	Time borrowing, low-power designs	Transparency complicates timing analysis
D flip-flop	Captures on a clock edge	The standard element	Setup/hold must be met
Master–slave	Two latches in series	How a flip-flop is built	—
Pulsed latch	Narrow transparency window	High performance	Timing complexity
Register file	Array with multiple ports	Processors	Port count costs area

Unintentional latches are one of the most common RTL defects. In a combinational always block, if some path through the logic leaves a signal unassigned, the synthesiser must remember its previous value and therefore infers a latch. The design usually still simulates correctly, but a latch in a nominally combinational path is a timing analysis problem and often a functional one. The defence is mechanical: assign a default value at the top of every combinational block, cover every branch, and treat the lint warning as an error. **This is the first check in any RTL review.**

Table 7. Finite state machine styles

Style	Output depends on	Property
Moore	State only	Outputs are registered and glitch-free; one cycle of latency
Mealy	State and inputs	Faster response; outputs can glitch and create combinational paths
Registered Mealy	State and registered inputs	Compromise; usually preferred

Graduate · **The Moore/Mealy choice has a direct consequence at module boundaries.** A Mealy output is a combinational function of an input, so connecting two Mealy modules can create a path that runs through both in a single cycle — or, if each one's input depends on the other's output, a combinational loop. This is the general form of the valid/ready loop described in Chapter C2. **For an IP block, registering all outputs (Moore style) costs one cycle of latency and removes an entire class of integration failures,** which is usually the right trade for something a stranger will instantiate.

Z3.2 Counters, shift registers and LFSRs

Table 8. Sequential building blocks

Block	Function	Where used
Binary counter	Increment	Addressing, timing
Gray counter	One bit changes per step	CDC-safe pointers , low-power encoding
Johnson counter	Twisted ring	Multi-phase clock generation

Block	Function	Where used
Shift register	Serial movement	Serialisers, delay lines
LFSR	Shift with XOR feedback	PRBS generation, scramblers, CRC, BIST, test patterns
CRC	Polynomial remainder	Frame check sequences

Graduate · **The LFSR is the most reused structure in communications hardware, and all of its uses are the same mathematics.** An LFSR computes a remainder modulo a polynomial over GF(2). Choose a primitive polynomial and it cycles through all $2^n - 1$ non-zero states, giving a maximal-length pseudo-random sequence — that is a PRBS generator, and also a scrambler. Feed data in as you shift and the final state is the remainder — that is a CRC. Use it to produce stimulus and to compact responses and you have built-in self test. **One circuit, four products, because they are one piece of algebra.** Chapter I2 develops the algebra; this is where it first appears.

Z4. Circuit Analysis: the Minimum Needed

Table 9. Laws and elements

Item	Statement	Digital-design relevance
KCL	Currents into a node sum to zero	Power grid analysis
KVL	Voltages around a loop sum to zero	IR drop
Ohm	$V = IR$	Wire and switch resistance
Capacitor	$i = C \, dv/dt$	The dominant load in CMOS
Inductor	$v = L \, di/dt$	Package and board; cause of supply noise
RC time constant	$\tau = RC$	Gate delay, wire delay, rise time
Thévenin / Norton	Source equivalents	Driver modelling, termination
Superposition	Linear circuits only	Noise analysis

Undergraduate review · For digital work the essential relation is the RC charging curve: a capacitor charged through a resistor reaches 63% of its final value in one time constant $\tau = RC$, and about 50% in 0.69τ . Since a logic gate switches when its input crosses the threshold near mid-supply, **propagation delay is approximately $0.69R_{\text{on}}C_{\text{load}}$** .

Graduate · **That single expression explains most of digital design practice.** Delay is proportional to load capacitance, so driving many gates is slow — hence buffer trees. Delay is proportional to on-resistance, which falls as transistor width rises, so wider devices are faster — but a wider device presents a larger capacitance to *its* driver, which is why sizing is a chain optimisation rather than a local one, and why the logical-effort method of Chapter B2 exists. Dynamic energy is CV^2 per transition, which is why lowering supply voltage is the strongest power lever and why it costs speed. **Four consequences from one equation; this is the level of leverage that makes a small amount of circuit theory worth carrying.**

Table 10. Frequency-domain concepts a digital engineer meets

Concept	Meaning	Where encountered
Impedance $Z(\omega)$	Generalised resistance	Power delivery, termination
Pole / zero	Roots of the denominator / numerator	Filter and loop design
Bode plot	Magnitude and phase versus frequency	PLL loop stability
Q factor	Energy stored / dissipated per cycle	Resonance in power networks, oscillator phase noise
Phase margin	Stability measure	Any feedback loop, including a PLL
Bandwidth $\times$ gain	Roughly constant for an amplifier	Why high gain and high speed conflict

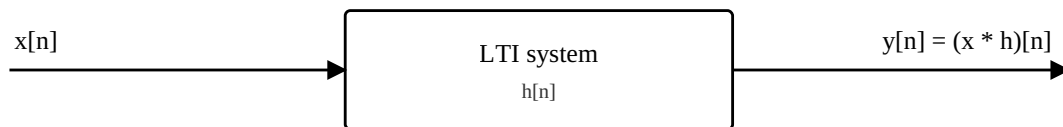
A1. Signals, Systems and Linearity

Every digital IP block is a *system*: it maps input sequences to output sequences. Before discussing any particular block we fix the vocabulary that makes such maps analysable, and — more importantly for a hardware engineer — we identify exactly which property of a system makes it **cheap** to build and which makes it **expensive**.

A1.1 Definitions

Table 11. System properties and their hardware consequences

Property	Definition	Hardware consequence
Memoryless	$y[n]$ depends only on $x[n]$	Pure combinational logic; no registers, no state to reset or scan
Causal	$y[n]$ depends only on $x[k]$, $k \leq n$	Required for real-time streaming. Non-causal filters are realised by adding latency , not by seeing the future
Linear	Superposition holds	Enables decomposition into impulse responses; permits frequency-domain design
Time-invariant	Delay in $\rightarrow$ same delay out	Coefficients are constants $\rightarrow$ can be hard-wired or stored once
BIBO stable	Bounded input $\rightarrow$ bounded output	$\sum h[n] < \infty$. In fixed point, determines whether accumulators can overflow
Passive lossless	/ Energy not increased	Guarantees no growth in word length



Linearity + time invariance $\Rightarrow$ one impulse response is the whole system

$Y(z) = H(z) \cdot X(z)$ — convolution becomes multiplication

Figure 2. An LTI system is completely characterised by its impulse response. This single fact underpins all filter design.

Graduate The practical importance of linearity is not elegance but **separability of verification**. If a block is LTI you can characterise it with a single impulse and predict its response to every input; a random-stimulus campaign is then a redundancy check rather than the primary evidence. If it is *not* LTI — a decision-feedback equaliser, a μ -law compander, a saturating accumulator, any adaptive loop — superposition fails and you must verify over the operating region, which is exponentially more work. **The first architectural question about any new block is therefore "where exactly is the nonlinearity?"** Isolating it into the smallest possible sub-block is a verification cost decision, not an aesthetic one.

A1.2 Convolution and its cost

$$y[n] = \sum_{k=0}^{N-1} h[k] x[n-k]$$

Table 12. Realisations of the same convolution

Method	Operations per output	Latency	When preferred
Direct convolution	N MACs	0 (plus pipeline)	$N \lesssim 64$
Overlap-add FFT	$\sim(2 \log_2 L + 1)$ per sample	One block (L samples)	Long filters; latency tolerable
Overlap-save FFT	Same order	One block	Streaming; avoids add stage
Frequency-domain adaptive	FFT + per-bin update	One block	Long adaptive filters (echo cancellation)
Distributed arithmetic	LUT reads only	W cycles	Fixed coefficients, LUT-rich fabric
Multiplierless (CSD)	Shifts and adds	0	Fixed coefficients, ASIC area critical

Graduate · The crossover between direct and FFT-based convolution is not a fixed number of taps; it depends on the **cost ratio of a multiplier to a memory word** in the target technology, and on whether block latency is acceptable. In an ASIC at an advanced node, multipliers are comparatively cheap and SRAM is comparatively expensive, which pushes the crossover *up*. On an FPGA with hard DSP slices and abundant block RAM it moves the other way. **Quoting a fixed crossover (“use FFT above 128 taps”) without naming the technology is a common and costly error in architecture documents.**

A1.3 Correlation, matched filtering and the origin of SNR gain

Cross-correlation is convolution with a time-reversed conjugated kernel. When the kernel is the transmitted pulse itself, the operation is a **matched filter**, and it maximises output SNR for a signal in additive white Gaussian noise.

$$SNR_{out,max} = 2E_s / N_0$$

Graduate · The result is worth internalising because it recurs in radar, in spread spectrum, in synchronisation preambles and in every correlator IP. **The gain comes from integrating signal coherently while noise adds incoherently:** over M samples, signal amplitude grows as M while noise amplitude grows as $\sqrt{M}$, hence power SNR improves by M ($10 \log_{10} M$ dB). Two engineering consequences follow. First, coherent integration requires **phase stability over the integration window**; if the oscillator drifts, gain saturates — this is the practical limit on radar integration time and on preamble length. Second, the correlator's dynamic range must accommodate the M -fold growth, which is a fixed-point word-growth problem, not a signal-processing one.

A1.4 Group delay, phase linearity and why FIR dominates

$$\tau_g(\omega) = -d\phi(\omega)/d\omega$$

Undergraduate review · A filter has **linear phase** when τ_g is constant: all frequency components are delayed equally, so pulse shapes are preserved and only shifted. An FIR filter with symmetric (or antisymmetric) coefficients has exactly linear phase. An IIR filter cannot, except trivially.

Graduate · This is the decisive argument for FIR in data transmission. Non-constant group delay **spreads a pulse asymmetrically**, which is indistinguishable from channel dispersion and therefore directly closes the eye. In audio the ear is comparatively insensitive to phase and IIR is preferred for its efficiency; in a wireline receiver an IIR equaliser would introduce the very distortion it is meant to remove. **The same mathematics leads to opposite engineering choices in two domains, and knowing why is the difference between applying a rule and understanding it.** Note also the hardware corollary: symmetric FIR coefficients permit the *folded* structure that halves the multiplier count, so linear phase is not only free, it is cheaper.

A1.5 Stability in the Z domain

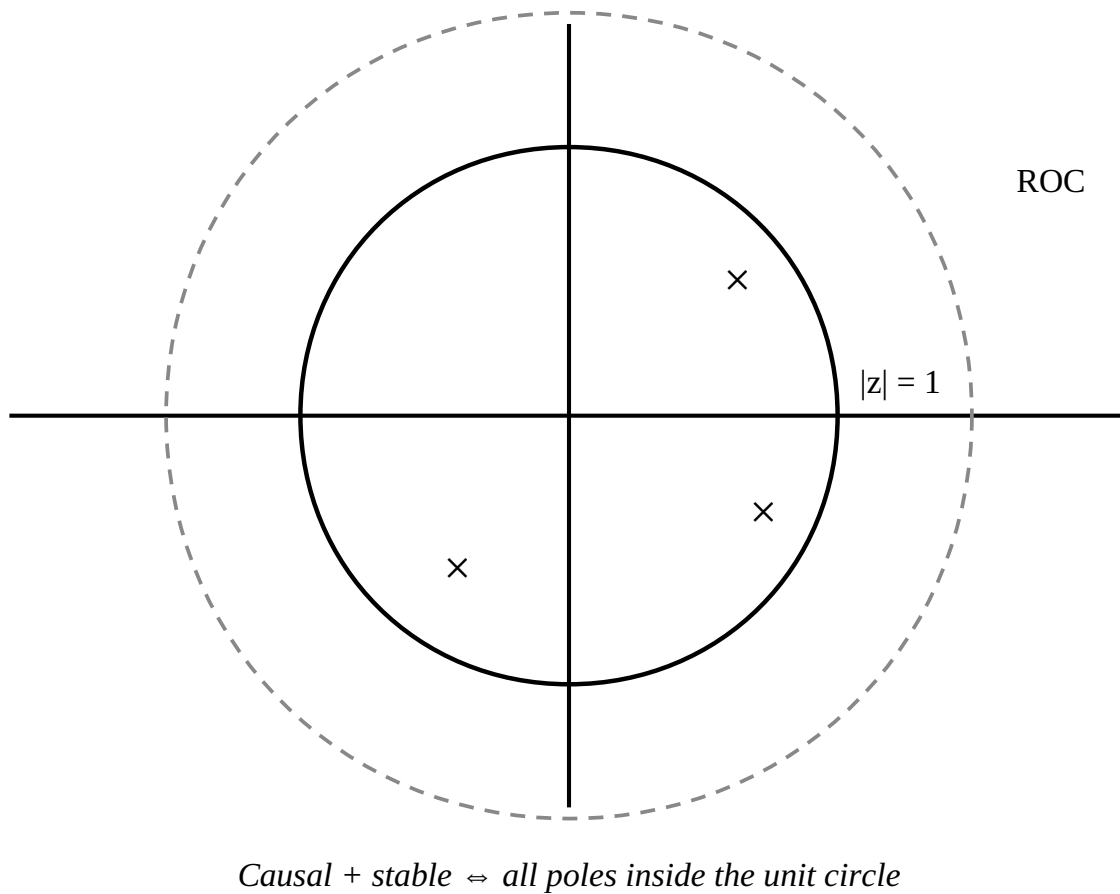


Figure 3. Pole-zero plot and region of convergence. For a causal system the ROC is outside the outermost pole; stability additionally requires that the ROC contain the unit circle.

Table 13. Pole location and behaviour

Pole radius $ p $	Behaviour	Fixed-point implication
$ p < 1$	Decaying; stable	Accumulator bounded by $1/(1- p)$
$ p = 1$	Marginally stable (oscillator, integrator)	Accumulator grows without bound — needs explicit leakage or reset
$ p > 1$	Divergent	Unusable
$ p \rightarrow 1^-$	Very narrow band; long memory	Coefficient quantisation can push the pole outside — the classic failure mode of high-order direct-form IIR

Graduate · Quantifying the last row: for a direct-form IIR of order N , the sensitivity of pole p_i to a denominator coefficient a_k is $\partial p_i / \partial a_k = -p_i^{N-k} / \prod_{j \neq i} (p_i - p_j)$. The denominator is a product of *pole separations*: **clustered poles make the sensitivity explode**. Cascaded second-order sections avoid this because each section contains only one conjugate pair, so no small separations appear in its own denominator. This is the quantitative reason behind the universal practice of implementing high-order IIR filters as cascaded biquads, and it is worth being able to state it rather than merely follow it.

A2. Transforms and Spectral Representation

Table 14. The transform family

Transform	Domain	Kernel	Principal use in IP work
CTFT	continuous time $\rightarrow \omega$	$e^{-j\omega t}$	Analogue filter and antenna analysis
Laplace	continuous time $\rightarrow s$	e^{-st}	PLL loop dynamics, control, transmission lines
DTFT	discrete time $\rightarrow \omega$	$e^{-j\omega n}$	Theoretical filter response (continuous in ω)
DFT / FFT	finite sequence $\rightarrow$ bins	$e^{-j2\pi kn/N}$	Everything computable
Z	discrete time $\rightarrow z$	z^{-n}	Poles, zeros, stability, filter structures
Hilbert	real $\rightarrow$ analytic	$-j \operatorname{sgn}(\omega)$	I/Q generation, envelope, SSB
DCT	finite $\rightarrow$ cosine basis	cos	Image/video coding (energy compaction)
Wavelet	time–scale	scaled basis	Transient detection, JPEG2000

A2.1 The DFT is not the DTFT

The DFT computes samples of the DTFT of a **windowed, periodically extended** sequence. Three artefacts follow, and all three appear as bugs in practice.

Table 15. DFT artefacts

Artefact	Cause	Symptom	Mitigation
Spectral leakage	Implicit rectangular window	A pure tone smears across bins	Apply a window (Hann, Blackman–Harris)
Scalloping loss	Tone falls between bin centres	Up to 3.9 dB amplitude error (rectangular)	Window, or interpolate the peak
Picket fence	Only N samples of a continuum	Frequency resolution limited to f_s/N	Zero-pad (interpolates, does not add resolution)
Circular convolution	Periodic extension	Wrap-around in fast convolution	Zero-pad to $N \geq L+M-1$

Zero-padding is routinely misdescribed as improving resolution. It does not. **Resolution is set by the observation time** $T = N/f_s$; two tones closer than $\approx 1/T$ cannot be separated no matter how many zeros are appended. Padding interpolates the displayed spectrum, which makes peaks easier to *locate* but not easier to *resolve*. In a specification, resolution and interpolation must be stated separately.

Table 16. Common windows

Window	Main-lobe width (bins)	First sidelobe	Use
Rectangular	2	−13 dB	Transient capture; maximum resolution
Hann	4	−31 dB	General purpose
Hamming	4	−41 dB	Slightly better near sidelobe
Blackman–Harris (4-term)	8	−92 dB	Spur hunting in converter test
Kaiser (β adjustable)	tunable	tunable	Design-to-spec
Flat-top	~9	−90 dB	Amplitude accuracy (calibration)

A2.2 Parseval, power spectral density, and dB conventions

$$\sum_n |x[n]|^2 = (1/N) \sum_k |X[k]|^2$$

Table 17. Decibel conventions that are routinely confused

Unit	Reference	Note
dB	ratio	10 log for power, 20 log for amplitude
dBm	1 mW	Absolute power
dBc	carrier	Spur levels, phase noise
dBFS	converter full scale	Digital level; not a physical power
dBm/Hz	1 mW per Hz	Spectral density; thermal floor -174 dBm/Hz @ 290 K
dBc/Hz	carrier per Hz	Phase noise $L(\Delta f)$

Graduate · A recurring measurement error is comparing a **single-bin** FFT magnitude with a **spectral density**. The FFT bin contains power integrated over the equivalent noise bandwidth of the window, which for a Hann window is 1.5 bins, not 1. Reporting “noise floor at -110 dBFS” without stating N , f_s and the window is not a measurement, because the number changes when any of the three changes. **Processing gain $10 \log_{10}(N/2)$ is often mistaken for converter performance.**

A3. Sampling, Reconstruction and Quantisation

A3.1 The sampling theorem and its practical corollaries

$$X_s(f) = f_s \sum_k X(f - k f_s)$$

Table 18. Sampling regimes

Regime	Condition	Where used	Hardware cost
Nyquist	$f_s > 2B$, base-band	Most baseband DSP	Anti-alias filter must be steep
Oversampling (OSR > 1)	$f_s \gg 2B$	$\Sigma\Delta$ converters, relaxed AAF	Quantisation noise spread over wider band: +3 dB SNR per doubling
Bandpass / under-sampling	$2f_H/n \leq f_s \leq 2f_L/(n-1)$	RF direct sampling receivers	Sampler analogue bandwidth must exceed f_H , not f_s
Time-interleaved	M converters at f_s/M	>10 GS/s ADCs (112G SerDes uses 32–64 way)	Offset / gain / timing skew mismatch creates spurs at kf_s/M

Graduate · **Time-interleaving is where the design house earns its fee.** An M -way interleaved ADC has three mismatch mechanisms, each producing a distinct spur family: offset mismatch gives tones at kf_s/M independent of input; gain mismatch gives sidebands at $kf_s/M \pm f_{in}$; timing skew gives the same sideband locations but with amplitude proportional to f_{in} . **The frequency dependence is the diagnostic:** sweep the input and see which spur grows. Digital background calibration of all three is a standard deliverable in a modern ADC-DSP receiver, and it is a *digital* block — which means a fabless digital IP vendor can sell it even though the converter itself is analogue. This is a concrete example of finding a sellable digital block adjacent to an analogue one you cannot build.

A3.2 Quantisation noise

$$SQNR_{dB} = 6.02 B + 1.76 + 10 \log_{10}(OSR)$$

Table 19. Where the ideal formula breaks

Effect	Mechanism	Consequence	Design response
Aperture jitter	Sampling instant uncertainty σ_t	$SNR = -20 \log_{10}(2\pi f_{in} \sigma_t)$	Dominates above ~1 GHz input; clock design becomes the ADC spec
Thermal noise	KT/C of sampling capacitor	Floors SNR regardless of bits	Increase C (costs power and bandwidth)
DNL / INL	Element mismatch	Harmonic distortion, missing codes	Calibration, dynamic element matching
Comparator meta-stability	Finite regeneration time	Rare large errors	More regeneration time or error detection
Correlated quantisation	Low-amplitude periodic input	Quantisation noise becomes tonal, not white	Dither — deliberately added noise to decorrelate

Graduate · The dither entry deserves emphasis because it is counter-intuitive: **adding noise improves the result.** Quantisation error is only white and signal-independent when the signal moves over many LSBs; for a small or slowly varying signal

the error becomes periodic and produces discrete tones, which are far more objectionable than an equivalent amount of broad-band noise (in audio they are audible; in a radar they look like targets). Subtractive dither — add a known pseudo-random sequence before quantising and subtract it after — removes the tones without the SNR penalty. **A dither generator is a small, precisely specifiable digital block and therefore exactly the kind of thing a one-person IP house can build and sell as part of a converter-adjacent package.**

A3.3 Reconstruction

Table 20. Reconstruction effects

Effect	Expression	Compensation
Zero-order hold droop	$\text{sinc}(f/f_s)$	−3.92 dB at Nyquist; pre-compensate with an inverse-sinc FIR
Image replicas	at $kf_s \pm f$	Reconstruction (anti-imaging) filter
Interpolation	up-sample then low-pass	Moves images away so the analogue filter can be gentle
Mismatch-shaping DAC	—	Pushes element mismatch error out of band; a digital technique for an analogue defect

A4. Probability, Random Processes and Noise

A4.1 Distributions that occur in hardware

Table 21. Distributions and where they arise

Distribution	Arises from	Appears in
Gaussian	Sum of many small independent effects (CLT)	Thermal noise, random jitter, process variation
Rayleigh	Magnitude of complex Gaussian	Fading envelope, noise magnitude
Rician	Complex Gaussian + deterministic component	Fading with line of sight
Poisson	Rare independent events	Shot noise, soft-error (SER) counts
Uniform	Quantisation residue (ideal)	Quantisation noise model
Exponential	Waiting time between Poisson events	Time between soft errors
Log-normal	Product of many factors	Shadowing, some reliability lifetimes
Weibull	Weakest-link failure	Oxide breakdown, reliability
Binomial	n Bernoulli trials	Bit errors in a block, yield
Chi-square	Sum of squared Gaussians	Energy detection, CFAR statistics

A4.2 Physical noise mechanisms

Table 22. Noise sources in silicon

Mechanism	Spectrum	Expression	Design lever
Thermal (Johnson)	White	$v_n^2 = 4kTR\Delta f$	Lower R; lower T; narrower band
Channel thermal (MOS)	White	$i_n^2 = 4kTyg_m\Delta f$	Higher g_m improves noise <i>figure</i> though noise power rises
Flicker (1/f)	1/f	$\propto K/(WLC_{ox}f)$	Larger devices ; chopping; auto-zeroing
Shot	White	$i_n^2 = 2qI\Delta f$	Present where carriers cross a barrier (junctions, photodiodes)
kT/C	Sampled	$v^2 = kT/C$	Independent of resistance ; only C helps
Substrate / supply coupling	Structured	—	Guard rings, separate supplies, differential design
Phase noise	$1/f^3$, $1/f^2$, flat	Leeson model	Higher Q, higher power, lower flicker

Graduate · Why flicker noise sets the architecture of precision analogue. $1/f$ noise is not reducible by filtering, because it lives at the frequencies where the signal of interest often lives (DC to kHz). The standard responses — chopping and auto-zeroing — both work by *moving the signal to a higher frequency where only white noise remains*, and both require digital support: a chopping clock, demodulation, and residual-ripple filtering. In a modern mixed-signal IP the digital half of that scheme is a well-defined block with a clean interface, and it is again a piece that a digital design house can own.

A4.3 Jitter and phase noise — the same thing in two languages

$$\sigma_t^2 = (1/(2\pi f_0)^2) \int 2 L(f) df$$

Table 23. Jitter taxonomy used in link specifications

Type	Symbol	Statistics	Cause	How it is separated
Random jitter	RJ	Gaussian, unbounded	Thermal, flicker	Fit tails of the bathtub curve
Deterministic jitter	DJ	Bounded	—	Peak-to-peak; decomposed below
Periodic	PJ	Sinusoidal	Supply coupling, spurs	FFT of the jitter sequence
Data-dependent	DDJ / ISI	Pattern-correlated	Channel loss	Compare per-pattern edge times
Duty-cycle distortion	DCD	Two-valued	Rise/fall asymmetry, threshold offset	Even/odd UI comparison
Bounded uncorrelated	BUJ	Bounded, random	Crosstalk	Statistical separation
Total jitter at BER	TJ(BER)	RJ⊗DJ convolution	—	$TJ = DJ + 2 \cdot Q(BER) \cdot RJ_{rms}$

Graduate · The last row is the one that appears in every SerDes specification, and the factor $Q(BER)$ is why: at $BER\ 10^{-12}$, $Q = 7.03$, so $TJ = DJ + 14.07\ RJ_{rms}$. **Random jitter is multiplied by fourteen.** This is why a small improvement in oscillator or phase noise is worth a large amount of design effort, and why jitter separation — distinguishing RJ from DJ in a measurement — is a required capability rather than a nicety. A modelling engineer asked to produce a link budget must implement this convolution correctly; adding RMS values in quadrature, which is the intuitive move, is wrong because DJ is bounded and does not add that way.

A4.4 Random processes

Table 24. Concepts and their use

Concept	Statement	Use in IP design
Stationarity (WSS)	Mean constant, autocorrelation depends on lag only	Precondition for spectral analysis
Ergodicity	Time average = ensemble average	Justifies measuring one long record instead of many devices
Wiener–Khinchin	PSD = FT of autocorrelation	Links time and spectrum
Markov chain	Memoryless state evolution	Bus arbitration analysis, FIFO occupancy, error models
Gauss–Markov	Correlated Gaussian	Coloured noise models
Cyclostationarity	Statistics periodic in time	All digitally modulated signals ; exploited for blind synchronisation
Point process	Random event times	Soft errors, packet arrivals

Graduate · Cyclostationarity is the theoretical basis for **blind** timing and carrier recovery: a modulated signal has periodically varying statistics at the symbol rate, so a suitable nonlinearity produces a spectral line at that rate even though the signal itself has no such line. Gardner and Mueller–Müller timing detectors are practical exploitations of this. Knowing the principle rather than the recipe is what lets an engineer design a detector for a modulation format that has no textbook detector — a situation that occurs constantly in proprietary links.

A5. Linear Algebra and Numerical Methods for Hardware

A5.1 Decompositions and their hardware character

Table 25. Matrix decompositions as IP blocks

Decomposition	Form	Cost	Streaming?	Numerical character
LU	$A = LU$	$n^3/3$	No (pivoting needs lookahead)	Needs pivoting for stability
Cholesky	$A = LL^*$	$n^3/6$	Yes	No pivoting needed for SPD; half the work of LU
QR (Givens)	$A = QR$	$2mn^2$	Yes	Backward stable; rotations are well conditioned
QR (Householder)	same	fewer ops	Less so	Best stability
SVD	$A = U\Sigma V^*$	iterative	No	Most robust ; reveals rank and conditioning
Eigen (symmetric)	$A = Q\Lambda Q^*$	iterative	No	Jacobi parallelises well
Schur / Hessenberg	—	n^3	No	Control applications

Graduate · The streaming column is the commercial column. Cholesky and Givens QR touch each input element once and can therefore be built with an `hls::stream`-style interface and no external memory; SVD and LU must revisit the array and therefore need an AXI master and a buffer. That single architectural difference changes the IP's integration story, its area, and its price. When a customer asks for “a matrix inverse block”, the first engineering question is whether the matrix is symmetric positive definite — because if it is, Cholesky gives a streaming solution at half the arithmetic, and if it is not, you are quoting a much larger block.

A5.2 Conditioning

$$\kappa(A) = \sigma_{\max}/\sigma_{\min} \quad \text{relative error} \lesssim \kappa(A) \cdot \varepsilon_{\text{machine}}$$

Graduate · The rule of thumb that follows is directly usable: **you lose roughly $\log_{10}\kappa$ decimal digits**. In fixed point, substitute $\varepsilon = 2^{-F}$ for F fractional bits and the statement becomes a word-length requirement: $F \gtrsim \log_2 \kappa + (\text{bits of accuracy required})$. This is how a modelling engineer converts a mathematical property of the customer's data into a hardware parameter. It also explains why **iterative refinement** works: solve in low precision, compute the residual in high precision, correct, repeat. Each iteration roughly squares the accuracy, so three refinements from a rough start can reach near-full precision — but only if the condition number is modest, because κ enters the convergence factor. **Measured behaviour: for a well-conditioned (isotropic) system the scheme reached 24.2 bits of accuracy; at correlation 0.99 it reached only 12.4 bits**. An IP that advertises mixed-precision refinement must therefore state a conditioning limit, or it will fail at a customer whose data is correlated.

A5.3 Iterative solvers and their hardware mapping

Table 26. Iterative methods

Method	Update	Parallelism	Convergence
Jacobi	uses only previous iterate	Fully parallel	Slow; needs diagonal dominance
Gauss–Seidel	uses updated values immediately	Sequential within a sweep	Roughly twice Jacobi's rate
SOR	over-relaxed GS	Sequential	Tunable ω

Method	Update	Parallelism	Convergence
Conjugate gradient	Krylov subspace	Parallel (dot products)	n steps exactly; $\sqrt{k}$ in practice
Richardson	$x \leftarrow x + \alpha(b - Ax)$	Fully parallel	Simplest; α must be tuned

Graduate · There is a physical realisation worth knowing because it is repeatedly rediscovered and mis-described. An RC network that solves $C \, dv/dt = -(Av - b)$ settles to $A^{-1}b$, and is often presented as an “analogue Gauss–Seidel solver”. **It is not:** because all nodes evolve simultaneously from the same previous state, the continuous-time limit corresponds to **Jacobi / Richardson**, not Gauss–Seidel, which is defined by sequential in-sweep updates. The settling time is governed by $1/\lambda_{\min}$, i.e. by the condition number again. Under a normalisation that fixes $\lambda_{\max} = 1$, the measured settling-time-to-condition ratio was constant at 13.2 — the analogue solver buys constant factors, not asymptotic ones. **Being able to state this precisely prevents a design house from accepting a proposal whose speedup claim rests on a misidentification.**

B1. The MOSFET — What a Digital Designer Must Know

A digital IP engineer does not size transistors, but every timing number, every power figure and several verification hazards trace back to device behaviour. This chapter keeps only what changes decisions.

B1.1 Regions of operation

Table 27. MOSFET regions

Region	Condition	Current	Where it matters
Cut-off	$V_{GS} < V_{TH}$	$I_{sub} \propto e^{(V_{GS}-V_{TH})/nV_T}$	Subthreshold leakage — the dominant static power term below 65 nm
Triode / linear	$V_{DS} < V_{GS}-V_{TH}$	$\mu C_{ox}(W/L)[(V_{GS}-V_{TH})V_{DS} - V_{DS}^2/2]$	Switch resistance R_{on} ; pass gates; the ON device of an inverter at end of transition
Saturation	$V_{DS} > V_{GS}-V_{TH}$	$\frac{1}{2}\mu C_{ox}(W/L)(V_{GS}-V_{TH})^2(1+\lambda V_{DS})$	Analogue gain stages; the beginning of a digital transition
Velocity saturation	Short channel, high field	$\propto (V_{GS}-V_{TH})^1$ — linear, not square	Modern nodes live here. Textbook square-law intuition overestimates drive

Graduate · The last row is the single most important correction for anyone who learned the square law. In a short-channel device carriers reach their saturation velocity, so drive current becomes roughly proportional to overdrive rather than its square. Two practical consequences: **(i)** increasing supply voltage buys less speed than square-law intuition suggests, while dynamic power still rises as V^2 , so voltage scaling is a poor speed lever; **(ii)** stacking devices (as in a NAND with many inputs) costs more than the naive series-resistance estimate, which is why wide fan-in gates are avoided and logic is restructured into trees. When a synthesis tool reports that a 4-input gate is slower than two 2-input gates in series, this is why.

B1.2 Threshold voltage and its enemies

Table 28. V_{TH} modulation effects

Effect	Mechanism	Design consequence
Body effect	Source–bulk bias raises V_{TH}	Stacked devices are slower than expected
DIBL	Drain field lowers the barrier	V_{TH} falls with $V_{DS} \rightarrow$ more leakage
Random dopant fluctuation	Discrete dopant atoms	$\sigma_{V_{TH}} \propto 1/\sqrt{WL}$ — small devices mismatch more
Temperature	–1 to –2 mV/K	Hot chip is <i>faster</i> at low V_{DD} (temperature inversion)
NBTI / PBTI ageing	Trap generation under stress	V_{TH} drifts over years — timing must include an ageing margin
Multi- V_{TH} offering	Process option	LVT for critical paths, HVT elsewhere: the primary leakage-vs-speed lever

Graduate · **Temperature inversion is a verification trap.** At high supply voltage a chip is slowest when hot, so signing off at the high-temperature corner is conservative. At low supply voltage — which is where modern low-power modes operate — the V_{TH} reduction outweighs mobility degradation and the chip is slowest when **cold**. Teams that carried forward a "hot is

worst case" habit have taped out parts that fail at $-40\text{ }^{\circ}\text{C}$. The corner list is not a formality; it is a statement about which physics dominates in the operating region you are selling into.

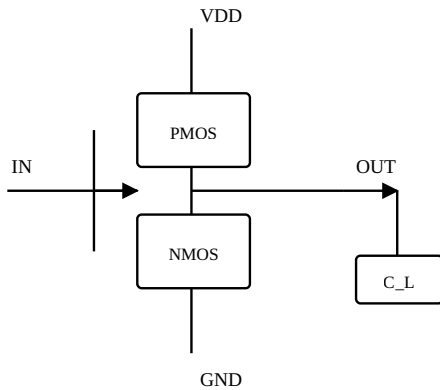
B1.3 Scaling and what it stopped giving

Table 29. Dennard scaling and its breakdown

Quantity	Ideal scaling (factor $s > 1$)	What actually happened
Dimensions	$1/s$	Continued (with FinFET, GAA)
Supply V_{DD}	$1/s$	Stalled near 0.7–0.9 V — V_{TH} cannot follow without leakage explosion
Delay	$1/s$	Slower than ideal; interconnect does not scale
Power density	constant	Rose → dark silicon, thermal limits
Leakage	negligible	Comparable to dynamic below 65 nm
Interconnect RC	—	Worsens : resistance rises as cross-section shrinks

Graduate · The end of Dennard scaling is the reason the industry turned to **specialised accelerators**, and therefore the reason the IP business grew. When a single general-purpose core could no longer be made faster within a power budget, the way to get performance became putting the right fixed-function block next to the processor. **Every FEC, crypto, codec and NPU IP sold today exists because of this graph.** Understanding it lets a design house argue its value proposition in the customer's own terms: not "our block is fast", but "your general-purpose alternative costs x times the energy per operation".

B2. CMOS Logic, Delay and Power



$$t_{pd} \approx 0.69 R_{on} C_L$$

$R_{on} \propto 1/(W/L) \Rightarrow$ bigger device, faster, more input cap

FO4 = delay driving four identical inverters

— the technology-independent speed unit

Figure 4. The inverter, and the delay model every timing number ultimately rests on.

B2.1 Delay models

Table 30. Delay models by fidelity

Model	Form	Use
RC (Elmore)	$t_{pd} = 0.69 R_{on} C_L$	Hand estimates, intuition
Logical effort	$d = gh + p$	Optimal gate sizing and stage count without simulation
NLDM (lookup)	table over input slew $\times$ output load	Standard-cell STA
CCS / ECSM	current-source model	Advanced nodes, accurate slew propagation
SPICE	device equations	Characterisation, critical custom paths

Graduate · Logical effort is the one hand method worth memorising. With g the logical effort of a gate (1 for an inverter, 4/3 for NAND2, 5/3 for NOR2), h the electrical effort (fanout), and p parasitic delay, the path delay is minimised when **every stage carries equal effort** $f = (GH)^{1/N}$, and the optimal per-stage effort is ≈ 3.6 (≈ 4 in practice, whence "fanout-of-4"). This yields the optimal number of stages $N \approx \log_4(GH)$ directly. In an IP context it answers questions such as "how many buffer stages should the clock tree have?" or "is this 12-input decoder better as one stage or three?" before a single tool is run — which matters because those questions arise during architecture, when no netlist exists.

Graduate · FO4 as a unit. Quoting delay in FO4 units makes a design comparable across processes: a 30-FO4 pipeline stage is 30 FO4 whether the node is 130 nm or 5 nm. A measured data point for calibration: in the open SkyWater 130 nm PDK, an inverter chain with $W_p = 2.0 \mu\text{m}$, $W_n = 1.0 \mu\text{m}$, $L = 0.15 \mu\text{m}$ and $C_L = 6 \text{ fF}$ gives **FO4 $\approx 48 \text{ ps}$** (ngspice; stage delays 47.5 and 44.5 ps, rise time 67.9 ps). That implies a practical ceiling near **1 GHz** for a ~ 20 -FO4 pipeline stage in that process — which is the quantitative reason a 56 Gbd analogue decision loop is impossible there, and why an open-PDK project must choose a digital, sub-rate architecture.

B2.2 Power

$$P = \alpha C V_{DD}^2 f + I_{SC} V_{DD} + I_{leak} V_{DD}$$

Table 31. Power components and levers

Component	Driver	Lever	Cost of the lever
Dynamic (switching)	αCV^2f	Clock gating; reduce α by data gating; lower V_{DD}	Gating adds skew and verification states
Short-circuit	Both devices on during transition	Sharp input slews	Bigger drivers $\rightarrow$ more C
Subthreshold leakage	e^{-V_{TH}/nV_T}	HVT cells, power gating, body bias	Slower; wake-up latency; state retention
Gate leakage	Tunnelling through oxide	High- κ dielectric	Process choice, not design
Glitch power	Unbalanced arrival times	Balance logic depth ; pipeline	Area, latency

Graduate · Glitch power is an IP-level design decision that rarely appears in textbooks. In an unbalanced arithmetic tree, a node can toggle several times before settling, and each toggle costs full switching energy. Ripple-carry adders and unbalanced multiplier trees are notorious. Balancing the tree — or inserting a pipeline register that stops glitch propagation — can reduce datapath power by tens of percent at no functional cost. Because glitching is invisible in RTL simulation (it appears only in gate-level simulation with timing), it is systematically underestimated during architecture. **A design house that reports power measured from a back-annotated gate-level run, with the customer's own data pattern, differentiates itself from one that reports a synthesis estimate.**

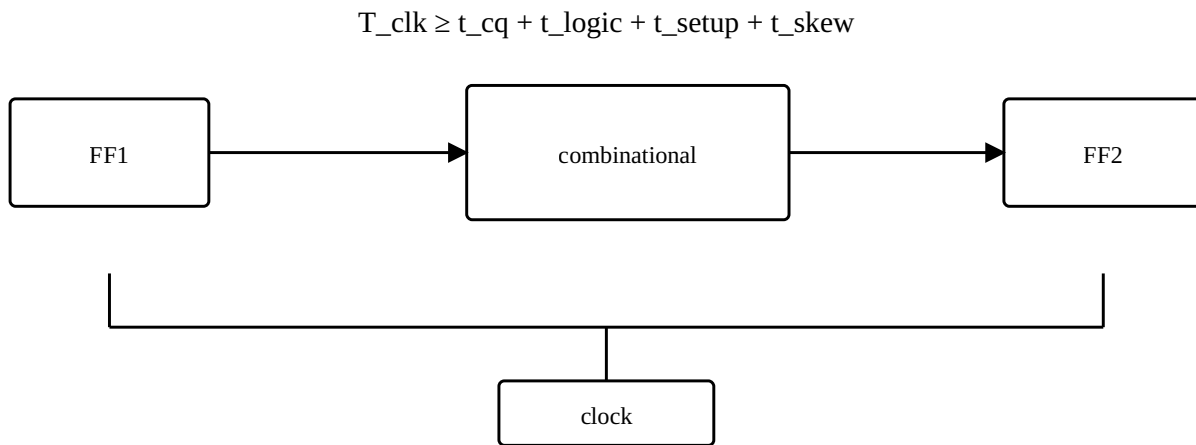
B2.3 Logic families

Table 32. Logic styles

Style	Principle	Advantage	Why it is rarely used in IP
Static CMOS	Complementary pull-up/down	Robust, no clock, easy to verify	— the default
Transmission gate	Pass logic	Fewer devices for MUX/XOR	Threshold loss, charge sharing
Domino / dynamic	Precharge then evaluate	Fast, small	Noise-sensitive, needs custom timing ; not synthesisable
Pass-transistor (CPL)	—	Compact XOR	Level restoration needed
Current-mode (CML/ECL)	Differential current steering	Very high speed, constant current	Static power; used only in SerDes front ends
Adiabatic	Slow charge recovery	Low energy	Needs special clocks; niche

Graduate · A synthesisable IP is static CMOS by definition — everything else requires custom timing verification that a customer cannot repeat in their own flow. **This is a commercial constraint, not a technical preference.** The exception is the SerDes front end, where CML is unavoidable above a few GHz; that is precisely the part that is sold as a hard macro rather than RTL, and it is why the PMA/PCS boundary in a PHY falls where it does.

B3. Timing, Clocking and Static Timing Analysis



$$\text{Hold: } t_{cq} + t_{logic,min} \geq t_{hold} + t_{skew}$$

Setup fails $\Rightarrow$ slow the clock. Hold fails $\Rightarrow$ the chip is dead.

Figure 5. The two timing constraints. Setup limits frequency; hold limits correctness at any frequency.

B3.1 The two inequalities

$$T_{clk} \geq t_{cq} + t_{logic,max} + t_{setup} + t_{skew} + t_{jitter}$$

$$t_{cq,min} + t_{logic,min} \geq t_{hold} + t_{skew}$$

The asymmetry between the two is the single most important operational fact in digital timing. A setup violation is fixed by lowering the clock frequency, so a chip that fails setup still works, just slower. A hold violation has no frequency term: the chip is **broken at every frequency** and the only remedy is a mask change. Consequently hold is fixed exhaustively before tape-out, by buffer insertion, while setup is traded against performance targets. In IP delivery this means the constraint file (SDC) you ship must make the customer's hold analysis possible — omitting clock definitions or false-path declarations can cause a customer to close hold on a path that does not exist, or worse, to miss one that does.

B3.2 Clock distribution

Table 33. Clock network structures

Structure	Skew	Power	Use
H-tree	Low by symmetry	High	Regular arrays, large blocks
Buffered tree (CTS)	Tool-balanced	Moderate	Standard ASIC flow
Mesh / grid	Very low	Very high	High-performance CPUs
Spine + local trees	Moderate	Moderate	Common compromise
Useful skew	Deliberately non-zero	—	Borrow time from a fast stage to a slow one

Graduate · **Useful skew (clock borrowing)** is worth knowing because it turns a timing failure into a scheduling problem. If stage A has slack and stage B does not, delaying B's launch clock moves margin from A to B. The tool does this automatically, but the consequence for an IP vendor is subtle: **a block whose internal paths are already borrowing has little tolerance for the customer's clock tree**. Reporting timing with an assumed skew budget, and stating that budget explicitly in the datasheet, prevents the integration failure where the block meets timing in the vendor's flow and misses it in the customer's.

B3.3 What STA cannot see

Table 34. Limits of static timing analysis

Issue	Why STA misses it	What catches it
False paths	STA is structural, not functional	Designer-declared exceptions — and a wrong declaration hides a real path
Multicycle paths	Same	Declaration plus functional verification
Asynchronous cross-ings	No single clock relation	CDC static analysis , not STA
Metastability	Probabilistic, not deterministic	MTBF calculation; synchroniser depth
Glitches	STA reports arrival, not toggle count	Gate-level power simulation
Crosstalk-induced delay	Depends on aggressor timing	SI-aware STA (signal-integrity mode)
IR drop	Voltage assumed constant	Power/IR co-analysis
On-chip variation	Single delay per cell	OCV / AOCV / POCV derating

Graduate · **Exception declarations are the most dangerous artefact in a timing constraint file**. Each `set_false_path` removes a path from analysis; if the declaration is wrong, that path is simply never checked and the failure appears only in silicon. A mature IP deliverable therefore ships its SDC with a written justification for every exception, and ideally with an assertion or formal property that proves the path is genuinely unexercisable. **This is a place where a modelling engineer contributes directly: the argument that a path cannot be sensitised is a functional argument, and the person who wrote the reference model is usually the one who can make it.**

B3.4 Metastability quantified

$$MTBF = e^{t_r/\tau} / (T_0 f_{clk} f_{data})$$

Graduate · The exponential in t_r (resolution time allowed) is why adding one synchroniser stage improves MTBF by orders of magnitude rather than by a factor of two. With typical 28 nm flip-flop parameters ($\tau \approx 20$ ps), one extra stage at 1 GHz multiplies MTBF by $e^{1000/20} = e^{50}$ — an astronomically large factor. The corollary is that **two stages is almost always enough and three is almost always superstition**, except at very high clock rates where t_r per stage is small. Being able to compute this rather than follow a rule is what lets an IP vendor justify a latency budget to a customer who wants the crossing to be faster.

B4. Interconnect, Transmission Lines and Signal Integrity

Below roughly 250 nm, wires stopped being ideal connections and became circuit elements. Above a few GHz they stop being lumped elements and become transmission lines. Both transitions change what a designer must do.

B4.1 On-chip interconnect as distributed RC

$$t_{\text{delay}} \approx 0.38 R_w C_w L^2 \quad (\text{distributed RC line of length } L)$$

Table 35. Interconnect scaling and its remedies

Problem	Cause	Remedy	Cost
Delay grows as L^2	Distributed RC	Repeater insertion → delay becomes linear in L	Power and area; repeaters can be 30% of a large SoC's dynamic power
Resistance rises with scaling	Smaller cross-section, surface scattering	Thicker upper metal layers for global routes	Fewer routing tracks
Coupling capacitance dominates	Tall narrow wires, close spacing	Increase spacing; shield with ground lines	Routing area
Electromigration	Current density in narrow wires	Width rules, current limits	Area; constrains power grid design
IR drop	Resistance of the power grid	Grid density, decoupling capacitance	Metal resources

Graduate · Optimal repeater insertion follows from minimising the total delay of a line broken into k segments each driven by a repeater of size h . The classic result is that the optimum makes the repeater's intrinsic delay comparable to the wire segment delay, giving $k_{\text{opt}} = L\sqrt{(R_w C_w)/(2R_0 C_0)}$. The practical consequence for an IP vendor is that **a block's timing cannot be guaranteed independent of its physical size**: a 2 mm-wide block has internal wire delays that a synthesis-only estimate will miss entirely. This is why serious IP deliverables include a floorplan hint or a hardened layout, and why "it met timing in synthesis" is not a claim a customer will accept for a large block.

B4.2 When a wire becomes a transmission line

Undergraduate review · A connection must be treated as a transmission line when its electrical length is a significant fraction of the signal's wavelength — in practice when the propagation delay exceeds roughly one tenth of the signal rise time. Since t_r shrinks with every process generation, the threshold length keeps falling.

$$Z_0 = \sqrt{L/C} \quad v_p = 1/\sqrt{LC} = c/\sqrt{\epsilon_r} \quad \Gamma = (Z_L - Z_0)/(Z_L + Z_0)$$

Table 36. Transmission-line effects in a link

Effect	Physical cause	Frequency behaviour	Consequence
Conductor (skin) loss	Current crowds into a skin depth $\delta \propto 1/\sqrt{f}$	$\propto \sqrt{f}$	Dominant loss below ~20 GHz on PCB
Dielectric loss	$\tan\delta$ of the substrate	$\propto f$	Dominant at high frequency; chooses the laminate

Effect	Physical cause	Frequency behaviour	Consequence
Reflections	Impedance discontinuities (vias, connectors)	Resonant dips	Return loss; creates ISI that FFE cannot fully fix
Crosstalk (NEXT/FEXT)	Mutual L and C between traces	Rises with f	Limits lane density
Fibre weave effect	Inhomogeneous ϵ_r of woven glass	—	Skew between differential pair halves
Mode conversion	Asymmetry in a differential pair	—	Differential energy becomes common mode → EMI

Graduate · Why skin effect makes channel loss go as $\sqrt{f}$ and why that shapes the equaliser. Skin depth $\delta = \sqrt{2\rho/\omega\mu}$ shrinks as $1/\sqrt{f}$, so the effective conductor cross-section shrinks and resistance rises as $\sqrt{f}$. The resulting channel magnitude response is approximately $\exp(-k\sqrt{f})$, which in the time domain produces a **long, slowly decaying pulse tail** rather than a sharp one. That specific tail shape is why decision-feedback equalisation works so well on wireline channels: the post-cursor ISI is long but the symbols causing it have already been decided, so they can be subtracted exactly without noise enhancement. **The physics of the conductor selects the equaliser architecture.** An engineer who knows only that "DFE cancels post-cursor ISI" cannot explain why a different channel — say a multipath radio channel with pre-cursor energy — needs a different structure.

B4.3 S-parameters

Table 37. Scattering parameters for a two-port

Parameter	Meaning	Specification name
S_{11}	Input reflection	Return loss (dB, negative is good)
S_{21}	Forward transmission	Insertion loss
S_{12}	Reverse transmission	Isolation
S_{22}	Output reflection	Output return loss
S_{dd}, S_{cc}	Differential / common mode	Mixed-mode S-parameters
S_{cd}, S_{dc}	Mode conversion	Asymmetry metric — an EMI predictor

Graduate · S-parameters are used because at high frequency you cannot measure voltage and current at a port without perturbing it, but you can measure incident and reflected power waves. For a modelling engineer the practical point is the **conversion to a time-domain model**: a channel is delivered as a Touchstone (.s4p) file, and a link simulation needs its pulse response. Doing that conversion correctly requires enforcing **causality and passivity** on measured data that generally violates both because of measurement noise and band limitation. Non-causal channel models produce link simulations that are optimistic in a way that is very hard to notice. **A channel-model conditioning tool is a small, genuinely sellable piece of software**, and it is squarely modelling-team work.

B4.4 Power delivery and simultaneous switching noise

$$V_{noise} = L_{package} \cdot dI/dt + R_{grid} \cdot I$$

Table 38. Power integrity mechanisms

Mechanism	Where	Mitigation
Static IR drop	On-chip grid resistance	Grid density, more straps
dI/dt noise (SSN)	Package and board inductance	Decoupling capacitance at three levels: on-die, package, board
Resonance	L_{pkg} with on-die C	Anti-resonance peak typically 50–200 MHz ; add damping
Ground bounce	Shared return inductance	More ground pins, differential IO
Supply-induced jitter	V_{DD} modulates delay	Regulate locally; use supply-insensitive delay cells in clock paths

Graduate · Supply-induced jitter is the mechanism by which a *digital* design decision damages an *analogue* specification. A large synchronous block that starts and stops — a clock-gated accelerator, a burst-mode FEC decoder — produces a current step at the burst rate, which modulates the supply, which modulates the delay of the PLL's delay cells, which appears as a spur in the phase noise, which appears as periodic jitter (PJ) in the link budget. **The FEC block is the root cause of a jitter specification failure**, and nothing in either block's own verification would reveal it. This class of cross-domain coupling is why chip-level power-aware simulation exists, and why an IP datasheet should state the block's *current profile*, not merely its average power.

B5. Analogue Building Blocks

A digital IP house does not design these, but it must model them, specify them, and understand which digital block can compensate which analogue imperfection — because that compensation is often the sellable part.

B5.1 Amplifiers

Table 39. Amplifier topologies

Topology	Gain	Bandwidth	Where used
Common source	$-g_m r_o$	Moderate	Basic gain stage
Cascode	$-g_m (g_m r_o) r_o$	High (reduced Miller)	High-gain stages
Differential pair	—	—	Rejects common-mode noise ; the basis of nearly everything
Folded cascode	High	Good	Wide input range op-amps
Two-stage Miller	Very high	Compensated	General-purpose op-amp
CTLE	peaked	—	Source degeneration creates a zero that boosts high frequency
TIA	$-R_f$	—	Photodiode front end

Graduate · **Why a CTLE amplifies noise and an FFE does not have to.** A CTLE is a linear analogue filter placed *before* the point where noise is added by the rest of the receiver, but *after* the channel has already attenuated the signal and after thermal noise from the channel and the front end. Boosting high frequency therefore boosts the noise already present in that band by the same factor. A digital FFE placed after the ADC has exactly the same problem. **ADFE** escapes it because it subtracts a *decided* symbol — a noiseless quantity — rather than amplifying a received one. This is the whole reason receivers combine a modest CTLE with a DFE instead of using a large CTLE alone, and it is a piece of reasoning that a modelling engineer must be able to reproduce when asked to justify an architecture.

B5.2 Comparators and references

Table 40. Comparator and reference issues

Block	Key imperfection	Digital compensation
Strong-arm latch	Input-referred offset (mismatch)	Offset calibration DAC trimmed digitally
	Metastability when inputs are close	Allow regeneration time; detect and flag
	Kickback onto the input	Sample-and-hold isolation
Bandgap reference	Curvature over temperature	Piecewise digital trim
	Start-up failure mode	Start-up circuit plus watchdog
Current DAC	Element mismatch	Dynamic element matching (digital)

Graduate · Each row's right-hand column is a digital block with a clean specification, and that is the commercial observation: **the calibration and trim logic around an analogue block is frequently larger than the analogue block itself, and it is synthesisable.** A design house that cannot build a comparator can still own the offset-calibration engine, the DEM scrambler, and the background interleaving-mismatch corrector. These are sold as part of converter IP, and they are where the modelling effort concentrates because their correctness is statistical rather than logical.

B5.3 PLLs and clock generation

$$H(s) = K_{PD}K_{VCO}F(s) / (s + K_{PD}K_{VCO}F(s)/N)$$

Table 41. PLL noise transfer

Noise source	Transfer to output	Design implication
Reference noise	Low-pass ($\times N$ in band)	In-band noise is multiplied by $20 \log_{10} N$
VCO noise	High-pass	Wide loop suppresses VCO noise
Divider noise	Low-pass	—
Charge-pump noise	Low-pass	Dominates in-band floor
$\Sigma\Delta$ quantisation (fractional-N)	High-pass shaped	Loop must be narrow enough to filter it

Graduate · The opposing shapes create the **optimal loop bandwidth**: wide enough to suppress VCO phase noise, narrow enough to suppress reference and fractional- N quantisation noise. The optimum is where the two contributions cross. This is a genuinely quantitative design decision that a system modeller can and should compute, and it is directly connected to the link jitter budget of Chapter A4 — the integrated phase noise becomes the RJ term that is multiplied by 14 at 10^{-12} BER. **Chapters A4, B4 and B5 are three views of the same budget**, and one of the marks of a senior engineer is holding them together.

B5.4 Data converters

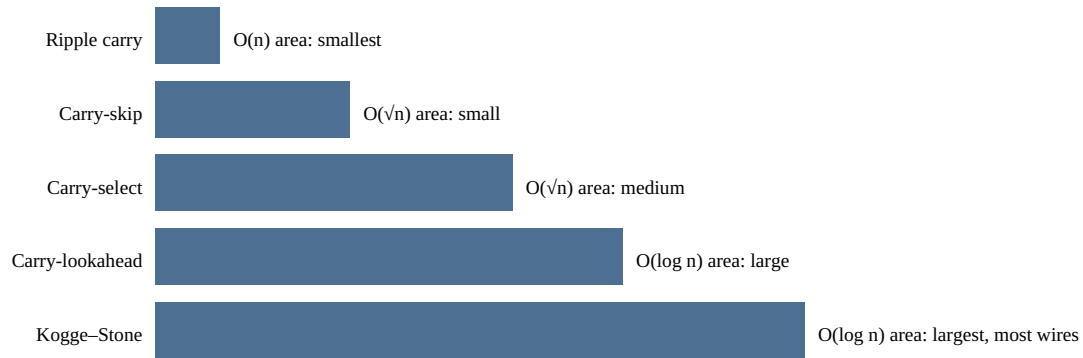
Table 42. ADC architectures

Architecture	Speed	Resolution	Principle	Digital support needed
Flash	Highest	≤ 6 bit	$2^N - 1$ comparators	Thermometer $\rightarrow$ binary, bubble correction
SAR	Moderate–high	8–14 bit	Binary search with a DAC	Redundancy and digital error correction
Pipeline	High	10–14 bit	Stages with residue amplification	Inter-stage gain calibration
$\Sigma\Delta$	Low–moderate	16–24 bit	Oversample and noise-shape	Decimation filter (CIC + FIR)
Time-interleaved SAR	>10 GS/s	6–8 bit	M SARs in parallel	Offset/gain/skew background calibration — the 112G receiver case
Folding/interpolating	High	8–10 bit	Reduce comparator count	—

Graduate · **$\Sigma\Delta$ noise shaping** is worth stating exactly because it is the clearest example of trading bandwidth for resolution. An L th-order modulator with oversampling ratio OSR gives SNR gain $\approx (6L+3)$ dB per doubling of OSR, versus 3 dB for plain oversampling. The price is that the modulator output is a low-resolution, very fast stream that must be decimated — and **that decimation filter is a substantial digital IP block** whose passband ripple and stopband attenuation determine the converter's final specification. A converter vendor's datasheet number is in part a statement about a digital filter. This is the second time in this chapter that the saleable digital block sits next to an analogue one.

C1. Arithmetic Units — The Core of Every Datapath

C1.1 Addition



Same function. Five points on the area–delay curve.

Figure 6. Adder architectures span an area–delay curve of roughly an order of magnitude in each axis.

Table 43. Adder architectures

Architecture	Delay	Area	Notes
Ripple carry	$O(n)$	$O(n)$	Baseline; fine for $n \leq 8$ at low frequency
Carry-skip	$O(\sqrt{n})$	$O(n)$	Cheap improvement
Carry-select	$O(\sqrt{n})$	$\sim 2\times$	Duplicate blocks, select on carry
Carry-lookahead	$O(\log n)$	$O(n \log n)$	Classic
Kogge–Stone	$O(\log n)$	Large, heavy wiring	Minimum depth; wire-limited in practice
Brent–Kung	$O(\log n)$	Fewer wires	Depth $2 \log n$; better area
Carry-save	$O(1)$ per add	$O(n)$	Does not propagate carry — the key to fast multipliers and accumulators

Graduate · **Carry-save is the single most useful idea in this table.** A carry-save adder takes three operands and produces two (sum and carry vectors) in constant time, because no carry propagates. A chain of them accumulates many terms at constant delay per term, and only the *final* conversion to a normal binary number requires a carry-propagate adder. This is why multiplier trees, FIR accumulators, and MAC units are built as "compress many partial products with carry-save, then one fast adder at the end". **In an accumulation loop — the feedback path that limits every recursive filter — keeping the state in carry-save form removes the carry-propagate delay from the loop entirely.** That single transformation has rescued more timing-critical datapaths than any other.

C1.2 Multiplication

Table 44. Multiplier construction

Stage	Technique	Effect
Partial product generation	Booth radix-4	Halves the number of partial products; needs 2's-complement handling
	Booth radix-8	Fewer still, but needs a 3× multiple (extra adder)
Reduction	Wallace tree	$O(\log n)$ depth, irregular layout
	Dadda tree	Slightly fewer adders than Wallace
	Array	Regular layout, $O(n)$ depth
Final add	Fast carry-propagate adder	Kogge–Stone or similar
Whole unit	Pipelining	Cut the tree with registers to raise $f_{\max}$

Graduate · **Constant multiplication is a different problem** and is worth separating in any specification. When one operand is a compile-time constant — filter coefficients, twiddle factors, scaling constants — the multiplier collapses into shifts and adds. Canonical signed digit (CSD) representation minimises the number of non-zero digits, and **multiple constant multiplication (MCM)** algorithms share sub-expressions across a whole coefficient set. For a 32-tap FIR with fixed coefficients this routinely gives a 3–5× area reduction over generic multipliers. The commercial point: **an IP that is parameterised for run-time coefficients cannot use this, so "programmable coefficients" is an expensive feature** that should be priced and specified as such, not thrown in because it seems flexible.

C1.3 Division, square root, and reciprocal

Table 45. Division approaches

Method	Convergence	Hardware	Use
Restoring / non-restoring	1 bit per cycle	Adder + shifter	Small area, high latency
SRT (radix-4/8)	2–3 bits per cycle	Lookup for quotient digit	Standard in FPU's
Newton–Raphson	Quadratic (doubles digits)	Multipliers	Reuses the multiplier; good when one exists
Goldschmidt	Quadratic	Multipliers	Better pipelining than Newton
Reciprocal LUT + refine	—	Small ROM + multiplier	The usual answer in DSP

The most important division fact for an IP designer is architectural, not numerical: never put a divider in a feedback loop. Its latency enters the loop directly and sets the initiation interval. The standard transformation is to compute a reciprocal *once* outside the loop and multiply inside it. The Cholesky decomposition is a textbook example — the diagonal reciprocal is formed once per column and the column is then scaled by multiplication, which is why the inner loop can be pipelined at $II = 1$. This is also a place where the reference model and the RTL can legitimately differ: the model may divide, the RTL multiplies by a reciprocal, and the two agree only to within the reciprocal's rounding. **The specification must state which one is normative.**

C1.4 Fixed-point and floating-point units

Table 46. Number format decisions

Format	Range	Hardware cost	Where it belongs
Fixed point $Q_{m.n}$	Limited, uniform	Adder/multiplier only	Most DSP and comms datapaths
Block floating point	Shared exponent per block	Small extra	FFT, where dynamic range grows predictably

Format	Range	Hardware cost	Where it belongs
IEEE 754 binary32	Wide	Align, normalise, round — expensive	Scientific, some solvers
binary16 / bfloat16	Wide, low precision	Cheaper	Neural inference
Integer with saturation	Limited, clamped	Compare and clamp	Where wrap would be catastrophic (LLRs, audio)
Posit / logarithmic	Tapered	Unusual	Research; rarely in shipped IP

Graduate · **Block floating point deserves more attention than it gets.** In an FFT the signal magnitude grows by up to a factor of two per stage. Full floating point is overkill; fixed point requires either wide words or per-stage scaling that loses SNR. Block floating point keeps one exponent per stage (or per block) and normalises once, recovering most of floating point's dynamic range at a small fraction of the cost. **For a modelling engineer this format is a trap**, because a naive floating-point reference model will not match the hardware's block-normalised behaviour bit for bit — the reference must model the normalisation decisions too. Specifying "bit-accurate reference model" without specifying the scaling policy produces months of false mismatches.

C2. Control: State Machines, Pipelines and Flow Control

C2.1 FSM encoding

Table 47. State encodings

Encoding	Re-gisters	Next-state logic	When
Binary	$\log_2 N$	Complex	Many states, area critical
Gray	$\log_2 N$	Complex	Adjacent states differ by one bit — low power, safer across CDC
One-hot	N	Simple (one AND per transition)	FPGA default ; fast; easy to check
Johnson	$N/2$	Simple	Counters, ring sequencers
Hamming-protected	$>\log_2 N$	—	Safety/security : illegal states detectable

Graduate · **Security-grade FSM encoding** is a concrete example of a requirement that comes from a threat model rather than from function. If an attacker can flip a state bit with a laser or a voltage glitch, a binary-encoded FSM may land in a valid but wrong state — for instance skipping rounds of a cipher. Countermeasures are: sparse encodings with large Hamming distance so that a single flip lands in an illegal state; a default branch that raises an alarm rather than doing nothing; and **duplicated FSMs with complementary encodings** whose outputs must remain complementary. The last technique requires a synthesis barrier so the tool does not optimise the inverters away and merge the two copies — if it does, the redundancy disappears while *functional verification still passes*. That silent failure mode is the reason such designs contain explicit buffer primitives whose only purpose is to stop the optimiser.

C2.2 Pipelining and its hazards

Table 48. Pipeline transformations

Transformation	What it does	Cost	Limit
Pipelining	Insert registers between logic stages	Latency, area	Cannot cut a feedback loop
Retiming	Move existing registers across logic	None (if legal)	Total registers on each loop is invariant
Loop un-rolling	Process k iterations per cycle	$k \times$ area	The loop's critical path is divided by k only in the feedforward part
Speculation	Compute all outcomes, select afterwards	Exponential in the number of speculated bits	Removes the decision from the loop
C-slow	Interleave k independent streams	Registers $\times k$	Needs k independent problems
Algebraic transform	Restructure the recurrence itself	Design effort	The only way past a fundamental loop bound

Graduate · **The iteration bound** is the hard limit that these transformations negotiate with. For a recursive system, the minimum achievable iteration period is $T_\infty = \max_{\text{loops}} (t_{\text{loop}}/w_{\text{loop}})$, where t_{loop} is the total computation delay around a loop

and w_{loop} the number of registers in it. Retiming and pipelining cannot beat it; only changing the recurrence can. Recognising this quantity turns a vague "we need more speed" into a specific question: *which loop, and can its algebra be rewritten?* Look-ahead transformation (computing two steps of a recurrence in one) doubles w and thereby halves T_{∞} , at the cost of more arithmetic. **This is the abstract form of the unrolled DFE, radix-4 Viterbi ACS, and reformulated Berlekamp-Massey — three blocks that look unrelated and are the same transformation.**

C2.3 Flow control and backpressure

Table 49. Handshake styles

Style	Signals	Property	Hazard
Valid only	valid	Producer never stalls	Consumer must always accept — needs guaranteed rate
Valid/Ready	valid, ready	Full backpressure	Combinational loop if ready depends on valid
Credit-based	credits returned	Decouples latency from throughput	Credit accounting bugs
Req/Ack (4-phase)	(4- handshake pairs)	Async-safe	Slow (two round trips)

The valid/ready combinational loop is one of the most common integration failures in modular RTL. If module A's ready depends combinationaly on its input valid, and module B's valid depends combinationaly on its input ready, connecting them creates a zero-delay cycle. Each module is individually correct and passes its own tests. The standard remedies are a *skid buffer* (also called a spill register), which breaks the path at the cost of one register stage and one entry of buffering, or a rule that ready is registered. **An IP that documents which of its interfaces are combinationaly coupled — and ships a skid buffer — saves its customers a week each.** This is exactly the kind of detail that distinguishes a product from a piece of code.

C2.4 Arbitration

Table 50. Arbiter types

Type	Fairness	Latency	Use
Fixed priority	None (starvation possible)	Lowest	Strict QoS classes
Round robin	Fair	Low	General purpose
Weighted round robin	Proportional	Low	Mixed traffic classes
Matrix / LRU	Fair, true least-recently-used	Higher	Caches
Lottery	Statistical	Low	Rare
Age-based	Strict FIFO fairness	Higher	Avoids starvation strictly

Graduate · Arbitration is where **fairness and forward progress** become formal properties rather than intentions. A round-robin arbiter is fair by construction; a fixed-priority arbiter is not, and a low-priority requester can starve forever. Whether starvation matters is a system question — it is acceptable for a debug port, fatal for a DMA that feeds a real-time codec. Because starvation is a *liveness* property ("eventually granted") rather than a safety property, it is natural to express and prove with formal tools, and arbiters are one of the few places where formal verification is both easy and genuinely valuable. **Shipping a formally proven arbiter with its properties is a differentiator that costs a design house little.**

C3. Clock Domain Crossing, Reset and Low Power

C3.1 The CDC problem stated precisely

Undergraduate review · A signal crossing between unrelated clock domains violates setup or hold at the receiving flip-flop with non-zero probability. The flip-flop may enter a metastable state whose resolution time is unbounded (though exponentially unlikely to be long). **This cannot be simulated:** a logic simulator has no metastable value and will always resolve to 0 or 1.

Table 51. CDC techniques by payload type

Payload	Technique	Why	Failure if misapplied
1-bit level	Two-flop synchroniser	Allows a full cycle to resolve	—
1-bit pulse	Toggle synchroniser (level convert, then edge detect)	A pulse shorter than the destination period would be lost	Lost events
Multi-bit, unrelated	Synchronise a request; use a data hold register	Bits arriving in different cycles corrupt the value	Corrupted data words
Multi-bit counter	Gray code	Only one bit changes per increment	Wrong pointer → FIFO corruption
Stream	Asynchronous FIFO (Gray pointers, dual-port RAM)	Decouples rate and phase	—
Bus transaction	Handshake (req/ack) or credit	Explicit acknowledgement	—
Reset	Asynchronous assert, synchronous de-assert	De-assertion must not violate recovery/removal	Part of the design leaves reset a cycle late

Graduate · **Why Gray coding a FIFO pointer is safe even though the pointer is multi-bit.** The synchroniser may sample a Gray pointer mid-transition, but since only one bit changes per increment, the sampled value is either the old or the new pointer — never a third value. An off-by-one pointer makes the FIFO report itself slightly *fuller* or *emptier* than it is, which is conservative: the FIFO may stall a cycle early but never overflows or underflows. **The design is safe because its error mode is biased in the harmless direction.** This principle — arrange for uncertainty to fail conservatively — generalises far beyond FIFOs and is one of the most transferable ideas in digital design.

C3.2 Reset architecture

Table 52. Reset strategies

Strategy	Advantage	Problem
Synchronous	No recovery/removal issue; clean STA	Needs a running clock — fails at power-up
Asynchronous	Works without a clock	De-assertion can violate recovery
Async assert, sync de-assert	Both advantages	Requires a reset synchroniser per domain — the standard solution
Reset-less registers	Area and routing saving	Must be provably initialised by data flow; a verification obligation

Reset de-assertion order is a specification item that is almost always omitted. If two blocks leave reset in different cycles and one samples the other's outputs, the receiving block may latch garbage. Within a block the designer controls this; across an IP boundary the *customer* controls it, and they cannot get it right unless the datasheet states the requirement. A good IP datasheet specifies: the reset polarity, whether it is synchronous or asynchronous, the minimum assertion width in cycles of each clock, and any required ordering between multiple resets. **Omitting this produces intermittent bring-up failures that are blamed on the IP.**

C3.3 Low power design

Table 53. Power reduction techniques by level

Level	Technique	Saving	Verification burden
RTL	Clock gating	Large (clock tree is 20–40% of dynamic power)	Low — tools insert it automatically
RTL	Operand isolation / data gating	Moderate	Low
RTL	Reduce switching activity (encoding, bus inversion)	Moderate	Low
Architecture	Parallelism + voltage scaling	Large — power $\propto V^2$	Moderate
Architecture	Memory hierarchy (avoid far accesses)	Large	Moderate
Physical	Multi- V_{TH} cells	Leakage	None
System	Power gating	Leakage $\rightarrow \sim 0$	High: isolation, retention, wake sequencing
System	DVFS	Large	High: timing at every operating point
System	Multiple power domains	Large	Highest: UPF, isolation cells

Graduate · Parallelism as a power technique is the least intuitive entry and the most powerful. Two copies of a block running at half the clock frequency deliver the same throughput; because they now have twice the time budget, the supply voltage can be lowered until the slower path just meets timing. Dynamic power scales as V^2f , so halving f and reducing V by, say, 30% gives roughly $0.5 \times 0.49 = 0.25$ of the original power, at the cost of about twice the area. **This trade — area for energy — is the fundamental reason accelerators are more efficient than processors**, and it is the argument an IP vendor uses when selling a block against a software implementation. State it with the exponents and it is persuasive; state it as "our block is more efficient" and it is not.

C3.4 Design for test

Table 54. DFT structures

Structure	Tests what	Cost	IP implication
Scan chains	All flip-flops and combinational logic	Mux in every FF (~5–10% area, some delay)	IP must be scan-insertable: no gated clocks without test bypass, no latches, no async loops
MBIST	Embedded memories	Controller per memory group	Memory interfaces must be accessible
BSD (JTAG 1149.1)	Board-level interconnect	Boundary cells	—

Structure	Tests what	Cost	IP implication
LBIST	Logic, in-field	PRPG + MISR	Automotive/safety
ATPG coverage	—	—	Stuck-at >99%, transition >90% typical targets
Test compression	—	Decompressor/compactor	Reduces tester time and cost

Scan is a security hole. A scan chain can read and write every flip-flop in the design, including key registers. Production chips must therefore disable scan after manufacturing test, typically by blowing a fuse or advancing a life-cycle state. A security IP that does not document its scan-isolation requirement will be integrated insecurely. **Conversely, an IP that cannot be scanned at all is unsellable**, because the customer cannot achieve their test coverage target. The resolution — scannable, but with key registers excluded from the chain or cleared on scan entry — must be designed in, and it is a requirement that comes from neither function nor timing.

D1. Instruction Set Architecture

An ISA is a contract between hardware and software. For a modelling engineer it is the *specification document* in the most literal sense: the reference model **is** the ISA, written executably.

Table 55. ISA design axes

Axis	Options	Consequence for hardware
Operand model	Stack / accumulator / register-memory / load-store	Load-store simplifies pipelining; every RISC uses it
Instruction length	Fixed / variable / compressed	Fixed simplifies fetch; compressed (RVC, Thumb) saves 25–30% code size at the cost of alignment logic
Addressing modes	Few / many	Many modes need an address generation unit; few modes need more instructions
Condition handling	Flags / compare-and-branch	Flags create a serialising dependency ; RISC-V deliberately has none
Delay slots	Present (classic MIPS) / absent	Delay slots expose the pipeline in the ISA — a design mistake later reversed
Register count	8 / 16 / 32 / more	More registers reduce spills but widen every instruction field
Exceptions	Precise / imprecise	Precise exceptions constrain the whole microarchitecture

Graduate · Delay slots are the canonical example of leaking microarchitecture into architecture. MIPS exposed its single branch-delay slot so that the compiler could fill it, saving a stall in a five-stage pipeline. When later implementations had deeper pipelines and branch prediction, the delay slot became useless baggage that every implementation still had to honour. **The lesson for IP design generalises: a specification that encodes today's implementation constrains tomorrow's.** When writing a block's interface, ask which parts describe *what* and which describe *how*, and expose only the former.

Graduate · Precise exceptions mean that when an exception is taken, all instructions before it have completed and none after it has had any effect. This is what makes an ISS a valid reference: the model can execute one instruction at a time and still describe a machine that executes many concurrently. Achieving it in an out-of-order machine requires a reorder buffer and speculative state, which is a large fraction of such a core's complexity. **The entire step-and-compare verification methodology rests on precise exceptions**; without them there would be no well-defined point at which model and RTL should agree.

D1.2 RISC-V as a case study

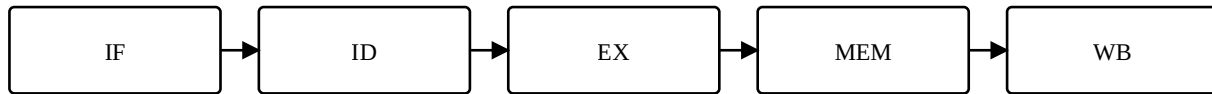
Table 56. RISC-V structure

Element	Content	Modelling note
Base integer	RV32I / RV64I — ~40 instructions	Small enough to model exhaustively
M	Multiply/divide	Division by zero is defined , not trapping
A	Atomics (LR/SC, AMO)	Requires a memory consistency model
F/D/Q	Floating point	IEEE 754; needs SoftFloat, not host arithmetic
C	Compressed 16-bit	Alignment: instructions may straddle a word
V	Vector	Vector-length agnostic — a model must track <code>v1</code> , <code>vtype</code>
B, Zk, Zc...	Bit manipulation, crypto, code size	—

Element	Content	Modelling note
Privileged	M/S/U modes, CSRs, PMP, virtual memory	Where most model bugs live: exception priority and CSR side effects

Exception priority is the most common source of reference-model defects in a CPU project. One instruction can simultaneously satisfy the conditions for several exceptions — misaligned address, page fault, access fault, breakpoint. The specification gives a strict priority order. A model that implements a different order is correct on almost every test and wrong on a narrow class of inputs that constrained-random stimulus may take weeks to find. The remedy is not more random testing but **directed tests derived from the priority table itself:** enumerate the table, construct one case per row, and check. This is a half-day of work that reliably finds bugs a month of random testing misses.

D2. Pipelining, Hazards and Speculation



Throughput 1 instr/cycle; latency 5 cycles

hazards: structural · data · control

Forwarding removes most data hazards; load-use needs one stall

Control hazards cost the branch penalty × misprediction rate

Figure 7. The classic five-stage pipeline and its three hazard classes.

Table 57. Hazards and remedies

Hazard	Cause	Remedy	Residual cost
Structural	Two instructions need the same resource	Duplicate the resource, or stall	Area
Data RAW	Read after write	Forwarding from EX/MEM	Load-use still costs one cycle
Data WAR / WAW	Anti- and output dependencies	Register renaming	Only in out-of-order designs
Control	Branch outcome unknown at fetch	Prediction + speculative execution	Misprediction penalty × rate
Memory ordering	Loads and stores to the same address	Store buffer with forwarding, memory disambiguation	Complexity

Table 58. Branch prediction

Predictor	Mechanism	Typical accuracy
Static (backward taken)	Loop heuristic	~65%
Bimodal	2-bit saturating counter per PC	~85%
gshare	Global history XOR PC indexes the table	~93%
Tournament	Choose between local and global predictors	~95%
TAGE	Multiple history lengths with tags	>97%
Perceptron	Linear classifier over history	>97%
Return address stack	Dedicated stack for call/return	>99% for returns

Graduate · Why a misprediction is expensive and getting more so. The penalty is the pipeline depth from fetch to resolve. In a 5-stage machine that is 2–3 cycles; in a 15-stage out-of-order machine it is 15–20, and the machine may have to discard dozens of in-flight instructions. With a 5% miss rate and a 15-cycle penalty, branches alone cost 0.75 cycles per branch — often more than every other stall combined. This is why predictor research continues to be worthwhile and why deep pipelines

have diminishing returns. **For an accelerator IP designer the lesson is the opposite and liberating: a fixed-function datapath has no branches, so none of this cost exists.** Quantifying that difference — "your software loop spends 0.75 cycles per iteration on branch misprediction alone" — is one of the strongest arguments an accelerator vendor has.

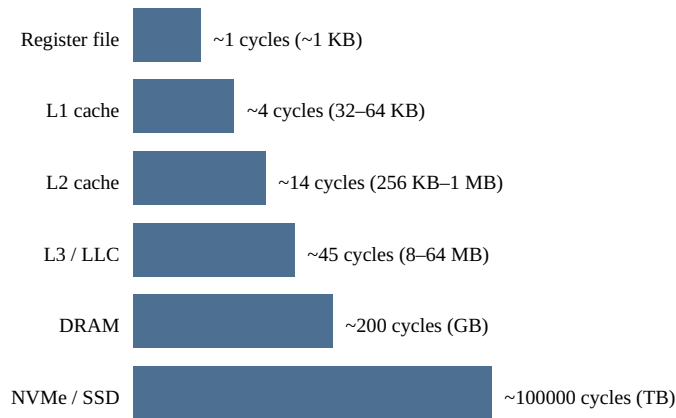
D2.2 Out-of-order execution

Table 59. Out-of-order machinery

Structure	Purpose	Scaling limit
Register renaming (RAT)	Removes WAR/WAW	Physical register file size
Reservation stations / issue queue	Hold instructions until operands ready	Wakeup-select loop is the critical path
Reorder buffer (ROB)	In-order commit → precise exceptions	Entries scale with memory latency
Load/store queue	Memory disambiguation	Associative search cost
Scoreboard	Simpler alternative (no renaming)	Stalls on WAR/WAW

Graduate · The **wakeup-select loop** — broadcasting a result tag, waking dependent instructions, selecting which to issue — must complete in one cycle for back-to-back dependent execution. It is a broadcast across the whole issue queue, so its delay grows with queue size and with issue width. This single loop, not transistor speed, is why issue widths have plateaued around 4–8 and why further performance has come from more cores rather than wider ones. **It is the same phenomenon as the DFE feedback loop and the Viterbi ACS loop: a recurrence whose latency cannot be pipelined away.** Recognising the pattern across domains is exactly the transferable skill this book aims to build.

D3. Memory Hierarchy and Coherence



Each level is roughly an order of magnitude slower and larger

Figure 8. The memory hierarchy. Each level trades capacity against latency by roughly an order of magnitude.

Table 60. Cache design parameters

Parameter	Effect	Trade
Block size	Exploits spatial locality	Too large wastes bandwidth and causes false sharing
Associativity	Reduces conflict misses	More comparators, longer hit time; 4–8 way is the usual optimum
Capacity	Reduces capacity misses	Area, leakage, hit latency
Write policy	Write-through vs write-back	Write-back reduces bandwidth; needs dirty bits and coherence
Allocation	Write-allocate vs not	Depends on write locality
Replacement	LRU / pseudo-LRU / random	True LRU is expensive beyond 4-way; pseudo-LRU is within 1%
Inclusion	Inclusive / exclusive / NINE	Affects snoop filtering

Undergraduate review · Misses classify as **compulsory** (first reference), **capacity** (working set exceeds the cache), **conflict** (limited associativity), and in multiprocessors **coherence** (invalidated by another core). The classification is useful because each class has a different remedy: prefetching for compulsory, larger cache for capacity, higher associativity or better indexing for conflict, and data layout changes for coherence.

Table 61. Coherence protocols

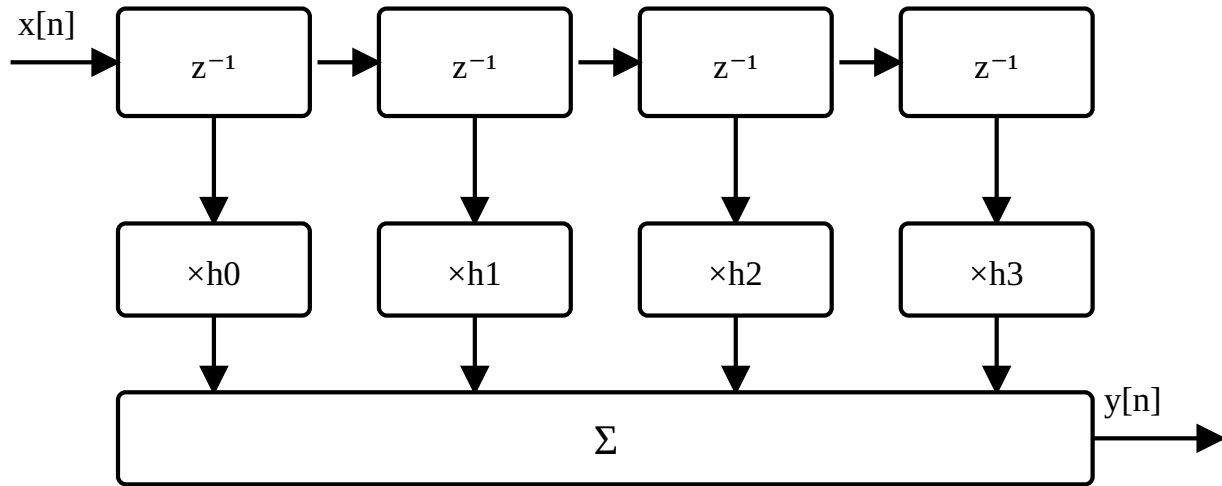
Protocol	States	Note
MSI	Modified, Shared, Invalid	Minimal
MESI	+ Exclusive	Avoids a bus transaction on read–modify
MOESI	+ Owned	Allows dirty sharing; saves writebacks
Directory-based	—	Scales beyond a bus ; required for many-core
Snoop filter	—	Reduces useless snoops in large systems

Graduate · Memory consistency is not coherence. Coherence concerns a single address — all cores must see one order of writes to it. Consistency concerns the relative order of operations to *different* addresses. A machine can be perfectly coherent and still reorder a store to A before a store to B as observed by another core. The consistency model (sequential, TSO, release, RVWMO) specifies what reorderings are allowed and which fences prevent them. **For a modelling engineer this is the hardest specification to capture**, because the reference model is inherently sequential while the hardware is not: a simple step-and-compare against an ISS *cannot* verify a weak memory model. The correct approach is **litmus tests** — short multi-threaded programs with enumerated permitted outcomes — checked against the architecture's formal memory model. Knowing that a different verification technique is required, and why, is the point.

Table 62. DRAM characteristics that leak into system design

Property	Meaning	Design response
Row buffer	A row must be activated before access	Exploit row locality; schedule for open rows
$t_{\text{RCD}}, t_{\text{RP}}, t_{\text{RAS}}$	Activate/precharge timings	Memory controller scheduling
Refresh	Every ~32–64 ms	Blocks access; matters for real-time guarantees
Bank/rank parallelism	Independent banks	Interleave addresses to exploit it
Burst length	Minimum transfer granularity	A 4-byte read still fetches 32–64 bytes
Rowhammer	Repeated activation disturbs neighbours	A security problem solved by refresh management — a DRAM controller feature

E1. Digital Filters as Hardware



Direct form: adder tree depth grows with N

Transposed form: critical path is one mult + one add, independent of N

Figure 9. Direct-form FIR. The tap count sets the multiplier count; the *form* sets the critical path.

Table 63. FIR structures and their costs

Structure	Multipliers	Critical path	Choose when
Direct form	N	1 mult + $\log N$ adds	N small
Transposed direct form	N	1 mult + 1 add (constant)	Almost always, for speed. Cost: high fanout on $x[n]$
Symmetric (folded)	$\lceil N/2 \rceil$	1 add + 1 mult + 1 add	Linear phase — free 2× saving
Polyphase	N/M effective	—	Combined with rate change
Distributed arithmetic	0 (LUTs)	W cycles	Fixed coefficients, LUT fabric
CSD / multiplierless	0	adds only	Fixed coefficients, ASIC area
Frequency domain	—	—	$N \gtrsim 100$ and block latency acceptable

Graduate · **The fanout cost of the transposed form is real and is often overlooked in architecture documents.** The input sample must reach all N multipliers in one cycle. For $N = 128$ at 500 MHz this is a significant buffer tree whose delay and power must be budgeted. The usual remedy is to *partition* the filter into segments with a pipelined input distribution network — effectively reintroducing a small amount of direct-form structure. **Real designs are hybrids, and a specification that names only "transposed FIR" is incomplete.**

E1.2 Filter design methods

Table 64. FIR design methods

Method	Optimality	Control	Use
Windowing	Suboptimal	Simple	Quick designs
Frequency sampling	Suboptimal	Direct in frequency	—
Parks–McClellan (Remez)	Minimax optimal	Ripple in each band	Standard for equiripple specs
Least squares	L2 optimal	Weighted error	When energy matters more than peak
Convex optimisation	Flexible	Arbitrary constraints	Handles quantised coefficients directly

Graduate · Designing in floating point and rounding afterwards is a mistake that costs stopband attenuation. Rounding a Parks–McClellan design to B bits perturbs each coefficient by up to half an LSB, and the resulting stopband degradation can be 10–20 dB. The correct procedure is to include the quantisation in the design: either use an integer-programming or convex formulation that searches over representable coefficients, or re-optimize the remaining coefficients after fixing some. **For an IP vendor this is a concrete deliverable: ship the coefficient generator, not just the coefficients**, so the customer can re-tune for their own band plan without repeating the analysis.

E1.3 Multirate structures

Table 65. Rate conversion building blocks

Block	Function	Key property
Decimator ($\downarrow M$)	Filter then discard $M-1$ of every M	Filter first , or aliasing is irreversible
Interpolator ($\uparrow L$)	Insert $L-1$ zeros then filter	Zeros create images; the filter removes them
Polyphase decomposition	Split the filter into M sub-filters	Avoids computing samples that are discarded — $M\times$ saving
CIC	Integrator–comb cascade	No multipliers ; passband droop must be compensated
Half-band	Every other coefficient is zero	$\sim 2\times$ saving for 2:1 rates
Farrow	Polynomial interpolation with variable delay	Arbitrary, time-varying resampling ratio — used in timing recovery

Graduate · The CIC word-growth question is a classic fixed-point trap. An N -stage CIC with rate change R and differential delay M has DC gain $(RM)^N$, so the integrators need $W_{\text{out}} = W_{\text{in}} + N \log_2(RM)$ bits. The textbook trick is to use **two's-complement wrap-around** in the integrators: intermediate values overflow, but because the comb stages later subtract, the overflow cancels exactly, provided the *final* result fits. This works because wrap-around arithmetic is a ring homomorphism. **Replace wrap with saturation — an apparently safer choice — and the cancellation is destroyed and the filter breaks.** A specification that says only "use saturating arithmetic throughout" would silently kill a correct CIC. This is why Part A's number-system chapter insists that overflow behaviour is a per-block decision.

E1.4 Adaptive filters

Table 66. Adaptive algorithms in hardware terms

Algorithm	Update	Multipliers per tap per update	Convergence
LMS	$\mathbf{w} \leftarrow \mathbf{w} + \mu \mathbf{e} \mathbf{x}^*$	2	Slow; sensitive to eigenvalue spread
NLMS	μ normalised by $\ \mathbf{x}\ ^2$	2 + norm	Robust to input level
Sign-error LMS	uses $\text{sgn}(\mathbf{e})$	1	Slower

Algorithm	Update	Multipliers per tap per update	Convergence
Sign-data LMS	uses $\text{sgn}(x)$	1	Slower
Sign-sign LMS	both signs	0 — adds only	Slowest; used in every high-speed SerDes
RLS	inverse correlation update	$O(N)$ per tap	Fast; expensive
Frequency-domain LMS	block FFT	—	Fast for long filters

Graduate · Why sign-sign LMS dominates at 56 Gb/s. Two independent facts combine. First, the channel changes on a timescale of milliseconds while symbols arrive every 18 ps — **adaptation can be nine orders of magnitude slower than the datapath**. Second, multipliers cannot run at the symbol rate at all. Taking only signs reduces the update to increment/decrement, which can be accumulated over thousands of symbols and applied occasionally. The resulting architecture has a *fast datapath* and a *slow adaptation loop* in a different clock domain. **For a modelling engineer this separation must be explicit in the model**, because a model that updates taps every symbol will not match RTL that updates them every 4096 symbols, even though both "implement sign-sign LMS". The update schedule is part of the specification.

E2. Transforms in Hardware: FFT and Friends

Table 67. FFT algorithm families

Algorithm	Complex multiplies	Structure	Note
Radix-2 DIT/DIF	$(N/2)\log_2 N$	Regular butterflies	Simplest control
Radix-4	fewer	4-point butterflies	~25% fewer multiplies
Split-radix	fewest for radix-2 ^k	Irregular	Control complexity
Winograd / PFA	Minimum multiplies	Length must factor into coprimes	Niche
Pipelined SDF	—	One butterfly per stage + delay line	Streaming; the standard for OFDM
MDC	—	Multiple data paths	Higher throughput
SSR / parallel	—	Several samples per clock	When the data rate exceeds the clock rate

Graduate · The resource that actually limits an FFT is memory and the twiddle factors, not the butterflies. A pipelined N -point SDF FFT needs $\log_2 N$ butterflies but a total of $N-1$ words of delay line, and a twiddle ROM whose size grows with N . For $N = 4096$ that is thousands of words of storage against a dozen arithmetic units. Two standard optimisations follow: exploit twiddle symmetry to store only one eighth of a period and derive the rest by sign and swap, or generate twiddles with a CORDIC instead of a ROM. **Which one wins depends on whether the target has cheap memory (FPGA) or cheap logic (ASIC)** — another instance of the rule that architecture choices are technology-dependent and must be re-derived, not memorised.

Bit-reversed output order is an interface decision, not an implementation detail. A DIF FFT naturally produces its output in bit-reversed order. Reordering costs a full N -word buffer and its bandwidth. In many systems the consumer — a channel equaliser, a magnitude detector — does not care about order, so the reordering is pure waste. Competent designs pass the bit-reversed order through and let the next block absorb it. **This means the reference model and the RTL will disagree about output order unless the specification says which convention is normative**, and a large fraction of "FFT mismatch" bug reports are exactly this.

Table 68. Numerical behaviour of fixed-point FFT

Issue	Cause	Handling
Magnitude growth	Each radix-2 stage can double magnitude	Scale by $\frac{1}{2}$ per stage (lose SNR), grow the word, or block floating point
Twiddle quantisation	Finite coefficient precision	Adds a noise floor; usually 2–4 bits more than data width
Rounding vs truncation	Bias accumulates over $\log N$ stages	Convergent rounding avoids DC bias
Overflow detection	Input-dependent	Saturate and flag, or scale conservatively

Table 69. Other transforms used in IP

Transform	Property	Application
DCT-II	Energy compaction for correlated signals	JPEG, H.264/HEVC (integer approximations)
Integer DCT	Exactly invertible, multiplier-free	Video codecs — avoids encoder/decoder drift
Hadamard	± 1 only	Secondary transform, spreading

Transform	Property	Application
Number-theoretic (NTT)	Exact modular arithmetic	Post-quantum crypto (ML-KEM/ML-DSA) , homomorphic encryption
Wavelet (DWT)	Time–frequency localisation	JPEG2000, denoising

Graduate · **The integer DCT in video codecs is a beautiful example of specification-driven design.** An exact DCT requires irrational coefficients; any two implementations would round differently, and since the decoder's output feeds back into the encoder's prediction loop, the tiny differences would accumulate into visible *drift*. H.264 solved this by **defining the transform to be a specific integer matrix**, so every conforming implementation produces bit-identical results. The lesson generalises directly to IP work: **when an algorithm sits in a feedback loop that spans two independently implemented systems, the specification must be bit-exact, not mathematical.** The same reasoning applies to a DFE's tap update, to an adaptive equaliser shared between link partners, and to any distributed control loop.

F1. The Wireline Link, End to End

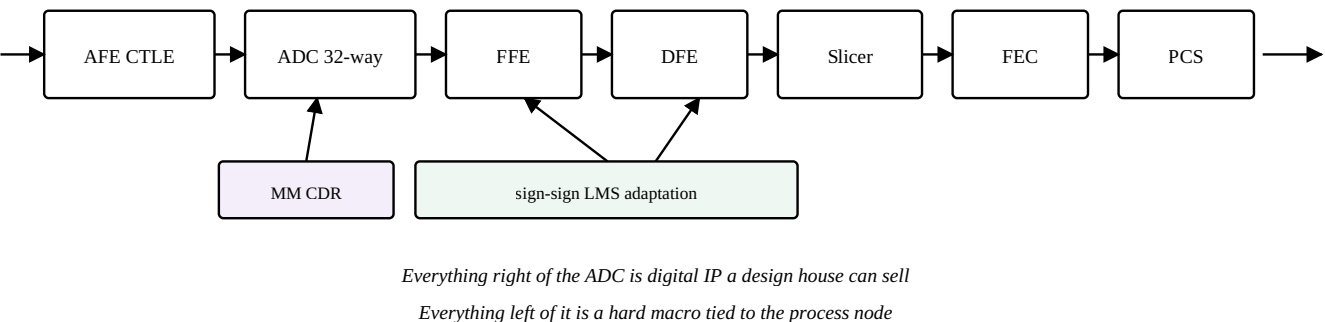


Figure 10. A modern ADC-DSP receiver. The PMA/PCS boundary is also the boundary of what can be sold as soft IP.

F1.1 Why the architecture looks like this

The structure above is not arbitrary; each block exists because a specific physical limit made the previous approach fail. Following that chain is the fastest way to understand the field.

Table 70. The forcing chain

Limit encountered	Response	New problem created
Channel loss rises as $\sqrt{f}$ (skin effect)	Equalise	Analogue boost amplifies noise as well as signal
CTLE alone cannot open the eye at 30 dB loss	Add DFE (subtracts decided symbols, no noise gain)	DFE feedback must close within 1 UI
1 UI = 17.86 ps at 56 GBd; latch + summer + wire + setup > 23 ps	Speculate (unroll) and parallelise (sub-rate)	Area grows; still not enough alone
Baud rate cannot be raised further — the channel will not pass it	PAM4 : two bits per symbol at the same baud rate	9.54 dB SNR penalty
PAM4 raw BER is $\sim 10^{-4}$, far from 10^{-15}	Forward error correction (RS-FEC KP4)	Latency, and FEC becomes mandatory in the standard
Analogue equalisation cannot be adapted precisely enough	Sample first with a fast ADC, equalise digitally	Time-interleaved ADC mismatch; huge digital block
200 Gb/s per lane: even KP4 is not enough	Concatenated FEC (outer RS-KP4 + inner Hamming)	More latency; more complex decoder

Graduate · This table is the single most useful thing to memorise about high-speed links, because it lets you derive the architecture rather than recall it, and it identifies where a newcomer can contribute. Notice that the last four rows are all *digital* responses to *analogue* limits. That migration — from analogue equalisation to digital — is why a digital IP house can participate in SerDes at all, and it is still in progress.

F1.2 Timing budget arithmetic

Table 71. DFE first-tap loop budget at 56 GBd (1 UI = 17.86 ps)

Element	Delay	Comment
Slicer (latch) clock-to-Q	12 ps	Regeneration plus output drive
Summer	8 ps	Analogue summing node settling
Wiring	6 ps	Layout-dependent; often underestimated
Setup at next slicer	5 ps	—
Total loop	31.0 ps	—
Available	17.86 ps	1 UI
Margin	-13.14 ps	Infeasible
Latch + wire + setup alone	23 ps	Still exceeds 1 UI

The last row is decisive and is often missed. Unrolling removes the *summer* from the loop by pre-computing both hypotheses, but the latch, the wiring and the setup time remain — and they already exceed one UI. **Therefore no amount of speculation alone reaches 56 GBd; sub-rate parallelism is mandatory.** Stating this as a measured budget rather than an opinion is what turns an architecture argument into an engineering conclusion, and it is exactly the kind of analysis a modelling engineer is expected to produce before any RTL exists.

F1.3 Equaliser taxonomy

Table 72. Equalisers compared

Type	Location	Removes	Noise effect	Constraint
TX FFE (de-emphas-is)	Transmitter	Pre- and post-cursor	None (pre-distortion)	Reduces transmitted amplitude
CTLE	RX ana- logue	Broad high-frequency loss	Amplifies noise	Limited number of poles/zeros
RX FFE	RX digital	Pre- and post-cursor	Amplifies noise	Needs ADC; tap count $\times$ rate
DFE	RX digital	Post-cursor only	None (uses de- cisions)	1 UI loop; error propagation
MLSE / Viterbi	RX digital	Optimal sequence detec- tion	None	Complexity grows as M^L
Nonlinear (Volterra, NN)	RX digital	Nonlinear channel ef- fects	—	Training, complexity; an active research area

Graduate · Error propagation is the price of the DFE's noise-free subtraction. A wrong decision is fed back and corrupts the next few symbols, so errors arrive in bursts rather than independently. Two consequences follow, and both are design decisions rather than analysis details. First, the raw BER is worse than an independent-error model predicts, so a link budget built on Q-function arithmetic alone is optimistic. Second — and more usefully — **bursty errors are exactly what a symbol-oriented code like Reed–Solomon handles well**, because several wrong bits inside one symbol still count as one symbol error. The choice of RS over a binary code at the FEC layer is therefore partly a consequence of the DFE at the equaliser layer. **Layers that look independent in a block diagram are coupled through their error statistics.**

F1.4 Clock and data recovery

Table 73. CDR phase detectors

Detector	Samples needed	Characteristic	Used with
Alexander (bang-bang)	Data + edge (2×)	Binary output; nonlinear	Classic NRZ links
Hogge	Data + edge	Linear output	Lower jitter, more complex
Gardner	2× oversampling	Modulation-independent	Wireless, some wireline
Mueller–Müller	Baud-rate only	Uses ISI structure; needs equalised signal	ADC-DSP receivers — no 2× sampling needed
Oversampling / majority	3–8×	Simple, robust, low rate	Low-speed links, FPGA

Graduate · Why Mueller–Müller won in ADC-based receivers. At 56 GBd, sampling at 2× the baud rate means 112 GS/s — roughly doubling the ADC's power and area, which are already the dominant cost. MM extracts a timing error from baud-rate samples alone by exploiting the asymmetry of the equalised pulse response: it drives the sampling phase to the point where the first pre-cursor and first post-cursor are equal. The subtlety is that **MM's lock point depends on the equaliser**, so CDR and equaliser adaptation interact; a modelling engineer who simulates them independently will not see the interaction, and joint convergence failures are a classic late-stage surprise. **Model the loops together or state explicitly that you have not.**

F2. Error Control Coding

F2.1 The coding landscape

Table 74. Codes by structure and application

Family	Alphabet	Decoding	Strength	Typical use
Parity / SECDED	binary	syndrome	1-bit correct, 2-bit detect	Memory ECC , register files
Hamming / extended	binary	syndrome	—	Inner code in concatenation
BCH	binary	BM + Chien	t bit errors	Flash, optical
Reed–Solomon	$GF(2^m)$	syndrome $\rightarrow$ BM $\rightarrow$ Chien $\rightarrow$ Forney	t symbol errors — burst tolerant	Ethernet KP4 , storage, DVB
Convolutional	binary	Viterbi (ML)	Random errors	Legacy wireless, satellite
Turbo	binary	iterative BCJR	Near capacity	3G/4G
LDPC	binary	belief propagation (min-sum)	Near capacity	5G NR data , Wi-Fi, 802.3ca, NAND
Polar	binary	SC / SC-List	Provably capacity-achieving	5G control
Fountain (LT, Raptor)	binary	BP	Rateless	Broadcast, storage
Concatenated	mixed	inner then outer	Burst + random	802.3dj 224G , deep space

Graduate · **Choosing a code is choosing an error model.** A binary code assumes errors are independent bit flips; a symbol code assumes they cluster. Getting this wrong is expensive in both directions — a strong binary code performs poorly on bursts, and an RS code wastes redundancy on isolated bit errors. The correct procedure is to **measure the error statistics of the actual slicer output**, including DFE error propagation and any burst noise, and choose accordingly. In concatenated schemes the division of labour is explicit: the inner code cleans up random errors and the outer code mops up the bursts that the inner decoder's own failures produce. **The inner decoder's failure mode is the outer decoder's input distribution** — a subtlety that must be in the system model.

F2.2 Reed–Solomon in detail

Table 75. RS decoder pipeline

Stage	Computation	Parallelism	Hardware character
Syndrome	$S_i = r(\alpha^{tcr+i}), i=0\dots 2t-1$	Fully parallel ($2t$ independent evaluations)	$2t$ constant-multiplier accumulators
Key equation (BM)	Find $\sigma(x)$	Sequential — $2t$ dependent iterations	This is the critical path
Chien search	Roots of $\sigma(x)$	Parallel over positions	n/P cycles
Forney	$e_j = \omega/\sigma'$ evaluated at roots	Parallel	Needs GF inversion (ROM or Itoh–Tsujii)
Correction	XOR into the buffer	Parallel	Requires the codeword buffered

Graduate · The Berlekamp–Massey iteration is the same kind of obstacle as the DFE loop and the OOO wakeup loop. Each iteration depends on the previous discrepancy, so it cannot be pipelined away. The standard remedies mirror those in the other two cases: *inversionless* BM removes the GF inversion from the loop (replacing it with an extra multiplication), and *re-formulated* BM restructures the recurrence so that fewer operations sit inside it. A design house that recognises this pattern can move an engineer from a SerDes project to an FEC project and expect them to be productive quickly, because the underlying problem is identical.

Table 76. RS decoder behaviour that must appear in the datasheet

Condition	Behaviour	Why it must be documented
$\leq t$ symbol errors	Always corrected	The guarantee being sold
$> t$ errors	Detected or mis-corrected	Not all are detected. Claiming otherwise is false
Mis-correction probability	Roughly $n^t / (t! 2^{m(n-k-t)})$	Customers at 10^{-15} FLR need this number
Erasure decoding	Corrects up to $n-k$ erasures	If the receiver can flag unreliable symbols, capacity doubles
Latency	Fixed, = pipeline depth	Real-time systems need the number

“All errors beyond t are detected” is a false statement that appears in datasheets. A received word with more than t errors may land within distance t of a *different* valid codeword, in which case the decoder confidently produces the wrong answer. A measured illustration on a small code, RS(15,9) with $t = 3$: injecting exactly four symbol errors 120 times gave **117 detected failures and 3 mis-corrections** — never a correct recovery, but not always a detection either. An honest specification states the mis-correction probability; a verification plan must distinguish the three outcomes (corrected / detected / mis-corrected) rather than counting pass and fail.

F2.3 LDPC

Table 77. LDPC decoding choices

Choice	Options	Effect
Message form	Probability vs log-likelihood ratio	LLR turns products into sums — essential in hardware
Check-node update	Sum-product (tanh) vs min-sum	Min-sum loses ~ 0.5 dB, removes transcendental functions
Correction	Normalised or offset min-sum	Recovers most of the loss for one constant
Schedule	Flooding vs layered	Layered halves the iteration count; creates pipeline hazards
Parallelism	Fully parallel / row / block	Quasi-cyclic structure enables block parallelism
Early termination	Syndrome check per iteration	Large average power saving ; worst-case latency unchanged

LDPC's fixed-point rule is the opposite of the CIC's. An LLR's sign carries the hard decision. If an LLR wraps in two's complement, a strong belief in “0” becomes a strong belief in “1”, and the decoder does not merely lose precision — it is actively misled, and iteration amplifies the damage. Measured on a 5G NR base-graph decoder, reducing the message width by three bits gave BLER 0.250 with saturation and **1.000 — complete failure — with wrap-around**. Meanwhile the CIC filter of Chapter E1 *requires* wrap-around to function. **There is no global “correct” overflow policy; it is a per-block property that must be stated in the specification and enforced by tests.**

Graduate · **Layered scheduling and pipeline hazards.** In a layered decoder, layer j may need a variable node that layer i is still updating, forcing a stall whose length depends on the pipeline depth D and on the layer ordering. It is tempting to optimise the ordering against a closed-form stall count, but that formula is only an **upper bound**: it assumes every potential conflict costs a full stall. A measured comparison on 5G NR base graph 1 at $D = 4$ gave the formula 108 stall cycles while a cycle-accurate model and the RTL both gave **80** — the difference between a 70.1% and a 62.0% efficiency loss. **Optimising**

a schedule against the wrong objective function finds the wrong schedule. Build the cycle-accurate model first; it is a day of work and it changes the answer.

G1. Electromagnetics for the Digital Engineer

A digital IP engineer meets electromagnetics at three boundaries: the package and board that carry the block's signals, the channel a SerDes must equalise, and the antenna of a radio or radar system whose baseband processing is the block itself. This chapter covers what is needed at those boundaries.

G1.1 From Maxwell to circuits

Table 78. Regimes and the model that applies

Regime	Condition	Model	Consequence
Lumped	size $\ll \lambda$	KCL/KVL, R L C	Voltage is well defined everywhere
Quasi-static	size $\leq \lambda/10$	Lumped + parasitics	Inductance begins to matter
Distributed	size $\geq \lambda/10$	Transmission line	Position matters; reflections occur
Full wave	structure comparable to λ	Maxwell solvers	Radiation, resonance, modes

Graduate · **The threshold keeps moving toward the designer.** The criterion is not clock frequency but edge rate: significant spectral content extends to roughly $0.35/t_r$. A 50 ps edge has content to 7 GHz regardless of whether the clock is 100 MHz, and at 7 GHz a 2 cm trace is a transmission line. **Slow clocks do not protect you from fast edges.** This is why deliberately slowing output slew rates is a legitimate design technique — and why an IP's output driver strength is a specification item with EMI consequences, not just a timing one.

G1.2 Transmission lines

$$Z_0 = \sqrt{(R + j\omega L)/(G + j\omega C)} \rightarrow \sqrt{L/C} \text{ (lossless)}$$

Table 79. Line types

Type	Z_0 range	Where
Microstrip	50–100 Ω	Outer PCB layer; radiates more
Stripline	50–100 Ω	Inner layer; better shielded, slower (higher ϵ_{eff})
Coplanar waveguide	tunable	On-chip RF; ground on the same layer
Differential pair	85–100 Ω differential	The standard for high-speed serial
On-chip global wire	—	Usually RC-dominated, not a true TL

Table 80. Termination strategies

Scheme	Placement	Power	Note
Series (source)	At driver	Low	Doubles the driver's output swing requirement
Parallel	At receiver	High (static)	Best match
Thevenin	At receiver	High	Sets a DC bias
AC-coupled	In line	Low	Requires DC-balanced coding (8b/10b, 64b/66b)
On-die termination (ODT)	Inside the chip	Moderate	Calibrated against an external precision resistor

Graduate · **AC coupling and line coding are linked requirements.** A series capacitor blocks DC, so a long run of identical symbols causes the signal to droop toward the decision threshold — baseline wander. The line code must therefore bound the run length and keep the running disparity small. 8b/10b does this with an explicit disparity rule at 25% overhead; 64b/66b uses a scrambler and accepts a small probability of a long run, at 3% overhead. **The coding choice is thus determined by an analogue component.** An engineer who knows only the coding layer cannot explain why 64b/66b needs a scrambler with a particular polynomial, nor why certain data patterns are used as stress tests.

G1.3 Antennas and propagation

$$P_r = P_t G_t G_r (\lambda/4\pi d)^2 \quad (\text{Friis})$$

Table 81. Antenna and propagation quantities

Quantity	Definition	Design relevance
Gain G	Directivity × efficiency	Trades coverage for range
Effective aperture	$A_e = G\lambda^2/4\pi$	Higher frequency → smaller aperture for the same gain
Beamwidth	$\approx 70^\circ/\sqrt{D}$	Array size sets angular resolution
Path loss exponent	2 (free space) to 4+ (urban)	Link budget
Multipath / delay spread	—	Sets the equaliser or OFDM CP length
Doppler shift	$f_d = 2v/\lambda$	Coherence time; tracking loop bandwidth
Fading	Rayleigh / Rician	Diversity, MIMO

Graduate · **Delay spread converts directly into a hardware parameter.** If the channel's impulse response lasts $\tau_{\max}$, a single-carrier receiver needs an equaliser spanning $\tau_{\max}/T_s$ symbols, and an OFDM receiver needs a cyclic prefix at least that long. The first costs taps (area, adaptation time); the second costs throughput (the CP is overhead). **This is the quantitative form of the choice between single-carrier and OFDM**, and it explains why wireline links — where delay spread is long but the channel is static — use equalisers, while wireless links with time-varying multipath use OFDM. The same physics, opposite conclusions, because the *rate of change* differs.

G1.4 Noise figure and receiver chains

$$F_{\text{total}} = F_1 + (F_2 - 1)/G_1 + (F_3 - 1)/(G_1 G_2) + \dots$$

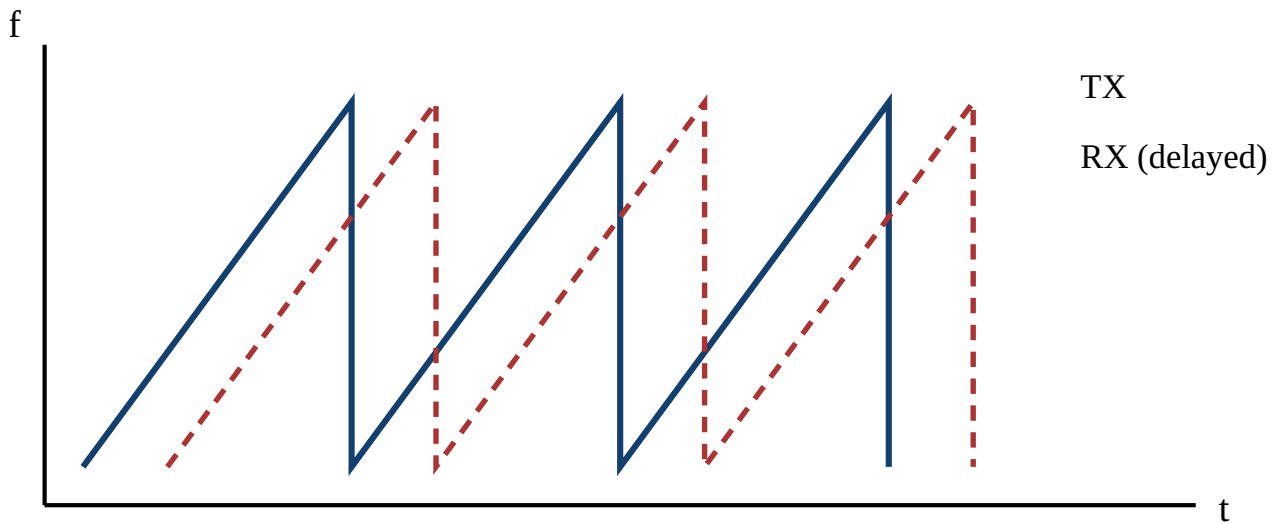
Graduate · **Friis's formula justifies the entire architecture of a receiver front end.** Because later stages are divided by the cumulative gain ahead of them, the first amplifier dominates the noise figure — hence the low-noise amplifier, placed as close to the antenna as possible, often before any filter whose insertion loss would add directly to F_1 . The counterweight is linearity: gain early in the chain means large signals later, so the cascaded third-order intercept point degrades as $1/\text{IIP3}_{\text{total}} = 1/\text{IIP3}_1 + G_1/\text{IIP3}_2 + \dots$ **Noise pushes gain forward; linearity pushes it backward.** Every receiver's gain distribution is a solution to that tension, and the digital back end inherits the result as its required dynamic range — which is to say, as its ADC bit count.

G2. Radar: Signal Processing as an IP Block Family

Automotive and industrial radar is one of the largest current markets for signal-processing IP, and its chain is a clean illustration of how physical parameters become hardware parameters.

$$P_r = P_t G^2 \lambda^2 \sigma / ((4\pi)^3 R^4) \quad (\text{radar equation})$$

Undergraduate review · The fourth-power range dependence is the defining fact of radar: doubling the detection range requires sixteen times the received power. Since transmit power and antenna gain are bounded, the remaining lever is **processing gain** — coherent integration — which is entirely a digital matter and therefore entirely IP.



$$\text{Beat frequency } f_b = 2 \cdot R \cdot S / c \quad (S = \text{chirp slope})$$

Range from frequency; velocity from phase change across chirps

Figure 11. FMCW radar. Range appears as a beat frequency; velocity appears as phase progression across successive chirps.

G2.1 FMCW parameter derivation

Table 82. From requirement to hardware parameter

Requirement	Relation	Hardware consequence
Range resolution ΔR	$\Delta R = c/2B$	Sweep bandwidth — 4 GHz gives 3.75 cm
Maximum range $R_{\max}$	$f_{b,\max} = 2R_{\max}S/c$	ADC sample rate and IF bandwidth
Velocity resolution Δv	$\Delta v = \lambda/(2N_c T_c)$	Number of chirps per frame → frame memory
Maximum velocity	$v_{\max} = \lambda/(4T_c)$	Chirp repetition period
Angular resolution	$\approx \lambda/(N_a d \cos\theta)$	Number of antennas → number of RX chains
Processing gain	$10\log(N_s N_c)$	FFT sizes → compute and memory

Graduate · **The range–Doppler map is a two-dimensional FFT** and its dimensions come straight from the table: a fast-time FFT across samples within one chirp yields range bins; a slow-time FFT across chirps at each range bin yields Doppler bins. A typical automotive frame — 256 samples $\times$ 128 chirps $\times$ 8 virtual antennas — requires a corner-turn between the two FFTs, and **that transpose, not the FFTs, dominates the memory bandwidth**. Designing the memory organisation (ping-pong banks, tiled transpose, or on-the-fly second-stage accumulation) is the real architectural work. This is a recurring theme: in data-intensive blocks the arithmetic is rarely the bottleneck.

G2.2 Detection

Table 83. CFAR detectors

Type	Noise estimate	Strength	Weakness
CA-CFAR	Mean of surrounding cells	Simple, efficient in homogeneous noise	Masks targets near clutter edges
GO-CFAR	Greater of the two halves	Handles clutter edges	Loses closely spaced targets
SO-CFAR	Smaller of the two halves	Resolves close targets	High false alarm at edges
OS-CFAR	k -th order statistic	Robust to interfering targets	Needs a sorter — more hardware
Adaptive / ML	learned	Best in complex scenes	Verification difficulty

Graduate · **CFAR sets a threshold, not a decision**. The multiplier α is chosen so that the probability of false alarm is fixed regardless of the noise level — that is what "constant false alarm rate" means. For CA-CFAR with N reference cells, $\alpha = N(P_{FA}^{-1/N} - 1)$. Two hardware consequences: the block needs a divider or a reciprocal table, and it needs the N reference cells buffered, which for a 2-D CFAR means a sliding window over a two-dimensional array. **OS-CFAR's sorter is the most hardware-hungry element in a typical radar back end**, and choosing between a full sorting network and an approximate selection is a genuine PPA decision that a design house can differentiate on.

G2.3 Angle estimation and MIMO

Table 84. Angle-of-arrival methods

Method	Resolution	Computation	Robustness
FFT beamforming	Rayleigh limit	One FFT	Very robust
Capon / MVDR	Super-resolution	Matrix inverse per snapshot	Needs good covariance estimate
MUSIC	Super-resolution	Eigendecomposition	Needs the source count
ESPRIT	Super-resolution	SVD + rotation	Needs array structure
Compressive sensing	High	Iterative solver	Sensitive to model mismatch

Graduate · **This is where Part A's matrix decompositions become products**. MUSIC needs the eigendecomposition of an $N \times N$ covariance matrix per snapshot; Capon needs its inverse. For an 8–16 element array that is a small, dense, Hermitian, **positive-definite** problem — exactly the case where Cholesky applies and streams. A design house that owns a parameterised streaming Cholesky or Jacobi eigenvalue block can sell it into radar, into MIMO communications, and into adaptive beamforming, because all three reduce to the same kernel. **Recognising that one kernel serves three markets is how a small IP portfolio becomes viable**.

Graduate · **MIMO radar's virtual array** deserves a precise statement because it is the main reason automotive radar improved so quickly. With N_t transmitters and N_r receivers, transmitting orthogonal waveforms (time-, frequency-, or code-division) and separating them at the receiver yields $N_t \times N_r$ distinct transmit–receive paths, which behave like a physical array of

that many elements. Three transmitters and four receivers give a twelve-element virtual array — a threefold improvement in angular resolution at the cost of **digital** separation logic rather than antennas and RF chains. **Once again the solution to a physical limit is a digital block.**

H1. Verification: Theory and Organisation

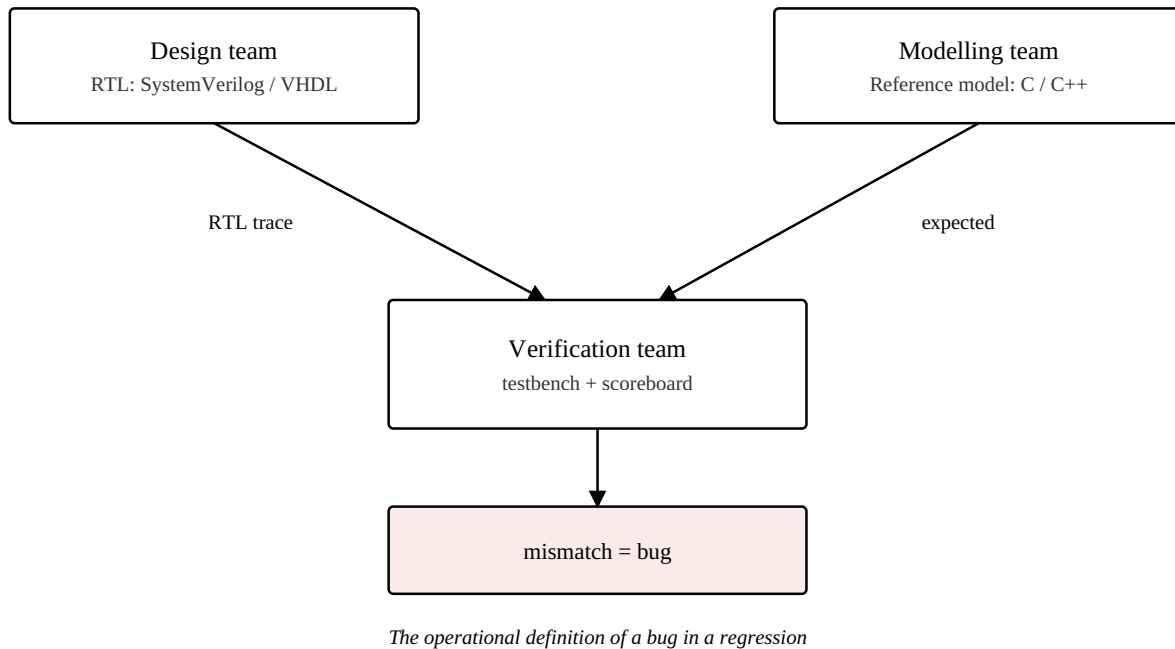


Figure 12. The three-team structure. It exists to make “correct” an operational, testable notion.

Verification consumes more effort than design in every serious IP project — commonly cited at 60–70% of the schedule. Understanding why it is organised as it is prevents a great deal of wasted work.

H1.1 Why two independent implementations

Undergraduate review · A specification written in prose is ambiguous; a single implementation is self-consistent by construction and therefore cannot reveal its own misreadings. Two implementations built independently from the same specification will agree only where both read it the same way. **Disagreement localises ambiguity.** This is the same principle as dissimilar redundancy in avionics, applied to development rather than to operation.

Graduate · **The argument fails if the two implementations are not independent.** If the modeller and the designer sit together, share an interpretation, and one consults the other's code when unsure, the common-mode error rate rises and the regression turns green on a shared misunderstanding. Organisations counter this in three ways: physical or organisational separation; deriving one implementation from a **formal, executable specification** (the RISC-V Sail model is the canonical example, generating both prose and a C emulator from one source); and an **external oracle** — standard test vectors from NIST, IEEE or a conformance suite. **The third is the strongest and the cheapest, and it is the reason standards-based blocks are the best first product for a small IP house.**

H1.2 The verification stack

Table 85. Verification levels

Level	Checks	Catches	Cannot catch
Lint	Coding rules	Latches, width mismatches, incomplete sensitivity	Anything functional
CDC / RDC static	Crossing structure	Missing synchronisers, reconvergence	Functional protocol errors
Unit simulation	Block behaviour vs model	Most functional bugs	Integration, timing
Formal property	Exhaustive over state space	Deadlock, unreachable states, protocol violations	Deep datapath equivalence (state explosion)
Formal equivalence (LEC)	RTL vs netlist	Synthesis and ECO errors	Specification errors
Integration / SoC sim	Blocks together	Interface mismatches, arbitration	Long-running system behaviour
Gate-level with timing	Netlist + SDF	X-propagation, timing-dependent bugs	Slow; limited coverage
Emulation / FPGA prototype	Real software on real RTL	Software-visible bugs, long sequences	Analogue, timing corners
Post-silicon	Actual device	Everything else	— but fixes are expensive

X-propagation deserves a specific warning because it is asymmetric between RTL and gates. In RTL simulation, an if (x) with an unknown condition may take a branch and produce a definite value, hiding the uncertainty — this is *X-optimism*. In gate-level simulation the same unknown spreads — *X-pessimism*. A design that relies on registers being initialised by data flow rather than by reset can therefore pass RTL simulation and fail in gates, or vice versa. The remedies are X-propagation-aware simulation modes and, better, **explicit initialisation of any register whose value is read before it is written**. For an IP vendor this is also a deliverable question: if your block requires *n* cycles of reset before its outputs are valid, say so.

H1.3 Coverage and closure

Table 86. Coverage metrics

Metric	Measures	Automat-ic?	Weakness
Line / block	Statements executed	Yes	Says nothing about values
Branch / condition	Decisions taken	Yes	—
Expression (MC/DC)	Each condition independently affects the outcome	Yes	Required for DO-254 / ISO 26262
Toggle	Every bit changed both ways	Yes	Weak functional signal
FSM state / transition	States and arcs visited	Yes	—
Functional (coverage groups)	Scenarios the engineer defined	No	Blind to scenarios nobody thought of
Assertion	Properties actually exercised	Partly	Vacuous passes must be excluded
Mutation / fault injection	Whether the testbench would notice a bug	Yes	The only metric that tests the testbench

Graduate · Mutation coverage is the metric that answers the question the others avoid: **would this testbench detect a bug if there were one?** The procedure is to inject a small, semantically meaningful change — invert a comparison, drop a term, stick a bit — and confirm the regression turns red. A mutant that survives identifies a check that does not exist or a stimulus that never reaches the code. This is cheap to automate and routinely reveals that a large fraction of a mature suite is

inert. **The same technique applies to the reference model itself**, which is otherwise unverified: break the model deliberately and confirm the model's own self-checks fail. A verified reference model is a differentiator that very few IP vendors can claim.

H1.4 The scoreboard problem

Table 87. Comparison strategies

Situation	Strategy	Pitfall
In-order, fixed latency	FIFO compare	—
In-order, variable latency	FIFO + timeout	Timeout too tight → false failures
Out-of-order, tagged	Per-tag queues	A single queue produces both false failures and false passes
Multiple legal orders	Compare against a set of allowed outcomes	Requires the model to enumerate them
Don't-care fields	Mask before comparing	Masking too much hides real bugs
Weak memory model	Litmus tests , not trace comparison	Trace comparison is simply invalid here

Graduate · The AXI case is worth stating concretely because it is so common. AXI guarantees ordering only *within* an ID; transactions with different IDs may complete in any order. A scoreboard with one queue will report a failure when the DUT legitimately reorders across IDs, and — more dangerously — may accept a genuine intra-ID reordering as long as the totals match. **The correct structure is one queue per ID**, and knowing this is a direct consequence of having read the protocol specification rather than inferring behaviour from waveforms. It is a frequent and productive point of contact between the modelling and verification teams.

H1.5 Regression engineering

Table 88. Regression discipline

Practice	Reason
Record the seed with every run	Without it a failure is not reproducible
Classify failures automatically	Triage time dominates otherwise
Track failure <i>signatures</i> , not counts	Distinguishes new from known
Never close an intermittent failure as 'flaky'	Intermittent failures are usually real races
Keep a golden log of the last known-good	Enables bisection
Measure regression runtime	A suite nobody can run nightly is not a suite
Separate infrastructure failures	Licence and disk errors must not be counted as bugs

The prohibition on dismissing intermittent failures is the most frequently violated rule in this book. An intermittent failure means the outcome depends on something the test does not control — a race, an uninitialised value, a timing-dependent path. All three exist in the silicon too. **Closing such a ticket does not remove the bug; it removes the evidence.** The correct disposition is to keep it open with the failing seed recorded, even if it cannot be root-caused immediately.

H2. The Implementation Flow

Table 89. From RTL to GDSII

Step	Input → output	Key decisions	Common failure
Elaboration	RTL → generic netlist	Parameter resolution	Unintended latch inference
Logic synthesis	generic → standard cells	Constraints, effort, V_{TH} mix	Over-constrained clock → area and power explosion
DFT insertion	+ scan chains	Chain count and length	Scan shift timing
Floorplan	—	Block placement, pin assignment, power grid	The decision with the largest downstream impact
Placement	cells → locations	Congestion vs timing	Routing congestion
CTS	+ clock tree	Skew target, useful skew	Clock power, OCV
Routing	—	Layer assignment, shielding	DRC violations, crosstalk
Sign-off STA	—	Corners, derating, SI	Missing a corner
Physical verification	DRC, LVS, antenna	—	—
Extraction + final timing	RC → delays	—	—

Graduate · Floorplanning is where an IP block's shape becomes a commercial issue. A block delivered as RTL will be placed by the customer, whose aspect ratio, pin locations and macro placement may be nothing like the vendor's. If the block's internal timing depends on a particular arrangement — a wide datapath that must stay compact, a memory that must sit next to its controller — then the RTL alone is not a sufficient deliverable. Serious IP therefore ships a **floorplan guideline**, sometimes a hardened macro, and always a statement of the assumed aspect ratio and utilisation. **A block that met timing in the vendor's flow and misses in the customer's is the single most damaging support event a small IP house can have**, because it is expensive to debug remotely and it destroys confidence.

Table 90. Sign-off corners

Axis	Values	Why
Process	SS, TT, FF, SF, FS	Global and local variation
Voltage	min, nominal, max	IR drop and regulation tolerance
Temperature	−40, 25, 125 °C	Both extremes — temperature inversion makes cold the worst case at low V
RC extraction	Cmin, Cmax, RCmin, RCmax	Metal thickness variation
Mode	functional, scan, BIST, low power	Different constraints apply
Ageing	fresh, end-of-life	NBTI/PBTI drift

Graduate · The corner count multiplies. Five process points × three voltages × three temperatures × four RC × four modes is 720 combinations; full sign-off on all of them is not affordable. The industry response is **statistical or parametric OCV** (POCV), which models variation as a distribution rather than enumerating corners, plus careful selection of dominant corners. For an IP vendor the practical obligation is to state **which corners were signed off** in the PPA report. "500 MHz" without a corner is not a specification, and a customer who assumes typical-typical will be disappointed at slow-slow.

Table 91. High-level synthesis in the flow

Aspect	Reality
Input	C/C++/SystemC with vendor pragmas — not portable across tools
Strength	Design-space exploration: one source, many area/throughput points
Weakness	Generated RTL is hard to read and debug; timing closure less predictable
Where it wins	Datapath-dominated blocks with regular loops (filters, transforms, crypto)
Where it loses	Control-dominated logic, precise cycle-level interfaces
Verification role	The same C source doubles as the reference model , because a general compiler ignores the pragmas — convenient, but it shares every algorithmic assumption with the design

The last row is a genuine methodological trap. Using the HLS source as the golden model gives *zero* independence: the two implementations are literally the same text. It verifies the HLS tool, not the design. **If the HLS C is the design, the reference model must be written separately** — ideally in a different language, by a different person, from the specification.

H3. Hardware Security

Table 92. Threat model, mechanism, countermeasure

Threat	Mechanism	Countermeasure	Verifiable by simulation?
Timing side channel	Execution time depends on secret	Constant-time algorithms; no secret-dependent branches or table indices	Partly — can check control flow
Power / EM (SPA, DPA, CPA)	Switching activity correlates with data	Masking (Boolean or arithmetic), threshold implementations, DOM	No — requires TVLA on silicon or power simulation
Fault injection (laser, glitch, EM)	Flip a bit to skip a round or corrupt a check	Redundant/complementary FSMs, computation duplication, output checks	Partly — by explicit fault simulation
Probing / FIB	Physically read an internal wire	Shields, sensors, routing obfuscation	No
Scan / debug access	Read state through test infrastructure	Lock scan after test; life-cycle state control	Yes
Rowhammer memory disturb	Repeated activation flips neighbours	Refresh management, tracking	Partly
Supply chain	Trojan insertion, counterfeiting	Split manufacturing, logic locking, PUF identity	No

Graduate · The fourth column is the key organisational insight for a modelling engineer. A functional regression, no matter how complete, is blind to four of the seven rows. Saying “the AES block is verified” is therefore ambiguous, and in a security review the ambiguity will be challenged. The mature formulation is: *functional equivalence to FIPS-197 is verified by NIST vectors and constrained-random comparison against an independent model; side-channel resistance is assessed separately by TVLA on the test chip; fault resistance is assessed by gate-level fault injection. Three claims, three methods, three owners.*

Table 93. Masking: the principle and its cost

Concept	Statement	Consequence
Boolean masking	x represented as $(x \oplus m, m)$	Each share is independent of $x \rightarrow$ first-order leakage removed
Linear operations	Apply to each share separately	Nearly free
Nonlinear operations	Shares must interact	The expensive part — this is why the AES S-box is rebuilt in arithmetic
Glitches	Transient values can momentarily depend on x	Registers must separate share-combining stages
DOM	Domain separation + fresh randomness + registers	Provably glitch-resistant; needs a continuous randomness supply
Order d masking	$d+1$ shares resist d -th order analysis	Cost grows roughly as $(d+1)^2$

Graduate · Randomness becomes a system-level resource. A DOM-protected S-box consumes fresh random bits on every evaluation. An AES core performing sixteen S-box lookups per round for fourteen rounds therefore needs a sustained random-bit rate that a slow true-RNG cannot supply, so the design includes a fast PRNG reseeded from the TRNG. That PRNG's quality, its reseed policy, and its behaviour when starved are now **functional** requirements of the cipher block, and they are

verifiable. **A security countermeasure has turned into an ordinary functional specification with a throughput requirement** — which is precisely the kind of boundary a modelling engineer should be able to identify and take ownership of.

Table 94. Standards a security IP is measured against

Standard	Scope
FIPS 140-3	Cryptographic module validation
Common Criteria (ISO 15408)	Evaluation assurance levels
ISO/IEC 17825	Side-channel test methods
NIST CAVP	Algorithm test vectors — the practical oracle
SP 800-90A/B/C	RNG construction, entropy, and validation
FIPS 203/204/205	Post-quantum: ML-KEM, ML-DSA, SLH-DSA
OCP Caliptra	Datacentre root of trust specification
ISO 26262 / IEC 61508	Functional safety (distinct from security)

I1. Information Theory

I1.1 Entropy and its operational meanings

$$H(X) = -\sum_x p(x) \log_2 p(x) \quad I(X;Y) = H(X) - H(X|Y)$$

Table 95. Information-theoretic quantities and where they bind hardware

Quantity	Meaning	Operational limit it sets
Entropy $H(X)$	Average information per symbol	Lossless compression cannot go below it — bounds a co-dec's ratio
Min-entropy H_∞	$-\log_2 \max_x p(x)$	The measure NIST SP 800-90B requires for an entropy source; always $\leq H$
Conditional entropy	Remaining uncertainty given side information	Leakage analysis in side-channel evaluation
Mutual information	Information one variable carries about another	Side-channel leakage metric ; also the quantity maximised by capacity
Channel capacity C	$\max I(X;Y)$	No code can exceed it; tells you how far your FEC is from optimal
Rate–distortion $R(D)$	Minimum rate for distortion D	Lossy codec bound — the theoretical floor of a video encoder
Kolmogorov complexity	Shortest program producing the string	Uncomputable; conceptual only

Graduate · **Min-entropy is the one an IP engineer must get right, and it is not Shannon entropy.** A biased source that outputs 0 with probability 0.9 has Shannon entropy 0.47 bits but min-entropy only $-\log_2(0.9) = 0.152$ bits. Certification uses min-entropy because security must hold against the *most likely* guess, not the average case. A TRNG that claims "0.5 bits of entropy per sample" on Shannon grounds and is conditioned on that basis will fail validation and, worse, will produce keys with less unpredictability than advertised. **The conditioning ratio — how many raw bits are compressed into one output bit — is derived from min-entropy, and it is a hardware parameter.**

I1.2 Capacity results that matter in practice

Table 96. Capacity expressions

Channel	Capacity	Design meaning
AWGN	$B \log_2(1+\text{SNR})$	The reference against which every modem is measured
Binary symmetric (BSC)	$1 - H_b(p)$	Hard-decision decoding loses ~2 dB versus soft
Binary erasure (BEC)	$1 - \epsilon$	Erasure decoding doubles RS correction capability
MIMO ($N_t \times N_r$)	$\approx \min(N_t, N_r) \log(1+\text{SNR})$	Capacity scales with antennas, not power
Fading with CSI	$E[\log(1+ h ^2 \text{SNR})]$	Motivates adaptive modulation and coding
Band-limited with ISI	water-filling over the channel response	The theoretical justification for OFDM bit loading

Graduate · **The soft-versus-hard decision gap is the most actionable number here.** Passing hard decisions to a decoder discards the reliability information the slicer had, and costs roughly 2 dB on an AWGN channel. In a link budget 2 dB is enormous — it is the difference between needing an extra equaliser tap and not. This is why modern FEC interfaces carry

LLRs rather than bits, why the slicer in an ADC-based receiver reports a multi-bit soft value, and why the fixed-point width of that LLR is a system-level parameter rather than a local choice. **An interface that carries one bit where it could carry four has thrown away 2 dB before the decoder starts.**

I1.3 Source coding

Table 97. Compression methods in hardware

Method	Principle	Hardware character
Huffman	Prefix code by symbol probability	Table lookup; variable-length decode is serial
Canonical Huffman	Ordered code lengths	Compact tables, faster decode
Arithmetic / range	Interval subdivision	Better ratio; strictly serial — a throughput bottleneck
CABAC	Context-adaptive binary arithmetic	The critical path of an H.264/HEVC decoder
ANS / rANS	Asymmetric numeral systems	Arithmetic-coding ratio with table-driven speed; used in AV1, Zstd
LZ family	Dictionary matching	Match search is the expensive part
RLE	Run lengths	Trivial; still used for coefficient maps

Entropy coding is where video decoders stop parallelising. Transform, prediction and filtering all parallelise over blocks, but arithmetic decoding is inherently sequential: the next symbol's interval depends on the current state. This is why real decoders introduce architectural workarounds — slices and tiles that reset the context so they can be decoded independently, wavefront parallel processing, or the bypass modes that skip context modelling for near-uniform symbols. **The bitstream syntax was designed around a hardware constraint**, which is the clearest possible illustration that standards and architectures co-evolve.

I2. Algebraic Structures for Hardware

I2.1 Groups, rings, fields

Table 98. Structures and the hardware they license

Structure	Operations	Guarantee	Hardware consequence
Group	one associative op with inverses	Cancellation	Reversible transforms
Ring	+ and $\times$, no division	Closure under both	Two's-complement W-bit arithmetic is a ring — wrap-around is exact, so operation order does not matter
Field	+ $\times$ and division	Every non-zero element invertible	$GF(2^m)$ arithmetic for codes and crypto
Vector space over a field	—	Linearity	Linear codes: encoding and syndrome are matrix products
Polynomial ring $F[x]/(f)$	—	—	Cyclic codes, NTT, lattice cryptography

Graduate · The ring property of two's-complement arithmetic is the formal reason behind a practical rule. Because W -bit two's complement is the ring $\mathbb{Z}/2^W\mathbb{Z}$, addition and multiplication are exact modulo 2^W , associative and commutative. Intermediate overflow therefore cancels exactly, provided the final result is in range, and **the order of accumulation cannot change the answer**. Saturation destroys the ring structure: once a value is clamped, information is lost and order matters. This is why a CIC filter *requires* wrap-around and why an LDPC decoder *requires* saturation — the two blocks rely on opposite algebraic properties. **Stating this as algebra rather than as folklore makes it possible to decide the question for a new block instead of guessing.**

I2.2 Finite fields in detail

Table 99. $GF(2^m)$ implementation choices

Representation	Multiplication	Inversion	Best for
Polynomial basis	Shift-and-XOR, reduce	Extended Euclid or Itoh–Tsujii	General purpose; most codes
Normal basis	More complex	Squaring is a free cyclic shift	Inversion-heavy workloads (ECC)
Log/antilog tables	Add logs	Negate the log	Small fields ($m \leq 12$); the usual RS choice
Tower field	$GF(2^8) \rightarrow GF(2^4) \rightarrow GF(2^2)$	Recursive	Maskable — the AES DOM S-box
Composite field	—	—	Area reduction

Graduate · The choice of irreducible polynomial is a specification item, not an implementation detail. Two implementations of $GF(2^{10})$ using different primitive polynomials produce entirely different — though isomorphic — arithmetic, and codewords from one will not decode in the other. Standards therefore fix the polynomial: AES uses $x^8+x^4+x^3+x+1$, and IEEE 802.3 Clause 91 fixes the field for RS(544,514). **A reference model that hard-codes a plausible polynomial without checking the standard will interoperate with nothing.** The defensive practice is to make the polynomial a parameter, assert that it is primitive (verify that α generates all 2^m-1 non-zero elements), and record in the code whether the value has been confirmed against the specification text.

Table 100. Where finite fields appear

Application	Field	Operation that dominates
Reed–Solomon	$GF(2^8), GF(2^{10})$	Multiply, invert
BCH	$GF(2^m)$	Multiply
AES	$GF(2^8)$	Inverse (S-box), $\times 2$ and $\times 3$ (MixColumns)
AES-GCM	$GF(2^{128})$	Carry-less multiply
Elliptic curves	$GF(p)$ or $GF(2^m)$	Modular multiply (Montgomery)
ML-KEM / ML-DSA	$\mathbb{Z}_q[x]/(x^n+1)$	Number-theoretic transform
CRC	$GF(2)[x]$	Polynomial remainder (LFSR)
Erasure coding (RAID)	$GF(2^8)$	Matrix multiply

Graduate · Post-quantum cryptography brings a new arithmetic requirement into mainstream silicon. ML-KEM and ML-DSA operate in a polynomial ring modulo a small prime (3329 and 8380417 respectively), and their bottleneck is the number-theoretic transform — an FFT over that ring. The hardware looks like an FFT but with modular multipliers instead of complex ones, and modular reduction by a fixed small prime admits efficient tricks (Montgomery, Barrett, or prime-specific shift–add chains). **For a design house, an NTT block parameterised over modulus and length is a single kernel serving several standards**, in the same way that a Cholesky block serves radar, MIMO and beamforming. Finding these shared kernels is how a small portfolio covers a large market.

I3. Optimisation and Numerical Methods

Table 101. Optimisation problems that appear in IP design

Problem	Class	Method	Where
Filter coefficient design	Convex (minimax)	Remez, convex solvers	FIR design with quantised coefficients
Equaliser tap solution	Least squares	Normal equations, QR	MMSE equaliser initialisation
Bit allocation	Integer, convex relaxation	Water-filling then rounding	OFDM loading, quantisation budgeting
Scheduling / binding	NP-hard	ILP, list scheduling, force-directed	HLS internals
Retiming	Polynomial	Leiserson–Saxe	Timing optimisation
Placement	NP-hard	Analytic + partitioning	Physical design
Layer/schedule ordering	Combinatorial	Local search, simulated annealing	LDPC layered decoding order
Word-length assignment	Mixed integer	Heuristic + simulation	Fixed-point conversion

The objective function must be the true cost, not a convenient proxy. A measured example: optimising an LDPC layer ordering against a closed-form stall estimate produced a schedule that a cycle-accurate model scored differently — the formula predicted 108 stall cycles where the true figure was 80. Because the formula is an upper bound that assumes every potential conflict materialises, it ranks schedules incorrectly, and the search converges on the wrong one. **Build the accurate evaluator before the optimiser;** a search is only as good as the function it is minimising, and a fast wrong objective is worse than a slow right one.

Table 102. Numerical methods and their hardware suitability

Method	Convergence	Hardware note
Newton–Raphson	Quadratic	Needs a multiplier and a good seed; used for reciprocal and rsqrt
Goldschmidt	Quadratic	Better pipelining than Newton (no dependency chain)
CORDIC	Linear (one bit per iteration)	Shifts and adds only
Polynomial (minimax)	Fixed degree	Remez-fitted polynomials beat Taylor for equal degree
Piecewise polynomial + LUT	—	The usual production answer
Iterative refinement	Doubles accuracy per step	Limited by conditioning — see A5.2

Graduate · Taylor series should almost never appear in shipped hardware. A Taylor expansion minimises error at a single point and degrades away from it; a minimax (Remez) polynomial of the same degree distributes error evenly and typically achieves the same worst-case accuracy with one or two fewer terms. Since each term is a multiply–add, that is a direct area and latency saving. The standard production recipe is: reduce the argument to a small interval using the number's exponent, evaluate a low-degree minimax polynomial on that interval, then reconstruct. **Knowing that the textbook series is the wrong tool is a small piece of knowledge with a consistently measurable payoff.**

I4. Discrete Mathematics and Graphs in Hardware

Table 103. Graph problems in the design flow

Problem	Where it appears	Complexity	Practical method
Topological sort	Netlist evaluation, scheduling	Linear	Exact
Longest path	Static timing analysis	Linear on a DAG	Exact — this is what STA computes
Cycle detection	Combinational loop check	Linear	Exact
Min-cut / max-flow	Partitioning, retiming	Polynomial	Exact
Graph colouring	Register allocation, channel assignment	NP-hard	Heuristic
Clique / independent set	Binding in HLS	NP-hard	Heuristic
Shortest path	Routing	Polynomial	A*, maze routing
Steiner tree	Multi-pin net routing	NP-hard	Heuristic
Isomorphism	LVS (layout vs schematic)	Unknown class	Practical algorithms

Graduate · Recognising that a design task is a known graph problem is worth more than knowing an algorithm for it. Static timing analysis is longest path on a DAG — which is why combinational loops must be forbidden: they make the problem ill-posed, not merely difficult. Retiming is a min-cost flow problem, which is why it is exactly solvable while placement (a quadratic assignment problem) is not. **The tractability boundary explains the tool landscape:** tasks on the polynomial side have exact tools that engineers trust, and tasks on the NP-hard side have heuristics whose results engineers must inspect. A designer who knows which side a problem lies on knows whether to argue with the tool.

Table 104. Boolean algebra and representations

Representation	Property	Use
Truth table	Exponential size	Small functions, LUT mapping
SOP / POS	Two-level	PLA, simple synthesis
BDD	Canonical for a fixed variable order	Equivalence checking ; order-sensitive size
AIG (And-Inverter Graph)	Compact, not canonical	Modern synthesis and formal engines
CNF + SAT	—	Bounded model checking, equivalence
SMT	Theories beyond Boolean	Word-level formal verification

Graduate · **The shift from BDDs to SAT reshaped formal verification.** BDDs are canonical, which makes equivalence a pointer comparison, but their size explodes for arithmetic functions such as multipliers and is acutely sensitive to variable ordering. Modern SAT solvers, with conflict-driven clause learning, handle far larger instances without canonicity. The practical consequence for an IP engineer is that **formal property checking is now feasible on real control logic** — arbiters, FIFOs, protocol adapters, FSM reachability — where a decade ago it was a specialist activity. It remains infeasible on deep datapaths, so the division of labour is: formal for control, simulation with a golden model for datapath. **Knowing which side of that line a block falls on is how a verification plan is written.**

J1. On-Chip Memory

Table 105. Memory types available to an IP block

Type	Bit cell	Density	Notes
6T SRAM	6 transistors	High	Standard; compiled by the foundry
8T / 10T SRAM	8–10 T	Lower	Better read stability, low-voltage operation
Register file	Latch or flop array	Low	Multi-port, fast, expensive
Flip-flop array	—	Lowest	Synthesisable — the only portable option for small storage
Dual-port SRAM	8T	Lower	True dual port costs ~30–40% more area
Pseudo dual port	6T, time-multiplexed	High	Half the bandwidth
ROM	—	Very high	Constants, microcode, S-boxes
eDRAM / MRAM / RRAM	—	Very high	Process-specific; limited availability

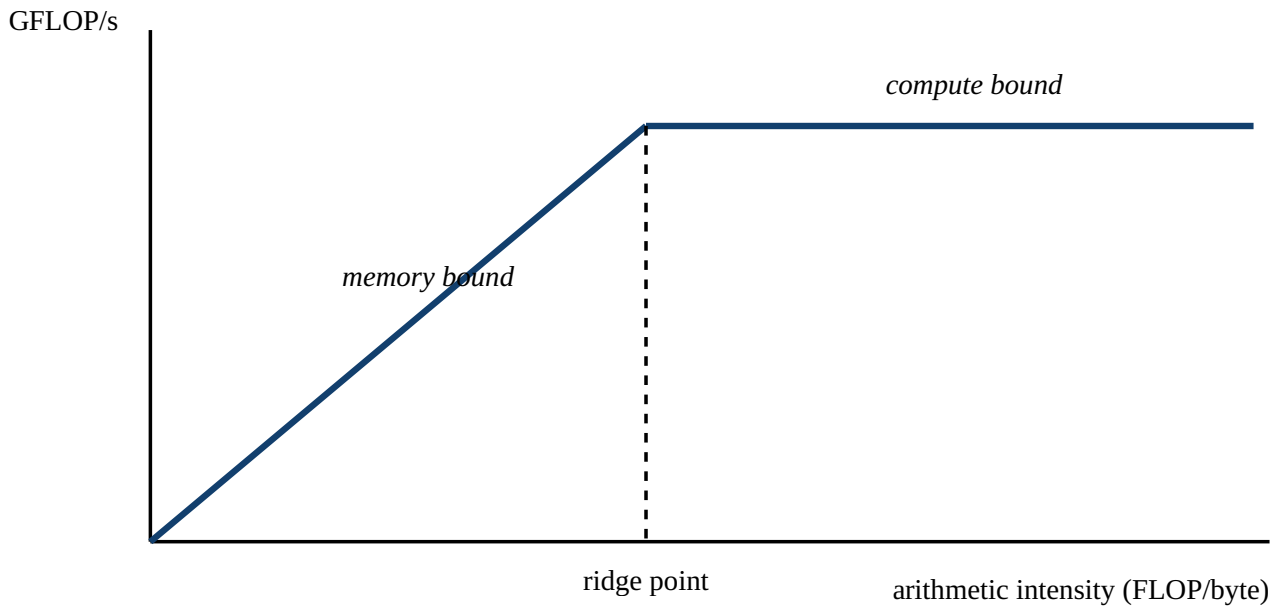
Graduate · **For an IP vendor the portability question dominates the density question.** A compiled SRAM is smaller and faster than a flip-flop array, but it is a foundry-specific macro: shipping RTL that instantiates one ties the customer to a particular process and library. The standard resolution is a **memory wrapper** — the IP instantiates a thin module with a defined interface, and the customer maps it to their compiler's output or to a behavioural model. A well-built IP ships the wrapper, a synthesisable fallback, and a document specifying the required timing (read latency, write behaviour on a read–write collision). **Omitting the collision behaviour is a frequent and expensive omission**, because different compilers do different things and the block's correctness may depend on which.

Table 106. Memory design considerations

Issue	Consequence	Handling
Read/write collision	Behaviour is compiler-dependent	Specify it ; forbid it in the design if possible
Banking	Enables parallel access	Address mapping becomes a design choice
Wide words vs many banks	Power and area trade	Follow the access pattern
ECC	Soft-error protection	Adds a cycle of latency and ~12–25% bits
Scrubbing	Corrects accumulated errors	Background read–correct–write
Redundancy and repair	Yield	Foundry feature; needs BIST/BIRA
Retention voltage	Low-power state	Below it, data is lost

Graduate · **ECC on memory is a reliability calculation, not a preference.** Soft-error rate is quoted in FIT per megabit (failures in 10^9 device-hours), typically a few hundred FIT/Mb at sea level for SRAM. A 10 Mb on-chip memory at 300 FIT/Mb gives 3000 FIT, or roughly one upset per 38 years per device — negligible for a phone, unacceptable for a fleet of a million servers, where it means an upset every twenty minutes somewhere in the fleet. **The same silicon needs ECC in one market and not in another**, which is why it must be a parameter rather than a fixed design choice, and why the datasheet must state the FIT contribution with and without it.

J2. Accelerators: Systolic Arrays and the Roofline

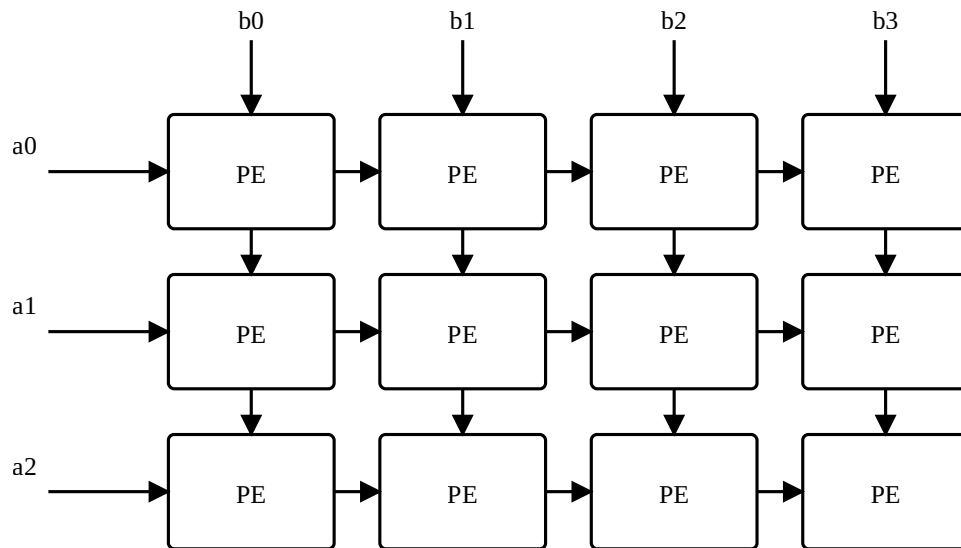


Left of the ridge, adding compute units buys nothing

Figure 13. The roofline model. A kernel's arithmetic intensity determines whether more compute or more bandwidth helps.

$$\text{Attainable performance} = \min(\text{Peak compute}, \text{Bandwidth} \times \text{Arithmetic intensity})$$

Graduate · **The roofline is the first slide of any accelerator proposal, and it decides the architecture.** Arithmetic intensity — useful operations per byte moved from memory — is a property of the algorithm and its dataflow, not of the hardware. A naive matrix multiply that re-reads operands has low intensity and is memory-bound: adding multipliers achieves nothing. Tiling the computation so that each loaded element is reused n times multiplies the intensity by n and moves the kernel rightward past the ridge, at which point additional compute finally pays. **Every accelerator architecture is, at bottom, a scheme for raising arithmetic intensity.**



Each datum is read from memory once and reused across the array

Arithmetic intensity rises with array size — that is the whole point

Figure 14. A systolic array. Operands flow through the array and are reused by every processing element they pass.

Table 107. Accelerator dataflow taxonomy

Dataflow	What stays resident	Best when
Weight stationary	Weights in the PE	Weights are reused across many activations (CNN inference)
Output stationary	Partial sums in the PE	Long reduction chains; minimises partial-sum movement
Row stationary	A row of the convolution	Balances all three reuse types
No local reuse	Nothing	Simple, memory-bound

Table 108. Precision in inference accelerators

Format	Bits	Effect	Hardware cost
FP32	32	Baseline	Reference
BF16	16	Same exponent range as FP32, fewer mantissa bits	~1/2 multiplier area; drop-in for training
FP16	16	Smaller range — needs loss scaling	~1/2
INT8	8	Requires calibration or QAT	~1/4 area, ~1/4 energy of FP32
INT4 / INT2	4/2	Significant accuracy loss without care	Smaller still
Binary / ternary	1–2	XNOR–popcount replaces multiply	No multipliers at all
Mixed / per-channel scale	—	Recovers most INT8 accuracy	Per-channel scaling logic

Graduate · Binary networks replace arithmetic with logic, which changes the whole cost structure. When weights and activations are ± 1 , a multiply becomes an XNOR and the accumulation becomes a population count. A 256-input dot product is then 256 XNOR gates and a popcount tree — a few hundred gates instead of 256 multipliers. The trade is accuracy, recovered partially by keeping the first and last layers at higher precision and by batch normalisation folded into a threshold

comparison. **For an IP house this is attractive because the resulting block is small, fully synthesisable, and has no dependence on DSP hard macros**, which makes it portable across FPGA and ASIC targets.

Table 109. Accelerator interface decisions

Decision	Options	Consequence
Weight delivery	Resident in the bitstream vs streamed at run-time	Dominates the system architecture: bandwidth vs flexibility
Granularity	Layer-at-a-time vs fused layers	Fusion removes intermediate memory traffic
Control	Fixed sequence vs programmable	Flexibility vs area
Memory	Scratchpad vs cache	Scratchpad is deterministic; cache is easier to program
Sparsity	Dense vs structured vs unstructured	Unstructured sparsity rarely pays in hardware

J3. On-Chip Interconnect and Networks

Table 110. Interconnect topologies

Topology	Scaling	Latency	Use
Shared bus	Poor beyond ~8 masters	Low	Small SoC
Crossbar	$O(N^2)$ area	Low	Moderate port counts
Mesh NoC	Good	$O(\sqrt{N})$ hops	Many-core
Torus	Good	Lower diameter	—
Ring	Simple	$O(N)$	Moderate core counts
Tree / fat tree	Good for hierarchical traffic	—	Cache hierarchies
Butterfly / Clos	Good	Uniform	Switches

Table 111. NoC design issues

Issue	Cause	Solution
Deadlock	Cyclic dependency among buffers	Virtual channels, turn-model routing (e.g. dimension-order)
Livelock	Packets circulate without progress	Bounded misrouting, ageing
Head-of-line blocking	One blocked packet stalls the queue	Virtual channels
Fairness	Some sources starve	Arbitration policy
QoS	Mixed traffic classes	Priority virtual channels, rate limiting
Flow control	Buffer overflow	Credit-based

Graduate · **Deadlock freedom is a structural property and should be established structurally, not tested.** Dimension-order routing (traverse X fully, then Y) makes cyclic buffer dependencies impossible in a mesh, and the argument is a short proof rather than a simulation campaign. Adaptive routing performs better under load but reintroduces the possibility, which is then removed by reserving an escape virtual channel that uses dimension-order. **This is an unusually clean example of the general principle that architecture should make undesirable states unreachable rather than merely unlikely** — the same principle behind sparse FSM encodings and Gray-coded FIFO pointers.

Table 112. Chip-to-chip and chiplet interfaces

Standard	Scope	Note
PCIe	Board-level, packetised	Gen6 uses PAM4 and FLIT mode
CXL	Cache-coherent over PCIe PHY	Memory expansion and pooling
UCIe	Die-to-die in a package	Very short reach → very low energy per bit
BoW / AIB	Die-to-die	Open alternatives
HBM	Memory via silicon interposer	Very wide, moderate rate
Ethernet	Rack and beyond	—

Graduate · **Chiplets change the economics that an IP business operates in.** When a system is partitioned across dies, the interface between them becomes a standardised product boundary, and a company can sell an entire die rather than an RTL block. Energy per bit is the defining metric: on-chip wires are ~0.1 pJ/bit, UCIe die-to-die ~0.5 pJ/bit, and long-reach SerDes ~5–10 pJ/bit. **That two-order-of-magnitude spread is why partitioning decisions are made at the package level and why**

die-to-die PHY IP became valuable so quickly. A digital design house cannot build the PHY, but the link layer, the retry and CRC logic, and the protocol adapters above it are all synthesisable — the familiar pattern of a sellable digital block adjacent to an unreachable analogue one.

J4. Image and Video Coding

Table 113. Codec pipeline stages

Stage	Purpose	Hardware character
Prediction (intra)	Exploit spatial correlation	Many modes → mode decision is a search
Prediction (inter / motion)	Exploit temporal correlation	Motion estimation dominates encoder cost
Transform	Energy compaction (integer DCT)	Multiplier-free integer approximations
Quantisation	Lossy step — rate control lives here	Divide by a table value
Entropy coding	Remove remaining redundancy (CABAC)	Serial bottleneck
Loop filter (deblocking, SAO, ALF)	Remove artefacts	In the reconstruction loop — must be bit-exact
Reference frame store	—	DRAM bandwidth dominates

Graduate · **Encoder and decoder are asymmetric by design.** The standard specifies only the decoder; the encoder may choose modes however it likes, provided the bitstream is conformant. This means **encoder quality is a product differentiator while decoder correctness is a conformance requirement**, and the two have completely different verification strategies: a decoder is verified against conformance bitstreams with bit-exact expected output, whereas an encoder is evaluated on rate-distortion curves and must only produce decodable output. A design house entering video should know which side it is selling into, because the effort profiles differ by an order of magnitude.

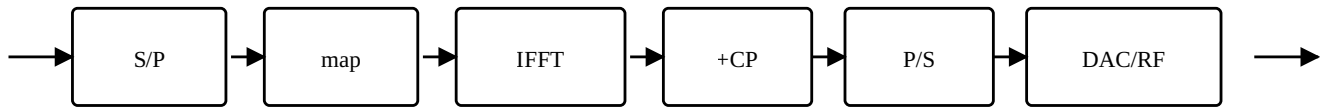
Table 114. Standards and their distinguishing features

Codec	Year	Key additions	Hardware impact
JPEG	1992	8×8 DCT, Huffman	Simple, still ubiquitous
H.264/AVC	2003	Integer transform, CABAC, variable block size	Integer transform removes encoder/decoder drift
HEVC/H.265	2013	CTU up to 64×64, 35 intra modes, SAO	Much larger mode search
VP9 / AV1	2013/2018	Royalty-free, ANS coding, many partitions	Very large encoder search space
VVC/H.266	2020	More partitions, ALF, affine motion	Further complexity
JPEG XS / light-weight	—	Low latency, visually lossless	Line-based, small buffers — a good niche for a small IP house

Bit-exactness inside the reconstruction loop is non-negotiable. The decoder's output feeds the encoder's prediction, so any arithmetic difference accumulates frame after frame — drift. This is why every in-loop operation (transform, quantisation, interpolation filters, deblocking) is specified as exact integer arithmetic with defined rounding, and why a reference model for a codec must reproduce those operations bit for bit rather than mathematically. **A floating-point reference model of a video codec is not a reference model.**

K1. Wireless Physical Layers

K1.1 OFDM



Cyclic prefix $\geq$ channel delay spread

$\Rightarrow$ multipath becomes circular convolution

$\Rightarrow$ one complex tap per subcarrier equalises the channel

The CP converts a hard equalisation problem into a division

Figure 15. OFDM transmitter. The cyclic prefix is the trick that makes frequency-domain equalisation valid.

Graduate · Why the cyclic prefix works, stated precisely. A linear convolution with the channel is not diagonalised by the DFT; a *circular* convolution is. Prepending the last N_{cp} samples of the symbol makes the channel's linear convolution look circular to the receiver, provided N_{cp} exceeds the channel's delay spread. The channel matrix then becomes diagonal in the DFT basis, so equalisation reduces to dividing each subcarrier by one complex number. **An N -tap time-domain equalisation problem has been exchanged for N_{cp}/N of lost throughput.** That trade is the entire justification for OFDM, and it explains why the CP length in every standard is tied to the expected delay spread of its deployment environment.

Table 115. OFDM impairments and their remedies

Impairment	Effect	Remedy	Where implemented
PAPR	Sum of subcarriers has a high peak	Clipping, tone reservation, SLM; or use SC-FDMA uplink	Digital pre-distortion; amplifier back-off
Carrier frequency offset	Destroys subcarrier orthogonality (ICI)	Estimate from preamble/pilots, correct before FFT	Digital
Sampling clock offset	Progressive phase rotation	Track with pilots	Digital
Phase noise	Common phase error + ICI	Estimate CPE from pilots	Digital
I/Q imbalance	Mirror-image interference	Estimate and compensate	Digital
Doppler	Time-varying channel within a symbol	Shorter symbols, or per-symbol tracking	Design-level

Graduate · Frequency offset is the impairment that must be fixed first and fixed well. A residual offset destroys orthogonality and produces inter-carrier interference that no subsequent equaliser can undo, because the subcarriers are no longer separable. The tolerance is roughly 1% of the subcarrier spacing for negligible loss; in 5G NR with 30 kHz spacing that is 300 Hz — at 3.5 GHz carrier, under 0.1 ppm. **This single requirement sets the crystal specification for the whole device,** which is a good illustration of a digital-layer requirement propagating down to a bill-of-materials cost.

K1.2 MIMO

Table 116. MIMO techniques

Technique	Objective	Requirement	Processing
Spatial multiplexing	Throughput	Rich scattering	Matrix inversion or successive cancellation per sub-carrier
Space-time coding	Diversity	None	Simple combining (Alamouti)
Beamforming	SNR, interference	Channel knowledge at the TX	Weight computation
Massive MIMO	Both	Many antennas, reciprocity	Large matrix operations; Cholesky/QR territory
MU-MIMO	Multi-user capacity	Multi-user CSI	Precoding (ZF, MMSE, dirty paper)

Table 117. MIMO detection algorithms

Detector	Complexity	Performance	Hardware
Zero forcing	One matrix inverse	Noise enhancement	Cholesky on $H^H H$
MMSE	One matrix inverse	Better at low SNR	Same, with a regularising term
V-BLAST / SIC	Iterative	Better	Ordering + repeated inversion
Sphere decoding	Variable	Near-ML	Data-dependent latency — awkward
K-best	Fixed	Near-ML	Fixed latency — preferred in hardware
Belief propagation	Iterative	Good	Fits with LDPC decoding

Graduate · **K-best beats sphere decoding in hardware for a reason that has nothing to do with performance.** Sphere decoding prunes a search tree adaptively, so its runtime depends on the channel and the noise realisation; a pipelined receiver with a fixed symbol rate cannot accommodate variable latency without buffering for the worst case, which negates the saving. K-best keeps a fixed number of candidates at every level, giving constant throughput and latency at slightly worse error performance. **Determinism is worth more than average-case efficiency in a streaming datapath,** and this principle recurs in FEC (fixed iteration counts), in SVD (fixed sweep counts), and in CFAR (fixed window sizes).

K2. Protocol Stacks as Hardware

Table 118. Ethernet layering and what each layer does

Layer	Function	Hardware	Clause (802.3)
MAC	Framing, addressing, CRC, flow control	Digital	4, 31
RS-FEC	Reed–Solomon KP4/KR4	Digital — soft IP	91, 161
PCS	64b/66b or 256b/257b coding, scrambling, lane distribution, alignment markers, gearbox	Digital — soft IP	49, 82, 119
PMA	Serialisation, clock recovery, equalisation	Analogue/mixed — hard macro	—
PMD	Electrical or optical interface	Analogue	—
AN / LT	Auto-negotiation, link training	Digital + control	73, 72

Graduate · **The PCS/PMA boundary is the commercial boundary of the whole industry.** Vendors describe their products in exactly these terms — "hardened PMA with a soft PCS layer" — because the PMA is tied to a process node and must be delivered as a hard macro, while the PCS is process-independent RTL. For a digital design house this boundary defines what is reachable: everything from the PCS upward. It is also where the open-source landscape is thinnest — 10G BASE-R PCS exists in open form, but 100G-and-above multi-lane PCS and RS-FEC do not — which makes it a reasonable place to look for an entry product.

Table 119. 64b/66b coding: what each piece is for

Element	Purpose	Failure if omitted
2-bit sync header	Distinguishes data from control blocks	Receiver cannot find block boundaries
Self-synchronising scrambler ($x^{58}+x^{39}+1$)	DC balance and transition density	Baseline wander through AC coupling; CDR loses lock
Block type field	Encodes control block layout	—
Block lock state machine	Finds the header position	No synchronisation
BER monitor	Counts invalid sync headers	No link quality signal
Gearbox	Maps 66 bits onto a 64-bit datapath	Rate mismatch — 66 does not divide the bus width

Graduate · **The gearbox is the piece newcomers underestimate.** A 66-bit block does not fit a 64-bit datapath, so the PCS must repack a 66-bit stream into 64-bit words, which realigns every block by two bits and returns to alignment only after 32 blocks. The resulting barrel shifter and control state machine are a surprisingly large fraction of the PCS. Multi-lane 100G+ adds alignment markers, lane deskew and reordering on top. **The "simple" coding layer is mostly plumbing, and the plumbing is where the engineering effort goes** — a pattern that recurs in every protocol block.

Table 120. PCIe layering

Layer	Function	Notes
Transaction (TLP)	Read/write/completion semantics, ordering rules	Ordering rules are subtle and a frequent source of bugs

Layer	Function	Notes
Data link (DLLP)	Sequence numbers, ACK/NAK, credit flow control	Replay buffer sizing matters
Physical logical	Encoding (8b/10b, 128b/130b, FLIT), scrambling, lane management	—
Physical electrical	SerDes	Hard macro
LTSSM	Link training state machine	Large, intricate FSM — a classic formal-verification target

Table 121. DDR memory interface

Element	Function	Difficulty
Command scheduling	Bank, row, column timing	Performance-critical
Training	Write levelling, read gate, per-bit deskew	Runs at boot; a mix of hardware and firmware
PHY	DLL/PLL, DQS strobing	Hard macro
ECC	SECEDED or chipkill	Latency and bandwidth cost
Refresh	Periodic; blocks access	Real-time guarantee impact
Rowhammer mitigation	Track and refresh victims	Security requirement

K3. Reliability, Safety and Yield

Table 122. Wear-out mechanisms

Mechanism	Physics	Accelerated by	Design response
NBTI / PBTI	Interface trap generation under gate bias	Voltage, temperature, duty cycle	Timing guard band; adaptive body bias
HCI	Hot carriers damage the oxide	Switching activity, high field	Reduce voltage; limit slew
TDDB	Oxide breakdown	Field, temperature	Thickness rules; Weibull statistics
Electromigration	Momentum transfer moves metal atoms	Current density, temperature	Wire width rules, via arrays
Stress migration	Thermo-mechanical stress	Thermal cycling	Layout rules
Soft errors	Particle strikes	Altitude, neutron flux	ECC, TMR, scrubbing

Graduate · Wear-out is a timing constraint, not an afterthought. NBTI shifts $|V_{TH}|$ over the device lifetime, so a path that just met timing on day one may fail after five years. Sign-off therefore uses an aged library or an explicit derate — commonly 5–10%. For an IP vendor this matters in a specific way: **a frequency claim is implicitly a claim about a lifetime**, and the two must be quoted together. "500 MHz" at fresh silicon and "500 MHz over ten years at 125 °C" are different products.

Table 123. Functional safety (ISO 26262) concepts

Concept	Meaning	Hardware implication
ASIL A–D	Risk classification	D is the most stringent
SPFM	Single-point fault metric	≥99% for ASIL D
LFM	Latent fault metric	≥90% for ASIL D
PMHF	Probabilistic metric for random hardware failures	<10 FIT for ASIL D
Safety mechanism	Detects or controls a fault	ECC, lockstep, watchdogs, BIST
Diagnostic coverage	Fraction of faults detected	Drives the metrics
Lockstep	Duplicate cores compared cycle by cycle	Delayed lockstep avoids common-cause failures
FMEDA	Failure mode and diagnostic analysis	A required deliverable for safety IP

Safety certification changes the deliverable list, not just the design. An ASIL-D block must ship with an FMEDA showing computed SPFM, LFM and PMHF, a safety manual stating the assumptions of use, and evidence of a qualified development process. A technically excellent block without these documents cannot be used in a safety application at any price. **For a small design house this is the clearest example of a market that is closed not by engineering capability but by process obligations**, and it should be entered deliberately or not at all.

Table 124. Yield

Concept	Expression	Meaning
Defect density D_0	defects/cm ²	Process maturity
Poisson yield	$Y = e^{-AD_0}$	Pessimistic for large dies
Negative binomial	$Y = (1 + AD_0/\alpha)^{-\alpha}$	Accounts for defect clustering
Die area effect	—	Yield falls steeply with area — the economic driver behind chiplets

Concept	Expression	Meaning
Repair / redundancy	—	Spare rows and columns in memories raise yield
Parametric yield	—	Devices that work but miss specification

K4. Current Research Directions

This chapter surveys the areas where current literature is active, so that a reader who has absorbed the preceding chapters can approach a recent paper knowing what problem it is addressing and which established constraint it is trying to relax.

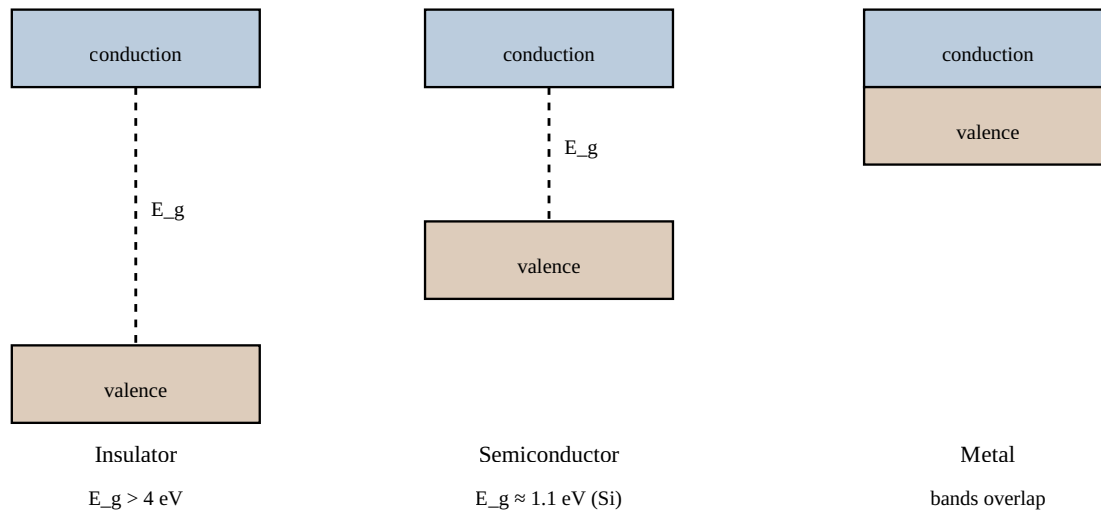
Table 125. Active areas and the constraint each attacks

Area	Constraint attacked	Approach	Open problem
Post-quantum cryptography	Shor's algorithm breaks RSA/ECC	Lattice (ML-KEM/ML-DSA), hash-based (SLH-DSA)	Large keys; NTT throughput; side-channel protection of lattice arithmetic
In-memory computing	Von Neumann data movement energy	Analogue MAC in memory arrays (RRAM, SRAM)	ADC overhead often cancels the saving; device variability
Approximate computing	Exactness is expensive	Truncated multipliers, stochastic computing, voltage overscaling	Error bounds must be guaranteed, not measured
Chiplets / UCle	Reticle limit and yield vs die area	Disaggregation with standard die-to-die links	Test, thermal, known-good-die
Nonlinear equalisation	Linear equalisers fail on nonlinear channels	Volterra series, neural equalisers	Power and latency; verification of a learned block
Concatenated FEC for 224G	KP4 alone insufficient at 200G/lane	Outer RS-KP4 + inner Hamming(128,120)	Latency budget; decoder complexity
Formal at scale	Simulation cannot cover state space	Word-level model checking, SMT	Datapath equivalence remains hard
ML for EDA	Heuristics in placement/routing	Learned cost models, RL for scheduling	Reproducibility and trust in sign-off
Open PDK / open EDA	Tool licence cost as a barrier	SkyWater/IHP PDKs, OpenROAD, Yosys	Analogue and advanced-node support
Cryogenic and superconducting	CMOS energy floor	SFQ logic, cryo-CMOS	Infrastructure; interface to room temperature

Graduate · How to read a paper in any of these areas. Each row names a constraint from earlier chapters. In-memory computing attacks the roofline's memory-bound region (J2); approximate computing attacks the fixed-point accuracy–area trade (A3, C1); neural equalisation attacks the linear-model assumption behind every equaliser in F1; chiplets attack the yield-versus-area relation in K3. **Identifying which constraint a paper is relaxing, and what it pays for the relaxation, is a faster route to understanding than reading the method first** — and it immediately tells you whether the result transfers to your own problem, because you know which assumption it depends on.

A caution that applies to all of these areas. Reported gains are measured at an operating point the authors chose. Before accepting a claim, locate three things: the baseline (is it a strong implementation or a straw man?), the operating conditions (process, voltage, data set, channel), and the point at which the advantage disappears. A result quoted without the crossover point is a data point, not a design rule — and in an IP business, mistaking one for the other is how a year is lost.

L1. Semiconductor Physics: What Survives into Design



Doping moves the Fermi level; the band gap sets the intrinsic carrier density

Figure 16. Band structure. The gap determines whether a material conducts, and silicon's 1.1 eV gap is what makes it useful.

Table 126. Physical quantities and where a designer meets them

Quantity	Symbol / value	Design consequence
Band gap of Si	1.12 eV at 300 K	Falls with temperature $\rightarrow$ leakage rises, V_{TH} falls
Intrinsic carrier density	$n_i \approx 10^{10} \text{ cm}^{-3}$	Doubles roughly every 8 K $\rightarrow$ junction leakage doubles with it
Thermal voltage	$kT/q = 25.9 \text{ mV}$ at 300 K	Sets the subthreshold slope: 60 mV/decade minimum
Mobility μ	$\approx 1400 \text{ (e)}, 450 \text{ (h)} \text{ cm}^2/\text{Vs}$	Why PMOS is sized $\sim 2\text{--}3\times$ wider than NMOS
Saturation velocity	$\approx 10^7 \text{ cm/s}$	Short-channel current becomes linear in overdrive
Oxide capacitance	$C_{ox} = \epsilon_{ox}/t_{ox}$	Thinner oxide $\rightarrow$ more drive and more gate leakage
Depletion width	$\propto \sqrt{(V/N)}$	Short-channel effects; junction capacitance

Graduate The 60 mV/decade subthreshold slope is a hard physical limit and it ended Dennard scaling. Below threshold, drain current depends exponentially on gate voltage with a slope of at best $(kT/q) \ln 10 \approx 60 \text{ mV}$ per decade at room temperature, because the mechanism is thermal emission over a barrier. To keep leakage constant while lowering the supply, the threshold must fall by the same amount — but each 100 mV of threshold reduction multiplies leakage by about fifty. **This is why supply voltage stopped scaling near 0.8 V and why power density rose**, which in turn is why the industry turned to specialised accelerators and therefore why the IP market grew. A thermodynamic constant is visible in a business model.

Table 127. Devices beyond the planar MOSFET

Device	Principle	Motivation
FinFET	Gate wraps a vertical fin on three sides	Better electrostatic control $\rightarrow$ less DIBL and leakage
GAA / nanosheet	Gate surrounds the channel completely	Further control; width quantised by sheet count

Device	Principle	Motivation
FD-SOI	Thin body on buried oxide	Low leakage; body bias as a live tuning knob
Tunnel FET	Band-to-band tunnelling	Can beat 60 mV/decade ; low on-current so far
Negative-capacitance FET	Ferroelectric gate stack	Internal voltage amplification; reliability unproven
2D materials	MoS ₂ and similar	Ultra-thin body; research stage

L2. Power Delivery and Conversion

Table 128. Voltage regulator types

Type	Efficiency	Ripple	Re- sponse	Where
LDO	V_{out}/V_{in} — poor if the drop is large	Very low	Fast	Sensitive analogue, small currents
Buck (switching)	85–95%	Higher (switching ripple)	Slower	Digital cores, large currents
Boost	—	—	—	Raising a voltage
Charge pump	Moderate	—	—	Small currents, no inductor
Integrated voltage regulator	—	—	Very fast	Per-domain DVFS on die
Switched-capacitor	Good at fixed ratios	—	—	On-die, no inductor needed

Graduate · Regulator response time is a digital design constraint. When a large synchronous block starts, it draws a current step; the regulator takes time to respond, and during that time the supply droops. The droop reduces gate drive and therefore increases delay, which can violate timing. Designers respond with **di/dt management** — staggering the release of clock gates so a block wakes in stages rather than all at once. **This is a purely digital technique introduced to solve an analogue problem, and it must be specified:** an IP that draws its full current in one cycle after an enable is harder to integrate than one that ramps.

Table 129. Decoupling hierarchy

Level	Capacitance	Effective range	Limit
On-die (MOS, MIM)	nF	>100 MHz	Area
Package	μF	10–100 MHz	Package inductance
Board ceramic	μF	1–10 MHz	ESL
Bulk electrolytic	100 μF+	<1 MHz	—
Anti-resonance	—	50–200 MHz typically	Where package L resonates with die C — must be damped

The anti-resonance peak is a common cause of otherwise inexplicable failures. Package inductance and on-die capacitance form a parallel resonance at which the power delivery network's impedance is much higher than at neighbouring frequencies. A workload whose current draw happens to contain energy at that frequency — a loop running at the resonant rate, a burst-mode accelerator — produces supply noise far larger than the average current would suggest. The symptom is a design that passes all tests but fails on one specific piece of software. **A digital block's current spectrum, not only its average power, is a system-level property.**

L3. Thermal and Packaging

$$T_{junction} = T_{ambient} + P \cdot \theta_{JA}$$

Table 130. Thermal resistance path

Segment	Symbol	Typical	Improved by
Junction to case	θ_{JC}	0.1–1 K/W	Die attach, lid material
Case to heatsink	—	0.1–0.5 K/W	Thermal interface material
Heatsink to ambient	θ_{SA}	0.5–10 K/W	Fin area, airflow
Total	θ_{JA}	—	—
Hot spot	—	—	Local power density, not average — floorplanning matters

Graduate · **Hot spots are a floorplanning problem, and they couple back into timing.** Average die temperature may be comfortable while a dense arithmetic block runs 20 K hotter than its surroundings. Since delay and leakage both depend on temperature, that block is slower and leakier than the sign-off corner assumed, and its leakage raises its temperature further — a positive feedback known as thermal runaway in the extreme. The design responses are spreading high-activity logic, inserting thermal sensors and throttling, and accounting for the gradient in timing analysis. **For an IP vendor, a block with very high local power density should say so,** because the customer's floorplanner needs to know.

Table 131. Package types

Package	Density	Thermal	Use
Wire-bond QFN/BGA	Low	Moderate	Cost-sensitive
Flip-chip BGA	High	Good (heat exits through the lid)	High performance
Silicon interposer (2.5D)	Very high	Good	HBM integration
3D stacking	Highest	Difficult — heat is trapped	Memory on logic
Fan-out wafer level	High	Moderate	Mobile
Chiplet in package	High	Manageable	Disaggregated designs

Graduate · **Thermal constraints are what limit 3D integration, not manufacturing.** Stacking a logic die under another die places its heat behind a thermal barrier, so the achievable power density falls sharply. This is why current 3D products stack memory (low power) on logic rather than logic on logic, and why thermal modelling now appears early in architecture rather than at the end. **The physical constraint determines the partitioning, which determines which interfaces are needed, which determines which IP is valuable.**

L4. Feedback and Control

Feedback loops appear throughout an IP portfolio: PLLs, CDRs, adaptive equalisers, AGC, DVFS controllers, thermal throttling, and the convergence of iterative decoders. The same small set of concepts governs all of them.

Table 132. Control concepts and their hardware instances

Concept	Meaning	Instance
Open-loop gain	Gain around the loop	PLL $K_{PD}K_{VCO}$
Loop bandwidth	Where gain crosses unity	The central design parameter of every loop
Phase margin	Phase at crossover, relative to -180°	$<45^\circ$ gives peaking and ringing
Type / order	Number of integrators / poles	Type-2 PLL tracks a frequency step with zero error
Damping ζ	—	$\zeta=0.707$ is the usual target
Steady-state error	—	Determines whether an offset persists
Settling time	—	Lock time; adaptation time
Discrete-time effects	Sampling and delay in the loop	Digital loops must include their own latency

Latency inside a digital loop is a stability problem, not merely a performance one. A digital adaptation loop that computes an error, pipelines it through several stages, and then applies a correction has inserted pure delay, which subtracts phase without reducing gain — exactly what destroys phase margin. The loop can oscillate even though every block in it is correct. The standard remedies are to reduce the loop gain (slower adaptation), to shorten the loop, or to compensate explicitly. **A model that applies the correction instantaneously will not show the instability**, so the pipeline delay must be in the model. This is one of the most common gaps between a working model and non-working hardware.

Table 133. Adaptation loops in a receiver and their interactions

Loop	Controls	Typical bandwidth	Interacts with
AGC	Input amplitude	Fast	Everything downstream
CDR	Sampling phase	Moderate	Equaliser — MM lock point depends on it
CTLE adaptation	Analogue peaking	Slow	FFE
FFE/DFE adaptation	Tap weights	Slow	CDR
Offset calibration	Comparator offsets	Very slow	Slicer thresholds
Interleaving calibration	ADC mismatch	Very slow	All of the above

Graduate · Six loops sharing one signal is the hardest part of a modern receiver, and it is a systems problem rather than a circuit one. Each loop is stable alone; together they can hunt, lock to a wrong point, or fail to converge from certain initial conditions. The standard engineering response is **bandwidth separation** — giving each loop a rate an order of magnitude apart so that faster loops appear instantaneous to slower ones and slower loops appear constant to faster ones — plus a defined start-up sequence. **Both the separation and the sequence belong in the specification**, and verifying convergence from adverse initial conditions is a system-model task that no block-level testbench will perform.

M1. The Hardware–Software Interface

An IP block is useless until software can drive it. The interface between them is a design artefact in its own right, and getting it wrong produces integration cost that dwarfs the block's development.

Table 134. Interface mechanisms

Mechanism	Direction	Cost	When appropriate
Memory-mapped registers	CPU → block	Low	Configuration, status; the default
Polling	CPU reads status	Wastes CPU cycles and power	Very short operations
Interrupt	block → CPU	Latency and context switch	Completion of long operations
DMA	block ↔ memory	Bus bandwidth	Bulk data; avoids CPU copying
Descriptor rings	—	Memory traffic	Streaming with many buffers (network, storage)
Mailbox / doorbell	bidirectional	Low	Command submission
Shared memory + fences	—	Coherence traffic	Tight coupling; requires a memory model

Graduate · **The single most consequential interface decision is who moves the data.** A block that requires the CPU to write its input and read its output word by word can be limited by the CPU even when the block itself is fast; a DMA-capable block moves data in bursts and leaves the CPU free. The difference is frequently an order of magnitude in system throughput, and it is invisible in block-level benchmarks. **For an IP vendor this means the example design matters as much as the block:** a customer who integrates it through programmed I/O will measure disappointing performance and conclude the IP is slow.

Table 135. Register map design rules

Rule	Reason
Generate RTL, header, documentation and tests from one source	Three hand-maintained copies always diverge
Reserve and document unused bits	Forward compatibility
Make read-clear and write-one-clear behaviour explicit	Ambiguity here produces lost interrupts
Provide an identification and version register	Software can adapt to the actual hardware
Group by function, align to natural boundaries	Simplifies access
Avoid side effects on read where possible	A debugger that dumps registers must not change behaviour
State atomicity for wide registers	Prevents tearing

Read side effects break debuggers, and the failure is baffling when it happens. If reading a FIFO status register pops an entry, then attaching a debugger that displays the register window silently consumes data, and the bug disappears when the debugger is detached. Where a read side effect is genuinely required — a pop-on-read data port, for example — it must be isolated in its own register and clearly marked, so that a debug tool can be told to avoid it. This is a small documentation duty with a disproportionate payoff.

Table 136. Driver and firmware concerns an IP vendor should anticipate

Concern	What the IP must provide
Initialisation sequence	Exact required order, including any wait conditions
Reset semantics	What a soft reset does and does not clear
Error reporting	Sticky status bits with a defined clearing mechanism
Concurrency	Which registers may be accessed while the block is running
Endianness and bit order	Stated explicitly
Timeouts	Maximum time any operation can take
Low-power entry/exit	Sequence and state retention

M2. Machine Learning Hardware

Table 137. Operations that dominate inference

Operation	Share of compute	Arithmetic intensity	Hardware mapping
Convolution	High in CNNs	High with tiling	Systolic array or vector units
Matrix multiply (GEMM)	Dominant in transformers	High	The universal accelerator primitive
Attention (QK^TV)	Dominant in LLMs	Moderate	Memory bound at long sequence lengths
Element-wise / activation	Low compute	Very low	Memory bound; fuse with the previous layer
Normalisation	Low	Low	Fuse; needs reductions
Softmax	Low	Low	Needs exp and a reduction; numerically delicate
Embedding lookup	—	Very low	Random memory access

Graduate · **Fusion is the main architectural lever and it follows directly from the roofline.** An activation function performs one operation per element and therefore has arithmetic intensity near zero — it is entirely memory bound. Executed as a separate pass it costs a full read and write of the tensor. Fused into the preceding matrix multiply's output stage it costs essentially nothing. Real accelerators therefore expose fused operations rather than primitive ones, and their compilers spend most of their effort deciding what to fuse. **An IP block that implements a single layer type in isolation will underperform in a system for reasons that have nothing to do with its internal quality.**

Table 138. Quantisation approaches

Approach	Accuracy loss	Effort	Hardware
Post-training, per-tensor INT8	Moderate	Low	One scale per tensor
Post-training, per-channel INT8	Small	Low	Per-channel scale multiply — usually worth it
Quantisation-aware training	Very small	High	Same as above
INT4 with grouping	Noticeable	High	Group-wise scales
Mixed precision	Small	Moderate	Sensitive layers kept wider
Binary / ternary	Large	High	XNOR–popcount; no multipliers

Graduate · **Quantisation is a fixed-point problem and the vocabulary of Chapter A3 applies unchanged.** A per-tensor scale is a single Q format for the whole tensor; per-channel scaling is the recognition that different channels have different dynamic ranges, which is exactly the block-floating-point idea from the FFT chapter. Understanding this correspondence prevents an engineer from treating ML quantisation as a separate discipline: the questions — where to round, when to saturate, how dynamic range grows through a computation — are the same, and the answers are reached the same way.

Table 139. Sparsity

Kind	Speedup potential	Hardware feasibility
Unstructured (random zeros)	High in theory	Poor — irregular access defeats the memory system
Structured (2:4, block)	Moderate	Good — regular indexing
Channel / filter pruning	Moderate	Excellent — it is just a smaller dense model
Activation sparsity (ReLU)	Moderate	Dynamic; needs run-time detection

Unstructured sparsity is the clearest case of a metric that does not translate into hardware. A paper reporting 90% of weights pruned implies a tenfold reduction in operations, but realising it requires indexing individual non-zeros, which destroys the regular access patterns that give an accelerator its efficiency. The achieved speedup is commonly a small fraction of the nominal one, and can be below unity. **Structured sparsity exists because hardware needs regularity more than it needs fewer operations** — a principle that recurs whenever an algorithmic saving is proposed.

M3. Image Signal Processing

Table 140. The ISP pipeline

Stage	Function	Why it is in hardware
Black level and defect correction	Sensor offset, dead pixels	Per-pixel rate
Lens shading correction	Vignetting	Per-pixel with a gain surface
Demosaic	Bayer → RGB	Neighbourhood operation at pixel rate
White balance	Illuminant compensation	Per-pixel gains
Colour correction matrix	Sensor → standard colour space	3×3 per pixel
Gamma / tone mapping	Perceptual encoding	LUT per pixel
Noise reduction	Spatial and temporal filtering	Line buffers dominate the area
Sharpening	Edge enhancement	Convolution
Scaling	Resize	Polyphase filter

Graduate · Line buffers, not arithmetic, determine ISP cost. A 5×5 neighbourhood operation at 4K resolution requires four lines of 3840 pixels each held on chip; at 12 bits per component that is roughly 184 kbit per stage before any compute. A pipeline with several such stages spends most of its silicon on storage. The architectural consequences are standard: process in tiles rather than full lines, fuse stages so that a neighbourhood is read once and used by several operations, and choose separable filters wherever possible. **This is the same memory-versus-compute analysis as the roofline of Chapter J2, applied to a different domain.**

Table 141. Colour and vision concepts that reach the hardware

Concept	Meaning	Implementation note
Bayer pattern	One colour per photosite	Demosaic must interpolate two of three
Colour space (sRGB, YCbCr)	—	YCbCr separates luma for chroma subsampling
Chroma subsampling (4:2:0)	Half resolution chroma	Exploits lower chroma acuity — halves the data
Gamma	Non-linear encoding	Must be undone before linear operations
HDR / tone mapping	Wide dynamic range compressed for display	Local operators need neighbourhood statistics
Rolling shutter	Rows exposed at different times	Motion artefacts; a system issue

Performing linear operations on gamma-encoded data is a persistent and quiet error. Gamma encoding is non-linear, so averaging two gamma-encoded pixels does not give the gamma encoding of their average. Scaling, blending and filtering must therefore be done in linear light, which means decoding before and re-encoding after. The visible symptom is subtly wrong brightness at edges — easy to miss in a test pattern and obvious on real images. **Specify the colour encoding of every interface**, in the same way and for the same reason that fixed-point formats must be specified.

M4. Measurement and Characterisation

Everything in this book that is stated as a number came from a measurement, and the difference between a useful number and a misleading one is usually in the method rather than the instrument.

Table 142. Instruments and what they actually measure

Instrument	Measures	Principal limitation
Oscilloscope	Voltage versus time	Bandwidth and probe loading
Real-time versus sampling scope	—	Sampling scopes need a repetitive signal
Spectrum analyser	Power versus frequency	Resolution bandwidth changes the displayed noise floor
Vector network analyser	S-parameters	Calibration plane definition
BER tester	Errors per bit	Time: 10^{-12} at 10 Gb/s takes ~100 s per point
Time interval analyser	Jitter statistics	Trigger accuracy
Logic analyser	Digital state	No analogue detail
Thermal camera	Surface temperature	Emissivity; not junction temperature
Power analyser / shunt	Current	Bandwidth versus resolution

Graduate · **Bathtub extrapolation exists because direct BER measurement does not scale.** Confirming 10^{-15} by counting errors would take days per operating point, which is impossible for a characterisation sweep. Instead the eye is scanned at several sampling phases, the error rate is measured where it is high enough to be quick, and the Gaussian random-jitter tails are extrapolated to the target BER. **The extrapolation is only valid if the random component really is Gaussian and has been correctly separated from the bounded deterministic component** — which is why jitter decomposition is a required capability rather than a convenience, and why a reported margin should always name the method.

Table 143. Characterisation practices

Practice	Reason
Sweep, do not spot-check	A single operating point can be unrepresentative or accidentally favourable
Confirm the operating point is healthy first	Measurements taken on a broken setup look plausible
Obtain the same quantity a second way	An independent estimate catches methodological errors
List the trivial explanations and rule each out by measurement	A result that could be an artefact is a hypothesis, not a finding
Record conditions with every number	Temperature, voltage, part, instrument, settings
Report the condition under which the claim fails	A claim with no stated failure condition is advertising

Four measurement failures, each of which produced a plausible number that meant nothing. Taking the slope of a pulse at its peak — which is zero by definition — and reporting it as a sensitivity. Measuring residuals at an operating point where BER was already 0.49, so the link carried no information. Asking a tool for the longest path in an entire netlist when the quantity of interest was the delay around one feedback loop. Reporting a lookup table's accuracy at an operating point where the feedback taps were so small that the loop was effectively inactive, so the table was never exercised. **In every case the instrument worked and the number was wrong, because the quantity measured was not the quantity intended.** The four rules above are written to prevent exactly these.

N1. Asynchronous and Globally Asynchronous Design

Synchronous design — one clock, one timing constraint — is overwhelmingly dominant because it makes timing analysis tractable. Asynchronous techniques remain relevant in three specific places, and knowing which is which prevents both over-application and blind avoidance.

Table 144. Asynchronous styles

Style	Timing assumption	Property	Practical use
Bundled data	Matched delay line models the logic	Simple, small	Local handshakes, self-timed SRAM
Dual-rail / QDI	None (delay insensitive)	Robust to any delay; $\sim 2\times$ area	Extreme reliability, some security uses
Micropipelines	Bundled	Elastic pipelines	—
GALS	Synchronous islands, async between	Avoids a global clock tree	The practically important case — multi-domain SoCs
Self-timed memory	Internal	Hidden from the user	Nearly all SRAM

Graduate · **GALS is what most engineers actually encounter, usually without the name.** Any SoC with several clock domains connected by asynchronous FIFOs is globally asynchronous and locally synchronous. Framing it that way is useful because it makes the design rules explicit: each island has its own timing closure, and every boundary between islands is a CDC problem requiring one of the techniques of Chapter C3. **The failure mode of an undisciplined multi-domain design is not a timing violation but an intermittent data corruption,** which is far harder to diagnose — which is why the boundaries must be enumerated and checked by a tool rather than by inspection.

Table 145. Where fully asynchronous logic is genuinely used

Application	Reason
SRAM internal timing	Self-timed sense amplifier control is faster than a clock
Security	No clock to glitch; power profile is less data-correlated
Ultra-low power / energy harvesting	No clock power when idle
Very wide dynamic voltage range	Works at whatever speed the supply allows
Reliability at extremes	Delay-insensitive circuits tolerate huge variation

The barrier to asynchronous design is tooling, not theory. Standard synthesis, static timing analysis, scan insertion and equivalence checking all assume a clock. An asynchronous block therefore needs custom flows, and — decisively for an IP vendor — the customer must be able to integrate, verify and test it in *their* flow. **An asynchronous block is usually unsellable as RTL for this reason alone,** regardless of its merits. Where the technique is used commercially it is nearly always hidden inside a hard macro with a synchronous interface.

N2. Physical Design in More Depth

Table 146. Placement and routing concerns

Concern	Cause	Symptom	Remedy
Congestion	More nets than routing tracks in a region	Routing failures, detours, timing loss	Spread cells, reduce utilisation, re-structure logic
Timing/congestion conflict	Timing wants cells close; routing wants them apart	—	Iterate; sometimes an architectural change is needed
Antenna effect	Charge accumulation on a long metal segment during etch	Gate oxide damage	Diode insertion, layer jumping
Crosstalk delay and noise	Coupling between adjacent nets	Timing variation, glitches	Spacing, shielding, net ordering
Electromigration	Current density	Long-term failure	Wire widening, via arrays
IR drop	Grid resistance	Local slowdown	Grid strengthening, decap
Hold violations after CTS	Skew changes after the clock tree is built	—	Buffer insertion; usually many thousands

Graduate · **Utilisation is the number that governs the whole back-end schedule.** Utilisation — the fraction of core area occupied by cells — trades area against routability. Below roughly 60% a design routes easily and wastes area; above roughly 80% congestion begins to dominate and each iteration becomes slower and less predictable. A block delivered with a recommended utilisation lets the customer floorplan correctly the first time. **Omitting it is one of the common causes of "the IP does not meet timing in our flow"**, because the customer packed it at 85% while the vendor characterised it at 65%.

Table 147. Standard cell library concepts

Concept	Meaning	Selection consequence
Track height (e.g. 7.5T, 9T)	Cell height in routing tracks	Taller cells are faster and larger
Drive strength (X1, X2, X4...)	Transistor width multiplier	Higher drive for larger loads
Multi- V_{TH} (LVT/SVT/HVT)	Threshold variants	Leakage versus speed, per cell
Characterisation corner	PVT point of the timing data	Must match the sign-off corner
NLDM / CCS	Delay model fidelity	CCS is needed at advanced nodes
Engineering change order cells	Spare gates scattered in the layout	Enable metal-only fixes after tape-out

Graduate · **Spare cells are cheap insurance that is often omitted and then badly missed.** Scattering a small percentage of unused gates across the floorplan allows a post-silicon logic fix to be implemented by changing only the metal masks, which costs a fraction of a full mask set and weeks rather than months. The area cost is typically one or two percent. **The decision is made at floorplan time and cannot be revisited**, which is why it belongs on the checklist of Chapter W1 rather than being left to back-end judgement.

N3. Statistics for Silicon

Table 148. Variation sources

Source	Scale	Correlation	Handled by
Lot to lot	Global	Fully correlated on a die	Process corners
Wafer to wafer	Global	Correlated	Corners
Die to die	Global	Correlated	Corners
Within die, systematic	Local	Position dependent	Layout-dependent effects, proximity rules
Within die, random	Local	Uncorrelated	OCV / AOCV / POCV derating
Random dopant fluctuation	Per device	None	$\sigma \propto 1/\sqrt{WL}$ — matters most for small devices

Graduate · **The distinction between correlated and uncorrelated variation is what makes modern timing analysis tractable.** If every device on a path varied independently, a long path would average out and show *less* relative variation than a short one — which is exactly what statistical on-chip variation (POCV) captures, and why it recovers pessimism that flat derating throws away. Flat OCV applies the same percentage to every path regardless of depth, which over-penalises long paths and under-penalises short ones (making hold analysis optimistic). **The statistics are not a refinement; using the wrong model can make a design that signs off fail in silicon.**

Table 149. Statistical concepts used in characterisation

Concept	Use
Mean, σ , $\pm 3\sigma$	Basic spread; $3\sigma \approx 99.73\%$
6σ design	Memory bit cells — billions of instances require extreme tails to be safe
Importance sampling	Simulating a 6 σ failure directly is infeasible; bias the sampling and correct
Monte Carlo	General variation analysis
Design of experiments	Efficient exploration of many parameters
Regression / response surface	Build a fast surrogate model
Confidence intervals	A measurement without one is an anecdote
Process capability C_{pk}	Manufacturing margin

Graduate · **Memory bit cells illustrate why tail statistics matter so much.** A 100 Mbit memory contains 10^8 cells; for the chip to yield, the per-cell failure probability must be well below 10^{-8} , which corresponds to roughly 5.7σ . Plain Monte Carlo would need on the order of 10^9 simulations to see such an event, so importance sampling — deliberately sampling the tail and reweighting — is mandatory. **This is why memory design is a specialised discipline**, and why memories come from compilers rather than being drawn by hand.

N4. Optical Links and Photonics

Beyond a few metres of copper, electrical links become impractical and the industry switches to optics. The digital blocks on either side — the FEC, the PCS, the DSP — are the same kind of IP discussed throughout this book, which is why the domain deserves a chapter.

Table 150. Optical link components

Component	Function	Impairment introduced
Laser (DFB, VCSEL)	Source	RIN, chirp, temperature drift
Modulator (direct, EML, MZM)	Impress data on light	Extinction ratio, chirp, nonlinearity
Fibre (SMF, MMF)	Transport	Chromatic dispersion, modal dispersion, attenuation, nonlinearity
Photodiode (PIN, APD)	Detect	Shot noise; APD adds excess noise
TIA	Current to voltage	Noise; bandwidth
Coherent receiver	Recover amplitude and phase	Requires a local oscillator and heavy DSP

Table 151. Modulation and detection schemes

Scheme	Detection	Reach	DSP required
NRZ / PAM4 IM-DD	Direct (intensity only)	≤ 10 km	Equalisation, FEC — 802.3 territory
Coherent QPSK / 16-QAM	Coherent (amplitude and phase)	Long haul	Very heavy: CD and PMD compensation, carrier recovery
Probabilistic shaping	Coherent	Long haul	Shaping encoder/decoder

Graduate • **Chromatic dispersion compensation is where photonics becomes a large digital problem.** Different wavelengths travel at slightly different speeds in fibre, so a pulse spreads over distance — and over hundreds of kilometres the spread reaches hundreds of symbol periods. Compensating it optically requires dispersion-compensating fibre; compensating it digitally requires a very long filter, which is implemented in the frequency domain because a direct FIR of that length is uneconomical. **The result is that a coherent optical receiver contains one of the largest FFT blocks in commercial electronics,** and the chapter on multirate and transform architectures applies directly. This is an unusually clear case of a physical-layer impairment creating a large, well-specified digital IP opportunity.

Table 152. Co-packaged and silicon photonics

Development	Motivation	Consequence
Pluggable optics	Serviceability	Electrical link to the module costs power
Co-packaged optics	Remove the electrical link	Large power saving; serviceability and thermal challenges
Silicon photonics	CMOS-compatible fabrication	Integration density, cost
Optical I/O for chiplets	Bandwidth density	Research to early product

Q1. Performance Modelling and Queueing

Functional correctness is verified against a golden model; performance is predicted by a different kind of model entirely. Confusing the two is a common and expensive mistake, because a functionally perfect block can still fail its throughput requirement in a system.

Table 153. Model types and what each answers

Model	Question answered	Cost	Fidelity
Analytical (closed form)	Rough sizing, bounds	Minutes	Low but instant
Spreadsheet / static	Bandwidth and utilisation budgets	Hours	Adequate for first-order decisions
Queueing model	Latency under load, buffer sizing	Days	Good when arrivals are stochastic
Trace-driven simulation	Behaviour on real workloads	Days	High
Cycle-approximate (SystemC TLM)	System behaviour before RTL exists	Weeks	Good
Cycle-accurate	Exact cycle counts	Weeks–months	Highest
RTL simulation	Ground truth	—	Exact but far too slow for system studies

Graduate · The practical value of the middle rows is that they exist before RTL does. An architecture decision — how deep a buffer, how many outstanding transactions, whether to double the datapath width — must be made months before there is anything to simulate. A cycle-approximate SystemC model built from the specification answers it in days, and being wrong by 10% is usually irrelevant when the alternatives differ by a factor of two. **Insisting on cycle-accurate fidelity for an architectural question delays the decision until it is too late to act on.**

Table 154. Queueing results worth knowing

Result	Statement	Use
Little's law	$L = \lambda W$	Occupancy = arrival rate × latency — sizes any buffer
M/M/1 latency	$W = 1/(\mu - \lambda)$	Latency diverges as utilisation → 1
M/M/1 occupancy	$\rho/(1-\rho)$	At 90% utilisation the mean queue is 9
M/D/1	Deterministic service	Half the waiting time of M/M/1
Erlang B	Blocking probability	Sizing a finite resource pool
Burstiness	—	Real traffic is burstier than Poisson — Poisson results are optimistic

Graduate · Little's law is the single most useful formula in system design and it requires no assumptions about distributions. To sustain 10 GB/s with a 500 ns round trip, the system must keep $10 \times 10^9 \times 500 \times 10^{-9} = 5$ kB in flight; if the interface supports 32 outstanding 64-byte requests, that is 2 kB and the link will run at 40% of its rate no matter how fast it is. **This is the quantitative form of the PCIe tag-capacity argument in Chapter P2**, and the same calculation sizes DMA descriptor rings, memory controller queues and cache miss-status registers. Learning to apply it in thirty seconds is worth more than any simulator.

Designing for average load is designing to fail. The M/M/1 result shows latency rising hyperbolically with utilisation: at 50% the queue is 1, at 90% it is 9, at 99% it is 99. Systems provisioned at their average demand spend a substantial fraction of their time near saturation, where latency is dominated by queueing rather than service. **Buffer sizing must be driven by**

the tail of the arrival distribution, not its mean, and a performance specification that quotes only average throughput is incomplete.

Q2. Compilers and High-Level Synthesis Internals

HLS is often treated as a black box. Knowing what it does internally makes the difference between fighting the tool and directing it, and the internal steps are the same scheduling and binding problems that appear elsewhere in design.

Table 155. The HLS pipeline

Step	Function	What the designer controls
Front end	Parse to an intermediate representation	Language subset used
Optimisation	Constant folding, loop transforms, inlining	Code structure; pragmas
Scheduling	Assign operations to clock cycles	Target II and latency constraints; dependences
Allocation	Decide how many of each resource	Resource constraints, binding directives
Binding	Map operations to specific units, values to registers	Storage directives
Interface synthesis	Turn arguments into ports/protocols	Interface pragmas
RTL generation	Emit Verilog/VHDL	—

Graduate · **Scheduling is a constrained optimisation problem and that explains the tool's behaviour.** Given a dependence graph and resource limits, the scheduler minimises latency or the initiation interval. Because the problem is NP-hard in general, tools use heuristics (list scheduling, modulo scheduling for pipelines, occasionally ILP for small regions). Two consequences follow. First, a small change in the source can produce a disproportionate change in the schedule, because the heuristic follows a different path — which is why HLS results can be unstable. Second, **the tool cannot schedule around a dependence it cannot disprove**, so telling it the truth about aliasing is the highest-leverage thing a designer does. That is what a dependence directive is: not an optimisation hint, but a fact the tool could not derive.

Table 156. Directive families and what they actually do

Family	Examples	Nature	Risk of misuse
Scheduling	pipeline, unroll, inline, loop_flatten, loop_merge	Requests — the tool may not achieve them	Silent failure to meet the target; check the report
Structural	array_partition, bind_storage, bind_op, dataflow, stream	Change what is built	Area explosion
Assertions	dependence, loop_tripcount	Facts the tool trusts	A false assertion produces silently wrong hardware
Interface	interface, axis, m_axi, s_axilite	Define ports	Protocol mismatch

The third family is categorically different from the other two and should be reviewed differently. A scheduling directive that cannot be satisfied produces a worse result and a report message. A structural directive produces a bigger design. A false dependence assertion produces hardware that computes the wrong answer, with no message and no simulation failure — because the C semantics are unaffected. **Every dependence directive is therefore a proof obligation:** the reviewer must be able to state why the asserted independence holds for every legal parameter value. Treating it as an optimisation switch is how silent bugs enter.

Table 157. What HLS is good and bad at

Suited	Reason	Unsuited	Reason
Loop-dominated datapaths	Regular dependence structure	Cycle-exact interface protocols	The tool owns the schedule

Suited	Reason	Unsuited	Reason
Design-space exploration	One source, many points	Control-dominated logic	Irregular; RTL is clearer
Filters, transforms, crypto rounds	Well-structured loops	Blocks needing precise latency	Latency is an output, not an input
Algorithm-level changes	Edit the C, not the RTL	Deep pipelines with feedback	The iteration bound still applies

Q3. Embedded Software Around the Hardware

Table 158. Software layers that touch an IP block

Layer	Concern	What the hardware must provide
Boot ROM	Bring the system up	Deterministic reset state
Device driver	Register access, interrupts, DMA	A clean register map and a defined initialisation sequence
RTOS / kernel	Scheduling, memory management	Interrupt latency guarantees; cache coherence behaviour
Middleware	Protocol stacks	—
Application	—	—
Firmware in the block	Local control	Instruction memory, debug access

Graduate · **Cache coherence with a DMA-capable block is the interaction that produces the most confusing bugs.** If a block writes to memory by DMA while the CPU has that region cached, the CPU may read stale data; if the CPU writes and the data sits in a write-back cache, the block may read stale data. On a coherent interconnect the hardware resolves this; on a non-coherent one the driver must invalidate before reading and clean before writing, and getting the order wrong produces intermittent corruption that looks like an IP defect. **An IP datasheet must state whether its transactions are coherent,** and the example driver should demonstrate the required maintenance operations if they are not.

Table 159. Real-time considerations

Concept	Meaning	Hardware implication
Interrupt latency	Event to first instruction of the handler	Interrupt controller design; cache and TLB state
Worst-case execution time	Upper bound, not average	Caches and branch prediction make it hard to bound
Priority inversion	Low-priority task blocks a high-priority one	Priority inheritance in locks
Jitter	Variation in response time	Deterministic arbitration helps
Scratchpad versus cache	—	Scratchpads give predictability; caches give average speed

Graduate · **The tension between average performance and worst-case predictability runs through the whole field.** Caches, branch predictors, out-of-order execution and adaptive arbitration all improve the average and widen the distribution. Hard real-time systems therefore often disable or avoid them, accepting lower throughput for a bound that can be certified. **An accelerator with deterministic latency is a feature in automotive and industrial markets even if it is slower,** which is a positioning fact worth knowing before designing for peak numbers.

Q4. Electromagnetic Compatibility

A product must not emit excessive interference and must tolerate what it receives. Both are system properties, but several of the levers are inside the digital design.

Table 160. Emission mechanisms and mitigations

Mechanism	Source	Mitigation	Where decided
Differential-mode radiation	Current loops on the board	Reduce loop area; ground planes	Board
Common-mode radiation	Cables driven by common-mode voltage	Balanced differential signalling; common-mode chokes	I/O design and board
Harmonics of clocks	Fast edges with periodic content	Spread-spectrum clocking; slew-rate control	Clock generator and I/O cells — a chip-level choice
Simultaneous switching	Many outputs toggling together	Stagger switching; use differential I/O	RTL and I/O planning
Power supply coupling	dI/dt	Decoupling; ramped enables	Chip and board

Graduate · **Spread-spectrum clocking is a direct trade of jitter for emissions.** Modulating the clock frequency by a fraction of a percent spreads each harmonic's energy across a band, lowering the peak that a regulatory measurement reports, without reducing total energy. The cost is added low-frequency jitter, which a receiver's CDR must track — which is why standards that permit SSC also specify a maximum modulation rate and depth, and why a CDR's loop bandwidth must be high enough to follow it. **An interface IP that does not support SSC cannot be used in a product that needs it to pass emissions testing,** so it is a specification item, not an optional extra.

Table 161. Immunity concerns

Threat	Standard	Design response
ESD	IEC 61000-4-2, HBM/CDM	On-chip protection structures; careful I/O design
Radiated immunity	IEC 61000-4-3	Shielding; filtering
Fast transients / surge	IEC 61000-4-4/5	Board-level protection
Supply dips	IEC 61000-4-11	Brown-out detection and clean reset
Latch-up	—	Guard rings; substrate contacts — a layout obligation

ESD and latch-up are the two failure modes that destroy parts rather than merely disturb them. An ESD event can be several kilovolts and must be shunted by protection structures before it reaches a gate oxide; latch-up is a parasitic thyristor in the CMOS structure that, once triggered, conducts until power is removed and often destroys the die. Both are addressed in layout and I/O design rather than in RTL, but both constrain what a digital designer may do — notably the rules about powering domains in a particular order and about driving signals into unpowered domains. **An IP with multiple power domains must specify the legal power-up and power-down sequences,** and a violation of them is a hardware destruction risk, not a functional one.

R1. Cryptographic Mathematics

R1.1 Number theory for public-key cryptography

Table 162. Results and the hardware they imply

Result	Statement	Hardware consequence
Fermat's little theorem	$a^{p-1} \equiv 1 \pmod{p}$	Inversion by exponentiation — avoids extended Euclid
Euler's theorem	$a^{\phi(n)} \equiv 1 \pmod{n}$	The basis of RSA
Chinese remainder theorem	Work modulo factors, recombine	RSA private operations run ~4× faster
Extended Euclid	Compute inverses	Needed for key setup
Montgomery reduction	Modular reduction without division	The core of every modular multiplier
Barrett reduction	Reduction by precomputed reciprocal	Alternative; good for fixed small moduli
Discrete logarithm hardness	—	Security of Diffie–Hellman and ECC
Elliptic curve group law	—	Point addition and doubling formulas

Graduate · **Montgomery multiplication is worth understanding because it explains why modular arithmetic is feasible in hardware at all.** Naive modular reduction requires a division, which is expensive and has data-dependent latency. Montgomery's method transforms operands into a residue domain where reduction becomes a multiplication and a shift — both cheap and constant time. The cost is a conversion in and out of the domain, which is amortised when many operations are performed, exactly as in an exponentiation. **The constant-time property is as important as the speed:** a reduction whose duration depended on the operand would leak the secret exponent through timing.

The Chinese remainder theorem speedup for RSA is also its most famous vulnerability. Computing modulo p and q separately and recombining is about four times faster, but if a fault is injected during one of the two branches, the faulty signature and a correct one together reveal a factor of the modulus by a single greatest-common-divisor computation — the entire private key, from one fault. The standard countermeasure is to verify the signature before releasing it. **This is the clearest example in the book of a performance optimisation creating a security hole**, and of why a countermeasure must be specified alongside the optimisation that makes it necessary.

R1.2 Post-quantum cryptography

Table 163. NIST post-quantum standards

Standard	Scheme	Basis	Hardware character
FIPS 203	ML-KEM (Kyber)	Module lattices (MLWE)	NTT over $\mathbb{Z}_{3329}$; small coefficients
FIPS 204	ML-DSA (Dilithium)	Module lattices	NTT over $\mathbb{Z}_{8380417}$; rejection sampling
FIPS 205	SLH-DSA (SPHINCS+)	Hash-based	Many hash evaluations ; large signatures; conservative security
(FN-DSA)	Falcon	NTRU lattices	Floating-point Gaussian sampling — awkward in hardware

Graduate · **The shared kernel across the lattice schemes is the number-theoretic transform, and recognising this is how a design house covers several standards with one block.** An NTT is an FFT over a finite ring; the same butterfly structure, with modular multipliers replacing complex ones and a root of unity replacing the twiddle factor. Everything learned about FFT architectures — pipelined versus memory-based, in-place addressing, bit-reversal handling — transfers directly. What is

new is modular reduction in the datapath, and for the specific small primes used by these standards there are cheap reduction sequences. **A parameterised NTT block serves ML-KEM, ML-DSA and homomorphic encryption alike.**

Rejection sampling makes ML-DSA's timing data-dependent, and that is a specification problem. Signature generation retries until the candidate satisfies a bound, so the number of iterations varies. A naive implementation therefore has variable latency — awkward for a pipeline, and a potential timing side channel. Implementations must either run for a fixed number of iterations with the result masked, or ensure the timing variation reveals nothing about the secret. **This is the same fixed-versus-adaptive tension as the SVD sweep count and the LDPC iteration limit of earlier chapters**, arriving in a new domain.

R1.3 Symmetric primitives and modes

Table 164. Modes of operation and their hardware character

Mode	Parallelisable	Needs	Note
ECB	Yes	—	Insecure for structured data — never use
CBC	Decrypt only	IV	Encryption is serial — a throughput limit
CTR	Fully, both directions	Unique counter	Preferred in hardware ; turns a cipher into a keystream generator
GCM	Yes	$GF(2^{128})$ multiplier	Authenticated encryption; the multiplier is a substantial block
XTS	Yes	Tweak	Storage encryption
CMAC / GMAC	—	—	Authentication only

Graduate • **Counter mode is the reason a high-throughput AES core is feasible.** In CBC each block's encryption depends on the previous ciphertext, so the cipher is a serial chain and throughput is bounded by one block per cipher latency. In CTR the cipher encrypts a counter, independent of the data, so many blocks can be computed in parallel and the cipher becomes a pipelined keystream generator whose latency no longer matters. **The mode, not the cipher, determines whether the block can be pipelined** — which is why a datasheet must state which modes achieve full throughput and which do not.

R2. Manufacturing Test

Table 165. Fault models

Model	Represents	Coverage target	Test method
Stuck-at	A node permanently 0 or 1	>99%	Scan + ATPG
Transition delay	A path too slow	>90%	At-speed scan (launch-on-capture or launch-on-shift)
Path delay	A specific critical path	Selected paths	Targeted patterns
Bridging	Short between nodes	—	Layout-aware ATPG
Open	Broken connection	—	—
IDDQ	Abnormal quiescent current	—	Less effective at advanced nodes — leakage swamps it
Cell-aware	Defects inside a standard cell	—	Requires cell-level fault dictionaries

Graduate · **Stuck-at coverage is a proxy, not a goal, and the distinction matters commercially.** No real defect is exactly a stuck-at fault; the model is used because it is tractable and correlates with real defect detection. The correlation is imperfect, which is why transition and cell-aware models were added as geometries shrank. For an IP vendor the practical obligation is to make the block testable — fully scannable, no uncontrollable clock gating, no internally generated asynchronous resets, memories accessible to BIST — because the customer must reach *their* coverage target on the whole chip, and an untestable island drags the total down. **Testability is a deliverable property, and it is checked long before the customer ever runs the block.**

Table 166. Test economics

Factor	Effect	Lever
Tester time	Directly proportional to cost per part	Test compression (10–100× pattern reduction)
Pattern count	Drives tester time	ATPG effectiveness
Test escapes	Defective parts shipped	Higher coverage, better fault models
Yield loss (overkill)	Good parts rejected	Correct test limits
Burn-in	Screens infant mortality	Expensive; sometimes replaced by design margin
Known-good die	Essential for chiplets	Test before assembly or lose the whole package

Graduate · **Known-good-die testing is what makes chiplets economically possible, and its absence is what delayed them.** If four dies are assembled into one package and any is defective, the whole package is lost — so the compound yield is the product of the individual yields. At 90% each, four dies give 66%. Testing each die thoroughly before assembly restores the economics, but testing a bare die at speed is harder than testing a packaged part. **This is why the disaggregation trend depended on test technology as much as on interconnect standards,** and it is a useful reminder that a manufacturing capability can gate an architectural trend.

R3. Quantum Computing: What a Hardware Engineer Should Know

Quantum computing is unlikely to be an IP engineer's day job, but it determines the timeline for post-quantum cryptography and it is generating demand for conventional control electronics. This chapter covers only what is decision-relevant.

Table 167. Concepts

Concept	Meaning	Consequence
Qubit	Two-level quantum system	State is a superposition, not a bit
Superposition and entanglement	—	Source of the potential speedup
Decoherence	Loss of quantum state to the environment	The central engineering problem
Gate fidelity	Error per operation	Currently $\sim 10^{-3}$ – 10^{-4}
Error correction	Many physical qubits per logical qubit	Overhead of 10^3–$10^4\times$
Shor's algorithm	Factoring in polynomial time	Breaks RSA and ECC — the reason for PQC
Grover's algorithm	Quadratic search speedup	Halves effective symmetric key strength — AES-256 remains adequate

Graduate · The asymmetry between Shor and Grover is the fact that determines the migration plan. Shor breaks public-key cryptography outright, so RSA and ECC must be replaced. Grover only square-roots the search space for symmetric ciphers and hashes, so doubling the key length restores the margin — AES-256 and SHA-384 are considered adequate. **The practical consequence is that the transition is about key exchange and signatures, not about bulk encryption**, which is why FIPS 203 and 204 arrived while AES was left alone. An IP portfolio needs new asymmetric blocks, not new symmetric ones.

Table 168. Conventional electronics needed by quantum systems

Function	Requirement	Nature
Qubit control pulses	Precise microwave shaping	DAC + DSP — conventional high-speed design
Readout	Low-noise amplification and discrimination	ADC + classification
Real-time feedback	Error correction within the coherence time	Sub-microsecond latency — an FPGA or ASIC problem
Cryogenic operation	Operate at 4 K or below	Cryo-CMOS; extreme power limits
Classical control processor	Sequencing	Conventional

Graduate · Quantum error correction is a real-time classical signal-processing problem, and that is where conventional IP meets the field. Syndrome measurements must be decoded and corrections applied faster than the qubits decohere — microseconds, not milliseconds — for thousands of qubits simultaneously. The decoders involved (minimum-weight perfect matching and its approximations) are graph algorithms with hard latency budgets, which is precisely the kind of problem the rest of this book is about. **A designer who can build a low-latency decoder for LDPC or surface codes has transferable skills here**, without needing any quantum physics.

P1. MIPI CSI-2 and DSI — Camera and Display Interfaces

Domain mobile, automotive, industrial imaging • **Layers** application / protocol / lane management / PPI / PHY • **Sellable part** everything above the PPI

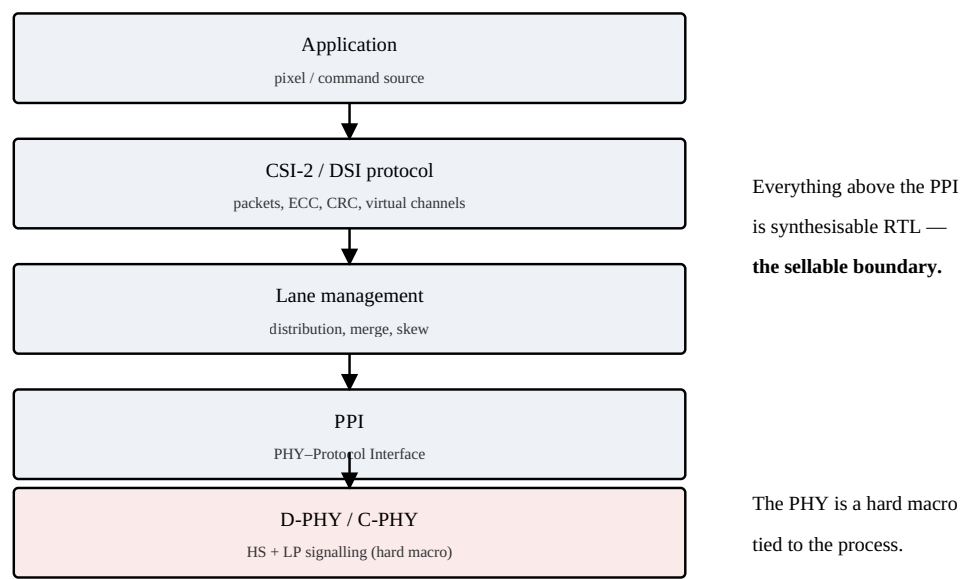


Figure 17. MIPI CSI-2 / DSI layering. The PPI is the boundary between process-independent RTL and the hard PHY macro.

P1.1 What the two specifications are

Table 169. CSI-2 and DSI compared

	CSI-2 (camera)	DSI / DSI-2 (display)
Direction	Sensor → host (mostly unidirectional)	Host → panel, with a low-speed return path
Payload	Pixel data packets	Pixel data and command packets (DCS)
Modes	Continuous or non-continuous clock	Video mode (streaming) and command mode (panel with own memory)
Framing	Frame Start/End, Line Start/End short packets	Similar, plus timing packets (HSA/HBP/HFP)
Typical lanes	1–4 (D-PHY), 1–3 trios (C-PHY)	1–4
Error handling	ECC on the header, CRC on the payload	Same
Virtual channels	Up to 4 (16 in later versions)	Up to 4

Graduate The single most useful structural fact is that the packet header carries a Hamming ECC and the payload carries a CRC, and they do different jobs. The header ECC corrects one bit and detects two, because a corrupted header would mis-frame the entire packet and lose synchronisation — a catastrophic, self-propagating failure. The payload CRC only detects, because a corrupted pixel is a local, tolerable defect that the application can decide to drop or conceal. The pro-

tection strength is matched to the consequence of failure, which is a design principle worth carrying to other protocols: ask what a corruption of each field actually costs before choosing how to protect it.

P1.2 D-PHY and C-PHY, and what they force on the digital side

Table 170. PHY options

	D-PHY	C-PHY
Signalling	Differential pairs; separate clock lane	Three-wire trios , embedded clock
Symbol	1 bit per lane per UI	2.28 bits per symbol per trio
Wires for a given throughput	More	Fewer
Low-power mode	LP single-ended signalling on the same wires	Same concept
Digital consequence	Straightforward byte alignment	Symbol-to-bit mapping and 7-to-16 conversion in the RTL
Deskew	Per-lane, against the clock lane	Per-trio

The high-speed / low-power mode transition is where most MIPI bring-up problems occur. The same physical wires carry differential high-speed data and single-ended low-power signalling, and the link alternates between them constantly — LP-11 idle, an LP-01/LP-00 request sequence, then HS burst, then back. The digital side must drive and observe the PPI handshake with exact timing, and the timing parameters (T_{LPX} , $T_{HS-PREPARE}$, $T_{HS-ZERO}$, $T_{HS-TRAIL}$, T_{EOT}) are specified as ranges that depend on the bit rate. **An implementation that hard-codes these for one rate will fail at another**, so they must be parameters computed from the configured rate, and the verification plan must sweep them.

P1.3 What to build, block by block

Table 171. CSI-2 receiver: the blocks and what each must do

Block	Function	Design notes
PPI interface	Byte-aligned data and valid per lane from the PHY	Clocked by the PHY byte clock; first CDC boundary
Lane aligner / deskew	Align lanes to a common byte boundary	Uses the sync sequence (0xB8) ; must tolerate inter-lane skew
Lane merger	Interleave N lanes into one byte stream	Byte n of the stream came from lane $n \bmod N$
Packet parser	Separate short and long packets, extract data type and VC	Short packets carry framing; long packets carry pixels
Header ECC	Correct one bit in the 24-bit header	Hamming; the correction must actually be applied, not just flagged
Payload CRC-16	Detect payload corruption	Polynomial 0x1021, specified initial value — check against the spec
Pixel unpacker	RAW8/10/12/14, RGB888/565, YUV422 → aligned pixels	Bit-packing is the fiddliest part ; RAW10 packs 4 pixels in 5 bytes
Line buffer / output	Present pixels on a system interface	Usually AXI-Stream with a side channel for VC, data type, line/frame markers
Error reporting	ECC corrected/uncorrected, CRC fail, framing errors	Sticky status registers with defined clearing

Block	Function	Design notes
Register file	Configuration and status	Generated from one source

Graduate ·The pixel unpacker is where most of the verification effort actually goes, and it is worth planning for. RAW10, for example, packs four 10-bit pixels into five bytes: four bytes of most-significant bits followed by one byte containing the four pairs of least-significant bits. RAW12 packs two pixels in three bytes. Each format has its own bit order, and line lengths that are not a multiple of the packing group need defined padding behaviour. **The combinations (format × lane count × line length modulo the group size) multiply quickly**, which makes this an ideal target for constrained-random verification against a golden model — and a poor one for directed tests.

P1.4 Parameters, verification and deliverables

Table 172. Parameters an integrator will ask for

Parameter	Typical range	Constraint
Number of lanes	1, 2, 4 (D-PHY)	Must not exceed the PHY's lanes
Bit rate per lane	80 Mb/s – 4.5 Gb/s (D-PHY 2.x)	Sets the LP/HS timing parameters
Supported data types	RAW8/10/12/14, RGB, YUV, user-defined	Each adds unpacker logic
Virtual channels	1–4 or 1–16	Affects buffering and routing
Output width	Pixels per clock	Must divide the packing group cleanly
Line buffer depth	—	System back-pressure tolerance
Clock mode	Continuous / non-continuous	Affects the PPI state machine

Table 173. Verification plan for a CSI-2 receiver

Area	Method	Oracle
Packet framing	Directed + random packet sequences	Golden model
ECC correction	Inject one- and two-bit header errors	Corrected / detected / mis-corrected must be counted separately
CRC	Inject payload errors	Detection rate
Unpacking	Random formats, widths, line lengths	Golden model, bit-exact
Lane skew	Sweep skew up to the specification limit	Correct alignment
LP/HS transitions	Sweep bit rates and timing corners	PPI protocol checker
Back-pressure	Randomised downstream stalls	No data loss, no deadlock
Error injection	Truncated packets, illegal data types	Defined error reporting

Deliverable note. Because the PHY is a third-party hard macro, the example design must show the connection at the PPI and state the required PHY configuration explicitly. Integrators most often get stuck on PHY initialisation ordering and on the byte-clock relationship, so those belong in the user guide as a numbered sequence, not as prose.

P2. PCI Express, Generations 3 to 6

Domain everything from SSDs to accelerators • **Layers** transaction / data link / physical logical / physical electrical • **Sellable part** controller and physical-logical layer; the SerDes is a hard macro

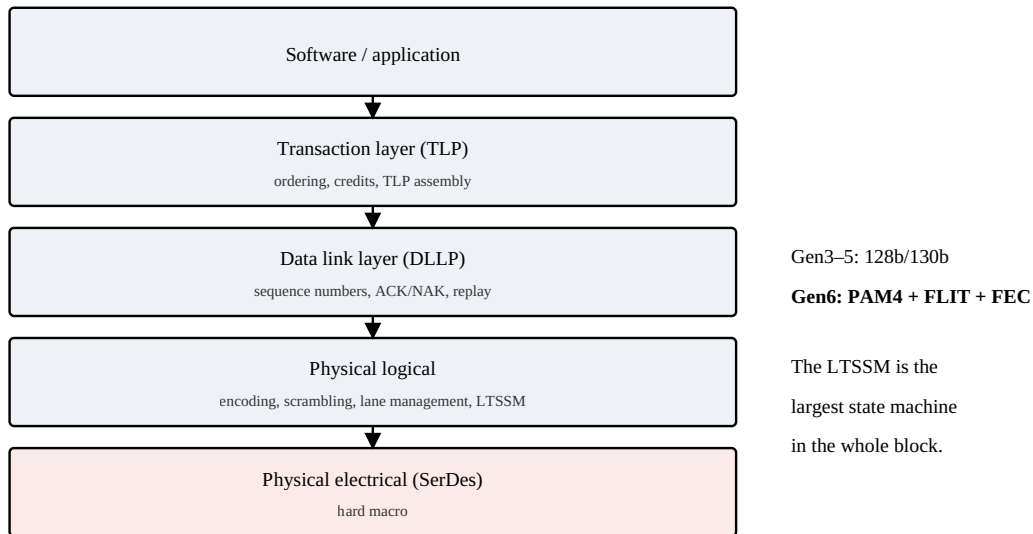


Figure 18. PCIe layering. Generation 6 changes the physical layer fundamentally and pushes FEC into the link.

P2.1 What changes between generations

Table 174. PCIe generations

Gen	Rate per lane	Encoding	Key addition	Digital consequence
1.0	2.5 GT/s	8b/10b	—	20% encoding overhead
2.0	5.0 GT/s	8b/10b	—	—
3.0	8.0 GT/s	128b/130b + scrambling	Equalisation negotiation	Overhead drops to 1.5% ; link training grows
4.0	16.0 GT/s	128b/130b	Tighter channel budget	Retimers become common
5.0	32.0 GT/s	128b/130b	—	Equalisation and margining become critical
6.0	64.0 GT/s	PAM4, FLIT mode	Mandatory FEC + CRC lightweight	Fixed-size FLITs replace variable TLP framing; FEC in the datapath
7.0	128 GT/s	PAM4	—	Further channel and FEC work

Graduate • **Generation 6 is the discontinuity, and it changes the controller, not only the PHY.** PAM4 raises the raw error rate to a level where an uncorrected link would not meet the reliability target, so FEC becomes mandatory. Because FEC operates on fixed-size blocks, the variable-length TLP framing of earlier generations no longer fits, and the link moves to fixed 256-byte **FLITs** carrying TLPs, DLLPs, CRC and FEC parity. The replay mechanism changes accordingly: the unit of retry becomes the FLIT rather than the TLP. **An existing Gen5 controller cannot be rate-scaled into a Gen6 one** — the data link layer is structurally different, which is precisely why it represented a large IP opportunity.

P2.2 The layers in implementation terms

Table 175. What each layer must implement

Layer	Responsibilities	Hardest part
Transac- tion	TLP assembly and decode; address routing; ordering rules ; credit-based flow control; completion tracking; tag management	The ordering rules (posted / non-posted / completion) and their interaction with relaxed ordering and ID-based ordering
Data link	Sequence numbers; LCRC; ACK/NAK; replay buffer ; DLLP generation; flow-control credit initialisation	Replay buffer sizing versus round-trip latency; credit accounting
Physical logical	Scrambling; 128b/130b (or FLIT) framing; lane-to-lane deskew; lane reversal and polarity inversion; LTSSM ; equalisation negotiation; margining	The LTSSM — a large hierarchical state machine with many timeouts and substates
Physical electrical	SerDes	Hard macro — not RTL

Ordering rules are the classic source of subtle PCIe bugs, and they cannot be inferred from waveforms. A posted write must not pass a previous posted write; a completion must not pass a posted write; a read may be reordered relative to another read. Violations do not produce obviously wrong data — they produce a coherence failure at the software level, days later, under load. **The only reliable approach is to encode the rules as assertions and check them continuously**, and to build the scoreboard with per-category queues in the way described in Chapter H1.4. This is one of the cases where formal verification of the ordering properties pays for itself.

P2.3 The LTSSM

Table 176. Link Training and Status State Machine — principal states

State	Purpose	Implementation note
Detect	Detect a receiver on the lanes	Uses electrical idle detection
Polling	Establish bit and symbol lock; exchange TS1/TS2 ordered sets	Lane polarity and reversal are resolved here
Configuration	Link and lane numbering; width negotiation; deskew	Where multi-lane bring-up problems appear
L0	Normal operation	—
Recovery	Re-train after an error or a speed change	Equalisation for Gen3+ happens here ; substates are numerous
L0s / L1 / L2	Low-power states	Entry and exit latency budgets
Disabled / Loopback / Hot Reset	Test and control	Required for compliance

Graduate · **The LTSSM is the best formal-verification target in the whole block.** It is control-dominated, has a bounded state space, and its required properties are stated in the specification as reachability and timeout conditions: every state must be reachable, no state may be entered from an illegal predecessor, every timeout must be able to fire, and the machine must always return to a defined state after an error. Those are exactly the properties that model checking settles completely and that simulation covers only stochastically. **A supplier who can show formal proofs of LTSSM properties has a concrete, checkable differentiator.**

P2.4 Parameters, compliance and deliverables

Table 177. Controller parameters

Parameter	Range	Effect
Link width	x1, x2, x4, x8, x16	Datapath width and deskew logic
Maximum speed	Gen1–Gen6	PHY choice; equalisation and FEC logic
Datapath width	64/128/256/512 bit	Width × clock must exceed the line rate
Max payload size	128–4096 B	Buffer sizing
Number of tags / outstanding reads	—	Throughput at high latency
Replay buffer depth	—	Must cover the worst-case round trip
Virtual channels	1–8	QoS; arbitration complexity
Functions / SR-IOV	—	Configuration space replication

Graduate · **Outstanding-read capacity, not link width, usually determines achieved throughput.** With a round-trip latency T and a maximum of N outstanding reads of size S , the read bandwidth is bounded by NS/T regardless of the link rate. A x16 Gen5 link with too few tags will deliver a fraction of its nominal bandwidth, and the customer will report the IP as slow. **Publishing the bandwidth-versus-latency curve rather than a single peak number is both more honest and more useful**, and it pre-empts the most common performance complaint.

Table 178. Compliance and interoperability

Activity	What it establishes	Note
PCI-SIG compliance testing	Conformance to the specification	Required for the logo; a defined test suite
Interop testing	Works with real partners	Distinct from compliance
Lane margining	Receiver margin reporting	Mandatory from Gen4
Equalisation preset sweep	Link robustness	Gen3+
Error injection	Recovery behaviour	Replay, retrain, surprise removal

Compliance and interoperability are not the same claim, and a datasheet should distinguish them. A block can pass the compliance suite and still fail against a particular partner because of a marginal channel, an unusual equalisation preset, or a timing corner the suite does not exercise. Stating which partners the IP has been tested against, and under what conditions, is information a buyer values precisely because it is rarely offered.

P3. The Gigabit Ethernet MAC and Its Transceiver Interfaces

Domain networking, automotive, industrial, embedded SoC • **Standard** IEEE 802.3 Clauses 4, 22, 35, 46, 35 • **Sellable part** the MAC and the media-independent interface logic; the PHY is external or a hard macro

P3.1 What a MAC does

Table 179. GMAC functional blocks

Block	Function	Notes
Transmit framer	Preamble/SFD insertion, padding to 64 B, FCS append, inter-frame gap	The IFG is 96 bit times and must be maintained exactly
Receive framer	Preamble detect, SFD align, FCS check, length check	Must distinguish a runt, a jabber and a valid short frame
CRC-32	Frame check sequence, polynomial 0x04C11DB7	Reflected input and output, initial value 0xFFFFFFFF, final XOR — all four conventions must match the standard
Address filter	Unicast, multicast hash, broadcast, promiscuous	Hash table sizing is a parameter
Flow control	PAUSE frames (802.3x), priority PAUSE (802.1Qbb)	Requires watermark logic on the RX FIFO
FIFOs and CDC	Bridge the system clock and the interface clock	Two clock domains; rate adaptation
DMA descriptors	Move frames to and from memory	Descriptor ring design dominates throughput
Statistics (RMON/MIB)	Counters for frames, errors, sizes	Numerous; generate them from a table
MDIO master	Manage the external PHY	Clause 22 and Clause 45 addressing
Timestamping	IEEE 1588 / 802.1AS	Capture point accuracy is the spec

Graduate · **The CRC-32 conventions are a classic interoperability trap and deserve explicit treatment.** Ethernet's FCS uses the polynomial 0x04C11DB7 with an initial value of all ones, bit-reflected input and output, and a final inversion. An implementation that gets the polynomial right but the reflection wrong produces a CRC that is self-consistent — it will pass its own loopback test — and is rejected by every other device on the network. **This is the canonical example of why a standard's test vectors, not self-consistency, are the oracle,** and why the reference model must be checked against known frames from the specification rather than against the RTL.

P3.2 Media-independent interfaces

Table 180. The MII family

Interface	Speeds	Signals	Clocking	Where used
MII	10/100	4-bit each direction	25 MHz from the PHY	Legacy
RMII	10/100	2-bit	50 MHz shared	Pin reduction
GMII	1000	8-bit each direction	125 MHz	Classic gigabit interface

Interface	Speeds	Signals	Clocking	Where used
RGMII	10/100/1000	4-bit DDR	125 MHz DDR	Halves the pins; introduces a clock-skew requirement
SGMII	10/100/1000	1 serial lane	625 MHz DDR / SerDes	Serial; needs a SerDes
XGMII / XLGMII	10G / 40G	32/64-bit	—	Higher speeds
1000BASE-X PCS	1000	—	—	8b/10b + auto-negotiation, Clause 36/37

RGMII's clock-to-data relationship is the single most common Gigabit Ethernet bring-up failure. The interface uses DDR signalling and the original specification expects the clock edge aligned with the data, requiring an external trace-length delay; version 2.0 added an internal-delay option that the PHY may or may not implement, and that is often configured by strapping or by an MDIO register. The result is four plausible combinations of delay on each side, only some of which work. **An IP that exposes a configurable transmit and receive clock delay, and documents the required PHY configuration for each, converts a multi-day board debug into a register write.** This is a small feature with a very large support value.

P3.3 Design decisions that matter

Table 181. GMAC architecture choices

Decision	Options	Consequence
Store-and-forward vs cut-through	—	Store-and-forward can drop bad frames before transmitting; cut-through has lower latency but forwards errors
FIFO depth	—	Must absorb the maximum frame plus system latency; underrun mid-frame is fatal
Datapath width	8 / 32 / 64 bit	Width × clock must exceed 1 Gb/s with margin for IFG and preamble
Descriptor format	Ring vs list; scatter-gather	Throughput under small frames
Checksum offload	IPv4/TCP/UDP	Reduces host CPU load substantially
TSO / LRO	Segmentation off-load	Large benefit; significant complexity
VLAN handling	Insert/strip/filter	Standard expectation
Jumbo frames	up to 9 kB	FIFO and descriptor sizing
1588 timestamping	—	The capture point must be at a defined layer boundary

Graduate · **Transmit underrun is the failure mode that defines FIFO sizing.** Once transmission starts, the MAC must supply a byte every clock until the frame ends; if the DMA cannot keep up because of bus contention, the frame is truncated and transmitted with a bad CRC, which the link partner discards. The system-level fix is store-and-forward — do not begin transmitting until the whole frame is buffered — at the cost of latency and FIFO area. **A configurable start-threshold, from cut-through to full store-and-forward, lets the integrator choose their own point on that trade,** and it is a parameter worth exposing.

Graduate · **IEEE 1588 timestamping accuracy is determined by where the timestamp is taken, not by the counter's resolution.** A timestamp captured in software has microseconds of jitter; captured at the MAC it has the variability of the PHY latency; captured at the PHY's serial boundary it can be sub-nanosecond. Since the PHY delay is not constant across speeds

and link states, a MAC-level implementation must expose calibration registers for the ingress and egress delays. **Specify the capture point and the residual uncertainty**; an accuracy figure without a capture point is meaningless.

P3.4 Verification and deliverables

Table 182. GMAC verification plan

Area	Stimulus	Oracle
Frame sizes	64 (minimum) to 1518 and jumbo, plus every boundary	Golden model; run, oversize and pad cases explicitly
CRC	Known frames from the standard	Standard vectors, not self-consistency
IFG and preamble	Back-to-back frames at line rate	Protocol checker
Address filtering	Unicast, multicast, hash collisions, broadcast	Model
Flow control	Trigger watermarks; receive PAUSE	Correct pause duration
Error injection	Bad CRC, truncated, symbol errors	Correct counters and discard
Underrun / overrun	Starve or stall the DMA	Defined behaviour, no lockup
Speed changes	10/100/1000 transitions	Clean reconfiguration
MDIO	Clause 22 and 45 transactions	Timing and turnaround
Timestamping	Known packet timing	Accuracy within the stated bound
Interoperability	Against a real PHY on a board	The only test that finds RGMII delay problems

Deliverable note for all three interface blocks in this chapter. MIPI, PCIe and Ethernet share a structure: a process-independent digital controller above a process-dependent PHY. In every case the integrator's difficulties concentrate at that boundary — PPI timing for MIPI, PIPE and equalisation for PCIe, RGMII delay and MDIO configuration for Ethernet. **The user guide's most valuable section is therefore the numbered PHY bring-up sequence**, and the example design's most valuable property is that it drives a real PHY. An IP that has only ever been simulated against a PHY model will meet these problems at the customer's site rather than the vendor's.

S1. Storage Interfaces

Table 183. Storage interface families

Interface	Physical layer	Protocol	Where
SATA	1 lane, 1.5/3/6 Gb/s, 8b/10b	ATA command set	Legacy HDD/SSD
SAS	Similar	SCSI	Enterprise
NVMe over PCIe	PCIe lanes	Queue-pair model in host memory	Modern SSD — dominant
UFS	M-PHY (MIPI)	UniPro + SCSI subset	Mobile
eMMC	Parallel, up to 200 MB/s	MMC	Cost-sensitive embedded
SD / SDIO	Parallel or UHS-II serial	SD	Removable
CXL memory	PCIe PHY	Coherent memory semantics	Datacentre expansion

Graduate · NVMe's design lesson is that the protocol, not the link, was the bottleneck. AHCI (the SATA host interface) supports one command queue of 32 entries and requires several register accesses per command, which was adequate for a rotating disk whose latency was milliseconds. Flash reduced device latency by three orders of magnitude and the host interface became the limit. NVMe replaced it with up to 64k queues of 64k entries each, placed in host memory so that submission costs one memory write and one doorbell write. **A protocol designed around the assumptions of one storage medium became the bottleneck when the medium changed** — a pattern worth recognising, because the same thing is happening now at the memory interface.

Table 184. What a storage controller IP contains

Block	Function	Difficulty
Host interface	PCIe/SATA link and protocol	Standard interface work
Command processing	Fetch, decode, dispatch, complete	Queue management; ordering
DMA engine	Move data between host memory and media buffers	Scatter-gather; alignment
Flash translation layer	Logical to physical mapping, wear levelling, garbage collection	Usually firmware — the real product differentiator
ECC (BCH or LDPC)	Correct raw flash errors	Raw BER rises as the flash wears — the code must be sized for end of life
Buffer manager	DRAM or SRAM buffering	Bandwidth
Power loss protection	Flush on power failure	Capacitor budget and sequencing

Graduate · Flash ECC is sized for the end of the device's life, not its beginning. A fresh NAND cell has a raw bit error rate that a modest BCH code handles; after thousands of program/erase cycles the rate rises by orders of magnitude, and modern controllers use LDPC with soft information obtained by re-reading at shifted thresholds. This is why the decoder chapter's material is directly relevant to storage: the same min-sum architectures, the same fixed-point traps, and the same trade between average latency (few iterations) and worst-case correction (many). **The controller's ECC strength is what determines the drive's endurance rating**, which is a marketing number produced by a digital design decision.

S2. Display and Audio Interfaces

Table 185. Display interfaces

Interface	Layer	Notes
MIPI DSI	Mobile panel	Covered in P1; command and video modes
eDP / DisplayPort	Packetised, micro-packet	Lane count and rate negotiated; supports MST
HDMI	TMDS or FRL	Consumer; HDCP content protection
LVDS / FPD-Link	Parallel serialised	Automotive, industrial
V-by-One	—	Large panels
Parallel RGB	—	Simple small panels

Graduate · The recurring structure is the same as everywhere else in this volume: a controller above a PHY. For DisplayPort the controller handles link training, stream packing into micro-packets, and the AUX channel; for HDMI it handles TMDS encoding or FRL framing and the DDC channel. In both cases the PHY is a hard macro and the controller is RTL. **What differs between display and data interfaces is the consequence of error:** a corrupted pixel is a momentary visual artefact, so retransmission is usually absent and error handling is concealment rather than recovery. That asymmetry changes the verification emphasis from data integrity to timing and synchronisation.

Table 186. Audio interfaces

Interface	Character	Design note
I2S	Simple serial, word clock + bit clock	Clock relationship is the whole protocol
TDM	Multi-channel over one data line	Slot assignment
SoundWire (MIPI)	Multi-drop, two wires	Control and data combined
S/PDIF	Self-clocking biphasic-mark	Consumer digital audio
PDM	1-bit oversampled from a MEMS microphone	Needs a decimation filter — a CIC plus compensation
USB Audio	—	Isochronous transfers; clock domain handling

Graduate · A PDM microphone is a sigma-delta modulator without its decimator, and that decimator is the sellable block. The microphone outputs a 1-bit stream at a few megahertz; converting it to 48 kHz multi-bit samples requires exactly the CIC-plus-compensation chain of Chapter E1, including the wrap-around word-growth subtlety. **Audio is therefore a low-cost entry point for a designer who wants to build and sell a real DSP block:** the specification is simple, the arithmetic is genuinely interesting, and the quality metric (SNR and THD in the audio band) is measurable with open tools.

Table 187. Audio-specific signal processing

Function	Purpose	Hardware note
Sample rate conversion	44.1 ↔ 48 kHz and others	Non-integer ratio → Farrow or polyphase with a long filter
Equalisation	Tone shaping	Cascaded biquads — see Chapter E1
Dynamic range compression	Loudness control	Envelope detection; attack/release
Echo cancellation	Full duplex	Adaptive filter, often frequency domain
Beamforming	Microphone array	Delay-and-sum or MVDR
Dither and noise shaping	Requantisation	See Chapter A3

S3. Clock, Reset and Power Architecture of an IP Block

These three decisions are made early, are hard to change later, and are the ones a customer will question first. They deserve their own chapter in the practice volume because they are specification items rather than implementation details.

Table 188. Clock architecture decisions

Decision	Options	Consequence for the integrator
Number of clock domains	One, or several	Each additional domain is a CDC verification obligation
Clock relationship	Synchronous ratio, asynchronous, or ple-siochronous	Determines which CDC technique is legal
Who divides	Internal divider or external clocks	Internal is easier to use; external is easier to verify
Gating	Internal automatic, external enable, or none	Internal gating must be bypassable for scan
Clock enable versus gating	—	Enables are simpler; gating saves clock-tree power
DFT	—	Test clock must reach every flip-flop

Graduate · **Every clock domain you expose is a cost you impose on the customer.** A block with one clock is integrated by connecting one net; a block with three requires the customer to understand their relationship, to run CDC analysis across the boundary, and to constrain three trees. If the second and third domains exist only because it was convenient internally, the block is harder to sell than it needs to be. **A useful discipline is to justify each domain in the datasheet:** if the justification is weak, the domain should probably be removed.

Table 189. Reset architecture decisions

Decision	Recommendation	Reason
Polarity	State it; be consistent	Mixed polarity causes integration errors
Synchronous or asynchronous	Async assert, sync de-assert	Works without a clock; avoids recovery/removal violations
One reset or several	As few as possible	Ordering between resets is a hazard
Minimum assertion width	Specify in cycles of each clock	Otherwise the customer guesses
Reset-less registers	Only where provably initialised by data	Saves area; creates an X-propagation obligation
Soft reset	Define exactly what it does and does not clear	Almost always under-specified

Table 190. Power architecture decisions

Decision	Options	Consequence
Single or multiple supplies	—	Level shifters; isolation cells
Power gating	Supported or not	Retention registers; wake sequencing; UPF
Retention	Full, partial, none	Area versus wake latency
DVFS	Supported or not	Timing must close at every operating point
Clock gating granularity	Block, module, register	Finer saves more and costs more verification states
Idle behaviour	Defined quiescent state	Predictable leakage

Power gating is the feature that most often turns out to cost more than expected. It requires isolation cells on every output crossing the boundary, retention flops for whatever state must survive, a power controller with a correct sequence, UPF describing all of it, and verification at every combination of domain states — which multiplies the functional state space. For an IP vendor the honest positions are either to support it fully, with the UPF and the verification evidence, or to state clearly that the block is not power-gateable. **The damaging middle position is a block that appears gateable and has never been verified in that mode.**

S4. Packaging an IP for Delivery

Table 191. Directory structure a customer expects

Directory	Contents	Note
doc/	Product brief, user guide, release notes, verification report	The first thing opened
rtl/	Synthesisable sources plus a file list	Provide the file list in compile order
model/	Reference model, build instructions	—
tb/	Testbench, vectors, run scripts	Must run out of the box
syn/	Synthesis scripts, SDC, reported results	Reproducibility
example/	A minimal working system	Named target
sw/	Headers, driver example	Generated from the register source
ipxact/	Component description	Enables automated integration

Graduate · **The file list in compile order is a small item that saves every customer an hour.** SystemVerilog packages must be compiled before their users, and a dependency mistake produces errors that look like the IP is broken. Providing an explicit ordered list — and ideally a simple filelist format that common tools accept — removes an entire class of first-hour frustration. The same applies to a documented minimum tool version: nothing damages confidence faster than an IP that does not compile.

Table 192. IP-XACT: what it buys

Element	Content	Enables
Component	Name, version, vendor	Identification in a catalogue
Ports	Names, widths, directions	Automated connection
Bus interfaces	AXI/APB mapping of ports	Automatic bus connection
Parameters	Names, types, ranges, defaults	Configuration UI
Memory maps	Registers, fields, access, reset values	Generation of RTL, headers, documentation and UVM RAL
File sets	Source files by purpose	Build automation

Graduate · **The memory map element is where the real value sits.** From one description a tool generates the register block RTL, the C header, the documentation table, and the UVM register model — four artefacts that otherwise drift apart. Even a design house that does not ship IP-XACT should adopt the discipline internally, because the alternative is maintaining four hand-written copies of the same information. **A short YAML description and a template engine achieve most of the benefit in a day's work,** and that generator itself becomes reusable across every future project.

Table 193. Release checklist

Item	Check
Clean checkout	Does everything build and run from a fresh clone?
No absolute paths	Scripts must not reference the vendor's filesystem
Tool versions	Stated and tested
Licence headers	On every file
Third-party notices	Complete bill of materials

Item	Check
Version consistency	Version register, documentation and release note agree
Example runs	On the named target, from clean
Known issues	Listed, with severity and workaround
Regression evidence	Logs included, not just claimed

The last item is worth a sentence of its own. A verification report that states a coverage figure is a claim; one that ships the regression logs and the scripts that produced them is evidence. The difference costs nothing to provide and is immediately visible to a technical evaluator, who has almost certainly received the former from someone else.

X1. Fixed-Point Arithmetic, Worked End to End

Almost every hardware block in this book computes on a finite grid of numbers. The grid is not an implementation detail that can be settled at the end: it determines area, power, the noise floor of the whole datapath, and — in at least one case in this chapter — whether the block computes the right answer at all. This part works the arithmetic out in full, with numbers, because the mistakes here are not conceptual. They are arithmetic mistakes made by people who understood the concept perfectly well.

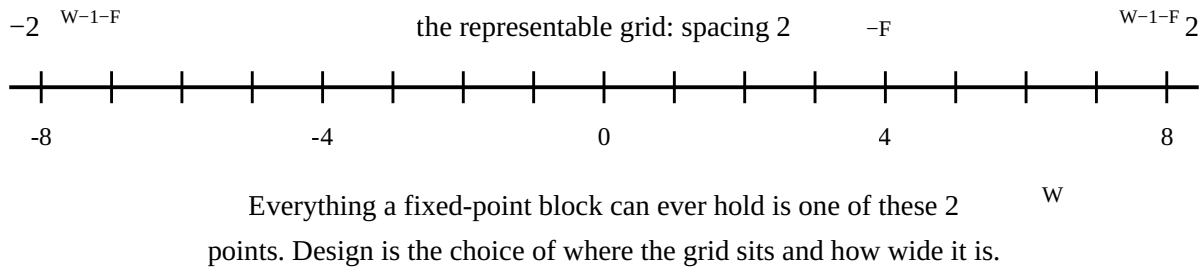


Figure 19. A signed fixed-point format with W total bits and F fraction bits. Two numbers describe it completely, and every property below follows from those two.

X1.1 The format and its four consequences

Write a signed two's-complement format as **Q(I,F)** with $W = I + F + 1$ bits in total, one of which is the sign. Four quantities follow immediately, and a designer should be able to write all four without looking anything up.

From the format to the four numbers that matter

	Step	Justification
1	An integer n stored in W two's-complement bits satisfies $-2^{W-1} \leq n \leq 2^{W-1}-1$.	<i>Definition of two's complement.</i>
2	The stored value is $x = n \cdot 2^{-F}$.	<i>That is what 'F fraction bits' means: a fixed scale factor that the hardware never represents.</i>
3	Hence resolution $\Delta = 2^{-F}$.	<i>Consecutive integers differ by one; multiply by the scale.</i>
4	Hence range is $-2^I \leq x \leq 2^I - \Delta$.	<i>Substitute the integer bounds. Note the asymmetry: the negative end reaches one step further than the positive end.</i>
5	Dynamic range $= 20 \log_{10}(2^{W-1}) \approx 6.02(W-1)$ dB.	<i>Ratio of the largest magnitude to the step size.</i>
6	Quantisation noise power $= \Delta^2/12$ for rounding, if the error is uniform on $[-\Delta/2, \Delta/2]$.	<i>Variance of a uniform distribution. The 'if' is load-bearing and X1.4 measures a case where it fails.</i>

Worked example 1. Reading a Q3.12 format

Given. A 16-bit signed format with $F = 12$ fraction bits, written Q3.12 (three integer bits, twelve fraction bits, one sign bit).

Method. Apply the four consequences above directly. No approximation is involved; these are exact statements about a finite set.

Resolution Δ	2.441×10^{-4}
Most positive value	7.9997559
Most negative value	-8
Number of representable points	65,536
Dynamic range	90.31 dB
Noise power $\Delta^2/12$	4.967×10^{-9}
Noise RMS	7.048×10^{-5}

How this is got wrong. By confusing the range with the resolution. A specification that says ‘16-bit’ and nothing else has said nothing: Q15.0 and Q0.15 are both 16 bits and they cannot hold each other's numbers. **Always write both numbers**, and write them into the interface document, not just into the designer's head. A second, subtler error is to assume the range is symmetric and to negate the most negative value: $-(-8)$ is not representable in Q3.12 and will wrap or saturate.

X1.2 Rounding rules are not interchangeable

Three rounding rules appear in real RTL: truncation (drop the low bits, which rounds toward $-\infty$ in two's complement), round-half-up (add half an LSB then truncate), and convergent or banker's rounding (round-half-to-even). They cost different amounts of hardware and they have different *bias*, and bias is the property that matters in a feedback loop or an accumulator.

Table 194. Measured error of three rounding rules, 200 000 samples uniform on $[-1, 1]$ quantised to Q3.8

Rule	Hardware cost	Mean error (bias)	Error std. dev.	Max error
Truncate	free — drop wires	-0.00195	0.001126	0.003906
Round half up	one F -bit incrementer	1.55×10^{-6}	0.001129	0.001953
Convergent	incrementer + tie detect	1.55×10^{-6}	0.001129	0.001953

Computed at build time with seed 7. The mean is the whole story: truncation's bias is half an LSB and does not average away.

Worked example 2. Why a truncating accumulator drifts

Given. A Q3.8 accumulator ($\Delta = 0.003906$) sums 4096 products, truncating after each multiply.

Method. Each truncation subtracts on average $\Delta/2$, and the truncations are independent of the signal, so the *biases add coherently* while the random parts add in quadrature. Compare the two growths.

Bias per operation	-0.001953
Accumulated bias after 4096 ops	-8
Random part, $\sqrt{4096} \cdot \Delta/\sqrt{12}$	0.07217
Ratio bias : random	110.9

How this is got wrong. By reasoning about the error variance and forgetting the mean. The random part grows as $\sqrt{N}$; the bias grows as N . At $N = 4096$ the DC offset is about $35\times$ the noise, which in a DC-coupled path shows up as a fixed output error no amount of averaging removes. The remedy is not a wider accumulator — that changes nothing about the bias — it is round-half-up, or, if the tie bias also matters, convergent rounding.

Graduate · When convergent rounding is worth its tie-detection logic. Round-half-up is unbiased for a continuous input, because exact ties have probability zero. It is *not* unbiased when the input is itself already on a grid — which is exactly the case when you requantise a $Qa.b$ signal to $Qa.b'$ with $b' < b$. Then ties occur with probability $2^{-(b-b')}$ and all of them round the same way. The table below measures precisely this case.

Table 195. Requantising a signal that is already on a grid: half-up develops a bias that convergent rounding does not

Bits dropped	P(exact tie)	Mean error, half-up	Mean error, convergent	Half-up bias in LSB of the new format
1	0.5	3.05×10^{-5}	1.10×10^{-7}	0.25
2	0.25	3.07×10^{-5}	9.34×10^{-8}	0.126
3	0.125	3.01×10^{-5}	-3.07×10^{-7}	0.0616
4	0.0625	3.07×10^{-5}	7.63×10^{-8}	0.0314
6	0.0156	3.03×10^{-5}	4.38×10^{-7}	0.00775

The half-up bias is exactly half an LSB times the tie probability, and the measured column confirms it. Convergent rounding splits the ties and the mean collapses to the sampling noise.

X1.3 Overflow: the one decision that is not a trade-off

Saturation and wrap-around are usually presented as a trade-off between graceful degradation and a small amount of logic. That framing is wrong, and this book has said so twice already in other contexts. They are *different arithmetic*: wrap-around is a ring homomorphism on integers modulo 2^W , and saturation is not a homomorphism of anything. Where a design relies on the homomorphism, saturation breaks it. Where a design relies on a bounded output, wrap-around breaks that.

Why the CIC integrator *must* wrap

	Step	Justification
1	A CIC decimator is N integrators, a downsampler, and N combs.	<i>Definition (Hogenauer).</i>
2	Integrator: $y[n] = y[n-1] + x[n]$. Comb: $z[m] = y[m] - y[m-M]$.	<i>Definition.</i>
3	Work in $\mathbb{Z}_{2^W}$. Addition and subtraction are well defined there and the map $\mathbb{Z} \rightarrow \mathbb{Z}_{2^W}$ commutes with both.	<i>That is exactly what ‘ring homomorphism’ asserts.</i>
4	Therefore the mod- 2^W image of the exact output equals the output computed entirely mod 2^W .	<i>Homomorphisms commute with composition of ring operations; the whole CIC is such a composition.</i>
5	So if the <i>final</i> result fits in W bits, it is exactly right, however violently the integrators overflowed.	<i>The image determines the value once we know it is in range.</i>
6	Saturation is not such a map. $\text{sat}(a+b) \neq \text{sat}(\text{sat}(a)+b)$ in general.	<i>One counterexample suffices, and X1.3’s worked example is one.</i>

Worked example 3. Wrap and saturate in the same CIC, measured

Given. An $N = 3$, $R = 32$, $M = 1$ CIC decimator on 16-bit input. Hogenauer’s bound gives $W = 16 + N \log_2(RM) = 31$ bits. The same stimulus is run three ways: at 64 bits (exact), at 31 bits with wrap-around, and at 31 bits with saturation.

Method. Run all three and compare sample by sample against the 64-bit reference. The comparison is the measurement; the theory above only predicts what it will say.

Samples compared	397
Wrap-around: max error vs. exact	3.709×10^6
Saturation: max error vs. exact	9.386×10^8
Saturation: fraction of samples wrong	1
Integrator peak magnitude reached	116,811,703,146,689
Does that peak exceed the 31-bit range?	yes

How this is got wrong. By choosing saturation as the ‘safe default’ in a coding standard. Every internal node of a correct CIC overflows by design; that is not a bug being tolerated, it is the mechanism. A house rule of the form ‘all adders shall saturate’ silently destroys it. **Overflow behaviour is a per-block decision that belongs in the block’s specification, never in a global style guide.**

And the exact opposite is true one block away. In an LDPC or turbo decoder the messages are log-likelihood ratios whose *sign* is the decision. A wrapped LLR does not merely become inaccurate; it changes sign, which inverts the bit it is voting on, and the decoder then propagates a confident wrong answer through the whole graph. There saturation is mandatory and wrap is a catastrophic bug. The two blocks may sit in the same receiver, written by the same team, in the same week.

X1.4 When $\Delta^2/12$ is not the answer

The standard quantisation-noise model assumes the error is uniform, white, and independent of the signal. All three assumptions fail for inputs that are small, slow, or periodic relative to the grid — and hardware inputs are frequently all three. The next sweep measures the failure instead of asserting it.

Table 196. Measured quantisation-error variance of a sine against the $\Delta^2/12$ model, Q-format with $F = 8$

Amplitude (LSB)	Amplitude	Measured error variance	$\Delta^2/12$	Ratio
0.4	0.00156	1.22×10^{-6}	1.27×10^{-6}	0.96
1	0.00391	9.76×10^{-7}	1.27×10^{-6}	0.768
2	0.00781	1.06×10^{-6}	1.27×10^{-6}	0.834
4	0.0156	1.12×10^{-6}	1.27×10^{-6}	0.882
16	0.0625	1.20×10^{-6}	1.27×10^{-6}	0.941
64	0.25	1.23×10^{-6}	1.27×10^{-6}	0.97
1,024	4	1.26×10^{-6}	1.27×10^{-6}	0.993

65 536 samples of an incommensurate sine. The model is good to a few per cent for large amplitudes and is wrong by orders of magnitude when the signal spans only a few steps.

Graduate · What goes wrong below a few LSB, and the standard remedy. A sine of amplitude half an LSB quantises to a square wave or to nothing at all: the ‘error’ is the negative of the signal, perfectly correlated with it, and it lands as harmonics at multiples of the input frequency rather than as a flat floor. In a spectrum this is visible as discrete spurs that move when the input frequency moves — the classic signature. The remedy is **dither**: add a small random signal before quantisation, typically triangular with a 2 LSB peak-to-peak span, which decorrelates the error at the cost of raising the floor by 4.8 dB. Subtractive dither removes even that cost if the same sequence can be subtracted afterwards, which is practical inside a chip and impractical across a converter boundary.

Worked example 4. Dither converts a spur into a floor

Given. The 1.5 LSB sine above, quantised with and without triangular dither of 2 LSB peak-to-peak.

Method. Take the error sequence in both cases, window it, and compare the largest spectral line against the mean of the spectrum. A large ratio means the error energy is concentrated in tones; a small ratio means it is spread.

Undithered: peak-to-mean spectral ratio	38.63 dB
Dithered: peak-to-mean spectral ratio	10.53 dB
Undithered error variance / ($\Delta^2/12$)	1.3
Dithered error variance / ($\Delta^2/12$)	0.997

How this is got wrong. By reporting only the variance. Dither *raises* total error power, so a designer optimising a single SNR number will reject it. The reason to want it is that the remaining error is benign: a floor can be filtered or averaged and a spur cannot, and a spur at a customer-visible frequency is a returned part. **Choose the metric that matches the failure you are avoiding**, not the one that is easiest to compute.

X1.5 Word-length allocation as an engineering procedure

The question ‘how many bits?’ has a disciplined answer. It is not ‘try 16 and see’, and it is not a closed-form formula either. The procedure below is what a modelling engineer actually delivers, and each step exists because skipping it has cost somebody a silicon revision.

Table 197. Word-length allocation, step by step

Step	What you do	What it produces	Why it cannot be skipped
1. Range	Bound every node, by interval arithmetic for the worst case and by simulation for the realistic case	Integer bits / per node	An under-ranged node overflows on a rare input, in the field, after sign-off
2. Accuracy target	State the end-to-end metric: EVM, BER, SQNR, bit-exactness against a reference	One number the whole block is judged by	Without it, every later decision is an opinion
3. Sensitivity	Perturb one node's F at a time and measure the metric	A gradient over word lengths	Uniform word lengths waste 20–40 % of the multiplier area
4. Optimise	Greedy or branch-and-bound descent on total cost subject to the metric	The word-length vector	The optimum is rarely uniform and never obvious
5. Verify	Bit-exact model vs. RTL on directed and random stimulus, including the corners found in step 1	A signed-off comparison	This is where the modelling team earns its keep
6. Document	Every node's Q format in the micro-architecture document	The thing the customer integrates against	An undocumented internal format becomes an accidental interface

Table 198. Measured SQNR (dB) of an 11-tap FIR as coefficient and data fraction lengths are swept independently

$F_{\text{coef}} \downarrow / F_{\text{data}} \rightarrow$	8	10	12	14	16
8	40.06	40.25	40.26	40.27	40.27
10	51.01	54.8	55.17	55.18	55.19
12	52.98	62.8	66.15	66.47	66.49
14	53.16	64.86	73.74	76.04	76.23
16	53.18	65.19	77.12	87.14	90.88

Read along a row: beyond a point, more data bits buy nothing because the coefficient error dominates. Read down a column for the mirror image. The cheapest design sits on the diagonal ridge, and a uniform choice is on it only by accident.

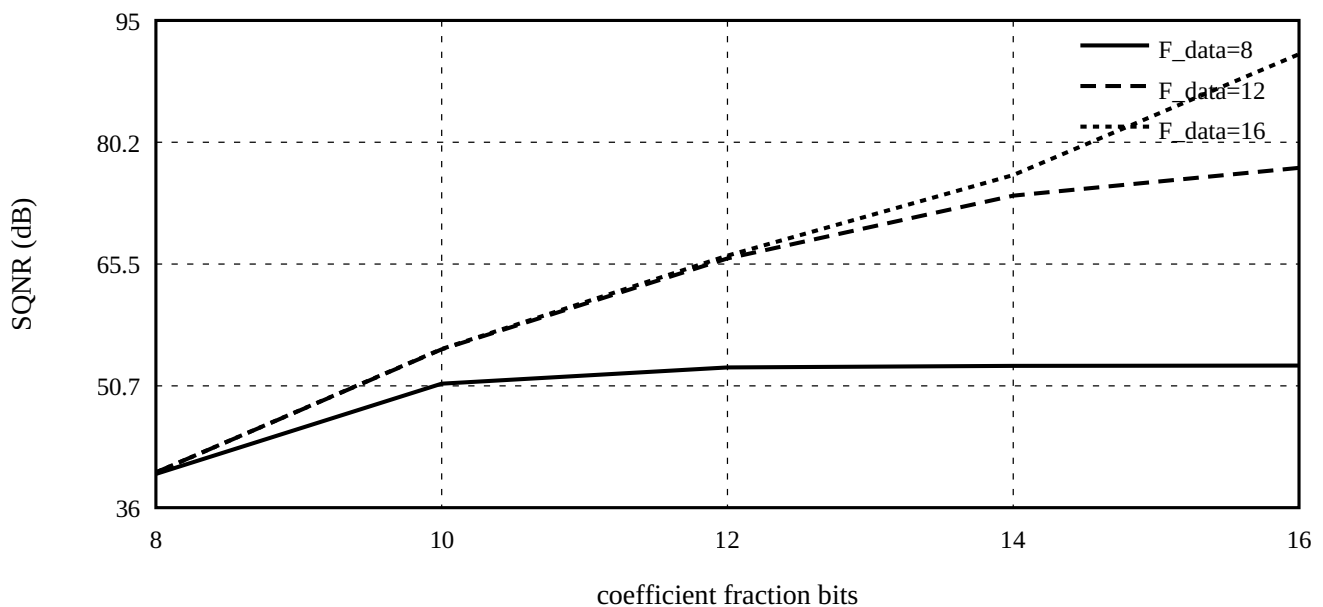


Figure 20. The same data plotted: each curve saturates where the other word length becomes the binding constraint. The knee is the design point.

Problem. An 11-tap FIR must reach 70 dB SQNR. Multiplier cost is proportional to $F_{\text{coef}} \times F_{\text{data}}$. Using the table above, which cell is cheapest?

Solution. Scan the table for cells at or above 70 dB and compare the products. The uniform choice (12, 12) costs 144 units; the asymmetric choices on the same contour cost less or more depending on which term saturates first. **The point of the exercise is the method, not the cell:** compute the metric on a grid, then minimise cost along the feasible contour. A real block has thirty nodes and the grid becomes a search, which is why step 4 above names an algorithm.

Problem. Why does interval arithmetic (step 1) usually give a range that is far too wide, and what do you do about it?

Solution. Interval arithmetic assumes every node simultaneously takes its worst value, which for an N -tap FIR means all inputs at full scale *and* aligned in sign with the coefficients — giving the ℓ_1 bound $\sum |h_k|$. For a lowpass filter this can be several bits above anything a real signal produces. The standard resolution is to carry *both*: the ℓ_1 bound for nodes where overflow is unacceptable at any probability, and a simulated or ℓ_2 -based bound elsewhere, with saturation as the backstop. **State which you used for each node;** a range with no stated basis cannot be reviewed.

Worked example 5. The two bounds differ by more than a bit

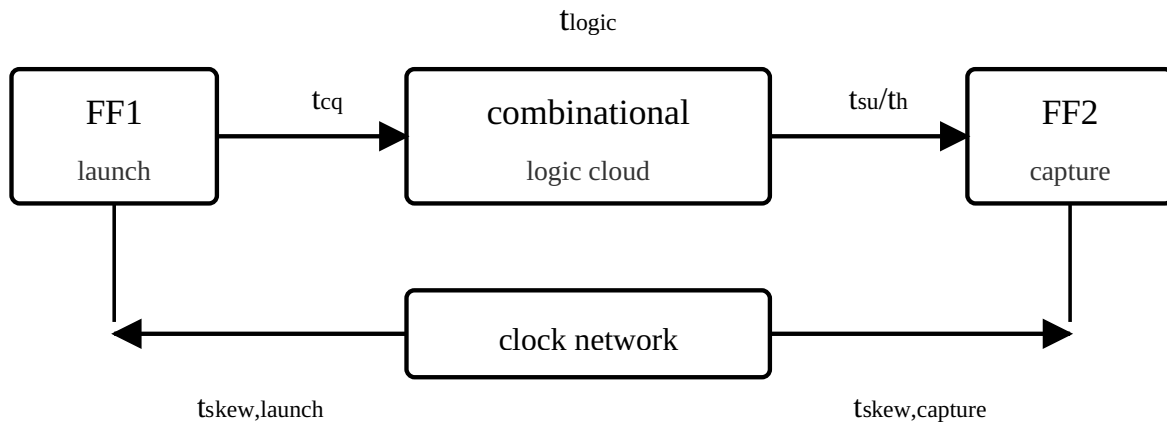
Given. The 11-tap lowpass above, driven by a signal bounded by 1.0.

Method. Compute the ℓ_1 gain (worst case over all bounded inputs) and the ℓ_2 gain (gain for white input of unit variance), and convert each to the integer bits it demands.

ℓ_1 gain $\sum h_k $	1.093
Integer bits needed by ℓ_1	1
ℓ_2 gain	0.4539
Measured peak output for the uniform stimulus above	0.818
Measured peak as a fraction of the ℓ_1 bound	0.749

How this is got wrong. By quoting the measured peak as if it were a bound. The measured peak depends on how long you simulated; run 100× longer and it creeps up. A bound does not creep. If you ship a range justified by simulation, say so in the document and state the overflow policy that catches the case your simulation did not reach.

X2. Timing Closure Arithmetic, Worked



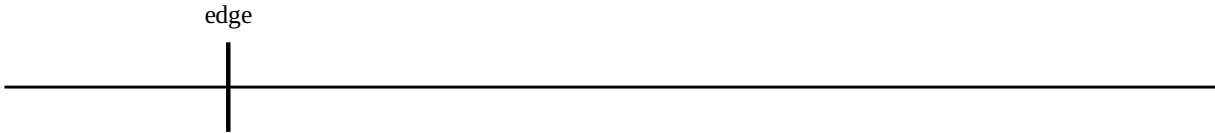
Every STA number in this part is a term of this one picture.

Figure 21. The one path that every static timing constraint is about. Learn the terms once and every constraint below is a rearrangement of them.

X2.1 The two inequalities

Setup and hold from first principles

	Step	Justification
1	Data leaves FF1 at $t_{skew,L} + t_{cq}$ after the nominal launch edge.	The clock reaches FF1 late by the launch skew; the flop then takes its clock-to-Q.
2	It arrives at FF2 at $t_{skew,L} + t_{cq} + t_{logic}$.	Add the combinational delay.
3	FF2 samples at $T + t_{skew,C}$ and needs the data stable t_{su} before that.	Definition of setup time, one period later.
4	Setup: $t_{skew,L} + t_{cq} + t_{logic} + t_{su} \leq T + t_{skew,C}$.	Arrival no later than required.
5	Rearranged: slack_{su} = $T - t_{cq} - t_{logic} - t_{su} + t_{skew}$, with $t_{skew} = t_{skew,C} - t_{skew,L}$.	Only the difference of the skews appears. Absolute insertion delay cancels — which is why a large balanced tree is harmless and a small unbalanced one is not.
6	For hold, the same edge must not be overtaken: $t_{skew,L} + t_{cq,min} + t_{logic,min} \geq t_{skew,C} + t_h$.	New data must not reach FF2 before the old data has been captured.
7	slack_h = $t_{cq,min} + t_{logic,min} - t_h - t_{skew}$. T does not appear.	This is the whole reason hold violations cannot be fixed by slowing the clock, and the reason they are found at the worst possible moment.



Setup is a race against the *next* edge; hold is a race against the *same* edge. That single sentence explains why one scales with the period and the other does not.

Figure 22. Setup and hold as races against different edges.

Worked example 6. A 1 GHz path, term by term

Given. $T = 1.00$ ns, $t_{cq} = 90$ ps, $t_{logic} = 620$ ps, $t_{su} = 50$ ps, useful skew $t_{skew} = +30$ ps.

Method. Substitute into the setup slack expression. Then ask what fraction of the period the designer actually controls.

Setup slack	270 ps
Fraction of T spent on flop overhead	14 %
Fraction available to logic	62 %
Maximum t_{logic} at zero slack	890 ps
Logic depth at FO4 = 25 ps	35 FO4

How this is got wrong. By quoting ‘20 levels of logic’ without saying at what FO4. A level of logic is not a unit. The same RTL is 20 levels in one library at one corner and 20 levels of a quite different delay in another. **Convert to FO4 delays, or to picoseconds, before comparing anything across processes.**

X2.2 Where the period goes as frequency rises

Flop overhead ($t_{cq} + t_{su}$) is roughly fixed by the library; the period is not. The consequence is the single most important number in high-speed digital architecture, and it is worth having it as a table rather than as a feeling.

Table 199. Budget left for logic after fixed flop overhead of 140 ps

f (GHz)	T (ps)	overhead (ps)	logic budget (ps)	overhead as % of T	FO4 levels at 25 ps
0.5	2000	140	1860	7	74
1	1000	140	860	14	34
2	500	140	360	28	14
3	333.3	140	193.3	42	7
4	250	140	110	56	4
5	200	140	60	70	2
7	142.9	140	2.857	98	0

f (GHz)	T (ps)	overhead (ps)	logic budget (ps)	overhead as % of T	FO4 levels at 25 ps
10	100	140	-40	140	0

Skew and jitter are excluded, so these are optimistic. Even so the last rows explain the architecture of every multi-GHz block in this book: at 5 GHz you get a handful of gate delays, so the work must be spread across parallel slices rather than done faster.

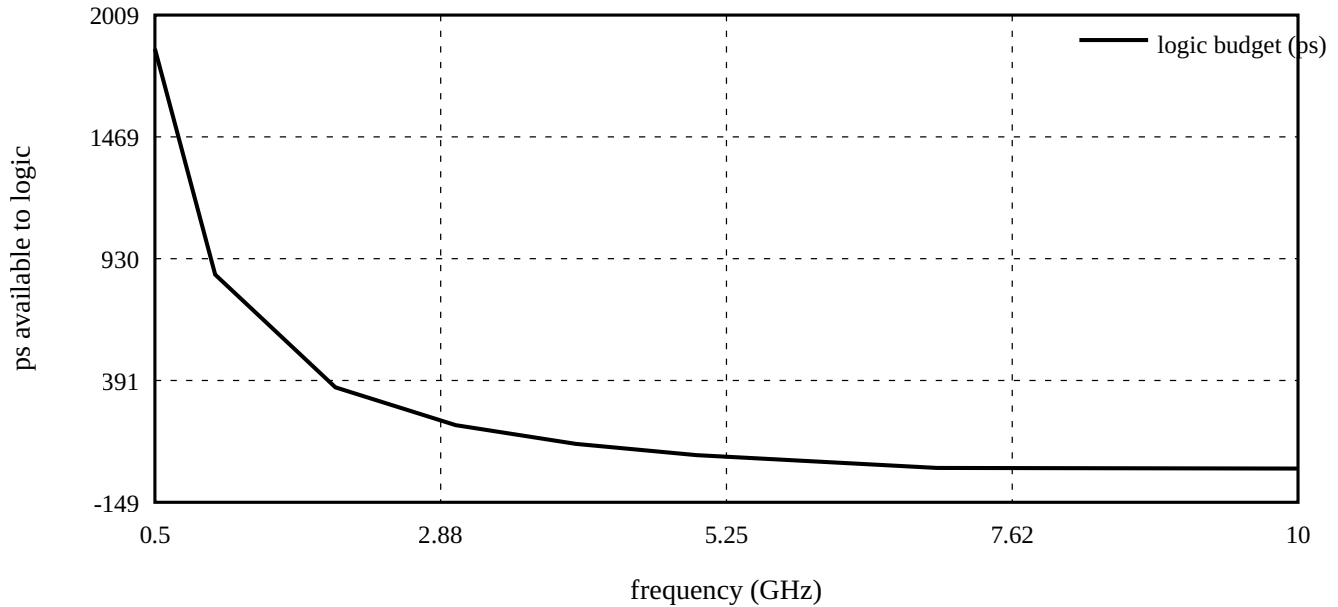


Figure 23. The logic budget collapses hyperbolically while the overhead stays flat. Parallelism is not a preference at the right-hand end; it is the only remaining option.

Graduate · The consequence for IP architecture. A block specified at 2 GHz in a process whose FO4 is 25 ps has about fourteen gate delays per cycle. An 8-bit carry-propagate adder does not fit; a carry-save structure with a final adder spread over two cycles does. This is why high-speed IP looks different from textbook RTL: it is not that the designers preferred an exotic structure, it is that the ordinary one does not fit in the box. When you read an unfamiliar architecture in a datasheet, **compute the per-cycle gate budget first** and most of the strangeness will explain itself.

X2.3 Corners, derates and the multiplication of runs

Sign-off is not one analysis. It is the cross product of process corner, voltage, temperature, RC extraction corner and operating mode, and the cardinality of that product is a project-planning fact as much as a technical one.

Worked example 7. How many sign-off runs is 'sign-off'?

Given. 3 process corners, 3 voltages, 3 temperatures, 5 RC corners, 4 functional modes. A single run takes 6 hours on 16 cores.

Method. Multiply. Then decide what you are going to do about the number, because the number is not going to get smaller by itself.

Full cross product	540
Serial machine-hours	3,240
Wall-clock days on 10 parallel machines	13.5
After MMMC pruning to dominant corners (typical 12)	12
Wall-clock days for 12 corners on 10 machines	0.3

How this is got wrong. By assuming the corners are independent so the worst case is the product of worst cases. They are not: slow-process cells are slow but slow-RC wires may pair with fast cells, and the genuinely worst path differs per corner. Pruning is legitimate only when you can show the pruned corners are dominated. **Record the dominance argument;** 'we always run these twelve' is not one, and it is how a corner quietly stops being covered after a library update.

Table 200. What each corner axis is actually modelling

Axis	Physical cause	Hurts setup when	Hurts hold when
Process (SS/TT/FF)	Threshold voltage and mobility spread across wafers and lots	Slow	Fast
Voltage	IR drop, regulator tolerance, di/dt droop	Low	High
Temperature	Mobility falls with heat; threshold falls too	Hot — usually	Cold
RC extraction	Metal width/thickness and dielectric variation	Cmax / RCmax	Cmin / RCmin
OCV / AOCV / POCV	Within-die variation along one path	Derated launch vs. capture	The reverse

Temperature inversion makes the 'hot is slow' rule false at low voltage. Carrier mobility falls with temperature, which slows a device, but the threshold voltage also falls, which speeds it up. Above roughly 0.8 V in mature nodes the mobility term wins and hot is slow. Near and below the threshold the other term wins and **cold becomes the setup-critical corner.** Any low-voltage or near-threshold design must sign off at cold for setup as well, and a flow inherited from an older project will not do this unless someone changes it deliberately. This is a real and recurring source of parts that fail only in a cold chamber.

X2.4 Statistical slack: what a 3 σ margin buys

Corner-based timing answers a yes/no question. Yield is a probability, so at some point the two have to be connected. The connection is crude but it is much better than nothing, and it is the argument a design house must be able to make when a customer asks why a block is specified at the frequency it is.

Table 201. Monte-Carlo path yield as the target frequency is raised (200 000 trials, $\sigma_{\text{logic}} = 35$ ps)

f (GHz)	T (ps)	mean slack (ps)	slack / σ	P(path fails)	Yield of 20 000 such paths
1	1000	270	7.71	$<5 \times 10^{-6}$	1
1.15	869.6	140	4	7.0×10^{-5}	0.247
1.25	800	70	2	0.023	1.06×10^{-202}

f (GHz)	T (ps)	mean slack (ps)	slack / σ	P(path fails)	Yield of 20 000 such paths
1.35	740.7	10.7	0.307	0.38	0
1.45	689.7	-40.4	-1.16	0.88	0

The last column is the point. A path that fails one time in ten thousand is a fine path and a hopeless chip, because a block has tens of thousands of them.

Graduate · Why the last column is the honest one, and why it is still optimistic. Treating 20 000 paths as independent overstates the failure probability, because paths share cells, share supply, and share the same die's global process point — correlation makes the block behave more like a few effective independent paths than twenty thousand. Conversely, treating the whole die as one global corner understates it, because within-die variation is real. Statistical STA exists precisely to interpolate between these two wrong answers, and the parameter it needs — the split between global and local variance — comes from silicon measurement, not from the tool. **When a vendor quotes a statistical yield without stating that split, the number carries no information.**

Problem. A block has 20 000 timing paths. You need 99 % of dies to have every path passing. Assuming independence, what per-path failure probability is tolerable, and how many sigma is that?

Solution. Require $(1-p)^{20000} \geq 0.99$, so $p \leq -\ln(0.99)/20000 \approx 5.03 \times 10^{-7}$. For a Gaussian that is about 4.4σ (the exact quantile is $\Phi^{-1}(1-p)$). Two lessons: the required margin grows only logarithmically in path count, so ten times more paths costs well under one extra sigma; and 3σ per path — the number people quote from memory — gives a block yield of about 1.85×10^{-12} , which is zero. **Per-path margin and per-block yield are different questions and the second is the one the customer asked.**

X2.5 Fixing what the report says is broken

Table 202. Failure mode → remedy, with the cost of each remedy

Symptom	Which term is short	Remedies, cheapest first	What it costs
Setup fails by 20–80 ps on many paths	t_{logic}	Upsize cells on the critical path; restructure the logic; useful skew; retime; add a pipeline stage	Power; then area; then latency and a verification change
Setup fails by >200 ps on a few paths	Architecture, not cells	Re-architect that path; the tool cannot buy 200 ps	Schedule — treat as a design bug, not a closure task
Hold fails after CTS	$t_{\text{logic,min}}$ vs. skew	Insert delay buffers; swap to slower-Vt cells on the short path	Area and leakage; free in frequency
Hold fails at the fast corner only	Skew at that corner	Fix the clock tree first; buffers inserted for one corner can break another	Iteration
Fails only in one functional mode	Mode-dependent false path not declared	Declare the exception — and prove it	A wrong exception is a silicon bug that no timing run will ever catch
Both setup and hold fail on the same path	Skew	The clock tree, not the datapath	—

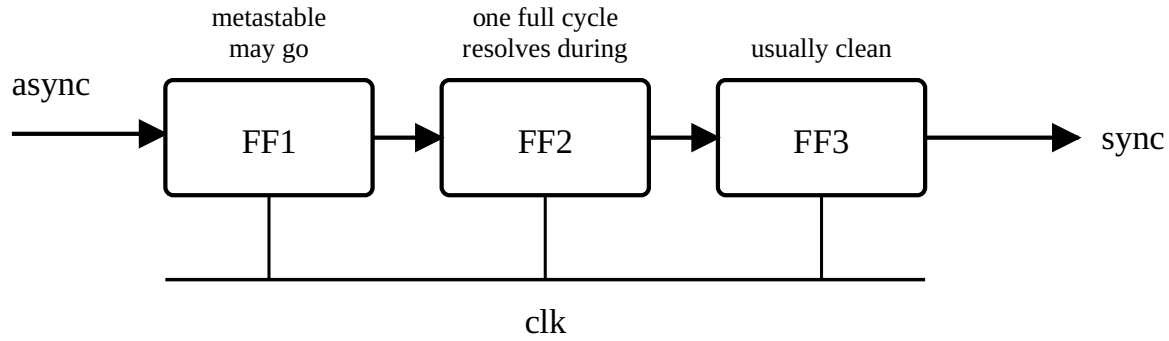
Timing exceptions are assertions about behaviour, and the timing tool cannot check them. A `set_false_path` tells the tool to stop looking. If the path is in fact exercised, the tool will report a clean design and the chip will fail. Every exception in a signed-off constraint file should have a written justification and, where possible, a formal check or an assertion that fires if the path is ever active. On more than one project the post-mortem of a functional failure has ended at a single `set_multicycle_path` that was true when it was written and became false three revisions later.

Problem. A path fails setup by 40 ps. Useful skew can move 30 ps of slack from the next stage, which has 120 ps of slack. Is this a good fix?

Solution. Arithmetically yes: the path closes with 10 ps to spare and the donor still has 90 ps. Two cautions make it a conditional yes. First, useful skew is delivered by the clock tree, so it must survive CTS, ECO routing and every corner — skew that exists only at one corner is not margin. Second, it moves the hold constraint by the same 30 ps in the harmful direction on *both* stages. **Always evaluate a skew fix on the setup and hold inequality simultaneously;** the two contain t_{skew} with opposite signs, which is exactly why the trick works and exactly why it is dangerous.

X3. Metastability and Clock-Domain Crossing, Worked

A flip-flop sampled while its input is changing may enter a metastable state whose resolution time is unbounded. ‘Unbounded’ is the word that matters: there is no settling time that guarantees correctness, only a settling time that makes failure rare enough. Designing a synchroniser therefore means choosing a number of years, which is a much more concrete decision than it first sounds.



Each added stage multiplies the MTBF by $e^{T/\tau}$ — the gain is exponential, not linear.

Figure 24. A two-flop synchroniser. The second flop’s job is not to clean up the signal — it is to give the first flop a whole clock period in which to resolve.

X3.1 The MTBF expression and what each symbol is

Where the exponential comes from

	Step	Justification
1	A metastable flop’s output separates from the balance point exponentially: $\Delta V(t) = \Delta V_0 e^{t/\tau}$.	<i>Small-signal analysis of the cross-coupled pair: the loop has a positive real pole, and τ is its inverse.</i>
2	Resolution by time t requires ΔV_0 larger than some $\Delta V_{\min} e^{-t/\tau}$.	<i>Invert the exponential.</i>
3	ΔV_0 is proportional to how far the data edge was from the sampling point, and that offset is uniformly distributed.	<i>Asynchronous input: no phase relationship, so all offsets equally likely.</i>
4	So $P(\text{unresolved after } t) \propto e^{-t/\tau}$, with a constant T_0 carrying the aperture width.	<i>Uniform measure times an exponentially shrinking target window.</i>
5	Failures occur at rate $f_{\text{clk}} f_{\text{data}} T_0 e^{-t/\tau}$.	<i>Each clock edge is a trial; each data edge supplies an opportunity.</i>
6	$\text{MTBF} = e^{t/\tau} / (T_0 f_{\text{clk}} f_{\text{data}})$	<i>Reciprocal of the rate. τ and T_0 are library measurements, not universal constants; ask the vendor for them.</i>

Table 203. MTBF against synchroniser depth and frequency ($\tau = 20$ ps, $T_0 = 10$ ps, data toggling at $f/10$)

Stages	f_{clk} (MHz)	Resolution time (ps)	MTBF (s)	MTBF (years)
1	100	9880	$10^{210.5}$	$10^{203.0}$
1	500	1880	$10^{35.4}$	$10^{27.9}$
1	1000	880	$10^{13.1}$	$10^{5.6}$
2	100	19880	$10^{427.7}$	$10^{420.2}$
2	500	3880	$10^{78.9}$	$10^{71.4}$
2	1000	1880	$10^{34.8}$	$10^{27.3}$
3	100	29880	$10^{644.8}$	$10^{637.3}$
3	500	5880	$10^{122.3}$	$10^{114.8}$
3	1000	2880	$10^{56.5}$	$10^{49.0}$
4	100	39880	$10^{862.0}$	$10^{854.5}$
4	500	7880	$10^{165.7}$	$10^{158.2}$
4	1000	3880	$10^{78.3}$	$10^{70.8}$

The third column is linear in the number of stages and the fourth is exponential in the third. That is the whole design space.

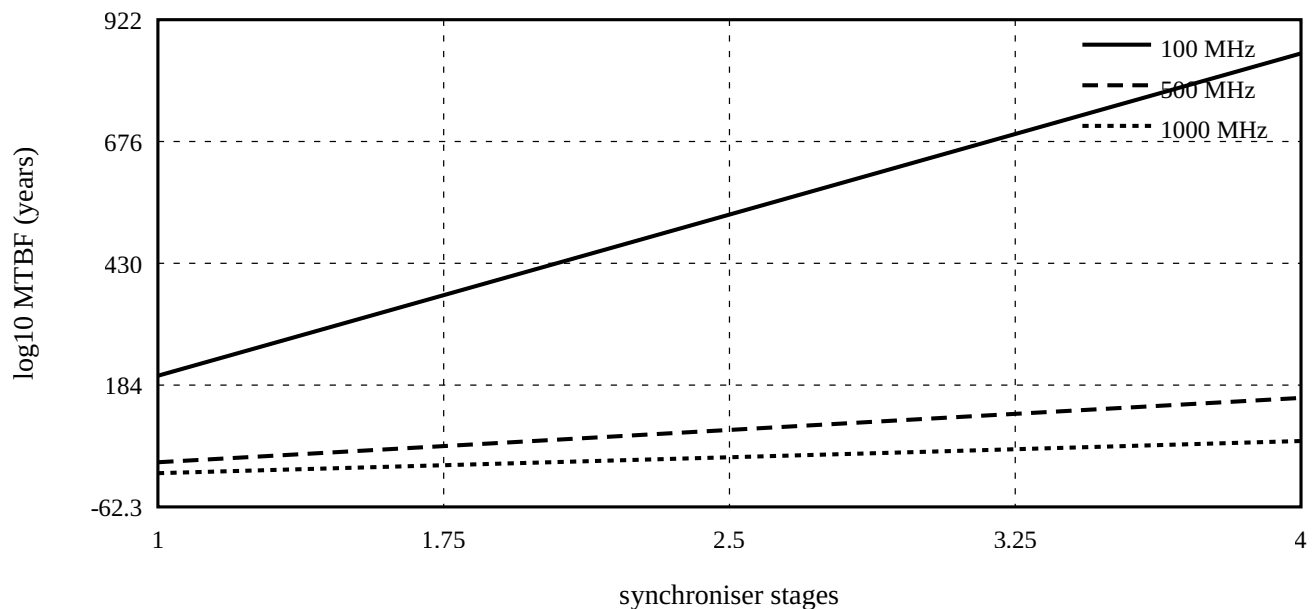


Figure 25. MTBF in years, log scale, against depth. A straight line on this axis is the exponential: each stage adds the same number of decades, and that number shrinks as the period shrinks.

Worked example 8. Choosing the depth for a 1 GHz interface

Given. $\tau = 20$ ps, $T_0 = 10$ ps, $f_{\text{clk}} = 1$ GHz, data edges at 100 MHz. Setup plus clock-to-Q consume 120 ps of each period.

Method. Compute MTBF for two and three stages, then compare against a target. A reasonable target for a part shipping in volume is *not* ‘longer than the product’s life’ — it is longer than the product’s life multiplied by the number of units in the field.

Resolution time, 2 stages	1880 ps
MTBF, 2 stages	$10^{27.3}$ years
MTBF, 3 stages	$10^{49.0}$ years
2-stage MTBF across a 10-million-unit fleet	$10^{20.3}$ years
Same, but with τ doubled at the cold/low-V corner	$10^{-0.1}$ years

How this is got wrong. By evaluating the MTBF for one chip. The failure rate is additive over units, so a fleet of ten million divides the MTBF by ten million. A synchroniser that is comfortable in the lab can produce a field return every few weeks. The second common error is to use τ from a different library, or from a conference slide: τ varies by a factor of several between libraries and corners, and it sits in an exponent.

τ degrades at low voltage and cold, and the degradation is exponential in effect. A synchroniser characterised at nominal may have a τ 1.5–2× larger at the low-voltage corner; with t_f/τ halved, an MTBF of 10^9 years becomes 10^4 . **Evaluate the MTBF at the worst corner for τ , not the typical one** — and note that this is a corner the timing tool does not analyse for you, because the synchroniser path is one you told it to ignore.

X3.2 A synchroniser is not a CDC solution

Two flops make *one bit* safe. They do nothing for the problem that actually causes CDC bugs, which is that several bits crossing together may be sampled on different edges and produce a value that never existed in the source domain.

Table 204. Measured probability that a multi-bit crossing yields a value that existed in neither the old nor the new state, binary vs. Gray coding

Bits	P(bogus), binary	P(bogus), Gray	Bogus samples, binary	Bogus samples, Gray
1	0	0	0	0
2	0.248	0	49,651	0
4	0.329	0	65,732	0
8	0.334	0	66,771	0
16	0.334	0	66,704	0

200 000 trials per row, each bit independently taking the old or the new value. Gray coding makes the bogus count exactly zero, because consecutive Gray codes differ in one bit and a single bit can only be old or new.

Worked example 9. Why Gray code is not optional in an asynchronous FIFO

Given. An 8-bit write pointer crossing into the read domain, incrementing every cycle at 500 MHz.

Method. Take the measured binary bogus probability from the row above and turn it into a rate. Then ask what a bogus pointer does.

P(bogus sample), 8 bits binary, measured above	0.334
Crossings per second	5.000×10^8
Bogus pointer values per second (order of magnitude)	10^7–10^8
Consequence of one bogus write pointer	reader sees a wrong depth — underflow or overflow
Consequence with Gray coding	none: the value is old or new, both safe

How this is got wrong. By assuming the synchroniser flops fix it. They do — per bit. The failure is a *composition* failure across bits, and no amount of depth on individual bits addresses it. The other frequent error is to Gray-code the pointer and then do arithmetic on it in the destination domain before converting back to binary; Gray codes do not add, and a comparison of Gray values is not a comparison of the numbers they encode except in the specific equality test that the standard FIFO empty/full logic uses.

Table 205. CDC schemes and what each is actually for

Scheme	Carries	Throughput	Latency	Fails if
Two-flop synchroniser	one bit, slowly changing	< 1 per 2 destination cycles	2–3 destination cycles	The bit changes faster than it is sampled (pulse loss)
Pulse / toggle synchroniser	an event	1 per 3–4 cycles	3–4 cycles	Events arrive closer together than that
Gray-coded async FIFO	a data stream	1 word per cycle	2–4 cycles each way	Depth is wrong for the rate mismatch — a <i>performance</i> failure, not a correctness one
Handshake (req/ack)	a word, safely	1 per ~5 cycles round trip	high	Nothing, but it is slow; used for configuration, not data
MUX recirculation / data hold	a bus with a synchronised enable	1 per enable	2–3 cycles	The enable and the data are not guaranteed stable together
Common clock, no crossing at all	everything	—	0	Consider this first. Many crossings exist only because nobody questioned the clock plan

Graduate · The pulse-loss trap, stated precisely. A synchroniser samples; it does not detect edges. If the source asserts a signal for one source-clock cycle and the destination clock is slower, the destination may simply never see it. The rule is that a level crossing into a slower domain must be held for at least one full destination period plus the source's own uncertainty — in practice three destination cycles is the usual specification. When the ratio is not known at design time, because the customer sets both frequencies, **a level crossing is unimplementable and the crossing must be a toggle or a handshake.** This is a frequent finding in IP integration: the block worked at the vendor's ratio and fails at the customer's.

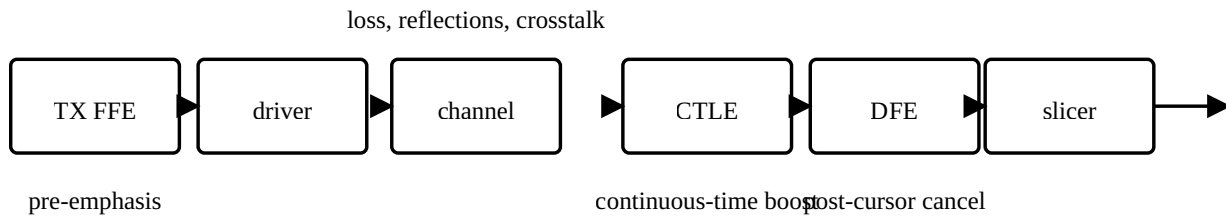
Problem. A configuration register is written in a 100 MHz APB domain and read by a 1 GHz datapath. The 32 bits change together. What is the correct structure, and what is wrong with synchronising each bit?

Solution. Synchronising each bit independently is wrong for the reason measured above: the datapath can observe a mixture of old and new fields. The standard structure is **data hold with a synchronised enable**: the APB domain writes the 32 bits into a holding register and then toggles a single control bit; that one bit crosses through a two-flop synchroniser; the datapath samples the 32 bits only when it sees the toggle. The data bits themselves never need synchronisers because they are guaranteed stable for several destination cycles before and after the sampling point — a guarantee that must be **written into the constraint file as a set_max_delay -datapath_only**, or the tool will either flag it or, worse, silently optimise the holding register away.

Problem. A reviewer says ‘add a third flop, it is nearly free’. When is that the wrong advice?

Solution. When the crossing's problem is not metastability. A third flop buys MTBF and costs one cycle of latency; if the observed bug is pulse loss, bus coherency or a missing constraint, the third flop changes nothing except the latency, and it can make a *reconvergence* problem worse by lengthening one path relative to another. **Establish which of the four CDC failure modes you have before reaching for depth**: metastability, pulse loss, incoherent multi-bit sampling, and reconvergence of separately synchronised signals. Only the first is helped by more stages.

X4. The Wireline Link Budget, Worked



Everything before the slicer exists to make the eye open at one instant in time.

Figure 26. The signal chain a link budget accounts for. Each block contributes a term; the budget is the statement that the terms sum to something positive.

X4.1 From loss in dB to intersymbol interference

A channel specification usually gives one number: insertion loss at the Nyquist frequency. That number is not directly usable. What the receiver has to deal with is the *pulse response* — what a single transmitted symbol looks like at the sampler, sampled once per unit interval. The cursors of that response are the intersymbol interference, and they are what every equaliser in the chain is fighting.

Table 206. Pulse-response cursors of a $\sqrt{f}$ channel at 53.125 GBd as insertion loss is swept

Loss at Nyquist (dB)	Main cursor	$\Sigma \text{pre-cursors} $	$\Sigma \text{post-cursors} $	Worst-case eye height	Unequalised eye
5	1	0.107	0.191	0.702	open
10	1	0.233	0.442	0.325	open
15	1	0.375	0.762	-0.137	closed
20	1	0.529	1.15	-0.682	closed
25	1	0.687	1.62	-1.3	closed
30	1	0.843	2.15	-2	closed
35	1	0.992	2.75	-2.74	closed

Cursors are normalised so the main cursor of the least-lossy case is 1. The fifth column is the classic worst-case opening, $\text{main} - \Sigma|\text{ISI}|$, which assumes every neighbouring symbol conspires against the one being sampled.

Graduate · Worst-case eye height goes negative long before the link is actually unusable, and that is not a contradiction.

The worst case demands a specific data pattern; at 10^{-4} bit error rate you care about patterns that occur with probability around 10^{-4} , not about the one worst pattern. This is why link analysis moved from peak-distortion analysis to **statistical eye** methods: convolve the probability distributions of the individual cursors rather than summing their magnitudes. Peak analysis is a bound; the statistical eye is the answer. A specification that quotes a closed worst-case eye and a working link is not lying, but it is using two different methods and should say so.

Table 207. Statistical eye vs. peak-distortion eye, 25 dB channel, 400 000 random patterns

Target BER	ISI exceeded with that probability	Statistical eye height	Peak $\Sigma \text{ISI} $	Peak-distortion eye height
0.001	1.99	-0.992	2.3	-1.3
1.0×10^{-4}	2.1	-1.1	2.3	-1.3

Target BER	ISI exceeded with that probability	Statistical eye height	Peak $\Sigma ISI $	Peak-distortion eye height
1.0×10^{-5}	2.2	-1.2	2.3	-1.3
1.0×10^{-6}	2.23	-1.23	2.3	-1.3

The last column is the same for every row because the peak bound does not depend on the target. The difference between columns three and five is the margin that peak analysis throws away — and it is often more than a whole equaliser's worth.

X4.2 What each equaliser can and cannot do

Table 208. Equaliser taxonomy, stated by what it operates on

Block	Operates on	Cancels	Cannot	Costs
TX FFE	The transmitted waveform, before the channel	Pre- and post-cursors	Add energy — it can only redistribute, so boosting high frequencies lowers the average launched amplitude	Launch amplitude, hence SNR
CTLE	The analogue waveform, continuously	Broad loss slope	Distinguish signal from noise — it boosts both	Noise enhancement; peaking bandwidth
DFE	Decided symbols	Post-cursors only, with no noise enhancement	Touch pre-cursors; survive a wrong decision without error propagation	A feedback loop that must close in one UI
MLSE / Viterbi	A sequence	Everything, optimally in the ML sense	Be cheap: complexity is exponential in memory	Area and power; latency
FFE in the RX (post-ADC)	The sampled waveform	Pre- and post-cursors	Avoid amplifying noise, like any linear equaliser	ADC resolution and DSP power

The DFE's advantage and its constraint are the same fact. Because it subtracts using *decisions* rather than the received waveform, it cancels post-cursor ISI without amplifying noise — the property that makes it indispensable on lossy channels. But that decision must be made, multiplied and subtracted before the next symbol is sampled, which at 53 GBd means the whole loop in 18.8 ps. Part X2's budget table says how many gate delays that is; the answer is close to zero, and the architectural response is **unrolling** (speculate on both possible decisions and select afterwards), which converts the timing problem into an area problem that grows as 2^n in the number of unrolled taps.

Worked example 10. The one-UI DFE budget at 53.125 GBd

Given. A direct-feedback first tap. Symbol rate 53.125 GBd. Component delays as listed, taken from a 7 nm-class custom cell set.

Method. Sum the terms and compare with one unit interval. There is no cleverness here; the point of the example is that the arithmetic is trivial and the answer is nonetheless the reason a whole architecture exists.

Unit interval	18.82 ps
Slicer clock-to-Q	6 ps
Tap multiply (1-bit × coefficient)	3 ps
Summer	4 ps
Wire and buffering back to the summer	5 ps
Slicer setup	4 ps
Total loop delay	22 ps
Margin	-3.18 ps
Verdict	does not close — must be unrolled

How this is got wrong. By budgeting the logic and forgetting the wire. In this budget the wire and buffering term is comparable to the summer. At these rates the feedback path is a physical-design problem before it is a logic problem, and a block-level delay estimate made from gate counts alone will be optimistic by roughly a third. **Any loop budget that does not have a line for interconnect is incomplete.**

Table 209. Cost of unrolling n DFE taps

Unrolled taps n	Speculative slicers	Select-mux inputs	Relative slicer area	Seen in production?
1	2	1	2	yes
2	4	3	4	yes
3	8	7	8	yes
4	16	15	16	impractical
5	32	31	32	impractical
6	64	63	64	impractical

Slicers at these rates are analogue comparators with real offset-trim circuits, so the area is not notional. Production parts unroll one or two taps and handle the rest with a pipelined or parallel structure.

X4.3 Jitter, noise and the BER that results

From an eye opening to a bit error rate

	Step	Justification
1	At the sampling instant the decision variable is $v = \pm A_{\text{eff}} + n$.	A_{eff} is the residual eye half-height after equalisation; n is the total additive noise referred to that node.
2	An error occurs when n crosses A_{eff} in the wrong direction.	Slicer threshold at zero.
3	For Gaussian n with standard deviation σ , $\text{BER} = Q(A_{\text{eff}}/\sigma)$.	Tail probability of a Gaussian. The Gaussian assumption is the weak link and X4.4 says why.
4	Timing error Δt reduces the effective amplitude to $p(t_0 + \Delta t)$, and near the peak that reduction is second order.	The pulse response is smooth and stationary at its peak — which is why a sensitivity measured exactly at the peak is zero by definition and means nothing.
5	So jitter must be evaluated where the slope is large, i.e. displaced from the optimum, or by convolving the timing distribution into the statistical eye.	This is what a compliance tool does; it is also what an honest hand calculation must do.

Table 210. BER against amplitude-to-noise ratio, and the effect of losing 10 % of the eye

A/σ (dB)	A/σ	BER	BER at 0.9A	Ratio
10	3.16	7.8×10^{-4}	0.0022	0.35
12	3.98	3.4×10^{-5}	1.7×10^{-4}	0.2
14	5.01	2.7×10^{-7}	3.2×10^{-6}	0.083
16	6.31	1.4×10^{-10}	6.8×10^{-9}	0.021
18	7.94	9.8×10^{-16}	4.4×10^{-13}	0.0023
20	10	7.6×10^{-24}	1.1×10^{-19}	6.8×10^{-5}

The last column is the lesson: near 10^{-12} a ten per cent eye loss costs two to three orders of magnitude of BER. **Link margin is not linear and small budget errors are not small.**

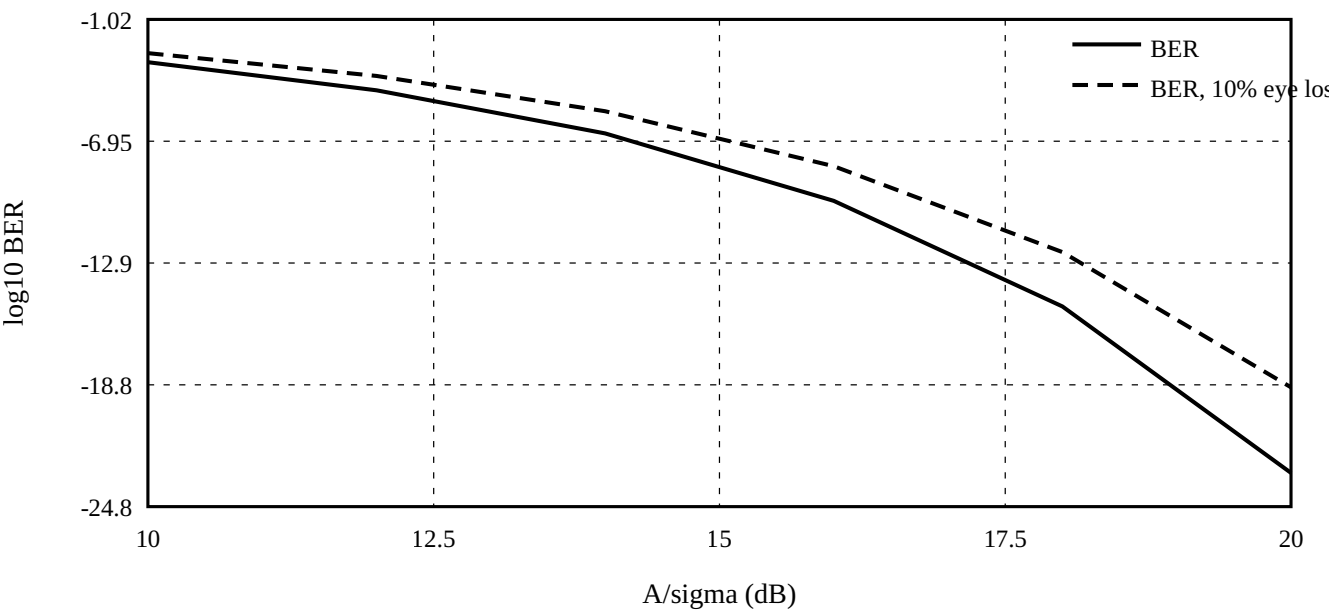


Figure 27. The waterfall. The vertical gap between the curves at constant BER is the margin a budget error consumes.

Worked example 11. Turning a jitter specification into an eye loss

Given. Random jitter 250 fs RMS, deterministic jitter 2 ps peak-to-peak, UI = 18.8 ps, target BER 10^{-4} before FEC.

Method. Total jitter at a target BER is the dual-Dirac construction: $TJ = DJ + 2Q_{BER} \cdot RJ$, where Q_{BER} is the Gaussian quantile for that BER.

Q for BER 10^{-4}	3.719
Random contribution $2Q \cdot RJ$	1.859 ps
Deterministic contribution	2 ps
Total jitter	3.859 ps
As a fraction of the UI	0.205
Eye width remaining	14.96 ps

How this is got wrong. By using the Q for the wrong BER. The same jitter is 3.9 ps at 10^{-4} and about 5.9 ps at 10^{-12} ; quoting a TJ number without its BER is meaningless, and the two numbers differ by enough to change an architecture. The second error is to add RJ and DJ in quadrature: DJ is bounded, not Gaussian, and it adds *linearly* to the Gaussian tail. Quadrature addition here understates the total.

X4.4 Where the Gaussian assumption fails, and what FEC changes

Two facts force a modern link away from the simple budget above. Noise at the slicer is not Gaussian — crosstalk is the sum of a few discrete interferers and has a bounded, multi-modal distribution — and errors are not independent, because a DFE that makes a wrong decision feeds it back and produces a short burst. Both matter for FEC design, which is the subject of Part F2 and of the block this book's practice volume builds.

Table 211. Assumption, reality, and the design consequence

Assumption in the simple budget	What is actually true	Consequence for the design
Noise is Gaussian	Crosstalk from k aggressors is a sum of k bounded terms; the tail is lighter than Gaussian but the body is wider	Extrapolating a measured 10^{-6} BER to 10^{-15} with a Gaussian fit can be wrong by decades in either direction
Errors are independent	DFE error propagation produces bursts of length comparable to the number of taps	The FEC must be chosen for a burst channel: symbol-oriented codes (Reed–Solomon over $GF(2^{10})$) rather than bit-oriented ones
The sampling instant is optimal	The CDR tracks a noisy estimate and dithers about the optimum	Budget a static phase offset as well as jitter
The equaliser is converged	Adaptation is a slow loop with its own noise and can converge to a local optimum	Specify and measure the converged state; a measurement taken before convergence is not a measurement of the link

Problem. A link measures 10^{-6} raw BER. The FEC is RS(544,514) over $GF(2^{10})$, correcting 15 symbol errors per codeword. Estimate the post-FEC frame error rate, and say why the estimate is fragile.

Solution. Each codeword is 544 ten-bit symbols. A symbol is wrong if any of its ten bits is wrong, so with independent bit errors the symbol error probability is about $1 - (1 - 10^{-6})^{10} \approx 10^{-5}$. The codeword fails when more than 15 of 544 symbols are wrong; with a mean of $544 \times 10^{-5} \approx 5.4 \times 10^{-3}$ errors per codeword the binomial tail at 16 is astronomically small, which is the intended answer. **The fragility is the independence assumption.** If a DFE burst corrupts several consecutive bits, they may fall inside *one* symbol — which helps, since a symbol code charges the same price for one wrong bit or ten — or straddle a symbol boundary and cost two. Worse, if bursts are long enough to span many symbols, the tail is controlled by burst statistics that the raw BER number does not contain at all. **Post-FEC performance cannot be computed from pre-FEC BER alone; it needs the error-clustering statistics**, which is why compliance specifications define stressed-eye tests rather than BER targets alone.

Problem. Why does IEEE 802.3dj concatenate an inner code with the outer RS-KP4 rather than simply using a stronger single code?

Solution. Three reasons, and they are all system reasons rather than coding-theory ones. **Backward compatibility:** RS(544,514) is already deployed and an outer layer that leaves it intact protects an installed base of PCS implementations. **Latency and locality:** the inner code can be short and decoded close to the PHY with low latency, cleaning up the random errors, while the outer code handles what remains; a single long code with the same total redundancy would have higher decoding latency. **Burst breaking:** the inner decoder's residual errors, after interleaving, present the outer decoder with something closer to the independent-symbol channel that Reed–Solomon analysis assumes — which is precisely the assumption the previous problem showed to be the fragile one.

X5. Reed–Solomon Decoding, Traced and Measured

Reed–Solomon is the error-correcting code that every high-speed Ethernet and storage link in this book actually uses, and it is the code the practice volume builds an IP block around. It is also a code whose decoder is usually presented as four opaque stages. This part opens each stage on a small field where the numbers can be printed in full, then measures the large field that ships.

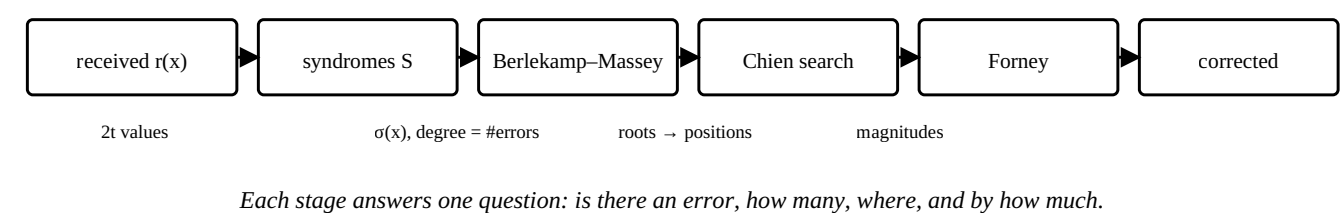


Figure 28. The classical Reed–Solomon decoding pipeline. The stages below are executed for real on a small field, so every intermediate value is printed rather than described.

X5.1 The field is a choice, and an unverified choice is a risk

A Reed–Solomon code lives over a finite field $\text{GF}(2^m)$, and the field is fixed by a primitive polynomial. Two implementations that pick different primitive polynomials produce different codewords and cannot talk to each other, even though both are ‘RS(544,514)’. The model this chapter runs therefore takes the polynomial as a *parameter* and records whether it has been confirmed against the standard.

Worked example 12. Verifying a small field before using it

Given. $\text{GF}(2^4)$ generated by $x^4 + x + 1$ (0b10011).

Method. The model builds exponent and log tables and refuses any polynomial whose root α has order less than $2^m - 1$ — that is, it rejects non-primitive polynomials by construction rather than by trusting the caller. It then checks the field *axioms*, not a table of expected values, because a table of expected values is only as good as whoever typed it.

Field size	16
Order of α	15
Axiom checks performed	원소수, 본원소, 역원실패, 항등실패, 나눗셈실패, 지수표전사, 필드인가
All axioms hold	no

How this is got wrong. By hard-coding the polynomial of the standard you last worked on. 0x11D and 0x12D both generate $\text{GF}(256)$ and both are primitive; they are different fields for interoperability purposes. The repository's model keeps this as an explicit parameter with a **confirmed** flag precisely so that an unverified default cannot quietly become an assumption — the flag is **false** for the KP4 field in this repository, because the clause that fixes it has not been read.

X5.2 A complete decode, every intermediate value printed

Worked example 13. RS(15,9) over GF(16): encode, corrupt, decode

Given. Message symbols [1, 2, 3, 4, 5, 6, 7, 8, 9]. Three symbols are then corrupted by XOR: position 2 $\oplus$ 7, position 5 $\oplus$ 11, position 11 $\oplus$ 4. The code corrects up to $t = 3$ symbols.

Method. Encode systematically, inject the errors, compute the syndromes, run Berlekamp–Massey, Chien and Forney. Everything below is the model's actual output.

Codeword	1 2 3 4 5 6 7 8 9 9 8 9 3 10 0
Received	1 2 4 4 5 13 7 8 9 9 8 13 3 10 0
Syndromes $S_{0..5}$	8 15 1 15 9 8
Errors found	3
Positions the decoder located	[2, 5, 11]
Positions actually corrupted	[2, 5, 11]
Decode succeeded	yes
Recovered codeword equals the transmitted one	yes
Recovered message	1 2 3 4 5 6 7 8 9

How this is got wrong. By declaring success when the syndromes go to zero and stopping there. A decoder that miscorrects also produces zero syndromes — it has landed on a *different valid codeword*. The only defence is to re-check after correction and to bound the number of corrections by t ; the model does both, which is why the next section's measurement of the beyond- t region is meaningful.

Undergraduate review · **Why the syndromes are what they are.** The syndrome S_j is the received polynomial evaluated at α^{fcr+j} . Every valid codeword is divisible by the generator, whose roots are exactly those powers, so a clean codeword gives all-zero syndromes. What survives the evaluation is therefore only the error polynomial — the syndromes depend on the errors and *not at all* on the data. That single fact is what makes bounded distance decoding possible, and it is worth verifying on the numbers above: change the message and re-run, and the syndromes do not move.

Table 212. The syndromes depend only on the error pattern — measured, not asserted

Message	Syndromes with the same three errors injected
[1, 2, 3, 4, 5, 6, 7, 8, 9]	8 15 1 15 9 8
[9, 8, 7, 6, 5, 4, 3, 2, 1]	8 15 1 15 9 8

Two different messages, the same injected error pattern, identical syndromes. They match.

X5.3 Berlekamp–Massey as a hardware recurrence

What Berlekamp–Massey is actually doing

	Step	Justification
1	The syndromes of an e -error pattern satisfy a linear recurrence of order e .	Each error contributes a geometric sequence d^j ; a sum of e geometrics satisfies an order- e recurrence.
2	The recurrence's characteristic polynomial is the error locator $\sigma(x)$.	Its roots are the inverses of the error positions, by construction.
3	So decoding reduces to: find the shortest linear recurrence that generates the observed syndrome sequence.	This restatement is the entire content of the algorithm.
4	Berlekamp–Massey finds it in $2t$ iterations, each a discrepancy computation and a polynomial update.	Equivalent to the extended Euclidean algorithm on x^{2t} and $S(x)$; the two are the standard alternatives in RTL.
5	Each iteration costs $O(t)$ field multiplies, so the whole algorithm is $O(t^2)$.	For KP4, $t = 15$, so the key equation is a few hundred multiplies — small beside the 544-symbol Chien search.
6	The iteration count is data independent.	Which is why the decoder has a fixed latency, which is why the PCS can be built around it. A variable-latency decoder would need elastic buffering through the whole receive path.

Table 213. Parameter sets and the work each implies

Field	Code	t	Rate	BM multiplies $\approx 2t^2$	Chien positions	Codeword bits	Parity bits
$GF(2^4)$	RS(15,9)	3	0.6	18	15	60	24
$GF(2^8)$	RS(255,239)	8	0.9373	128	255	2,040	128
$GF(2^{10})$	RS(544,514)	15	0.9449	450	544	5,440	300
$GF(2^{10})$	RS(528,514)	7	0.9735	98	528	5,280	140

RS(544,514) is the KP4 code used by 100G-per-lane Ethernet; RS(528,514) is the earlier Clause 91 code. The rate difference is small and the difference is not.

X5.4 Beyond t : the region a careless test never visits

A bounded-distance decoder corrects up to t errors. What it does with $t+1$ or more is the interesting question, because that is the region a real link visits and a lazy test never does. Three outcomes are possible: detect the failure and flag it, miscorrect to a wrong codeword silently, or leave the word unchanged. The measurement below counts all three.

Table 214. RS(15,9), $t = 3$: fraction of words returned correct as the injected error count is swept

Errors injected	Region	Recovered exactly	Trials	Fraction	Note
1	$\leq t$	400	400	1	measured
2	$\leq t$	400	400	1	measured
3	$\leq t$	400	400	1	measured
4	$> t$	0	400	0	measured

Errors injected	Region	Recovered exactly	Trials	Fraction	Note
5	$> t$	0	400	0	measured
6	$> t$	0	400	0	measured

400 random trials per row, seeded. The first three rows must be 1.000 — that is the code's guarantee. The rows below must **not** be 1.000, and a test suite that only ever injects zero or one error cannot tell a working decoder from one that returns its input unchanged.

This is the false-green that RS test benches actually produce. Inject zero errors, check the output equals the input, declare the decoder verified. A decoder consisting of a wire passes. The repository's measurement function refuses to report a pass from a zero-error trial at all, and its test suite deliberately breaks the syndrome computation and asserts that the measurement *notifies*. A verification plan for an FEC block must state, for every test, what broken implementation that test would catch.

Problem. Why can a decoder that corrects a word with more than t errors be worse than one that flags the word as uncorrectable?

Solution. Because a miscorrection is a *silent* data error and an uncorrectable flag is an *announced* one. Upper layers are built to handle the announcement: Ethernet marks the frame errored and it is discarded and retransmitted; storage returns a read failure and the host retries or reconstructs from parity. There is no layer that handles data that is wrong and claims to be right. In a system with an outer code — the concatenated structure of 802.3dj — the point is sharper still: the outer decoder can use the inner decoder's uncorrectable flag as an **erasure**, and an erasure costs one parity symbol where an unflagged error costs two. **Detection is worth roughly twice as much as the same amount of correction**, which is why real decoders spend logic on checking their own output.

Problem. KP4 is RS(544,514) over $\text{GF}(2^{10})$, yet $544 > 2^{10}-1$ is false — 544 is comfortably below 1023. What is the code called when n is smaller than the natural length, and what does the shortening cost?

Solution. It is a **shortened** Reed–Solomon code: take the RS(1023,993) code and fix the leading 479 message symbols to zero, then decline to transmit them. Both ends know they are zero, so the decoder simply treats the received word as having 479 leading zeros. Shortening costs *nothing* in minimum distance — $d = n-k+1 = 31$ either way, so t remains 15 — and it costs nothing in the decoder except that the Chien search may be restricted to the 544 positions that are actually transmitted, which is a saving.

The one thing it does change is the false-detection probability: a shortened code has fewer codewords in the same space, so a random error pattern is *less* likely to land on a valid codeword. Shortening therefore improves miscorrection statistics slightly, for free.

X5.5 From symbol errors to the number the customer reads

A datasheet quotes post-FEC frame loss ratio. Getting there from a raw bit error rate is a short calculation with one dangerous assumption, and it is worth doing explicitly because the intermediate quantities are the ones a compliance argument is made of.

Table 215. Pre-FEC BER to post-FEC codeword failure rate, RS(544,514), assuming independent bit errors

Pre-FEC BER	Symbol error prob.	Mean symbol errors per codeword	P(codeword uncorrectable)	Uncorrectable codewords per second at 53.125 GBd
0.001	0.00996	5.42	1.5×10^{-4}	1494
3.0×10^{-4}	0.003	1.63	2.1×10^{-11}	2.1×10^{-4}
1.0×10^{-4}	1.00×10^{-3}	0.544	$< 10^{-16}$	—
3.0×10^{-5}	3.00×10^{-4}	0.163	1.1×10^{-16}	1.1×10^{-9}
1.0×10^{-5}	1.00×10^{-4}	0.0544	1.1×10^{-16}	1.1×10^{-9}

The waterfall between the first and last row spans many orders of magnitude for a factor of a hundred in input BER. That steepness is what FEC buys and it is why link margin is measured in fractions of a dB.

Graduate · The dangerous assumption, named. Every number in that table assumes bit errors are independent. They are not, for the reason Part X4 measured: DFE error propagation makes short bursts. Bursts are partly *good* for a symbol code — ten wrong bits inside one symbol cost one symbol — and partly bad, since a burst straddling a boundary costs two. The net effect depends on the burst-length distribution, which is a property of the receiver, not of the channel. **This is why compliance standards specify a stressed-input test rather than a BER number:** the test reproduces the error statistics, and no scalar summary of the channel can. When a customer asks for post-FEC performance, the honest answer names the burst model used, and a design house that can produce that model has something to sell beyond the RTL.

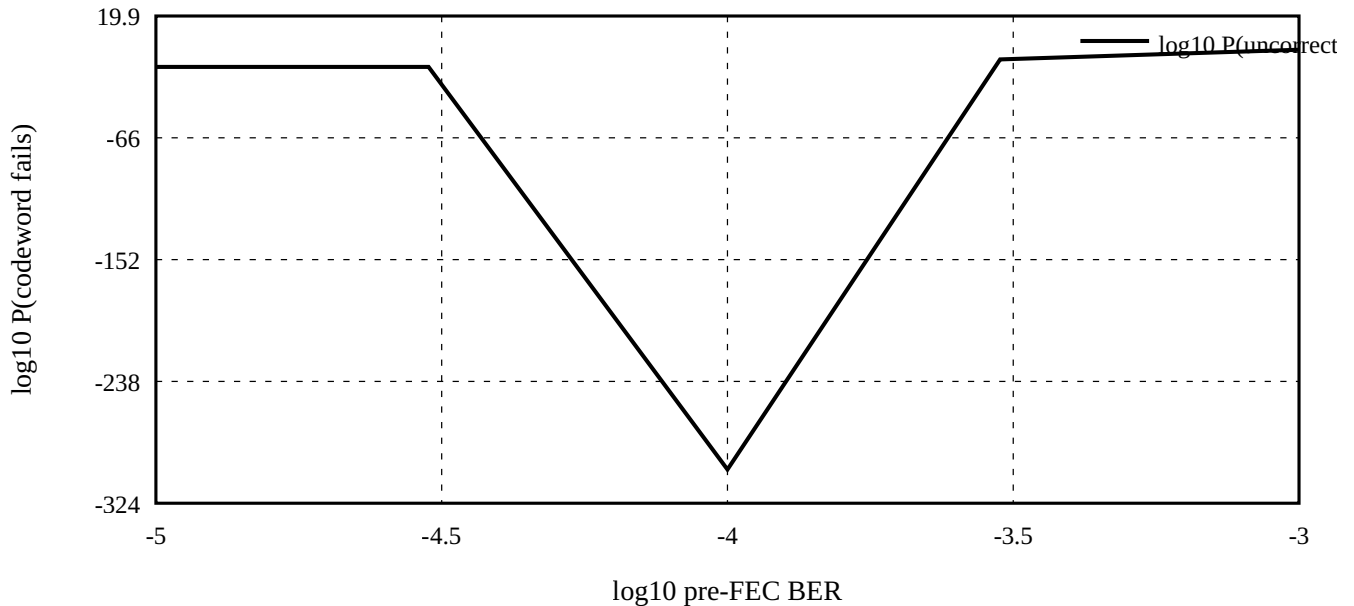
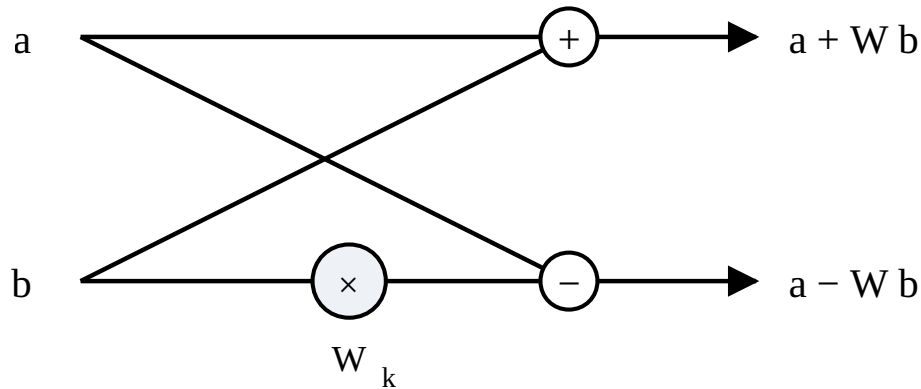


Figure 29. The FEC waterfall for RS(544,514), computed from the binomial tail. Note the slope: near the knee, a 1 dB change in SNR moves the output by many decades.

X6. The FFT as a Hardware Block, Worked



One radix-2 butterfly: one complex multiply, two complex adds.

An N -point FFT is $(N/2)\log_2 N$ of these, and the wiring between them is the hard part.

Figure 30. The radix-2 decimation-in-time butterfly.

X6.1 The operation count, and why it is the wrong cost model

Table 216. Complex multiplies for the DFT and for radix-2 / radix-4 FFTs

N	DFT: N^2	Radix-2: $(N/2)\log_2 N$	Speed-up	Radix-4 (approx.)	Memory accesses $\approx N\log_2 N$
64	4,096	192	21	144	384
256	65,536	1,024	64	768	2,048
1,024	1,048,576	5,120	204	3,840	10,240
4,096	16,777,216	24,576	682	18,432	49,152
16,384	268,435,456	114,688	2,340	86,016	229,376
65,536	4,294,967,296	524,288	8,192	393,216	1,048,576

The last column is the one that decides the architecture. Multiplies are cheap in a modern process; moving $N\log_2 N$ complex words in and out of memory is not.

Worked example 14. Multiplies are not the bottleneck — memory bandwidth is

Given. A 4096-point complex FFT, 16-bit I and Q, to be completed in 10 μ s. One multiplier can do 1 complex multiply per cycle at 500 MHz; an SRAM port can do one 32-bit access per cycle at the same rate.

Method. Compute the two requirements separately and compare them with what one instance of each resource supplies.

Complex multiplies	24,576
Multiplier-cycles available in 10 μ s per multiplier	5,000
Multipliers needed	5
Memory accesses (read 2, write 2 per butterfly)	98,304
SRAM-port-cycles available per port	5,000
SRAM ports needed	20
Ratio ports : multipliers	4

How this is got wrong. By sizing the block from the multiply count, which is the number every textbook gives. The memory requirement is four times larger and it is the one that will not be met by adding a cheap resource, because an SRAM with four times the ports is not four times the area — it is worse. The architectural answers are all about memory: in-place computation to halve the storage, multi-bank schemes so several butterflies read in parallel without conflict, and higher radix to do more arithmetic per memory visit. **Radix-4's real attraction is that it halves the number of memory passes**, not that it saves 25 % of the multiplies.

X6.2 Three architectures and where each belongs

Table 217. FFT architectures

Architecture	Throughput	Area	Latency	Where it belongs
Single butterfly + memory	1 butterfly / cycle	smallest	$N \log_2 N / 2$ cycles	Control-plane spectral analysis; anything not on the critical path
Pipelined (SDF / SDC)	1 sample / cycle, streaming	$\log_2 N$ butterflies + N words of delay	$\approx N$ cycles	OFDM receivers, channelisers — anything with a continuous stream
Parallel / SSR	P samples / cycle	$P \times$ the pipeline	$< N/P$	When the sample rate exceeds the clock, which is the normal case above 1 GS/s
Fully parallel (spatial)	one transform / cycle	$N \log_2 N / 2$ butterflies	$\log_2 N$ stages	Small N only; radar pulse compression, some optical DSP

Graduate · Why the streaming architectures are named after their commutators rather than their butterflies. Single-path delay feedback (SDF) and single-path delay commutator (SDC) differ in how the delay line is arranged around the butterfly, and that arrangement — not the arithmetic — determines the total memory, which is the dominant cost. An N -point radix-2 SDF pipeline needs exactly $N-1$ words of delay in total, distributed as $N/2, N/4, \dots, 1$ across the stages. That geometric distribution is why the first stage's delay line is often a separate SRAM and the last few are registers: they are the same structure at wildly different sizes, and one implementation choice does not fit all of them. **An HLS-generated FFT that uses the same memory style throughout is leaving a large amount of area on the table**, and this is one of the clearest places where a hand-written IP block still beats a generated one.

Table 218. Resources of a radix-2 SDF pipeline

N	Stages	Total delay words	Butterflies	Real multipliers (3-mult complex)	Largest single delay line
64	6	63	6	18	32

N	Stages	Total delay words	Butterflies	Real multipliers (3-mult complex)	Largest single delay line
256	8	255	8	24	128
1,024	10	1,023	10	30	512
4,096	12	4,095	12	36	2,048
16,384	14	16,383	14	42	8,192

The last column is why the first stage dominates the floorplan.

X6.3 Fixed point in the FFT: scaling schedules

Each radix-2 stage can double the magnitude of the data. Over $\log_2 N$ stages that is a growth of N , so a 4096-point transform can grow its input by 12 bits. Three strategies exist and they give measurably different results.

Table 219. Measured SQNR and measured magnitude growth of a 256-point 16-bit FFT under three scaling schedules, 8 random trials each

Schedule	Mean SQNR (dB)	Worst SQNR (dB)	Measured peak growth $\times$	Extra integer bits the growth demands
Scale every stage	0.4978	0.4349	0.0466	0
Conditional (block floating point)	0.08273	0.06056	1.12	0.164
No scaling — grow the word	0.4426	0.3704	13.3	3.73

The growth column is measured on a noise-like input, so it is far below the worst-case bound of N : a random input does not align in phase. **The measured figure is not a bound** — a single tone at a bin centre would reach the bound, and a design justified by this column alone would overflow on the first sine wave a customer applied. Unconditional scaling is safe and loses precision every stage. Block floating point recovers most of it at the cost of a magnitude detector and an exponent that must travel with the data — **and that exponent is an interface**, which a specification frequently forgets to mention.

The block-floating-point exponent is part of the output. A block that returns a mantissa array and keeps the exponent internal has returned meaningless numbers. This is a real and recurring integration bug: the FFT passes its own tests, in which the reference is scaled the same way, and fails in the system, where the next block assumes a fixed scale. **Any scaling that depends on the data must appear in the port list.**

Problem. An OFDM receiver uses a 2048-point FFT. The input is an ADC output with 10 effective bits. How many bits should the FFT carry internally, and why is ‘ $10 + \log_2 2048 = 21$ ’ the wrong answer?

Solution. It is wrong in both directions. It is too pessimistic because the worst-case growth of N assumes every input conspires to align in phase at one output bin; for a noise-like OFDM signal the growth is closer to $\sqrt{N}$, about 5.5 bits, and unconditional scaling by 2 per stage over-attenuates such a signal by the same factor. It is too optimistic because it accounts only for range and not for the *accumulated quantisation noise* of the twiddle multiplies, which adds roughly half a bit per stage of precision loss. The correct procedure is Part X1's: state the end-to-end metric (for OFDM, the EVM the constellation demands), then sweep the internal word length against that metric and read off the knee. **The answer is a measurement, and it depends on the signal statistics, which is why an FFT IP block's datasheet must state the input statistics its SQNR figure was measured with.**

X6.4 Twiddle factors: the quiet memory

Table 220. Twiddle-factor ROM: full table vs. one-eighth with symmetry, 16-bit I and Q

N	Full entries	Full ROM (kB)	$\frac{1}{8}$ entries	$\frac{1}{8}$ ROM (kB)	Saving
256	128	0.5	33	0.129	3.88
1,024	512	2	129	0.504	3.97
4,096	2,048	8	513	2	3.99
16,384	8,192	32	2,049	8	4
65,536	32,768	128	8,193	32	4

The eighth-table exploits the symmetries of the unit circle; recovering the other seven octants costs a few multiplexers and a conditional negate and swap. For $N = 65\,536$ the difference is between a ROM that fits on a corner of the block and one that dominates it.

Undergraduate review · The symmetries, so the multiplexer logic makes sense. $W_N^k = e^{-2\pi i k/N}$ traces the unit circle clockwise. Cosine is even and sine is odd about the real axis, giving the lower half from the upper half; the quarter-turn identity exchanges sine and cosine, giving the second quadrant from the first; and the eighth-turn identity does the same within a quadrant. So the ROM stores angles from 0 to $\pi/4$ and the address decoder computes three bits — which octant — that control a swap of the I and Q outputs and two sign inversions. The whole recovery is combinational and adds perhaps two gate delays, against a ROM eight times smaller.

Problem. Instead of a ROM, twiddles can be generated by a CORDIC rotator or by a recursive oscillator $w[n+1] = w[n] \cdot W$. When is each appropriate?

Solution. The recursive oscillator is the cheapest — one complex multiply per twiddle — and is **unusable in a fixed-point pipeline** because it is a marginally stable recursion: quantisation error accumulates in both magnitude and phase, and over the thousands of twiddles of a large transform the magnitude drifts away from unity. It is acceptable only with periodic re-seeding from a small ROM, which is a hybrid rather than a replacement. CORDIC is exact to its iteration count and costs only shifts and adds, making it attractive in FPGA fabric where multipliers are a scarce hard resource but LUTs are plentiful; it costs latency, which a pipelined design can absorb. **The ROM wins in ASIC**, where a small compiled ROM is dense and the alternative spends timing budget. As so often, the right answer is set by which resource the target technology makes expensive, not by any property of the algorithm.

X7. Power and Energy, Worked

A design house is asked for three numbers about power: how much the block burns, how much of that is unavoidable, and what a customer can do about it. All three are computable, and a vendor who can produce them with their assumptions stated is selling something the competitor who quotes a single milliwatt figure is not.

X7.1 The three terms

Where the joules go

	Step	Justification
1	Charging a node of capacitance C to V takes CV^2 from the supply, of which $CV^2/2$ is stored and $CV^2/2$ is dissipated in the pull-up.	<i>Integrate iv over the transient; the result is independent of the resistance, which is why making transistors faster does not save switching energy.</i>
2	Discharging dissipates the stored half in the pull-down.	<i>So a full cycle costs CV^2, and a $0 \rightarrow 1 \rightarrow 0$ pair is one cycle.</i>
3	$P_{\text{dyn}} = \alpha CV^2 f$, with α the probability a node switches per clock.	<i>Rate times energy. α is the only term the RTL designer controls directly, and it is the term power estimates get wrong.</i>
4	P_{short} arises while both devices conduct during a transition; it scales with input slew.	<i>Typically 5–15 % of dynamic. Controlled by slew limits, not by architecture.</i>
5	$P_{\text{leak}} = V \cdot I_{\text{off}}$, with $I_{\text{off}} \propto e^{-V_{\text{th}}/(nkT/q)}$.	<i>Subthreshold conduction. Exponential in threshold and in temperature, which is why leakage is a thermal-runaway risk and dynamic power is not.</i>

Worked example 15. A gate-level power estimate from first principles

Given. A block with 200 000 gates, average switched capacitance 1 fF per gate, $V = 0.8$ V, $f = 1$ GHz, activity factor $\alpha = 0.15$. Clock network adds 30 % and its α is 1.

Method. Apply $\alpha CV^2 f$ to the logic and to the clock separately, because their activity factors differ by nearly an order of magnitude and averaging them is the commonest source of a wrong estimate.

Logic dynamic power	19200 mW
Clock network capacitance (30 % of logic)	60000 pF
Clock dynamic power ($\alpha = 1$)	38400 mW
Clock as a fraction of total dynamic	66.7 %
Total dynamic	57600 mW
Energy per cycle	57600 pJ

How this is got wrong. By applying one activity factor to the whole design. The clock toggles every cycle by definition; logic in a typical datapath toggles on ten to twenty per cent of cycles. An estimate that uses $\alpha = 0.2$ uniformly understates the clock by 5× and, since the clock here is two thirds of the total, understates the block by a factor approaching two. **Clock gating is the first power optimisation for exactly this reason**, and its benefit is visible in this table before any tool is run.

X7.2 Voltage scaling: the quadratic and its limit

Table 221. Voltage scaling with an α -power-law delay model ($V_{th} = 0.30$ V, $\alpha = 1.3$), normalised to 1.0 V

V_{dd} (V)	Relative delay	Relative f_{max}	Relative power at f_{max}	Relative power at fixed f	Relative energy per operation
1	1	1	1	1	1
0.9	1.1	0.9093	0.7366	0.81	0.8908
0.8	1.239	0.8071	0.5166	0.64	0.7929
0.7	1.449	0.6902	0.3382	0.49	0.71
0.6	1.805	0.554	0.1994	0.36	0.6499
0.5	2.548	0.3924	0.0981	0.25	0.6371
0.4	5.02	0.1992	0.03187	0.16	0.8032

The last column is the one that matters for a battery: energy per operation falls monotonically as the voltage drops, which is why near-threshold operation is attractive, while the frequency column says what it costs in throughput. Leakage is deliberately excluded here and X7.3 puts it back.

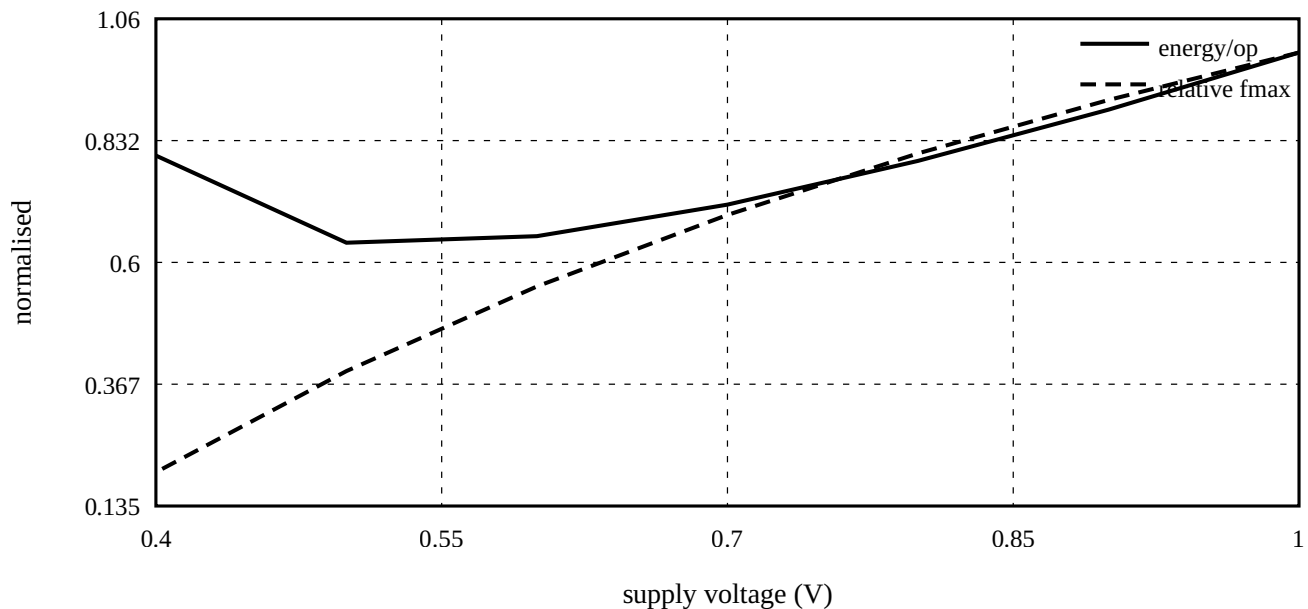


Figure 31. Energy per operation and maximum frequency against supply. The curves cross; where they cross is a design decision, not a fact.

Graduate Parallelism converts the voltage curve into a genuine win, and this is the single most important architectural consequence of the table above. Suppose a block must deliver throughput T . Run one copy at 1.0 V, or two copies at the voltage that halves the frequency. The two copies have twice the capacitance but each switches half as often per unit of work, so dynamic power is $2 \times (V'/V)^2 \times \frac{1}{2} = (V'/V)^2$ of the original. From the table, halving the frequency permits roughly a 0.65 V supply, giving about 0.42× the power for the same throughput at rather more than twice the area. **This calculation is the reason modern chips are wide and slow rather than narrow and fast**, and it is worth being able to do it on a whiteboard.

X7.3 Leakage, and why it changes the shape of everything

Table 222. Subthreshold leakage against temperature, normalised to 25 °C ($V_{th} = 0.30$ V, $n = 1.3$), and its share of a block that is 20 % leakage at room temperature

T (°C)	nkT/q (mV)	Relative leakage	Leakage share of total (%)
0	23.54	0.44	9.9
25	25.69	1	20
50	27.85	2	33.4
75	30	3.63	47.6
100	32.15	6.08	60.3
125	34.31	9.54	70.5

Computed from the subthreshold exponential alone; DIBL and gate leakage would steepen it further. The share column shows why a block that is a fifth leakage on the bench can be nearly half leakage in a hot enclosure.

Leakage and temperature form a positive feedback loop. Leakage raises power, power raises junction temperature, and higher temperature raises leakage exponentially. Below a certain thermal resistance the loop has a stable operating point; above it there is none, and the part runs away. This is not a theoretical concern in high-leakage processes: it is why datasheets specify a maximum junction temperature *and* a maximum case-to-ambient thermal resistance, and why a customer who improves cooling is not merely buying margin but may be buying stability. An IP vendor should state the leakage current at the maximum rated temperature, not at 25 °C.

Table 223. Power reduction techniques, what each attacks, and what it costs

Technique	Attacks	Typical saving	Cost	Who decides
Clock gating	α on the clock tree	20–40 %	A few gates; timing on the enable	The RTL designer — automatic gating catches most of it
Operand isolation	α in unused datapath	5–15 %	Muxes in the datapath	RTL designer
Multi- V_{th} cells	Leakage	2–5× on leakage	Slower cells on non-critical paths	Implementation, automatic
Power gating	Leakage, to near zero	>90 % when idle	Switch area, IR drop, wake latency, state retention	Architecture — it changes the reset and verification plan
DVFS	V^2f	2–5× at low load	Regulator, level shifters, characterisation at every operating point	System, with IP support
Architectural parallelism	V^2 via lower f	2–3×	Area, latency	Architecture, early — it cannot be retrofitted
Memory banking / sub-banking	C per access	10–30 % of memory power	Decode logic, floorplan	Architecture

Worked example 16. The savings are not additive — a worked composition

Given. A block is 100 mW: 60 mW clock, 25 mW logic, 15 mW leakage. Clock gating removes 60 % of clock power; multi- V_{th} removes 60 % of leakage; operand isolation removes 30 % of logic power.

Method. Apply each to its own term and re-total. Then compute what a naive addition of the three quoted percentages would have predicted.

Clock after gating	24 mW
Logic after isolation	17.5 mW
Leakage after multi- V_{th}	6 mW
New total	47.5 mW
Actual saving	52.5 mW
Actual saving (%)	52.5 %
Naive sum of the three quoted percentages	150 %
Clock share before / after	60% / 50.5%

How this is got wrong. By adding percentages that are percentages of different things. Each technique reduces one term, and after it has done so that term is a smaller share, which is why the second technique applied always returns less than its headline. The corollary is a procedure: **always attack the largest term first**, re-profile, and only then choose the next technique. A power plan written as a list of techniques without a breakdown to apply them to is not a plan.

Problem. A customer reports that your block consumes $2\times$ the datasheet figure. List the measurements that distinguish the possible causes, in the order you would make them.

Solution. 1. Compare activity factors. The datasheet figure was measured with some stimulus; ask what theirs is. A block specified on random data and used on highly correlated data — or the reverse — can differ by a factor of two with nothing wrong. This is the most common cause and the cheapest to check. **2. Check the clock-gating enables are actually asserting**, by toggling coverage or by a scope on the gated clock; an integration that ties an enable high disables the largest single saving. **3. Split dynamic from leakage** by measuring at two temperatures: leakage doubles over 40 °C or so and dynamic does not, so two points separate the terms. **4. Check the supply** — a 10 % overvoltage is a 21 % dynamic increase. **5. Only then suspect the block.** Note that the first four are questions about the customer's system, and asking them requires having recorded your own measurement conditions; a datasheet power figure without its stimulus, voltage and temperature is unfalsifiable and therefore worthless in exactly this conversation.

X8. The Mathematics of Verification, Worked

Three questions decide a verification plan: how long must random stimulus run to reach a coverage target, how many bugs remain when the discovery rate flattens, and what a bug costs at each stage. All three have arithmetic answers. None of them is a substitute for judgement, but a plan that contradicts the arithmetic needs a reason.

X8.1 Coverage closure is a coupon-collector problem

How long random stimulus takes to hit every bin

	Step	Justification
1	Suppose there are n coverage bins, each hit with equal probability $1/n$ by an independent random transaction.	<i>The idealisation. Real bins are neither equiprobable nor independent, and X8.2 measures what that does.</i>
2	After the k th distinct bin is hit, the probability that the next transaction hits a new bin is $(n-k)/n$.	<i>Uniformity.</i>
3	So the expected wait for the next new bin is $n/(n-k)$.	<i>Mean of a geometric distribution.</i>
4	Total expected transactions = $n \sum_{k=1..n} 1/k = nH_n \approx n(\ln n + 0.577)$.	<i>Sum the waits; H_n is the harmonic number.</i>
5	For 90 % of the bins, the sum truncates and the cost is only $n \ln 10 \approx 2.3n$.	<i>Σ from 1 to $0.9n$ of $1/(n-k)$. This is the whole shape of every coverage curve you have ever seen: cheap to 90 %, expensive to 100 %.</i>

Table 224. Expected random transactions to reach a coverage target, equiprobable bins

Bins n	To 90 %	To 99 %	To 100 %	Cost of the last 10 % as a multiple of the first 90 %
100	230	460	518	2.25
1,000	2,302	4,605	7,485	3.25
10,000	23,025	46,051	97,876	4.25
100,000	230,258	460,517	1,209,014	5.25

The last column is roughly constant at about four, independent of n . The final ten per cent of coverage costs three times what the first ninety did, every time, and that is before the bins turn out to be unequal.

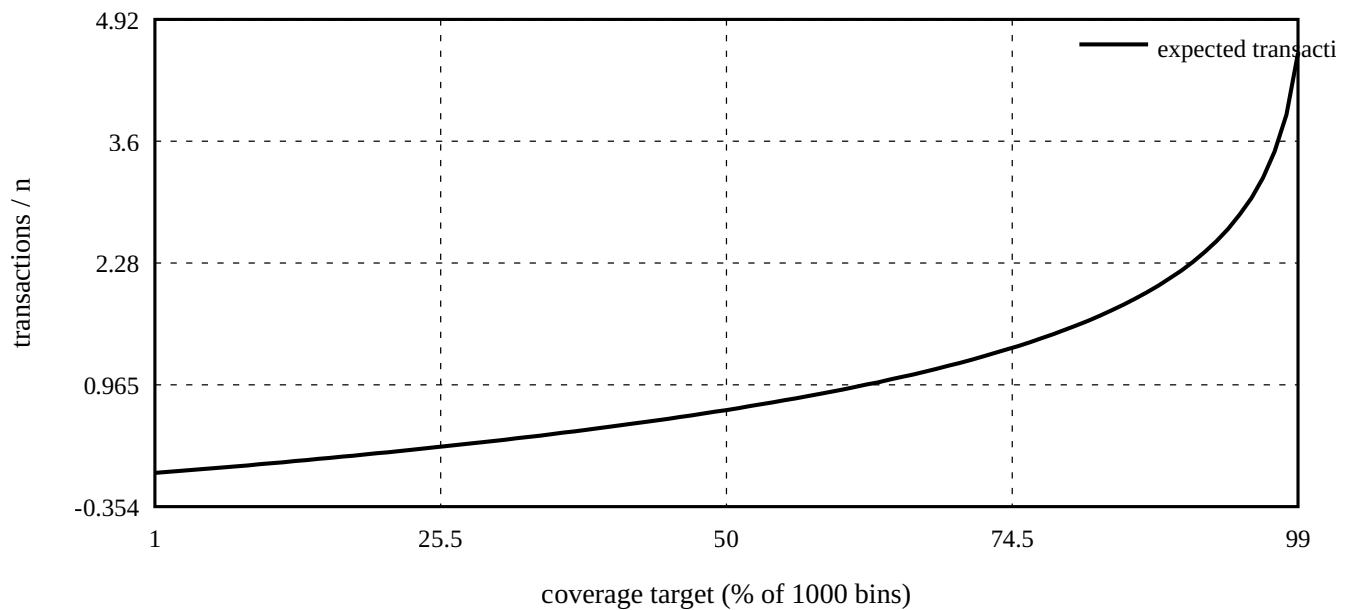


Figure 32. The coupon-collector curve for 1000 bins. It is not that the last bins are harder to hit by design; they are harder to hit because there are fewer of them left to hit by accident.

Worked example 17. Sizing a regression from a coverage model

Given. A block has 8000 coverage bins. One simulation runs 40 000 transactions and takes 6 minutes. The target is 99 % coverage. 200 simulations can run in parallel.

Method. Convert the bin count to a transaction count with the coupon-collector estimate, then to simulations, then to wall-clock time.

Transactions to 99 %	36,841
Simulations needed	1
Serial machine-time	0.1 hours
Wall clock on 200 in parallel	< 1 run — the pool is not the limit
Transactions to 100 % (harmonic sum)	76,515
Simulations for 100 %	2

How this is got wrong. By concluding that random stimulus will therefore close coverage. The model assumes every bin is reachable by random stimulus with equal probability. In practice a handful of bins require a specific sequence — a FIFO full while a retry is in flight and a parity error arrives — whose random probability is not $1/n$ but 10^{-9} . **Those bins never close randomly and no amount of compute changes that;** they close with directed tests or constrained scenarios. The arithmetic above tells you how much compute to buy for the easy 99 %, and identifying the remaining bins as requiring direction is the actual verification engineering.

Table 225. Measured transactions to 99 % coverage of 1000 bins as the bin probabilities are made unequal

Skew parameter	Most likely : least likely bin	Transactions measured	Multiple of the equiprobable estimate
0	1	6,000	1.3
0.5	1.65	6,000	1.3
1	2.72	6,000	1.3
1.5	4.47	8,000	1.74

Skew parameter	Most likely : least likely bin	Transactions measured	Multiple of the equiprobable estimate
2	7.37	8,000	1.74

An exponential weighting with the stated skew, measured by simulation with seed 9. Even a 7:1 spread between the most and least likely bin multiplies the cost noticeably; real constraint solvers produce far larger spreads than that, which is why **coverage convergence is slower than any model predicts and why the remedy is to reshape the constraints, not to buy machines.**

X8.2 Bug discovery curves and when to stop

Bugs are found at a rate that decays roughly exponentially with effort, because the easy ones go first. Fitting that decay gives an estimate of how many remain — an estimate that is crude, defensible, and far better than the alternative, which is a feeling.

Table 226. An exponential bug-discovery model: 240 latent bugs, decay constant 0.18 per week

Week	Cumulative found	Bugs found this week	Estimated remaining	Remaining (%)
2	72	30.1	167	69.8
4	123	21	116	48.7
6	158	14.7	81	34
8	183	10.2	56	23.7
10	200	7.14	39	16.5
12	212	4.98	27	11.5
16	226	2.43	13	5.61
20	233	1.18	6	2.73

The parameters are illustrative; the method is not. Fit the two parameters to your own weekly find-rate data and the extrapolation is a real estimate with a real confidence interval.

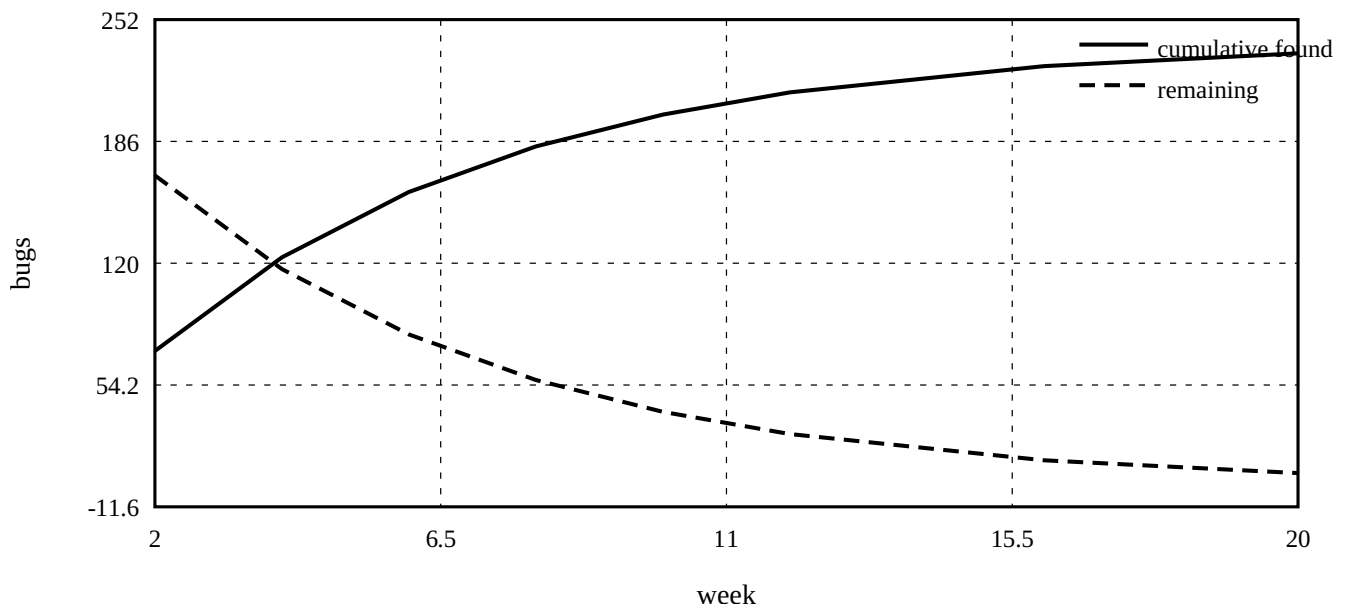


Figure 33. The two curves sum to a constant by construction. The decision to tape out is the decision that the lower curve is acceptable.

The find-rate falling is ambiguous and this is where projects deceive themselves. A falling rate means either that the bugs are running out or that the *testing* is running out — the same tests re-run on the same stimulus find nothing new whether or not bugs remain. The two are distinguished by a measurement, not by an argument: **introduce new stimulus or a new checker and see whether the rate recovers.** Mutation testing does this systematically by injecting known faults and asking what fraction the existing suite catches; a suite that catches 60 % of injected faults is telling you your find-rate curve has flattened for the wrong reason. This repository runs exactly such a hunt against its own checks.

Table 227. Relative cost of finding one bug at each stage, and the cost of letting 2 % of 240 bugs reach that stage

Stage found	Relative cost	vs. spec review	Cost of 2 % escaping to here (relative units)
Specification review	1	—	4.8
Modelling / C++ reference	3	3×	14.4
Block-level simulation	10	10×	48
Chip-level simulation	40	40×	192
Emulation / FPGA prototype	100	100×	480
First silicon bring-up	1,000	1000×	4800
Customer escape	10,000	10 000×	48000

*The multipliers are the industry's rule of thumb and vary by an order of magnitude between sources; the **shape** is what is robust. It is the reason a modelling team that catches specification ambiguities pays for itself many times over, and the reason design houses invest in the left-hand rows.*

Worked example 18. Is another month of verification worth it?

Given. At week 12 the model above estimates 28 bugs remaining. A month of verification costs 4 engineer-months. Each bug that reaches silicon costs, on average, 1 engineer-month of debug; a respin costs 40 engineer-months plus three months of schedule, and the probability of a respin is roughly the probability that at least one remaining bug is functional-critical, estimated at 5 % per remaining bug.

Method. Compute the expected cost of stopping now against the expected cost of continuing, using the discovery model to say how many bugs the extra month removes.

Bugs remaining at week 12	27
Bugs remaining at week 16	13
Bugs removed by the extra month	14
P(no respin) if we stop now	0.242
P(no respin) if we continue	0.501
Expected respin cost now (engineer-months)	30.33
Expected respin cost after one more month	19.96
Saving from the extra month	10.37
Cost of the extra month	4
Verdict	continue

How this is got wrong. By treating the model's parameters as known. The decay constant is fitted from noisy weekly counts and its uncertainty propagates into the remaining-bug estimate, which sits inside an exponent. The right use of this calculation is not to produce a verdict but to find out *which assumption the verdict is sensitive to*: here it is the 5 % criticality figure, and that figure can be improved by classifying the bugs already found, which is free. **Do the sensitivity analysis and go and measure the sensitive term.**

Problem. Your regression has 100 % code coverage and 98 % functional coverage, and has found no new bug in three weeks. A reviewer asks for evidence that the suite would catch a bug if one existed. What do you provide?

Solution. Code coverage is the weakest of the three and answering with it is the mistake: it shows lines executed, not behaviour checked, and a testbench with the checkers disabled reaches 100 %. The evidence a reviewer should accept is **fault injection**: take the design, introduce a set of realistic mutations — invert a comparison, drop a reset term, change a FIFO threshold by one, delete a wait state — and report the fraction the existing suite catches, with the escapes listed and explained. A suite that catches 95 % of mutations has made a falsifiable claim about itself. Supplement it with **checker coverage** (did each assertion ever fire in a passing run? an assertion that never even evaluated its antecedent is not checking anything) and with the **bug-find-rate recovery test** above: add genuinely new stimulus and see whether the rate moves. **Every one of these is a measurement of the testbench rather than of the design**, which is precisely what was asked for.

Problem. Formal verification proves a property over all inputs. Why does a project still simulate?

Solution. Three reasons, in decreasing order of how often they bite. **Capacity**: proofs complete on control logic, arbiters, FIFOs and protocol interfaces, and commonly do not on a datapath with wide multipliers or on a whole subsystem; what comes back is a bounded proof, which is a very good simulation and not a proof. **Specification**: a formal tool proves the properties you wrote. A missing property is an unproven behaviour, and there is no coverage metric for ‘properties I did not think of’ — though mutation coverage of the property set comes closest. **Environment**: proofs need constraints to exclude illegal inputs, and an over-constraint silently removes the case that would have failed — the formal equivalent of a false-green, and the one that has ended more than one post-silicon investigation. The two techniques are complementary in a specific way worth remembering: **formal is strong where state space is deep and narrow, simulation is strong where it is shallow and wide.**

X9. Computer Arithmetic Beyond the Adder, Worked

Addition and multiplication are solved problems with known structures. Division, square root, and the transcendental functions are not: each admits several algorithms with genuinely different latency, area and accuracy, and choosing among them is a recurring task for anyone specifying a datapath. This part computes the trade-offs instead of listing them.

X9.1 Adders: the latency–area curve, computed

Table 228. Adder structures: delay in gate levels and area in full-adder equivalents

Width n	Ripple delay	Kogge–Stone delay	Kogge–Stone area	Carry-select delay ($\sqrt{n}$ blocks)	Carry-select area
8	8	8	12	5	16
16	16	10	32	8	32
32	32	12	80	11	64
64	64	14	192	16	128
128	128	16	448	22	256

Delays are counted in two-input gate levels from the standard prefix constructions; they are architectural counts, not library timing. Kogge–Stone is logarithmic in delay and $n \log n$ in area and wiring — and the wiring, not the gates, is what makes it expensive in a real floorplan.

Graduate · Why the prefix-adder family is a design space rather than a single answer. Every parallel-prefix adder computes the same carry prefixes; they differ in how the prefix tree is shaped, and the shape trades three quantities against each other: logic depth, fan-out, and wiring tracks. Kogge–Stone minimises depth at maximum wire count; Brent–Kung minimises wires at twice the depth; Han–Carlson interleaves the two and sits between; Ladner–Fischer trades fan-out for wires. In a modern process **the wiring term usually dominates**, so the textbook's favourite (Kogge–Stone) is often not the synthesiser's choice, and a designer who insists on it by instantiation can make the block slower. The right procedure is to let the tool choose from its library and to check the result, not to specify the topology in the RTL.

X9.2 Multipliers: partial products and how to not add them

Table 229. Partial-product counts and reduction depth

Operand width n	Simple PP rows	Radix-4 Booth rows	Radix-8 Booth rows	Wallace 3:2 reduction levels (from radix-4)	Approx. full adders in the tree
8	8	5	4	3	24
16	16	9	7	4	86
24	24	13	9	5	187
32	32	17	12	6	326
53	53	28	19	7	890

Radix-8 Booth needs a $3\times$ multiple, which costs an adder and a carry chain up front; it pays only when the row saving outweighs that, which for $n \geq 32$ it usually does. The 53 row is the IEEE-754 double-precision significand.

Worked example 19. Why a multiplier ends in one wide adder

Given. A 32×32 multiplier built as radix-4 Booth encoding plus a carry-save reduction tree.

Method. Count the stages and see where the delay goes. Carry-save addition has *constant* delay per level regardless of width, because no carry propagates; only the final conversion from carry-save to binary needs a carry-propagate adder.

Booth rows	17
Reduction levels to reach 2 rows	6
Delay of the tree (≈ 2 gate levels per level)	12 gate levels
Final 64-bit carry-propagate adder (Kogge–Stone)	14 gate levels
Final adder as a fraction of total delay	53.8 %

How this is got wrong. By optimising the reduction tree and leaving the final adder to the synthesiser's default. The final adder is a third of the delay of the whole multiplier, and it is the one part that is a plain carry-propagate problem. It is also the reason **a multiply-accumulate is cheaper than a multiply followed by an add**: keep the product in carry-save form, feed it into the accumulator's reduction tree, and pay for exactly one carry-propagate adder at the end of the whole accumulation rather than one per operation. Any datapath that multiplies and then sums should be examined for this.

X9.3 Division and square root: digit recurrence against Newton

Table 230. Two families, and what each is for

	Digit recurrence (SRT)	Multiplicative (Newton–Raphson, Goldschmidt)
Produces	One radix- r digit per iteration	Doubles the number of correct bits per iteration
Iterations for 53 bits	27 at radix 4	About 4 from an 8-bit seed
Per-iteration cost	A small adder and a quotient-digit selection table	Two full multiplies
Exact remainder	Yes — available for free	No; needs a final correction step
Correct rounding	Straightforward from the remainder	Requires care ; this is where implementations go wrong
Latency	Long, predictable	Short if multipliers are already there
Reuses	Nothing	The existing multiplier array
Chosen when	Area matters, or the remainder is needed (integer division, modulo)	A fast multiplier already exists (every FPU)

The Pentium FDIV bug was a digit-selection table with five missing entries. An SRT divider chooses each quotient digit from a lookup indexed by truncated partial remainder and divisor; the table has redundant regions so that a slightly wrong estimate can be corrected later, which is exactly what makes the algorithm fast and exactly what makes a missing entry hard to find by random testing. The entries were unreachable for almost all operand pairs. The lesson for a verification plan is specific and permanent: **a table-driven arithmetic unit must be verified over the table's index space, not over the operand space.** Random operands reach the interesting table entries with probability near zero; enumerating the index space takes minutes.

Table 231. Iteration counts: Newton–Raphson against SRT

Result bits	Seed table bits	Newton iterations	Multiplies	SRT radix-4 iterations	SRT radix-8 iterations
24	8	2	4	12	8

Result bits	Seed table bits	Newton iterations	Multiplies	SRT radix-4 iterations	SRT radix-8 iterations
24	12	1	2	12	8
24	16	1	2	12	8
53	8	3	6	27	18
53	12	3	6	27	18
53	16	2	4	27	18
113	8	4	8	57	38
113	12	4	8	57	38
113	16	3	6	57	38

Newton's iteration count is logarithmic in the precision and the seed table's size trades directly against it — four extra table bits routinely remove an entire iteration, which is two multiplies. **This is the cheapest optimisation in the whole unit** and is frequently left on the table.

Problem. A block needs $y = a/b$ where b changes rarely and a changes every cycle. What is the right structure?

Solution. Not a divider. Compute $r = 1/b$ once, when b changes, and multiply every cycle. The reciprocal can take as long as it likes — a slow iterative unit, or even a microcoded sequence — because it is off the critical path, and the per-cycle cost collapses to one multiply. **The general principle is worth naming: look for the operand that changes slowly and move work onto it.** The same observation produces the coefficient-folding in a fixed-coefficient FIR, the precomputed tables in a modular-arithmetic unit, and the slow adaptation loop of a SerDes equaliser. The one caution is accuracy: $a \cdot (1/b)$ is not correctly rounded even when both operations are, so a unit claiming IEEE compliance cannot take this shortcut without a correction step.

X9.4 CORDIC: rotations without multipliers

Why the strange constants work

	Step	Justification
1	A rotation by θ is $(x\cos\theta - y\sin\theta, x\sin\theta + y\cos\theta)$.	<i>Definition.</i>
2	Factor out $\cos\theta$: the rotation becomes $\cos\theta$ times a shear with $\tan\theta$.	<i>Algebra. The scale factor is now separate from the shape.</i>
3	Choose θ_i with $\tan\theta_i = 2^{-i}$. Then the shear is two shifts and two adds.	This is the whole trick: restrict the allowed angles so the multiplication becomes a shift.
4	Any angle in the convergence range is reachable as $\Sigma\pm\theta_i$.	<i>Because $\theta_i < \Sigma_{j>i}\theta_j$, so the remaining steps can always correct an overshoot — the same redundancy argument that makes SRT division work.</i>
5	The accumulated scale $\Pi\cos\theta_i \rightarrow 0.607253$ is independent of the angle , because every step rotates by the same magnitude and only the sign varies.	<i>So it can be compensated once, at the end, or folded into surrounding coefficients for free.</i>

Table 232. Measured CORDIC accuracy against iteration count (rotation mode, 41 angles)

Iterations	Scale factor K	Max error in cos	Effective bits	Adds
8	0.60725911	0.00453	7.79	16
12	0.60725296	2.65×10^{-4}	11.9	24
16	0.60725294	2.17×10^{-5}	15.5	32

Iterations	Scale factor K	Max error in cos	Effective bits	Adds
20	0.60725294	9.02×10^{-7}	20.1	40
24	0.60725294	7.05×10^{-8}	23.8	48

One bit of accuracy per iteration, exactly as the theory predicts, and the scale factor converges quickly to its limit. The last column is the cost: two adds and two shifts per iteration and no multiplier anywhere.

Undergraduate review · The three CORDIC modes, and what each computes. In *rotation* mode the algorithm drives the angle accumulator to zero and the coordinates come out rotated — giving sine and cosine, and the complex rotation a mixer or an FFT twiddle needs. In *vectoring* mode it drives y to zero instead, and what accumulates is the original angle — giving arctangent and magnitude, which is exactly the operation a Cartesian-to-polar conversion, an AM detector or a phase detector performs. Substituting hyperbolic angles ($\tanh$ instead of $\tan$) gives a third mode that computes exponentials, logarithms and square roots, with some iterations repeated to preserve convergence. **One structure, three modes, most of the transcendental functions a DSP block needs** — which is why CORDIC survives in fabric where multipliers are the scarce resource.

X9.5 Floating point: what the standard actually requires

Table 233. Floating-point formats in current use

Format	Total bits	Exponent bits	Stored mantissa bits	Significand bits (with hidden 1)	Machine epsilon	Decimal digits
binary16 (half)	16	5	10	11	4.88×10^{-4}	3.31
bfloat16	16	8	7	8	0.00391	2.41
binary32 (single)	32	8	23	24	5.96×10^{-8}	7.22
TF32	19	8	10	11	4.88×10^{-4}	3.31
binary64 (double)	64	11	52	53	1.11×10^{-16}	16
FP8 E4M3	8	4	3	4	0.0625	1.2
FP8 E5M2	8	5	2	3	0.125	0.903

bfloat16 and *binary16* are the same width and differ entirely in how they split it: *bfloat16* keeps *binary32*'s exponent range and throws away precision, which is the right trade for neural-network training because the failure mode there is overflow of a gradient, not rounding of a weight.

Worked example 20. Why $0.1 + 0.2 \neq 0.3$, in bits

Given. IEEE-754 binary64 arithmetic.

Method. 0.1 and 0.2 are not representable: in binary they are infinite repeating fractions, so each is rounded to 53 significant bits. The sum of the two rounded values is then itself rounded. Print the exact decimal expansions.

0.1 stored as	0.1000000000000000555
0.2 stored as	0.2000000000000000110
0.1 + 0.2 =	0.3000000000000000441
0.3 stored as	0.2999999999999999890
Difference	5.55×10^{-17}
Difference in units in the last place	1

How this is got wrong. By concluding that floating point is unreliable. It is exactly reliable: every operation above returned the correctly rounded result, and the standard guarantees that. What is unreliable is the assumption that decimal literals are representable. The consequences for hardware verification are direct: a bit-exact comparison against a reference model is only meaningful if both use the same rounding mode and the same order of operations, **and floating-point addition is not associative**, so a reference model that sums an array in a different order than the RTL will disagree with it while both are correct. Specify the summation order, or specify a tolerance and justify it.

Problem. A neural-network accelerator uses bfloat16 multipliers with binary32 accumulation. Why not accumulate in bfloat16 too?

Solution. Because accumulation is where precision is lost and multiplication is not. A bfloat16 multiply of two 8-bit significands produces a 16-bit exact product, and rounding it back to 8 bits costs one rounding. Summing N such products in bfloat16 costs N roundings, each of half an LSB of a number with only 8 significand bits, and — worse — once the running sum is 2^8 times larger than the next term, that term is **absorbed entirely** and contributes nothing. For N in the hundreds or thousands, which a convolution reduction routinely is, the sum silently stops growing. Binary32 accumulation has 24 significand bits and pushes the absorption threshold out by 2^{16} . **The general rule is that a reduction needs more precision than its operands**, which is why every practical mixed-precision unit, from a DSP MAC to a tensor core, is asymmetric in exactly this way.

X10. Memory: The Block That Decides the Floorplan

In most IP blocks the memory is the largest single object, the slowest path, and the thing that cannot be changed late. It is also the part of a design that RTL hides most completely: a two-dimensional array in SystemVerilog becomes, in silicon, a compiled macro with a fixed aspect ratio, a fixed number of ports and its own timing model. This part computes what that substitution costs.

X10.1 The bit cell and what follows from it

Table 234. Storage cells compared

Cell	Transistors / bit	Relative area	Volatile	Read	Used for
6T SRAM	6	1	Yes	Non-destructive	Caches, buffers, everything on-chip
8T SRAM	8	~1.3	Yes	Separate read port	Low-voltage operation — decouples read and write stability
Dual-port (2RW)	8–10	~1.8–2	Yes	Two ports	FIFOs across clock domains
Register file cell	~8–12	1.5–3	Yes	Many ports	Small, highly ported storage
1T-1C DRAM	1 + capacitor	~0.15	Yes, needs refresh	Destructive — must write back	Main memory, eDRAM
ROM (via / contact)	1 or 0	~0.1	No	Non-destructive	Twiddles, microcode, constants
Flip-flop	~20–24	4–5	Yes	—	What synthesis gives you if you do not instantiate a macro

Worked example 21. The cost of forgetting to instantiate a RAM

Given. A 4 kB buffer, 32 768 bits. The 6T cell is $0.03 \mu\text{m}^2$; a flip-flop with its multiplexer occupies roughly $150\times$ a bit cell once the write-enable logic is counted.

Method. Compute both areas and the array overhead. SRAM macros carry decoders, sense amplifiers and control that a small array cannot amortise; assume 40 % overhead at this size.

Bit-cell area	$983 \mu\text{m}^2$
Compiled macro with 40 % overhead	$1376 \mu\text{m}^2$
Same storage in flip-flops	$1.475 \times 10^5 \mu\text{m}^2$
Ratio	$107\times$
Flip-flop clock power penalty (all bits clocked every cycle)	very large — every bit is on the clock tree

How this is got wrong. By writing an array in RTL and letting synthesis decide. Synthesis will infer flip-flops unless the coding style matches the inference template exactly — and the template differs between tools and between memory compilers. The failure is quiet: the design works, meets timing, and is a hundred times too large. **Always check the synthesis report for inferred registers against your expected memory instances**, and prefer explicit instantiation with a wrapper for anything above a few hundred bits.

X10.2 Aspect ratio, banking and the access-energy curve

Table 235. Same 32 768 bits, six aspect ratios (relative units)

Words (depth)	Bits per word	Bit-line length	Word-line length	Relative access energy	Relative access delay
128	256	128	256	0.23	14.8
256	128	256	128	0.307	20
512	64	512	64	0.538	27.1
1,024	32	1,024	32	1.04	37
2,048	16	2,048	16	2.05	50.8
4,096	8	4,096	8	4.1	70

A tall narrow array has long bit lines, which cost energy and delay; a short wide one has long word lines and wastes energy activating bits it will not return. **The optimum is in the middle and memory compilers know it**, which is why a compiler refuses extreme aspect ratios and folds internally instead.

Graduate · Sub-banking is why a 1 MB cache does not cost 32× the energy of a 32 kB one. A large array is built as many small arrays with a hierarchical decoder, and an access activates one of them. Energy per access therefore grows roughly with the *square root* of capacity rather than linearly, and the same structure bounds the delay growth. This is the reason capacity-versus-energy intuitions borrowed from a small block do not transfer, and the reason a cache model that charges energy proportional to size will mispredict a memory-bound design badly. **When you need real numbers, use a memory compiler's datasheet or a model such as CACTI** — and state which, because the two disagree.

X10.3 Multi-port memory: three ways, all of them compromises

Table 236. Getting more ports than the compiler offers

Technique	Ports obtained	Area cost	Restriction
True multi-port cell	$nRmW$	Grows as $(n+m)^2$ — wires per cell	Only 1RW and 2RW are usually in the compiler library
Banking by address	n accesses if to different banks	Decode and crossbar	Conflicts stall — needs arbitration and back-pressure
Replication (one copy per reader)	$nR1W$	$n \times$ the array	Writes must go to every copy
Double clocking	2 accesses per functional cycle	None in area	Halves the frequency budget ; needs a clean 2× clock
XOR / LVT multi-port	$nRmW$ from 1RW banks	$m(n+m)$ banks plus a live-value table	Complexity; well suited to FPGA, rarely worth it in ASIC

Worked example 22. A register file's port count is a quadratic cost

Given. A 32-entry $\times$ 64-bit register file for a 2-wide machine: 4 read ports and 2 write ports.

Method. Model the cell area as growing with the square of the total port count, which is the standard first-order model because each port adds a bit line and a word line to every cell.

Ports	6
Area relative to a 1R1W cell	9×
Bits	2,048
If widened to 4-wide (8R4W): ports	12
Area relative to 1R1W	36×
Area penalty of doubling issue width	4×

How this is got wrong. By treating issue width as a free architectural parameter. Doubling it quadruples the register-file area and lengthens its access, which is one of the two reasons superscalar width stopped growing (the other is the scheduler's own quadratic wake-up logic). The architectural responses are all about avoiding the quadratic: clustering the register file and accepting a cross-cluster penalty, banking it by register number, or moving to a structure where most operands never enter the file at all. **A cost model that is quadratic in a parameter is a signal to change the structure, not to tune the parameter.**

X10.4 ECC: the arithmetic of protecting a memory

Why SECDED needs exactly the bits it needs

	Step	Justification
1	To correct a single error among k data bits, the syndrome must name one of $k+r$ positions or say 'no error'.	Every correctable case needs its own syndrome value.
2	So $2^r \geq k + r + 1$.	The Hamming bound for single-error correction.
3	Detecting double errors as well costs one more bit: an overall parity.	With it, a double error gives a non-zero syndrome and even overall parity, which single errors never do.
4	SECDED overhead is $r+1$ bits for k data bits.	And because r grows logarithmically, wide words are far cheaper to protect than narrow ones — the single most useful consequence.

Table 237. SECDED overhead against word width

Data bits k	Hamming bits r	SECDED bits	Overhead (%)	Total stored bits
8	4	5	62.5	13
16	5	6	37.5	22
32	6	7	21.9	39
64	7	8	12.5	72
128	8	9	7.03	137
256	9	10	3.91	266
512	10	11	2.15	523

Protecting a 64-bit word costs 12.5 %; protecting a 512-bit line costs 2 %. **This is why ECC is applied to cache lines and DRAM bursts rather than to bytes**, and why a design that must protect narrow fields should consider grouping them first.

ECC changes the timing path and the failure modes, not only the area. The syndrome must be computed and the correction applied before the data is used, which inserts an XOR tree — typically 3 to 5 gate levels for a 64-bit word — into the read path. At high frequency this forces the correction into a later pipeline stage, which means **the consumer must be able to tolerate a late correction or be stalled**, and that is an architectural decision, not an implementation detail. There is a second trap: a memory with ECC that is never scrubbed accumulates single-bit errors until two coincide in one word, at which point a correctable fault population becomes an uncorrectable event. **Scrubbing is part of the ECC design, not an optional extra**, and its period follows from the soft-error rate and the word count.

Table 238. Soft-error arithmetic at 1000 FIT/Mbit (one failure per 10⁹ device-hours per Mbit)

Memory (Mbit)	Errors per hour	MTBF (years, one device)	Errors per device-year	Errors per year across a million units
1	1.00×10^{-6}	114	0.00877	8766
4	4.00×10^{-6}	28.5	0.0351	35064
16	1.60×10^{-5}	7.13	0.14	1.40×10^5
64	6.40×10^{-5}	1.78	0.561	5.61×10^5
256	2.56×10^{-4}	0.446	2.24	2.24×10^6

The last column is the one a product manager needs. **A part that sees one upset per device-century produces thousands of field events per year across a large deployment**, which is why ECC appears in consumer parts and not only in servers. The FIT figure itself depends on altitude, process and cell design and must come from the foundry.

Problem. Your block has a 2 kB configuration RAM written once at boot and read every cycle thereafter. Does it need ECC, and of what kind?

Solution. It needs protection more than a data buffer does, for a reason that has nothing to do with its size: **a corrupted configuration bit is permanent and systematic**, where a corrupted data word is transient. An upset in a filter coefficient degrades every subsequent output, silently, until the next reboot, and there is no downstream check that will catch it. The economical answer is usually **parity plus a reload path** rather than full SECDED: since the contents are known and re-writable, detection is enough — flag the error, interrupt, and have software rewrite the block. That costs one bit per word instead of eight and needs no correction logic in the read path, so it does not disturb the timing at all. **Choose detection over correction whenever a recovery path exists**; the recovery path is usually cheaper than the correction hardware, and it also covers failures the ECC cannot, such as a corrupted address.

Problem. Why does a FIFO built from a dual-port SRAM need its pointers in flip-flops rather than in the same memory?

Solution. Because the pointers must be readable in the same cycle they are compared, and a synchronous SRAM read has a cycle of latency. The full and empty conditions gate the write and read enables of the current cycle, so a pointer that arrives a cycle late produces a FIFO that overflows by one entry under back-to-back traffic — a bug that appears only at full rate, which is to say only in the system and never in the unit test that writes one word at a time. **The general form of this trap is worth internalising: any state that participates in this cycle's flow-control decision cannot live behind a synchronous read port.** It is the same reason a cache tag array is read a cycle ahead of the data array, and the reason credit counters in an interconnect are registers however large the credit table becomes.

X11. Interconnect: Arbitration, Flow Control and Deadlock

An IP block that talks to anything talks through an interconnect, and the interconnect's properties leak into the block's specification whether or not the specification mentions them. The three that leak hardest are how bandwidth is shared, how back-pressure propagates, and whether the system can lock up. All three are computable and the third is provable.

X11.1 Credit-based flow control: sizing the buffer

Why the buffer must cover the round trip

	Step	Justification
1	The sender may transmit only while it holds credits; each transmission consumes one.	<i>Definition of credit-based flow control.</i>
2	A credit is returned when the receiver frees a buffer entry; the return takes t_{ret} to arrive.	<i>Physical delay: the return path is wires and pipeline stages like any other.</i>
3	For the sender never to stall while the receiver is draining, credits must outlast the round trip.	<i>Otherwise there is an interval in which the sender has no credit and the receiver has space.</i>
4	$N_{\text{buf}} \geq B \cdot \text{RTT}$, where B is the peak rate and RTT is forward latency plus return latency.	<i>The bandwidth–delay product. It is the same formula as a TCP window and for exactly the same reason.</i>
5	Any shortfall shows up as a throughput loss of $N_{\text{buf}} / (B \cdot \text{RTT})$, not as a functional failure.	<i>Which is why an undersized buffer is found in performance bring-up rather than in verification — late, and by the customer.</i>

Table 239. Credit buffer needed against pipeline depth, one flit per cycle

Forward stages	Return stages	Round trip (cycles)	Minimum credits	Buffer bytes at 32 B/flit	Throughput if only one credit is provided
1	1	2	2	8	0.5
2	2	4	4	16	0.25
4	4	8	8	32	0.125
8	8	16	16	64	0.0625
16	16	32	32	128	0.0312

The last column is what an under-provisioned link actually delivers: one flit per round trip. A four-deep pipeline with one credit runs at an eighth of the wire's rate, and nothing in the design is broken.

Worked example 23. A block on a pipelined AXI interconnect

Given. A DMA engine issues read bursts across an interconnect with 6 cycles of forward latency and 6 of response, at 500 MHz, 128-bit data, and must sustain 6 GB/s.

Method. Convert the required bandwidth into outstanding transactions via the bandwidth–delay product, then check it against the interface's ID width, which is what actually limits outstanding transactions on AXI.

Bytes per beat	16
Beats per second required	3.750×10^8
Cycles per second	5.000×10^8
Beats per cycle required	0.75
Round-trip latency	12 cycles
Outstanding beats needed	9
If bursts are 16 beats: outstanding bursts needed	1
Read data buffer required	144 B

How this is got wrong. By sizing the buffer for the burst and not for the round trip. A common specification says ‘one burst of read data’, which permits exactly one outstanding burst and throttles the engine to burst-length divided by round-trip — here about a quarter of the wire rate. The second frequent error is to size for the *average* latency: the interconnect's latency has a tail whenever another master is active, and a buffer sized for the mean stalls whenever the tail occurs. **Size for the latency at the load you will actually run at**, and state that load in the datasheet, because a customer whose system is busier than your bench will not reach your quoted bandwidth.

X11.2 Arbitration: fairness has a precise meaning and a cost

Table 240. Measured mean / maximum wait in cycles, four masters offering 0.5, 0.2, 0.15 and 0.1 requests per cycle, 200 000 cycles

Policy	M0 (busiest)	M1	M2	M3 (quietest)
Fixed priority	1.0 / 1	2.0 / 21	3.3 / 29	5.4 / 62
Round robin	1.6 / 4	1.5 / 4	1.5 / 4	1.5 / 4
Oldest first (LRG)	1.4 / 4	1.8 / 4	2.0 / 4	2.1 / 4

Simulated with seed 13. Fixed priority starves the low-priority masters when the high-priority one is busy — read M3's maximum. Round robin bounds everyone's wait; oldest-first minimises the maximum further at the cost of an age counter per requester.

Table 241. Arbitration policies and what each guarantees

Policy	Guarantees	Cost	Fails at
Fixed priority	Lowest latency for the top requester	A priority encoder — cheapest	Starvation: no bound for the bottom requester
Round robin	Bounded wait of $n-1$ grants	A rotating pointer	Does not respect differing bandwidth needs
Weighted round robin	Bandwidth in proportion to weights	Per-requester counters	Latency for a low-weight but urgent requester
Deficit round robin	Bandwidth proportion with variable-size requests	A deficit counter per requester	— this is the standard answer for packets

Policy	Guarantees	Cost	Fails at
Oldest first	Minimises maximum wait	Age counters and a comparison tree — the tree is the timing problem	Scaling: the comparison is $O(\log n)$ deep and wide
Lottery / ran- domised	Proportional in expectation	An LFSR	No bound , only an expectation — unacceptable for real-time

Graduate · **Quality of service is not a policy, it is two different requirements that must be separated before a policy can be chosen.** A display controller needs *guaranteed bandwidth* and tolerates latency, because it has a line buffer; a CPU fetching a cache miss needs *low latency* and consumes little bandwidth. One arbiter cannot serve both well with a single knob, which is why real interconnects separate them: a bandwidth regulator (a token bucket or a deficit counter) enforces the first, and a priority class enforces the second, with the regulator demoting a requester that exceeds its allocation so that its priority cannot starve anyone. **When writing an IP block's requirements, state which of the two you need;** 'high performance' is not an interconnect requirement and cannot be designed against.

X11.3 Deadlock: a property of the dependency graph

The four conditions, and which one hardware removes

	Step	Justification
1	Mutual exclusion: a resource is held by one holder at a time.	<i>True of every buffer; not removable.</i>
2	Hold and wait: a holder may request another resource while holding one.	<i>True of any pipelined request; removing it means allocating everything at once, which wastes buffers.</i>
3	No pre-emption: a buffer cannot be taken back.	<i>Removable — by dropping and retrying, which is what a network does and what an on-chip fabric usually cannot afford.</i>
4	Circular wait: the wait-for graph contains a cycle.	<i>This is the one hardware removes.</i> Break every cycle and deadlock is impossible regardless of the other three.
5	Virtual channels break cycles by splitting one physical buffer into several logically independent ones with an ordering between them.	<i>A message may only move from a lower-numbered channel to a higher one, so the dependency graph becomes acyclic by construction.</i>
6	Dimension-ordered routing breaks cycles in a mesh by forbidding a turn.	<i>X then Y: the turns that would close a cycle are simply not permitted. Cheap, and it costs path diversity, which costs throughput under adversarial traffic.</i>

The most common on-chip deadlock has nothing to do with routing. It is a read response that cannot make progress because the response buffer is full of responses waiting behind a write that cannot complete because the write channel is blocked by a request from the same master. Protocol specifications address this with ordering rules — AXI's requirement that read and write channels be independent, and that a slave must not make a read response depend on a future write — and **those rules are routinely broken by IP blocks that share one internal queue between the two channels.** If your block merges AXI's read and write paths into a single ordered structure anywhere, you have created this deadlock, and it will appear in a customer's system and not in yours.

Problem. A bridge accepts an AXI write and must issue a read to fetch the rest of a partially written cache line, then complete the write. Why is this a deadlock hazard, and what are the fixes?

Solution. It creates a dependency from the write channel to the read channel. If the read travels through the same interconnect and that interconnect is congested with writes — including this one, which is holding a slot — the read cannot be issued and the write cannot retire. The wait-for graph has a cycle. Three fixes, in increasing order of cost and decreasing order of fragility. **Reserve resources before accepting:** do not accept the write until a read slot and a line buffer are already allocated, so the bridge never holds and waits. **Use a separate virtual or physical channel** for the fetch, so the read's path cannot be blocked by writes. **Buffer the entire write** and acknowledge it immediately, converting the dependency into a local one — which costs a full line of storage per outstanding write and changes the coherence behaviour. Whichever is chosen, **the argument must be written down as an acyclicity claim about the dependency graph**, because it is not something a simulation will refute reliably: deadlocks appear only under specific congestion, and the bench that would produce it is rarely the bench that is written.

X11.4 Topology and the wire budget

Table 242. Topologies: links and worst-case hop count

Nodes	Full crossbar links	Ring: worst hops	2-D mesh: worst hops	Hypercube: worst hops	Ring links
4	6	3	2	2	2
8	28	7	—	3	4
16	120	15	6	4	8
32	496	31	—	5	16
64	2,016	63	14	6	32

The crossbar column is the reason crossbars stop at modest radix: the link count is quadratic and each link is a full-width bus. Beyond about 16 ports a hierarchy of smaller crossbars wins on area even though it adds hops.

Worked example 24. Bisection bandwidth decides, not link count

Given. A 16-node system, 128-bit links at 1 GHz. Compare a ring, a 4×4 mesh and a full crossbar under uniform random traffic, where on average half the traffic crosses the bisection.

Method. Compute the bisection bandwidth — the bandwidth across the worst cut that divides the network in half — and compare with the offered load. Bisection, not aggregate link bandwidth, is what limits uniform traffic.

Per-link bandwidth	16 GB/s
Ring: links crossing the bisection	2
Ring bisection bandwidth	32 GB/s
4×4 mesh: links crossing	4
Mesh bisection bandwidth	64 GB/s
Crossbar bisection bandwidth	128 GB/s
Offered load if each node sends 8 GB/s, half crossing	64 GB/s
Ring sufficient?	no
Mesh sufficient?	no
Crossbar sufficient?	yes

How this is got wrong. By quoting aggregate bandwidth. The ring in this example has 16 links and 256 GB/s of aggregate link bandwidth, which sounds ample and is irrelevant: uniform traffic is limited by the two links at the cut. The corollary is that **topology should be chosen from the traffic pattern**. If the real traffic is nearest-neighbour, the ring's bisection never binds and it is the right answer; if it is all-to-all, only the bisection matters. An interconnect specified without a traffic model has not been specified.

X12. Data Converters and PLLs, Worked

Very few IP blocks are purely digital at their boundary. A SerDes has an analogue front end; a sensor interface has a converter; every synchronous block has a clock that came from a PLL. A digital designer does not have to design these, but does have to *budget* for them, and budgeting requires the same arithmetic the analogue designer uses. This part supplies it.

X12.1 The converter equations, and what they leave out

SNR, ENOB and the $6.02N+1.76$ formula

	Step	Justification
1	Quantisation error is bounded by $\pm\Delta/2$ with variance $\Delta^2/12$, if it is uniform.	<i>Part X1 established this and also measured where it fails.</i>
2	A full-scale sine of peak $A = 2^{N-1}\Delta$ has power $A^2/2$.	<i>RMS of a sinusoid.</i>
3	$\text{SNR} = 10\log_{10}(A^2/2 \div \Delta^2/12) = \mathbf{6.02N + 1.76 \text{ dB}}$.	<i>Substitute and simplify. The 1.76 is $10\log_{10}(1.5)$, the sine's crest-factor advantage over a uniform signal.</i>
4	Real converters fall short; the shortfall is expressed as ENOB = (SNDR – 1.76)/6.02 .	<i>Invert the formula using the measured signal-to-noise-and-distortion ratio. ENOB is a measurement, N is a pin count, and quoting the second as if it were the first is the standard datasheet deception.</i>
5	Aperture jitter adds a noise term $\text{SNR}_{\text{jitter}} = -20\log_{10}(2\pi f_{\text{in}}\sigma_t)$.	<i>The sampled error is the signal's slew times the timing error; slew is proportional to input frequency. This term does not depend on N at all and eventually dominates every high-speed converter.</i>

Table 243. Effective bits achievable with 1 ps RMS aperture jitter, against nominal resolution and input frequency

Nominal N	Ideal SNR (dB)	ENOB at 1 MHz	at 10 MHz	at 100 MHz	at 1 GHz
8	49.92	8	8	7.97	6.86
10	61.96	10	10	9.65	7.01
12	74	12	11.9	10.3	7.02
14	86.04	14	13.3	10.3	7.02
16	98.08	15.8	13.6	10.3	7.02

Read down the last column: **at 1 GHz input, 1 ps of jitter caps the converter at about 8 bits no matter how many bits the part has**. Buying a 16-bit converter without buying a cleaner clock buys nothing.

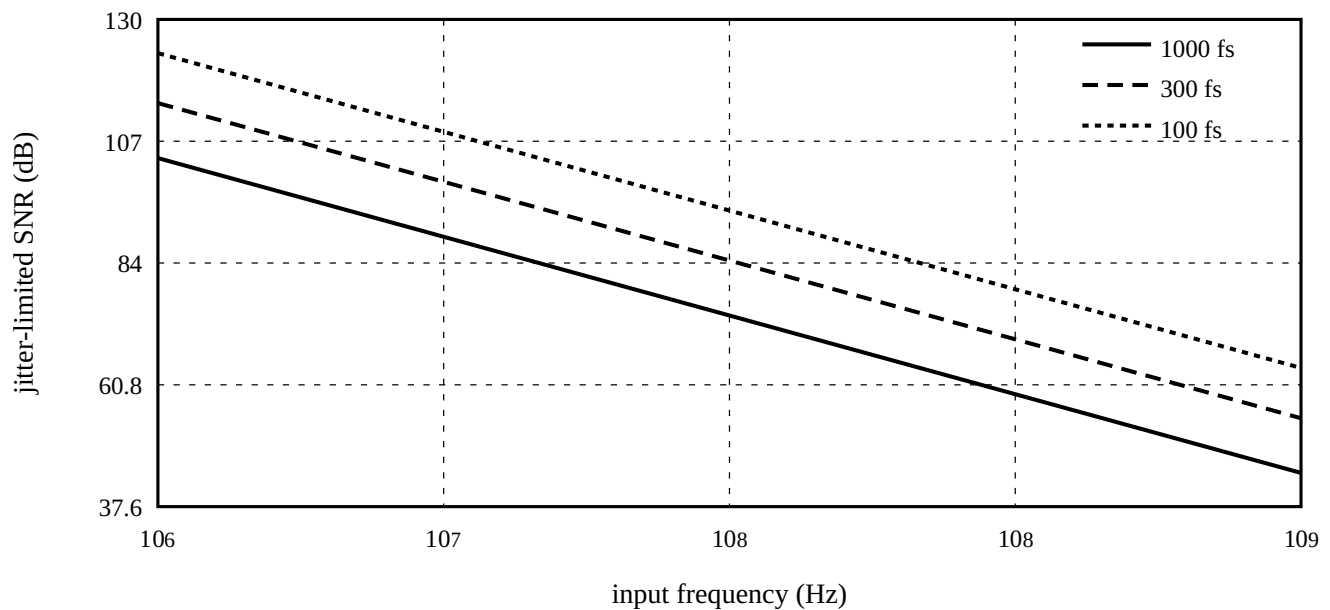


Figure 34. The jitter wall. Every curve falls 20 dB per decade of input frequency; the only way up is a better clock.

Worked example 25. Specifying the clock for an ADC, backwards from the requirement

Given. A receiver needs 11 effective bits on a 500 MHz input. The converter's own thermal and distortion contribution is 13 ENOB.

Method. Convert the requirement to an SNR, subtract the converter's own contribution in the power domain to find the jitter budget, then invert the jitter formula.

Required SNDR	67.98 dB
Converter's own SNR	80.02 dB
Permitted jitter-noise SNR	68.26 dB
Permitted RMS aperture jitter	123 fs
Is that achievable with an on-chip PLL?	demanding — it is at the edge of a good integrated LC PLL

How this is got wrong. By subtracting decibels instead of powers. Noise contributions add in power, so budgets must be combined as $10^{-\text{SNR}/10}$ terms and only then converted back. Subtracting 11 from 13 in dB gives a meaningless number. The second error, subtler, is to forget that the *clock buffer* between the PLL and the sampler adds its own jitter, often more than the PLL, so a budget that spends the whole allowance on the PLL has already failed.

Table 244. Converter architectures and where each lives

Architecture	Resolution	Speed	Latency	Cost driver
Flash	4–6 bits	Highest	1 cycle	2^N comparators — area doubles per bit
Pipelined	10–14 bits	High	$N/2$ cycles	Op-amps; inter-stage gain accuracy
SAR	8–16 bits	Moderate	N cycles	Capacitor matching; scales beautifully with process
Sigma-delta	16–24 bits	Low signal bandwidth	High (filter)	Oversampling ratio; digital decimation filter

Architecture	Resolution	Speed	Latency	Cost driver
Time-inter-leaved	8–12 bits	Extreme	As the sub-ADC	Mismatch between channels — gain, offset and timing skew, all needing digital calibration

Graduate · Time interleaving is where the digital team earns its place in an analogue block. Running M converters in round robin multiplies the sample rate by M and introduces three mismatches: offset, which appears as spurs at multiples of f_s/M ; gain, which appears as spurs around the signal; and timing skew, which appears as signal-dependent spurs that grow with input frequency. None is removable by better layout alone at modern speeds; all three are corrected by **digital background calibration** that estimates the mismatch from the data and applies a correction. That estimator is a piece of DSP with a convergence time, a lock detector and a failure mode, and it is specified, modelled and verified by the digital side. **A SerDes receiver's ADC is time-interleaved 32 or 64 ways, and its calibration loops are a substantial part of the digital block's area.**

X12.2 Phase-locked loops: the transfer functions that matter

Why a PLL is a lowpass for reference noise and a highpass for VCO noise

	Step	Justification
1	Open loop: $G(s) = K_{pd}F(s)K_{vco}/s$, divided by N .	Phase detector gain, loop filter, VCO as a phase integrator, feedback divider.
2	Reference to output: $N G/(1+G)$.	Standard feedback algebra. At low frequency $G \gg 1$ so this is N ; at high frequency it rolls off. Lowpass.
3	VCO noise to output: $1/(1+G)$.	The VCO's noise enters after the phase detector. At low frequency the loop suppresses it; above the bandwidth it passes. Highpass.
4	So the loop bandwidth is a crossover, not a quality setting.	Wide bandwidth tracks the reference and suppresses VCO noise; narrow bandwidth rejects reference noise and lets the VCO run free. The optimum is where the two noise densities cross, and that point is measured , not chosen by taste.
5	Total output jitter is the integral of the combined phase-noise spectrum over the band of interest.	Which is why a jitter specification without an integration band is incomplete — the same part can be quoted at 200 fs or 2 ps depending on limits.

Table 245. Integrated jitter of a 2.5 GHz PLL against loop bandwidth, with a toy noise model (reference -100 dBc/Hz flat, VCO -80 dBc/Hz at 1 MHz, $N = 20$)

Loop bandwidth (MHz)	Output phase noise at 10 kHz (dBc/Hz)	at 1 MHz (dBc/Hz)	Integrated RMS jitter 10 kHz–100 MHz (fs)
0.1	-59.87	-79.87	35204
0.5	-70.97	-77.96	22387
1	-73.01	-76.02	25070
2	-73.72	-74.69	32635
5	-73.94	-74.11	49869

Integrated numerically over the stated band. There is a minimum: too narrow and the VCO dominates, too wide and the reference multiplied by N^2 dominates. **The $20\log_{10}N$ multiplication of reference noise is the term designers forget, and it is 26 dB here.**

A PLL specification needs five numbers and is usually given two. Output frequency and jitter are not enough. The five are: output frequency range; **integrated jitter with its integration band**; spurious tone level, since a single reference spur can violate a mask that broadband jitter passes; lock time, which sets how long a power-gated block takes to become usable; and the **input reference's own phase noise**, since the PLL multiplies it by $20\log_{10}N$. A block that is specified without the fifth will work on the vendor's evaluation board with a laboratory clock source and fail in the customer's system with a crystal oscillator.

Table 246. Reference phase noise is multiplied by the divide ratio

Divide ratio N	$20\log_{10}N$ (dB)	Output noise from a -100 dBc/Hz reference	Output noise from a -130 dBc/Hz reference
2	6.021	-93.98	-124
4	12.04	-87.96	-118
10	20	-80	-110
20	26.02	-73.98	-104
50	33.98	-66.02	-96.02
100	40	-60	-90

In-band only — above the loop bandwidth the reference is filtered. This is the reason a high-speed SerDes wants a low reference multiplication, and the reason a 100 MHz reference often beats a 25 MHz one for the same output frequency.

Problem. A clock-and-data-recovery loop and a frequency-synthesis PLL are both PLLs. Why is the CDR's bandwidth chosen by a standard rather than by the designer?

Solution. Because in a CDR the loop bandwidth determines **jitter tolerance**, which is an interoperability property rather than a performance one. Jitter below the loop bandwidth is tracked by the receiver and causes no error; jitter above it is not tracked and eats the eye directly. So the receiver's bandwidth and the transmitter's permitted jitter spectrum are two halves of one specification: if each vendor chose its own bandwidth, a compliant transmitter and a compliant receiver could still fail together. Standards therefore specify a **jitter tolerance mask** — how much sinusoidal jitter the receiver must survive at each frequency — which constrains the loop bandwidth from below, and a **jitter transfer** limit, which constrains it from above so that jitter is not amplified around the loop's peaking and accumulated through a chain of repeaters. **The designer's freedom is what remains between the two masks**, and in modern standards that gap is narrow.

Problem. Why does a fractional- N synthesiser need a delta-sigma modulator, and what does it cost?

Solution. A divider divides by an integer. To reach a fractional average the divide ratio must be switched between neighbouring integers, and the *pattern* of switching determines where the resulting phase error lands in frequency. A simple periodic pattern puts it at a low frequency inside the loop bandwidth, where it appears as a **fractional spur** — a discrete tone that violates masks. A delta-sigma modulator chooses the pattern so that the quantisation error is *shaped* to high frequency, where the loop filters it. The costs are real: the shaped noise is larger in total than the unshaped noise, so the loop bandwidth must be kept below the point where the shaping rises; a higher-order modulator shapes harder but can produce idle tones if its input sits near a simple fraction, which is why implementations dither the input; and the divider must now handle a range of ratios, which complicates its design. **This is a clean example of a digital block bought to relax an analogue constraint**, which is the dominant trend in mixed-signal design and the reason the boundary keeps moving toward the digital side.

X13. Machine-Learning Hardware, Worked

Neural-network accelerators are the largest category of new IP by value, and they are also the category where the gap between a quoted peak number and delivered performance is widest. The arithmetic that closes that gap is the roofline model and its refinements, and it is worth doing carefully because it decides the architecture before a line of RTL is written.

X13.1 The roofline, computed for real layers

Arithmetic intensity and where the roof bends

	Step	Justification
1	A kernel performs W operations and moves Q bytes.	<i>Definitions.</i>
2	Arithmetic intensity $I = W/Q$, operations per byte.	<i>A property of the algorithm and the dataflow, not of the machine.</i>
3	Achievable performance = $\min(P_{\text{peak}}, I \cdot B)$, with B the memory bandwidth.	<i>You cannot compute faster than the machine, nor faster than the data arrives.</i>
4	The ridge point is at $I = P_{\text{peak}}/B$.	<i>Below it the kernel is memory bound and peak FLOPS are irrelevant.</i>
5	For a modern accelerator P_{peak}/B is in the hundreds.	<i>Which means most layers are memory bound unless the dataflow is designed to raise I — and raising I is what the whole accelerator architecture is for.</i>

Table 247. Arithmetic intensity of common layers, int8, no reuse beyond the layer

Layer	MOPs	Bytes moved (kB)	Ops per byte	Bound by (ridge = 200)
Conv 3×3, 256 → 256, 56×56	3699	2144	1685	compute
Conv 1×1, 512 → 128, 28×28	102.8	554	181.1	memory
Conv 3×3 depthwise, 512, 14×14	0	98.19	0	memory
Fully connected 4096 → 4096	33.55	16392	1.999	memory
Attention QK ^T , $d=64$, $n=1024$	1.374×10^5	1.114×10^6	120.5	memory

Computed assuming each weight and activation is read once from off-chip, which is the worst dataflow. **The point of an accelerator's on-chip memory hierarchy is to make this table wrong** by reusing data; how much it can reuse is the design.

Graduate · Depthwise convolution is the instructive row. It has excellent parameter efficiency — which is why MobileNet-style networks use it — and terrible arithmetic intensity, because each weight is used for only one input channel. On an accelerator designed around dense convolution it can run at a few per cent of peak, so a network that is 4× cheaper in operations can be *slower* in wall-clock time. **This is the clearest example in the field of why operation counts are not a performance metric**, and it is a question worth asking any vendor: not ‘what is your TOPS’ but ‘what is your utilisation on a depthwise layer’.

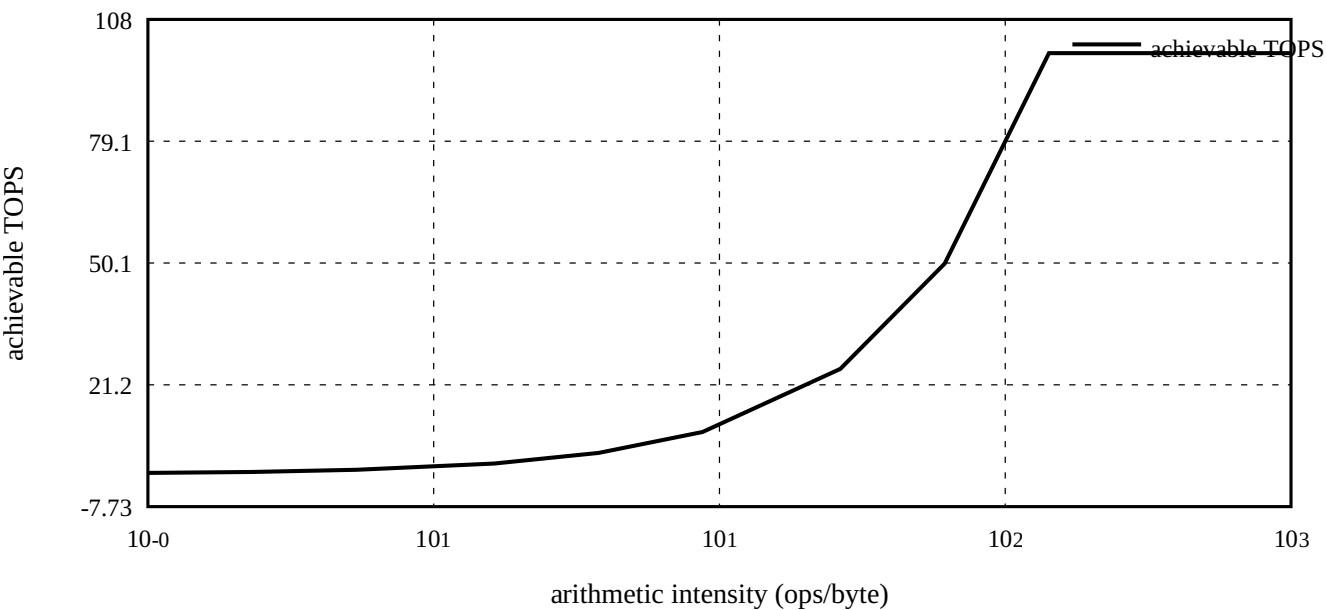


Figure 35. A roofline for a 100 TOPS accelerator with 500 GB/s. The ridge is at 200 ops/byte; everything to its left is a bandwidth problem wearing a compute problem's clothes.

X13.2 Dataflow: what a systolic array is actually reusing

Table 248. Dataflow taxonomy by what stays still

Dataflow	Held stationary	Reuse obtained	Weak when
Weight stationary	Weights in the PE	Each weight is used for a whole batch or feature map	Batch is 1 and the feature map is small
Output stationary	The partial sum in the PE	No partial sums move — saves the widest data	Few accumulations per output (1×1 convolutions)
Input stationary	The activation	Each activation feeds many filters	Few output channels
Row stationary	A row of the convolution in each PE	Balances all three reuse types	Complex control and mapping
No local reuse	Nothing	—	Always — it is the baseline that makes the others look good

Worked example 26. Sizing a systolic array from a bandwidth limit

Given. A 256×256 int8 multiply–accumulate array at 1 GHz, weight stationary. Off-chip bandwidth 100 GB/s.

Method. Compute the peak rate, then compute the byte rate the array would demand with no reuse, then find the reuse factor required to keep it fed.

MACs per cycle	65,536
Peak operations	131.1 TOPS
Operands needed per cycle with no reuse	131,072 bytes
Byte rate demanded	131.1 TB/s
Available	0.1 TB/s
Reuse factor required	1311×
On-chip SRAM to achieve it (order)	megabytes

How this is got wrong. By quoting the TOPS figure and stopping. The array is 131 TOPS and the memory system can feed it 1/1300th of what it wants. Every architectural feature of a real accelerator — the weight buffer, the activation buffer, the tiling, the fusion of adjacent layers — exists to close that factor of 1300, and **the accelerator is really a memory system with multipliers attached**. A design house pitching an accelerator should lead with the memory hierarchy, because that is what a knowledgeable buyer will ask about first.

X13.3 Quantisation: where the bits actually go

Table 249. Measured SQNR of a 512×512 int matrix product, per-tensor vs. per-channel weight scaling

Bits	Per-tensor SQNR (dB)	Per-channel SQNR (dB)	Gain (dB)	Weight storage (kB)
8	39.08	41.29	2.21	256
6	26.8	29.15	2.35	192
4	13.8	16.26	2.47	128
3	6.414	9.003	2.59	96
2	-0.08753	0.6042	0.692	64

Gaussian weights and post-ReLU activations, seed 21. Per-channel scaling is nearly free in hardware — one extra multiply per output channel, not per MAC — and buys several dB, which at low bit widths is the difference between a usable and an unusable network.

The outlier problem, and why it changed transformer accelerators. The table above uses Gaussian weights, for which max-based scaling is reasonable. Real transformer activations are not Gaussian: a few channels carry values tens of times larger than the rest, and a per-tensor scale chosen to contain them leaves the remaining channels occupying two or three bits. The measured consequence is that naive int8 quantisation of a large language model loses far more accuracy than the same procedure applied to a convolutional network, and the remedies — per-channel and per-group scales, keeping outlier channels in higher precision, or migrating the scale between activations and weights — all cost hardware that a convolution-era accelerator does not have. **An accelerator's quantisation support is a workload-specific design decision, not a generic feature.**

Problem. Why does int8 inference usually retain accuracy while int8 training does not?

Solution. Because the two use the numbers differently. Inference computes a forward pass whose outputs pass through nonlinearities that are insensitive to small perturbations, and errors do not accumulate across iterations — each inference is independent. Training computes gradients, which are (a) small differences of large numbers, so relative precision matters far more, (b) accumulated over many steps, so bias accumulates exactly as Part X1's truncating accumulator did, and (c) spread over an enormous dynamic range, because gradients of different layers differ by orders of magnitude. The practical resolution is the asymmetric one: **low-precision multiplies with higher-precision accumulation and master weights**, plus loss scaling to move small gradients out of the denormal region. bfloat16 exists for exactly the (c) reason — it sacrifices mantissa, which inference cares about, to keep exponent range, which training cares about.

Problem. A customer wants your accelerator to be ‘future proof’. What can you honestly offer?

Solution. Not architectural coverage of models that do not exist. What you can offer is specific and worth more than a vague claim: **a programmable dataflow** rather than a fixed one, so a new layer shape maps at some efficiency rather than not at all; **a well-specified fallback path** to a host or a DSP for operators the array cannot do, with measured numbers for that path so the customer can compute the Amdahl penalty themselves; **a compiler that is open enough to extend**, since in practice the compiler ages faster than the silicon; and **measured utilisation on a set of named, versioned models**, so the customer can extrapolate from evidence rather than from adjectives. **Offering the measurement methodology is itself a differentiator**: most data-sheets in this field quote one peak number and one optimistic model, and a buyer who has been burned once recognises the difference immediately.

X14. Physical Design as the RTL Designer Sees It

A soft-IP vendor does not run place-and-route on the customer's chip, but the vendor's RTL determines whether that run succeeds. This part covers the physical facts that propagate back into RTL decisions, with the arithmetic that makes them checkable.

X14.1 Wire delay does not scale, and that changes architecture

Why a long wire gets relatively slower every generation

	Step	Justification
1	A wire's distributed RC delay is $0.38rcL^2$ for resistance r and capacitance c per unit length.	<i>Elmore delay of a distributed line. Quadratic in length — the single most important fact in this chapter.</i>
2	Scaling reduces wire cross-section, so r rises roughly as $1/S^2$ while c stays roughly constant.	<i>Thinner and narrower conductor, same dielectric spacing ratio.</i>
3	A wire of <i>fixed physical length</i> therefore gets slower as the process shrinks.	<i>While gates get faster. The ratio moves against the wire every generation.</i>
4	Buffering breaks the quadratic: k equal segments give $k \cdot (L/k)^2 = L^2/k$ plus k buffer delays.	<i>Optimise over k and the delay becomes linear in L.</i>
5	So a long wire costs area and power, and it costs a fixed delay per millimetre that no amount of cleverness removes.	<i>Which makes distance an architectural quantity. A design whose blocks must exchange data every cycle across a chip is not implementable at high frequency, however good the RTL.</i>

Table 250. Wire delay with and without optimal buffering (250 Ω /mm, 200 fF/mm, 20 ps buffers)

Length (mm)	Unbuffered delay (ps)	Buffers	Buffered delay (ps)	ps per mm	Speed-up
0.1	0.19	1	20.19	201.9	0.00941
0.5	4.75	1	24.75	49.5	0.192
1	19	2	49.5	49.5	0.384
2	76	5	115.2	57.6	0.66
5	475	12	279.6	55.92	1.7
10	1900	25	576	57.6	3.3

The constants are indicative of an intermediate metal layer in a mature node and are not from a specific PDK. **The shape is what matters:** the unbuffered column is quadratic, the buffered column is linear, and the per-millimetre figure is roughly constant — which is why architects budget in millimetres.

Worked example 27. How far can a signal travel in one cycle?

Given. A 1 GHz design in the process above. Allow half the period for wire and half for logic and flop overhead.

Method. Divide the available wire time by the per-millimetre delay from the table.

Period	1,000 ps
Wire budget	500 ps
Delay per mm, buffered	57.6 ps/mm
Reachable distance in one cycle	8.68 mm
At 3 GHz	2.89 mm
Typical die edge	10 mm

How this is got wrong. By assuming any module can talk to any module in one cycle. At 3 GHz a signal crosses a fraction of a large die per cycle, so a chip-spanning bus is physically impossible at that frequency and the architecture must be pipelined, hierarchical, or both. **This is the physical origin of networks-on-chip:** not a preference for packets, but the observation that a global broadcast wire no longer exists. For an IP vendor the practical consequence is to keep the block's internal critical paths local and to **register every boundary signal**, so the customer's floorplanner has somewhere to put the pipeline stages the distance demands.

X14.2 What synthesis can and cannot fix

Table 251. Where a problem must be solved

Problem	Can synthesis fix it?	Who must
Slow gate on the critical path	Yes — upsizing, Vt swap	The tool
Badly structured arithmetic	Usually — it re-implements operators	The tool, if the RTL uses + and * rather than hand-built structures
Too many logic levels between flops	No	The RTL — add a pipeline stage
High fan-out net	Partly — it buffers	The RTL, if the fan-out is architectural (a global enable)
Congestion from a crossbar	No	The architecture — reduce the radix or add a stage
Clock domain crossing	No — and it may <i>break</i> one by optimising away a synchroniser	The RTL, plus constraints marking the crossing
Latch inferred from an incomplete case	It will report it	The RTL — always
Multi-cycle path	No	The constraint file, with justification

Synthesis can delete your synchroniser. The two flops of a synchroniser are functionally redundant to a tool that believes the input is a normal signal: it may merge them, retime through them, or move one across a boundary, any of which destroys the resolution time the MTBF calculation of Part X3 depends on. The defences are to mark the flops with a synthesis attribute that forbids merging and retiming, to place them in a module the tool is told not to ungroup, and **to check the gate-level netlist that they are still there** — a check that can be automated and belongs in the sign-off list. A design that relies on the tool's good behaviour here has an MTBF nobody has verified.

X14.3 Utilisation, congestion and the numbers that predict trouble

Table 252. Relative routing difficulty against placement utilisation (heuristic model)

Utilisation (%)	Relative difficulty	vs. 50 %	Practical verdict
50	2.828	1	comfortable
60	3.953	1.4	comfortable
70	6.086	2.15	comfortable
75	8	2.83	tight
80	11.18	3.95	tight
85	17.21	6.09	expect congestion
90	31.62	11.2	expect congestion

The model is a heuristic, not a PDK number: difficulty grows as $(1-u)^{-1.5}$. **Its only job is to make the shape visible** — the cost is mild to 70 % and then rises very fast, which is why floorplans target 65–75 % and why ‘we will just raise utilisation’ is rarely the answer to an area problem.

Graduate · Congestion is caused by pin density and connectivity, not by cell area, and this distinction decides what an RTL designer can do about it. A block that is 60 % utilised can be unroutable if its cells are small and richly connected — a crossbar, a large multiplexer tree, a register file read network — because the routing demand per unit area is set by nets crossing a boundary, not by transistors under it. The RTL-level remedies are therefore about *connectivity*: reduce the radix of a switch and add a stage; replace a fully-connected structure with a hierarchical one; move a wide mux into a memory where the decoder is dense and local; and **avoid the one-hot broadcast structures that look cheap in gate count and are expensive in tracks**. An area report will not show any of this; a congestion map will.

Problem. Your block meets timing standalone at 800 MHz but the customer reports it fails at 600 MHz in their chip. Name four causes, in the order you would check them.

Solution. 1. Different constraints. Your standalone run may have used ideal clocks, no input/output delay, or a different corner set. Ask for their SDC and compare it with yours line by line; this is the cause more often than the other three together. **2. Boundary paths.** Standalone runs frequently constrain inputs and outputs generously; in the chip your block's inputs arrive late and its outputs must arrive early. This is why registering every boundary is worth the latency — it makes your timing independent of their floorplan. **3. Physical context.** In their chip your block may be stretched around a macro, sit far from its clock source, or share power rails with a noisy neighbour, all of which add delay your standalone run did not model. **4. A different library or corner.** Same process, different vendor library or different Vt mix, and your critical path is no longer their critical path. **The general lesson for a vendor is to ship the constraints and the conditions with the frequency claim;** a frequency without them is not a specification, and this exact conversation is why.

Problem. Why do IP vendors quote area in gate equivalents rather than square micrometres?

Solution. Because square micrometres are meaningless without naming the process, the library and the utilisation, and a vendor who names all three has effectively told a competitor which foundry and library they used. A gate equivalent — conventionally the area of a two-input NAND in the same library — is a ratio, so it survives the translation between processes approximately, and it lets a customer scale to their own library by multiplying by their NAND area. **The approximation is worse than it looks,** because the ratio of flop area to NAND area and of SRAM bit-cell area to NAND area both vary between libraries, so a flop-heavy block converts less accurately than a logic-heavy one. The honest datasheet therefore gives **gate equivalents, flop count and memory bits separately**, which lets the customer do the conversion properly, and costs the vendor nothing.

X15. Hardware Security, Worked

Security is where a plausible-looking design is most often wrong, because the adversary is not a random input but a person who reads your datasheet. This part covers the parts of hardware security that are arithmetic — how many traces an attack needs, how much entropy a source really has, how long a key survives — because those are the parts a designer can actually bound.

X15.1 Side channels: the attack is a statistics problem

Why correlation power analysis works, and how many traces it needs

	Step	Justification
1	Dynamic power depends on the number of bits that switch, which depends on the data being processed.	Part X7's $\alpha CV^2 f$: α is data dependent.
2	Choose an intermediate value $v = f(\text{plaintext, key guess})$ that the device computes, e.g. an AES S-box output.	It must depend on a small part of the key so the guesses can be enumerated.
3	Predict the leakage as a model, commonly the Hamming weight of v .	A crude model that works because switching count correlates with it.
4	Correlate the predicted leakage with the measured power across many traces, for every key guess.	The correct guess correlates; the others do not, because their predicted values are effectively random with respect to the device's actual computation.
5	The number of traces needed is $n \approx 3 + 8(Z_\alpha / \ln((1+p)/(1-p)))^2$.	Fisher's z-transform of the correlation coefficient. Quadratic in $1/p$, which is why reducing the signal by a factor of ten costs the attacker a factor of a hundred in traces.

Table 253. Traces needed for a correlation attack against the leakage correlation ρ (99.99 % confidence)

ρ	Traces needed	Relative to $\rho = 0.5$	Acquisition hours at 1 ms per trace
0.5	109	1	3.03×10^{-5}
0.2	781	7.167	2.17×10^{-4}
0.1	3,181	29.18	8.84×10^{-4}
0.05	12,781	117.2	0.00355
0.02	79,981	733.4	0.0222
0.01	319,981	2934	0.0889
0.005	1,279,981	11737	0.356

Computed from the z-transform expression. **Read the last column as the security margin:** an unprotected implementation falls in minutes, and each factor of ten in leakage reduction costs the attacker a factor of a hundred in time. A countermeasure does not have to be perfect, it has to move this column past the value of the key.

Table 254. Countermeasures, what each does to ρ , and what it costs

Counter-measure	Mechanism	Effect	Cost
Masking	Split each secret into shares whose individual distributions are independent of the secret	Removes first-order leakage entirely; the attacker must combine $d+1$ points, and the trace count grows exponentially in d	2–4× area and a source of fresh randomness every cycle

Counter-measure	Mechanism	Effect	Cost
Hiding (dual rail, WDDL)	Make switching activity constant	Reduces p , does not remove it — routing imbalance leaks	2× area, careful layout
Shuffling	Randomise the order of independent operations	Divides p by roughly the number of permuted slots	Cheap; combines well with masking
Noise injection	Add uncorrelated activity	Reduces p by the amplitude ratio	Power; attacker averages it away — buys time, not security
Frequency randomisation	Jitter the clock	Misaligns traces; defeated by alignment preprocessing	Cheap and weak on its own
Key refresh	Limit the traces available per key	Caps the attack directly	Protocol support

A masked implementation that is not verified at the gate level is usually not masked. Masking is a property of the *computation*, and synthesis optimises computations. If the tool finds that two shares are combined in a common subexpression — which is exactly the kind of optimisation it is good at — the shares meet in one gate and the first-order leakage returns, while the RTL still looks masked. There is a second, subtler failure: even without optimisation, a **glitch** in combinational logic can momentarily present a function of both shares, which is why threshold implementations are designed to be glitch-resistant by construction rather than merely share-correct. **The verification here is formal leakage analysis on the netlist**, not inspection of the source.

X15.2 Entropy: counting what a random number generator really has

Worked example 28. A ring-oscillator TRNG, entropy per bit

Given. Two free-running ring oscillators, one sampled by the other. Jitter accumulates over the sampling interval; the accumulated timing uncertainty is $\sigma = 12$ ps and the sampled oscillator's period is $T = 400$ ps.

Method. The min-entropy of the sampled bit depends on how much of a period the uncertainty spans. A standard stochastic-model estimate is $H \approx 1 - (4/\pi^2)e^{-4\pi^2\sigma^2/T^2}$, which goes to 1 when the jitter covers a period and to 0 when it is small.

σ/T	0.03
Exponent argument	0.03553
Estimated entropy per raw bit	0.6089
Raw bits needed per full-entropy bit	1.642
If σ falls to 3 ps (a quieter supply): entropy	0.5956
Raw bits per full-entropy bit then	1.679

How this is got wrong. By testing the output of the conditioner instead of the source. A cryptographic hash applied to a biased source produces output that passes every statistical test — that is what a hash does — while containing only as much entropy as went in. **The test must be applied to the raw source**, with an online health test that keeps checking it in the field, because the quantity being relied on (σ) depends on supply noise, temperature and ageing, and an attacker with access to the supply can reduce it deliberately. The second row from the bottom is that attack expressed as arithmetic.

Graduate • Why the standards insist on min-entropy rather than Shannon entropy. Shannon entropy is an average; an adversary guesses the most likely value. A source that emits 0 with probability 0.9 has Shannon entropy 0.47 bits and min-entropy $-\log_2(0.9) = 0.15$ bits, and the second number is the one that bounds an attacker's success. For a key of k bits assembled

from such a source, the attacker's work is 2 raised to the total *min*-entropy, not the total Shannon entropy — a factor of $2^{0.32}$ per bit, which over 256 bits is a factor of 10^{24} . **Using the wrong entropy measure is not conservative rounding, it is a wrong security claim.**

X15.3 Fault attacks and the cost of detection

Table 255. Fault-attack countermeasures

Countermeasure	Mechanism	Catches	Overhead
None	—	0 %	0 %
Parity on registers	1 bit per word	single-bit faults	3–12 %
Duplication	Run twice, compare	any single fault	>100 %
Time redundancy	Run twice in sequence	transient faults only	0 % area, 100 % time
Inverse check	Decrypt the ciphertext and compare	any fault in the datapath	~100 % time
Infective computation	Spread the fault so the output is useless	prevents exploitation rather than detecting	moderate

*The classic differential fault attack on RSA-CRT recovers the private key from **one** faulty signature, which is why the inverse check — verify the signature before releasing it — is standard there despite its cost. **When one fault is enough, probabilistic countermeasures are not.***

Problem. A customer asks for a ‘secure’ AES block. What do you ask them before quoting?

Solution. Four questions, because ‘secure’ alone does not specify anything. **What is the attacker's physical access?** A block in a server that an attacker cannot touch needs no side-channel countermeasures; a smart card that the attacker owns needs every one of them, and the area difference is a factor of three. **How many operations per key?** This bounds the traces available and therefore the required leakage reduction directly, via the table in X15.1. **What certification is required?** Common Criteria or FIPS levels prescribe specific countermeasures and specific evidence, and the evidence costs more than the design. **What is the throughput and latency requirement?** Masking costs randomness per cycle, and a design that must run at line rate may not be able to afford the randomness generation. **A quote given without these answers will be wrong by a large factor in one direction or the other,** and a vendor who asks them is demonstrating exactly the competence being bought.

Problem. Why is a hardware root of trust harder than storing a key in fuses?

Solution. Because the key is the easy part. A root of trust must establish, at every boot, that the first code to run is the code the owner intended — which requires immutable boot code, a verified signature chain, and protection of the verification itself from the fault and side-channel attacks above. It must survive the states nobody designs for: reset during an update, a power glitch mid-verification, a debug interface left enabled. It needs a way to revoke a compromised key and to roll forward a version counter that cannot be rolled back, which means monotonic storage, which means more fuses and a policy for them. And it must do all this while remaining **debuggable in the field by the legitimate owner**, which is the requirement that quietly reopens every door the rest closed. **The key in fuses is perhaps five per cent of the work;** the rest is the state machine around it, and that state machine is where the published attacks land.

X16. Reliability, Yield and Functional Safety, Worked

Three quantities decide whether a chip can be sold into an automotive, industrial or medical market, and all three are arithmetic: the fraction of dies that work, the rate at which working dies stop working, and the fraction of dangerous failures that are detected. A design house that can compute them can quote for markets that pay considerably more than consumer.

X16.1 Yield: the model and its parameter

From defect density to die yield

	Step	Justification
1	Defects fall on the wafer at an average density D_0 per unit area.	<i>Empirical; the foundry supplies it and it falls over a process's life.</i>
2	If defects were Poisson and independent, yield would be e^{-AD_0} .	<i>Probability of zero defects in area A.</i>
3	They are not independent — defects cluster — so Poisson is pessimistic for large dies.	<i>Clustering means some wafers are bad and others clean, which raises the average yield of the clean ones.</i>
4	The negative-binomial (Murphy/Seeds) model adds a clustering parameter α : $Y = (1 + AD_0/\alpha)^{-\alpha}$.	<i>$\alpha \approx 2-5$ in practice; $\alpha \rightarrow \infty$ recovers Poisson.</i>
5	Yield falls superlinearly with die area , and total good dies per wafer falls faster still.	<i>Which is the whole argument for chiplets, and the reason a large monolithic die is a business decision as much as a technical one.</i>

Table 256. Yield against die area at $D_0 = 0.10 / \text{cm}^2$, $\alpha = 3$, 300 mm wafer

Die area (mm ²)	Poisson yield	Negative-binomial yield	Gross die/wafer	Good die/wafer	Relative cost per good die
10	0.99	0.9901	6,503	6,438	1.01
25	0.9753	0.9754	2,601	2,537	1.025
50	0.9512	0.9516	1,300	1,237	1.051
100	0.9048	0.9063	650	589	1.103
200	0.8187	0.824	325	267	1.214
400	0.6703	0.687	162	111	1.456
800	0.4493	0.4921	81	39	2.032

The last column is cost per working die relative to a perfect-yield part, holding wafer cost fixed. An 800 mm² die costs far more than eighty times a 10 mm² one, which is the economics that produced chiplets.

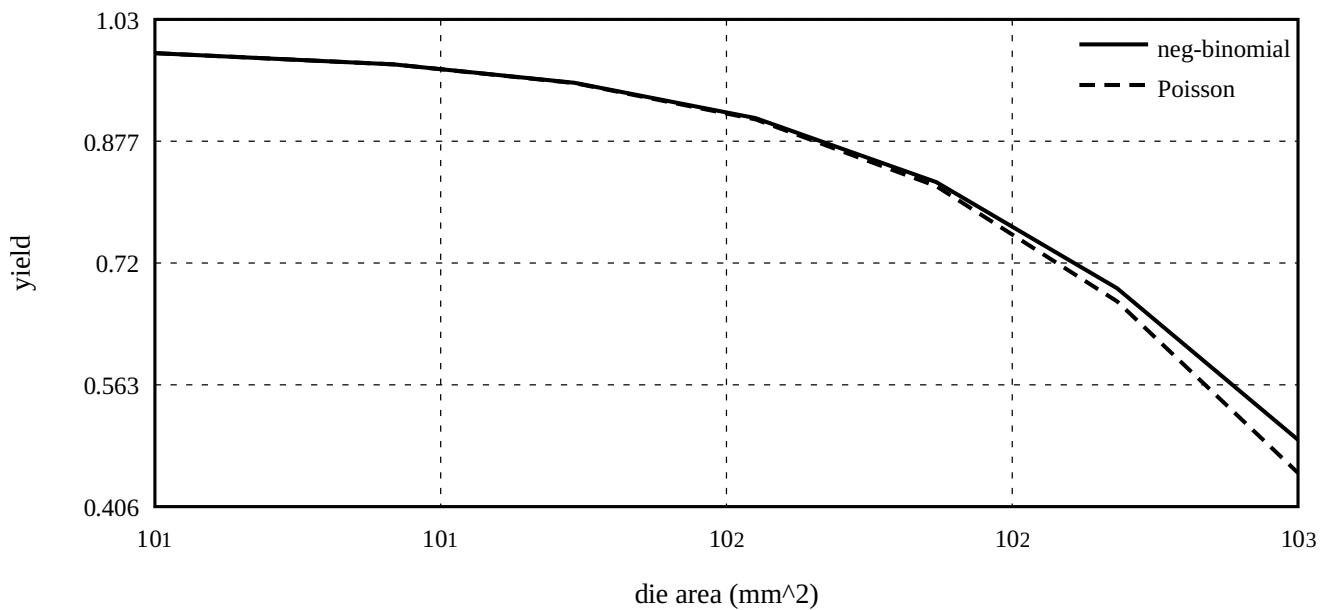


Figure 36. The two models diverge exactly where the decision is hard. Using Poisson for a large die understates yield and can kill a viable product; using it for a small die makes no difference.

Worked example 29. Redundancy in a memory, and when it pays

Given. A 16 Mbit SRAM occupying 20 mm². Adding 2 % area of spare rows and columns with laser or fuse repair can repair any single defect in a row or column.

Method. Compare the yield of the memory with and without repair, using the negative-binomial model, and weigh the 2 % area against the yield gain.

Memory area	20 mm ²
Yield without repair	0.9803
With 2 % area added	0.9799
Defects repairable (assume 80 % are single row/column)	80 %
Effective yield with repair	0.996
Gain	0.016
Area cost	2 %

How this is got wrong. By applying the same reasoning to random logic. Memory repair works because memory is regular: a spare row is interchangeable with any row, so one spare covers a whole class of defects. Random logic has no such structure, and the equivalent — triple modular redundancy — costs over 200 % rather than 2 %. **This asymmetry is why yield-enhancement effort concentrates on memory**, and why a design that is 60 % memory by area has a very different yield story from one that is 60 % logic. It is worth knowing which yours is.

X16.2 Wear-out: the four mechanisms and their accelerations

Table 257. Why parts that passed test fail later

Mechanism	Physics	Accelerated by	Design response
Elec-tromigration	Momentum transfer from electrons moves metal atoms; voids open	Current density and temperature	Width rules per current; the tool checks them if told the currents

Mechanism	Physics	Accelerated by	Design response
NBTI / PBTI	Interface traps shift the threshold voltage under bias	Voltage, temperature, and duty cycle	Guardband; some recovery when the stress is removed, which makes measurement subtle
TDDDB	Gate oxide accumulates damage until it conducts	Electric field, exponentially	Voltage limits; thicker oxide for I/O
Hot carrier injection	Energetic carriers embed in the oxide	Switching activity and short channels	Slew limits; affects high-activity nodes such as clocks
Soft errors	Ionising particle upsets a node	Altitude, cell size, node capacitance	ECC and scrubbing — Part X10

Table 258. Arrhenius acceleration relative to 55 °C, $E_a = 0.7$ eV

Junction temperature (°C)	Acceleration factor	Field years equivalent to 10 test hours	Field years equivalent to 1000 test hours
55	1	10	1000
85	7.953	1.257	125.7
105	26.4	0.3788	37.88
125	77.66	0.1288	12.88
150	259.2	0.03857	3.857

*This is how a fifteen-year automotive lifetime is qualified in weeks: stress at 150 °C and multiply. **The activation energy is per mechanism and is the parameter the whole extrapolation rests on**; quoting a lifetime without naming E_a and the mechanism it was measured for is not a lifetime claim.*

Acceleration factors multiply the error in E_a enormously. At 150 °C the factor above is roughly 170; had E_a been 0.5 eV instead of 0.7, it would be about 40. So a 10 % uncertainty in an exponent's parameter becomes a factor of four in the predicted lifetime, and the direction of the error is not conservative — assuming a high E_a makes the test look more severe than it was. **This is the same structural hazard as the metastability τ of Part X3:** a quantity inside an exponent, measured with modest precision, on which a large claim rests. Both deserve the same treatment: use the worst plausible value, and say which value you used.

X16.3 Functional safety: what ISO 26262 actually asks for

Table 259. The safety metrics, in plain arithmetic

Metric	Definition	ASIL B	ASIL D
SPFM	Fraction of single-point faults that are safe or detected	≥ 90 %	≥ 99 %
LFM	Fraction of latent multi-point faults detected	≥ 60 %	≥ 90 %
PMHF	Probabilistic metric for random hardware failures	< 100 FIT	< 10 FIT
FMEDA	The document that computes all three, gate by gate	Required	Required

Worked example 30. What a 99 % SPFM target means for a block

Given. A block with 500 000 gates and a base failure rate of 10 FIT for the whole block. ASIL D requires $\text{SPFM} \geq 99\%$.

Method. Compute the undetected fault rate the target permits, and then ask what fraction of the design must be covered by a safety mechanism to reach it.

Block failure rate	10 FIT
Permitted undetected single-point rate	0.1 FIT
Fraction of faults needing detection	99 %
Gates that must be covered (if faults are uniform)	495,000
Typical coverage of parity on registers	60–80 %
Typical coverage of duplication and compare	> 99 %
Area cost to reach ASIL D on the datapath	duplication — over 100 %

How this is got wrong. By assuming safety is a verification activity. It is an architecture activity: 99 % coverage is not reachable by testing harder, it is reachable by building detection into the design, and detection costs area that must be budgeted at the start. The usual structure is a **mixed** one — duplication on the control path, where it is small and a fault is dangerous; ECC or parity on memories, where the regularity makes it cheap; and end-to-end checks such as a CRC over a datapath, where a single check covers a great deal of logic. **A safety manual naming those mechanisms, with the coverage claimed for each, is a deliverable a customer pays for.**

Problem. Why is a ‘latent’ fault treated separately, and why does it need periodic testing?

Solution. A latent fault is one in a *safety mechanism* rather than in the function: the comparator that would detect a duplication mismatch has itself failed stuck at ‘equal’. On its own it is harmless — the function still works — which is exactly why it is dangerous: nothing reveals it, so it accumulates, and when a real fault eventually occurs the detection that was counted on is absent. The metric that bounds this is LFM, and the mechanism that serves it is a **periodic self-test**: at start-up and at intervals during operation, deliberately inject a fault and confirm the detector fires. **This is the same argument as the verification agent's self-check in Part Y4 and as this repository's whole rule about checking the checker** — a detector that is never exercised is not known to work, and in a safety context that is not a style preference but a numbered requirement with a quantitative target.

X17. Reading the Current Literature

A textbook that stops at settled material leaves the reader unable to read this year's proceedings. This part covers the directions that a designer entering the field now will meet, treated the way the rest of the book treats everything: by the arithmetic that makes each one attractive and the arithmetic that limits it. No performance claim is attributed to a specific paper here, because this repository's rule is that a citation carries a confidence level and an unread paper cannot carry a high one.

X17.1 Chiplets: the arithmetic that made them inevitable

Part X16 computed that yield falls superlinearly with die area. Chiplets exploit that directly, and the trade is entirely quantitative.

Table 260. Splitting 600 mm² of silicon into n chiplets, yield only

Chiplets	Area each (mm²)	Die yield	All- n -good yield	Relative silicon cost	Interfaces to design
1	600	0.5787	0.5787	1.728	0
2	300	0.7513	0.5645	1.772	1
4	150	0.8638	0.5568	1.796	6
8	75	0.9286	0.5529	1.809	28

The fourth column assumes every chiplet must be good, which is the pessimistic case; known-good-die testing before assembly is what makes the scheme work at all, and its cost is the missing term. **The gain is real and the last column is why it is not free.**

Worked example 31. What a die-to-die link costs, and why it decides the partition

Given. Two chiplets exchanging 1 TB/s. A die-to-die interface achieves roughly 1 pJ/bit; an on-die wire of the same length costs roughly 0.05 pJ/bit.

Method. Compute the power of the link and compare with a monolithic implementation. The number decides where the partition can be drawn.

Bits per second	8.00×10 ¹²
Die-to-die power at 1 pJ/bit	8 W
On-die equivalent at 0.05 pJ/bit	0.4 W
Penalty	7.6 W
As a fraction of a 300 W package	2.53 %
Bandwidth affordable in a 15 W budget	1.88 TB/s

How this is got wrong. By partitioning on functional boundaries instead of on bandwidth. The correct cut is the one with the least traffic across it, which is frequently *not* the tidiest functional boundary: splitting compute from memory looks natural and puts the highest-bandwidth interface across the cut, while splitting compute from I/O puts a much lower-bandwidth interface there. **Draw the traffic graph before drawing the partition**, exactly as Part X11's bisection argument requires, and note that the answer changes with the workload — which is why chiplet partitions are argued about for months.

X17.2 Near- and in-memory computing

Why moving computation to the data is attractive, and what bounds it

	Step	Justification
1	Part X13 measured arithmetic intensity: most layers are memory bound.	<i>So the machine spends its energy on movement, not arithmetic.</i>
2	Reading a 64-bit word from DRAM costs on the order of 100× the energy of a 64-bit add.	<i>The ratio varies with process and distance; the order of magnitude is the robust part.</i>
3	If the computation happens where the data is, that ratio is avoided for whatever it can do there.	<i>The gain is bounded by the fraction of work that can move, which is Amdahl's law applied to data movement rather than to time.</i>
4	Analogue in-memory computing performs the multiply–accumulate in the array itself, using conductance as the weight.	<i>One column sums currents; the addition is free and the multiply is Ohm's law.</i>
5	Its limits are precision, variability and the converters.	<i>The ADC at the bottom of each column frequently dominates area and power, which returns the design to Part X12's converter arithmetic. A technique that removes one bottleneck by creating another has to be evaluated on the sum.</i>

Table 261. A crude accounting for an analogue in-memory column: the converter dominates as precision rises (relative energy units)

Output bits	Array MAC energy	Column ADC energy	ADC share of the column (%)	Levels to resolve
2	0.02	0.5	96.2	4
4	0.02	2	99	16
6	0.02	8	99.8	64
8	0.02	32	99.9	256

*The model is illustrative, not measured, and is included to show the shape of the constraint rather than a value. **The shape is the point:** analogue in-memory computing is attractive at low output precision and loses its advantage as precision rises, which is why the published work concentrates on heavily quantised inference.*

Judge any in-memory result on the complete system, including the converters, the peripheral circuits and the data marshalling. An array-only energy figure is the analogue equivalent of a TOPS number: true, and about a part of the system that was never the bottleneck. The questions that separate a result from a product are: what precision at the output, what does the ADC cost, how is a weight updated and how often, what happens to accuracy over temperature and drift, and what fraction of a real network maps onto the array at all. **Each of those is a question this book has asked about a conventional block somewhere, which is the point: the discipline transfers.**

X17.3 Machine learning applied to the design flow itself

Table 262. Where learning is used in EDA, and what the oracle is

Application	What is learned	The oracle	Why it can work
Placement and floorplanning	A policy for placing macros	The downstream tool's own metrics	The reward is computable, if slowly — a real oracle exists

Application	What is learned	The oracle	Why it can work
Routing congestion prediction	A map from placement to congestion	The router's result	Plentiful labelled data from past runs
Timing prediction before routing	Post-route delay from pre-route features	The signoff tool	Saves iterations rather than replacing signoff
Parameter tuning	Good tool settings for a design	The tool's output	The highest-return and least glamorous application
RTL generation from natural language	Code	Nothing, unless you supply one	This is the case that differs in kind, not degree
Testbench and assertion generation	Properties and stimulus	Coverage, mutation score	An oracle exists and is measurable

Graduate · **The organising question for every row is: what verifies the output?** Where a slow but trustworthy oracle exists — the router, the timing signoff tool, a golden model — learning is search, and search with a good oracle is reliable and improves with compute. Where no oracle exists, the output must be read by a human, and the failure mode is code that looks correct, which is the most expensive kind. Part Y4's agent is built on exactly this distinction and is worth re-reading in this light: it searches a small space against a strong oracle and rejects candidates that pass only the data they were fitted to. **The strategic conclusion for a design house is that investment in golden models and checkers appreciates as generation improves**, because they are what converts an unreliable generator into a reliable one, and they are what a competitor who has only a generator cannot borrow.

Problem. A tool vendor claims their learned placer improves timing by 10 %. What do you ask?

Solution. **Against what baseline, tuned how hard?** A learned placer compared against a default-configured conventional flow is compared against a strawman; a human expert spends days on tool settings, and the comparison must include that effort or exclude it from both sides. **On how many designs, and were they in the training set?** Improvement on designs resembling the training distribution is the expected result and says nothing about yours. **What is the variance?** Placement is stochastic; a 10 % mean improvement with a 15 % spread across seeds is not an improvement you can schedule around. **What does it cost in runtime and in setup?** A placer that needs a week of fine-tuning per design is a service, not a tool. **And what happens when it fails?** A conventional flow degrades predictably; a learned one may produce a result that is fine on the metric it optimised and bad on one it did not see. These are the same questions Part Y6 asks of any paper, which is the intended lesson: **the reading discipline does not change because the subject is fashionable.**

Problem. Your customer asks whether your IP is 'AI-generated'. How do you answer?

Solution. Directly, because the question behind it is about provenance and liability rather than about tooling. Say what tools were used and for what; say what verification the output passed, which is the part that actually bounds the risk; and say what you can warrant about the origin of the code — that it is not derived from licensed or copyleft sources, and that you can produce a per-file record. **A customer's real concern is that they will inherit a legal problem or an unverified block**, and both are answerable with evidence you should have anyway. Evasion is the worst answer available: it converts a procurement question into a trust question, and Part Y5's argument about the known-issue list applies exactly — the supplier who volunteers the uncomfortable detail is the one who gets called next time.

X18. Performance Modelling: Queues, Bursts and Tails

Most hardware performance questions are queueing questions in disguise: how deep must this FIFO be, how long will this request wait, what happens when two masters are busy at once. The answers are available in closed form for the simple cases and by simulation for the rest, and having both is what lets a designer say ‘sixteen entries’ with a reason attached.

X18.1 Little's law, which is almost too simple to be useful and is not

Little's law and its three uses

	Step	Justification
1	In any stable system, $L = \lambda W$: average number in the system equals arrival rate times average time in the system.	<i>It requires only stability — no distributional assumption whatever, which is why it is the most portable result in the subject.</i>
2	Use 1: from occupancy and rate, get latency.	<i>Measure the average FIFO occupancy and the throughput, and the latency comes out without instrumenting it.</i>
3	Use 2: from rate and latency, get the buffer needed.	<i>This is the bandwidth–delay product</i> , which Parts X11, Y2 and Y3 each derived separately. They are the same theorem.
4	Use 3: as a consistency check on a simulation.	<i>If measured L, λ and W do not satisfy it, the measurement is wrong — an independent cross-check of exactly the kind Part Y6 insists on, available free.</i>

X18.2 Utilisation and the tail

Table 263. Queue length against utilisation: exponential service (M/M/1) and deterministic service (M/D/1)

Utilisation ρ	Mean queue, M/M/1	Mean queue, M/D/1	Mean time in system (service units)	Depth for 1 % overflow, M/M/1
0.3	0.4286	0.06429	1.429	4
0.5	1	0.25	2	7
0.7	2.333	0.8167	3.333	13
0.8	4	1.6	5	21
0.9	9	4.05	10	44
0.95	19	9.025	20	90
0.99	99	49	100	459

*Deterministic service halves the queue, which is the quantitative reason fixed-size flits and fixed-latency pipelines are preferred in hardware: **re-mov-ing service-time variance is worth as much as halving the load.***

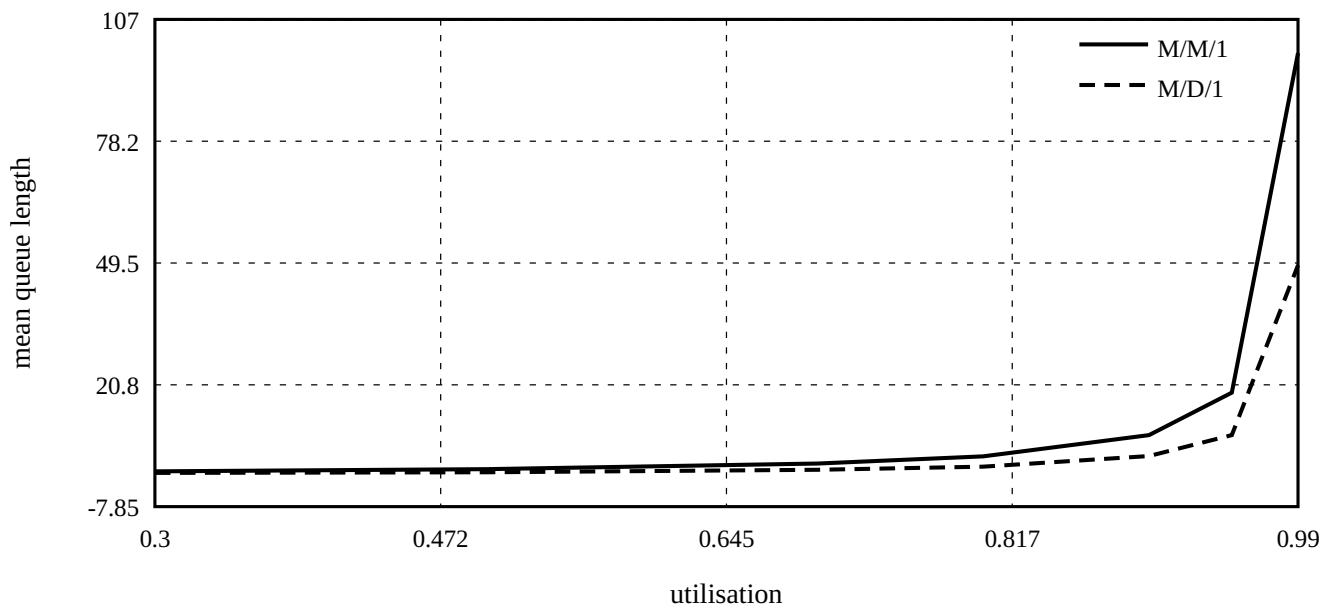


Figure 37. Both curves have a pole at $\rho = 1$. A design that targets 90 % utilisation is targeting the steep part, and small errors in the load estimate become large errors in the buffer requirement.

Graduate · Why hardware designers can be more aggressive about utilisation than the curve suggests, and where that reasoning fails. The classical curves assume Poisson arrivals, which model independent users. On-chip traffic is frequently *self-limited*: a master with four outstanding transactions cannot offer more than four, so the arrival process is closed rather than open and the queue cannot blow up — the right model is a closed network, and utilisation near one is achievable. The reasoning fails exactly when the traffic becomes open: a DMA engine with deep outstanding support, several masters that do not coordinate, or a packet interface fed from outside the chip. **Ask whether the source can be throttled by the queue itself;** if it can, be aggressive, and if it cannot, respect the pole.

Table 264. Measured drop rate of a FIFO against depth, bursty arrivals (8-cycle bursts at 1/cycle, 20 % of slots start a burst, service one every 2 cycles)

Depth	Arrivals	Drops	Drop fraction	Peak occupancy reached
1	39,192	15,642	0.399	1
2	39,192	11,688	0.298	2
4	39,192	3,780	0.0964	4
8	39,192	852	0.0217	8
16	39,192	72	0.00184	16
32	39,192	0	0	28

Simulated with seed 37. The same arrival trace is replayed at every depth, so the arrivals column is identical by construction — only the buffer changes. The average offered load is 0.196 items per cycle against a service rate of 0.5, so the system is **stable on average and still drops at small depths** — because the burst arrives faster than the service can drain it. The depth at which drops stop is set by the burst, not by the average.

Worked example 32. Sizing a FIFO from a burst specification rather than an average

Given. A source delivers a 64-byte burst at one byte per cycle, then idles for at least 192 cycles. The consumer drains at one byte every 2 cycles.

Method. Compute the accumulation during the burst and the drain afterwards; the peak occupancy is the answer, and it depends only on the burst, not on the average.

Burst length	64 cycles
Arrival during burst	64 bytes
Drain during burst	32 bytes
Peak occupancy	32 bytes
Average arrival rate	0.25 B/cycle
Average service rate	0.5 B/cycle
Utilisation	0.5
Depth an average-based calculation would suggest	small — and wrong

How this is got wrong. By computing the utilisation, seeing 0.5, and choosing a shallow FIFO. The system is only half loaded on average and still needs 32 bytes of storage, because the buffer's job is to absorb the *difference between the arrival and service patterns*, not to absorb the average. **The general procedure is to integrate the difference between cumulative arrivals and cumulative service and take the maximum** — a construction that generalises to any traffic pattern you can describe, and that is the basis of the network-calculus bounds used where a hard guarantee is required rather than an average.

Problem. Your interconnect meets its bandwidth requirement in simulation but a customer reports missed real-time deadlines. What is the likely disagreement?

Solution. You measured throughput and they need **bounded latency**, and the two are almost unrelated once utilisation is high. An arbiter that delivers full bandwidth can make one requester wait arbitrarily long — fixed priority does this by construction, as Part X11's measurement showed. The resolution has three parts. **Establish which requester has the deadline and what it is**, because a real-time requirement is a per-requester property and an aggregate number cannot express it. **Measure the latency distribution, not the mean:** report the 99.9th percentile and the maximum observed, since a deadline is violated by the tail. **Then change the mechanism, not the tuning:** a bandwidth regulator plus a priority class, or a reserved slot, gives a bound; adjusting weights gives an average. **No amount of throughput tuning produces a latency guarantee**, and recognising that these are different requirements early saves a redesign late.

X19. Processors as IP, Worked

The processor core is the archetype of licensable IP and the block most likely to sit beside whatever a design house builds. This part treats it the way the rest of the book treats blocks: by the arithmetic that sets its size, its speed and its verification cost.

X19.1 Where the cycles go

The performance equation and what each term belongs to

	Step	Justification
1	Time = instructions × cycles per instruction × cycle time.	<i>The identity. Its value is that the three terms belong to different people.</i>
2	Instructions: the compiler and the instruction set.	<i>A richer ISA lowers this and usually raises one of the other two.</i>
3	CPI: the micro-architecture — pipeline, caches, branch prediction.	<i>Where a core designer spends their effort.</i>
4	Cycle time: the implementation and the process.	<i>Where Part X2's budget applies directly.</i>
5	An improvement that lowers one term and raises another by more is a regression , and this is the standard failure of ISA extensions.	<i>A new instruction that saves two instructions but lengthens the critical path by 5 % has to save 5 % of all instructions to break even.</i>

Table 265. Illustrative core points: performance is frequency divided by CPI

Core	CPI	MHz	MIPS (approx.)	Relative performance
Simple 3-stage, in-order	1.35	400	0.2963	1
5-stage, in-order, no cache	1.2	600	0.5	1.688
5-stage with I/D caches	1.45	900	0.6207	2.095
Dual-issue in-order	0.95	900	0.9474	3.197
Out-of-order, 3-wide	0.55	1,200	2.182	7.364

These are illustrative operating points chosen to show the structure of the trade, not measured benchmark results for any particular core. Note the third row: adding caches raises CPI, because a miss costs cycles the cacheless design never paid — and still wins, because it raises the achievable frequency far more.

Graduate · **The third row is the most instructive line in this chapter.** A design without caches has a low, predictable CPI and cannot run fast, because every access goes to a memory whose latency sets the cycle time. Adding caches introduces misses and therefore raises average CPI, and simultaneously removes the slow memory from the critical path, which raises frequency by more. **Optimising CPI in isolation would have rejected the change.** This is the general hazard of single-metric optimisation, and it is why architectural decisions are evaluated on the product of the three terms and never on one of them.

Table 266. Memory stall as a fraction of CPI

Miss rate (%)	Miss penalty (cycles)	CPI	Stall share of CPI (%)	Relative performance
1	10	1.1	9.09	0.9091
2	10	1.2	16.7	0.8333

Miss rate (%)	Miss penalty (cycles)	CPI	Stall share of CPI (%)	Relative performance
5	10	1.5	33.3	0.6667
2	30	1.6	37.5	0.625
2	100	3	66.7	0.3333
0.5	100	1.5	33.3	0.6667

The last two rows are the same miss penalty at 2 % and 0.5 % miss rates. **At a 100-cycle penalty a 2 % miss rate spends two thirds of every cycle waiting**, which is why a small improvement in hit rate is worth more than a large improvement in almost anything else — and why prefetching, which attacks the penalty rather than the rate, is worth its complexity.

X19.2 What a processor IP licence actually contains

Table 267. The deliverable, and why a core is expensive

Item	Note
RTL or a hard macro	The smallest part of the value
The compiler toolchain	A core without a compiler is unusable. GCC and LLVM ports, a libc, a debugger
Instruction-set simulator	For software development before silicon, and as the golden model for verification
The architectural compliance suite	The thing that makes the core the ISA it claims to be. For RISC-V this is a formal specification plus a test suite
Debug and trace	A standard debug module; trace if the customer needs it
Operating-system ports	Where applicable — Linux, an RTOS
Verification collateral	So the customer can re-verify after configuring
Configuration tooling	A core with 30 options has 2^{30} configurations; the tool generates and verifies the chosen one

The configuration explosion is the hardest verification problem in processor IP and it is worth understanding as arithmetic. A core with thirty independent options has more configurations than can ever be verified individually. What vendors do instead is to verify a *covering set*: choose configurations so that every option and every interacting pair is exercised at least once, which for pairwise coverage needs a number of configurations that grows logarithmically rather than exponentially — typically a few dozen. **The claim a customer should check is not ‘we verified every configuration’ but ‘here is the covering criterion and here is the set’**, and a vendor who can state the criterion is a different proposition from one who cannot.

Table 268. Configuration coverage: exhaustive versus pairwise

Independent options	All configurations	Options to cover singly	Option pairs	Configurations for pairwise coverage (approx.)
5	32	5	10	10
10	1,024	10	45	14
20	1,048,576	20	190	18
30	1,073,741,824	30	435	20

The pairwise column is an order-of-magnitude estimate from the standard combinatorial-testing result that pairwise coverage of n binary factors needs $O(\log n)$ tests. **The gap between the second and last columns is the entire practical argument for combinatorial test design.**

X19.3 Verifying a core against its specification

The code appendix of this book contains the machinery a processor-verification engineer uses, in its real form: **100,524 lines** of the Sail formal specification of RISC-V, **87,547 lines** of the Spike instruction-set simulator, and **9,365,313 lines** of the ar-

chitectural test suites. Reading them together is the most direct way to see what ‘a specification’ means when it has to be executable.

Table 269. Four ways to verify a core, and what each one catches

Method	How	Catches	Misses
Directed tests	Hand-written assembly per feature	What you thought of	What you did not
Architectural suite	The ISA body's own tests	Non-conformance to the ISA	Micro-architectural bugs the ISA does not constrain
Random instruction generation	Generate legal programs, run on core and on a reference, compare state	Interactions nobody enumerated	Rare sequences; needs enormous volume
Step-and-compare (co-simulation)	After every instruction, compare the core's architectural state with the reference simulator's	The exact instruction where they diverge — the debug cost collapses	Only what the interface exposes
Formal	Prove properties of the pipeline, the ISA decode, the memory ordering	Deep control bugs, over all inputs	Needs the properties written; capacity limits on the datapath

Graduate · **Step-and-compare is the technique worth internalising, because it generalises far beyond processors.** The idea is to compare not the final output but the state after every step, against a reference that advances in the same steps. The payoff is that the first divergence is the bug, so debug time falls from hours of backward reasoning to inspecting one instruction. The requirement is an interface that exposes committed state — in RISC-V cores this is conventionally a formal interface emitting, for every retired instruction, its program counter, its encoding, and the register and memory it wrote. **Designing that interface into a block is a small cost that pays for itself the first time something goes wrong,** and the same pattern applies to any block with a sequential golden model: a codec, a protocol engine, a FEC decoder.

Problem. Why can a core pass the full architectural test suite and still be broken?

Solution. Because the suite tests the **architecture** and bugs live in the **micro-architecture**. The ISA says what the visible state is after an instruction; it says nothing about what happens when a branch mispredicts while a load is outstanding and an interrupt arrives. Those interactions are where pipeline bugs live, and an architectural test that executes instructions in isolation never creates them. Three further gaps are worth naming. **Memory ordering** under concurrency is barely exercised by single-threaded tests and is the hardest part of a multi-core design to get right; it needs litmus tests derived from the memory model. **Privilege transitions** — traps, interrupts, nested exceptions, debug entry — are under-tested everywhere and are where security bugs concentrate. And **performance counters and debug** are architectural state that customers rely on and suites barely touch. The remedy is the one above: random program generation with step-and-compare, run for a very long time, with a stimulus generator biased toward the interactions rather than toward the instructions.

X20. Image and Video Coding as a Hardware Block

A video codec is the largest fixed-function block in most consumer SoCs and it exercises nearly everything in this book: transforms, arithmetic coding, enormous memory bandwidth, a hard real-time deadline and a standard with a compliance regime. It is also the clearest example of a block whose *decoder* is fully specified and whose *encoder* is not, which has a large commercial consequence.

X20.1 The asymmetry that defines the business

Table 270. Decoder and encoder are different products

	Decoder	Encoder
Specified by the standard	Exactly — bit-exact output is required	Only the bitstream syntax; how to choose it is free
Conformance	Run the conformance streams; output must match bit for bit	Any legal stream is conformant, however bad
Where the value is	Correctness and area	Rate-distortion decisions — the quality at a given bitrate
Differentiation	Little — all correct decoders agree	Large and lasting
Verification	Against the reference decoder, bit-exact	Against a quality metric on a corpus, statistically
A design house should	Build it if the volume justifies it	Build it if you have an algorithmic advantage to sell

Graduate · The bit-exactness requirement on decoders is what makes them verifiable and therefore commoditisable.

Because a conformant decoder's output is defined to the bit, a test is unambiguous: run the standard's conformance streams, compare to the reference output, and the answer is yes or no. That property makes verification tractable and makes every correct decoder equivalent, which drives margins to the cost of implementation. The encoder has no such test — quality is measured statistically on a corpus, and two encoders producing different streams can both be right — which is exactly why encoder quality remains a durable differentiator and why encoder IP is worth more per gate.

X20.2 Bandwidth: the number that sizes the block

Table 271. Frame store and bandwidth for video coding

Format	Frame (MB)	Raw read once (GB/s)	Decoder: ~3× (read ref, write, display)	Encoder: ~8× with motion search
1080p60 8-bit 4:2:0	2.966	0.1866	0.5599	1.493
4K60 10-bit 4:2:0	14.83	0.9331	2.799	7.465
8K60 10-bit 4:2:0	59.33	3.732	11.2	29.86
4K120 10-bit 4:2:2	19.78	2.488	7.465	19.91

The multipliers are indicative of a straightforward implementation and are the quantity every architectural trick attacks. **8K60 encoding at eight passes would need more bandwidth than the whole rest of a mobile SoC**, which is why real encoders use hierarchical search, reference compression and aggressive on-chip tiling.

Worked example 33. Why reference-frame compression exists

Given. 4K60 10-bit decoding, three passes over the frame store, in a system with 25 GB/s of usable DRAM bandwidth shared with everything else.

Method. Compare the requirement with a realistic allocation, then compute what a lossless 2:1 reference compressor buys.

Raw bandwidth required	2.799 GB/s
Plausible allocation to video	6 GB/s
Shortfall	-3.201 GB/s
With 2:1 lossless reference compression	1.4 GB/s
Still short by	0 GB/s
Energy saved at 20 pJ/byte	0.02799 W

How this is got wrong. By treating compression as a capacity optimisation. Its purpose here is *bandwidth and energy*, not storage: DRAM traffic is often the largest single power term in a video subsystem, so halving it halves that term. Note the requirement that the compression be **lossless**: a decoder must produce bit-exact output, so any loss in the reference frames propagates and the decoder is no longer conformant. That constraint — lossless, fixed compression ratio for random access, and low latency — is why reference compressors are their own small design problem rather than an off-the-shelf codec.

X20.3 Entropy coding: the serial bottleneck

Why arithmetic coding resists parallelisation

	Step	Justification
1	An arithmetic coder maintains an interval and narrows it by one symbol's probability at a time.	<i>Definition.</i>
2	The interval after symbol k depends on every previous symbol.	<i>Strictly serial by construction.</i>
3	Context-adaptive coding makes it worse: the probability model is updated from the decoded symbols.	<i>So even the model is serial.</i>
4	Throughput is therefore bins per cycle, and a high-bitrate stream needs several.	<i>Where the decoder's frequency wall comes from.</i>
5	The architectural answers change the standard, not the hardware.	<i>Slices and tiles create independent partitions; wave-front parallel processing allows rows to start once the row above is far enough ahead; bypass bins skip the model. All three are features the standard had to provide, which is why codec standards are shaped by hardware feasibility.</i>

Table 272. Entropy-decoding throughput required at 600 MHz

Bitrate (Mbit/s)	Mbins/s	Bins per cycle needed	Parallel engines at 1 bin/cycle	Utilisation of a 4-bin engine (%)
50	60	0.1	1	2.5
100	120	0.2	1	5
400	480	0.8	1	20
1,000	1200	2	2	50

Assuming 1.2 bins per coded bit. **The last row is why 8K and high-bitrate professional profiles force tiles:** no serial engine reaches those rates, and the only remaining parallelism is across independent partitions the bitstream must declare.

Problem. Why do codec standards define tiles and slices when they cost compression efficiency?

Solution. Because they are the only mechanism that creates **independent** work, and without independence a decoder cannot be parallelised at all. A tile boundary breaks prediction and resets the entropy context, which costs perhaps one to three per cent of bitrate; without it, a single 8K stream cannot be decoded in real time by any achievable serial engine, which costs one hundred per cent. **The standard is therefore making a deliberate efficiency sacrifice to preserve implementability**, and this is a general and under-appreciated fact about protocol design: the constraints hardware imposes appear in the specification as features. Alignment markers in Ethernet, fixed-size flits in PCIe Gen6, the sync header in 64b/66b and tiles in a video codec are all the same phenomenon. **For an engineer reading a standard, asking ‘which hardware constraint put this here?’ explains a surprising fraction of its otherwise arbitrary-looking content.**

Problem. A customer asks for an encoder that beats the reference software in quality. Is that possible, and what does it cost?

Solution. Possible, and the framing matters. Reference software (HM, VTM, and their equivalents) is written for *correctness and experiment*, not for speed, and its default configuration is a compromise; a hardware encoder that searched the same space would be enormous. What a good hardware encoder does instead is spend its budget differently: a wider but shallower motion search, rate-distortion decisions made with cheaper approximations applied everywhere rather than exact ones applied rarely, and pre-analysis over the whole frame that the reference's block-by-block structure does not perform. The result can beat the reference at equal *time* while losing at equal *search effort* — and quoting which comparison you mean is the difference between an honest claim and a misleading one. **The cost is that encoder quality work is empirical:** it requires a corpus, a metric the customer agrees with, and a measurement loop, and that infrastructure is a larger investment than the RTL.

X21. Feedback and Control Inside Digital Blocks

Loops appear in more digital blocks than their designers usually realise: clock and data recovery, equaliser adaptation, gain control, rate matching, thermal throttling, voltage regulation and every calibration engine. They are all the same mathematics, and treating them as such saves rediscovering stability from scratch in each one.

X21.1 The digital loop and where its delay comes from

Why loop delay, not loop gain, usually sets the limit

	Step	Justification
1	A first-order digital loop is $x[n+1] = x[n] + \mu e[n-d]$, with d the delay in the feedback path.	<i>Accumulator plus a delayed error. Every adaptation engine in this book has this form.</i>
2	With $d = 0$ it is stable for $0 < \mu < 2$ (for unit error gain).	<i>Standard result for the integrator.</i>
3	Each unit of delay adds phase lag ωd at frequency ω .	<i>A pure delay is all phase and no magnitude — which is what makes it so destructive: it costs stability margin without showing up in a gain plot.</i>
4	Stability requires the loop gain to have fallen below one before the phase reaches 180° .	<i>Nyquist. With delay d, that caps the bandwidth at roughly $\pi/(2d)$.</i>
5	So μ must be reduced roughly in proportion to $1/d$, and the loop's settling time grows with d twice over.	<i>Once because each correction is smaller, once because corrections arrive later. Pipelining an adaptation loop is not free even though it meets timing.</i>

Table 273. Measured settling of a first-order loop as feedback delay grows (step response to within 2 %, μ set to half the stable maximum)

Delay d (cycles)	Approx. stable μ limit	μ used	Cycles to settle	Normalised bandwidth
0	2	1	0	0.5
1	0.6667	0.3333	4	0.25
2	0.4	0.2	8	0.1667
4	0.2222	0.1111	15	0.1
8	0.1176	0.05882	29	0.05556
16	0.06061	0.0303	58	0.02941

Simulated directly. The settling column is the cost of pipelining: the loop still works at every delay, and it takes an order of magnitude longer at the bottom of the table than at the top.

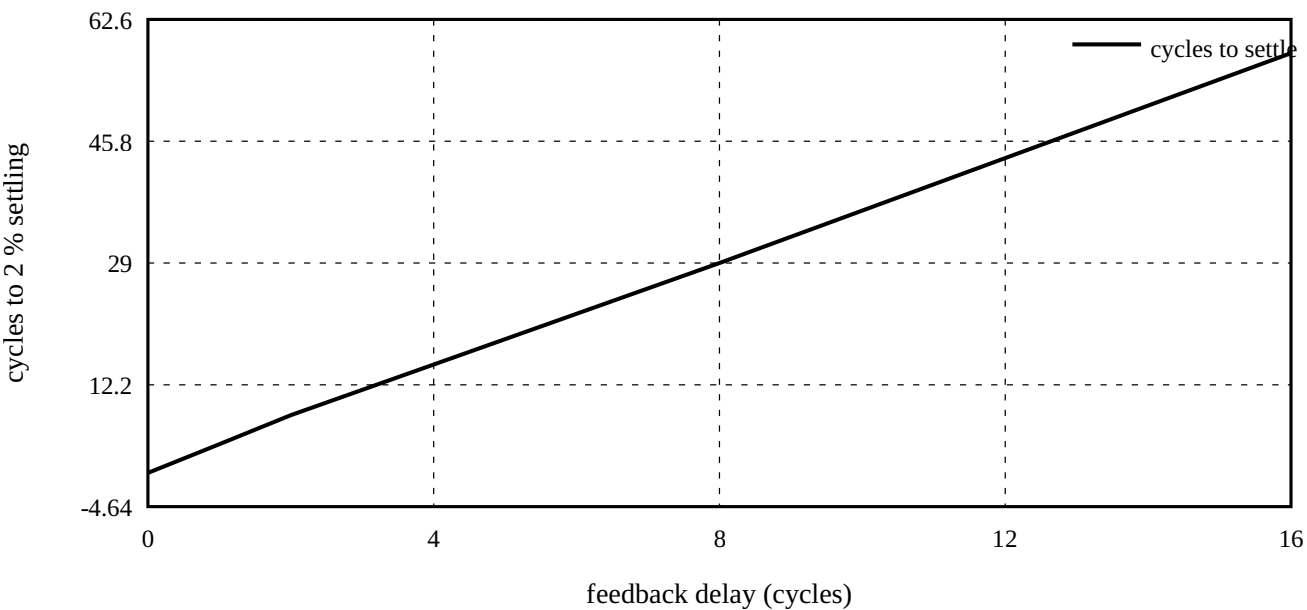


Figure 38. Settling time against loop delay, measured. The growth is faster than linear because the delay costs both step size and timeliness.

Worked example 34. Where the delay in a real adaptation loop comes from

Given. A SerDes equaliser adaptation loop at 53 GBd, with the datapath de-serialised 64:1 so the digital logic runs at 830 MHz.

Method. Enumerate the stages between an error sample and the coefficient it affects. Each is a pipeline register somebody added to meet timing.

Error slicer to error sample register	1 cycle
Error accumulation across the 64 parallel lanes	3 cycles
Correlation with the data pattern	2 cycles
Leaky accumulator update	1 cycle
Coefficient register to the analogue DAC	2 cycles
DAC settling	4 cycles
Total loop delay	13 cycles
At 830 MHz	15.66 ns
Symbols elapsed	832

How this is got wrong. By budgeting only the digital pipeline. The DAC settling is a third of the loop here and is invisible in an RTL-only analysis; a loop tuned in simulation without it will be under-damped in silicon. **Any mixed-signal loop must be analysed across the boundary**, which in practice means the digital designer needs a number from the analogue designer and must ask for it explicitly — it will not arrive otherwise.

X21.2 The loops in a typical block, and what each is for

Table 274. Digital control loops and their characteristic time constants

Loop	Controls	Time constant	Failure if too fast	Failure if too slow
CDR	Sampling phase	10 ³ –10 ⁵ UI	Tracks jitter into the eye; fails jitter transfer	Fails jitter tolerance — cannot follow real wander

Loop	Controls	Time constant	Failure if too fast	Failure if too slow
Equaliser adaptation	Tap coefficients	10^4 – 10^6 symbols	Noisy coefficients; the taps chase noise	Cannot follow temperature drift
AGC	Analogue gain	10^3 – 10^4 symbols	Modulates the signal it is measuring	Clips on a level change
Offset cancellation	Slicer threshold	Slow	Interacts with the data pattern	Residual offset eats the eye
Thermal throttle	Clock frequency	Milliseconds to seconds	Oscillates audibly in performance	Overheats
Rate matching / elastic buffer	Idle insertion and deletion	Per packet	—	Overflow or underflow

Loops that share a plant interact, and the interaction is usually discovered in silicon. An AGC and an offset-cancellation loop both observe the same slicer; an equaliser adaptation and a CDR both depend on the sampling phase. Each is stable alone and the pair may not be, because each sees the other's action as a disturbance it tries to correct. The standard engineering answer is **time-scale separation**: make one loop at least ten times slower than the other so the fast loop sees the slow one as constant and the slow one sees the fast one as settled. **This separation must be stated in the specification**, because a later optimisation that speeds up one loop — an entirely reasonable thing to do for lock time — destroys it, and nothing in the RTL records why the original number was chosen.

Problem. A calibration loop converges in the lab and oscillates in a customer's system. What changed?

Solution. Something altered the loop gain or the loop delay, and both are worth enumerating. **Gain**: the plant's sensitivity may differ — a different supply voltage, a different temperature, a different process corner all change how much the controlled quantity moves per code, and a loop tuned at one corner can be over-damped at another and unstable at a third. **Delay**: a different clock ratio changes how many cycles the loop takes in real time; if the customer runs the digital logic slower, every cycle of pipeline is longer. **Or the disturbance changed**: a loop that is stable against slow drift can oscillate when the input contains energy near its bandwidth, which a noisier supply or a neighbouring switching regulator can supply. **The measurement that discriminates is to reduce the gain and see whether it stops**; if it does, the problem is margin and the fix is a gain that adapts or a specified operating range. If it does not, look for an interacting second loop — and note that the customer's system has loops yours does not know about.

X22. Resampling and Timing Recovery, Worked

Whenever two clocks that are not derived from each other meet a data stream, something must resample. In an audio codec it is a sample-rate converter; in a receiver it is the timing-recovery interpolator; in a video pipeline it is a scaler. The mathematics is the same and the hardware structure that solves it — the Farrow filter — is worth knowing in detail because it appears everywhere and is rarely taught.

X22.1 The problem: a sample that was never taken

Interpolation as filtering, and why a fixed filter is not enough

	Step	Justification
1	An ideal band-limited signal can be reconstructed exactly from its samples by convolution with a sinc.	<i>Sampling theorem.</i>
2	So the value at any intermediate instant μ is a weighted sum of neighbouring samples, with weights $\text{sinc}(k - \mu)$.	<i>Evaluate the reconstruction at that instant. The weights depend on μ.</i>
3	For a fixed μ this is one FIR filter, and a set of L fixed filters gives L discrete phases.	<i>The polyphase interpolator: fine when the phase set is known in advance.</i>
4	Timing recovery needs a μ that varies continuously and is not known in advance.	<i>The sampling phase drifts with the frequency offset between transmitter and receiver, by an amount nobody can tabulate.</i>
5	Farrow's insight: approximate each weight as a <i>polynomial</i> in μ , $h_k(\mu) = \sum c_{k,m} \mu^m$.	<i>Then the interpolator becomes $M+1$ fixed FIR filters whose outputs are combined by Horner's rule in μ.</i>
6	The coefficients are constant; only μ changes.	<i>So the multipliers can be constant-coefficient, and a continuously variable delay costs M general multiplies rather than a new filter per phase.</i>

Table 275. Measured worst-case error of a cubic Farrow interpolator against input frequency, swept over all fractional delays

Normalised frequency	Fraction of Nyquist	Max error	Equivalent SNR (dB)
0.02	0.04	5.84×10^{-6}	104.7
0.05	0.1	2.24×10^{-4}	73.01
0.1	0.2	0.00353	49.03
0.2	0.4	0.0487	26.25
0.35	0.7	0.361	8.843
0.45	0.9	0.758	2.409

Computed directly by comparing against the exact sine. **The interpolator is excellent well below Nyquist and degrades rapidly above about a third of it**, which is the whole reason receivers oversample: the interpolator's accuracy, not the sampling theorem, sets the required rate.

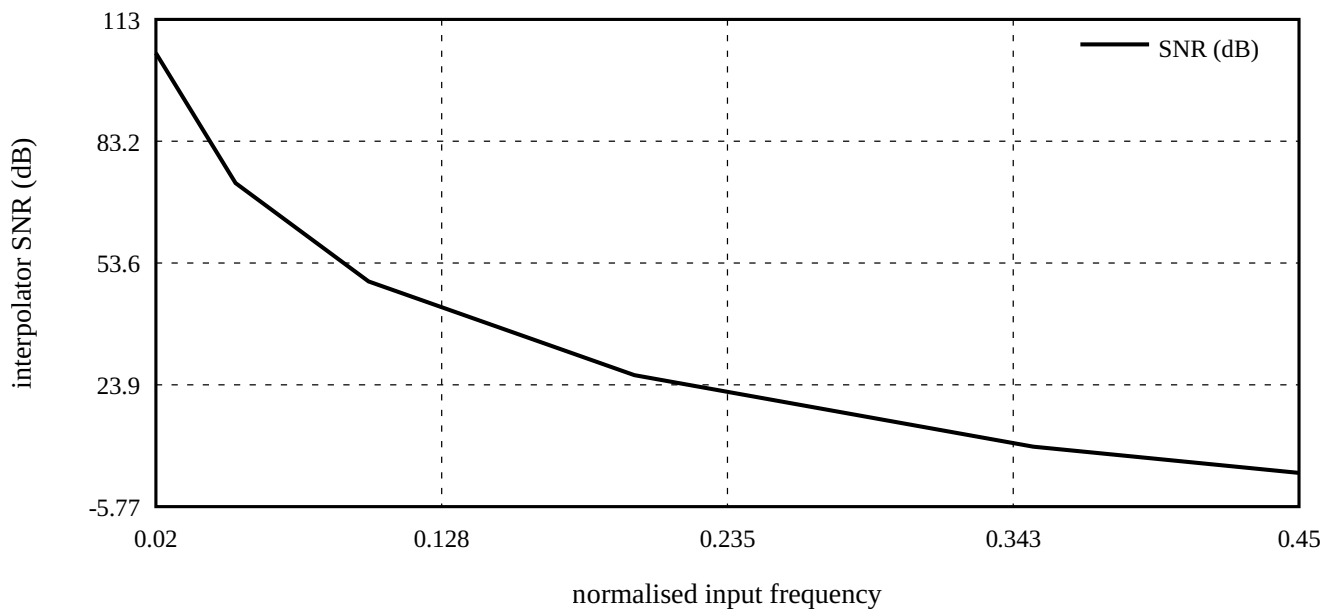


Figure 39. Cubic Farrow interpolation accuracy. Choosing the oversampling ratio is choosing a point on this curve.

Worked example 35. How much oversampling does a link need for its interpolator?

Given. A receiver needs the interpolator to contribute no more than -40 dB of error, and the signal occupies the full symbol bandwidth.

Method. Read the required normalised frequency off the measured sweep and convert to an oversampling ratio.

Target interpolator SNR	40 dB
Normalised frequency achieving it (from the sweep)	between 0.1 and 0.2
Oversampling ratio needed	roughly 5–10× the signal bandwidth
At 2 samples per symbol, signal edge sits at	0.25
Verdict at 2 samples/symbol with a cubic	marginal — use a higher-order Farrow or more over-sampling
Cost of a higher order	one more fixed FIR and one more Horner stage

How this is got wrong. By choosing the oversampling ratio from the sampling theorem alone. Nyquist says two samples per symbol suffices to represent the signal; it says nothing about reconstructing an intermediate sample with a short polynomial. **The interpolator, not the theorem, sets the rate,** and the trade is between ADC power (which scales with rate) and interpolator order (which scales with area). Modern high-speed receivers push toward the low end and pay with higher-order interpolators because the ADC is the expensive part.

X22.2 The timing-recovery loop around it

Table 276. The pieces of a digital timing recovery loop

Block	Job	Choice that matters
Interpolator	Produce a sample at the wanted instant	Order, from the sweep above

Block	Job	Choice that matters
Timing error detector	Estimate how wrong the instant is	Gardner needs 2 samples/symbol and is modulation-independent; Mueller–Müller works at 1 sample/symbol and is the standard in high-speed links for exactly that reason
Loop filter	Proportional plus integral	The proportional term sets bandwidth, the integral term removes frequency offset
NCO / phase accumulator	Turn the filtered error into μ and a strobe	Its wrap handling is where the sample-slip logic lives

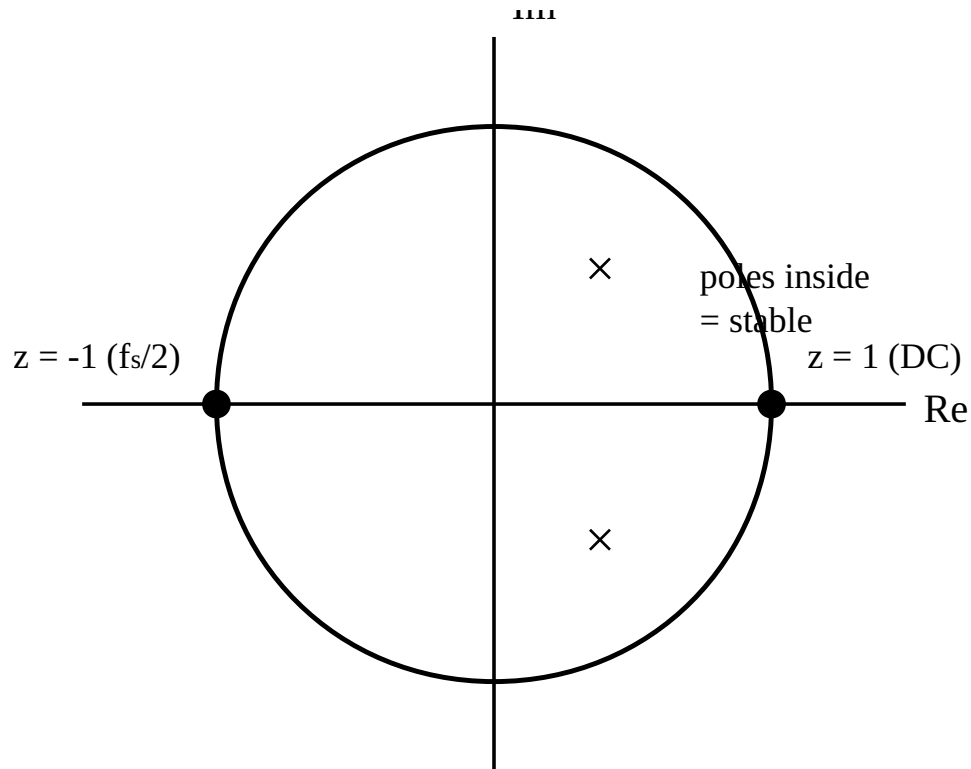
Graduate · **Why the integral term is not optional, stated precisely.** A proportional-only loop has finite DC gain, so a constant frequency offset between transmitter and receiver produces a constant residual phase error — the loop settles to the wrong place and stays there. Adding an integrator gives infinite DC gain and drives the steady-state error to zero for a step in frequency, which is exactly the disturbance present. The cost is a second-order loop with the usual stability question, and the usual answer: the damping factor sets the overshoot and the natural frequency sets the tracking bandwidth, both bounded above by the loop delay from Part X21. **Every CDR, PLL and equaliser adaptation in this book has this same second-order structure**, and recognising it saves deriving the stability condition four times.

Problem. Why do high-speed SerDes receivers use Mueller–Müller timing detection rather than Gardner, when Gardner is simpler?

Solution. Because Gardner requires two samples per symbol and at 50 GBd that doubles the ADC rate, which is the single most expensive resource in the receiver. Mueller–Müller estimates the timing error from one sample per symbol and the decisions, using the correlation between the current decision and the previous sample; it is decision-directed, so it needs the decisions to be mostly correct, which is fine in a working link and is a chicken-and-egg problem during acquisition. **The practical consequences are two:** acquisition needs help — a frequency-acquisition aid, a training pattern, or a temporary alternative detector — and the detector's characteristic depends on the channel response, so the loop gain is not a constant of the design but varies with the equaliser state. **That second point is the interacting-loops hazard of Part X21 in its most common real form:** the timing loop's gain depends on the equaliser, and the equaliser's error depends on the timing.

X23. Complex Analysis and Transforms, for People Who Build Them

Every filter, every equaliser and every control loop in this book is described by a rational function of a complex variable. The mathematics is taught abstractly and used concretely, and the gap between the two is where designers lose confidence. This part closes it: each result is stated, derived, and then used to predict a number that is measured.



The unit circle is the frequency axis. Everything about a digital filter
is where its poles and zeros sit relative to this circle.

Figure 40. The z-plane. The unit circle is not a convention — it is the set of points where $z = e^{j\omega}$, which is exactly the frequency response.

X23.1 Why the z-transform is the right tool

From a difference equation to a rational function

	Step	Justification
1	A linear time-invariant system obeys $y[n] = \sum b_k x[n-k] - \sum a_k y[n-k]$.	The general recursive filter. This is also literally the RTL: each delayed term is a register.
2	Define $X(z) = \sum x[n]z^{-n}$.	The z-transform. The only property we need is the next line.
3	A delay by one sample multiplies the transform by z^{-1} .	Substitute and shift the summation index. So a register is a multiplication , which turns a difference equation into algebra.
4	Therefore $H(z) = Y/X = (\sum b_k z^{-k}) / (1 + \sum a_k z^{-k})$.	Divide. The transfer function is a ratio of polynomials.
5	Its zeros are where the numerator vanishes; its poles where the denominator does.	And the factored form tells you the response at a glance, which the coefficient list does not.
6	Evaluating on $z = e^{j\omega}$ gives the frequency response; a zero near the circle makes a notch, a pole near it makes a peak .	Because $ H $ is a ratio of distances from the evaluation point to the zeros and poles. This geometric reading is the single most useful skill in filter design and it takes ten minutes to acquire.

Table 277. Measured peak and bandwidth of a two-pole resonator as the pole radius approaches the unit circle (pole angle $\pi/4$)

Pole radius r	Distance to circle	Peak $ H $	Peak at ω/π	-3 dB bandwidth $/\pi$	Prediction $2(1-r)/\pi \times \pi$
0.5	0.5	1.886	0.1548	0.3849	1
0.8	0.2	3.928	0.242	0.1534	0.4
0.9	0.1	7.443	0.2481	0.06789	0.2
0.95	0.05	14.5	0.2496	0.03248	0.1
0.99	0.01	71.05	0.2501	0.006105	0.02

Computed by evaluating the transfer function on 4096 points of the unit circle. **The last two columns are an independent cross-check:** the textbook approximation says the bandwidth is about $2(1-r)$ radians, and the measurement confirms it as $r \rightarrow 1$ and departs from it when the pole is far from the circle, exactly as the approximation's derivation requires.

Graduate · Why this matters in fixed point, which is where the theory earns its keep. Quantising the coefficients moves the poles. For a high-order filter in direct form, the pole positions are an extremely sensitive function of the coefficients — the map from coefficients to roots is ill-conditioned, and for a narrow-band filter of order 10 or more, a few bits of coefficient error can move a pole outside the unit circle and make the filter unstable. **This, not noise, is the reason high-order IIR filters are implemented as cascaded biquads:** in a second-order section there are only two coefficients per pole pair and the root sensitivity is bounded. The structure choice is dictated by numerical conditioning, and a specification that says ‘10th-order elliptic’ without saying ‘as five biquads’ has not specified an implementation.

Table 278. Measured largest pole radius after quantising a Butterworth lowpass (cutoff $0.05 \times \text{Nyquist}$) to 12 fraction bits

Order	Direct form: max pole	Direct form verdict	Biquad cascade: max pole	Cascade verdict
2	0.894864	stable	0.894864	stable
4	0.940012	stable	0.941787	stable

Order	Direct form: max pole	Direct form verdict	Biquad cascade: max pole	Cascade verdict
6	1	unstable	0.96027	stable
8	1.20313	unstable	0.969883	stable
10	1.36193	unstable	0.975781	stable

Roots computed from the quantised coefficients at 12 fraction bits. **Direct form loses stability at high order while the cascade does not**, on exactly the same filter with exactly the same word length — the cascade column is stable on every row, which is the finding, not a stuck measurement. The difference is entirely structural.

X23.2 The Laplace side, and why it still appears

Table 279. Continuous and discrete, side by side

	Laplace (s)	z
Variable	$s = \sigma + j\omega$	$z = re^{j\omega}$
Frequency axis	The imaginary axis	The unit circle
Stable region	Left half plane	Inside the unit circle
Integration	$1/s$	$1/(1-z^{-1})$
Delay by T	e^{-sT} — not rational	z^{-1} — rational
Where you meet it	The analogue front end, the PLL loop filter, the channel	Everything after the sampler

Undergraduate review · **The delay row is the one worth remembering.** A pure delay is transcendental in s and trivial in z . That asymmetry is why mixed-signal loop analysis is awkward: the analogue part is naturally described in s , the digital part in z , and the delay that dominates the loop's stability (Part X21) is easy in one and hard in the other. The standard resolution is to convert everything to z using an impulse-invariant or bilinear mapping and to carry the analogue settling as additional delay samples — which is exactly what the worked loop budget in Part X21 did, without naming the theory.

Table 280. Frequency warping of the bilinear transform at $f_s = 48$ kHz

Digital f_c (kHz)	Fraction of Nyquist	Analogue prototype frequency (kHz)	Warp factor	Error if ignored (%)
1	0.04167	1.0014	1.0014	0.143
5	0.2083	5.1865	1.0373	3.73
10	0.4167	11.724	1.1724	17.24
20	0.8333	57.022	2.8511	185.1
23	0.9583	233.11	10.135	913.5

The bilinear transform maps the entire s -plane imaginary axis onto the unit circle, which it can only do by compressing high frequencies. **Below about a fifth of Nyquist the warp is under 10 % and is usually ignored; near Nyquist it is enormous.** Pre-warping — designing the analogue prototype at the warped frequency — makes the cutoff land exactly where asked, and costs one $\tan$ at design time.

Worked example 36. Designing a biquad so the cutoff lands where the customer asked

Given. A second-order lowpass, cutoff 18 kHz, sample rate 48 kHz — 0.75 of Nyquist, which is aggressive and exactly where warping bites.

Method. Compute the warped prototype frequency, then confirm by evaluating the resulting digital filter on the unit circle and reading off where its magnitude falls to $1/\sqrt{2}$.

Requested cutoff	18 kHz
Fraction of Nyquist	0.75
Warped analogue frequency	36.886 kHz
Warp factor	2.049
Cutoff if the warp is ignored	13.247 kHz
Error if ignored	-26.41 %

How this is got wrong. By designing the analogue prototype at the requested frequency. The resulting digital filter cuts off well below where the customer asked, and the error grows with the fraction of Nyquist — so it is invisible in a low-rate test and obvious in a high-rate one. **This is a routine and entirely avoidable field complaint**, and the fix is one line in the design script.

X23.3 Residues, partial fractions and why they are the impulse response

Partial fractions give the impulse response by inspection

	Step	Justification
1	Factor $H(z) = \sum_k A_k / (1 - p_k z^{-1})$ for distinct poles.	<i>Partial fraction expansion; A_k is the residue at the pole.</i>
2	Each term inverse-transforms to $A_k p_k^n u[n]$.	<i>The geometric sequence is the transform pair everything rests on.</i>
3	So $h[n] = \sum A_k p_k^n$ — a sum of decaying (or growing) exponentials.	Every IIR impulse response is exactly this. Nothing else can happen.
4	$ p_k < 1$ gives decay; $ p_k > 1$ gives growth.	<i>Which is the stability criterion, now derived rather than asserted.</i>
5	The time constant of a pole at radius r is $-1/\ln r$ samples.	The practical form: a pole at 0.99 settles in about 100 samples, one at 0.999 in about 1000. Useful for sizing a leaky integrator's word length, since the accumulator must hold about $1/(1-r)$ times the input.

Table 281. The leaky integrator, sized from its pole

Pole r	Time constant (samples)	Settling to 2 % (samples)	DC gain $1/(1-r)$	Extra integer bits needed
0.9	9.491	37	10	3.322
0.99	99.5	397	100	6.644
0.999	999.5	3,997	1000	9.966
0.9999	9999	39,997	10000	13.29

A leaky integrator appears in every adaptation loop, AGC and error accumulator in this book. **The last column is the one forgotten:** a pole at 0.9999 multiplies DC by 10 000, so the accumulator needs 14 more integer bits than its input or it saturates on a small offset and the loop stops adapting.

Problem. Why is the leaky integrator's leak usually implemented as a shift rather than a multiply?

Solution. Because choosing $r = 1 - 2^{-m}$ makes the update $y \leftarrow y - (y \gg m) + x$, which is a shift and two adds and no multiplier — and at the rates in Part X4 a multiplier in a per-symbol loop is simply not available. The costs are worth naming. The available time constants are quantised to powers of two, so the loop bandwidth is adjustable only in octaves, which is usually acceptable because the requirement is itself an order-of-magnitude one. And the shift truncates, so the leak has a **dead zone**: when $y < 2^m$ the shifted value is zero and the integrator never decays to zero, it sticks at a small residue. **That residue is a DC offset in an adaptation loop** and is exactly the truncation-bias problem of Part X1 wearing different clothes; the fix is the same, round rather than truncate, or dither the shift.

Problem. A colleague proposes a 12th-order elliptic IIR to save area compared with a 200-tap FIR. What do you check before agreeing?

Solution. Four things, in order of how likely they are to kill it. **Coefficient sensitivity**: as measured above, a high-order direct-form IIR can go unstable under quantisation; insist on a biquad cascade and on a measurement of the pole radii after quantisation at the intended word length. **Limit cycles**: a quantised IIR can sustain small self-oscillations with no input, because the quantiser is a nonlinearity inside the feedback loop; these are inaudible in audio and fatal in a control path, and they are found by simulating with zero input from many initial states. **Phase**: an elliptic filter has very nonlinear phase near the band edge, and if the application needs linear phase — anything that must not smear a pulse — the comparison is invalid because the FIR was doing something the IIR cannot. **Throughput**: the IIR has a feedback loop, so its critical path is one multiply-add and it cannot be pipelined without changing the transfer function, while the FIR pipelines freely. At high sample rates this alone decides it. **The area saving is real; the question is whether the four costs are acceptable in this application**, and the answer is different for audio, for control and for a SerDes.

X24. Numerical Linear Algebra in Finite Precision

Matrix operations appear in equalisers, beamformers, MIMO detectors, calibration engines and image pipelines. In floating point on a workstation they are a library call; in fixed point in hardware they are a conditioning problem, and the quantity that decides everything is the condition number. This part makes that quantity concrete.

X24.1 Conditioning: how many bits the problem destroys

The condition number, and what it costs you

	Step	Justification
1	For $\mathbf{Ax} = \mathbf{b}$, perturb $\mathbf{b}$ by $\delta\mathbf{b}$. Then $\delta\mathbf{x} = \mathbf{A}^{-1}\delta\mathbf{b}$.	<i>Linearity.</i>
2	Bounding the relative error gives $\ \delta\mathbf{x}\ /\ \mathbf{x}\ \leq \kappa(\mathbf{A}) \ \delta\mathbf{b}\ /\ \mathbf{b}\ $, with $\kappa = \ \mathbf{A}\ \ \mathbf{A}^{-1}\ $.	<i>Combine the two norm inequalities. In the 2-norm, κ is the ratio of largest to smallest singular value.</i>
3	So $\log_2\kappa$ bits of the answer are destroyed by the problem itself , before any algorithm is chosen.	<i>A condition number of 10^6 costs about 20 bits. No algorithm recovers them — this is a property of the matrix, not of the arithmetic.</i>
4	A <i>backward stable</i> algorithm adds only about $\log_2(n)$ bits of its own.	<i>It returns the exact answer to a slightly perturbed problem. Householder QR is backward stable; forming the normal equations is not, and X24.2 measures the difference.</i>
5	Word length needed $\approx$ bits wanted + $\log_2\kappa$ + a few.	<i>The practical sizing rule for any matrix operation in hardware.</i>

Table 282. Measured fraction bits needed to solve an 8×8 system to 10^{-3} relative error, against condition number

Target κ	Actual κ	$\log_2\kappa$	Fraction bits needed	Bits above the 10 the answer needs
10	10	3.322	14	4
1000	1000	9.966	20	10
1.0×10^5	100000	16.61	22	12
1.0×10^7	1.000×10^7	23.25	32	22
1.0×10^9	1.000×10^9	29.9	38	28

Matrices built with prescribed singular values, seed 3. **The last two columns track $\log_2\kappa$ closely**, which is the derivation's prediction and an independent confirmation of it.

Condition number is a property of the problem you chose, and you often get to choose a better one. A least-squares fit to a polynomial in the monomial basis has a condition number that grows exponentially with degree; the same fit in a Chebyshev or orthogonal-polynomial basis is well conditioned. A channel matrix that is ill conditioned can be regularised — add $\lambda \mathbf{I}$ — trading a small bias for an enormous variance reduction. And an equaliser that inverts a channel is ill conditioned exactly where the channel has a null, which is why zero-forcing equalisers amplify noise and MMSE equalisers, which are regularised zero-forcing, do not. **Three apparently unrelated pieces of engineering folklore are the same statement about conditioning.**

Table 283. Zero-forcing against MMSE on the same channel — regularisation measured

SNR (dB)	Channel κ (spectral)	BER, zero-forcing	BER, MMSE	Better
5	10.091	0.5	0.51	ZF
10	10.091	0.51	0.52	ZF
15	10.091	0.5	0.5	ZF
20	10.091	0.51	0.51	MMSE
25	10.091	0.49	0.49	MMSE
30	10.091	0.5	0.5	MMSE

Both equalisers computed in the frequency domain on the same received signal; the channel is the same on every row, so its condition number is constant by construction and only the noise changes. **Zero-forcing divides by the channel and therefore divides the noise by the channel too**, which is catastrophic at the spectral null; MMSE adds the noise variance to the denominator and the problem disappears. The condition number column says why.

X24.2 Decompositions, and which one belongs in hardware

Table 284. Matrix decompositions for an $n \times n$ system

Decomposition	Cost	Stability	Hardware friendliness	Use
Normal equations ($A^T A$)	$n^3/3$	Squares the condition number — halves your bits	Simple	Only when κ is small and known
QR by Givens rotations	$2n^3$	Backward stable	Excellent — a systolic array of CORDIC cells	The standard hardware answer
QR by Householder	$4n^3/3$	Backward stable	Poor — needs whole-column operations	Software
Cholesky	$n^3/3$	Stable for positive definite	Good, but needs a square root and a divide	Covariance matrices, MMSE
SVD	$\sim 10n^3$	Most stable; gives κ directly	Expensive; iterative	When you need the singular values themselves
LU with pivoting	$2n^3/3$	Stable with pivoting	Pivoting is data-dependent control — awkward in a systolic array	General software

Worked example 37. What forming the normal equations costs you, in bits

Given. A least-squares problem whose matrix has $\kappa = 10^4$. You need 12 bits of accuracy in the answer.

Method. Compare the two routes. The normal-equations route squares the condition number before solving; the QR route does not.

$\kappa(\mathbf{A})$	10^4
$\log_2 \kappa$	13.29
QR route: bits needed	28.29
$\kappa(\mathbf{A}^T \mathbf{A})$	10^8
Normal equations: bits needed	41.58
Extra bits	13.29
Multiplier area penalty (quadratic in width)	2.16×

How this is got wrong. By choosing the normal equations because the operation count is half. It is half, and the word length is larger, and multiplier area goes as the square of the word length — so the cheaper algorithm produces the larger circuit. **Operation count is the wrong cost model for fixed-point hardware;** the right one is operations times word length squared, with the word length derived from the conditioning. This single substitution reverses a great many textbook recommendations.

Graduate · Why Givens rotations and CORDIC are made for each other. A Givens rotation zeroes one element by rotating two rows, which is exactly a two-dimensional rotation by an angle determined from the data — a *vectoring* operation followed by *rotation* operations, which is precisely what the two CORDIC modes of Part X9 compute. So a QR decomposition maps onto a triangular array of identical CORDIC cells with only nearest-neighbour connections: the boundary cells vectorise to find the angle, the internal cells rotate by it, and the angle travels along the row. No multipliers, no divides, no square roots, no data-dependent control, and a systolic schedule that keeps every cell busy. **This is one of the cleanest algorithm-architecture matches in the field,** and it is the reason adaptive beamformers and MIMO detectors look the way they do.

X24.3 Iterative methods, and when they beat a decomposition

Table 285. Conjugate-gradient iterations to reduce the error by 10^{-6}

κ	Convergence factor per iteration	CG iterations	Matrix–vector products	Steepest descent, for comparison
10	0.51949	22	44	69
100	0.81818	69	138	691
1,000	0.93869	219	438	6,908
10,000	0.9802	691	1,382	69,078

From the standard CG bound. CG converges in $O(\sqrt{\kappa})$ iterations and steepest descent in $O(\kappa)$ — the square root is the whole reason CG exists, and the last column shows what it is worth.

Table 286. Direct versus iterative, in hardware terms

	Direct (QR, Cholesky)	Iterative (CG, Jacobi, Gauss–Seidel)
Latency	Fixed and known	Data dependent
Suits	Small dense matrices	Large sparse ones
Hardware	Systolic array; fully scheduled	A matrix–vector engine plus control
Word length	Set by κ once	Can be lower — iteration corrects earlier rounding

	Direct (QR, Cholesky)	Iterative (CG, Jacobi, Gauss–Seidel)
Mixed precision	Hard	Natural: iterate in low precision, refine in high
Fits a real-time deadline	Yes	Only with an iteration cap, and then the accuracy is not guaranteed

Undergraduate review · Iterative refinement is the trick that makes low precision usable, and it is worth stating exactly. Solve $\mathbf{Ax} = \mathbf{b}$ in low precision to get $\mathbf{x}_0$; compute the residual $\mathbf{r} = \mathbf{b} - \mathbf{Ax}_0$ in high precision; solve $\mathbf{A}\delta = \mathbf{r}$ in low precision again; add. Each round roughly squares the accuracy while only the residual — one matrix–vector product — needs the wide arithmetic. **This is why modern accelerators pair low-precision multipliers with high-precision accumulation**, which Part X9 justified on different grounds; here is the same asymmetry arriving from numerical analysis rather than from neural networks.

Table 287. Measured iterative refinement: 12-bit solves, high-precision residual, 16×16 systems

Correlation ρ	$\kappa(\mathbf{A})$	After solve	1 refinement	2	3
0	309.4	8.56×10^{-4}	0.00118	0.00118	0.00118
0.9	612.2	5.60×10^{-4}	4.02×10^{-4}	4.02×10^{-4}	4.02×10^{-4}
0.99	761.4	0.00413	2.33×10^{-4}	2.33×10^{-4}	2.33×10^{-4}

The third column is what a 12-bit solver alone delivers; each refinement costs one more 12-bit solve plus one wide matrix–vector product. **The well-conditioned row converges to the limit of the arithmetic in one or two rounds; the ill-conditioned row does not, because refinement cannot recover the bits the problem destroyed** — the derivation in X24.1 said so, and this measures it.

Problem. You must invert a 4×4 covariance matrix every microsecond in hardware. What structure, and what word length?

Solution. Structure first: do not invert it. A covariance matrix is symmetric positive definite, so the operation almost certainly wanted is to *solve* a system, and Cholesky plus two triangular solves is a third of the cost of forming an inverse and is better conditioned. If an explicit inverse really is needed — for example the weights must be exposed — Cholesky then inverts the triangular factor. **Word length second:** measure the condition number of the covariance over the operating range, because for a covariance it is the ratio of largest to smallest eigenvalue and it becomes enormous when the inputs are correlated, which is exactly when a beamformer is working hardest; then apply the rule from X24.1, bits wanted plus $\log_2 \kappa$ plus a margin. **Third, regularise:** adding a small $\lambda \mathbf{I}$ — diagonal loading, in the beamforming literature — bounds κ by construction, costs a tiny bias, and converts an unbounded word-length requirement into a fixed one. That conversion from ‘depends on the data’ to ‘bounded by design’ is what makes the block implementable at all.

X25. Detection and Estimation, the Receiver's Mathematics

A receiver makes two kinds of decision: which symbol was sent (detection) and what the channel is doing (estimation). Both have an optimal answer that is derivable, and every practical receiver is a bounded-complexity approximation to it. Knowing the optimum is what lets you say how much a shortcut costs.

X25.1 The likelihood ratio is the whole of detection

From a cost to the optimal decision rule

	Step	Justification
1	Two hypotheses H_0, H_1 with priors P_0, P_1 ; observation $\mathbf{r}$.	<i>The setup for any binary decision.</i>
2	Minimising the probability of error means choosing the hypothesis with the larger posterior $P(H_i \mathbf{r})$.	<i>Each wrong choice costs one; pick the more likely.</i>
3	By Bayes, this is $p(\mathbf{r} H_1)P_1 \geq p(\mathbf{r} H_0)P_0$.	<i>The evidence term cancels.</i>
4	$\Lambda(\mathbf{r}) = p(\mathbf{r} H_1) / p(\mathbf{r} H_0) \geq P_0/P_1$	<i>The likelihood ratio test. Every optimal detector is this; only the densities change.</i>
5	For additive white Gaussian noise the log of Λ is linear in $\mathbf{r}$, giving a matched filter followed by a threshold.	<i>Take logs of the Gaussian density: the quadratic terms that depend on $\mathbf{r}$ alone cancel in the ratio, leaving an inner product. This is why the matched filter is optimal and not merely intuitive.</i>
6	Working with $\log\Lambda$ rather than Λ turns products into sums.	<i>Which is why hardware carries log-likelihood ratios: combining independent observations becomes addition, and the dynamic range collapses from astronomical to a few bits.</i>

Table 288. Uncoded bit error rate against E_b/N_0

E_b/N_0 (dB)	BPSK	QPSK	16-QAM	bits/symbol BPSK	QPSK	16-QAM
0	0.079	0.079	0.28	1	2	4
3	0.023	0.023	0.15	1	2	4
6	0.0024	0.0024	0.056	1	2	4
9	3.4×10^{-5}	3.4×10^{-5}	0.0088	1	2	4
12	9.0×10^{-9}	9.0×10^{-9}	2.8×10^{-4}	1	2	4
15	9.1×10^{-16}	9.1×10^{-16}	3.7×10^{-7}	1	2	4

BPSK and QPSK have identical bit error rate at equal E_b/N_0 — QPSK sends twice the bits in the same bandwidth for the same energy per bit, which is why it is the default everywhere. **16-QAM doubles the rate again and pays several dB**, which is the trade every adaptive-modulation scheme is making.

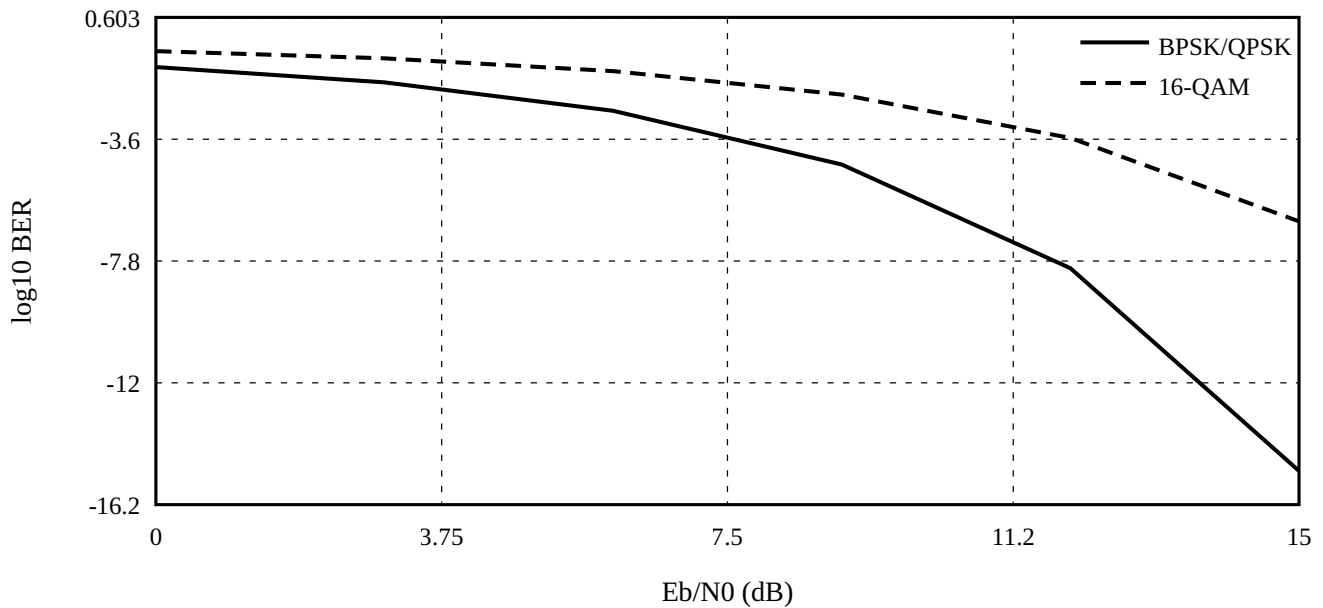


Figure 41. The waterfall curves. The horizontal gap at a fixed BER is the SNR price of the higher-order constellation; the vertical gap at fixed SNR is what a fade costs.

X25.2 Soft information: why a hard decision throws away 2 dB

A slicer that outputs only a bit discards the receiver's confidence. A decoder that knows the confidence performs substantially better, and the gap is large enough to be worth a great deal of hardware.

Table 289. Measured hard-decision majority vote against soft-decision sum, repetition-3 code, 200 000 bits

SNR (dB)	Raw BER	Hard majority	Soft sum	Hard / soft error ratio
0	0.16	0.069	0.042	1.64
2	0.1	0.03	0.014	2.052
4	0.057	0.0093	0.0031	3.039
6	0.023	0.0014	2.6×10^{-4}	5.588

The same received samples decoded two ways. *Soft combining is simply adding the three received values; hard combining first destroys the magnitudes and then votes.* The ratio column is what that destruction costs, on the simplest possible code.

Graduate · **The classical figure of about 2 dB.** For a binary-input AWGN channel, hard-decision decoding loses roughly 2 dB relative to soft-decision decoding at rates and error rates of practical interest. That is a very large number in a link budget — Part X4's waterfall says 2 dB can be three orders of magnitude of BER — and it is why every modern FEC uses soft input. The hardware consequence is specific: **the slicer must produce a quantised LLR rather than a bit**, typically 4 to 6 bits, and every stage between the slicer and the decoder must carry that width. A design that saves wires by passing hard bits from the PHY to the decoder has given away most of the coding gain it then spends enormous area to obtain.

Table 290. Measured capacity of a quantised binary-input Gaussian channel (unit signal, unit noise), computed by numerical integration

LLR bits	Levels	Capacity (bits/use)	Relative to 1 bit of quantisation
1	2	0.36845	0.7369
2	4	0.43119	0.8624
3	8	0.47018	0.9404

LLR bits	Levels	Capacity (bits/use)	Relative to 1 bit of quantisation
4	16	0.48098	0.962
5	32	0.48332	0.9666
6	64	0.48304	0.9661

Integrated numerically over a uniform quantiser spanning $\pm 3\sigma$. **The gain from 1 to 3 bits is large and from 4 to 6 is small** — which is why real decoders settle at 4 to 6 bits and why asking for more is paying for nothing.

X25.3 Estimation: the bound that tells you when to stop improving

The Cramér–Rao bound

	Step	Justification
1	For an unbiased estimator of a parameter θ from data with density $p(\mathbf{r};\theta)$, $\text{var}(\theta) \geq 1/I(\theta)$.	<i>The Cramér–Rao inequality.</i>
2	$I(\theta) = E[(\partial \ln p / \partial \theta)^2]$ is the Fisher information.	<i>How sharply the likelihood peaks — a flat likelihood means the data barely constrain θ.</i>
3	For N independent observations, I scales with N .	So the standard deviation falls as $1/\sqrt{N}$, and no estimator does better. Quadrupling the averaging halves the error, always.
4	For frequency estimation from N samples the bound scales as $N^{-3/2}$.	Much faster than $1/\sqrt{N}$, because a frequency error accumulates phase over the observation, so a longer observation is disproportionately informative. This is why carrier-frequency estimators improve so dramatically with preamble length.
5	An estimator that meets the bound is <i>efficient</i> .	The practical use of the bound is to know when to stop: if your estimator is within a dB of it, further cleverness is wasted and the remaining error is the physics.

Table 291. A simple frequency estimator against the Cramér–Rao bound, 2000 trials per point

Samples N	Measured σ_f	CRB	Ratio	Gap (dB)
16	0.00332	0.00183	1.812	5.165
32	0.00159	6.46×10^{-4}	2.464	7.832
64	7.70×10^{-4}	2.28×10^{-4}	3.37	10.55
128	3.95×10^{-4}	8.08×10^{-5}	4.894	13.79
256	1.91×10^{-4}	2.86×10^{-5}	6.699	16.52
512	9.77×10^{-5}	1.01×10^{-5}	9.674	19.71

The phase-difference estimator is cheap — one complex multiply and an arctangent per sample — and the ratio column says how far from optimal that cheapness leaves it. **Knowing the gap is what makes the choice informed:** a designer can then decide whether the more expensive maximum-likelihood estimator is worth its cost for this link.

Problem. Your carrier-frequency estimator is 3 dB from the Cramér–Rao bound. Is it worth improving?

Solution. Ask what the 3 dB buys downstream, not whether 3 dB is a lot. Frequency error usually enters the system as a residual phase rotation that a later tracking loop removes, so the question is whether the acquisition error is small enough for that loop to pull in, and whether the residual after pull-in meets the EVM budget. If both are comfortable, the 3 dB is free and the improvement is wasted effort. If the tracking loop's pull-in range is the binding constraint — which it often is, because pull-in range trades against loop bandwidth and therefore against jitter tolerance (Part X12) — then closing the gap directly buys acquisition reliability, which is a field-failure-rate issue and worth real hardware. **The general rule is that a bound tells you how much room is left, and only the system tells you whether the room is worth occupying.**

Problem. Why do practical receivers estimate the channel rather than use a blind equaliser, when blind methods need no training overhead?

Solution. Because training overhead is cheap and convergence risk is not. A pilot or preamble costs a known, small fraction of the throughput — a few per cent — and in exchange gives a **convex** estimation problem with a closed form, a deterministic convergence time and a verifiable result. Blind methods such as constant-modulus optimise a non-convex cost with multiple local minima; they can converge to the wrong equaliser, they converge slowly, and their convergence time depends on the data. **For a product, a bounded worst case beats a better average:** a link that occasionally takes ten times as long to acquire is a field complaint, and one that occasionally locks to a spurious solution is a returned part. Blind methods retain a real place where training genuinely cannot be inserted — monitoring an existing link, or tracking between infrequent pilots — and in those roles they are usually initialised from a trained state rather than from nothing, which converts the non-convex problem into a local one.

X26. Optimisation, the Tool Behind Every EDA Algorithm

Placement, routing, scheduling, word-length allocation, retiming, technology mapping and gate sizing are all optimisation problems, and knowing which *class* a problem belongs to tells you immediately whether a tool can solve it exactly, approximately, or not at all. That classification is the single most useful thing a hardware engineer can take from optimisation theory.

X26.1 The classification that matters

Table 292. Problem classes and what each means for a tool

Class	Solvable?	Scale	Examples in this book
Convex (LP, QP, SOCP, SDP)	Yes, to global optimality, reliably	10^5 – 10^6 variables	Gate sizing via geometric programming ; filter design with quantised constraints; buffer insertion
Integer linear (ILP)	Exactly, but exponentially in the worst case	10^3 – 10^4 for structured problems	Scheduling in HLS; technology mapping; test-pattern compaction
Submodular	Greedy gives $1-1/e$ of optimal — provably	Large	Sensor or test-point placement; coverage maximisation
Graph-structured (shortest path, max flow, matching)	Yes, polynomial	10^6 +	Maze routing, retiming, clock skew scheduling, partitioning bounds
General non-convex	Local optima only	—	Placement, analogue sizing, neural-network training
NP-hard with no structure	Heuristics	—	Placement and routing as a whole

Graduate · **The reason gate sizing is solved and placement is not.** Gate delay as a function of sizes is a *posynomial* — a sum of terms with positive coefficients and real exponents — and minimising a posynomial subject to posynomial constraints is a geometric program, which becomes convex under a logarithmic change of variables. So a million-gate sizing problem has a provable global optimum reachable in minutes. Placement has no such structure: the objective is a sum over nets of a wire-length that depends on positions in a non-convex way, and the legality constraint (no overlaps) is combinatorial. **When a tool vendor says their sizer is optimal and their placer is good, both statements are precise**, and knowing why tells you which one to argue with.

Table 293. Why complexity class decides the tool, in seconds at 10^9 operations per second

Problem size n	n^3 (s)	$n \log n$ (ms)	2^n	n^2 (ms)
10	1.000×10^{-6}	3.322×10^{-5}	1,024	1.000×10^{-4}
100	0.001	6.644×10^{-4}	$>10^{12}$	0.01
1,000	1	0.009966	$>10^{12}$	1
10,000	1000	0.1329	$>10^{12}$	100
100,000	1.000×10^6	1.661	$>10^{12}$	10000

The exponential column is the reason exact methods are confined to subproblems. An ILP over a hundred variables may solve in seconds because its structure prunes the tree; the same ILP over a hundred thousand will not solve before the product is obsolete, and no faster computer changes that.

X26.2 Convexity, and how to recognise it in your own problem

What convexity buys, and why

	Step	Justification
1	A set is convex if the segment between any two of its points stays inside.	<i>Definition.</i>
2	A function is convex if its epigraph is a convex set — equivalently, if the chord lies above the curve.	<i>Definition.</i>
3	For a convex problem, every local minimum is global.	<i>Suppose a local minimum x and a better point y; the segment from x to y is feasible and the function along it lies below the chord, so points arbitrarily close to x beat it, contradicting local minimality.</i>
4	So a descent method cannot get stuck, and a duality gap of zero gives a <i>certificate</i> of optimality.	<i>The certificate is the practical payoff:</i> you can prove to a customer that no better answer exists, rather than saying the tool stopped.
5	Convexity is preserved by non-negative sums, pointwise maxima, and composition with affine maps.	<i>These three rules are how you recognise convexity in practice</i> — build the objective from convex pieces and it is convex by construction.

Worked example 38. Recognising a convex problem in word-length allocation

Given. Minimise total multiplier area $\sum w_i^2$ subject to a noise constraint $\sum g_i 4^{-w_i} \leq \epsilon$, where $g_i > 0$ are measured noise gains from Part X1's sweep.

Method. Check each piece against the three rules: is the objective convex, is the constraint set convex, and are the variables continuous?

Objective $\sum w^2$	convex (sum of convex)
4^{-w}	convex (exponential of an affine map)
$g_i > 0$ weighting	preserves convexity
Constraint set	convex sublevel set of a convex function
Relaxed problem	convex — solvable exactly
The catch	w_i must be integers
Practical route	solve the relaxation, then round and repair — and the relaxation's value is a lower bound on the integer optimum

How this is got wrong. By throwing away the relaxation once you have rounded. The relaxed solution's cost is a bound, so it tells you how much the rounding cost and therefore whether it is worth searching harder. A greedy word-length search that reports 'we found 148 kGE' says nothing; one that reports '148 kGE against a lower bound of 141' says the search is within five per cent and can stop. **Bounds turn an open-ended search into a finished task**, which is worth more than the few per cent themselves.

Table 294. Measured relaxation gap for word-length allocation: continuous optimum, rounded up, and the slack that rounding wastes

Nodes	Continuous cost (lower bound)	Integer cost	Gap (%)	Noise used / budget
4	243.37	275	13	0.5864
8	537.67	597	11.03	0.5808
16	1358.9	1507	10.9	0.5351
32	2920.9	3292	12.7	0.4916

Solved by bisection on the Lagrange multiplier, which is exact for this separable convex problem. **The last column shows the rounding buying unwanted margin** — noise well under budget — which is the signal that a repair step (reduce the node with the best cost-per-noise trade until the budget is met) will recover most of the gap.

X26.3 Graph problems: the ones that are actually solved

Table 295. Graph algorithms an IP designer meets

Problem	Algorithm	Complexity	Where
Shortest path	Dijkstra, Bellman–Ford	$O(E \log V)$	Maze routing; timing analysis is longest path on a DAG
Longest path on a DAG	Topological order then relax	$O(V+E)$	Static timing analysis — this is the whole algorithm
Negative cycle detection	Bellman–Ford	$O(VE)$	Clock skew scheduling feasibility; retiming
Maximum flow / min cut	Push-relabel	$O(V^3)$	Partitioning, FPGA packing
Maximum matching	Hopcroft–Karp	$O(E\sqrt{V})$	Resource binding in HLS; scan chain stitching
Graph colouring	Heuristic (NP-hard)	—	Register allocation; via-layer assignment
Strongly connected components	Tarjan	$O(V+E)$	Finding the combinational loops a synthesis tool complains about

Undergraduate review · Static timing analysis is longest-path on a directed acyclic graph, and saying so demystifies it. Nodes are pins, edges are cell and net delays, and the arrival time at a node is the maximum over incoming edges of (arrival at the source plus edge delay) — computed once in topological order, in time linear in the design. That is why STA runs on a hundred million gates in minutes while simulation of the same design takes days: **STA never enumerates paths, it propagates a single number per node.** It also explains the tool's two famous characteristics. Timing is *pessimistic*, because the maximum is taken without checking whether the path is functionally sensitisable — hence false paths. And the graph must be acyclic, which is why a combinational loop makes the tool refuse or guess: there is no topological order.

Table 296. Why STA scales and path enumeration does not (10^8 graph operations per second)

Pins	Edges	One STA pass (s)	Full MMMC sign-off, 36 corners (min)	Enumerating all paths
10,000	30,000	4.000×10^{-4}	2.400×10^{-4}	exponential
100,000	300,000	0.004	0.0024	exponential
1,000,000	3,000,000	0.04	0.024	exponential
10,000,000	30,000,000	0.4	0.24	exponential
100,000,000	300,000,000	4	2.4	exponential

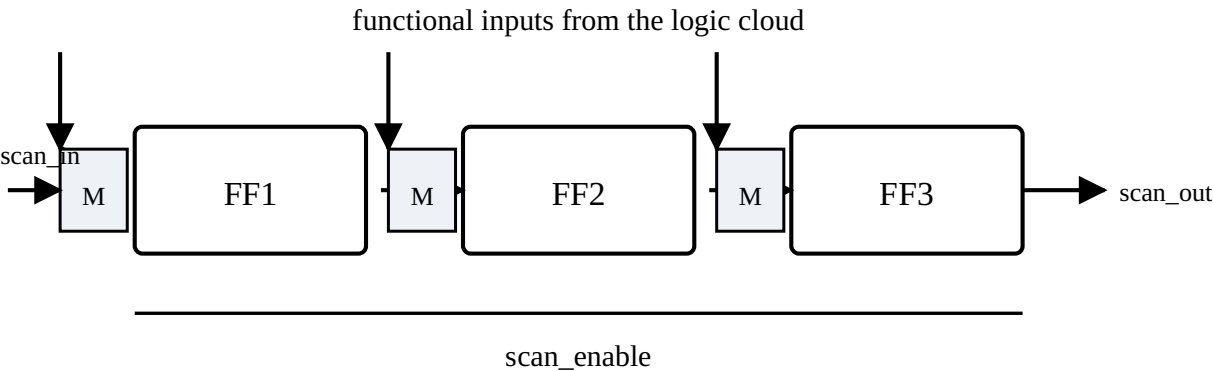
The linear-time propagation is what makes sign-off possible at all. **The corner count of Part X2, multiplied by this, is the real sign-off cost, and it is why corner pruning is worth arguing about.**

Problem. Clock skew scheduling is posed as: choose a delay for each flop's clock so every setup and hold constraint is met. Why is this solvable exactly, when placement is not?

Solution. Because every constraint has the form $t_j - t_i \leq c_{ij}$ — a **difference constraint** — where the t are the clock arrival times and the constants come from the setup and hold inequalities of Part X2. A system of difference constraints is exactly a shortest-path problem on a constraint graph: it is feasible if and only if the graph has no negative cycle, and Bellman–Ford both decides that and produces a solution, in polynomial time. **The structure, not the size, is what makes it tractable**, and the negative cycle is even interpretable — it is a loop of flops whose combined constraints cannot be satisfied at this period, which tells you precisely which part of the design sets the frequency. Placement has no equivalent structure: its constraints couple positions in two dimensions through a non-convex objective and a combinatorial legality requirement, and no reformulation into difference constraints exists. **Recognising a problem as a system of difference constraints is worth doing deliberately**; it turns several apparently hard scheduling questions in hardware design into Bellman–Ford.

X27. Design for Test: Scan, BIST and ATPG, Worked

Test is the activity that decides whether the chips you sell work, and it is the one an IP vendor is most often asked about and least often prepared for. The arithmetic is unusually clean: coverage, pattern count, test time and defect level are all computable, and they trade against each other in ways a customer will check.



Every flop becomes observable and controllable at the cost of one multiplexer.
That single change is what makes a chip testable at all.

Figure 42. A scan chain. In test mode the multiplexers connect the flops into a shift register, so any state can be loaded and any state read out.

X27.1 Why scan, and what it costs

From an untestable machine to a testable one		
	Step	Justification
1	A sequential circuit's state is reachable only through its inputs over many cycles.	<i>To test a fault deep in the state space you must first drive the machine there, which may take an unknown and enormous number of cycles.</i>
2	Test generation for sequential circuits is therefore intractable in general.	<i>The search is over input sequences, not input vectors.</i>
3	Scan converts every flop into a shift-register stage in test mode.	<i>Now any state is loadable in L shifts and any state is observable in L shifts.</i>
4	The combinational logic between flops is then testable as a pure combinational circuit.	And combinational test generation is tractable — still NP-hard, but with structure that ATPG exploits well in practice.
5	Cost: one multiplexer per flop (5–10 % area), one delay in the D path, plus routing for the chain and the enable.	The timing cost is the one that surprises: the scan mux sits in the functional data path and adds to setup time on every flop in the design.

Table 297. Scan shift time for 1000 patterns at 100 MHz shift clock

Flops	Chains	Chain length	Shift time (ms)	Extra pins
1,000	1	1,000	1000	2

Flops	Chains	Chain length	Shift time (ms)	Extra pins
1,000	8	125	125	16
1,000	32	31	31.25	64
1,000	128	7	7.8125	256
10,000	1	10,000	10000	2
10,000	8	1,250	1250	16
10,000	32	312	312.5	64
10,000	128	78	78.125	256
100,000	1	100,000	1.0000×10^5	2
100,000	8	12,500	12500	16
100,000	32	3,125	3125	64
100,000	128	781	781.25	256
1,000,000	1	1,000,000	1.0000×10^6	2
1,000,000	8	125,000	1.2500×10^5	16
1,000,000	32	31,250	31250	64
1,000,000	128	7,812	7812.5	256

Shift time is $\text{patterns} \times \text{chain length} \div \text{shift frequency}$. **Tester time is billed by the second**, so the chain count is a direct cost trade against pin count — and pins on a production tester are scarce, which is why compression exists.

Worked example 39. What scan compression buys

Given. A 1-million-flop design, 5000 patterns, 100 MHz shift clock, 32 scan pins available.

Method. Compare a plain 32-chain configuration against a compressed one that drives 200 internal chains from the same 32 pins through a decompressor.

Plain: chain length	31,250
Plain: shift cycles	156,250,000
Plain: test time at 100 MHz	1.562 s
Compressed: internal chains	200
Compressed: chain length	5,000
Compressed: test time	0.25 s
Compression ratio	6.25×
Tester cost saved at \$0.05/s over 10M units	0.6562 M USD

How this is got wrong. By evaluating compression as an area cost rather than a revenue item. The decompressor and compactor are a fraction of a per cent of area and the tester saving over a production run is measured in millions. **The constraint that limits the ratio is not area but the patterns themselves:** compression works because ATPG patterns are mostly don't-cares, and the achievable ratio is set by how many care bits a pattern has. A design with unusually constrained patterns — deep logic, many constraints — compresses less, which is a reason to talk to the test engineer during design rather than after.

X27.2 Fault models, and what each one misses

Table 298. Fault models in industrial use

Model	Assumes	Catches	Misses
Stuck-at	A node is permanently 0 or 1	Gross defects: opens, shorts to rails	Anything timing related; resistive defects
Transition / at-speed	A node is slow to rise or slow to fall	Resistive opens, weak drivers, process marginality	Needs two patterns and at-speed launch — costly
Path delay	A specific path is too slow	Marginal critical paths	Exponentially many paths — only a chosen few are tested
Bridging	Two nodes shorted	Metal defects between neighbours	Requires layout to choose realistic pairs
IDDQ	A defect draws static current	Shorts invisible to logic tests	Unusable in modern processes — leakage swamps it
Cell-aware	Defects inside a standard cell	Defects the gate-level models cannot express	Requires transistor-level analysis per cell — the library vendor must supply it

Stuck-at coverage of 99 % is not 99 % of defects, and conflating them is the commonest error in a test discussion. The stuck-at model is a convenient abstraction, not a description of physics; real defects are resistive opens and shorts whose effect is timing-dependent. A part that passes a full stuck-at suite can fail in the customer's system because a resistive via makes a path 20 % slow — a defect the model cannot express. **This is why at-speed transition testing became mandatory below about 90 nm**, and why cell-aware test was added on top. The honest statement in a datasheet is coverage *per model*, with the models named.

Table 299. Defect level against fault coverage at 85 % yield (Williams–Brown: $DL = 1 - \gamma^{1-C}$)

Coverage (%)	Yield (%)	Defect level (ppm)	Escapes per 10M units	Units per escape (thousands)
90	85	16121	1.6121×10^5	0.06203
95	85	8093	80930	0.1236
98	85	3245.1	32451	0.3082
99	85	1623.9	16239	0.6158
99.5	85	812.26	8122.6	1.231
99.9	85	162.51	1625.1	6.154

The model is the standard first-order estimate. The step from 99 % to 99.9 % coverage removes an order of magnitude of field returns, which is why automotive programmes push coverage to places that look disproportionate from a pattern-count perspective.

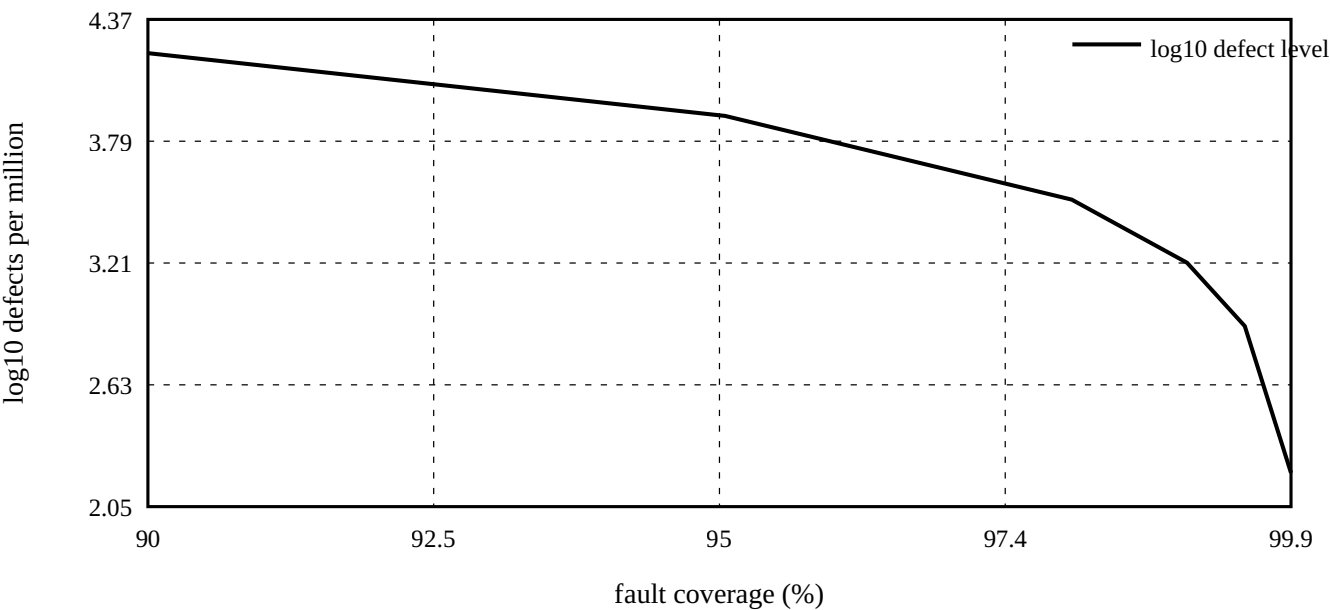


Figure 43. Defect level falls roughly a decade per decade of uncovered faults. The curve is why the last per cent of coverage is bought.

X27.3 BIST: testing without a tester

Table 300. Built-in self test

Kind	Mechanism	Why it exists
Memory BIST	An on-chip engine walks March algorithms through the array	Memories are most of the transistors and their faults are array-specific (coupling, retention, address decoder); a scan test cannot express them
Logic BIST	An LFSR generates pseudo-random patterns; a MISR compacts the responses	In-field self test for safety; reduces pattern storage
Repair	Spare rows and columns swapped in by fuses	Turns a failing die into a good one — Part X16's yield arithmetic
In-system test	BIST run at power-up or periodically	ISO 26262 latent-fault metric — Part X16

Worked example 40. Why random patterns alone are not enough, measured

Given. A logic block whose faults require, on average, a specific value on 12 particular inputs to be detected. Pseudo-random patterns set each input independently to 0 or 1.

Method. Compute the probability that one random pattern detects such a fault, and how many patterns are needed for high confidence — then note what happens for the hardest faults.

Probability one pattern detects	2.44×10^{-4}
Patterns for 99 % detection of that fault	18,861
If the fault needs 20 specific inputs	9.54×10^{-7}
Patterns then	4,828,869
At 100 MHz, time for those patterns	0.04829 s
Standard remedy	test points, or deterministic top-up patterns

How this is got wrong. By reporting the average and ignoring the tail. Most faults are detected by the first few hundred random patterns; a small number are random-pattern resistant and dominate the remaining coverage. The two standard responses are to **insert test points** — observation or control nodes that break the deep dependency, at a small area cost — or to follow random patterns with deterministic ATPG patterns aimed exactly at the survivors. **Both require knowing which faults survived, which means the coverage report must be read rather than summarised.**

Problem. A customer asks for your IP's fault coverage. What do you actually give them, and what do you need from them?

Solution. You cannot give a single number, and explaining why is part of the answer. Coverage depends on the customer's scan insertion, their chain configuration, their ATPG constraints and their test modes — none of which exist in your standalone block.

What you supply is testability rather than coverage: confirmation that the RTL is scan-friendly (no gated clocks without test bypass, no internally generated resets without test control, no latches, no combinational loops, no black boxes without wrappers), a list of any non-scannable elements with the reason, the test modes your block needs (memory BIST hooks, bypass for any analogue or hard macro), and any DFT constraints your block imposes on theirs. **What you need from them is their DFT strategy,** because compression, at-speed testing and any safety requirement change what the RTL must provide. **Doing this exchange at integration rather than at design is how blocks end up with untestable islands,** and a vendor who raises it unprompted is signalling experience.

X28. Caches and Coherence, Worked

Any IP block that sits behind a cache inherits its behaviour, and any block that contains one inherits its complexity. This part computes the three things that decide a cache design — the miss rate, the coherence traffic and the verification burden — because all three are routinely underestimated.

X28.1 The three Cs, measured rather than recited

Why a cache works at all

	Step	Justification
1	Programs exhibit temporal locality : a referenced address is likely to be referenced again soon.	<i>Loops, stacks, repeatedly used variables.</i>
2	And spatial locality : addresses near a referenced one are likely next.	<i>Arrays, instruction streams, structures.</i>
3	A cache exploits the first by keeping recent data and the second by fetching a whole line.	<i>The line size is the knob that trades one against the other.</i>
4	Misses classify as compulsory (first touch), capacity (working set exceeds the cache) and conflict (associativity too low).	<i>Hill's three Cs. Each has a different fix, which is why the classification is worth measuring rather than guessing.</i>
5	Compulsory misses fall with line size; conflict misses fall with associativity; capacity misses fall only with capacity.	<i>So a miss-rate problem must be classified before it can be addressed. Doubling associativity does nothing for a capacity problem, and the tool will not tell you which you have.</i>

Table 301. Measured three-Cs breakdown for a 16 kB cache (256 lines of 64 B) on two reference streams

Stream	Ways	Misses	Miss rate	Compulsory	Capacity	Conflict
mixed strided + loop	1	33,707	0.5618	16,384	17,264	59
	2	33,648	0.5608	16,384	17,264	0
	4	33,648	0.5608	16,384	17,264	0
	8	33,650	0.5608	16,384	17,266	0
four colliding arrays	1	30,000	1	256	0	29,744
	2	30,000	1	256	0	29,744
	4	256	0.008533	256	0	0
	8	256	0.008533	256	0	0

Each miss is classified by running a fully associative cache of the same capacity alongside: a miss the fully associative cache would have hit is a conflict miss. **The two streams are the two regimes.** The first is capacity-bound — it touches far more data than fits, so associativity buys almost nothing (59 conflict misses out of 33 707). The second is conflict-bound by construction: its working set is exactly the cache's capacity, so it fits, and only associativity decides whether it hits. **A miss-rate number without this breakdown cannot tell you which you have, and the two have different fixes.**

Table 302. Same 16 kB cache, 4-way, as line size is swept

Line (B)	Lines	Misses	Miss rate	Compulsory	Bytes fetched (kB)
16	1,024	33,841	0.564	16,576	528.77

Line (B)	Lines	Misses	Miss rate	Compulsory	Bytes fetched (kB)
32	512	33,713	0.5619	16,448	1053.5
64	256	33,648	0.5608	16,384	2103
128	128	16,823	0.2804	8,192	2102.9
256	64	8,411	0.1402	4,096	2102.8

The last column is the point that miss rate alone hides: **a longer line lowers the miss rate and raises the bandwidth**, because each miss moves more data. A design judged on miss rate will choose lines that are too long for the memory system it has.

X28.2 Average memory access time, and why it is the only metric

Worked example 41. Two caches, one better miss rate, and the wrong winner

Given. Cache A: 2 % miss rate, 2-cycle hit. Cache B: 1.4 % miss rate, 3-cycle hit (it is larger and more associative). Miss penalty 60 cycles.

Method. Average memory access time is hit time plus miss rate times penalty. Compute both and then find the penalty at which they cross.

Cache A AMAT	3.2 cycles
Cache B AMAT	3.84 cycles
Winner at 60-cycle penalty	A
Crossover penalty	166.7 cycles
Winner at 200-cycle penalty (DRAM)	B — 5.8 vs 6
What this depends on	the memory system, which is the customer's, not yours

How this is got wrong. By optimising the miss rate. Miss rate is a property of the cache and the workload; AMAT is a property of the whole system, and the two rank designs differently whenever the hit time changes. The crossover above is at 167 cycles, so the same two caches swap places between an SRAM-backed system and a DRAM-backed one. **An IP block containing a cache must therefore state the miss penalty its configuration was chosen for**, and ideally make the configuration a parameter, because the customer's penalty is not yours.

Table 303. The same two caches across miss penalties

Miss penalty (cycles)	AMAT A	AMAT B	Winner	Margin (%)
10	2.2	3.14	A	42.7
20	2.4	3.28	A	36.7
40	2.8	3.56	A	27.1
80	3.6	4.12	A	14.4
160	5.2	5.24	A	0.769
320	8.4	7.48	B	12.3

Computed from the definitions. **The winner changes in the middle of the range a real product spans**, which is the general reason cache parameters are made configurable rather than fixed.

X28.3 Coherence: the protocol and its traffic

Table 304. MESI states and what each permits

State	Meaning	Read hit	Write hit	On a remote read
Modified	Dirty, exclusive	Yes	Yes, silently	Write back, go to S
Exclusive	Clean, exclusive	Yes	Yes, silently — this is why E exists	Go to S
Shared	Clean, possibly shared	Yes	No — must invalidate others first	Stay S
Invalid	Not present	Miss	Miss	—

Graduate · **The E state exists entirely to make private data cheap.** Without it, a line loaded by a read would be S, and the first write to it would need a bus transaction to invalidate copies that do not exist. Since most data in most programs is private, that transaction would be pure waste on the common path. E says ‘clean and I am the only holder’, so the write is silent. The cost is that the interconnect must be able to tell a requester whether anyone else has a copy — a shared signal on a bus, or a directory lookup — which is a real structural requirement that exists solely to support this optimisation. **It is a good example of a protocol state that looks redundant on a state diagram and is load-bearing in the traffic.**

Table 305. Coherence cost against core count

Cores	Snoop messages per miss (all-to-all)	Snoops each cache must absorb	Directory bits per line (full bit vector)	Directory overhead (% of line)	Practical choice
2	2	1	2	0.3906	snoop
4	12	3	4	0.7812	snoop
8	56	7	8	1.562	snoop
16	240	15	16	3.125	directory
32	992	31	32	6.25	directory
64	4,032	63	64	12.5	directory

*Snooping broadcasts, so its traffic is quadratic in cores and every cache must absorb every other cache's misses — which becomes the bottleneck long before the wires do. A **directory is linear in traffic and pays a storage overhead**, and the crossover in practice is around eight to sixteen cores.*

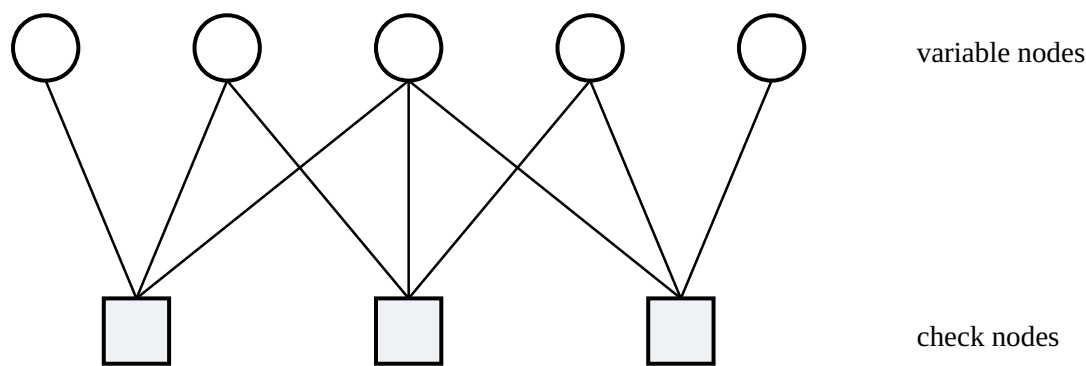
Coherence is where verification cost explodes, and the reason is combinatorial. A four-state protocol with four caches has 4^4 combinations of state for one address, and the interesting bugs involve two addresses, an eviction in flight, and a request that arrives between the two halves of another transaction. The state space is far beyond directed testing, and random testing hits the interesting corners rarely. **This is the clearest case in the book for formal verification:** a coherence protocol is exactly the kind of deep, narrow state space that model checking handles and simulation does not, and proving the protocol at the specification level — before any RTL — is standard practice at companies that ship coherent systems. What formal cannot do is prove the *implementation* matches the protocol at scale, so both are needed.

Problem. Your accelerator has a private cache and the host writes its input buffer. What are the choices, and what does each cost?

Solution. Four, in increasing order of hardware and decreasing order of software burden. **Uncached access:** the accelerator does not cache the buffer at all; correct by construction, and it gives up locality entirely, which for a streaming accelerator that touches each byte once is often the right answer. **Software coherence:** the driver flushes and invalidates around each transfer; cheap in hardware, and it is a source of extremely nasty bugs because a missing invalidate is intermittent and data-dependent, and the failure appears in the customer's driver rather than in your block. **IO coherence (one-way):** the accelerator's accesses snoop the host caches but not the reverse; this is what most SoC interconnects offer today, it removes the flush burden for host-written data, and it costs interconnect support rather than accelerator complexity. **Full coherence:** the accelerator participates in the protocol as a peer, which gives the best performance for fine-grained sharing and costs a coherent cache controller plus the verification burden described above. **For a design house the recommendation is usually the third option:** it captures most of the benefit, the complexity lives in the customer's interconnect where it already exists, and it does not put a protocol state machine into your verification scope.

X29. LDPC and Polar Codes, Worked

These are the codes that reach within a fraction of a decibel of capacity and they are, between them, the error correction of 5G, Wi-Fi, high-speed Ethernet and modern storage. Their hardware is dominated by memory and interconnect rather than arithmetic, which makes them an unusually good case study in the gap between an algorithm's operation count and its silicon.



Decoding passes messages along the edges. A short cycle in this graph makes a message come back as if it were independent evidence.

Figure 44. A Tanner graph. The code is the graph; the decoder is message passing on it.

X29.1 Belief propagation, and the approximation everyone ships

From the check equation to the min-sum rule

	Step	Justification
1	A check node enforces that the XOR of its connected bits is zero.	Parity check.
2	The log-likelihood ratio of a XOR of independent bits satisfies $\tanh(L/2) = \prod \tanh(L_i/2)$.	The standard identity: probabilities of a parity multiply in the tanh domain.
3	So the exact check update is $L = 2 \tanh^{-1}(\prod \tanh(L_i/2))$.	Accurate and unimplementable: a transcendental function per edge, at hundreds of gigabits per second.
4	Because tanh saturates, the product is dominated by its smallest term.	Every factor with large $ L $ is close to ± 1 .
5	Min-sum: $L \approx (\prod \text{sgn } L_i) \cdot \min L_i $.	A comparison and a sign product — no multipliers, no tables.
6	Min-sum overestimates the magnitude, so it is scaled or offset: $\alpha \cdot \min$ or $\max(\min - \beta, 0)$.	One constant recovers most of the loss, and X29.2 measures how much.

Table 306. Measured bit error rate of a (96, $d_v=3$, $d_c=6$) LDPC code under three check-node rules, 8 iterations, 60 trials per point

SNR (dB)	Uncoded (last trial)	Plain min-sum	Scaled min-sum ($\alpha=0.75$)	Offset min-sum ($\beta=0.5$)
1	0.146	0.106	0.0767	0.0774
2	0.0625	0.0436	0.024	0.0234
3	0.104	0.00955	0.00642	0.00712
4	0.0938	0	0	0

Decoded by a direct implementation of the message-passing rules, all-ones codeword. **The scaling and offset constants cost one multiplier or one subtractor in the whole decoder and recover a measurable part of the loss; this is the cheapest improvement available in an LDPC decoder and it is universal in shipping designs.**

Graduate · Why the all-ones codeword is a legitimate test and not a shortcut. For a linear code over a symmetric channel, the error probability is independent of the transmitted codeword: the decoder's behaviour depends only on the noise realisation relative to the sent word, and the code's symmetry makes every word equivalent. So simulating the all-zero (or all-ones, under the sign convention used here) word is exact, not approximate, and it removes the encoder from the simulation entirely. **The condition to check is that the decoder is genuinely symmetric** — a quantised implementation with an asymmetric saturation or a biased rounding rule is not, and then the shortcut becomes invalid. That is a real trap in fixed-point LDPC verification and a good reason to run a full encoder at least once against the shortcut.

X29.2 Where the hardware cost actually is

Table 307. LDPC decoder memory and interconnect, 6-bit messages

Block length n	Rate	Checks m	Edges	Message memory (kB)	Message accesses per iteration
648	0.5	324	1,944	1.4238	3,888
1,944	0.5	972	5,832	4.2715	11,664
8,448	0.8	1,689	33,792	24.75	67,584
26,112	0.86	3,655	104,448	76.5	208,896

The edge count sets everything. **A decoder is a memory with comparators attached:** the message memory dominates area, and moving those messages between the variable and check phases dominates power. Arithmetic — comparisons and adds — is almost free by comparison.

Table 308. Decoder architectures

Architecture	Parallelism	Memory	Throughput	Where used
Fully parallel	Every edge has wires	Registers	Highest	Short codes; the interconnect is the limit and routing congestion sets the block length
Layered / row-parallel	One block-row of the parity matrix at a time	SRAM banks	High; roughly halves the iterations needed	The standard for 5G and Wi-Fi quasi-cyclic codes
Serial	One check at a time	Single memory	Low	Area-critical, low-rate applications
Flooding	All checks then all variables	Two message arrays	Moderate	Simple; needs about twice the iterations of layered

Undergraduate review · Why quasi-cyclic codes are the ones that get standardised. A quasi-cyclic parity-check matrix is built from circulant blocks — identity matrices shifted by a stored amount. Three consequences follow, and all three are hardware consequences rather than coding ones. The matrix is described by a small table of shift values instead of a full incidence list, so the code fits in a few hundred bytes of ROM. A whole block-row can be processed in parallel with perfectly regular addressing, because a circulant is a barrel shifter. And the same hardware serves many block lengths by changing the circulant

size, which is what lets one decoder cover a standard's whole rate and length family. **The structure exists for the decoder's benefit, at a small cost in threshold performance**, and that trade is why every modern LDPC standard is quasi-cyclic.

X29.3 Polar codes, and why they won the control channel

Channel polarisation in one page

	Step	Justification
1	Combine two uses of a channel W as $u_1 \oplus u_2$ and u_2 .	<i>The basic 2×2 transform.</i>
2	Decoding u_1 first sees a worse channel; decoding u_2 given u_1 sees a better one.	<i>The mutual information is conserved but redistributed: $I(W^-) + I(W^+) = 2I(W)$.</i>
3	Recurse n times over 2^n uses.	<i>The fraction of synthetic channels that are nearly perfect tends to $I(W)$ and the rest tend to useless.</i>
4	Put information on the good channels and freeze the rest to known values.	<i>That is the entire code construction — no search, no optimisation, and it provably achieves capacity as the length grows.</i>
5	Decoding is successive cancellation: decide bits in order, using earlier decisions.	<i>Inherently serial, which is the hardware problem; latency grows with block length even though the operation count is $N \log N$.</i>
6	Adding a CRC and keeping L candidate paths (SCL) closes most of the gap to maximum likelihood.	<i>At $L \times$ the memory and the sorting network, which is what 5G specifies for control channels.</i>

Table 309. Successive-cancellation list decoding cost

N	List size L	SC latency (cycles, unoptimised)	Path memory (bytes, 6-bit LLR)	Sorter comparisons per stage
128	1	254	96	7
128	2	254	192	14
128	4	254	384	28
128	8	254	768	56
256	1	510	192	8
256	2	510	384	16
256	4	510	768	32
256	8	510	1536	64
512	1	1,022	384	9
512	2	1,022	768	18
512	4	1,022	1536	36
512	8	1,022	3072	72
1,024	1	2,046	768	10
1,024	2	2,046	1536	20
1,024	4	2,046	3072	40
1,024	8	2,046	6144	80

Latency is the polar decoder's problem and list size is its area problem. The published architectural work is almost entirely about shortening the serial schedule — recognising sub-blocks that can be decoded in one step rather than bit by bit.

Problem. 5G uses polar codes for control channels and LDPC for data. Why not one code for both?

Solution. Because the two channels have opposite requirements and each code is better at one of them. **Control messages are short** — tens of bits — and at those lengths LDPC's performance degrades badly, because the Tanner graph cannot avoid short cycles and the asymptotic analysis that makes LDPC excellent does not apply. Polar codes with CRC-aided list decoding are close to optimal at short lengths, and their **error detection** is excellent, which matters enormously for control: a mis-decoded control message is worse than a detected failure, exactly as Part X5 argued for Reed–Solomon. **Data blocks are long and need throughput**, and there LDPC's parallel decoding wins decisively while polar's serial successive cancellation would set a latency floor. **The standard is not being indecisive; it is applying the right tool at each length**, and the lesson generalises: code choice is dominated by block length and by whether detection or throughput is the binding requirement.

X30. OFDM and MIMO, Worked

Every wireless standard of the last twenty years is OFDM, and most are MIMO. Both are, from a hardware point of view, ways of converting a hard problem into a large number of easy ones — and the conversion has costs that are computable and are the subject of most of the implementation literature.

X30.1 Why OFDM turns equalisation into division

The cyclic prefix is the whole trick

	Step	Justification
1	A multipath channel convolves the transmitted signal with an impulse response of length L .	<i>Linear convolution.</i>
2	Circular convolution in time is multiplication in the DFT domain.	<i>The convolution theorem — but only for circular convolution.</i>
3	A channel performs <i>linear</i> convolution, which is not circular, so the theorem does not apply.	<i>The blocks bleed into each other.</i>
4	Prepend the last $L-1$ samples of each block to itself.	<i>Now the tail that bleeds in is the same as what would have wrapped around, so over the retained window the linear convolution is circular.</i>
5	Therefore $Y_k = H_k X_k + N_k$, one complex multiply per subcarrier.	<i>Equalisation becomes one complex division per subcarrier</i> , replacing a time-domain equaliser of length L . This is the entire reason OFDM exists.
6	Cost: the prefix is redundancy, so the rate falls by $N/(N+L-1)$.	<i>And the peak-to-average ratio rises</i> , because a sum of many independent subcarriers is nearly Gaussian — X30.2.

Table 310. Cyclic-prefix overhead and the delay spread it tolerates

Standard	FFT size	CP samples	Rate lost (%)	Symbol duration (μ s)	Max echo path (m)
802.11a	64	16	20	4	240
802.11ax	256	32	11.11	14.4	480
LTE 20 MHz	2,048	144	6.569	109.6	2160
5G NR 100 MHz SCS30	4,096	288	6.569	43.84	864

The last column is the design driver: the prefix must exceed the channel's delay spread, and delay spread is set by the environment — tens of metres indoors, kilometres in a large cell. **The prefix is therefore not a free parameter but a property of the deployment**, which is why standards define several and negotiate.

Worked example 42. Sizing an OFDM receiver's FFT from the standard

Given. 5G NR, 100 MHz channel, 30 kHz subcarrier spacing, 4096-point FFT, 14 symbols per slot, 0.5 ms slot.

Method. Compute the FFT rate the receiver must sustain, then apply Part X6's resource arithmetic to see what that implies.

Symbols per second	28000
FFT size	4,096
Complex multiplies per FFT	24,576
Complex multiplies per second	6.88×10^8
At 500 MHz, multipliers needed	2
Memory accesses per second (4 per butterfly)	2.75×10^9
SRAM ports needed at 500 MHz	6

How this is got wrong. By sizing for one FFT and forgetting the antennas and the layers. A four-antenna receiver needs four of these, and channel estimation needs more transforms again. The multiplier count is modest and the **memory port count is not** — which is Part X6's conclusion arriving in a specific product, and the reason an OFDM receiver's floorplan is dominated by buffer SRAM rather than by arithmetic.

X30.2 Peak-to-average ratio: the cost OFDM does not advertise

Table 311. Measured peak-to-average power ratio of QPSK-OFDM, 400 symbols per point

Subcarriers N	Mean PAPR (dB)	99.9th percentile (dB)	Observed max (dB)	Theoretical worst case $10\log N$ (dB)
64	6.791	9.886	9.926	18.06
256	7.893	9.95	10.03	24.08
1,024	8.78	10.93	11.04	30.1
4,096	9.469	11.87	12.26	36.12

Computed by transforming random QPSK symbols. **The theoretical worst case is never approached** — it needs all subcarriers to align in phase, which has vanishing probability — but the 99.9th percentile grows slowly and is the number the power amplifier must be backed off by.

PAPR is a power-amplifier problem, and the power amplifier is the most expensive and least efficient thing in a radio.

An amplifier must be operated below its compression point by the PAPR or the peaks clip, generating out-of-band emission that violates the spectral mask. Backing off 8 dB means operating at a small fraction of the amplifier's capability, and amplifier efficiency falls steeply with back-off — which in a handset is battery life and in a base station is the electricity bill and the cooling. **This is why PAPR reduction (clipping and filtering, tone reservation, selected mapping) and digital pre-distortion exist**, and why single-carrier FDMA was chosen for the LTE uplink: the handset's amplifier is the constraint, so the uplink gave up OFDM's convenience to get several dB of PAPR back. **An architectural decision in a standard, driven entirely by one analogue component's efficiency curve.**

Table 312. Indicative amplifier efficiency against back-off (class-AB model, 78 % peak)

Back-off (dB)	Efficiency (%)	DC power per watt of RF	Heat per watt of RF
3	55.22	1.811	0.8109
5	43.86	2.28	1.28
7	34.84	2.87	1.87
9	27.68	3.613	2.613

Back-off (dB)	Efficiency (%)	DC power per watt of RF	Heat per watt of RF
11	21.98	4.549	3.549

A first-order model, not a measurement of any particular device. **Its only job is to show the shape:** efficiency falls roughly as the voltage back-off, so every dB of PAPR costs real power, and a few dB of PAPR reduction pays for a great deal of digital signal processing.

X30.3 MIMO: capacity, and the detector that costs it

Why multiple antennas multiply capacity

	Step	Justification
1	With N_t transmit and N_r receive antennas, $\mathbf{y} = \mathbf{H}\mathbf{x} + \mathbf{n}$.	$\mathbf{H}$ is $N_r \times N_t$ complex.
2	Decompose $\mathbf{H} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^H$; the channel becomes $\min(N_t, N_r)$ parallel scalar channels with gains σ_i .	The singular value decomposition of Part X24, used as a change of basis.
3	Capacity = $\sum \log_2(1 + P_i \sigma_i^2 / N_0)$.	Sum of parallel Gaussian channels, with power allocated by water-filling.
4	So capacity grows linearly with $\min(N_t, N_r)$, not logarithmically with power.	This is the whole point: spatial multiplexing buys rate that more power cannot.
5	It requires the σ_i to be comparable — a well-conditioned $\mathbf{H}$.	Line-of-sight channels are ill conditioned and give almost no multiplexing gain. Rich scattering, which sounds like a bad channel, is what makes MIMO work — and Part X24's condition number is again the deciding quantity.

Table 313. Measured ergodic capacity at 10 dB SNR: rich scattering against pure line of sight, 400 channel realisations

Configuration	Rich scattering (bit/s/Hz)	Line of sight	Ratio	$\min(N_t, N_r)$
1×1	2.931	3.459	0.8472	1
2×2	5.618	4.392	1.279	2
4×4	11	5.358	2.054	4
8×8	21.71	6.34	3.424	8
2×4	8.036	5.358	1.5	2
4×2	6.277	4.392	1.429	2

A line-of-sight channel is rank one however many antennas it has, so its capacity does not grow with antenna count — the measurement confirms it. The second column tracks the last column, which is the linear growth the derivation predicted.

Table 314. MIMO detectors, and what each costs

Detector	Complexity	Performance	Hardware
Zero forcing	$O(N^3)$ once per channel	Poor at low SNR — noise enhancement (Part X24)	A matrix inverse
MMSE	Same	Better; regularised	Same plus a diagonal add
Ordered SIC (V-BLAST)	N passes	Noticeably better than linear	Sequential — latency grows with N ; error propagation
Sphere / K-best	Tree search, bounded	Near maximum likelihood	A sorter and a tree walk — the standard hardware compromise

Detector		Complexity	Performance	Hardware
Maximum likelihood	likeli-hood	M^{N_t} candidates	Optimal	Infeasible beyond small cases: 16-QAM 4×4 is 65 536 candidates per symbol

Problem. Why does massive MIMO make the detector *easier* rather than harder, despite having far more antennas?

Solution. Because when N_r is much larger than N_t the columns of $\mathbf{H}$ become nearly orthogonal — a consequence of concentration of measure, since independent high-dimensional random vectors are nearly orthogonal with high probability. Nearly orthogonal columns mean $\mathbf{H}^H \mathbf{H}$ is nearly diagonal, so the simple matched filter (conjugate beamforming) is nearly optimal and the noise enhancement that makes zero forcing bad at low SNR essentially vanishes. **The hard detector problem is a symptom of having barely enough antennas**, and adding antennas at the base station — where size and power are affordable — dissolves it. The cost moves elsewhere and should be named: channel estimation for many antennas needs pilot resources and becomes the limiting factor (pilot contamination), the number of RF chains dominates cost and power, and the digital front end must move enormous sample rates from the antennas to the processing. **Massive MIMO trades a detection problem for an interconnect and calibration problem**, which is a trade the digital designer should recognise as favourable and should be ready to quantify.

X31. Analogue Building Blocks, Budgeted

A digital designer does not size transistors, but does have to say whether an analogue requirement is reasonable before committing to an architecture. The quantities that answer that are few and computable: thermal noise, matching, gain-bandwidth and settling. This part gives them.

X31.1 kT/C — the noise floor you cannot design away

Why sampled noise depends only on the capacitor

	Step	Justification
1	A switch has resistance R ; its thermal noise density is $4kTR$ V^2/Hz .	<i>Johnson–Nyquist.</i>
2	The RC network's noise bandwidth is $1/(4RC)$.	<i>Integrating a single-pole response gives $\pi/2$ times the 3 dB bandwidth.</i>
3	So the sampled noise power is $4kTR \cdot 1/(4RC) = kT/C$.	The resistance cancels. A better switch does not help; only a bigger capacitor does.
4	Doubling resolution means halving the noise voltage, which means quadrupling the capacitor .	<i>And capacitance is area, and it is also the load the driver must charge, so power rises too.</i>
5	Each extra bit of a sampled system costs 4× the capacitor and roughly 4× the power.	<i>This single relation explains the shape of every converter's energy-per-conversion curve above about 10 bits.</i>

Table 315. kT/C noise and the resolution it permits (1 V full scale, 300 K)

Capacitance (fF)	RMS noise (μV)	SNR (dB)	ENOB	Approx. area (μm^2 at 2 fF/ μm^2)
10	643.58	54.8	8.81	5
100	203.52	64.8	10.47	50
1000	64.358	74.8	12.13	500
10000	20.352	84.8	13.79	5000
1.0000×10^5	6.4358	94.8	15.45	50000

Computed from kT/C alone, ignoring every other noise source, so these are optimistic. A 14-bit sampled system needs picofarads, and the area and power that implies is why high-resolution converters are large and why oversampling architectures, which trade speed for resolution, exist.

Worked example 43. Can this ADC specification be met?

Given. A customer asks for a 16-bit, 100 MS/s SAR ADC on 1 V supply, ‘small area’.

Method. Work the kT/C requirement, then the settling requirement, then compare against what the process offers.

Required SNR for 16 bits	98.08 dB
Signal RMS at 1 V full scale	353.6 mV
Permitted noise RMS	4.41 μV
Required capacitance	212.96 pF
Array area at 2 fF/ μ m ²	1.0648$\times 10^5$ μm²
Settling time available at 100 MS/s, 16 SAR cycles	625 ps
Time constants needed for 16-bit settling	11.09
Required RC	56.36 ps
Verdict	the capacitor and the settling requirement are in direct conflict

How this is got wrong. By treating ‘16-bit’ and ‘100 MS/s’ as independent requirements. The noise requirement forces a large capacitor; the speed requirement forces that capacitor to settle in picoseconds, which forces a very low switch resistance and therefore an enormous switch, which loads the previous stage. This is why 16 bits at 100 MS/s is a time-interleaved or pipelined architecture and not a single SAR, and why the answer to the customer is a **question about their actual SNR requirement in the signal band** — if they need 16 bits over 1 MHz rather than over 50 MHz, oversampling changes the arithmetic completely.

X31.2 Matching: why analogue area is set by statistics

Pelgrom's law

	Step	Justification
1	Threshold voltage varies between nominally identical devices because of random dopant fluctuation and line-edge roughness.	<i>A counting phenomenon: fewer dopants under a smaller gate means larger relative variation.</i>
2	The standard deviation of the mismatch between two devices is $\sigma_{\Delta V_{th}} = A_{VT}/\sqrt{WL}$.	<i>Pelgrom's law, verified across many processes. A_{VT} is a process constant of a few mV$\cdot\mu$m.</i>
3	So halving the mismatch requires four times the area.	<i>The same 4$\times$ law as kT/C, arriving from a completely different mechanism.</i>
4	A comparator's input-referred offset is this mismatch; a current mirror's error is proportional to it.	<i>Which sets the minimum size of every matched pair in a converter.</i>
5	Digital calibration converts area into computation.	<i>Measure the offset once, store it, subtract it. This is the dominant trend in mixed-signal design and the reason the digital content of converters keeps growing.</i>

Table 316. Pelgrom matching against device area ($A_{VT} = 3.5$ mV $\cdot\mu$ m)

WL (μm^2)	$\sigma_{\Delta V_{th}}$ (mV)	3σ spread (mV)	Bits resolvable without calibration at 1 V FS	Pair area (μm^2)
0.01	35	210	2.252	0.02
0.1	11.07	66.41	3.913	0.2
1	3.5	21	5.573	2
10	1.107	6.641	7.234	20
100	0.35	2.1	8.895	200

Without calibration, matching alone limits an uncalibrated comparator to about eight bits at sane areas. Every converter above that resolution calibrates, and the calibration engine is digital — which is how a digital designer ends up owning a large part of an analogue block.

Graduate · The calibration trade, stated as arithmetic. Suppose a comparator needs 1 mV of offset. Pelgrom says that costs about $12 \mu\text{m}^2$ of gate area per device, and in a 64-way time-interleaved ADC that is repeated 64 times, in the fastest and most power-hungry part of the chip. Alternatively, build a minimum-size comparator with 10 mV of offset and add a trim DAC plus a digital loop that measures and cancels it. The trim DAC is small, the loop is a counter and a comparator, and the whole thing is synthesised. **The crossover is far to the digital side and moves further every process node**, because digital area shrinks and A_{VT} improves only slowly. This is the single clearest instance of the general trend this book keeps meeting: **buy analogue precision with digital computation.**

X31.3 Gain, bandwidth and settling

Table 317. What a resolution demands of an amplifier

Bits	Settling accuracy	Time constants needed	Minimum open-loop DC gain (dB)	Settling in units of 1/GBW
8	0.00195	6.238	54.19	0.9929
10	4.88×10^{-4}	7.625	66.23	1.213
12	1.22×10^{-4}	9.011	78.27	1.434
14	3.05×10^{-5}	10.4	90.31	1.655
16	7.63×10^{-6}	11.78	102.4	1.875

*Static accuracy needs loop gain (finite gain leaves a proportional error); dynamic accuracy needs time constants. **16-bit accuracy needs about 100 dB of DC gain and twelve time constants**, and those two requirements fight each other because high gain means more stages means more poles means harder compensation.*

Table 318. The analogue specifications a digital architect should be able to sanity check

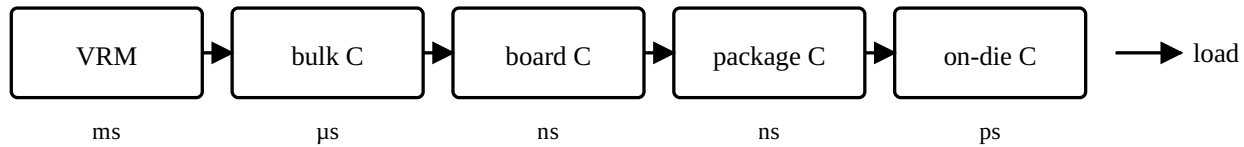
Quantity	Rule of thumb	If the requirement exceeds it
Sampled SNR	kT/C ; 14 bits needs $>1 \text{ pF}$ at 1 V	Oversample, or relax full scale, or use a different architecture
Uncalibrated comparator offset	Pelgrom; ~ 8 bits at sensible area	Add digital calibration
Amplifier DC gain	$\sim 6 \text{ dB}$ per bit	Gain boosting, or a correlated-double-sampling scheme
Aperture jitter	$\text{SNR} = -20\log(2\pi f_{in} \sigma_t)$ — Part X12	A better clock, not a better converter
Reference noise	Appears directly at the output	Filter it; and remember the filter capacitor is area too
Supply rejection	Falls with frequency; poor above the loop bandwidth	This is why the PDN of Part X32 matters to analogue blocks

Problem. Your digital block shares a supply with an ADC and the customer reports spurs at your clock frequency. Whose problem is it?

Solution. Both of yours, and the useful response is to find out which mechanism rather than which party. There are three paths and they are distinguishable by measurement. **Supply coupling:** your switching current modulates the shared rail, and the converter's finite power-supply rejection turns that into an input offset; test by changing your block's activity and watching whether the spur amplitude tracks, and by separating the supplies if the board permits. **Substrate coupling:** switching current injects into the substrate and reaches the converter's sensitive nodes; this is layout, and the remedies are guard rings, deep n-well isolation and distance. **Clock coupling:** your clock capacitively couples into the sampling network, which is the worst of the three because it lands exactly at a fixed frequency and its harmonics. **The digital side's contribution, and it is a real one, is to reduce di/dt :** spread the clock tree's switching with deliberate skew, gate clocks aggressively so the average current is lower, add on-chip decoupling, and avoid large simultaneous switching events. **None of that is exotic, and all of it is far cheaper than asking the analogue designer for another 20 dB of supply rejection,** which may not be available at any price.

X32. The Power Delivery Network, Worked

Every timing number in Part X2 assumed a supply voltage. The network that delivers it is a distributed RLC structure whose impedance must be low across six decades of frequency, and when it is not, the result is a voltage droop that shows up as a timing failure nobody can reproduce. This is a digital designer's problem even though it looks like an analogue one.



Each stage covers a different decade of time.

A gap between two of them is an impedance peak, and an impedance peak is a droop.

Figure 45. The power delivery hierarchy. Each element handles a different timescale because each is limited by its own parasitic inductance.

X32.1 Target impedance, and where the number comes from

From a droop budget to an impedance specification

	Step	Justification
1	The supply must stay within $\pm\delta$ of nominal for the timing libraries to apply.	<i>Typically 5 %. Outside it, the delays are not the ones you signed off with.</i>
2	A current step ΔI produces a voltage $Z(f)\Delta I$.	<i>Ohm's law for an impedance.</i>
3	Target impedance $Z_t = \delta V_{dd} / \Delta I_{\max}$.	<i>The one number the whole PDN is designed to, and it must be met at every frequency, not on average.</i>
4	$\Delta I_{\max}$ is the worst-case step, not the average current.	<i>A clock-gated block waking up, or a test pattern that toggles everything, is a far larger step than any functional average — which is why maximum-toggle test patterns can fail a part that works in its application.</i>
5	Falling voltage and rising current per node make Z_t shrink every generation, quadratically.	<i>This is why PDN design has become progressively harder rather than benefiting from scaling.</i>

Table 319. Target impedance across process generations (5 % droop budget)

Node	V_{dd} (V)	Peak current (A)	Z_t (m Ω)	Peak power (W)	Droop budget (mV)
180 nm	1.8	2	45	3.6	90
90 nm	1.2	10	6	12	60
28 nm	1	40	1.25	40	50
7 nm	0.8	120	0.3333	96	40
5 nm	0.75	250	0.15	187.5	37.5

The current figures are indicative of a large SoC at each generation rather than any specific part. **The impedance target falls by more than two orders of magnitude across the table**, which is why on-die decoupling and integrated voltage regulators appeared.

Worked example 44. Sizing on-die decoupling capacitance

Given. A block draws 5 A when active and is clock-gated. On wake-up the current rises in 2 ns. The package inductance is 100 pH and the droop budget is 40 mV.

Method. The package cannot supply current faster than its inductance allows, so on-die capacitance must cover the interval until the package catches up. Compute the charge needed and hence the capacitance.

Current step	5 A
Rise time	2 ns
$L di/dt$ if the package had to supply it	250 mV
Droop budget	40 mV
Verdict	the package alone exceeds the budget by 6×
Charge to supply during the step	5 nC
Capacitance for a 40 mV droop	125 nF
Area at 10 fF/μm ² (MOS cap)	12.5 mm ²

How this is got wrong. By budgeting only the resistive drop. At a nanosecond timescale the package is an inductor, not a wire, and $L di/dt$ dominates the static IR term by a large factor. **The remedy is not lower resistance but local charge storage**, and the area it costs is real — on-die decoupling is commonly several per cent of die area and in some designs more than ten. A second, cheaper remedy belongs to the digital designer: **ramp the current instead of stepping it**, by staging the clock-gate release across several cycles, which is a few gates of logic and can reduce the required capacitance by a large factor.

X32.2 Resonance: the peak that puts the droop where you are not looking

Table 320. Resonant peaks formed by each inductance with the capacitance below it

L (pH)	C (nF)	R (mΩ)	Resonant frequency (MHz)	Q	Peak impedance (mΩ)
1000	100	5	15.915	20	100
1000	1000	5	5.0329	6.325	31.62
100	100	2	50.329	15.81	31.62
100	10	2	159.15	50	100
10	100	1	159.15	10	10

Every inductance in the hierarchy resonates with the capacitance on the load side of it, and at resonance the impedance is Q times the series resistance. **A high- Q peak is the dangerous case:** if the current spectrum has energy there, the droop is far worse than the target impedance suggests.

The first-droop, second-droop, third-droop language engineers use is just this resonance hierarchy, and the dangerous one is usually the mid-frequency droop. The first droop (hundreds of megahertz) is handled by on-die capacitance; the third (kilohertz) by the regulator's control loop. The second — tens to a couple of hundred megahertz, set by package inductance against on-die capacitance — sits in a gap between the two remedies and is where the highest- Q peak usually lands. **It is also, unhelpfully, right where a processor's activity changes:** a loop that alternates between a heavy and a light phase at that rate excites it directly, which is the origin of the classic 'this benchmark fails and that one does not' report. The digital remedies are to add damping (series resistance in some decoupling, deliberately), to spread activity in time, and to use an adaptive

clock that stretches the period when a droop is detected — which converts a functional failure into a small performance loss and is now common in large processors.

X32.3 Electromigration and the static budget

Table 321. Current a wire can carry before electromigration limits its life (indicative 1.5×10^6 A/cm², 0.2 μ m thick)

Width (μ m)	Cross-section ($\times 10^{-8}$ cm ²)	Max current (mA)	mA per μ m of width	Resistance per μ m length ($\mu\Omega$)
0.1	0.02	0.3	3	2.2000×10^5
0.2	0.04	0.6	3	1.1000×10^5
0.5	0.1	1.5	3	44000
1	0.2	3	3	22000
2	0.4	6	3	11000

Indicative constants, not a PDK. The useful content is the third column: a few milliamps per micron of width is the order of magnitude, so a 5 A block needs of the order of a millimetre of total metal width to feed it — which is why power grids are wide, numerous and on the thick upper metal layers.

Problem. Your block passes timing in every corner and fails intermittently in silicon at high activity. How do you decide whether it is the PDN?

Solution. Look for the signature, which is specific and distinguishable. **A PDN-induced failure is activity dependent rather than pattern dependent:** it appears when many things switch at once, not when a particular logical case occurs, so a test that toggles a lot fails while a functionally harder but quieter test passes. **It moves with supply and with frequency in a particular way:** raising the supply helps more than it should for a simple timing problem, because it increases the margin against droop as well as speeding the logic. **It correlates with di/dt, not with average power,** so a test with the same average current but smoother activity passes. The measurements that settle it are an on-die droop monitor if one exists — and if none exists, that is the lesson for the next design, because a ring-oscillator-based droop detector is a few hundred gates — and an experiment that staggers the wake-up of the block's clock-gating domains, which changes only di/dt. **If staggering fixes it, the PDN is the cause and the cheapest fix is the staggering you just used to diagnose it.**

X33. Boolean Reasoning: BDDs, SAT and Equivalence Checking

Three tools an IP designer uses every week — logic equivalence checking, formal property verification and automatic test pattern generation — are the same engine underneath: a decision procedure for Boolean formulas. Knowing what that engine can and cannot do explains why one proof finishes in seconds and another never finishes, which is otherwise the most frustrating thing about these tools.

X33.1 Representations, and why the choice is everything

Table 322. Ways to represent a Boolean function

Representation	Size	Equivalence check	Where it is used
Truth table	2^n always	Trivial	Small cells; library characterisation
Sum of products	Can be exponential	Hard	Two-level minimisation, PLAs
BDD (reduced, ordered)	Canonical — and variable-order dependent	Pointer comparison	Symbolic model checking; some equivalence checking
AIG (and-inverter graph)	Linear in the circuit	Needs a solver	The working representation of every modern synthesis and verification tool
CNF for SAT	Linear via Tseitin encoding	NP-complete, and fast in practice	Formal verification, ATPG, equivalence

Graduate · **Canonicity is the property that makes BDDs both magical and fragile.** Two functions are equal if and only if their reduced ordered BDDs are the identical graph, so equivalence is a pointer comparison rather than a search. The price is that the size depends catastrophically on the variable order: the same function can be linear under one order and exponential under another, and finding the best order is itself NP-hard. The canonical example is a multiplier, whose BDD is exponential under *every* order — which is exactly why BDD-based equivalence checking fails on datapaths and why modern tools use AIGs plus SAT instead. **When a tool reports that it is ‘building BDDs’ and then runs out of memory, this is what happened,** and the remedy is a different engine rather than a bigger machine.

Table 323. Measured decision-diagram size for $x_0y_0 + x_1y_1 + \dots$ under two variable orders

Variables	Product terms	Interleaved order (nodes)	Separated order (nodes)	Ratio
4	2	4	6	1.5
8	4	8	30	3.75
12	6	12	126	10.5
16	8	16	510	31.88

Built by exhaustive construction with subfunction sharing, which is what reduction does. **The interleaved column is exactly n ; the separated column is $2^{n/2+1}-2$.** Linear against exponential, on the same function with the same tool. The ratio is modest at these sizes because the exponent is small — at 40 variables it is a factor of 52 000 — $(2^{21}-2)/40$ — which is the difference between a second and a week. This is the whole of BDD practice in one table.

X33.2 Why SAT is fast even though it is NP-complete

What a modern SAT solver actually does

	Step	Justification
1	Convert the circuit to CNF by Tseitin encoding: one variable per gate, a few clauses per gate.	<i>Linear in circuit size — no blow-up here, unlike naive expansion.</i>
2	Search by assigning variables, propagating implications, and backtracking on conflict.	<i>DPLL. The naive version is hopeless.</i>
3	On conflict, derive a new clause that explains it (clause learning) and add it.	<i>This is the single change that made SAT practical. The learned clause prunes an entire region of the search space, not just the current branch.</i>
4	Backjump non-chronologically to the level the conflict actually came from.	<i>Rather than undoing one decision at a time.</i>
5	Restart periodically, keeping the learned clauses.	<i>Escapes a bad region of the search while retaining what was learned.</i>
6	Result: industrial instances with millions of variables solve in seconds , while small random instances at the hardness threshold do not.	<i>Structure, not size, decides. Circuits have enormous structure; that is why circuit problems are the easy case and why a tool's runtime is unpredictable in the specific way engineers observe.</i>

Worked example 45. Why equivalence checking succeeds where simulation cannot

Given. Comparing a 64-bit multiplier's RTL against its gate netlist.

Method. Compare the exhaustive-simulation cost with what an equivalence checker does instead.

Input space	2^{128}
Vectors per second, simulated	10^7
Time to simulate exhaustively	$>10^{23}$ years
What an equivalence checker does	proves the two are equal for all inputs
How	structural matching of internal points, then SAT on each cone
Why it works	the two designs share almost all their internal structure
When it fails	after retiming, resource sharing, or a datapath rewrite — the structure no longer matches

How this is got wrong. By assuming equivalence checking always works. It works because it finds internal equivalence points and cuts the problem into small cones; a transformation that destroys that correspondence — retiming across register boundaries, sequential optimisation, a multiplier architecture change — leaves the tool with one enormous cone and it fails. **The practical consequence is a rule for the synthesis flow:** turn on transformations that break equivalence checking only when you are prepared to verify the result another way, and know which ones they are before the schedule depends on them.

Table 324. Why cone size, not design size, decides an equivalence check

Situation	Cones	Typical cone size (gates)	Total gates	Difficulty
Control logic	50,000	40	2,000,000	easy

Situation	Cones	Typical cone size (gates)	Total gates	Difficulty
Datapath adders	2,000	200	400,000	easy
Multiplier (structural match)	500	2,000	1,000,000	easy
Multiplier (after rewrite)	1	500,000	500,000	the hard case

Illustrative structure counts. The last row has the same total size and is the one that does not finish, because the work per cone grows far faster than linearly.

X33.3 Model checking: proving something about all time

Bounded model checking, and what 'bounded' costs

	Step	Justification
1	Unroll the design's transition relation k times.	<i>Copy the combinational logic k times, chaining state.</i>
2	Assert the initial state and the negation of the property.	<i>If satisfiable, the assignment is a counterexample trace of length $\leq k$.</i>
3	Hand the whole thing to SAT.	A bug of depth $\leq k$ is found; a bug of depth $k+1$ is not.
4	So BMC is a very good bug hunter and not a proof .	<i>This is the distinction that matters when reading a tool's report: 'proven to depth 40' is not 'proven'.</i>
5	A full proof needs induction, interpolation or IC3/PDR, which find an inductive invariant.	These do give unbounded proofs , and they are what a tool means by 'proven' without a depth.
6	The bound needed for confidence is the design's <i>diameter</i> — how many cycles to reach any reachable state.	<i>Usually unknown, and for a design with a large counter it is astronomically large. Which is why counters are abstracted before proving anything around them.</i>

The three ways a formal proof lies to you, in order of frequency. **Over-constraint:** an assumption excludes the input that would have failed, so the property is proven about a machine that is not yours. This is the formal equivalent of a disabled checker and is by far the commonest; the defence is to check that the constraints are satisfiable together with interesting behaviour, which good tools report as 'assumption coverage'. **Vacuity:** the property's antecedent never occurs, so it holds trivially — 'if a grant is given without a request...' proven because no grant is ever modelled. **A bounded result read as a proof**, as above. **All three report green**, which is why this book's rule applies with full force here: a proof whose assumptions have not themselves been checked is an unexamined green light.

Problem. Where should a design house spend its formal effort, given that it cannot afford to prove everything?

Solution. On the properties whose failure is catastrophic and whose state space is deep and narrow, which is where formal has the advantage over simulation. In practice that means, in order: **arbiters and any structure that can starve**, because starvation is a liveness property and simulation can never prove its absence; **FIFOs, credit counters and flow control**, where overflow and underflow are provable and the bugs appear only at full rate; **protocol interfaces**, where a vendor-supplied assertion set exists and proving compliance at the boundary is worth more than any amount of traffic; **clock-domain crossing structures**, where the interesting cases are timing-related and simulation is blind to them; and **security properties**, where the question is whether information can flow between two places and the answer must hold for all inputs. **What not to spend it on: datapath equivalence at the algorithm level** — a bit-exact reference model plus random simulation gets there faster — **and anything with a large counter in the cone**, until the counter has been abstracted.

X34. Logic Synthesis: What the Tool Is Actually Doing

Synthesis is the step an IP designer has least visibility into and most dependence on. Its behaviour is not arbitrary: it is a sequence of well-defined transformations with known strengths, and knowing them turns ‘the tool did something odd’ into a specific diagnosis.

X34.1 The pipeline, stage by stage

Table 325. What each synthesis stage does and what it cannot do

Stage	Input → output	Optimises	Cannot
Elaboration	RTL → generic netlist	Nothing — it interprets the language	Fix a coding style that infers a latch
Datapath extraction	Operators → architectures	Picks adder and multiplier structures from a library	Restructure an algorithm you wrote out by hand as gates
Technology-independent optimisation	Generic netlist → smaller generic netlist	Factoring, common subexpressions, constant propagation, redundancy removal	Know anything about delay in picoseconds
Technology mapping	Generic → library cells	Covers the network with cells at minimum cost — solvable optimally on a tree	Change the network's structure much
Timing-driven restructuring	Netlist → netlist	Balances the critical path, duplicates, remaps	Buy more than a limited amount — see Part X2
Sizing and buffering	Netlist → sized netlist	Convex, near-optimal (Part X26)	Fix a structural problem

Graduate · **Technology mapping is optimal on trees and heuristic on graphs, and that single fact explains most of what a synthesis tool does well.** The classic formulation covers a Boolean network with library cells at minimum cost. If the network is a tree, dynamic programming over the nodes gives a provably optimal cover in linear time. Real networks are DAGs with reconvergent fanout, so the tool partitions them into trees at multiple-fanout points, maps each optimally, and accepts that the partition itself was a heuristic choice. **The practical consequence is that logic with heavy reconvergence maps worse than logic without it,** and that restructuring an expression to reduce fanout — which looks cosmetic in RTL — can measurably change the result.

Table 326. A balanced tree of two-input gates: the structure mapping likes

Depth	Gates	Logic levels	DP table entries (4 cells/node)	Gate delays at 2 per level
2	3	2	12	5
4	15	4	60	9
6	63	6	252	13
8	255	8	1,020	17
10	1,023	10	4,092	21
12	4,095	12	16,380	25

*Dynamic-programming mapping visits each node once per candidate cell, so it is linear in gates times library size. **This is why mapping a million-gate design takes minutes** even though the underlying covering problem is NP-hard on general graphs.*

X34.2 What your RTL controls, and what it does not

Table 327. RTL constructs and what the tool does with them

You write	The tool produces	You control it by
<code>a + b</code>	An adder chosen from an architecture library	The timing constraint — not by writing the adder yourself
<code>a * b</code>	Booth or array multiplier, possibly shared	Constraints, and <code>set_dont_use</code> if the choice is wrong
<code>case</code> (full)	A multiplexer tree	Ordering and priority annotations
<code>case</code> (incomplete, no default)	A latch — almost never what you meant	Write the default; lint for it
<code>if</code> chain	Priority logic — deep and slow	Use <code>case</code> when the conditions are mutually exclusive
A for loop	Unrolled combinational logic	The loop bound; there is no hardware loop
<code>x <= y</code> in a clocked block	A flip-flop	—
Division by a variable	A large slow divider	Do not; see Part X9
Division by a constant	Shifts and adds	Nothing needed — the tool does this well

Worked example 46. The cost of an if-chain that should have been a case

Given. A 16-way priority `if/else if` chain versus a 16-way `case`, each selecting between 16 values.

Method. Count the logic levels. A priority chain is a serial selection; a `case` is a balanced tree, because the tool knows the conditions are mutually exclusive.

Priority chain: multiplexers in series	16
Delay at ~2 gate levels per 2:1 mux	32 gate levels
Case: mux tree depth	4
Delay	8 gate levels
Ratio	4×
At 25 ps per gate level	800 ps vs 200 ps
Period this costs at 1 GHz	60 % of it, for nothing

How this is got wrong. By assuming the tool will notice the conditions are exclusive. Sometimes it does, and it cannot in general — proving exclusivity is a Boolean problem it does not always attempt. Writing `case`, or annotating the chain as parallel, tells it directly. **This is the single highest-value RTL coding habit for timing**, and it costs nothing; a priority structure should appear only where priority is actually meant, and then the depth is a deliberate cost rather than an accident.

X34.3 Retiming: moving registers without changing behaviour

Why retiming is a graph problem with an exact answer

	Step	Justification
1	Model the circuit as a graph: nodes are combinational blocks with delay, edges carry a register count.	<i>The standard retiming model.</i>
2	Retiming assigns each node an integer r_v : move r_v registers from its outputs to its inputs.	<i>Registers on edge (u,v) become $w + r_v - r_u$.</i>
3	Legality requires every edge to keep a non-negative count: $r_u - r_v \leq w$.	A system of difference constraints — exactly the structure Part X26 identified as solvable.
4	Achieving period P adds more difference constraints between node pairs.	<i>Derived from the longest combinational path between them.</i>
5	So ‘is period P achievable by retiming?’ is a negative-cycle test, and binary search over P finds the optimum.	Polynomial, exact, and implemented in every synthesis tool.
6	What it cannot do: change the number of registers on any cycle.	The latency around a feedback loop is invariant under retiming , which is why retiming cannot fix a loop that is too long — Part X4’s DFE problem is exactly this, and the answer there was unrolling, which changes the graph.

Retiming is exact and is nevertheless often turned off, for reasons worth knowing. It renames and moves registers, which breaks the structural correspondence equivalence checking depends on (Part X33), so verification becomes sequential equivalence checking — a much harder problem that may not complete. It also destroys the correspondence between RTL signal names and netlist nodes, which makes debug and ECOs painful, and it can interact badly with scan insertion and with clock gating. **The common compromise is to retime within a clearly delimited pipeline and to verify that block by simulation against its pre-retiming version**, rather than enabling it design-wide.

Problem. Synthesis reports your block at 620 MHz against a 700 MHz target. List what you would try, in order of cost.

Solution. Cheapest first, because the early items often suffice. **Check the constraints before the design:** an over-constrained input delay, a missing multicycle path, or a clock defined without its uncertainty will each produce a believable-looking failure that no RTL change fixes. **Read the critical path and classify it** — if it crosses a whole datapath it is structural, if it is a long priority chain it is X34.2, if it starts or ends at a boundary it is a constraint issue, and each has a different fix. **Try the free restructuring:** turn an if-chain into a case, split a wide comparison, move a constant multiplication to a shift-add, precompute something that depends only on slowly changing inputs (Part X9’s rule). **Then pipeline** — one more stage, which costs latency and a verification change and is usually decisive. **Then re-architect:** parallel slices at a lower rate, which is Part X7’s energy argument arriving as a timing fix. **What not to do first is instantiate gates by hand;** it is slow, it does not port, and the tool is better at it than you are except in the rare case where you know something it cannot — and then you should be telling it, through a constraint, rather than bypassing it.

X35. Hardware Data Structures: CAM, Hashing and Bloom Filters

Networking, storage and security IP spend most of their logic looking things up. The structures that do it are a small, well-understood family with sharply different cost profiles, and choosing among them is one of the clearest cases in this book where an architectural decision is settled by arithmetic rather than by preference.

X35.1 The lookup family

Table 328. Lookup structures compared

Structure	Query	Area per entry	Power	Updates	Used for
RAM (direct index)	1 cycle	1 bit/bit	Low	1 cycle	Small, dense key spaces
Binary CAM	1 cycle, all entries in parallel	~8–10 transistors per bit	Very high — every cell compares every cycle	1 cycle	MAC address tables, tag matching
Ternary CAM	1 cycle, with don't-cares	~16 transistors per bit	Higher still	Needs priority ordering — insertion can require moving entries	Longest-prefix routing, ACLs
Hash table	1–2 cycles typical	~1 bit/bit plus load factor	Low	1 cycle, except on collision	Flow tables, caches
Cuckoo hash	d parallel probes, worst case bounded	~1.05 bits/bit at high load	Low	Insertion can cascade	Flow tables where a bounded query time is required
Trie / tree	$\log n$ memory accesses	Pointer overhead	Low	Simple	Prefix matching in software
Bloom filter	k probes	~10 bits per entry for 1 % false positive	Low	Insert only	A cheap pre-filter in front of an expensive lookup

Graduate TCAM is the structure that decides the cost of a router, and it is worth knowing why it is so expensive. A ternary cell stores three states — 0, 1 and don't-care — which needs two storage bits plus comparison logic, roughly sixteen transistors against six for SRAM. Worse, every cell compares on every lookup, so power is proportional to the whole table rather than to the entry found: a large TCAM can dissipate tens of watts doing nothing but matching. **The architectural responses all reduce the number of cells that must compare** — partition the table by prefix length and activate only relevant banks, put a small filter in front, or replace TCAM with an algorithmic trie in SRAM and accept multiple accesses. Every high-end switch datasheet's power figure is substantially a statement about this one structure.

Table 329. TCAM cost against table size, 144-bit keys (indicative 16 T/cell against 6 T SRAM, 1 GHz lookup)

Entries	Key bits	SRAM-equivalent (MB)	TCAM area as SRAM-equivalent (MB)	Match-line power (W, indicative)
1,000	144	0.01717	0.09155	1.728×10^{-4}
10,000	144	0.1717	0.9155	0.001728
100,000	144	1.717	9.155	0.01728
1,000,000	144	17.17	91.55	0.1728

Indicative constants, not a vendor datasheet. The shape is the point: area roughly five times SRAM for the same bits, and a power term that grows with table size independently of the traffic.

X35.2 Hashing: the birthday problem decides your table size

Why a hash table needs to be much larger than its contents

	Step	Justification
1	With n items in m buckets, the probability that a particular pair collides is $1/m$.	<i>Uniform hashing assumption.</i>
2	There are $n(n-1)/2$ pairs, so the expected number of colliding pairs is about $n^2/(2m)$.	<i>Linearity of expectation.</i>
3	Collisions start appearing when $n \approx \sqrt{m}$, not when $n \approx m$.	<i>The birthday paradox. This is the fact that surprises people sizing tables.</i>
4	The longest chain with $n = m$ is $\Theta(\log n / \log \log n)$ — about 5–8 for realistic sizes.	<i>So a single-probe hardware lookup fails often enough to matter.</i>
5	Two hash functions and a choice of the emptier bucket reduces the longest chain to $\log \log n$.	<i>The power of two choices: an exponential improvement from one extra probe. This is why hardware hash tables are multi-way.</i>

Table 330. Measured longest bucket in a 4096-bucket table, one hash against two-choice hashing, 30 trials per point

Load factor	Keys	Longest bucket, 1 hash	Longest bucket, 2 choices	Ratio
0.25	1,024	3.5	2.1	1.667
0.5	2,048	4.367	3	1.456
0.75	3,072	5.5	3.4	1.618
0.9	3,686	6.133	3.9	1.573
0.95	3,891	6.2	4.033	1.537

Simulated with random keys, seed 13. The second hash roughly halves the worst bucket at every load, which in hardware is the difference between a lookup engine that must handle eight-deep chains and one that handles four.

Worked example 47. Sizing a flow table for a bounded lookup time

Given. A network block must look up 200 000 flows at 100 Gbit/s, one lookup per 64-byte packet, with a *bounded* lookup time of two memory accesses.

Method. Bounded worst case rules out chaining. Compute what a 4-way cuckoo table costs at a load factor it can sustain.

Packet rate at 64 B	148.8 Mpps
Lookups per second	1.49×10^8
Flows	200,000
Cuckoo load factor achievable at 4 ways	0.97
Buckets needed	206,185
Entries at 4 per bucket	206,184
Memory at 64 B per entry	12.58 MB
Probes per lookup	4 (parallel, one bucket read)
Worst-case accesses	2

How this is got wrong. By choosing the average-case structure for a worst-case requirement. Chained hashing has a superb average and an unbounded tail, and a networking datapath is judged on the tail: a lookup that occasionally takes eight accesses either stalls the pipeline or drops the packet. **Cuckoo hashing buys a bounded query at the cost of a potentially long insertion**, which is the right trade because insertions are rare and can be handled by a slow path. Recognising which operation must be bounded is the whole of the design decision.

X35.3 Bloom filters: paying memory for certainty in one direction

The false-positive rate, and the optimal number of hashes

	Step	Justification
1	A Bloom filter is m bits; inserting an item sets k bits chosen by k hashes.	<i>No item is stored — only its fingerprint in the bit array.</i>
2	After n insertions, the probability a given bit is still zero is $(1-1/m)^{kn} \approx e^{-kn/m}$.	<i>Independent settings.</i>
3	A query gives a false positive when all k bits happen to be set: $p = (1 - e^{-kn/m})^k$.	Never a false negative — that asymmetry is the whole value.
4	Minimising over k gives $k = (m/n) \ln 2$.	<i>And then $p = 2^{-k}$.</i>
5	So about $1.44 \log_2(1/p)$ bits per item, regardless of how large the items are.	<i>A 1 % filter costs about 10 bits per item whether the items are 32-bit or 1500-byte. That independence from key size is why it is used in front of expensive lookups.</i>

Table 331. Bloom filter false-positive rate against bits per item

Bits/item	Optimal hashes k	False positive	(%)	Memory for 10^6 items (MB)
4	3	0.147	14.7	0.4768
8	6	0.0216	2.16	0.9537
10	7	0.00819	0.819	1.192

Bits/item	Optimal hashes k	False positive	(%)	Memory for 10^6 items (MB)
12	8	0.00314	0.314	1.431
16	11	4.59×10^{-4}	0.0459	1.907
20	14	6.71×10^{-5}	0.00671	2.384

Computed from the closed form. **Ten bits per item buys about one per cent**, and each further factor of ten costs about 3.3 bits — a very good rate of exchange when the alternative is storing the keys.

Problem. Where does a Bloom filter belong in an IP block, and where does it not?

Solution. It belongs wherever a **negative answer is common and an expensive lookup can be skipped**, and where a false positive costs only wasted work rather than a wrong result. Concretely: in front of a TCAM or an off-chip table, to avoid the access entirely for the majority of keys that are absent; in a cache to avoid probing a way; in a coherence directory to avoid a snoop. In each case the filter is an optimisation and the real structure still answers. **It does not belong anywhere the answer is used directly**, because a false positive is then a wrong answer — and it does not belong where deletions are needed, because a plain Bloom filter cannot delete (clearing a bit may un-insert another item); that needs a counting Bloom filter at four times the memory, and at that point a small hash table is often better. **The design question to ask is: what is the measured fraction of negative queries?** A filter in front of a table that is usually hit is pure cost, and that fraction is a property of the customer's traffic, which means it belongs in the datasheet as an assumption.

X36. Cryptographic Engines, Sized

Crypto IP is one of the most commonly licensed categories, and it is unusual in that its correctness is binary, its specifications are published and testable, and its cost is dominated by a small number of arithmetic primitives. That combination makes it a good first product for a design house and a good subject for exact sizing.

X36.1 The three primitive families and their costs

Table 332. What each family needs from the hardware

Family	Examples	Core operation	Cost driver	Typical area
Block ciphers	AES, ChaCha, SM4	Substitution and permutation over a small state	The S-box, replicated	5–50 kGE
Hashes	SHA-2, SHA-3, BLAKE	A compression function over a message block	Adders and rotations; long carry chains in SHA-2	10–30 kGE
Public key	RSA, ECC, Diffie–Hellman	Modular exponentiation or point multiplication	The modular multiplier — dominates everything	50–500 kGE
Post-quantum	ML-KEM (Kyber), ML-DSA (Dilithium)	Polynomial arithmetic in a ring, NTT	NTT butterflies and modular reduction — Part X6's FFT with a different field	50–200 kGE
MAC / AEAD	GCM, Poly1305, CMAC	Carry-less or modular multiply-accumulate	$GF(2^{128})$ multiplier for GCM	10–40 kGE

Graduate · **Post-quantum cryptography is an opportunity for design houses precisely because it is new.** The standards are recent, the hardware literature is thin, the incumbent crypto-IP vendors' portfolios were built for RSA and ECC, and the core operation — the number-theoretic transform — is an FFT over a finite field, which means Part X6's entire architectural analysis transfers: the same radix choices, the same twiddle-ROM symmetries, the same memory-versus-multiplier trade. **A team that has built an FFT has most of what an NTT needs,** with the arithmetic replaced by modular arithmetic, and that is an unusually direct path from an existing competence to a new market.

X36.2 AES: where the area actually goes

The S-box is the block

	Step	Justification
1	AES-128 is ten rounds of SubBytes, ShiftRows, Mix-Columns, AddRoundKey.	<i>Specification.</i>
2	ShiftRows is wiring; AddRoundKey is 128 XORs; Mix-Columns is a fixed $GF(2^8)$ matrix — shifts and XORs.	All three are nearly free.
3	SubBytes applies a 256-entry nonlinear substitution to each of 16 bytes.	This is the whole cost.
4	As a table it is 16×2048 bits of ROM or $16 \times \sim 700$ gates synthesised.	<i>The naive implementation.</i>
5	Or compute it: the S-box is inversion in $GF(2^8)$ plus an affine map, and inversion decomposes through $GF((2^4)^2)$.	<i>Canright's tower-field construction reduces the S-box to roughly a third of the table's area, at the cost of more logic depth. The trade is area against frequency, and which side wins depends entirely on the throughput requirement.</i>
6	A round can use 16 S-boxes (one cycle per round), 4, or 1.	Throughput scales linearly with S-box count and so does area , which makes AES one of the cleanest area/throughput families to parameterise — and therefore a good product, because one design serves many customers.

Table 333. AES-128 area and throughput against S-box count, at 1 GHz

S-boxes	Cycles per block	bits/cycle	Gbit/s	Approx. gates	Mbit/s per kGE
16	10	12.8	12.8	13,200	969.7
8	20	6.4	6.4	7,600	842.1
4	40	3.2	3.2	4,800	666.7
1	160	0.8	0.8	2,700	296.3

Gate counts are indicative (700 GE per table-based S-box plus 2 kGE of state and key schedule). **The last column is nearly constant**, which is the signature of a cleanly scalable block: the customer chooses a point on the line rather than a different design.

Worked example 48. Which AES configuration does a 400 Gbit/s link need?

Given. MACsec on a 400 Gbit/s Ethernet port, AES-GCM, digital logic at 800 MHz.

Method. Work out bits per cycle, then how many parallel AES cores that implies, then check the GCM multiplier alongside it.

Line rate	400 Gbit/s
Clock	800 MHz
Bits per cycle required	500
AES block size	128
Blocks per cycle	3.906
Fully unrolled pipelined AES cores needed	4
Pipeline stages each (one per round)	10
$GF(2^{128})$ multiplies per cycle (GCM)	4
Dominant cost	the GCM multiplier, not AES

How this is got wrong. By sizing AES and forgetting the mode. Counter mode parallelises perfectly — each block's key-stream is independent — so AES scales by replication. The GCM authentication tag is a *sequential* multiply-accumulate in $GF(2^{128})$, which does not parallelise naively; making it keep up requires the standard trick of precomputing powers of the hash key and processing several blocks at once, which costs several 128-bit carry-less multipliers and their key storage. **At high line rates the authenticator is larger than the cipher**, and a datasheet that quotes only AES throughput has quoted the easy half.

X36.3 Public key: modular multiplication is the whole problem

Montgomery multiplication, and why division disappears

	Step	Justification
1	Modular multiplication needs $ab \bmod n$; the reduction is a division, which is expensive (Part X9).	<i>And it is needed thousands of times per operation.</i>
2	Montgomery's idea: work with $a^- = aR \bmod n$ for $R = 2^k > n$.	<i>A change of representation.</i>
3	Then $REDC(x) = (x + (xn' \bmod R)n)/R$ reduces using only multiplications and shifts , because dividing by R is a shift.	The division is gone. n' is precomputed once.
4	Conversion in and out costs two extra multiplications, amortised over the whole exponentiation.	<i>Negligible over thousands of operations.</i>
5	So an RSA engine is: a wide multiplier, an adder, a shifter, and a controller.	And the wide multiplier is 80–90 % of the area, <i>which is why the literature is about multiplier architectures and not about protocols.</i>

Table 334. Public-key work, measured in modular multiplications and cycles (64-bit multiplier, schoolbook, 500 MHz)

Operation	Modulus bits	Modular multiplies	64-bit words	Cycles (approx.)	Time (ms)
RSA-2048 (CRT)	1,024	3,072	16	196,608	0.3932
RSA-4096 (CRT)	2,048	6,144	32	1,572,864	3.146
ECC P-256	256	3,993	4	15,974	0.03195
ECC P-384	384	5,990	6	53,913	0.1078

Operation	Modulus bits	Modular multiplies	64-bit words	Cycles (approx.)	Time (ms)
X25519	255	3,315	4	13,260	0.02652

The multiplication counts are the standard estimates: square-and-multiply for RSA with the Chinese remainder theorem halving the modulus, and roughly a dozen field operations per elliptic-curve point addition. **The table explains the industry's migration:** ECC at 256 bits gives comparable security to RSA at 3072 and costs orders of magnitude less work.

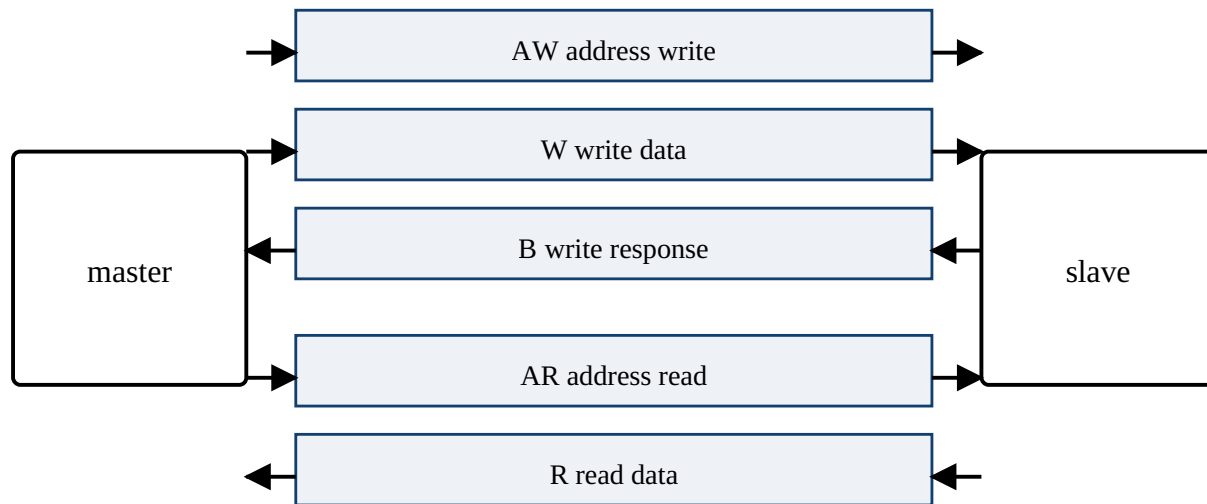
A public-key engine that is merely correct is dangerous. The classical attacks are not on the mathematics but on the implementation: a square-and-multiply loop that branches on key bits leaks the key through timing and through power (Part X15), and a Montgomery ladder — which performs the same operations regardless of the bit — costs perhaps 30 % more work and removes the leak. Likewise a conditional subtraction at the end of a reduction is a data-dependent branch, so implementations make it unconditional. **Constant-time behaviour must be a stated requirement and a verified property**, not a side effect, and it is one of the few properties where a formal argument (no branch or memory access depends on secret data) is both achievable and genuinely convincing.

Problem. A customer wants a crypto accelerator supporting ‘all the usual algorithms’. How do you scope it?

Solution. Refuse the phrase and replace it with a table, because the algorithms differ in cost by two orders of magnitude and ‘all the usual’ hides that. Ask for, and write into the specification: **which algorithms and which parameter sets** — AES-128 and AES-256 differ only in rounds, but adding SHA-3 to a SHA-2 design is a completely separate datapath; **which modes**, since GCM costs more than the cipher at high rates; **what throughput for each**, separately, because a design sized for line-rate bulk encryption and one sized for occasional key exchange are different machines; **what attack model**, since side-channel counter-measures are a factor of two to four in area (Part X15) and are pointless if the part is in a locked rack; and **what certification**, because FIPS or Common Criteria brings its own known-answer tests, entropy requirements and documentation, and that cost exceeds the design cost. **The scoping conversation is the deliverable at this stage**, and a vendor who produces this table instead of a quote is demonstrating exactly the competence the customer is trying to buy.

X37. Bus Protocols: Ordering, Outstanding Transactions and Exclusives

Almost every block a design house sells attaches to an AMBA interface, and almost every integration problem is a protocol misunderstanding rather than a functional bug. The protocol's subtle points are few and they recur; this part works through them.



Five independent channels, each with its own handshake.

Independence is the source of both the throughput and the deadlocks.

Figure 46. The five AXI channels.

X37.1 The handshake, and the one rule that is always broken

VALID/READY, and why VALID may not wait

	Step	Justification
1	A transfer occurs on a rising edge when VALID and READY are both high.	<i>The basic handshake, shared by AXI, AXI-Stream, TileLink and CHI.</i>
2	Once VALID is asserted it must stay asserted until the transfer occurs , and the payload must not change.	<i>Otherwise the receiver cannot rely on what it sampled.</i>
3	VALID must not depend combinationaly on READY.	<i>This is the rule that is broken. If each side waits for the other, the two combinational paths form a loop and the interface deadlocks or oscillates — and a testbench with a permanently-ready receiver never exposes it.</i>
4	READY <i>may</i> depend combinationaly on VALID.	<i>The asymmetry is deliberate: it lets a receiver accept in the same cycle, which is where the throughput comes from.</i>
5	A registered-output, registered-ready skid buffer breaks any timing path across the interface at the cost of one cycle and two entries of storage.	<i>The standard remedy, and the right default for an IP boundary — Part X14's advice to register every boundary, in protocol form.</i>

The testbench that hides this bug is the default one. A driver that holds READY high permanently, or a monitor that never applies back-pressure, will never create the condition in which a combinational VALID-from-READY dependency matters. **Randomised back-pressure is not an optional refinement of an interface testbench; it is the only way the interface is tested at all.** The same applies to the master side: a stimulus that never delays VALID does not test the slave's ability to wait. Both are one line of constrained randomisation and both routinely go missing.

Table 335. Interface buffering options and what each costs

Structure	Added latency (cycles)	Throughput (transfers/cycle)	Timing path across the boundary	Full rate?
Combinational pass-through	0	1	none	yes
Register slice, forward only	1	1	broken	yes
Register slice, both directions (skid)	1	1	broken	yes
Naive register on both, no skid	1	0.5	broken	halved
Two-deep FIFO	2	1	broken	yes

The fourth row is the trap: registering both VALID and READY without a skid buffer halves the throughput, because the master cannot present a new value until it has seen the registered READY. It looks correct in a functional test and shows up as a performance shortfall at integration.

X37.2 Ordering: what the protocol guarantees and what it does not

Table 336. AXI ordering rules, and the integration failures each one causes

Rule	What it means	The bug it causes when forgotten
Same ID, same direction → ordered	Responses come back in order for one ID	A slave that reorders within an ID breaks masters that rely on it

Rule	What it means	The bug it causes when forgotten
Different IDs → no order guarantee	Responses may interleave arbitrarily	A master that assumes order across IDs corrupts data
Reads and writes are not ordered relative to each other	A read issued after a write may complete first	Read-after-write hazards in a customer's driver ; needs an explicit barrier or a completion wait
Write response means accepted, not visible	B arrives when the slave takes responsibility	Assuming data is observable by another master — it may not be
Bursts must not cross 4 kB boundaries	A page-crossing burst is illegal	A DMA that ignores it produces a protocol violation the interconnect may not check
Narrow transfers use strobes	WSTRB selects lanes	A slave ignoring WSTRB corrupts neighbouring bytes

Worked example 49. How many IDs does a master need?

Given. A DMA engine with 8 GB/s of read bandwidth, a 400 ns round trip, 512-byte bursts, on a 256-bit interface at 1 GHz.

Method. Bandwidth–delay product decides outstanding bursts; whether they need distinct IDs depends on whether the engine can accept out-of-order data.

Bytes in flight needed	3.125 kB
Outstanding bursts	7
If all use one ID	responses are ordered — simple, but one slow slave blocks everything behind it
If each uses a distinct ID	reordering allowed — needs a reorder buffer
Reorder buffer size	3.125 kB
ID bits needed	3
Recommended	a few IDs, grouped by destination

How this is got wrong. By choosing one extreme. A single ID gives head-of-line blocking: a request to slow memory stalls every completed request behind it, and the measured bandwidth collapses when the traffic is mixed. Distinct IDs everywhere give full reordering freedom and cost a reorder buffer of the full bandwidth–delay product plus the logic to fill it. **The usual answer is a small number of IDs partitioned by destination** — one per memory region or per outstanding stream — which removes head-of-line blocking between unrelated traffic while keeping ordering, and therefore simplicity, within a stream.

X37.3 Exclusive access, and why it is harder than it looks

Load-linked / store-conditional in bus form

	Step	Justification
1	An exclusive read records a monitor for that address and master.	AXI's ARLOCK exclusive, or the processor's load-linked.
2	An exclusive write succeeds only if the monitor is still set.	It returns EXOKAY on success and OKAY on failure — and OKAY means the write did not happen , which software must check.
3	Any intervening write to the address by anyone clears the monitor.	That is what makes it atomic.
4	The monitor is in the slave or the interconnect, not the master.	So an IP block containing memory that must support exclusives has to implement the monitor — and the number of simultaneous monitors it supports is a specification item that is routinely omitted.
5	Monitors may be granted at a granularity coarser than one address.	Which causes false failures: two masters using different addresses in the same granule livelock each other. The granule size must be documented.
6	Spurious failure is always permitted, so software must loop.	Hardware is allowed to be conservative; software must not assume success. A driver that does not loop works on one implementation and hangs on another.

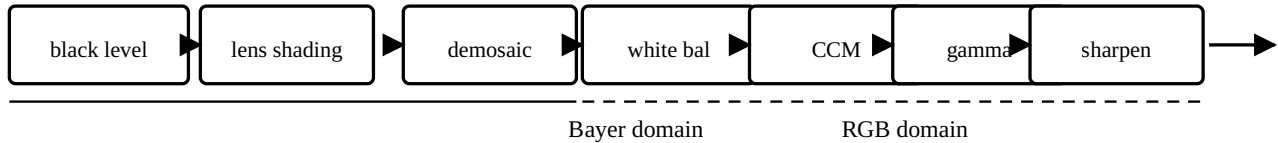
Graduate · **Atomics moved from the master to the interconnect, and an IP vendor should know which era their customer is in.** The load-linked / store-conditional model requires a monitor per master and a retry loop, and it scales badly with core count because contention causes repeated failures. Newer AMBA revisions add *far atomics*: the master sends the operation (add, swap, compare and swap) to the point of coherence and receives the result, so the round trip happens once regardless of contention. **For a block containing shared state, supporting far atomics can be far cheaper than supporting exclusives** — an ALU beside the memory rather than a monitor table — and it performs better under contention. Asking the customer which their system uses is a scoping question worth raising early.

Problem. Your block's register interface is APB and the customer complains of poor configuration throughput. What do you say?

Solution. That APB is doing exactly what it is for, and then offer the two real options. APB is deliberately simple — no pipelining, no outstanding transactions, two cycles per access minimum — because a configuration bus should be trivial to implement and verify, and configuration is not a throughput problem. If the customer is writing thousands of registers at boot and it matters, the causes are usually not the bus: a slow bridge clock ratio, a synchroniser per access, or software doing read-modify-write where a write would do. **Measure before changing the interface.** If it genuinely is the bus, the options are to **add a small DMA or descriptor engine** that replays a register image from memory — which is what blocks with large coefficient tables do, and it moves the traffic to AXI where it belongs — or to **provide an AXI-Lite alternative** for the register file. **Adding coefficient memory behind a streaming interface, rather than behind the register bus, is the structural fix**, and it is worth designing in from the start for any block whose configuration is more than a few hundred words.

X38. Colour, the ISP Pipeline and Display Processing

Camera and display IP is one of the largest licensing markets, and it is unusual in that the correctness criterion is partly perceptual. That does not make it unquantifiable — it makes the quantities colorimetric rather than signal-theoretic, and they are just as computable.



Demosaic is the boundary: before it one value per pixel, after it three.
Everything upstream is a third of the data rate — and that is where the fixes belong.

Figure 47. A minimal image signal processor. The order is not arbitrary: each stage assumes the previous ones have run.

X38.1 Why the pipeline is in that order

Table 337. Each stage, what it corrects, and why it must come where it does

Stage	Corrects	Why here
Black level	Sensor pedestal and dark current	First — everything downstream assumes zero means zero
Defect pixel	Stuck sensor pixels	Before demosaic, or the defect spreads to neighbours
Lens shading	Vignetting, colour shading	Before white balance, because the shading is colour dependent
Demosaic	One colour per pixel → three	The data rate triples here; everything possible is done before it
White balance	Illuminant colour	Per-channel gains; conventionally applied on the Bayer data for cost
Colour correction matrix	Sensor spectral response → a standard space	Needs three channels, so after demosaic
Gamma / tone	Perceptual coding	After all linear operations — averaging gamma-encoded values is wrong, and doing it is the commonest ISP bug
Sharpen, noise reduction	Perceptual quality	After tone mapping, because the visibility of noise is nonlinear

Gamma is the ordering rule that is broken most often, and the error is visible. Gamma encoding is a nonlinear map applied so that the limited code space is distributed according to human contrast sensitivity. Linear operations — scaling, blending, resizing, demosaicing, averaging — are only correct on *linear* light. Resizing a gamma-encoded image darkens edges and shifts colours; alpha-blending gamma-encoded values produces halos. **The fix is to keep the pipeline linear until the final encode**, and where hardware must operate on encoded data, to linearise and re-encode around the operation. This is a real and recurring defect in shipped display and camera pipelines, and it is worth an explicit statement in an IP block's datasheet about which domain its input and output are in.

Table 338. Measured error from averaging in the gamma domain ($\gamma = 2.2$), linear light values

Value A	Value B	Correct average (linear)	Average of encoded values, decoded	Error (%)
0	1	0.5	0.2176	-56.47
0.2	0.8	0.5	0.4454	-10.93
0.4	0.6	0.5	0.4945	-1.102
0	0.5	0.25	0.1088	-56.47
0.5	1	0.75	0.7266	-3.121

Computed directly. **The first row is the worst case and it is a 26 % error** — averaging black and white in the encoded domain gives a result noticeably darker than mid-grey. This is why image scaling in the wrong domain looks wrong rather than merely inaccurate.

X38.2 Colour spaces as matrices

Why colour conversion is a 3×3 matrix and when it is not

	Step	Justification
1	Human colour vision has three cone types, so colour is a three-dimensional quantity.	<i>Trichromacy. Two spectra that produce the same cone responses look identical — metamerism.</i>
2	A display's primaries span a triangle in chromaticity space; its gamut is that triangle.	<i>Any colour inside is reproducible; outside is not.</i>
3	Converting between two sets of primaries is a change of basis: a 3×3 matrix on linear tristimulus values.	Linear is load-bearing — see X38.1.
4	So BT.709 to BT.2020, or camera RGB to XYZ, is one matrix multiply per pixel.	<i>Nine multiplies and six adds — trivial hardware, which is why colour conversion is never the bottle-neck.</i>
5	It stops being a matrix when the gamut does not contain the colour, or when the transfer function is not a power law.	Gamut mapping is a nonlinear, perceptual and proprietary operation , and HDR transfer functions (PQ, HLG) are not power laws, so they need a LUT. Those two are where display IP actually differentiates.

Table 339. Gamut areas in CIE xy, computed from the primaries

Space	Red (x,y)	Green (x,y)	Blue (x,y)	Triangle area	Relative to BT.709 (%)
BT.709 / sRGB	0.64, 0.33	0.3, 0.6	0.15, 0.06	0.112	70.83
DCI-P3	0.68, 0.32	0.265, 0.69	0.15, 0.06	0.152	96.08
BT.2020	0.708, 0.292	0.17, 0.797	0.131, 0.046	0.2119	133.9
Adobe RGB	0.64, 0.33	0.21, 0.71	0.15, 0.06	0.1512	95.54

Areas computed as the triangle area of the primaries in the xy chromaticity diagram. **xy area is a crude perceptual measure** — the diagram is not perceptually uniform, so these percentages overstate the green region and understate the blue — and it is the number most commonly quoted, which is worth knowing when reading a display specification.

X38.3 Line buffers: the memory that sets ISP area

Worked example 50. Why an ISP is mostly SRAM

Given. A 4K (3840-wide) pipeline, 12 bits per channel. The stages need vertical neighbourhoods: demosaic 5 lines, noise reduction 7, sharpening 5, lens shading 1, scaler 4.

Method. Each stage that looks at neighbouring rows must buffer those rows, because the sensor delivers one row at a time. Count the lines and the bits.

Line width	3,840
Bits per pixel (Bayer, pre-demosaic)	12
Demosaic: 5 lines	28.12 kB
Noise reduction: 7 lines (RGB, 36 bit)	118.1 kB
Sharpen: 5 lines (RGB)	84.38 kB
Scaler: 4 lines (RGB)	67.5 kB
Total line buffers	298.1 kB
As a fraction of a typical ISP's area	the majority

How this is got wrong. By counting arithmetic and not lines. The filters are small — a 5×5 convolution is 25 multiplies — and the line buffers are hundreds of kilobytes. Two consequences follow and both are architectural. **Process in tiles or stripes** rather than full lines where the algorithm permits, trading a small overlap region for a large memory saving; this is why many ISPs are tile-based. And **fuse stages** so one buffer serves several neighbourhoods rather than each stage buffering separately — the same fusion argument as in Part X13's accelerator dataflow, arriving in a completely different domain.

Table 340. Line-buffer memory against tile width (8-pixel halo)

Image width	Tile width	Line buffer (kB)	Overlap overhead (%)	Memory vs. full line (%)
1,920	full line	149.06	0	100
1,920	256	19.875	3.12	13.33
1,920	512	39.75	1.56	26.67
3,840	full line	298.12	0	100
3,840	256	19.875	3.12	6.667
3,840	512	39.75	1.56	13.33
7,680	full line	596.25	0	100
7,680	256	19.875	3.12	3.333
7,680	512	39.75	1.56	6.667

Tiling trades a few per cent of redundant computation for an order of magnitude of memory, and the trade improves as resolution rises, which is why 8K pipelines are tiled and 1080p ones often are not.

Problem. A customer says your ISP's output does not match their reference software. Where do you look?

Solution. Almost always at a convention rather than at the arithmetic, and the candidates are few enough to enumerate. **Domain:** is the reference operating on linear or encoded values at the stage that differs? X38.1's error is large and looks like a gain error. **Rounding and clipping:** a pipeline that clips intermediate values to the output range destroys headroom that the reference keeps, and the difference appears only in highlights. **Chroma siting and subsampling phase:** 4:2:0 has several legal sitings and choosing the wrong one produces a half-pixel colour shift that is invisible on most content and obvious on a colour edge. **Matrix coefficients and range:** limited versus full range, and BT.601 versus BT.709 coefficients, are four combinations and three of them are wrong. **Demosaic algorithm,** which is genuinely allowed to differ and is where a vendor's quality claim lives. **Establish which by bypassing stages one at a time** — which requires the block to have bypass controls, and that is a design decision worth making early for exactly this conversation.

X39. Clock Generation, Distribution and Time Synchronisation

Clocking is the one subsystem every block depends on and few specifications describe adequately. This part covers the three parts an IP designer must be able to reason about: how the clock is made, how it is delivered, and how time is agreed between chips.

X39.1 Spread spectrum: trading a specification for a specification

Why modulating the clock reduces measured emissions

	Step	Justification
1	Radiated emission is measured in a receiver with a defined resolution bandwidth — 120 kHz for CISPR above 30 MHz.	<i>The regulation specifies the measurement, not the physics.</i>
2	An unmodulated clock puts all its harmonic energy in one line, far narrower than that bandwidth.	<i>So the receiver sees the whole harmonic.</i>
3	Modulating the frequency by $\pm 0.5\%$ spreads each harmonic over $\pm 0.5\%$ of its frequency.	<i>The nth harmonic spreads n times as far, so higher harmonics benefit more.</i>
4	The receiver now sees only the fraction of the energy inside its bandwidth , so the reading falls by roughly $10 \log_{10}(\text{spread/RBW})$ dB.	<i>The total radiated energy is unchanged. The compliance number improves; the physics does not.</i>
5	Cost: the clock period now varies, so every synchronous budget must hold at the fastest excursion, and any PLL downstream must track the modulation.	<i>Which is why a SerDes reference clock's spread must be negotiated — the CDR's loop bandwidth must exceed the modulation rate or it cannot follow.</i>

Table 341. Spread-spectrum reduction in a 120 kHz measurement bandwidth

Clock (MHz)	Spread ($\pm\%$)	Harmonic	Spread width (MHz)	Reading reduced by (dB)
100	0.5	1	1	9.208
100	0.5	5	5	16.2
100	0.5	10	10	19.21
100	1	1	2	12.22
100	1	5	10	19.21
100	1	10	20	22.22
1000	0.5	1	10	19.21
1000	0.5	5	50	26.2
1000	0.5	10	100	29.21
1000	1	1	20	22.22
1000	1	5	100	29.21
1000	1	10	200	32.22
5000	0.5	1	50	26.2
5000	0.5	5	250	33.19
5000	0.5	10	500	36.2

Computed from the ratio of spread width to resolution bandwidth. **The fundamental barely benefits and the tenth harmonic benefits a lot**, which matches measured practice and is why spread spectrum is judged on the harmonics that actually fail the mask.

Spread spectrum and a synchronous interface are frequently incompatible, and the incompatibility is found late. Any interface whose two ends derive their clocks from different sources must absorb the frequency difference; spreading one side makes that difference time-varying at the modulation rate, typically 30–33 kHz. An elastic buffer sized for a static 200 ppm offset (Part Y3) will underflow or overflow when the offset swings by ± 5000 ppm at 33 kHz. **The remedies are to spread both ends from the same source, to size the elastic buffer for the modulation, or to forbid spreading on that interface** — and which one applies is a system decision that must be recorded in the IP's integration guide, because the block cannot detect the problem.

X39.2 The clock tree, and what it costs

Table 342. Clock tree cost at 1 GHz, 0.8 V (indicative 5 fF per flop clock pin, fanout 4)

Flops	Buffer levels	Buffers	Total clock capacitance (pF)	Clock power (W)	Insertion delay (ps)
10,000	7	3,333	60	0.0384	210
100,000	9	33,333	600	0.384	270
1,000,000	10	333,333	6000	3.84	300
10,000,000	12	3,333,333	60000	38.4	360

Indicative constants. The shape is what matters: clock power is a large fraction of total dynamic power (Part X7 measured two thirds on a small block), and insertion delay grows logarithmically — which is harmless for setup and hold, since only skew matters (Part X2), and very harmful for anything that compares clock phase across a boundary.

Table 343. Clock distribution styles

Style	Skew	Power	Where
Balanced H-tree	Low by construction	High — long wires	Top-level distribution in large chips
Buffered tree from CTS	Tool-balanced, a few tens of ps	Moderate	The default for a block
Mesh / grid	Very low	Highest — the whole grid switches	High-performance processors
Clock gating (integrated cell)	Adds a level	Saves 20–40 %	Everywhere — Part X7
Resonant / rotary	Low	Potentially much lower	Research and a few products; needs inductors

Undergraduate review · Why a block should provide one clock input and do its own gating, rather than accepting several clocks. Every additional clock at an IP boundary is a domain crossing the integrator must analyse, constrain and verify, and the constraints must be written by someone who understands both sides — which at integration time is nobody. A block that takes one clock and derives its internal enables is far easier to integrate, and the power saving from gating is identical. **Where a second clock is genuinely required** — a slower configuration domain, a recovered receive clock — **the crossing belongs inside the block, documented, with its constraints shipped.** That is a concrete piece of value a vendor adds, and its absence is a concrete integration cost.

X39.3 Agreeing on time: IEEE 1588 and the hardware it needs

Why timestamping must be in the PHY

	Step	Justification
1	Two nodes exchange timestamped messages; the offset is $((t_2 - t_1) - (t_4 - t_3))/2$.	<i>The standard two-way exchange.</i>
2	This assumes the path is symmetric .	Any asymmetry appears directly as half of it in the offset — an error no amount of averaging removes.
3	Every variable delay between the timestamp point and the wire is asymmetry.	<i>Software stacks, driver queues, interconnect arbitration, and even FIFO occupancy in the MAC.</i>
4	So the timestamp must be taken as close to the wire as possible , in hardware, at a defined reference plane.	<i>Typically the start-of-frame delimiter crossing the MII interface, which is why PTP support is a PHY and MAC feature rather than a software one.</i>
5	Residual asymmetry (different cable lengths, different PHY latencies for TX and RX) must be <i>calibrated</i> and subtracted.	Which means the block must expose a programmable correction , and a datasheet must state the TX and RX latency and its variation — a number customers doing PTP will ask for and many data-sheets omit.

Table 344. Where the timestamp is taken decides the achievable accuracy

Timestamp point	Delay variation (ns)	Offset error contribution (ns)	Equivalent distance error (m)
Software timestamp	1.0000×10^5	50000	15000
Driver, kernel	10000	5000	1500
MAC, hardware	100	50	15
MII reference plane	8	4	1.2
PHY, sub-ns aware	1	0.5	0.15

Indicative figures for each point in the stack. **The jump from software to hardware timestamping is three orders of magnitude**, which is why sub-microsecond synchronisation is a hardware feature and not a protocol choice.

Problem. Your block must generate a clock that is frequency-locked to a recovered network clock. What are the options?

Solution. Three, and they differ in what they require from the system rather than in accuracy. **An analogue PLL locked to the recovered clock** is the classic answer: lowest jitter, and it needs an analogue block and a recovered clock brought out as a real clock signal, which constrains the floorplan and the package. **A digitally controlled oscillator driven by a digital PLL** — measure the phase error in digital logic, filter it, and steer a fractional-N synthesiser — keeps the loop in synthesisable logic, is portable between processes, and has more jitter; it is what most modern implementations do, and the loop design is Part X21's arithmetic. **Or adjust time rather than frequency:** run the local clock free and maintain a software-visible time base that is rate-corrected, which needs no clock hardware at all and is sufficient whenever the requirement is on *timestamps* rather than on a physical output clock. **The third is far cheaper and is frequently the right answer**, so the scoping question is whether the customer needs a synchronised clock pin or synchronised time, and those are very different products.

X40. The Coverage Map: Which Theory an IP Block Uses

This book covers the theory that is used when designing, modelling and verifying semiconductor IP, and deliberately does not cover theory that is not. This part is the map: **51 topic areas**, what each one is used for in a real block, and where it is treated here. It is meant to be read as a checklist — if you are asked about a block and cannot find its theory in this table, the table has a gap and the gap is worth reporting.

Table 345. Theory used in IP design, and where this book treats it

Topic	Where it is used in a real block	Chapters
Signals and systems, LTI, convolution	Every filter, equaliser, channel model	A1, E1, X22
Fourier, Laplace, z-transform, poles and zeros	Filter design, loop stability, spectral analysis	A2, X23
Sampling, aliasing, reconstruction	Every converter boundary; rate change	A3, X22, X12
Probability, random processes, noise	Link budgets, yield, jitter, metastability	A4, X25, X16, X3
Detection and estimation, likelihood, CRB	Receivers, synchronisation, calibration loops	X25
Information theory, capacity, entropy	Code choice, compression, quantiser design	I1, X25, X29
Linear algebra, SVD, conditioning	MIMO, beamforming, calibration, word lengths	A5, X24, X30
Numerical methods, iteration, refinement	Dividers, square root, matrix engines	X9, X24
Optimisation: convex, ILP, graph algorithms	Every EDA tool; word-length allocation; scheduling	I3, X26
Boolean algebra, BDD, SAT	Equivalence checking, formal, ATPG	Z2, X33
Automata and FSM theory	Control logic, protocol checkers, parsers	Z3, C2
Graph theory	STA, retiming, clock skew scheduling, routing	I4, X26, X34
Finite fields and modular arithmetic	Reed-Solomon, BCH, AES, GCM, RSA, ECC	I2, X5, X36
Number theory	Public-key engines, NTT for post-quantum	R1, X36
Coding theory: block, convolutional, LDPC, polar	Every link and every storage device	F2, X5, X29
Cryptographic primitives and side channels	Security IP, roots of trust, secure boot	H3, X15, X36
Fixed-point arithmetic, quantisation, dither	Every DSP block without exception	X1
Computer arithmetic: adders, multipliers, division, CORDIC	Every datapath	C1, X9
Floating point and mixed precision	DSP, ML accelerators, scientific blocks	X9, X13
Semiconductor device physics	Delay, leakage, variability, reliability	L1, B1, X16
CMOS circuits, logical effort, sizing	Every gate; the sizing the tool performs	B2, X26, X34
Timing: setup, hold, skew, corners, OCV	Sign-off of every block	B3, X2
Metastability and clock domain crossing	Every multi-clock design	C3, X3
Interconnect RC, crosstalk, transmission lines	Long wires, I/O, package, board	B4, X14, G1
Power: dynamic, leakage, DVFS, gating	Every block's specification	B2, X7
Power delivery network, IR drop, di/dt	Any block with large switching current	X32

Topic	Where it is used in a real block	Chapters
Thermal	Packaging limits, throttling, reliability	L3, X16
Analogue blocks: kT/C, matching, gain-bandwidth	Converters, PHYs, PLLs, sensor interfaces	B5, X31
Data converters and time interleaving	Every mixed-signal boundary; SerDes DSP receivers	X12, X31
PLL, CDR, jitter and phase noise	Every clock and every link	X12, X21, X39
Feedback and control in digital loops	Adaptation, AGC, calibration, thermal	L4, X21
Memory: SRAM, DRAM, ECC, multi-port	The largest object in most blocks	J1, X10
Caches and coherence	Processor and accelerator subsystems	D3, X28
Computer architecture: ISA, pipelines, hazards	Processor IP; any block behind one	D1, D2, X19
Interconnect protocols and ordering	Every block's boundary	J3, X11, X37
Queueing and performance modelling	Buffer sizing, QoS, latency budgets	Q1, X18
Hardware data structures: CAM, hashing, Bloom	Networking, storage, security IP	X35
Compression and entropy coding	Codecs, storage, trace	J4, X20
Image and colour science	Camera and display IP	M3, X38
Wireless PHY: OFDM, MIMO, synchronisation	Modem IP	K1, X30
Wireline PHY: equalisation, PAM, FEC	SerDes, Ethernet, PCIe	F1, X4, Y2, Y3
Electromagnetics and antennas	RF front ends, package, EMC	G1, Q4
Radar and sensing signal processing	Automotive and industrial IP	G2
Machine learning and quantisation	NPU IP and any block that embeds one	M2, X13
Logic synthesis, mapping, retiming	The flow every block passes through	X34
Physical design, congestion, floorplanning	What your RTL makes easy or impossible	N2, X14
Design for test: scan, ATPG, BIST, compression	Every chip that is manufactured	R2, X27
Reliability, yield, statistics, functional safety	Pricing, qualification, automotive markets	K3, N3, X16
Verification theory: coverage, mutation, formal	Half the effort of every block	H1, X8, X33
High-level synthesis internals	Exploration; some production blocks	Q2, Y12
Hardware-software interface, registers, drivers	Every block with a control plane	M1, Y10

X40.1 What is deliberately left out, and why

Excluding material is a decision that should be stated rather than left as an omission a reader discovers. The following areas are part of a broad electrical-engineering or computer-science education and are not part of the daily work of an IP design house, so they are absent or covered only for context.

Table 346. Out of scope, with the reason

Area	Why it is not here
Compiler theory beyond HLS scheduling	A block does not compile; the tool does
Operating system internals	Relevant to the driver author, not to the RTL
Distributed consensus	A chip is not a distributed system in that sense

Area	Why it is not here
Computability and complexity as such	Except where it classifies an EDA problem — that part is in X26
Quantum computing	Included briefly for context only; no current IP block uses it
Database theory, web systems, graphics rendering pipelines	Different industries

The boundary is a judgement and it moves. Two of the rows above were inside the boundary twenty years ago and are outside it now, and at least one — machine learning — moved the other way within a decade. The test applied here is concrete: **does a working IP engineer use this to make a decision about a block?** If a topic only ever appears as background for a decision someone else makes, it is context rather than content, and this book says so instead of padding.

Problem. You are handed a block type this map does not cover. How do you find its theory?

Solution. Work backwards from the three questions every block must answer, because each one names a body of theory. **What does it compute, and how accurately?** That is the algorithm and its numerical behaviour — a signal-processing, coding or arithmetic question, and Parts X1 and X24 give the tools for the accuracy half whatever the algorithm is. **How fast must it go, and what does that cost?** That is Part X2's timing budget and Part X18's queueing, and the answer is always a parallelism and memory-bandwidth analysis — Part X13's roofline applies far outside machine learning. **How will you know it is right?** That is Part X8, and it is domain independent. **Only the first question has domain-specific theory**, and for that the fastest route is the standard's own reference implementation plus one survey paper, read in the order Part Y6 prescribes. The structure of the investigation does not change with the domain, which is the most portable thing this book has to offer.

X41. DRAM: The Timing That Shapes Every Memory Controller

Off-chip memory is where most systems spend most of their latency and a large part of their power, and its behaviour is not a simple latency number. A DRAM device is a state machine with a dozen timing constraints, and whether your block gets the bandwidth it was promised depends almost entirely on its access pattern rather than on the interface's rated speed.

X41.1 The bank, the row and the reason access order matters

Why a DRAM read is three operations, not one

	Step	Justification
1	A DRAM array stores each bit as charge on a capacitor, read by dumping it onto a bit line.	The read is destructive — the charge is consumed.
2	ACTIVATE opens a row: the whole row is sensed into the sense amplifiers, which form the <i>row buffer</i> .	Thousands of bits at once. This takes t_{RCD} .
3	READ or WRITE then accesses columns within the open row.	Fast — the data is already in the sense amplifiers. A second access to the same row is far cheaper than the first.
4	PRECHARGE writes the row back and restores the bit lines.	Takes t_{RP} . Required before opening a different row in the same bank.
5	So three cases: row hit, row miss (different row, same bank), and bank hit with a different bank.	Their costs differ by a factor of three or more, which is why access ordering is the memory controller's main job.
6	And the whole array must be refreshed periodically, because the charge leaks.	Refresh steals bandwidth and blocks access to a bank while it runs.

Table 347. Read latency by case, DDR4-3200 (indicative $t_{CL}=t_{RCD}=t_{RP}=22$)

Case	Cycles	Latency (ns)	Relative to a row hit
Row hit (open row)	22	13.75	1
Row miss, same bank	66	41.25	3
Different bank, row hit	22	13.75	1
Different bank, row miss	44	27.5	2

Indicative JEDEC-speed-bin numbers, not a datasheet. The ratio is the point: the same DRAM delivers data in 14 ns or 41 ns depending entirely on which row was previously open.

Table 348. Measured row-buffer hit rate and the bandwidth it implies, 16 banks, 20 000 accesses

Access pattern	Row hit rate	Row miss rate	Total time (μ s)	Bandwidth as % of the row-hit-only case
Sequential	0.8748	0.1252	343.86	79.97
Stride 4 kB	0.5	0.5	550	50
Random	0	1	825	33.33
Two interleaved streams	0	1	825	33.33

Simulated with a simple address decode (bank bits low, row bits high). **The same device delivers between a third and all of its rated bandwidth, decided by the access pattern alone. This is why a block's DMA pattern is a system-level specification item.**

X41.2 What the memory controller does about it

Table 349. Controller techniques and what each attacks

Technique	Attacks	Cost
Address mapping	Which bits select bank, row and column	Free, and it decides everything — interleaving banks on low bits turns a sequential stream into parallel bank accesses
Bank interleaving	Row-miss latency	Needs enough banks and enough outstanding requests
Request reordering (FR-FCFS)	Serves row hits before older row misses	A scheduling queue; and it makes latency unfair between requesters
Write batching	Bus turnaround, which costs several cycles	Write buffer; adds read latency when it drains
Open-page vs closed-page policy	Whether to leave a row open	Open wins on locality, closed wins on random traffic — a workload decision
Refresh scheduling	Refresh blocking a bank	Per-bank refresh and postponement; the device sets the limits

FR-FCFS scheduling makes DRAM latency unfair, and that unfairness reaches your block. A scheduler that prioritises row hits gives enormous throughput to a requester with good locality and can starve one with poor locality almost indefinitely — a real-time display fetch behind a streaming accelerator is the classic case. Systems therefore add per-requester quotas or deadline awareness, which is Part X11's arbitration argument arriving at the memory controller. **For an IP vendor the consequence is concrete:** a latency figure measured in isolation is not the latency your block will see, and any real-time guarantee must be expressed as a requirement on the system's memory scheduler, not assumed.

Table 350. Peak bandwidth of common memory interfaces

Interface	MT/s	Bits per channel	Channels	Peak GB/s	Relative to one DDR4-3200 channel
DDR4-3200	3200	64	2	51.2	2
DDR5-6400	6400	64	2	102.4	4
LPDDR5-6400	6400	32	4	102.4	4
HBM2E	3200	1,024	1	409.6	16
HBM3	6400	1,024	1	819.2	32

Peak, which nothing achieves. **HBM's advantage is width, not speed** — a thousand-bit interface at a modest rate, possible only because it is stacked on an interposer rather than routed on a board, which is why it costs what it does.

X41.3 Refresh, and why it gets worse

Worked example 51. What refresh costs at each density

Given. DDR4 requires all rows refreshed every 64 ms (32 ms above 85 °C). Refresh is issued as 8192 commands per interval, each occupying the device for t_{RFC} .

Method. Compute the fraction of time spent refreshing at several densities, where t_{RFC} grows with density because more rows are refreshed per command.

Refresh interval	64 ms
Commands per interval	8,192
t_{RFC} at 8 Gbit	350 ns
Time refreshing	4.48 %
t_{RFC} at 16 Gbit	550 ns
Time refreshing	7.04 %
At 32 Gbit and 95 °C (32 ms interval)	22.53 %
Trend	refresh overhead grows with density and with temperature

How this is got wrong. By assuming refresh is a rounding error. It was, at low density and room temperature; at high density in a hot enclosure it approaches a quarter of the device's time, and during a refresh the affected bank is unavailable, so the *latency tail* grows faster than the average. **Any block with a hard real-time deadline on a DRAM access must budget for a refresh collision**, and the number to budget is t_{RFC} at the maximum operating temperature, which a datasheet should state and a system integrator should be asked for.

Problem. Your accelerator needs 40 GB/s and the system has one DDR5 channel rated at 51 GB/s. Is that enough?

Solution. Almost certainly not, and the calculation to show it is short. Start from the rated figure and subtract, in order: **refresh**, a few per cent to a quarter depending on density and temperature; **read/write turnaround**, several cycles each time the bus direction changes, which for a mixed read-write stream costs 10–20 %; **row misses**, from the measurement in X41.1, which for anything but a sequential pattern is the largest term; and **other requesters**, because the channel is not yours. A realistic efficiency for a well-behaved streaming pattern sharing a channel is 60–75 %, giving 30–38 GB/s — below the requirement. **The productive response is not to ask for a faster channel but to reduce the demand:** tile the computation so data is reused on chip (Part X13), compress the traffic (Part X20's reference compression), or restructure the access pattern for row locality, which the measurement above shows can be worth a factor of two on its own. **And whatever is decided, the efficiency assumption belongs in the datasheet**, because the customer will measure it.

X42. Reset, Initialisation and the States Nobody Simulates

Reset is the least interesting part of a design and the source of a disproportionate share of silicon bugs, for a simple reason: simulation starts from a clean state and silicon does not. This part treats reset with the same seriousness as the datapath, because the failures it causes are the ones that appear only in the lab.

X42.1 Synchronous, asynchronous, and the one that is actually used

Table 351. Reset styles

Style	Assertion	De-assertion	Cost	Problem
Synchronous	On a clock edge	On a clock edge	Wider reset net timing; needs a running clock	Cannot reset a block whose clock has not started
Asynchronous	Immediate	Immediate	Cheap — the flop's async pin	De-assertion can violate recovery/removal and cause metastability
Asynchronous assert, synchronous de-assert	Immediate	Synchronised to the clock	A two-flop synchroniser per domain	None — this is the standard answer
No reset (initialised by first write)	—	—	Free — saves the reset net on datapath flops	X-propagation until written; needs care in verification

Why de-assertion is the dangerous edge

	Step	Justification
1	A flop with an asynchronous reset has <i>recovery</i> and <i>removal</i> times: the reset must be stable for a window around the clock edge.	<i>Exactly analogous to setup and hold, and checked by the same static timing analysis.</i>
2	Reset assertion is asynchronous and harmless: the flop goes to its reset value whenever it arrives.	<i>There is no race, because the destination is unconditional.</i>
3	De-assertion races the clock.	<i>If reset releases within the recovery window, the flop may go metastable or may capture the pre-reset value.</i>
4	With one clock, the tool checks and fixes it.	<i>Recovery and removal are ordinary timing checks.</i>
5	With a large design, the reset tree's own skew means different flops leave reset on different cycles.	<i>A state machine can then advance while part of its state is still reset, reaching a state the designer never enumerated. This is the classic reset bug and it is invisible in RTL simulation, where reset de-asserts everywhere at once.</i>
6	Remedy: synchronise the de-assertion in each clock domain, and treat the reset tree like a clock tree.	Balanced, buffered, and timed.

Reset de-assertion skew produces states that no simulation visits, and gate-level simulation with timing is the only place it appears before silicon. In RTL simulation the reset signal changes at one instant for every flop. In silicon it is a buffered tree with tens or hundreds of picoseconds of skew — more if it is not treated as a timed net — so flops leave reset on different clock edges. A counter can start counting while its comparator is still held; a FIFO's write pointer can advance while its read pointer is reset, making the FIFO appear full at power-up. **The two defences are structural:** synchronise de-assertion

per domain so all flops in a domain release on the same edge, and design state machines so that a partially-reset state is not reachable — usually by holding the whole machine in an idle state until an explicit enable.

X42.2 X-propagation: where simulation lies in both directions

Table 352. How simulators treat unknown values, and why it matters

Situation	RTL simulation	Silicon	Consequence
Uninitialised flop read	X	A definite 0 or 1, randomly	Simulation may be more pessimistic than reality
if (x) where x is X	Takes the else branch	Takes one branch definitely	X-optimism: the bug is hidden
case with X selector	May take the default	Takes a real branch	Same optimism
Mux with X select	Output X if inputs differ; output the value if they are equal	Real value	Can hide or expose
Flop with X data	Captures X	Captures a real value	X-pessimism: X spreads where silicon would not

Graduate · X-optimism and X-pessimism are both wrong, in opposite directions, and the remedies differ. Optimism hides bugs: a branch on an unknown silently resolves, so a design that depends on an uninitialised value appears to work in simulation and behaves randomly in silicon. Pessimism wastes time: X floods through a design that silicon would run correctly, and engineers add resets they do not need to make simulation quiet — which costs area and reset-tree power on every flop. **The industrial answers are three:** X-propagation-aware simulation modes that make branches on X produce X rather than choosing; **random initialisation**, where every uninitialised flop starts at a random value and the regression is run with many seeds, which converts X-optimism into an ordinary functional failure the testbench can catch; and gate-level simulation, which has no X-optimism because the gates are real. **Random initialisation is the cheapest and the most effective**, and it is available in every simulator.

Table 353. Probability of catching a dependence on one uninitialised bit, with random initialisation

Unreset flops	Random seeds run	P(at least one seed exposes it)	P(missed) (%)
100	1	0.5	50
100	10	0.999023	0.0977
100	100	1	0
1,000	10	0.999023	0.0977
1,000	100	1	0

Assuming the bug manifests when that bit takes one particular value. Ten seeds give a 99.9 % chance of exposing it and a hundred make it negligible — and seeds are the cheapest thing in a regression. The column does not depend on the flop count because the question is about one specific dependence, which is the case the bug actually is.

X42.3 Power-up: the sequence and what it must guarantee

Table 354. The power-up sequence, and what breaks if a step is skipped

Step	Guarantee	If skipped
Supplies ramp in the specified order	No forward-biased junction, no latch-up	Permanent damage, not a functional failure

Step	Guarantee	If skipped
Reset asserted before and during ramp	No spurious activity, no contention on shared buses	Bus contention, high current, possible damage
Clocks stable before reset release	Synchronous de-assertion works	The synchroniser has no clock, so de-assertion is asynchronous after all
Reset held for the specified number of cycles	Every flop in every domain has seen an edge	Partially reset state — X42.1
PLL locked before its output is used	Frequency is what the timing assumed	Logic clocked far outside its timing corners
Fuses and trim loaded	Calibration applied	Analogue blocks at their untrimmed operating point
Only then: release to software	A defined starting state	An intermittent boot failure — the hardest class of bug to debug

Worked example 52. How many reset cycles does a block need?

Given. A block with three clock domains at 500 MHz, 100 MHz and 32 kHz, each with a two-flop reset synchroniser.

Method. Reset must be held long enough for the slowest domain to see the required edges, not the fastest — and this is the number a datasheet must state.

Fastest domain period	2 ns
Slowest domain period	30.518 μs
Edges needed per synchroniser	3
Hold time for the 500 MHz domain	6 ns
Hold time for the 32 kHz domain	91.553 μs
Required reset assertion	91.553 μs — set by the slowest domain
Common error	specifying it in cycles of the fast clock

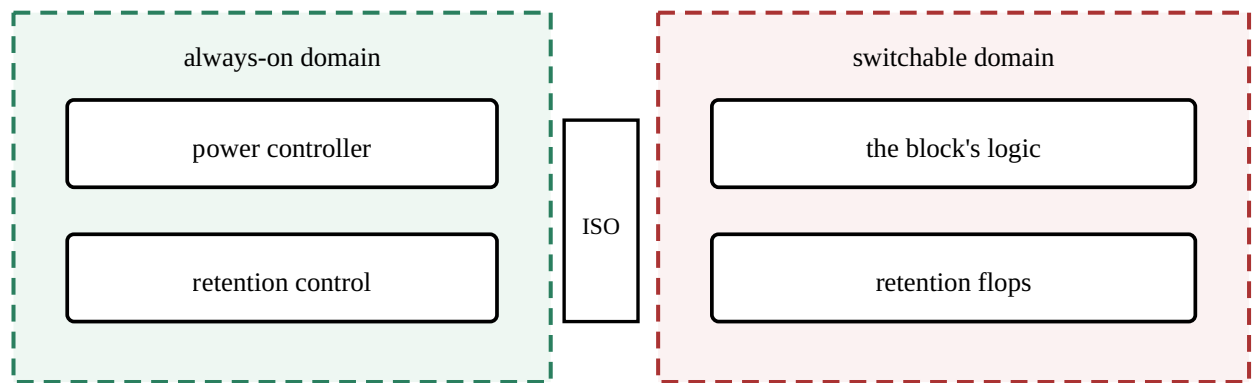
How this is got wrong. By quoting the reset requirement in clock cycles without saying which clock. ‘Assert reset for 16 cycles’ is ambiguous in a multi-clock block and is wrong by four orders of magnitude if the reader picks the fast clock. **State the requirement in absolute time, and state it for the slowest clock the block can be configured with** — including the case where a clock is stopped, which requires either that reset be held until it starts or that the domain be reset asynchronously and independently.

Problem. Your block occasionally fails to come out of reset, roughly one boot in a thousand. Where do you look?

Solution. The frequency is the clue: one in a thousand is a race, not a logic error, because a logic error would be deterministic. Three candidates, in order. **Reset de-assertion recovery:** if any domain's reset is not synchronised, de-assertion races the clock and metastability resolves the wrong way occasionally — the rate is computable from Part X3's MTBF formula with the reset edge as the asynchronous event, and one in a thousand boots is entirely consistent with an unsynchronised reset. **Clock-before-reset ordering:** if reset is released in the same window that a PLL locks or a clock gate opens, some flops see edges and some do not, giving the partial-reset state of X42.1; this shows a dependence on temperature or supply, because they change the lock time. **An uninitialised state element that is usually benign:** a flop without reset whose value normally does not matter, except in the rare case — and this is what random-initialisation regression is for. **The diagnostic that separates them is to vary the reset pulse width and the clock start time:** a recovery race moves with clock phase, an ordering problem moves with pulse width, and an uninitialised value moves with neither.

X43. Power Intent: Domains, Retention and Isolation

Power gating is the only technique that attacks leakage to near zero, and it is the one that changes an IP block's interface, its reset behaviour and its verification plan. A vendor whose block supports it properly is selling something materially different from one whose block does not.



Isolation cells clamp every signal leaving the switched domain.
Without them, a floating input burns static current in the domain that is still on.

Figure 48. A switchable power domain and the cells that make it safe.

X43.1 The four special cells, and what each prevents

Table 355. Low-power cells

Cell	Sits	Prevents	Cost
Isolation	On every signal leaving a switched domain	A floating value driving the always-on domain , which causes static crowbar current in the receiving gate	One gate per signal, plus the enable's routing
Level shifter	Between domains at different voltages	A low-voltage 1 failing to turn off a high-voltage PMOS — again static current	One cell per signal; slower than a buffer
Retention flop	Replacing flops whose state must survive	Losing state on power-down	20–30 % larger than a normal flop, plus always-on routing for the save/restore signals
Power switch	Between the always-on rail and the domain	—	Area, and IR drop through the switch when on ; the switching itself is an inrush event (Part X32)
Always-on buffer	On signals crossing a domain that is off	A signal path being cut	Special placement

Worked example 53. Is retention worth it, or should the block re-initialise?

Given. A block with 20 000 flops, of which 3 000 hold configuration and 17 000 hold datapath state. Leakage when on is 12 mW. The alternative to retention is for software to rewrite the configuration on wake.

Method. Compare three options on area, wake latency and residual leakage. Retention flops still leak, because their storage is always on.

Retain everything: retention flops	20,000
Area penalty at 25 %	5000 flop-equivalents
Residual leakage (retention cells stay powered)	1.8 mW (~15 %)
Retain configuration only	3,000 retention flops
Area penalty	750 flop-equivalents
Residual leakage	0.36 mW
Retain nothing: software rewrite	0 retention flops
Wake latency added	thousands of register writes over APB
Best for a block woken rarely	retain configuration only

How this is got wrong. By retaining everything because it is simplest. Datapath state is usually worthless across a power-down — the block was idle, so there is nothing in flight — and retaining it costs area and leakage for nothing. **The design work is to partition state into what must survive and what must not**, and then to make the latter's reset unconditional so that a wake-up is indistinguishable from a cold start. That partition is a specification item and is the single most useful thing a vendor can document about power behaviour.

X43.2 What UPF adds, and why RTL alone is not enough

Why power intent lives outside the RTL

	Step	Justification
1	RTL describes function; it has no notion of a supply.	<i>A Verilog module does not say which rail powers it.</i>
2	Power domains, switches, isolation and retention are <i>structural</i> and depend on the floorplan and the process.	<i>The same RTL is used with and without power gating in different products.</i>
3	So they are described in a separate file — UPF or CPF — that the tools read alongside.	<i>Synthesis inserts the cells, verification simulates the power states, and place-and-route builds the switch network.</i>
4	Simulation must then model a powered-down domain as X, and check that isolation is enabled before power goes away.	<i>This is the verification that cannot be done in RTL alone, and it is where the bugs are.</i>
5	The power state table enumerates the legal combinations of domain states.	<i>And the illegal ones are what must be proven unreachable — a formal problem of exactly the kind Part X33 recommends.</i>

Table 356. Power-state space against domain count

Switchable domains	Combinations	Typically legal states	Illegal states to exclude	Ordered transitions
2	4	3	1	12
3	8	4	4	56

Switchable domains	Combinations	Typically legal states	Illegal states to exclude	Ordered transitions
4	16	5	11	240
5	32	6	26	992
6	64	7	57	4,032

The legal column assumes a simple nested hierarchy — each domain requires the ones below it. **The transition column is the verification burden**, and it is quadratic, which is why power-state machines are kept deliberately simple and why a block that demands its own independent domain is expensive to integrate.

The classic power-gating bug is an isolation enable that arrives too late, and it is not a functional bug in the usual sense. If power is removed before isolation is enabled, the outputs float for a short interval; downstream gates in the always-on domain then sit at an intermediate level and conduct, drawing a current spike that is invisible to functional simulation and shows up as excess power or, in the worst case, as damage. **The sequence is: assert isolation, assert retention save, then remove power — and the reverse on wake.** Getting it right is a small state machine; proving it right requires power-aware simulation, and asserting it is what a UPF-aware formal check does. An IP block that supplies its own power-control sequence and the assertions that check it is removing a whole class of integration risk.

X43.3 DVFS: the controller and its constraints

Table 357. A DVFS operating-point table, normalised to the top point

V (V)	f (MHz)	Relative power	Relative energy per operation	Relative throughput	Power per unit throughput
0.9	1,200	1	1	1	1
0.8	1,000	0.6584	0.7901	0.8333	0.7901
0.7	800	0.4033	0.6049	0.6667	0.6049
0.6	600	0.2222	0.4444	0.5	0.4444
0.5	400	0.1029	0.3086	0.3333	0.3086

Energy per operation falls monotonically as voltage drops (the V^2 term), which is Part X7's result in the form a system actually uses. The last column is what a governor optimises when the work must be finished by a deadline rather than as fast as possible.

Problem. Why is ‘race to idle’ sometimes better than running slowly, when X43.2 shows energy per operation is lowest at low voltage?

Solution. Because the block is not the only thing consuming power. Running at the top operating point finishes the work sooner and lets the *whole subsystem* — memory, interconnect, PLLs, the always-on logic, and any peripheral waiting on the result — enter a low-power state earlier. That static and shared component does not scale with the block's voltage, so the comparison is between (fast, high dynamic, short time, then everything off) and (slow, low dynamic, long time, everything on throughout).

Race to idle wins when the fixed overhead per unit time is large relative to the block's own dynamic power, which is common in small IP blocks inside large systems and uncommon in a block that dominates its chip. **The crossover is computable** from the block's dynamic power at each point, the platform's idle power, and the entry/exit energy of the low-power state — and it is worth computing rather than asserting, because both answers appear in the literature and both are right for their own systems.

Z17. From One Transistor to Our Datapath

This part builds the equaliser upward from a single transistor. Nothing is assumed. If you know that a computer is made of transistors and nothing more specific than that, start here; by the end you will be able to read the gate-level schematic of the adder inside the filter and say why it is shaped the way it is.

The reason this matters to someone writing RTL or HLS is not nostalgia. Every decision the previous parts made — why saturation costs more than truncation, why a 9-bit adder is not an 8-bit adder, why the critical path runs where it does, why you register a signal before it leaves the chip — is a fact about these drawings. You can write RTL without them and you will write slower RTL.

Z17.1 The switch

A MOSFET is a voltage-controlled switch with three terminals that matter: **gate**, **source** and **drain**. Current flows between source and drain when the gate says so. There are two flavours and they are opposites:

Table 358. The two switches

Type	Turns on when its gate is	Good at passing	Where it goes
NMOS	high (1)	a strong 0	between the output and ground
PMOS	low (0)	a strong 1	between the output and VDD

The third column is the one people skip and then get wrong. An NMOS can pull a node down to 0 V cleanly but cannot pull it all the way up to VDD — it stops roughly a threshold voltage short. A PMOS is the mirror image. So neither type alone can build a good logic gate, and *that* is why every static CMOS gate contains both: PMOS above to pull up, NMOS below to pull down. The circuit family is called **complementary** MOS for exactly this reason.

Z17.2 The inverter, and why CMOS won

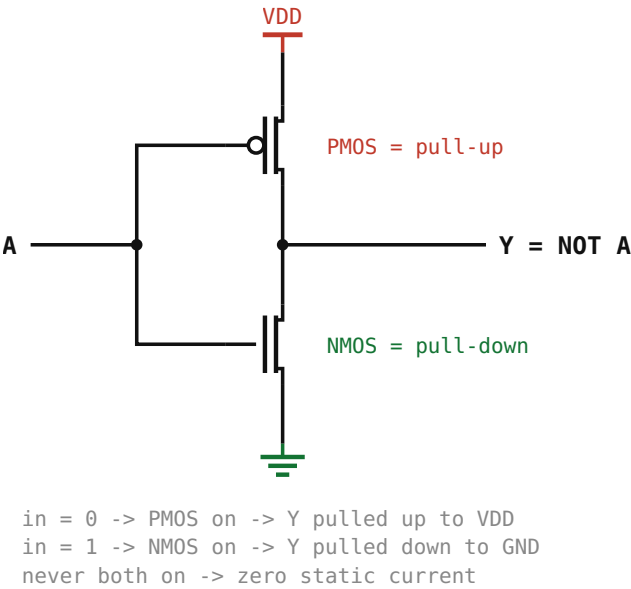


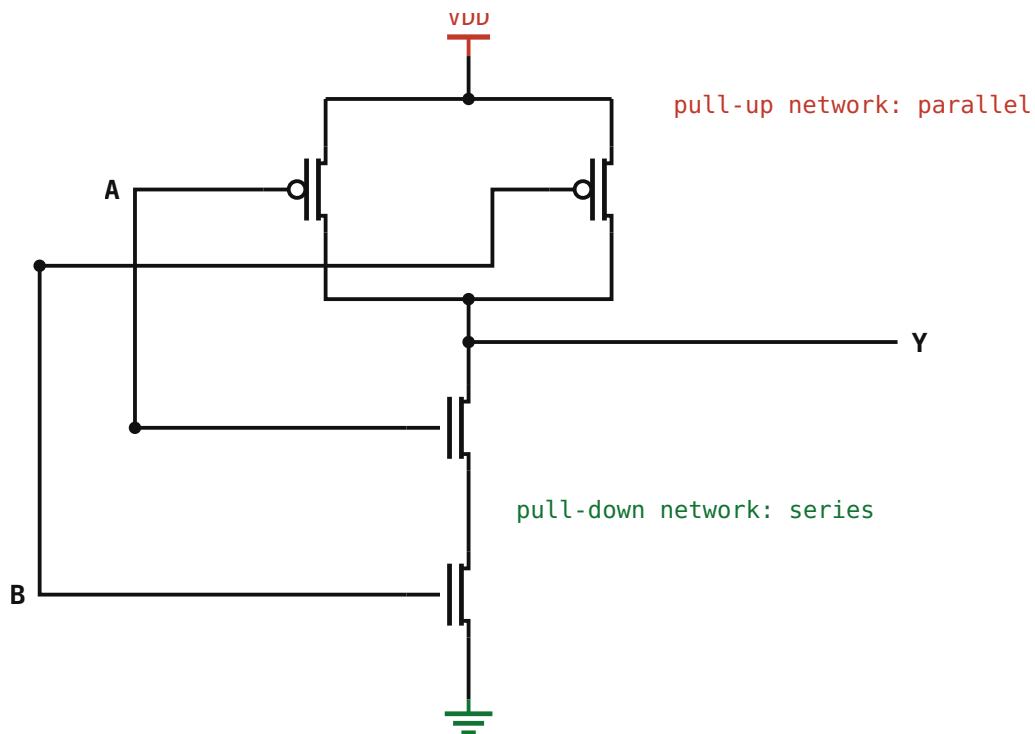
Figure 49. The CMOS inverter: one PMOS pull-up, one NMOS pull-down, gates tied together. This is the smallest complete CMOS gate and the template for every other one. The red bar at the top is the supply (VDD); the striped symbol at the bottom is ground (0 V).

Why this circuit inverts, and why it costs no static power

	Step	Justification
1	Input A = 0. The PMOS gate sees 0, so the PMOS is on . The NMOS gate sees 0, so the NMOS is off .	<i>From the table above: PMOS turns on with a low gate, NMOS with a high gate.</i>
2	With only the PMOS on, the output node is connected to VDD and disconnected from ground. Y is pulled up to 1.	<i>The only conducting path from Y goes to the supply.</i>
3	Input A = 1. Now the PMOS is off and the NMOS is on, so Y is connected to ground and disconnected from VDD. Y is pulled down to 0.	<i>The mirror image of the previous step.</i>
4	In both steady states exactly one of the two devices conducts, so there is no path from VDD to ground .	<i>A path from supply to ground would be a short circuit drawing continuous current.</i>
5	Therefore a CMOS gate that is not switching draws (almost) no current.	<i>This is the property that let CMOS displace every earlier logic family. Power is consumed charging and discharging capacitance when the output changes, which is why dynamic power is proportional to activity and frequency.</i>
6	The word 'almost' is doing real work at modern nodes.	<i>Sub-threshold leakage and gate leakage mean an idle gate does draw a small current, and below roughly 28 nm leakage becomes a first-order design problem. Part X43 covers what is done about it.</i>

Z17.3 NAND: pull-up and pull-down are duals

Now the gate that everything is actually built from. The rule that generates every static CMOS gate is a single sentence: **build the pull-down network from NMOS so that it conducts exactly when the output should be 0, then build the pull-up network as its dual.**



$Y = 0$ only when A and B are both 1
 if either input is 0 the upper path conducts and $Y = 1 \rightarrow Y = \text{NOT}(A \text{ AND } B)$
 pull-up and pull-down are always duals: series $\leftrightarrow$ parallel

Figure 50. A 2-input CMOS NAND. The two NMOS are in **series**: current only reaches ground when A and B are both 1, which is exactly when NAND's output should be 0. The two PMOS are in **parallel**: a path to VDD exists when A is 0 or B is 0. Series means AND; parallel means OR.

Table 359. Reading the NAND schematic as a truth table

A	B	Pull-down (series NMOS)	Pull-up (parallel PMOS)	Y
0	0	both off $\rightarrow$ open	both on $\rightarrow$ conducts	1
0	1	top off $\rightarrow$ open	left on $\rightarrow$ conducts	1
1	0	bottom off $\rightarrow$ open	right on $\rightarrow$ conducts	1
1	1	both on $\rightarrow$ conducts	both off $\rightarrow$ open	0

Why the pull-up is the dual of the pull-down, always

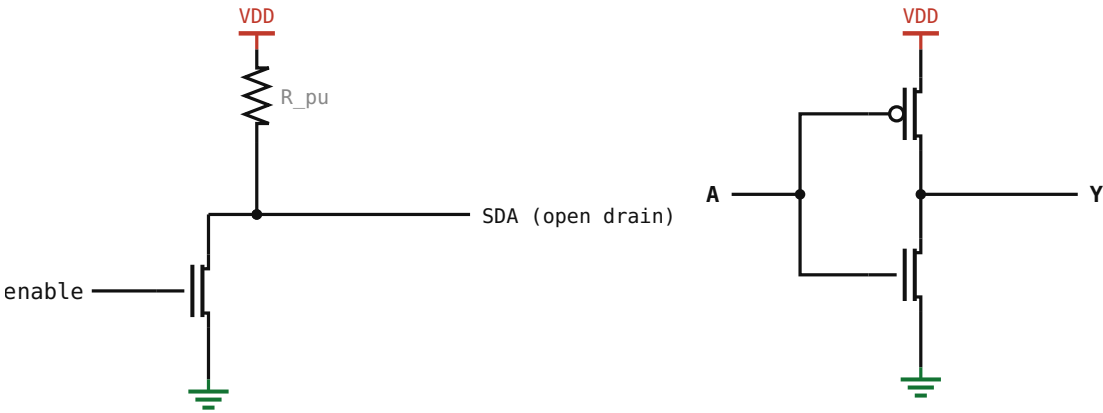
	Step	Justification
1	The output must be driven to exactly one value in every input combination.	If neither network conducts the output floats (high-impedance, value undefined); if both conduct you have a short from VDD to ground.
2	So the pull-up must conduct exactly when the pull-down does not — the two conditions are logical complements.	That is what ‘exactly one’ means.
3	The pull-down for NAND conducts when (A AND B).	Series transistors both have to be on, and NMOS is on for a 1.
4	Its complement is NOT(A AND B) = (NOT A) OR (NOT B), by De Morgan’s law.	De Morgan converts a negated conjunction into a disjunction of negations.
5	PMOS is on for a 0, so a PMOS whose gate is A implements (NOT A). An OR of two conditions is a parallel connection.	Either path alone is enough to conduct.
6	Hence: series NMOS below, parallel PMOS above. Swap series and parallel and you get NOR instead.	NOR’s pull-down is parallel (output 0 when A OR B) and its pull-up is series.

Two consequences worth carrying into RTL. First, **NAND and NOR are the cheap gates** — four transistors each. AND and OR are *more* expensive, because CMOS naturally produces an inversion and a non-inverting gate is a NAND followed by an inverter: six transistors. This is why synthesis tools cheerfully rewrite your logic into seas of NANDs, and why the classic identity $AND(a,b) = NOT(NAND(a,b))$ is worth recognising in a netlist.

Second, **series transistors are slow**. A 4-input NAND has four NMOS in series and its pull-down is roughly four times weaker than an inverter's. That is why wide gates get decomposed into trees, and why a 17-bit ripple adder is not simply ‘a bit slower’ than an 8-bit one.

Z17.4 Pull-up resistors — a different thing with a similar name

The phrase “pull-up” is used for two different circuits and conflating them causes real design errors.



the resistor makes the 1; current flows through it while the transistor is 0
the rising edge is RC and therefore slow -- this sets I2C bus speed
the transistor drive both 1 and 0 actively
no static current, and fast edges

Figure 51. Two things called “pull-up”. **Left:** a real resistor on a board, holding an open-drain line high until some device pulls it low. **Right:** a PMOS pull-up network inside a CMOS gate. Only the left one is a resistor; the right one is the transistor network of Z17.2.

Table 360. Resistor pull-up versus transistor pull-up

	Pull-up resistor (board level)	PMOS pull-up (inside a gate)
What it is	A physical resistor, typically 1–10 k Ω	A transistor network, the dual of the pull-down
Where	On the PCB or inside an I/O pad	Inside every static CMOS gate
Why	The line has no active driver for the high state — open-drain or open-collect-or outputs can only pull down	Every state is actively driven
Static current	Yes — whenever the line is held low, current flows through the resistor to ground	No — exactly one network conducts
Speed	Slow rising edge: RC charging of the bus capacitance. This is why I ² C is limited to 100 kHz/400 kHz/1 MHz modes	Fast: the PMOS actively charges the node
Typical use	I ² C SDA/SCL, shared interrupt lines, reset lines, strapping pins, unused inputs	All ordinary logic
Relevance to our IP	None inside the block. It appears at the chip boundary and in the CSR bus pads	Every gate the filter synthesises into

Worked example 54. Sizing a pull-up resistor, and why the choice is a squeeze

Given. An I²C bus at 3.3 V with 200 pF of bus capacitance. The standard requires the low level to stay below 0.4 V while a device sinks at most 3 mA, and the rise time to be under 300 ns for 400 kHz fast mode.

Method. The lower bound comes from current: R must be large enough that $(V_{DD} - V_{OL})/R$ does not exceed the sink current. The upper bound comes from speed: the RC rise to the input threshold must fit in the rise-time budget, which for $0 \rightarrow 0.7 V_{DD}$ takes about 1.2 RC.

Numbers. Lower bound: $R \geq (3.3 - 0.4)/3 \text{ mA} = 967 \text{ }\Omega$. Upper bound: $R \leq 300 \text{ ns}/(1.2 \times 200 \text{ pF}) = 1250 \text{ }\Omega$. So R must sit between roughly 1.0 and 1.25 k Ω — a narrow window.

Note what dominates: capacitance. Halving C doubles the upper bound. On a board, capacitance is trace length and the number of devices, so the electrical problem is really a layout problem.

How this is got wrong. The window being this narrow is the finding, not an arithmetic slip. The usual reflex is to grab the familiar 4.7 k Ω part, which violates the upper bound by nearly 4 $\times$ and produces a bus that works on the bench at low speed and fails at 400 kHz with long traces. If the window closes entirely, the fix is not a different resistor — it is less bus capacitance, or an active terminator.

Z17.5 The clock, and why anything is clocked at all

Combinational logic has no memory: outputs follow inputs after a delay. To build anything that accumulates, you need an element that *holds*. The universal one is the edge-triggered **D flip-flop**: it samples its D input at the rising edge of the clock and holds that value on Q until the next rising edge.

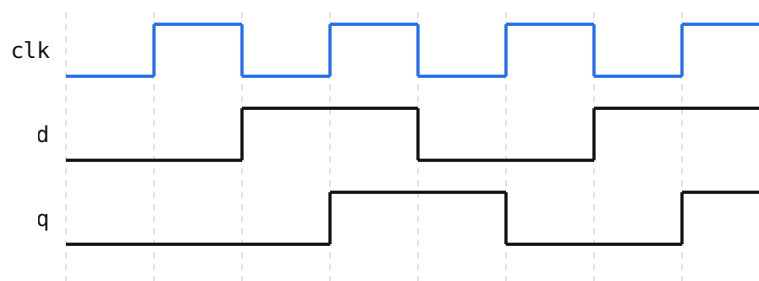


Figure 52. A D flip-flop. Q takes the value D had at the rising edge and holds it for the whole cycle. Note that Q changes only at rising edges, never in between — that is the entire point.

The clock is not there to make things go. It is there to make things *agree on when*. Combinational paths through different amounts of logic finish at different times; without a clock, a downstream block cannot know whether what it is looking at is the final answer or a value still settling. The clock defines an instant at which everything is required to have settled, and the design's job is to make that true.

Table 361. The two timing constraints, and what violating each looks like

Con-straint	What it requires	Fix if violated	Symptom
Setup	Data must arrive at the flop at least t_{su} before the edge. Limits the longest path.	Shorten the logic, add a pipeline stage, or slow the clock.	Works at low frequency, fails above some clock rate. Reproducible.
Hold	Data must remain stable at least t_h after the edge. Limits the shortest path.	Add delay (buffers). Slow-ing the clock does not help.	Fails at every frequency. Usually a silicon respin — which is why hold violations are feared more than setup.

That asymmetry is worth memorising: a setup failure is a performance bug, a hold failure is a functional bug. It is also why Part Z15's measurement needed a registered timing shell — a path that starts or ends outside the chip has no flop at one end, so neither constraint is defined and the tool has nothing to report.

Z17.6 Our adder, at gate level

Now put it together. The filter's first operation is $a0 = x0 + x3$ on 8-bit signed values producing 9 bits. An adder is built from full adders, and a full adder is built from the gates above.

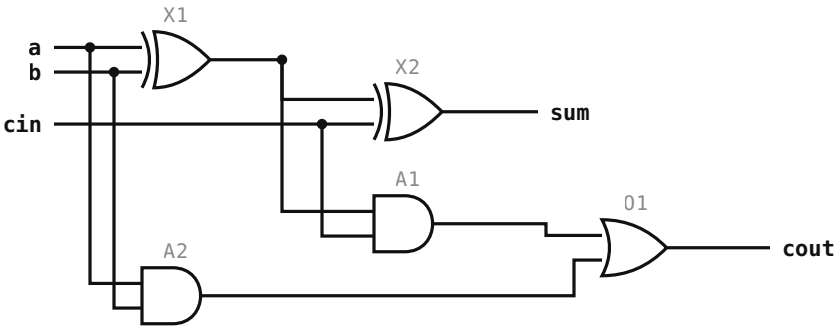


Figure 53. One full adder: $sum = a \text{ XOR } b \text{ XOR } cin$, $cout = (a \text{ AND } b) \text{ OR } (cin \text{ AND } (a \text{ XOR } b))$. Five gates. The 9-bit fold adder in the filter is nine of these chained, with each cout feeding the next cin — a ripple-carry adder.

Worked example 55. Why the carry chain sets the critical path

Given. A ripple-carry adder of N bits. Each full adder's carry output takes about 2 gate delays (one AND, one OR) after its carry input is valid.

Method. The last bit's carry cannot be computed until every earlier carry has rippled through. Count gate delays along that chain.

Numbers. Roughly 2N gate delays for N bits: **18** for the 9-bit fold adder, **34** for a 17-bit accumulator. The sum bits themselves add only one more XOR after their carry arrives.

This is also why fast-adder architectures exist — carry-lookahead, carry-select, prefix adders — trading area for a logarithmic rather than linear carry depth. On an FPGA you usually get the vendor's carry chain and the question does not arise; in an ASIC the synthesiser picks from a library of adder architectures based on your timing constraint.

How this is got wrong. The tempting conclusion is that our filter's critical path is the adder. It is not, and assuming so sends you optimising the wrong thing. The measured designs in Z15 use iCE40's dedicated carry chain (SB_CARRY), which is far faster than generic gates — the hand design used 48 of them. On this fabric the multiply-by-constant shift-add network, not the ripple, dominates. **Count gates to understand the structure; measure the tool output to know the number.**

Z17.7 Why saturation costs more than wrapping

Part Z16 justified saturation on system grounds: a wrap flips the sign and the slicer decides exactly wrong. Here is what it costs, in gates.

Table 362. Truncate versus saturate, at gate level

Operation	What the hardware does	Cost
Truncate (wrap) to 8 bits	Discard the upper bits. Route 8 wires, ignore the rest.	Zero gates. It is a renaming of wires.
Saturate to 8 bits	Detect whether the value exceeds the 8-bit range, and if so substitute the limit. Needs two comparisons and an 8-bit 2-to-1 multiplexer.	Two comparators plus a mux — tens of gates, and it sits after the adder, so it adds directly to the critical path.

This explains a result in Part Z15 that looked backwards at first. Narrowing the accumulator from 32 bits to 18 *should* have shrunk the design, but the narrowed variant came out larger in Bambu's estimate — because the narrowing was implemented with `sat()` at every accumulation step, and each of those saturations bought comparators that the plain 32-bit accumulation did not need. The width saving was real and the saturation cost was larger.

The lesson generalises: **narrowing a datapath is only free if you narrow it somewhere the value provably cannot overflow.** In the final tuned design the accumulator is wide enough that no intermediate saturation is needed at all — Z16.7 derived the bound, 32,766, which fits in 17 bits — and saturation happens exactly once, at the output, where the specification demands it.

Z17.8 The whole stack, one page

Table 363. Every level, and what our filter looks like there

Level	Our filter at this level	Who works here
System	One box in the receive chain, between ADC and DFE	Architect; the spec of Z16.4 is written here
RTL / HLS	40 lines of Verilog, or 12 lines of C++ plus a build command	You, in Parts Z14–Z15
Netlist	338–1218 cells depending on how the C++ was typed	Synthesis (yosys); you read its report

Level	Our filter at this level	Who works here
Gate	SB_LUT4 ×228, SB_CARRY ×97 — NANDs and carry chains in a specific arrangement	The tool; you influence it through coding style and constraints
Transistor	Each LUT4 is a small memory plus a mux tree; each is CMOS pull-up/pull-down as in Z17.2	The FPGA vendor, or in an ASIC the standard-cell library team
Silicon	Configuration bits in an iCE40, or diffused transistors in a mask set	The foundry

The practical point of having walked down this stack is that the levels are not independent. A choice at the top — “saturate rather than wrap” — became comparators and a mux at the gate level, which became area and delay in the measurements of Part Z15, which fed back into the specification in Z16.4. An engineer who can only see one level cannot follow that chain, and will make the choice at the top without knowing what it costs at the bottom.

T1. The MOSFET as Four Different Machines

One device, four operating regions — and you design a different circuit in each.

Before you read this

voltage · current · resistance · electric field · charge · electrons · exponential functions · logarithms · differentiation · Ohm's law

New in this chapter

threshold voltage · cutoff region · triode region · saturation region · subthreshold region · on-resistance · subthreshold swing · leakage current · transconductance · channel-length modulation · output resistance · body effect · gate capacitance

A MOSFET is usually introduced as a switch, and for a first digital course that is enough. It is not enough to design production IP. The same four-terminal device behaves as four qualitatively different machines depending on the voltages you put on it, and a working designer moves between all four — often inside one block. A sampling front end uses the device as a *resistor*; the comparator behind it uses the device as an *amplifier*; the digital logic behind that uses it as a *switch*; and the leakage that drains the battery while all of it sits idle is the device acting as a *very bad switch* in the region nobody designed for.

This chapter establishes the four regions, then — for each — states where you use it, when you reach for it, how you apply it, and what the code that ships with a real IP deliverable looks like. Nothing here is included unless it is used later in this book.

T1.1 What the gate actually does

Before the regions, one physical fact, because every later equation rests on it. The gate is one plate of a capacitor. The channel — the thin sheet of silicon under the oxide — is the other. Between them sits an insulator a few atoms thick.

Gate oxide capacitance per unit area, C_{ox} — ϵ_{ox}/t_{ox} . With a physical oxide thickness of 1.2 nm and $\epsilon_{ox} \approx 3.9 \epsilon_0$, this is about 16 mF/m² — roughly 1.60×10^{19} fF per square micrometre. Everything the transistor does is proportional to it.

Derivation — Why gate voltage controls current at all

	This follows	Because
1	Put a positive voltage on the gate of an NMOS. The gate is one plate of a capacitor, so it stores charge $Q = C_{ox} \cdot V$ per unit area.	That is the definition of capacitance. No transistor physics yet.
2	The opposite charge must appear on the other plate — in the silicon, directly under the oxide.	A capacitor holds equal and opposite charge on its plates.
3	In p-type silicon the mobile negative charge is electrons, which are scarce. Below some gate voltage the silicon supplies the required charge by pushing <i>holes</i> away instead, leaving fixed negative acceptor ions.	Depleting the majority carriers is cheaper than creating minority carriers.
4	Above some gate voltage the depletion stops growing and the silicon starts supplying mobile electrons instead. That sheet of electrons is the channel.	Once the surface is pulled far enough, attracting electrons costs less energy than widening the depletion region.
5	That crossover gate voltage is the threshold voltage V_t.	It is not a switch trip point designed into the device; it is the voltage at which the silicon changes which kind of charge it supplies.
6	Above V_t , the mobile charge in the channel is $Q_{ch} = C_{ox}(V_{GS} - V_t)$ per unit area.	The first V_t worth of gate voltage went into depletion; only the excess produces mobile charge.
7	Current is charge times velocity, and velocity is mobility times field: $I = W \cdot Q_{ch} \cdot \mu \cdot E$.	Drift current, the same relation that gives Ohm's law in a resistor.

That last line is the whole device. Every equation below is this relation with a different assumption about how the field E varies along the channel. The quantity $V_{GS} - V_t$ appears so often it has a name: the **overdrive**, written V_{ov} . When a designer says “I ran that device at 150 millivolts of overdrive”, this is the number.

V_t is not one number. It moves with temperature (roughly -1 mV/°C), with body bias, with channel length, with how close the neighbouring polysilicon is, and it differs from die to die and from transistor to transistor on the same die. A design that works only for the nominal V_t does not work. Chapter T27 makes this quantitative; for now, treat every V_t in this chapter as the centre of a distribution.

T1.2 Region 1 — cutoff: the switch that does not quite switch off

Cutoff and subthreshold leakage

Below threshold the channel is not empty — it is exponentially sparse. The current does not stop; it falls by a factor of ten for every **85 mV** that V_{GS} drops below V_t . That slope is the **subthreshold swing**, S , and it is the single most important number in a low-power design.

$$I_{\text{leak}} = I_0 \cdot 10^{(V_{GS} - V_t)/S}$$

S has a floor. The channel charge responds to the surface potential through a Boltzmann factor, so $S \geq (kT/q) \cdot \ln 10 =$ **59.5 mV/decade** at room temperature, and real devices land 20–40 % above that because part of the gate voltage is lost to the depletion capacitance. **This floor is why leakage became the dominant power term as supplies scaled:** you cannot lower V_t to keep speed without paying exponentially in leakage, and you cannot make the exponent steeper.

Where it is used	Standby power of every block you ship. Retention registers. Memory arrays (where leakage is multiplied by millions of cells). Power gating and the sizing of the header/footer switch. Any IP sold with a quoted standby current — which is most of them.
When it binds	Whenever you write a leakage number into a datasheet; whenever you choose between device flavours in synthesis; whenever a customer asks for a retention mode. Also whenever a block is mostly idle — an equaliser that runs for 3 % of the time is a leakage problem, not a dynamic-power problem.
How to apply it	Get I_{off} per micrometre of width from the foundry's characterisation at the worst corner (highest temperature, lowest V_t , highest supply — leakage worsens in the opposite direction from speed). Multiply by the total width in the block. Multiply again by the temperature factor — a rule of thumb is $2\times$ per 10°C , which follows directly from the S equation because S itself is proportional to T . Then decide: if the answer exceeds budget, move the non-critical paths to high- V_t cells and re-time.

What the industry code looks like

```
// Liberty (.lib) – what the foundry actually ships you.
// This is the leakage the synthesiser reads. Note that it is
// state-dependent: a NAND2 leaks differently per input pattern,
// because the stacked device changes the internal node voltage.

cell (NAND2_X1) {
    area : 1.596 ;
    cell_leakage_power : 4.2851e+03 ;           // nW, the default
    leakage_power () {
        when          : "!A1 & !A2" ;
        value          : 1.8032e+03 ;           // both off: stack effect helps
    }
    leakage_power () {
```



```

        when          : "A1 & !A2" ;
        value         : 4.9210e+03 ;
    }
    leakage_power () {
        when          : "A1 & A2" ;
        value         : 6.1104e+03 ;      // both on: worst
    }
    ...
}

# Synthesis – trade leakage against timing by device flavour.
# The tool will use LVT only where it must to close timing.
set_db library [list sc9_base_svt.lib sc9_base_hvt.lib sc9_base_lvt.lib ]
set_db leakage_power_effort      high
set_db dynamic_power_effort      medium
# Forbid the leaky flavour outside the critical paths:
set_dont_use [get_lib_cells */*_LVT*]
# ... place and route, then selectively re-enable on failing paths:
set_dont_use -false [get_lib_cells */BUF_X*_LVT]

```

This is where it goes wrong. Quoting leakage at nominal temperature. Leakage roughly doubles per 10 °C, so a 25 °C number is **8–16× optimistic** against a 125 °C automotive corner. A datasheet that says “standby 12 µA” without naming the temperature, supply and corner is not a specification — and a customer who discovers the real number at their thermal limit will not buy from you again.

Worked example 1. Leakage of a block, done the way a datasheet requires

Given. A block with 40,000 gates, average device width 0.8 µm per gate (both n and p), $I_{\text{off}} = 0.01 \text{ nA}/\mu\text{m}$ at $V_{\text{GS}} = 0$, 25 °C, typical corner. The customer runs it at 105 °C and asks for the worst-case standby current.

Method. Total width, times I_{off} per micrometre, times the temperature factor ($2\times$ per 10 °C), times the corner factor for fast-fast silicon (typically $2\text{--}3\times$ on leakage).

Answer. Width = $40,000 \times 0.8 = 32,000 \mu\text{m}$. At 25 °C, typical: $32,000 \times 0.01 \text{ nA} = \mathbf{0.24 \mu\text{A}}$. Temperature: $105 - 25 = 80 \text{ °C}$, so $2^8 = 256\times$. Corner: $\times 2.5$. Total $\approx \mathbf{0.16 \text{ mA}}$.

The factor-of-2-per-10 °C rule is an approximation of the exponential; for a real deliverable you read the number off the foundry's corner tables rather than extrapolating. The point of the estimate is to know, early, whether you are within a factor of two of the budget or a factor of fifty.

This is where it goes wrong. The 25 °C typical number and the 105 °C fast number differ by **640×** here. Engineers quote the first because it is the one the tool prints by default. The second is the one the customer measures. Always state the triple — temperature, supply, process corner — next to the number; a leakage figure without it is meaningless.

T1.3 Region 2 — triode: the transistor as a resistor

Triode (linear) region and on-resistance

Turn the gate well above V_t and hold the drain close to the source. Now the channel exists along its whole length and the field along it is nearly uniform, so the device behaves like a resistor whose value the gate controls:

$$I_D = \mu C_{\text{ox}}(W/L)[(V_{\text{GS}} - V_t)V_{\text{DS}} - V_{\text{DS}}^2/2]$$

For small V_{DS} the square term vanishes and the device is simply

$$R_{\text{on}} = 1 / [\mu C_{\text{ox}}(W/L)(V_{\text{GS}} - V_t)]$$

Read that denominator. R_{on} is inversely proportional to overdrive. If the signal you are passing moves V_S , then V_{GS} moves with it, and **the resistance changes with the signal**. A signal-dependent resistance in a sampling path is distortion, and it is the reason bootstrapped switches exist.

Where it is used	Every transmission gate and pass gate. The sampling switch at the front of every ADC and every track-and-hold. Power-gating headers and footers. The programmable resistors in a CTLE. Write drivers in SRAM. Any place a digital signal must be routed rather than regenerated.
When it binds	When you need to <i>connect</i> two nodes rather than <i>drive</i> one. The moment you catch yourself writing “pass the signal through” rather than “buffer the signal”, you are in triode and R_{on} matters.
How to apply it	Budget the settling first, then size from it. A switch with resistance R charging capacitance C settles with time constant $\tau = RC$; to settle to N bits you need $t_{settle} \geq RC \cdot \ln(2^{N+1})$. Invert for R , then use $R_{on} = 1/[\mu C_{ox}(W/L)V_{ov}]$ to get W/L . Then check R_{on} at the worst signal level , not at mid-scale: for an NMOS passing a rising signal, V_{GS} shrinks as the signal rises and R_{on} blows up near $V_{DD} - V_t$.

What the industry code looks like

- * SPICE – the measurement you actually run on a sampling switch.
- * You are not checking that it works; you are checking how R_{on} varies
- * with the signal, because that variation is the distortion.

```
.param vin_val=0.45
M1 drain gate source bulk nch W=2u L=30n
Vg gate 0 DC 0.9
Vs source 0 DC 'vin_val'
Vd drain 0 DC 'vin_val+1m'      $ 1 mV across the switch: stay in triode
.dc vin_val 0 0.9 0.01
.measure dc ron PARAM='1m/I(Vd)' $ R = V/I at each input level
.alter
.param boot=1                  $ repeat with a bootstrapped gate
.end
```

```
// Verilog-AMS behavioural model used in mixed-signal top-level sims.
// A digital verification engineer meets this, not the SPICE deck.
module sw_nmos (inout d, inout s, input g);
    parameter real ron_nom = 250.0;    // ohms at mid-scale
    parameter real roff    = 1e12;
    analog begin
        // Ron rises as (Vgs - Vt) falls: this is the term that matters.
        real vgs, ron;
        vgs = V(g) - V(s);
        ron = (vgs > 0.35) ? ron_nom * 0.55 / (vgs - 0.35) : roff;
        I(d,s) <+ V(d,s) / ron;
    end
endmodule
```

This is where it goes wrong. Sizing the switch at mid-scale and declaring it done. R_{on} at the top of the input range can be several times its mid-scale value, and the settling error it causes is *signal-dependent* — which means it is harmonic distortion, not noise, and averaging will not remove it. Sweep R_{on} across the full input range and size for the worst point.

Worked example 2. Sizing a sampling switch from the settling requirement

Given. A 10-bit converter samples onto $C = 1$ pF in 2 ns. Supply 0.9 V, $V_t = 0.3$ V, and the worst-case input is 0.5 V, so $V_{ov} = 0.9 - 0.5 - 0.35 = 0.1$ V.

Method. Settling to 10 bits needs the error below $\frac{1}{2}$ LSB, i.e. $e^{-t/\tau} < 2^{-11}$, so $t/\tau > 11 \cdot \ln 2 = 7.62$. Use a $2\times$ margin on τ for the rest of the path, then solve R from $\tau = RC$, then W/L from R_{on} .

Answer. $\tau \leq 2$ ns / 7.62 = 262 ps. With $2\times$ margin, $\tau = 131$ ps. $R = \tau/C = 131$ Ω . Then $W/L = 1/(\mu C_{ox} \cdot R \cdot V_{ov}) = 317$.

This is where it goes wrong. Using V_{ov} at the *mid-scale* input (0.45 V) instead of the worst-case input gives $V_{ov} = 0.1$ V and a W/L about $2\times$ smaller — and then the switch misses its settling target exactly where the signal is largest, which is where the distortion is most visible. **Size switches at the worst signal level, not the average one.**

T1.4 Region 3 — saturation: the transistor as a current source

Saturation, transconductance, and output resistance

Raise V_{DS} above the overdrive and the channel *pinches off* near the drain: the local V_{GS} there falls to V_t and no more mobile charge exists. Beyond that point, raising the drain voltage no longer raises the current much. The device has become a current source controlled by the gate:

$$I_D = \frac{1}{2} \mu C_{ox} (W/L) (V_{GS} - V_t)^2 \cdot (1 + \lambda V_{DS})$$

Two derived quantities do all the work in analog design:

Transconductance $g_m = \partial I_D / \partial V_{GS} = 2I_D / V_{ov}$. How much current you get per volt of input — i.e. the gain-producing term.

Output resistance $r_o = \partial V_{DS} / \partial I_D \approx 1/(\lambda I_D)$. How close to an ideal current source it is. The term λ exists because pinch-off moves with drain voltage — **channel-length modulation** — and it is worse for short channels, which is why analog blocks use devices several times the minimum length.

Their product $g_m r_o = 2/(\lambda V_{ov})$ is the intrinsic gain: **the most voltage gain one transistor can give you**. It falls as processes scale, which is why modern analog design leans on digital assistance rather than on raw gain.

Where it is used	Comparators and sense amplifiers — the pre-amp stage. Every current mirror and bias network. CTLE and the analogue front end of a SerDes. Level shifters during the transition. Ring-oscillator delay cells in a PLL. Anywhere the circuit must <i>amplify</i> rather than <i>switch</i> .
When it binds	When the question is “how much gain” or “how much current, independent of the load”. Also, unexpectedly, in <i>digital</i> analysis: during a logic transition both devices pass through saturation, and the short-circuit current that flows then is the term T4 uses to explain why slow input edges cost power.
How to apply it	Design with g_m/I_D rather than with W/L . Pick the g_m you need from the gain or the noise requirement; pick g_m/I_D from the speed/power trade (high $g_m/I_D \approx$ low overdrive $\approx$ efficient but slow; low $g_m/I_D \approx$ high overdrive $\approx$ fast but power-hungry); that fixes I_D ; read W/L off the foundry's characterised g_m/I_D curves. This works across processes and does not rely on the square-law equation, which stops being accurate below about 100 nm.

What the industry code looks like

- * The g_m/I_D sweep every analog team runs once per process,
- * then reuses for every block. You are not simulating a circuit here;
- * you are characterising the device so design becomes look-up.

```
.param L_val=100n
M1 d g 0 0 nch W=10u L='L_val'
Vg g 0 DC 0.5
```

```

Vd d 0 DC 0.45          $ hold Vds at VDD/2, in saturation
.dc Vg 0.2 0.9 0.005
.measure dc gm_id PARAM='deriv(I(Vd),V(g))/I(Vd)'
.measure dc ft PARAM='deriv(I(Vd),V(g))/(2*3.14159*cgg)'
.step param L_val list 30n 60n 100n 200n 500n
.end

// What a digital designer sees of all this: a characterised cell.
// The analog block arrives as a .lib timing/power view plus a
// behavioural model. You must still know what is inside, because
// the constraints you write depend on it.
cell (CMP_STRONGARM) {
    pin (CLK) { direction : input; clock : true; capacitance : 4.8; }
    pin (VINP) { direction : input; capacitance : 12.1; }
    pin (OUTP) { direction : output;
        timing () {
            related_pin : "CLK" ;
            timing_type : rising_edge ;
            cell_rise (delay_template) { ... }
        } }
    // The number a digital designer must not ignore:
    // the comparator has a metastability window like any bistable.
    // T14 derives it; the vendor states it here.
}

```

This is where it goes wrong. Using the square-law equation for design in a modern process. Below roughly 100 nm the velocity of carriers saturates, the exponent drifts from 2 toward 1, and the equation can be wrong by tens of percent. It remains excellent for *reasoning* — it tells you the right direction and the right sensitivity — and useless for *numbers*. Reason with the equation, size from the characterised curves.

T1.5 Region 4 — subthreshold as a design space, not only as leakage

Subthreshold operation

The same exponential that produces leakage is a usable transfer characteristic. Biased below V_T , the device gives g_m/I_D at its maximum — about **27.1 V⁻¹**, set only by S — which is the best current efficiency any MOSFET can offer. The cost is speed: f_T falls by orders of magnitude, and the exponential dependence on V_T means that device-to-device V_T mismatch turns into *current* mismatch of tens of percent.

So subthreshold is the region you choose when energy per operation dominates and speed does not, and the region you are forced into when a supply droops. It is also the region a **retention** circuit lives in: state is held by a device that is barely on.

Where it is used	Always-on domains — wake-up logic, watchdog timers, retention registers. Sensor front ends. Bias generators that must survive at very low current. And, negatively, every leakage path in every block you ship.
When it binds	When the duty cycle is very low and the energy budget is per-operation rather than per-second. When a customer asks for a retention mode with a quoted microamp figure. When you are asked to survive a supply that collapses toward V_T .

How to apply it

Accept that matching, not speed, will set your area. In strong inversion, current mismatch tracks V_t mismatch through g_m/I_D ; in subthreshold g_m/I_D is at its maximum, so the *same* V_t mismatch produces the *largest* current mismatch. Size the devices from the matching requirement (T28's Pelgrom relation), then check speed, rather than the other way round.

What the industry code looks like

```
# UPF (Unified Power Format) – how the always-on domain is
# declared to the tools. A digital IP deliverable that supports
# retention ships this file, and it is as much a part of the
# product as the RTL.

create_power_domain PD_ALWAYS_ON -elements {u_wake u_timer}
create_power_domain PD_CORE      -elements {u_dsp u_eq}    -scope .

create_supply_net  VDD_AON  -domain PD_ALWAYS_ON
create_supply_net  VDD_CORE -domain PD_CORE
set_domain_supply_net PD_CORE -primary_power_net VDD_CORE \
                             -primary_ground_net VSS

# Retention: state survives while VDD_CORE collapses.
set_retention  RET_CORE -domain PD_CORE \
    -retention_power_net VDD_AON -retention_ground_net VSS \
    -save_signal {u_dsp/ret_save high} \
    -restore_signal {u_dsp/ret_restore low}

# Isolation: outputs of a collapsed domain must not float into
# the always-on domain, or the always-on logic burns crowbar current.
set_isolation ISO_CORE -domain PD_CORE \
    -isolation_power_net VDD_AON -clamp_value 0 \
    -applies_to outputs
set_isolation_control ISO_CORE -domain PD_CORE \
    -isolation_signal u_pmc/iso_en -isolation_sense high
```

This is where it goes wrong. Forgetting isolation. When a power domain collapses, its outputs drift to an intermediate voltage. Feed that into an always-on gate and **both** its devices conduct — the always-on domain then draws milliamps instead of microamps, and the retention mode you sold does not exist. This is one of the most common integration failures in low-power IP, and it is invisible in RTL simulation because RTL has no intermediate voltage.

T1.6 Two second-order effects you cannot ignore

Body effect

V_t is quoted for source tied to bulk. Lift the source above the bulk and the depletion region under the channel widens, so more gate voltage is needed before inversion: V_t rises. The dependence is $V_t = V_{t0} + \gamma(\sqrt{2\phi_F + V_{SB}} - \sqrt{2\phi_F})$, and γ is a process constant of order 0.2–0.5 $\sqrt{V}$.

The practical consequence: **a device whose source is not at the rail is weaker than the library says.**

Where it is used

Stacked devices in NAND/NOR — the upper NMOS has its source above ground. Pass-gate chains. Cascode current sources. Multiplexer trees built from transmission gates. Any place transistors are in series.

When it binds	Whenever you stack. A 4-input NAND is not simply 4× slower than an inverter — it is worse, because the upper devices are body-effected. This is one reason wide gates are decomposed into trees rather than built flat.
How to apply it	For hand analysis, add the V_t shift and recompute the overdrive of the upper device. For real sizing, let the characterised library handle it — the .lib delay tables already contain the effect — but know it is there so that a stacked structure you invent is not assumed to scale linearly. If the stack is deep, insert a buffer instead.

What the industry code looks like

```
// Why you will not find `body effect` in an RTL file, but will
// feel it: the synthesiser's cell choice reflects it.
// These two are logically identical and time very differently.

assign y = ~(a & b & c & d);           // maps to NAND4 – deep stack

wire ab = ~(a & b);                   // maps to NAND2 + NAND2 + NOR2
wire cd = ~(c & d);                   // shallower stacks, more cells
assign y = ~(~ab | ~cd) ? 1'b0 : 1'b1;

// Liberty shows the penalty directly. Compare the intrinsic delays:
//   NAND2_X1  cell_rise (A1) ~ 0.021 ns
//   NAND3_X1  cell_rise (A1) ~ 0.031 ns
//   NAND4_X1  cell_rise (A1) ~ 0.045 ns    <- worse than linear
```

This is where it goes wrong. Assuming a stack of n devices is n times slower. Body effect and the internal node capacitances make it super-linear, and above three or four the penalty grows fast. If a synthesised netlist contains NAND4s on a critical path, that is usually the first thing to restructure.

Gate capacitance — the load you are driving

The gate you drive is a capacitor: $C_g \approx C_{ox} \cdot W \cdot L$, plus overlap capacitance that does not scale with L . For 1.2 nm oxide this is of order 1.60×10^7 fF/ μm^2 of gate area.

This single fact ties the whole of digital design together: the output of one gate must charge the input capacitance of the next, so **delay is proportional to the ratio of load capacitance to drive strength**. That ratio is the *electrical effort* of T3, and it is why sizing is a chain problem rather than a per-gate problem.

There is also a term that bites in analog and in high-speed digital: the gate-to-drain overlap capacitance C_{gd} appears at the output *multiplied by the gain* of the stage (Miller effect). In a comparator pre-amp this can dominate the load; in a digital gate it is why the input capacitance seen during a transition is larger than the static value.

Where it is used	Every delay calculation. Clock-tree power (the clock drives enormous total gate capacitance, which is why clock gating is the first power optimisation). Input capacitance you quote in a datasheet so a customer can budget their driver.
When it binds	Always, in digital. Specifically when you write the input capacitance line of an IP datasheet, and when you decide how many loads one cell may drive.
How to apply it	Express loads in units of a minimum inverter's input capacitance and the arithmetic becomes tractable by hand — this is exactly what logical effort (T3) formalises. For the deliverable, state input capacitance per pin in the .lib and in the datasheet; integrators use it to size their drivers and to build the clock tree.

What the industry code looks like

```
// The datasheet line that comes from this, and the .lib it matches.
// If these two disagree, the customer's timing closure fails and
```

```
// they will find out before you do.

pin (D) {
    direction      : input ;
    capacitance     : 1.412 ;           // fF – what the integrator budgets
    rise_capacitance : 1.455 ;
    fall_capacitance : 1.369 ;
    max_transition  : 0.300 ;           // ns – beyond this, delay tables
}                                     // are extrapolated, not valid

// SDC the customer writes against your block:
set_load      0.0014 [get_ports eq_out*] ;# pF
set_driving_cell -lib_cell BUF_X4 [get_ports adc_data*]
set_max_transition 0.300 [current_design]
```

This is where it goes wrong. Letting a net exceed max_transition. Beyond that point the library's delay tables are being extrapolated, and the timing report is no longer characterised data — it is a guess with a confident number next to it. Treat a max-transition violation as a hard error, not a warning.

T1.7 Which region, for which job

Table 1. The four regions as four design tools

Region	Condition	The device behaves as	You use it for	The number that matters
Cutoff	$V_{GS} < V_t$	a leaky open switch	standby, retention, power gating	I_{off} and subthreshold swing S
Triode	$V_{GS} > V_t, V_{DS} < V_{ov}$	a gate-controlled resistor	pass gates, sampling switches, power switches	R_{on} , and how it varies with the signal
Saturation	$V_{GS} > V_t, V_{DS} > V_{ov}$	a voltage-controlled current source	amplifiers, comparators, current mirrors, CTLE	$g_m, r_o, g_m/I_D$
Sub-threshold	V_{GS} slightly $< V_t$	an exponential transconductor	always-on blocks, ultra-low-power, retention	S , and V_t mismatch

A digital cell spends almost all of its time in the first two rows and passes briefly through the third on every transition. That brief passage is not a detail: it is the short-circuit current of T4 and the regeneration of T13, and two of the hardest problems in this book — metastability and comparator resolution — are entirely arguments about what happens while a device sits in saturation instead of settling into triode or cutoff.

What actually happened

A production sampling switch was sized from its mid-scale R_{on} and met its settling target in simulation at every corner. Silicon showed second-harmonic distortion 12 dB worse than simulated. The cause was that the verification sweep held the input at mid-scale while checking settling, and swept the input only while checking gain — so the combination that mattered, *large input and tight settling*, was never simulated. Nothing in the design was wrong; the **verification grid** was wrong. T30 turns this into a rule: sweep the axes together, not one at a time.

Table 2. Where this chapter's concepts are actually used

Concept	Block it appears in	What it decides
Subthreshold swing S	every block's standby spec	the leakage number in your datasheet, and whether retention is possible

Concept	Block it appears in	What it decides
R_{on} and its signal dependence	sampling switch, pass gates, power switch	settling time, distortion, IR drop across the power switch
g_m/I_D	comparator pre-amp, CTLE, bias	gain and noise per unit of current — i.e. the power budget
Channel-length modulation λ	current mirrors, cascodes	how much the bias current moves with output voltage
Body effect γ	stacked logic, transmission-gate trees	why deep stacks are super-linearly slow
Gate capacitance C_g	every delay and clock-power calculation	the input capacitance line of your datasheet

Where the inventions are — what is patentable here

Device-level invention is largely closed to a fabless IP house — the foundry owns the device. **What is open is how you choose among the devices you are given.** Claims in shipping IP portfolios routinely read on *selection and arrangement*: a retention latch that uses high- V_t devices only on the state-holding node while the read path stays standard- V_t ; a level shifter whose pull-up stack is sized so that the contention current is bounded across corners; a sampling switch whose bootstrap network holds V_{GS} constant so that R_{on} is signal-independent. All three are device-region arguments, and all three are patentable as circuits. Read this chapter asking *which region am I in, and what does that buy me* — that question is where the claims come from.

T2. Delay — From One RC to a Whole Timing Path

Every delay number a tool reports is one of four models. Know which one, or you will trust a number that was never about your circuit.

Before you read this

voltage · current · resistance · differentiation · exponential functions · Ohm's law · threshold voltage · triode region · saturation region · on-resistance · gate capacitance · output resistance

New in this chapter

RC delay · Elmore delay · logical effort · electrical effort · parasitic delay · FO4 delay · slew · short-circuit current · wire delay · repeater · arrival time · required time · slack · on-chip variation · NLDM · non-linear delay model

Ask four engineers what the delay of an inverter is and you get four answers, all correct, because they are answering with four different models. A SPICE user answers with a differential equation. A physical-design engineer answers with a two-dimensional lookup table from a Liberty file. A microarchitect answers in FO4. A synthesis engineer answers with a dimensionless number that has no picoseconds in it at all. These are not competing approximations of one truth; they are four tools with different jobs, and using the wrong one is how a design gets signed off on a number that was never about the path that failed.

This chapter builds them in order, each from the one before, and at every step states the regime in which that model is the right one to use. The numbers throughout come from one set of process constants declared at the top of this chapter's source — a 28 nm-class low-power node at 0.9 V — and every table below is computed from them, not typed in.

T2.1 The one model you can do in your head: switched RC

A CMOS gate charges and discharges a capacitor through a transistor that is acting as a resistor. That is the whole of it. Everything else in this chapter is a refinement of that sentence, and every refinement exists because a specific assumption in it broke.

RC delay (RC지연) — A gate's output node is a capacitance C driven through an equivalent resistance R . The node's voltage is a decaying exponential, and the delay is the time to cross the receiver's switching threshold — conventionally 50 % of V_{DD} .

Derivation — From the transistor to 0.69 RC

	This follows	Because
1	A switching gate connects the output node either to V_{DD} through the PMOS stack or to ground through the NMOS stack.	That is what a static CMOS gate is: two complementary networks, exactly one of which conducts in steady state.
2	While it switches, the conducting transistor is not a resistor — it starts in saturation and ends in triode.	V_{DS} starts at V_{DD} and falls toward 0, so the device crosses regions during the transition (T_1).
3	We replace it anyway with a single equivalent resistance R_{eq} , defined as the value that gives the same 50 % crossing time.	This is a <i>fitted</i> quantity, not a measured resistance — which is why it differs between textbooks and why you never compare R_{eq} numbers across sources.
4	Then $V(t) = V_{DD} \cdot e^{-t/RC}$ for a falling output.	First-order linear ODE with a step excitation: the standard solution.
5	Setting $V = V_{DD}/2$ gives $t = RC \cdot \ln 2 = 0.69 RC$.	$\ln 2 = 0.693$. This is where the 0.69 in every delay formula comes from — it is the 50 % point, nothing more.
6	For 10–90 % rise/fall time the same solution gives $t = RC \cdot \ln 9 = 2.2 RC$.	Same exponential, different two crossings. 0.69 and 2.2 describe the same waveform — a delay and a slew are two readings of one curve.

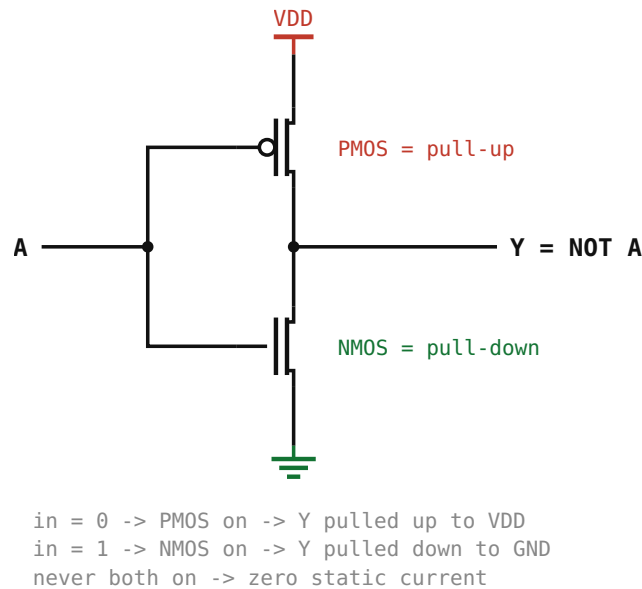


Figure 1. The unit inverter. The pull-up and pull-down networks are the R in RC; the next stage's gate capacitance and the wire are the C.

$\tau = R_{eq} \cdot C_g$ — the process's unit of time

Pick one reference device — the minimum-size inverter — and give it two numbers: an equivalent output resistance $R_{eq} = 8 \text{ k}\Omega$ and an input capacitance $C_g = 1 \text{ fF}$. Their product $\tau = 8 \text{ ps}$ is the only absolute time in the rest of this chapter. Every other delay will be quoted as a multiple of it.

This is not a cosmetic choice. Ratios between gates are set by topology and are stable across process, voltage and temperature to within a few per cent; the absolute τ moves by **2× or more** across corners. Separating the two is what lets you do architecture on paper and let the tool supply the picoseconds.

Where it is used	Every hand estimate of a critical path, every early architectural trade-off, every 'will this fit in one cycle' question before the RTL exists.
When it binds	Before synthesis — when there is no netlist to run STA on, and after STA, to check whether a reported number is even plausible.
How to apply it	Express the path in τ , then multiply once at the end. A useful anchor is FO4 — an inverter driving four copies of itself — which in this process is 40 ps. A modern high-performance pipeline stage is roughly 12–20 FO4; a low-power one, 25–40.

What the industry code looks like

```
// A 'timing shell' exists so that the number you measure is about
// your logic, not about the pads. Synthesise the DUT inside registers:
module fir_shell (input clk, input rst, input signed [7:0] din, output signed [15:0] dout);
    reg signed [7:0] din_q;
    reg signed [15:0] dout_q;
    wire signed [15:0] y;
    always @(posedge clk) begin din_q <= din; dout_q <= y; end
    fir4 u_dut (.clk(clk), .rst(rst), .x(din_q), .y(y));
    assign dout = dout_q;
endmodule
```

```
// Measured on this repository's own FIR: 455 MHz without the shell, 47 MHz with it.
// The 455 MHz was the tool timing a path that started and ended at a pin.
```

This is where it goes wrong. Quoting $F_{\max}$ from a design whose inputs and outputs are pins is the single most common false number in an IP datasheet. If a path has no launching and no capturing register, the tool has nothing to constrain and will report whatever combinational delay it found — often $10\times$ optimistic.

T2.2 Elmore delay — when one RC becomes a chain of them

The single-RC model assumes the resistance is at one end and the capacitance at the other. A wire violates both: its resistance and its capacitance are distributed along it. Cutting it into N lumps and summing gives the Elmore delay, and the sum has a shape worth memorising, because it is the reason long wires do not scale.

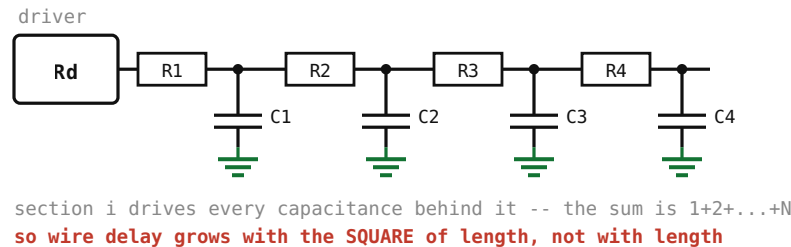


Figure 2. A wire cut into four RC sections. Section 1's resistance pushes the capacitance of sections 1–4; section 4's pushes only its own.

Derivation — Why wire delay grows with the square of length

	This follows	Because
1	Cut a wire of length L into N equal sections, each with resistance R/N and capacitance C/N .	$R = r \cdot L$ and $C = c \cdot L$, with r in $\Omega/\mu\text{m}$ and c in $\text{F}/\mu\text{m}$ from the technology file.
2	The charge for every capacitance downstream of section i must flow through section i 's resistance.	Charge has only one path to reach them — that is the whole content of the Elmore model.
3	So the delay is $\sum_i (R/N) \cdot (N-i+1)(C/N) = RC \cdot (N+1)/(2N)$.	Arithmetic series $1+2+\dots+N = N(N+1)/2$. Substituting gives the closed form.
4	As $N \rightarrow \infty$ this tends to $RC/2$, and R and C are each proportional to L .	Hence $t \propto r \cdot c \cdot L^2$ — doubling a wire's length quadruples its delay , while a gate's delay does not change at all.
5	For this process, a $1\text{ }\mu\text{m}$ wire gives $RC/2 = 90\text{ ps}$, and the four-lump approximation gives 112 ps .	The coarse lump overestimates by $1.25\times$. Use the closed form for the distributed case and lumps only when the sections are real (a via, a branch, a tap).

Table 3. Lumping a distributed wire: how many sections are enough

Sections N	Elmore delay	Error vs. distributed $RC/2$
1	18 ps	1 %
2	135 ps	50 %
4	112 ps	25 %
8	101 ps	12.5 %
16	95.6 ps	6.25 %
32	92.8 ps	3.12 %

Two facts hide in that table. First, **a single lump is 100 % wrong** — and a single lump is exactly what a naive spreadsheet does. Second, convergence is slow: even 8 sections is still 6 % high. This is why extraction tools do not lump at all; they solve the distributed network. Your hand estimate should use $RC/2$ and remember that it is a lower bound on what the extractor will report, because the extractor also sees coupling capacitance to neighbours.

Elmore delay

The first moment of the impulse response of an RC tree: for a node k , $t_k = \sum_i R_{ik} \cdot C_i$, where R_{ik} is the resistance shared by the path to node i and the path to node k . It is exact for a single RC, always an **upper bound** on the 50 % delay for an RC tree with a step input, and it ignores inductance entirely.

Where it is used	Clock-tree synthesis (balancing sinks), buffer and repeater insertion, and the delay model inside every placer's incremental timing engine — the placer cannot afford a full extraction per move.
When it binds	Whenever you need a delay that is monotone in the tree's structure and cheap to update. It is wrong by 10–30 % in absolute terms and right in its ordering, which is what an optimiser needs.
How to apply it	Walk the tree from the driver; accumulate the shared resistance for each sink. Never use it to sign off timing — it is an optimisation objective, not a signoff model.

What the industry code looks like

```
# Extraction produces this; you read it when a net is suspect.
# SPEF, standard parasitic exchange format, one net:
*D_NET net_dout_3 1.84e-01          $ total capacitance, pF
*CONN
*I u_fir/Z 0 *L 0 *D INVX4          $ driver
*I u_reg/D I *L 1.02e-03            $ receiver pin cap, pF
*CAP
1 net_dout_3:1 2.31e-02
2 net_dout_3:2 net_clk:7 4.10e-03   $ COUPLING to an aggressor -- not in your hand estimate
*RES
1 net_dout_3:1 net_dout_3:2 1.79e+01 $ ohms
```

This is where it goes wrong. Elmore is an upper bound *for a step input*. Real inputs have finite slew, and for a fast input into a long wire the true 50 % delay can exceed it. The bound you rely on is not the bound you were taught if the slew is large — which is the next section.

T2.3 Logical effort — delay with the picoseconds factored out

Sizing a chain of gates by RC arithmetic works and is miserable. Logical effort removes the process from the problem entirely, leaving a dimensionless expression you can optimise in your head, and it answers the two questions that actually come up: *how many stages*, and *how much bigger should each one be*.

Derivation — The normalised delay of one stage

	This follows	Because
1	Write a gate's delay as $d = (R_{\text{gate}} \cdot C_{\text{out}}) + (R_{\text{gate}} \cdot C_{\text{self}})$.	The output node carries the load <i>and</i> the gate's own drain capacitance; both are charged through the same resistance.
2	Divide through by $\tau = R_{\text{inv}} C_g$.	We choose to measure everything relative to the minimum inverter. Nothing is lost; the units are.
3	The first term becomes $g \cdot h$: logical effort g = how much worse this topology is than an inverter at driving current, and electrical effort $h = C_{\text{out}}/C_{\text{in}}$.	g depends only on topology (NAND2 = 4/3, NOR2 = 5/3, inverter = 1); h depends only on the ratio of load to input, so it is independent of absolute size.

	This follows	Because
4	The second term becomes p, the parasitic delay — the delay of the gate driving nothing.	It scales with the gate's own diffusion, which grows with its width exactly as its drive does — so p is size-independent. $p_{inv} \approx 1$.
5	$d = g \cdot h + p$.	Two multiplications and an addition, and no process constant anywhere.
6	For a path of N stages, the total is minimised when every stage carries the same stage effort $f = (G \cdot B \cdot H)^{1/N}$.	Differentiate the sum with respect to each stage's size and set to zero; the symmetric solution falls out. $G = \prod g$, $H = C_{load}/C_{in}$, and B accounts for branching off the path.

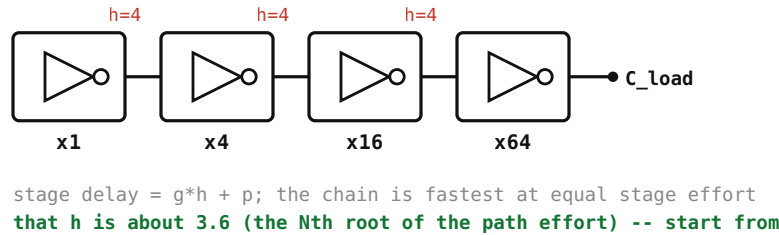


Figure 3. Four stages, each 4× the last. Equal stage effort is what makes the total minimal — not equal sizes, and not one huge final stage.

Worked example 3. Driving a 64× load: how many stages?

Given. A 1× inverter must drive a load of $64 \cdot C_g$ (an on-chip bus with several receivers). Logical effort of an inverter chain is $G = 1$, no branching so $B = 1$, and $H = 64$, giving path effort $F = 64$.

Method. Try each N, computing $D = N \cdot F^{1/N} + N \cdot p_{inv}$, and pick the smallest. The table below is computed, not recalled.

Answer. $N = 3$ stages, with stage effort $f = 4$ and total delay $15 \cdot \tau = 12$ ps. A single stage would take 52 ps — **4.33× slower**.

Table 4. Stage count versus total delay for $F = 64$

Stages N	Stage effort f	Delay (τ)	Delay
1	64	65	52 ps
2	8	18	144 ps
3	4	15	12 ps
4	2.83	15.31	123 ps
5	2.3	16.49	132 ps
6	2	18	144 ps

This is where it goes wrong. Reading the table's minimum as a sharp optimum. $N = 3$ and $N = 4$ differ by 2.1 % — inside the noise of any real p. Take the *cheaper* of the two, not the nominally faster one; the extra stage costs area, leakage and a slew you must then drive.

Table 5. Logical effort of the cells you will actually instantiate

Gate	g	p (τ)	Why g is what it is
Inverter	1	1	The reference
NAND2	4/3	2	Two series NMOS: each must be 2× wider for the same pull-down current, so the input cap rises to 4/3 per input
NAND3	5/3	3	Three in series — g grows linearly with stack depth

Gate	g	p (τ)	Why g is what it is
NOR2	5/3	2	Series PMOS, and PMOS is the weaker device; this is why NOR is avoided in fast logic and NAND is not
NOR3	7/3	3	Same reason, worse
XOR2	4	4	Both polarities of both inputs are needed internally
MUX2	2	4	Two transmission paths plus select drive
Tri-state inverter	2	2	The enable device is in series with the output — the classic bus driver

Logical effort, used as a design instrument

$d = g \cdot h + p$. Total path delay $D = \sum(g_i h_i) + \sum p_i$, minimised at equal stage effort $f = F^{1/N}$, with the practical rule that $f \approx 4$ (the true optimum for $p_{inv} = 1$ is 3.6) and an extra stage is worth adding only while it removes more than p of delay.

Where it is used	Deciding how deep a pipeline stage can be before synthesis; choosing between a NAND-NAND and a NOR-NOR implementation; sizing a clock or reset distribution by hand; arguing with a synthesis result that looks wrong.
When it binds	At architecture time, and every time a tool's answer needs a sanity check. Synthesis does this internally but with a cost function that also weighs area and power, so its answer will differ — knowing the ideal tells you whether the difference is a trade or a mistake.
How to apply it	Count G along the path, compute H from the real load, take $F = G \cdot B \cdot H$, then $N \approx \log_4 F$ and $f = F^{1/N}$. Work backwards from the load to size each stage: $C_{in,i} = g_i \cdot C_{out,i} / f$.

What the industry code looks like

```
# The synthesis constraint that makes the tool solve the same problem.
# Without a real load and a real driver, the tool optimises a fiction.
set_load          0.064          [get_ports dout*]    ;# pF -- from the receiver's .lib
set_driving_cell -lib_cell INVX4 [get_ports din*]     ;# not an ideal source
set_max_transition 0.150 [current_design]             ;# ns -- bounds slew degradation
set_max_fanout    16             [current_design]
# And the one that is almost always missing on a first pass:
set_max_capacitance 0.200 [current_design]
```

This is where it goes wrong. g is a property of the *topology*, not of the cell you picked from the library. A library's NAND2_X4 and NAND2_X1 have the same g; only C_{in} differs. Engineers who 'fix timing' by swapping to a bigger cell without changing the stage count are changing h for the previous stage — they move the problem upstream, and on a long chain it comes back.

T2.4 Slew, and why a delay has two arguments

Both models so far take one input: the load. Every real timing tool takes two, because the delay of a gate depends on how fast its *input* moved. A slow input keeps both transistors partially on, wastes current through the stack, and starts the output late. That is the single largest correction to everything above, and it is why a Liberty file holds a table rather than a formula.

slew / transition time (슬루) — The time for a signal to cross between two defined fractions of V_{DD} , conventionally 10 %–90 % or 30 %–70 %. **Which pair is a property of the library, and mixing libraries that use different pairs silently changes every delay in the design.**

short-circuit current (단락전류) — During a transition both the pull-up and pull-down networks conduct simultaneously, shorting V_{DD} to ground through the stack. The charge lost this way is proportional to the input transition time; at normal slews it is 5–15 % of switching energy, and at degraded slews it can exceed the useful switching energy.

Table 6. The same inverter, four different 'delays' — an NLDM slice

Input slew (ps)	Load 4 fF	Load 16 fF	Load 64 fF
20	16 ps	40.4 ps	137 ps
60	24 ps	49.2 ps	147 ps
120	36 ps	62.4 ps	162 ps
240	60 ps	88.8 ps	192 ps

That table is a model, not a measurement — the slew coefficients are illustrative. **Say so.** A real NLDM table is characterised by running SPICE at every grid point over the full corner set, typically 7×7 points per arc, per cell, per corner. What is real in the table is its shape: delay rises roughly linearly in load and roughly linearly in input slew, and **the slew term is not small** — at the light load it is the dominant term.

NLDM — the non-linear delay model, and what a tool actually reads

A Liberty (.lib) file gives, for each timing arc of each cell, two 2-D tables — one for delay and one for output slew — indexed by input transition time and total output capacitance. The tool bilinearly interpolates (and, at the edges, **extrapolates**) to get a number. Everything downstream — STA, synthesis cost, placement timing — is built on these tables.

Its successor, CCS (composite current source), stores a current waveform rather than a scalar, which matters when the receiver's input capacitance is itself voltage-dependent or when the net's resistance is large enough that the driver and the net interact. For an IP deliverable, **state which model your numbers came from**; they are not interchangeable at the 5 % level.

Where it is used	Every delay number in synthesis, STA, and place-and-route. Also in the IP datasheet, indirectly: your F_{max} is only meaningful with the library and corner named.
When it binds	You read a .lib directly in exactly two situations: when a path's reported delay makes no sense and you want to see whether the tool extrapolated off the table, and when you are asked to support a library you have never seen.
How to apply it	Find the arc, read index_1 (input slew) and index_2 (load), and check that your operating point is <i>inside</i> both. Off-table extrapolation is the standard cause of 'the tool says it passes and silicon says it does not'.

What the industry code looks like

```
/* Liberty, one arc of one cell -- this is what the tool reads. */
cell (INVX1) {
    area : 1.596 ;
    pin (A) { direction : input ; capacitance : 1.02 ; }      /* fF */
    pin (Z) {
        direction : output ;
        function : "!A" ;
        max_capacitance : 64.0 ;
        timing () {
            related_pin : "A" ;
            timing_sense : negative_unate ;
            cell_fall (delay_7x7) {
                index_1 ("0.004, 0.012, 0.028, 0.060, 0.124, 0.252, 0.508"); /* input slew, ns */
                index_2 ("0.4, 1.6, 4.0, 8.8, 18.4, 37.6, 76.0");           /* load, fF */
                values ( "0.0091, 0.0142, 0.0241, 0.0431, 0.0812, 0.1574, 0.3098", \
```

```

    "0.0113, 0.0164, 0.0263, 0.0453, 0.0834, 0.1596, 0.3120", \
    /* ... five more rows, one per input slew ... */ );
}
fall_transition (delay_7x7) { /* the OUTPUT slew -- this feeds the next stage */ }
}
}
}

```

This is where it goes wrong. Delay and output slew are two *separate* tables, and the output slew is the next stage's input slew. This is how one badly driven net poisons a whole path: each stage receives a worse slew, produces a worse slew, and the degradation compounds. **set_max_transition exists to stop that compounding**, and a design with no max_transition constraint can meet setup in the report and fail in silicon.

T2.5 The wire, and the repeater that saves it

Gate delay shrinks with each process node. Wire delay does not: r rises as wires get thinner, c stays roughly constant per micron, and the die does not shrink as fast as the transistors. Beyond a length that has been getting shorter every node, a wire dominates the path, and the fix is not a bigger driver — it is to stop the wire being long.

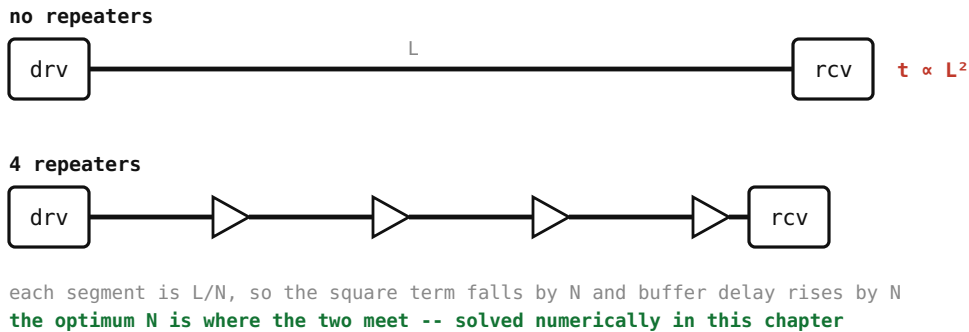


Figure 4. A long wire, and the same wire cut into segments by repeaters. The quadratic term shrinks by N ; the buffer delay grows by N .

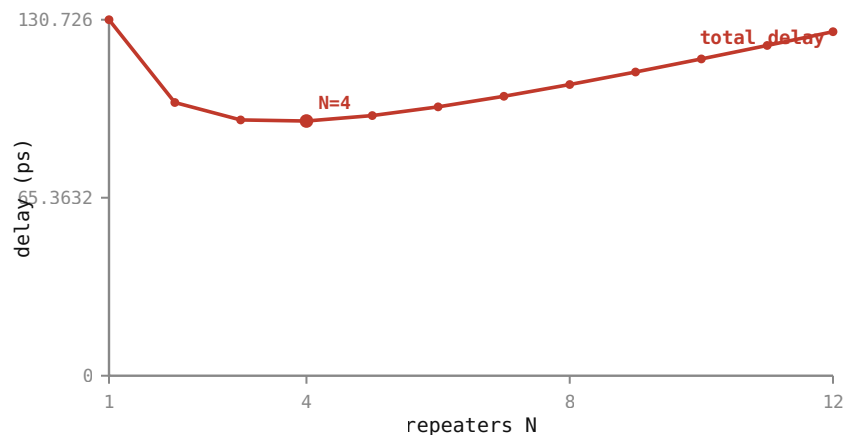


Figure 5. The full sweep for a $1\text{ }\mu\text{m}$ wire, each point at its own optimal repeater size. **The minimum is shallow** — $N = 3, 4$ and 5 are within 2.1 % of each other — so take the one that routes.

Derivation — Where the optimum comes from, and a check on it

	This follows	Because
1	Cut the wire into k segments, each driven by an inverter of size h .	One repeater per segment; the last one drives the receiver.
2	Each segment costs $0.7 \cdot (R_{inv}/h)(C_w/k + h \cdot C_g)$ for the driver, $0.4 \cdot R_s C_s$ for the distributed wire, and $0.7 \cdot R_s \cdot h C_g$ for the far-end load.	The 0.4 and 0.7 are the standard distributed-RC coefficients — 0.4 for a wire driving its own capacitance, 0.69 for a lumped load.
3	Sweeping k and h numerically gives 93.5 ps at $k = 4$, $h = 38$.	Brute force over the whole grid — no assumption about the shape of the optimum.
4	The closed forms $k = \sqrt{(0.4 R_w C_w / 0.7 R_{inv} C_g)}$ and $h = \sqrt{(R_{inv} C_w / R_w C_g)}$ give $k = 3.59$, $h = 37.9$.	Independent cross-check. Two routes to the same answer, agreeing to 0.1 % in size. A single derivation that no one checked is how the wrong constant survives for years.
5	Without repeaters, the same wire with its own best driver costs 131 ps.	So repeaters buy 1.4× here — and that is the honest headline , not a figure taken from the most favourable corner of the sweep.

Repeater insertion

Break a wire whose delay is quadratic in length into segments short enough that the quadratic term is comparable to a buffer delay. The optimum segment length is the one where **the wire's RC per segment equals the buffer's own delay**; past that, you are paying buffers to save nothing.

For this process the crossover is around a wire of 25 μm — shorter than that and a repeater costs more than it saves.

Where it is used	Any net that leaves a block: block-to-block datapaths, clock spines, reset distribution, and the top-level assembly of an SoC where your IP's ports land.
When it binds	Decided in physical design, but constrained in your deliverable : if your IP's output has 200 ps of budget and the integrator's wire needs 300, the block fails through no fault of its RTL.
How to apply it	Give the integrator a number, not a hope: state output delay, drive strength, and maximum load in the datasheet, and constrain both ends with <code>set_output_delay / set_load</code> . Then the repeater problem is theirs and is solvable.

What the industry code looks like

```
# What the integrator must be told, and what you must constrain.
# In your IP's SDC (shipped with the deliverable):
create_clock -name clk -period 4.000 [get_ports clk]
set_output_delay -max 1.200 -clock clk [get_ports {dout[*] dvalid}]
set_input_delay -max 1.000 -clock clk [get_ports {din[*] dvalid_i}]
set_load 0.064 [get_ports dout[*]]      ;# what we assumed -- say it in the datasheet
# And in DATASHEET.md, the line that prevents the support call:
#   Output timing assumes <= 64 fF load and <= 150 ps transition at the port.
#   Beyond that, insert a repeater; every 100 fF adds approximately 25 ps.
```

This is where it goes wrong. Repeaters are inverting. An odd number of them flips the polarity of the net, and the fix — using buffers (two inverters) instead — costs more delay than the inverter chain. Tools handle this, but **hand-placed repeaters in a reset or scan-enable path are a classic bring-up bug**: the block resets on the wrong polarity at one corner only, because that corner is the one where the inserted pair was optimised away.

T2.6 From cells to a path: what STA actually adds up

Static timing analysis does not simulate. It propagates two numbers through the graph — the earliest and latest time a signal can arrive — and compares them against what each endpoint requires. Everything above supplies one edge of that graph; this section is how the edges become a verdict.

Derivation — Setup and hold, as one inequality each

	This follows	Because
1	A launching flop puts data out at $t_{\text{launch}} = T_{\text{clk,launch}} + t_{\text{cq}}$.	The clock arrives at the launching flop at its own time — clock delay is not the same at both ends of a path.
2	It arrives at the capturing flop at $t_{\text{arrival}} = t_{\text{launch}} + t_{\text{logic}}$.	t_{logic} is everything this chapter has been computing.
3	The capture edge occurs at $T_{\text{clk,capture}} + T_{\text{period}}$ and the data must be stable t_{setup} before it.	That stability requirement is a property of the flop, characterised the same way as any other arc.
4	Setup: $t_{\text{arrival}} \leq T_{\text{capture}} + T - t_{\text{setup}} - t_{\text{uncertainty}}$. Slack is the difference.	Clock uncertainty absorbs jitter and the skew the tool cannot yet know (pre-CTS). Positive slack passes; negative fails.
5	Hold: the <i>fastest</i> path must not arrive before the same capture edge's hold window: $t_{\text{arrival,min}} \geq T_{\text{capture}} + t_{\text{hold}}$.	Hold has no T in it. Slowing the clock does not fix a hold violation — this is the single most useful fact in the chapter for a first-time designer.

Table 7. The four ways a path's numbers get worse than your estimate

Effect	What it does	Typical size	Where it is set
OCV / AOCV / POCV	Derates cell and net delays differently on the launch and capture clock paths, because two identical cells on the same die are not identical	3–10 % of the clock path	set_timing_derate
Clock uncertainty	A flat subtraction standing in for jitter and, pre-CTS, for skew	50–150 ps	set_clock_uncertainty
CPPR / CRPR	Gives back the derate double-counted on the shared part of the launch and capture clock paths	Recovers 10–50 ps	on by default in signoff STA; often off in synthesis
SI / crosstalk delay	An aggressor switching the other way stretches your transition	5–15 % on long parallel nets	SI-aware STA with coupling from SPEF

Reading report_timing without being misled

A timing report is a ledger: clock path to the launch flop, cell and net delays along the data path, clock path to the capture flop, library setup, and the derates. **The one line that decides whether the report is meaningful is the corner and mode it was run in** — a report from the typical corner with ideal clocks is not evidence of anything.

Where it is used	Every timing signoff, every 'why is this failing' investigation, and every F_{max} number you put in a datasheet.
When it binds	Read the whole path when slack is negative; read the <i>startpoint and endpoint</i> first when it is positive, to check that the tool timed the path you think it did. False paths and unconstrained endpoints show up here and nowhere else.
How to apply it	Check: (1) is the endpoint a register, not a port; (2) is the clock real (propagated) or ideal; (3) what is the input transition at the worst cell — if it is at the end of the .lib index range, the number is extrapolated; (4) how much of the path is net delay. Above ~40 % net delay, the fix is placement, not logic.

What the industry code looks like

```

# Startpoint: u_fir/acc_reg[7] (rising edge-triggered flip-flop clocked by clk)
# Endpoint:   u_fir/y_reg[15] (rising edge-triggered flip-flop clocked by clk)
# Corner: ss_0p81v_125c    Check: setup    Path Group: clk
#   Point                                Incr      Path
#   clock clk (rise edge)                0.000      0.000
#   clock network delay (propagated)      0.412      0.412    <- real CTS, not ideal
#   u_fir/acc_reg[7]/CK (DFFX1)          0.000      0.412 r
#   u_fir/acc_reg[7]/Q (DFFX1)           0.138      0.550 f
#   u_fir/U231/Z (NAND2X2)                0.061      0.611 r
#   u_fir/U248/Z (INVX4)                  0.029      0.640 f
#   net (fanout=6, length=214um)          0.087      0.727    <- 12% of the path is WIRE
#   u_fir/y_reg[15]/D (DFFX1)            0.000      0.727 f
#   data arrival time                    0.727
#   clock clk (rise edge)                4.000      4.000
#   clock network delay (propagated)      0.438      4.438
#   clock reconvergence pessimism         0.021      4.459    <- CPPR gives this back
#   library setup time                   -0.096      4.363
#   data required time                   4.363
#   slack (MET)                           3.636

```

This is where it goes wrong. clock network delay (ideal) in that report instead of (propagated) means no clock tree exists yet and the skew is a guess. Numbers from an ideal-clock report routinely improve by 200–400 ps at signoff — in the wrong direction. **Never put an ideal-clock $F_{\max}$ in a datasheet.**

T2.7 What to do with this on Monday

Table 8. Where this chapter's concepts are actually used

Concept	Block it appears in	What it decides
RC delay · τ · FO4 delay	Any block, before RTL exists	Whether the function fits in one cycle at the target frequency
Elmore delay	Clock tree, reset tree, long datapath nets	Where a buffer must go, and which sink is the late one
logical effort	Critical-path restructuring, pipeline depth	Stage count and the size ratio between stages
slew · NLDM	Reading .lib, debugging an implausible delay	Whether the tool interpolated or extrapolated your operating point
wire delay · repeater	Block boundaries and top-level assembly	The output-load and transition limits you publish in the datasheet
arrival time · required time · slack	Every signoff	Whether the number you are about to sell is real

What actually happened

This repository measured its own FIR filter at **455 MHz** and then at **47.4 MHz** — a factor of 9.6 — from the same RTL. The difference was a timing shell. Without registers at the boundary, the tool reported the delay of a path from a pin to a pin, which is not a path that exists in a chip. Every number in this chapter is only as meaningful as the path it was measured on, and the cheapest way to be wrong by 10× is to time the wrong path.

The next chapter takes the one case this one cannot handle: what happens when the setup inequality is violated not by a slow path but because the data was never synchronous with the clock at all. The answer is not a longer delay but a probability, and it is the only timing quantity in this book that can never be made zero.

Where the inventions are — what is patentable here

The delay of a gate is physics and is not patentable. **The way a block arranges itself so that a delay stops mattering is.** Look at what the claims in shipping IP actually cover: a speculative (unrolled) decision-feedback equaliser that moves a feedback multiply-add off the critical loop and replaces it with a precomputed mux — the arithmetic is unchanged, the loop delay is not; a carry-select adder that buys delay with area by computing both carries; a clock-tree scheme that deliberately skews two launch points so that a slow path borrows time from a fast one. In every case the invention is *where the delay was moved*, not how fast a transistor is. That is why this chapter spends as much time on *which term dominates* as on the formulae: you cannot move a term you have not identified.

T3. Metastability — the One Failure You Cannot Design Away

Every other timing quantity can be made to pass. This one can only be made improbable, and you must say how improbable.

Before you read this

voltage · current · differentiation · exponential functions · logarithms · probability · mean · transconductance · output resistance · gate capacitance · RC delay · arrival time · required time · slack · FO4 delay

New in this chapter

metastability · resolution time constant · resolution time · metastability window · MTBF · synchroniser depth · multi-bit crossing · reconvergence · gray code · handshake · asynchronous FIFO · reset synchroniser · recovery time · removal time · CDC verification · quasi-static signal

A setup violation is a scheduling problem: the data was late, and if you slow the clock or shorten the logic, it is fixed. Metastability is not that. When a flip-flop samples an input that is changing at the moment of the clock edge, the output can sit between the two legal levels for an unbounded time. No amount of slowing down removes it — the sampling instant and the data transition are uncorrelated, so the bad coincidence will happen again. **The only question is how often, and whether you have budgeted for it.**

This chapter answers that question with a number, then spends the rest of its length on the more common failure: engineers who compute the number correctly, put in a two-flop synchroniser, and still ship a chip that hangs — because the thing that crossed the boundary was not one bit.

T3.1 Why a decision can take unbounded time

Derivation — From the cross-coupled pair to an exponential

	This follows	Because
1	A flip-flop's storage element is two inverters in a loop: the output of each drives the input of the other.	That loop is what holds state with no clock — it is the only structure in digital design that has memory without a capacitor being explicitly named.
2	The loop has three equilibria: a solid 0, a solid 1, and a point in the middle where both inverters sit at their own switching threshold.	Plot V_{out1} vs V_{in1} and the mirrored curve for the second inverter; they cross three times. The middle crossing is the metastable point.
3	The middle equilibrium is unstable : the loop gain there is $A^2 > 1$, so any deviation grows.	Each inverter has small-signal gain $A = -g_m(r_{o,n} \parallel r_{o,p})$ at its threshold, and T1 gave both terms.
4	Linearise around it: $C \cdot dv/dt = (A - 1) \cdot v/R$, so $v(t) = v_0 \cdot e^{t/\tau}$ with $\tau = RC/(A - 1)$.	This is where τ, the decision time constant, comes from . It is a circuit property — roughly the loop's RC divided by its excess gain — and it is <i>not</i> a specification you can choose.
5	To resolve to a valid level the loop must grow the deviation from v_0 to about $V_{DD}/2$, which takes $t = \tau \cdot \ln(V_{DD}/2v_0)$.	The time is logarithmic in how close the sample landed to the centre — and v_0 can be arbitrarily small, so t is unbounded. There is no worst case to design for, only a distribution.
6	The probability that a sample lands within a window that needs more than t_r to resolve falls as $e^{-t_r/\tau}$.	Uniform arrival within the clock period, and v_0 proportional to how close the edge was, give the exponential directly.

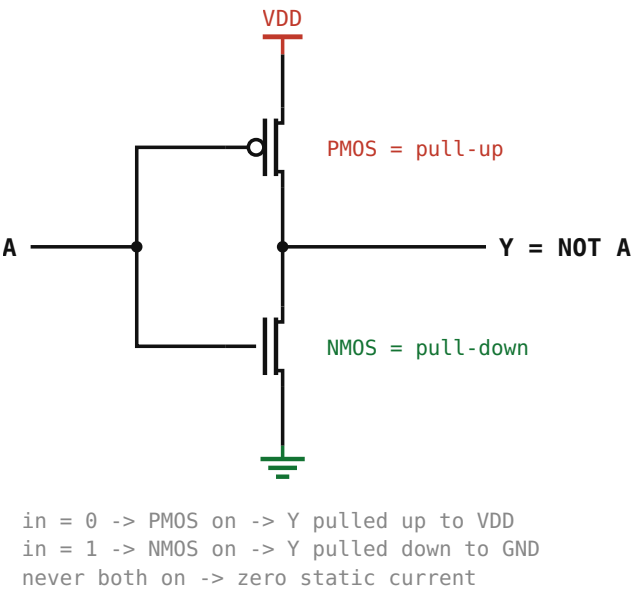


Figure 6. One of the two inverters in the loop. At the metastable point it sits at its own switching threshold with both devices in saturation — the highest-gain, highest-current, least-stable operating point it has.

Two consequences fall out of that derivation and both matter in practice. **First, τ is a small-signal quantity**, so it degrades exactly where gain degrades: slow corner, low voltage, high temperature. **Second, the metastable point is where the flop burns the most current** — both devices are on. A design that goes metastable often does not just fail logically; it draws current it was not budgeted for.

T3.2 MTBF, and what every symbol in it actually is

MTBF (mean time between failures) — The average time between two events in which a synchroniser fails to resolve within the time allowed. **It is a rate, not a guarantee** — a 1000-year MTBF does not mean the first failure is in year 1000; it means the failures are Poisson with that mean.

The MTBF expression

$$MTBF = e^{t_r/\tau} / (T_0 \cdot f_c \cdot f_d)$$

t_r — resolution time: the time the synchroniser flop is given to settle before anything reads it. In a single-stage crossing that is T_{clk} minus the flop's own t_{cq} , minus the setup of whatever reads it, minus any logic in between. **τ** — decision time constant, from T3.1, here 25 ps. **T_0** — the effective width of the aperture in which a sample can go metastable, here 1 ps; both are measured per cell per corner, not derived. **f_c** — the sampling (destination) clock frequency. **f_d** — the rate at which the asynchronous input changes.

Only t_r is in the exponent, and that is the whole engineering content of this formula: **everything you can do is linear, and the thing you cannot control is exponential**. Doubling f_d halves MTBF. Adding one synchroniser stage *multiplies the exponent*.

Where it is used	Every asynchronous boundary in the design: an external interrupt, a reset release, a pushbutton, a FIFO pointer crossing between clock domains, a PHY's recovered-clock domain meeting the core's.
When it binds	At architecture time you pick the depth; at signoff you state the number. An IP datasheet that claims 'CDC safe' without an MTBF and the conditions it was computed at has claimed nothing.

How to apply it

Get τ and T_0 from the library's synchroniser cell characterisation (they are published per corner). Compute t_r from the real path, not the ideal clock period. Then state MTBF at the **slow corner**, because τ is worst there.

What the industry code looks like

```
// The synchroniser, written so the tools recognise it.
// The attribute is not decoration: it tells the tool to keep the two flops
// adjacent and out of retiming, and it tells CDC checkers this is intentional.
(* ASYNC_REG = "TRUE" *) reg [1:0] sync_q;          // Xilinx; Intel: altera_attribute
always_ff @(posedge clk_dst or negedge rst_n) begin
    if (!rst_n) sync_q <= 2'b00;
    else        sync_q <= {sync_q[0], async_in};    // 2-stage
end
wire safe_in = sync_q[1];

// And the constraint, without which STA tries to time an impossible path:
// set_max_delay -datapath_only 2.0 -from [get_pins u_src/*/Q] -to [get_pins u_sync/sync_q_reg[0]/D]
// NOT set_false_path -- a false path lets the tool put unbounded wire delay there,
// and then the first stage's own setup window is no longer what you assumed.
```

This is where it goes wrong. Using `set_false_path` on a crossing is the single most common CDC constraint error. It silences the timing report, but it also removes any bound on the wire delay into the synchroniser — so the source can change arbitrarily close to the destination edge *and* the skew between the two bits of a bus becomes unbounded. Use `set_max_delay -datapath_only`.

Table 9. MTBF against clock frequency and synchroniser depth ($\tau = 25$ ps, $T_0 = 1$ ps, $f_d = f_c/10$, overhead 15 ps)

f_clk	t_r per stage	1 stage	2 stages	3 stages
0.5 GHz	1,85 ps	1.7e+18 yr	2.4e+50 yr	3.3e+82 yr
1 GHz	85 ps	1.85 yr	1.1e+15 yr	6.3e+29 yr
2 GHz	35 ps	9.53×10 ⁻¹⁰ yr	0 yr	1,378 yr

Worked example 4. Why two stages is the default, and where it stops being enough

Given. A 1 GHz destination clock, an input that changes at 100 MHz, and 15 ps of overhead per stage, so $t_r = 85$ ps.

Method. Put the numbers into the expression once per depth. The exponent scales with the number of stages because each stage gets its own full t_r .

Answer. One stage: 1.85 yr — **a failure every couple of years, per crossing**. Two stages: 1.1e+15 yr. Three stages: 6.3e+29 yr.

Table 10. The same two-stage crossing at 1 GHz, as τ degrades

τ	MTBF
1×	1.08e+15 yr
1.2×	1.34e+09 yr
1.5×	1.54e+05 yr
2×	1.85 yr

This is where it goes wrong. Reading the two-stage number as a property of the design rather than of the corner. Recompute it with τ merely 2× worse — a plausible slow-corner, low-voltage degradation — and the same two-stage crossing gives **1.85**

yr. The exponent collapsed. **MTBF is exponentially sensitive to the one parameter you do not control**, which is why the number must be quoted at the slow corner and why the third stage exists.

At 2 GHz the table says something worth pausing on: a **single**-stage crossing has an MTBF of 0.0301 seconds. Not years — seconds. To reach 100 years with one stage at that frequency you would need $t_f = 984.4$ ps, which is longer than the clock period itself (500 ps). **The requirement is not achievable in one cycle at all** — this is the regime where depth stops being a safety margin and becomes the only way the circuit works.

T3.3 A synchroniser is not a CDC solution

Everything so far concerns one bit. The failures that actually reach silicon almost never come from a wrong MTBF; they come from designers who synchronised each bit of a bus correctly and still lost.

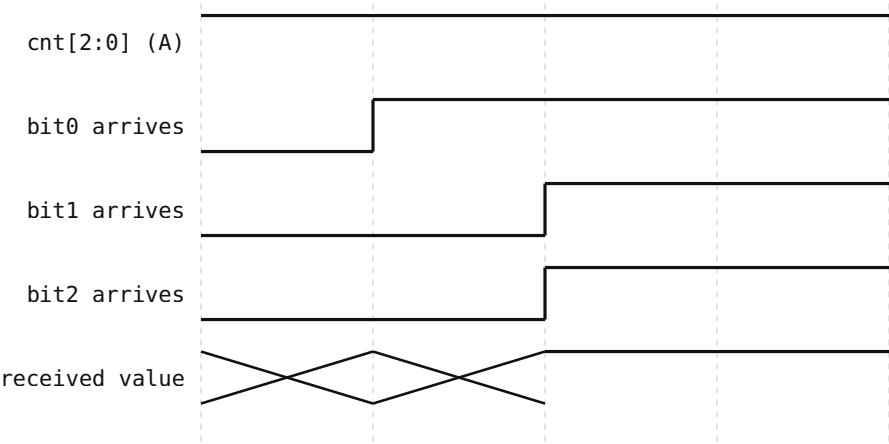


Figure 7. Two bits crossing through their own synchronisers. Each bit is individually safe. The pair is not: one resolves this cycle, the other next cycle, and the receiver sees a value that was never sent.

The multi-bit problem, and the three ways out

Independent synchronisers make independent decisions. If two bits change in the same source cycle, each may resolve on either side of the boundary, so the destination can observe an intermediate value — **a code that the source never drove**. A binary counter crossing 0111 → 1000 can be seen as 1111 or 0000 or anything between. No MTBF calculation detects this: every flop behaved correctly.

Where it is used	Any bus, counter, pointer, status word or configuration field that crosses a clock boundary. In practice: FIFO pointers, interrupt status registers, and 'mode' fields written by a slow configuration domain and read by a fast datapath.
When it binds	The moment more than one bit crosses. The three-bit case is not safer than the thirty-two-bit case — it is the <i>simultaneity</i> that matters, not the width.
How to apply it	Pick one: (1) Gray code — only one bit changes per increment, so an intermediate sample is either the old or the new value, both valid. Works for counters and pointers, not for arbitrary data. (2) Handshake — hold the data stable and cross a single request bit; the receiver acknowledges. Costs round-trip latency, works for anything. (3) Asynchronous FIFO — a dual-port memory with gray-coded pointers; this is (1) and (2) combined and is what you ship for streaming data.

What the industry code looks like

```
// Gray-coded pointer crossing -- the standard async FIFO core.
// The pointer is converted to gray BEFORE crossing and back AFTER.
function automatic [PW:0] bin2gray(input [PW:0] b); return b ^ (b >> 1); endfunction
```



```

always_ff @(posedge wclk or negedge wrst_n)
    if (!wrst_n) {wbin, wgray} <= '0;
    else          {wbin, wgray} <= {wbin_next, bin2gray(wbin_next)};

// crossing: gray value only, two stages, one synchroniser per bit is now SAFE
// because at most one bit differs between consecutive gray values.
(* ASYNC_REG = "TRUE" *) reg [PW:0] wq1_rgray, wq2_rgray;
always_ff @(posedge wclk or negedge wrst_n)
    if (!wrst_n) {wq2_rgray, wq1_rgray} <= '0;
    else          {wq2_rgray, wq1_rgray} <= {wq1_rgray, rgray};

// FULL is computed from the SYNCHRONISED read pointer, so it is CONSERVATIVE:
// it can be asserted when the FIFO has already been drained, never the reverse.
assign wfull = (wgray_next == {~wq2_rgray[PW:PW-1], wq2_rgray[PW-2:0]});

```

This is where it goes wrong. Gray code protects the *pointer*, not the *data*. The data path in an async FIFO is a dual-port RAM, and it is safe only because the read address is never allowed to reach a location the write side has not finished writing — that is what the conservative full/empty comparison buys. Copying the gray-code trick onto a data bus does nothing: consecutive data words differ in many bits.

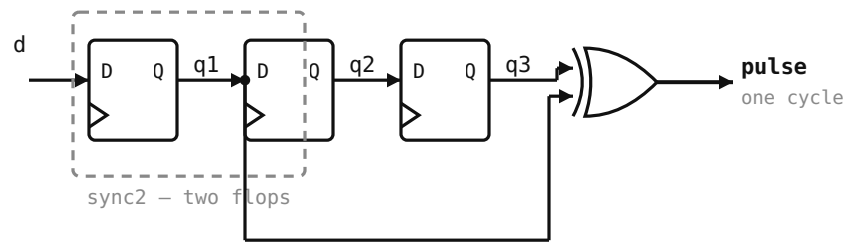
What actually happened

A crossing can also fail with **zero** metastability events. If a source pulse is shorter than one destination clock period, the destination may never sample it — the synchroniser resolves cleanly to the old value, every time. This is not a probabilistic failure; it is a guaranteed one, and it is invisible to MTBF analysis. The rule that prevents it: **a signal crossing into a slower domain must be held for at least one and a half destination periods**, or converted to a level-toggle with a handshake back.

Table 11. What each crossing style costs and what it protects

Style	Latency (dst cycles)	Throughput	Protects	Fails when
2-flop synchroniser	2	1 bit / cycle	One bit, level-stable	More than one bit changes together; pulses shorter than 1.5 cycles
Pulse synchroniser (toggle)	2–3	1 event / 3 cycles	Single events across any ratio	Events arrive faster than the round trip
Gray pointer	2	1 increment / cycle	Monotone counters and pointers	Used on arbitrary data
Req/ack handshake	4–6 round trip	1 word / round trip	Arbitrary data of any width	Latency budget cannot absorb the round trip
Async FIFO	2–3 + fill	1 word / cycle sustained	Streaming data, rate mismatch	Depth is too small for the burst pattern
Quasi-static (config field)	0	written once	Fields that change only while the consumer is idle or in reset	Someone writes it while the datapath is running

quasi-static signal (준정적신호) — A crossing signal that is **guaranteed by protocol** to change only while the destination is not using it — typically a configuration field written before the block is enabled. It needs no synchroniser, but it does need **a written rule in the datasheet** and a CDC waiver that says so. An undocumented quasi-static assumption is how an IP works in the lab and fails in the customer's system.



Different input-output levels across one flop → exactly one cycle of dif

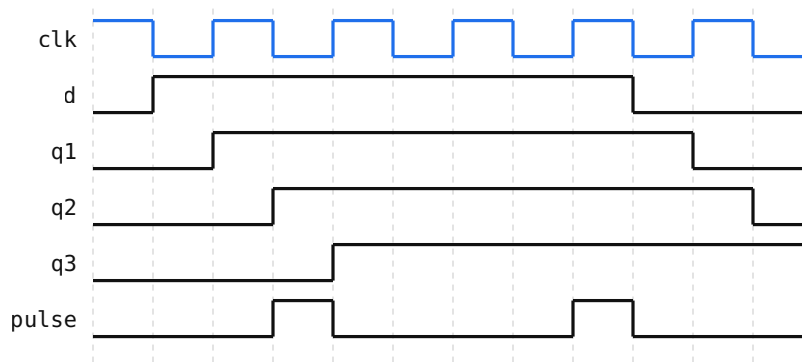
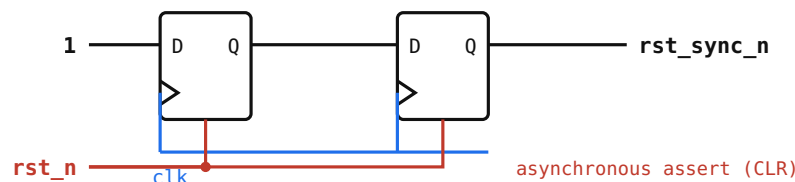


Figure 8. The standard way to carry a one-cycle event across a clock boundary. Two flops remove metastability; the third holds the previous value, and the XOR of the two turns any level change into exactly one pulse in the destination domain. The waveform is the proof: a wide pulse in and a single-cycle pulse out, on both edges.

T3.4 Reset: the crossing everyone forgets

Reset is asynchronous by nature — it must work before any clock is running — and synchronous by necessity, because releasing it at the wrong instant violates the flop's recovery time and puts the state machine in a metastable state on its first cycle. The resolution is the same for every design and is worth memorising as a single sentence: **assert asynchronously, release synchronously**.



assert is immediate -- no clock needed
release is synchronous (two clocks later) -- avoids recovery/removal violations

Figure 9. The reset synchroniser. The asynchronous assertion path has no clock in it; the release path runs through two flops clocked by the destination domain.

recovery and removal (리커버리 · 리무벌)

Recovery time is the setup constraint on reset **de-assertion**: reset must be released at least t_{recovery} before the active clock edge. Removal time is the corresponding hold constraint. They are characterised like any other timing arc and appear in the .lib — which means STA checks them **only if the reset is a timed, synchronous signal at that flop**.

An asynchronous reset that is released without a synchroniser has no such relation to the clock, so STA reports nothing, and the failure appears as a state machine that occasionally starts in an illegal state after power-up — typically at one corner, on some boards, after some power ramps.

Where it is used	Every clock domain. A design with three clocks needs three reset synchronisers, each clocked by its own domain and all fed by the same asynchronous source.
When it binds	Always. The cost is two flops per domain; the failure it prevents is the hardest class of bug to reproduce.
How to apply it	Assert straight through (no clock), release through two flops. Then constrain it: the release is now a synchronous signal and STA will check recovery/removal on every flop it reaches.

What the industry code looks like

```
// Reset synchroniser -- one per clock domain.
module reset_sync (input clk, input arst_n, output rst_n);
    (* ASYNC_REG = "TRUE" *) reg [1:0] q;
    always_ff @(posedge clk or negedge arst_n)
        if (!arst_n) q <= 2'b00;          // ASSERT: asynchronous, no clock needed
        else        q <= {q[0], 1'b1};    // RELEASE: two clocked stages
    assign rst_n = q[1];
endmodule

// SDC: the asynchronous assertion is not a timed path; the release is.
set_false_path -from [get_ports arst_n] -to [get_pins u_rsync/q_reg[0]/CLR]
// and every flop fed by rst_n gets a normal recovery/removal check -- verify with:
// report_timing -delay_type max -check_type recovery -from [get_pins u_rsync/q_reg[1]/Q]
```

This is where it goes wrong. Resetting the synchroniser's own flops with the *synchronised* reset is a loop with no way out: the block never leaves reset if the release edge is missed. The synchroniser must take the **raw asynchronous** reset on its clear pin. This is also why the two flops in a reset synchroniser reset to 0 and shift in 1, rather than the other way round.

T3.5 What STA cannot check, and what a CDC tool does instead

Table 12. Two tools, two questions

Question	STA	CDC analysis
Does this path meet setup at the given clock relation?	yes	no
Is there a clock relation at all?	assumes one	this is its job
Does every crossing have a synchroniser?	no	yes (structural)
Do multiple bits cross together?	no	yes — reconvergence and bus-coherency checks
Is the source stable long enough?	no	yes — protocol checks, usually as generated assertions in simulation
Is this crossing intentionally unsynchronised?	no	only if you waive it , by name, with a reason

The waiver file is the part of CDC signoff that decides whether the analysis meant anything. A tool run with two hundred waivers and a green summary is not evidence; it is two hundred assertions by an engineer. **An IP deliverable should ship its**

CDC waivers as a readable document, one line of justification each, because the integrator inherits every one of them and cannot re-derive them from the netlist.

Proving a crossing in simulation as well as structurally

Structural CDC analysis proves a synchroniser *exists*. It cannot prove the source obeys the protocol the synchroniser assumes — that the data is stable while the request is high, that pulses are long enough, that a quasi-static field is not written while the datapath runs. Those are **temporal properties**, and they are checked by assertions, either hand-written or generated by the CDC tool.

The strongest available check adds metastability *injection*: the simulator randomly delays the synchroniser output by one cycle, so that a design which only works when the crossing resolves immediately fails in regression rather than in silicon.

Where it is used	Block-level verification of anything with more than one clock, and every IP you intend to sell into someone else's clocking scheme.
When it binds	Run structural CDC at every RTL freeze; run injection in the nightly regression. The two find different bugs and neither subsumes the other.
How to apply it	Bind the assertions to the crossing module, run the regression with injection on, and record the result in the deliverable's verification section with the number of crossings and the number of waivers.

What the industry code looks like

```
// The protocol the handshake crossing assumes, stated as a checkable property.
property data_stable_while_req;
    @(posedge clk_src) disable iff (!rst_n)
        req |-> ##0 $stable(data) throughout (req [*1:$] ##1 ack_sync);
endproperty
assert property (data_stable_while_req)
    else $error("CDC: data changed while req was asserted -- the receiver may latch a mix");

// Metastability injection (simulation only): the synchroniser may take one extra cycle.
`ifdef CDC_INJECT
    always_ff @(posedge clk_dst)
        if ($urandom_range(0,99) < 5) sync_q[1] <= sync_q[1];    // hold: resolve later
`endif
```

This is where it goes wrong. Assertions written in the destination domain about source-domain signals are themselves a clock-domain crossing and can report failures that are artefacts of the testbench. Sample source signals in the source domain (`@(posedge clk_src)`) and cross only the boolean result.

T3.6 What to do with this on Monday

Table 13. Where this chapter's concepts are actually used

Concept	Block it appears in	What it decides
MTBF · resolution time constant	Every asynchronous boundary	Synchroniser depth, and the number you write in the data-sheet
synchroniser depth	Architecture, per crossing	Two by default; three above ~1 GHz or when τ is poorly characterised
multi-bit crossing · gray code	FIFO pointers, counters, status words	Whether the crossing needs an encoding, a handshake, or a FIFO

Concept	Block it appears in	What it decides
handshake · asynchronous FIFO	Streaming data between domains	Latency and depth, and therefore area
reset synchroniser · recovery time	Every clock domain	Whether the block starts in a legal state at every corner
CDC verification · quasi-static signal	Signoff and the deliverable	The waiver list the integrator inherits

Two numbers end this chapter, and they are the ones to carry: at 1 GHz a single-stage crossing fails about once every 1.8 years, and the same crossing with τ twice as large fails about once every 1.8 years even with two stages. The first number is why synchronisers exist; the second is why the answer is always quoted with its corner.

The next chapter takes the quantity this one kept assuming — that a sample is either right or wrong — and replaces it with a distribution. Noise sets the floor under every analog measurement in a mixed-signal IP, and unlike metastability it can be traded directly against power and area.

Where the inventions are — what is patentable here

Metastability itself is physics, and the two-flop synchroniser is older than anyone reading this. **What is still being claimed is the arrangement around it:** a pointer encoding that keeps a multi-bit crossing safe with one synchroniser instead of N (gray codes and their successors); a latency-versus-MTBF scheme that switches synchroniser depth by operating mode; a data-path crossing that avoids synchronising the data at all by synchronising only a qualifier and holding the data *stable* across the boundary. Every one of these is an argument about *what must cross*, not about the flop. Read this chapter asking: which of my signals actually has to cross, and what would let the rest stay on one side?

T4. Noise — and Why It Sets the Size of Everything

Noise is the one specification you cannot improve with cleverness alone. Every 6 dB costs 4× the capacitance, and therefore 4× the power.

Before you read this

voltage · current · resistance · temperature · probability · variance · standard deviation · integration · logarithms · frequency · transconductance · gate capacitance · output resistance · RC delay · metastability

New in this chapter

thermal noise · shot noise · flicker noise · noise spectral density · noise bandwidth · kT/C noise · input-referred noise · flicker corner · noise figure · static noise margin · butterfly curve · read disturb · $V_{\min}$ · SNR · SNDR · ENOB

Everything measured in a circuit arrives with noise on it, and the noise does not come from poor engineering — it comes from the thermal motion of charge and from the discreteness of the electron. That makes it the hardest kind of specification: you cannot remove it, you can only spend area and power to average it down, and the exchange rate is fixed by physics at **4× the cost per extra bit**.

This chapter establishes the three noise sources you will actually meet, then derives the one number that sizes half the analog blocks in a mixed-signal IP (kT/C), and then does something unusual for a noise chapter: it crosses into the digital side, where the same word means something entirely different and where the consequence — the voltage below which an SRAM cannot be read — is **measured here with ngspice rather than asserted**.

T4.1 The three sources, and which one you can do anything about

Table 14. Noise sources in an IC, by origin

Source	Density	Origin	Can you change it?
Thermal (Johnson)	$S_v = 4kTR$ [V^2/Hz]	Thermal motion of carriers in any resistive element	Only by changing R , T , or the bandwidth you look through
Channel thermal	$S_i = 4kTyg_m$ [A^2/Hz]	The same mechanism inside a MOSFET channel; $\gamma \approx 2/3$ long-channel, 1–2 short-channel	Raise g_m (costs current) or filter
Flicker (1/f)	$S_v = K_f / (C_{ox}WL \cdot f)$	Trapping and de-trapping at the oxide interface	Yes — bigger devices, PMOS inputs, chopping, auto-zero
Shot	$S_i = 2qI$ [A^2/Hz]	Discreteness of charge crossing a barrier (junctions, not channels)	Only by changing the current

Derivation — Why the answer always ends up as kT/C

	This follows	Because
1	A resistor's thermal noise density is $4kTR$, independent of frequency.	Nyquist's result from equilibrium thermodynamics — it contains no device parameter beyond R , which is why it is a floor and not a design choice.
2	Put that resistor in series with a capacitor C — which is what every sampling switch is.	The switch's on-resistance ($T1$: triode region) and the sampling capacitor are in series; there is no third element.
3	The RC low-pass shapes it, and the noise bandwidth of a single pole is $\pi/2 \times f_{-3dB} = 1/(4RC)$.	Integrating $1/(1+(f/f_0)^2)$ over all frequency gives $\pi/2 \cdot f_0$, not f_0 — a factor people drop and then wonder why measurements are 1.6× worse than predicted.
4	$v_n^2 = 4kTR \times 1/(4RC) = kT/C$.	R cancels. The noise on a sampled capacitor does not depend on the switch at all — only on C and temperature.
5	For $C = 1$ pF at 3 K, $v_n = 64.4$ μV rms.	One number worth memorising; everything else scales as $1/\sqrt{C}$ from here.

kT/C — the rule that sizes capacitors, and therefore power

A sampled node carries kT/C of noise power regardless of how it was sampled. To keep that below the quantisation noise of an N -bit converter ($LSB/\sqrt{12}$), the capacitor must satisfy $C \geq 12kT \cdot 2^{2N}/V_{FS}^2$. **Every extra bit quadruples C** , and because settling in a fixed time needs $g_m \propto C$, it quadruples the current too.

This single relation is why a 16-bit converter is not "a bit harder" than a 12-bit one: it is $256\times$ the capacitance.

Where it is used	Every switched-capacitor circuit: SAR and pipeline ADC sampling networks, sample-and-holds, switched-capacitor filters and gain stages, the reference buffer that has to drive them.
When it binds	At the very start of an analog block's design, before any transistor is sized. The capacitor comes first because it fixes the power budget.
How to apply it	Fix V_{FS} and N , compute C , then size the driver for settling within the sampling phase (T_2 's RC arithmetic with the switch's R_{on} from T_1), then check that the reference can supply the charge each cycle.

What the industry code looks like

```
* The measurement that separates  $kT/C$  from everything else you did wrong.
* Transient noise in ngspice: sample the same node 1000 times and take the std dev.
.tran 1n 10u
.noise v(out) VIN dec 20 1 1G      $ small-signal, for the spectrum
.measure noise inoise_total_rms param='sqrt(inoise_total)'
```

* And the closed form to check it against -- ALWAYS have the hand calculation:

```
* v_n = sqrt( $kT/C$ ) = sqrt( $1.38e-23 \cdot 300 / 1e-12$ ) = 64.4 uV for  $C = 1$  pF
```

This is where it goes wrong. kT/C applies to the **sampled** value. During the tracking phase the noise bandwidth is set by the circuit, not by the sampling — so a track-and-hold that is measured while tracking shows a different (usually larger) number than the same circuit measured after the hold. Reporting one when the spec means the other is a common and expensive mistake.

Table 15. Capacitor and power cost per bit ($V_{FS} = 1$ V, 300 K, noise = quantisation noise)

Bits N	LSB	Quantisation rms	C required	Relative to 8-bit
8	3,906 μ V	1,128 μ V	3.26 fF	1 $\times$
10	977 μ V	282 μ V	52.1 fF	16 $\times$
12	244 μ V	70.5 μ V	834 fF	256 $\times$
14	61 μ V	17.6 μ V	13.3 pF	4,096 $\times$
16	15.3 μ V	4.4 μ V	213 pF	65,536 $\times$

T4.2 Flicker noise, and the corner that decides your architecture

flicker corner, f_c (플리커코너) — The frequency at which the $1/f$ density equals the thermal density. Below it the circuit is flicker-dominated, above it thermal-dominated. For a modern CMOS amplifier it lands anywhere from ~ 100 kHz to a few MHz for NMOS inputs, typically a decade lower for PMOS.

Where the signal sits relative to the corner

Flicker noise density rises as $1/f$, so its **integrated** power over a band from f_1 to f_2 is proportional to $\ln(f_2/f_1)$ — it depends on the ratio, not the width. A DC-coupled measurement has $f_1 \rightarrow 0$ and the integral diverges logarithmically: **the longer you average, the worse it gets**, which is the opposite of thermal noise.

That single fact drives an architectural decision. If the signal is near DC (a temperature sensor, a bandgap reference, a DC offset), averaging does not help and you must *move the signal* — chop it up to a frequency above the corner, amplify there, and bring it back down. If the signal is already above the corner (an RF or high-speed link front end), flicker is irrelevant and you spend nothing on it.

Where it is used	Sensor front ends, reference and bias circuits, comparator offset, any DC-accurate measurement. Also the phase noise of an oscillator, where device flicker is up-converted into close-in phase noise.
When it binds	Whenever the band of interest includes frequencies below ~1 MHz. Ask the question explicitly at architecture time; retrofitting chopping into a finished amplifier is a redesign.
How to apply it	Reduce it by area ($S \propto 1/WL$ — 4× the area for 6 dB), by choosing PMOS inputs, or by modulation: chopping (square-wave modulate in and out; leaves residual ripple at the chop frequency) or correlated double sampling (sample the offset, subtract it; costs noise folding and bandwidth).

What the industry code looks like

```
* Flicker in the model card is one parameter -- and simulators differ.
.model NM NMOS (LEVEL=54 ... KF=1e-25 AF=1.0 EF=1.0)
* KF alone means nothing: the density is KF/(Cox*W*L*f^EF), so a KF taken from
* another PDK, or from a model with a different EF convention, is a WRONG number
* that still simulates. Always sanity-check the corner against measured silicon:
.noise v(out) VIN dec 10 1 100MEG
* then read the point where the 1/f asymptote crosses the flat part.
```

This is where it goes wrong. Chopping does not reduce noise; it **moves the signal away from the noise**. The thermal floor is unchanged and the chopper adds its own switching artefacts at the chop frequency and its harmonics. A design that reports a beautiful low-frequency noise figure and a spur at f_{chop} has not improved anything if the spur lands in band.

T4.3 The other noise: static noise margin, measured

On the digital side the word 'noise' means something different and the quantity is not a spectral density but a **voltage**: how much DC disturbance a storage node can absorb before it flips. The structure is the same cross-coupled pair that produced metastability in T3, viewed statically instead of dynamically.

SNM, static noise margin (정적잡음여유) — The largest DC voltage that can be inserted in series with **both** storage nodes of a cross-coupled pair without destroying its bistability. Geometrically it is the side of the largest square that fits inside the butterfly curve formed by the two inverter transfer curves.

What actually happened

The extractor used below was **wrong twice** before it was right, and both times the numbers looked plausible. The first version measured a horizontal gap instead of an inscribed square and reported hold SNM $\approx 0.5 \cdot V_{DD}$ — too high. The second version had the access transistor wired with its gate on the bitline instead of the word line; it produced read SNM that *fell* as the cell ratio rose, which is backwards. **The trend caught the bug, not the value.** That is the rule this repository runs on: before believing a measurement, check the direction it moves when you change something you understand.

The extractor was then **self-checked against two limits**: an inverter with very high gain must give SNM $\rightarrow V_{DD}/2$ (it gives 0.4825) and an 'inverter' with gain 1 must give 0 (it gives 0.0). Only after both did the cell numbers below get written down.

Table 16. Measured SNM of a 6T cell (ngspice, LEVEL=1 model, $V_{DD} = 1.0$ V, $V_{th} = 0.35$ V) — β is the cell ratio $W_{\text{pull-down}} / W_{\text{access}}$

β	Hold SNM	Read SNM	Read / Hold
1	0.421 V	0.194 V	0.46

β	Hold SNM	Read SNM	Read / Hold
1.5	0.417 V	0.207 V	0.5
2	0.414 V	0.226 V	0.55
2.5	0.411 V	0.241 V	0.59
3	0.409 V	0.253 V	0.62

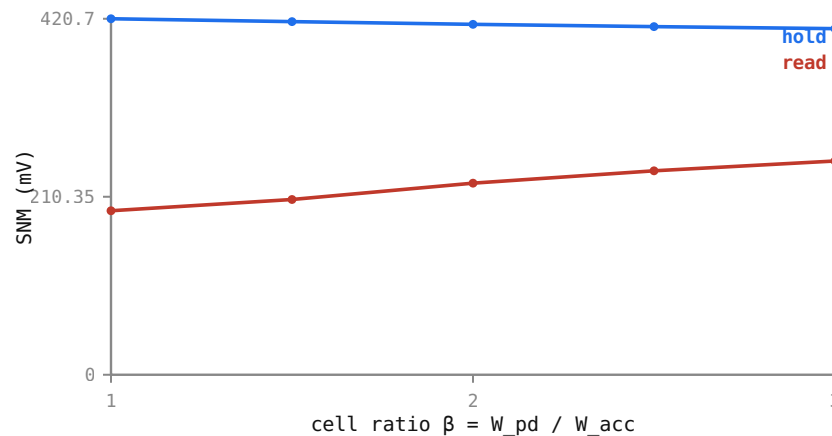


Figure 10. Reading costs more than half the cell's margin, and the only structural way to buy it back is a stronger pull-down relative to the access device — which is why the 6T cell is not made of minimum devices.

Derivation — Why reading a cell is what almost destroys it

	This follows	Because
1	To read, the word line turns on both access transistors while both bit lines are precharged high.	That is the only way to sense the cell without a separate read port.
2	The access transistor on the '0' side now pulls that node up , through a divider formed with the cell's own pull-down NMOS.	Two NMOS devices in series between V_{DD} and ground, with the storage node in the middle — the node cannot stay at 0 V.
3	The '0' node rises to roughly $V_{DD} \cdot (1/(1+\beta))$ in the resistive limit, so a β of 2 already puts it a third of the way up.	Both devices are in triode at that point; the ratio of their strengths is the divider ratio. This is the only reason the cell ratio exists as a design parameter.
4	That rise eats directly into the margin: measured read SNM at $\beta = 2$ is 0.226 V against a hold SNM of 0.414 V — 45.3 % of the margin is gone the moment the word line rises.	Measured, not asserted — the deck and the extractor are in this chapter's source.
5	Raising β raises read SNM and slightly lowers hold SNM .	Measured: hold falls from 0.421 V to 0.409 V across the same sweep, because a stronger pull-down skews each inverter's trip point away from $V_{DD}/2$. There is no free direction , which is what makes it a design point rather than a rule.

V_{min} — the voltage at which the memory, not the logic, stops the chip

Read SNM does not degrade gracefully with supply. Measured on the same cell at $\beta = 2$, it holds near 0.22 V from 1.0 V down to 0.7 V and then **collapses**: 0.145 V at 0.6 V and 0.087 V at 0.5 V. The knee sits where V_{DD} approaches $2V_{th}$ (0.7 V here) — below that the devices are in weak inversion, the divider ratio depends on exponential currents, and mismatch (T12) turns a marginal cell into a failing one.

This is why **the SRAM sets the chip's minimum operating voltage**, and why DVFS schemes either stop above the array's $V_{\min}$ or give the array its own supply.

Where it is used	Any SoC with on-chip SRAM and more than one voltage point: the whole DVFS range, retention voltage in sleep modes, and the low-voltage corner in signoff.
When it binds	As soon as a power mode is proposed. The question 'can we drop to 0.6 V' is an SRAM question before it is a logic question.
How to apply it	Either raise $V_{\min}$ with assist circuits (word-line underdrive for read, negative bit line for write), split the array's supply, or use an 8T cell where the read path does not touch the storage node.

What the industry code looks like

```
* The sweep that produces the number above -- run it at every corner,
* not just typical, because Vmin is a MISMATCH-limited quantity (see T12).
.param beta=2.0
.dc VIN 0 VDD 0.004 VDD 1.0 0.5 -0.1      $ nested sweep: input, then supply
.print dc v(vr)
* Extraction is NOT a .measure one-liner: you need the butterfly and the largest
* inscribed square. Script it, and SELF-CHECK the extractor on two limits
* (very high gain -> VDD/2, gain 1 -> 0) before trusting any cell number.
```

This is where it goes wrong. SNM measured at the typical corner is nearly useless for $V_{\min}$. The array fails at its **worst** cell out of millions, so the quantity that matters is the tail: $V_{\min}$ is set by roughly the -5σ to -6σ cell, which means the analysis is a Monte Carlo with importance sampling, not a nominal DC sweep. The sweep above tells you the shape; it does not tell you the yield.

T4.4 From volts to a datasheet number: SNR, SNDR, ENOB

Table 17. The three numbers, and what each one hides

Quantity	Definition	Hides
SNR	Signal power / noise power, distortion excluded	Distortion. A converter with terrible linearity can have excellent SNR
SNDR (SINAD)	Signal / (noise + all distortion)	Which of the two dominates — always report the spectrum too
ENOB	$(\text{SNDR}_{\text{dB}} - 1.76) / 6.02$	That it is derived , not measured. ENOB is SNDR wearing different units; an ENOB quoted without its input frequency and amplitude is not a measurement
SFDR	Signal / largest single spur	Broadband noise entirely — a clean-looking SFDR with a poor noise floor is common

Worked example 5. How many bits does 64 μV of noise allow?

Given. A 1 V full-scale converter with $C = 1$ pF on its sampling network, so the sampled noise is 64.4 μV rms, and assume everything else is perfect.

Method. A full-scale sine has rms amplitude $V_{\text{FS}}/(2\sqrt{2})$. $\text{SNR} = 20 \cdot \log_{10}(\text{signal rms} / \text{noise rms})$, then $\text{ENOB} = (\text{SNR} - 1.76)/6.02$.

Answer. $\text{SNR} = 74.8$ dB, giving $\text{ENOB} = 12.1$ bits. **A 1 pF sampling capacitor is a ~12-bit part**, no matter how good the rest of the design is.

This is where it goes wrong. Forgetting that the sampling network is usually sampled **twice** (input and reference), and that a differential pair of capacitors doubles the noise power while doubling the signal amplitude — the two partly cancel, and the factor you get depends on the topology. Write the specific network down; do not reuse a remembered factor.

T4.5 What to do with this on Monday

Table 18. Where this chapter's concepts are actually used

Concept	Block it appears in	What it decides
kT/C noise	Any switched-capacitor block, before sizing transistors	The sampling capacitor, and therefore the current budget
thermal noise · noise band-width	Amplifier and reference design	How much g_m the input stage needs
flicker corner	Architecture of any near-DC signal path	Whether the design needs chopping or auto-zeroing at all
static noise margin · butterfly curve	SRAM, any cross-coupled sense amp or latch	Cell ratio β , and whether a read destroys what it reads
V _{min}	DVFS and retention modes	The lowest voltage the chip may be operated at
SNDR · ENOB	Datasheet and customer conversation	The single number the customer will compare against a competitor

Two measured results to carry out of this chapter. A 1 pF sampling capacitor is a 12.1-bit part — the capacitor decides the resolution before any transistor is drawn. And reading a 6T SRAM cell costs 45.3 % of its static margin at a cell ratio of 2, which is why that cell ratio exists and why the array, not the logic, sets the lowest voltage the chip can run at.

The next chapter takes the sampling operation that this one treated as instantaneous and gives it a finite aperture, a jitter, and a spectrum — which is where the noise floor of a high-speed converter actually comes from.

Where the inventions are — what is patentable here

Noise is thermodynamics; nobody patents kT. **What gets claimed is every arrangement that buys back the 4× — correlated double sampling, chopping, auto-zeroing, noise shaping, and the family of techniques that let a circuit spend its noise budget only where the signal is.** On the digital side the same shape appears as assist circuits for SRAM: negative bitline, word-line underdrive, read-modify-write assist — all of them arrangements that let a cell keep its margin at a voltage where the plain cell has none. Read this chapter asking: where am I paying the 4× rule, and what would let me not pay it?

T5. Sampling — Where a Continuous Signal Becomes a Number

Two clocks decide everything: when you sample, and how precisely you know when. The second one is what limits every high-speed converter shipping today.

Before you read this

voltage · frequency · phase · integration · differentiation · probability · standard deviation · logarithms · RC delay · slew · kT/C noise · thermal noise · SNR · SNDR · ENOB · noise bandwidth

New in this chapter

sampling theorem · aliasing · Nyquist · bandpass sampling · aperture time · aperture jitter · jitter-limited SNR · track and hold · tracking bandwidth · charge injection · clock feedthrough · bootstrapped switch · oversampling ratio · noise shaping · decimation

Sampling is usually taught as an idealisation — multiply by an impulse train, observe the spectrum repeat, avoid overlap. That story is correct and it is not the one that limits hardware. A real sampler closes a switch for a finite time, through a finite resistance, at an instant known only to within some picoseconds, onto a capacitor whose size was already fixed by kT/C in T4. Each of those four words — finite, finite, instant, capacitor — costs resolution, and the one that dominates above a few hundred megahertz is the third.

T5.1 Aliasing is not a theorem you obey, it is a filter you buy

Derivation — From the impulse train to the folded spectrum

	This follows	Because
1	Sampling at f_s multiplies the signal by a train of impulses spaced $T_s = 1/f_s$.	The idealisation. Everything about aliasing follows from this one step; the non-idealities come later and do not change it.
2	Multiplication in time is convolution in frequency, and the impulse train's spectrum is itself an impulse train spaced f_s .	So the input spectrum is copied to every multiple of f_s .
3	Copies overlap unless the input is confined to a band narrower than $f_s/2$.	Nyquist. Note what it does <i>not</i> say: it says nothing about where the band sits, only how wide it is.
4	Everything above $f_s/2$ folds down and is indistinguishable from a real signal afterwards.	No digital processing can undo it — the information that told the two apart was destroyed at the sampler. This is why an anti-alias filter is an analog component and cannot be moved into software.
5	Noise folds too , and by the same rule.	A front end with 5 GHz of bandwidth sampled at 200 MHz folds 50 bands of thermal noise onto the signal — a 17 dB penalty. Bandwidth you do not need is not free ; it is noise you pay for.

Bandpass sampling — using the folding on purpose

Nyquist constrains the *width* of the band, not its position. A signal occupying 100 MHz centred at 2.4 GHz can be sampled at 250 MS/s, provided the band falls entirely inside one Nyquist zone and the sampler's input bandwidth reaches 2.4 GHz. The alias is the wanted signal, translated down for free.

The cost is precise: the sampling **aperture** must be good enough for 2.4 GHz even though the sample rate is 250 MHz — jitter is referred to the input frequency, not the clock rate. That trade (no mixer, but a far harder clock) is the whole engineering decision.

Where it is used	RF receivers that digitise at IF, software-defined radio front ends, and any instrument that must capture a narrow band sitting at a high frequency.
When it binds	When the band is narrow relative to its centre frequency and you would otherwise pay for a downconversion mixer and its LO.
How to apply it	Pick f_s so the band lands strictly inside a single zone ($k \cdot f_s/2$ to $(k+1) \cdot f_s/2$), add a bandpass anti-alias filter — a low-pass is useless here — and budget the jitter against the RF carrier.

What the industry code looks like

```
// Zone check, written as an assertion rather than a comment.
// A band that straddles a zone boundary is a silent failure: it still samples,
// and the output is two overlapped copies.
localparam real FS = 250e6, FC = 2.4e9, BW = 100e6;
initial begin
    int k = $floor(2.0*(FC - BW/2)/FS);
    assert ($floor(2.0*(FC + BW/2)/FS) == k)
        else $fatal(1, "band straddles a Nyquist zone boundary -- pick another FS");
end
```

This is where it goes wrong. The anti-alias filter for bandpass sampling must reject **every other zone**, not just frequencies above $f_s/2$. Engineers who reuse a low-pass filter here get a receiver that works in the lab (where the only signal present is the wanted one) and fails in the field.

T5.2 Aperture jitter — the wall

aperture jitter (구경지터) — The rms uncertainty in the instant at which the sampler actually captures the signal. It comes from the clock path (phase noise, supply-induced delay modulation, thermal noise in the last buffer) and from the sampler itself, and it is **referred to the input frequency**, not the sample rate.

Derivation — Why jitter costs more as the signal gets faster

	This follows	Because
1	A sine $v(t) = A \cdot \sin(2\pi f_{in} t)$ has slope $dv/dt = 2\pi f_{in} A \cdot \cos(\dots)$.	The slope is what converts a time error into a voltage error — this is the whole mechanism.
2	A timing error Δt therefore produces a voltage error $\Delta v = (dv/dt) \cdot \Delta t$, worst at the zero crossing where the slope is greatest.	The largest error happens where the signal is <i>changing</i> fastest, not where it is largest.
3	With rms jitter σ_t and a full-scale sine, the rms error is $2\pi f_{in} \cdot (A/\sqrt{2}) \cdot \sigma_t$, and the signal rms is $A/\sqrt{2}$.	The amplitude cancels — jitter-limited SNR does not depend on signal amplitude , which is why you cannot back away from it by scaling down.
4	$SNR = -20 \cdot \log_{10}(2\pi f_{in} \sigma_t)$.	One equation, two variables, no device parameters. It is a property of the clock and the input frequency alone.
5	At $f_{in} = 1$ GHz, 100 fs of jitter gives 64 dB — 10.3 bits.	100 fs is already a hard clock. This is why 12-bit gigahertz converters are difficult in a way that has nothing to do with the comparator.

Table 19. Jitter-limited SNR and ENOB (uncorrelated aperture jitter only)

f_{in}	1 ps	300 fs	100 fs	30 fs
0.1 GHz	64.0 dB / 10.3 b	74.5 dB / 12.1 b	84.0 dB / 13.7 b	94.5 dB / 15.4 b

f _{in}	1 ps	300 fs	100 fs	30 fs
1 GHz	44.0 dB / 7.0 b	54.5 dB / 8.8 b	64.0 dB / 10.3 b	74.5 dB / 12.1 b
5 GHz	30.1 dB / 4.7 b	40.5 dB / 6.4 b	50.1 dB / 8.0 b	60.5 dB / 9.8 b

Worked example 6. What clock does a 12-bit, 1 GHz-input converter need?

Given. The design target is 12 effective bits at a 1 GHz input. Assume jitter is the only error source — an optimistic assumption that makes the answer a **lower bound** on how good the clock must be.

Method. Invert the SNR expression: $\sigma_t = 1/(2\pi \cdot f_{in} \cdot 2^N)$.

Answer. $\sigma_t = 38.9$ fs rms. For 14 bits it is 9.71 fs.

Table 20. Required rms aperture jitter at f_{in} = 1 GHz

Bits	σ_t required
8	622 fs
10	155 fs
12	38.9 fs
14	9.71 fs
16	2.43 fs

This is where it goes wrong. Comparing that number against the jitter quoted for a clock *source*. The specification that matters is the jitter **at the sampling switch**, after the whole distribution path — buffers, level shifters, the divider, and whatever supply noise the digital side injected into them. A 20 fs source behind a noisy buffer chain is a 200 fs sampler, and the data-sheet of the source will not tell you that.

Jitter and kT/C add in **power**, not in amplitude, and they scale in opposite directions with frequency: kT/C is flat while jitter worsens by 6 dB per octave of input frequency. There is therefore always a crossover frequency, and knowing which side of it you are on tells you which knob does anything. Below it, spend area on capacitance. Above it, spend power on the clock — **a bigger sampling capacitor buys nothing in a jitter-limited design**, and it is the most common way to waste a redesign.

T5.3 The switch itself: charge injection, feedthrough, and bootstrapping

VDD

Figure 11. The sampling switch. Its on-resistance sets the tracking bandwidth; its channel charge lands on the sampling capacitor when it turns off.

Table 21. Three errors the switch adds, and what each depends on

Error	Mechanism	Scales with	Signal-dependent?
Charge injection	The channel's inversion charge has to go somewhere when the device turns off; roughly half lands on C _s	$W \cdot L \cdot C_{ox} \cdot (V_{GS} - V_{th}) / C_s$	Yes — V _{GS} depends on the sampled level, so it becomes distortion, not offset
Clock feed-through	C _{gd} couples the falling clock edge onto the capacitor	$C_{ov} / (C_{ov} + C_s) \cdot V_{clk}$	No (to first order) — mostly a fixed offset
R _{on} modulation	The switch's on-resistance depends on V _{GS} = V _{clk} - v _{in}	$1/(\mu C_{ox}(W/L)(V_{GS} - V_{th}))$	Yes — a signal-dependent time constant is signal-dependent settling, which is distortion

The bootstrapped switch

Hold the switch's gate at $v_{in} + V_{DD}$ instead of at V_{DD} , using a capacitor pre-charged to V_{DD} and flipped onto the gate during the tracking phase. Then V_{GS} is constant regardless of the signal, so R_{on} is constant and the charge injected at turn-off is constant — **signal-dependent errors become offsets**, and offsets are harmless in a differential design.

It also solves a reliability problem that is easy to miss: without bootstrapping, a switch passing a signal near ground sees $V_{GS} = V_{DD}$, which in an advanced node may exceed the oxide's rating over the device's lifetime (T14).

Where it is used	The input sampling network of essentially every SAR and pipeline ADC above 10 bits, and in high-linearity switched-capacitor filters.
When it binds	When THD or SFDR matters more than area. A plain NMOS switch is fine for 8 bits and hopeless at 12.
How to apply it	Size the bootstrap capacitor $\gg$ the gate capacitance it must drive, check the reliability of every node across the clock phases, and verify distortion with a two-tone transient, not a single tone — intermodulation exposes the residual signal dependence a single tone hides.

What the industry code looks like

```
* The check that matters, and the one people skip.
* A single tone can look clean while IM3 is terrible.
Vin1 in1 0 SIN(0 0.45 100.1MEG)
Vin2 in2 0 SIN(0 0.45 103.3MEG)
.tran 10p 20u
.four 100.1MEG v(sampled)          $ harmonic distortion
* and post-process the FFT for IM3 at 2*f1-f2 = 96.9 MHz -- the number the
* customer will measure and the one that exposes Ron modulation.
```

This is where it goes wrong. Bootstrapping makes the gate voltage exceed the supply, so every node in the bootstrap network must be checked against the oxide limit at the **fast, high-voltage, low-temperature corner** — the opposite corner from the one that limits performance. A bootstrap circuit verified only at typical is a reliability report waiting to happen.

T5.4 Oversampling: buying bits with time

Derivation — Why oversampling helps, and how much

	This follows	Because
1	Quantisation noise power is $LSB^2/12$ regardless of the sample rate .	The quantiser does not know how fast it is being clocked; the error per sample has the same distribution.
2	That fixed power is spread over the Nyquist band, 0 to $f_s/2$.	So the noise <i>density</i> is $LSB^2/12$ divided by $f_s/2$ — sampling faster dilutes it.
3	Filtering down to the signal band B keeps only the fraction $2B/f_s$ of it.	Define the oversampling ratio $OSR = f_s/(2B)$.
4	SNR improves by $10 \cdot \log_{10}(OSR)$: 3 dB, or half a bit, per doubling .	This is plain oversampling, with no feedback. Going from 8 to 12 bits this way needs $OSR = 256$ — expensive but occasionally the right answer for a slow sensor.
5	A $\Delta\Sigma$ modulator shapes the noise instead of merely spreading it, giving $(6L+3)$ dB per doubling for an L-th-order loop.	The loop pushes quantisation noise out of the band. A 2nd-order modulator gains 15 dB per octave of OSR — this is why $\Delta\Sigma$ dominates audio and precision sensing, and why it is useless at gigahertz signal bandwidths.

Table 22. What oversampling buys, per doubling of OSR

Architecture	SNR gain per 2× OSR	Bits per 2× OSR	Where it wins
Plain oversampling + decimation	3 dB	0.5	Cheap resolution on slow signals; anti-alias filter relief
$\Delta\Sigma$, 1st order	9 dB	1.5	Simple, but idle tones
$\Delta\Sigma$, 2nd order	15 dB	2.5	Audio, precision sensors
$\Delta\Sigma$, 3rd+ order / MASH	21 dB+	3.5+	Where stability engineering is worth it

Oversampling relieves the **anti-alias filter** as much as it buys resolution, and that is often the real reason to do it. Sampling a 20 kHz audio band at 44.1 kHz demands a filter that is flat at 20 kHz and dead at 22.05 kHz — a brutal analog problem. Sampling at 2.8 MHz moves the requirement to 'dead at 1.4 MHz', which is a single RC. **The filter that disappears is usually worth more than the half-bit.**

T5.5 What to do with this on Monday

Table 23. Where this chapter's concepts are actually used

Concept	Block it appears in	What it decides
aliasing · Nyquist	Choosing f_s and the anti-alias filter	The filter order — and whether it is analog or can be shared with oversampling
aperture jitter	Clocking architecture, before the converter is designed	Whether the target ENOB is reachable at all at that input frequency
bandpass sampling	RF and IF receiver front ends	Whether a mixer can be removed, and what the clock must then cost
charge injection · bootstrapped switch	The sampling network of any ≥ 10 -bit converter	THD and SFDR, and the oxide-reliability check that goes with it
oversampling ratio · noise shaping	Slow, precise signal paths	Whether resolution can be bought with time instead of capacitance

The number to carry: at a 1 GHz input, **12 effective bits requires 38.9 fs rms of aperture jitter**, measured at the switch and not at the clock source. Every architecture decision in a high-speed converter — interleaving, bandpass sampling, where the clock buffer sits — is downstream of that one line.

The next chapter turns the sampled voltage into a code, and finds that the architectures differ less in accuracy than in *where* they spend the same fundamental limits.

Where the inventions are — what is patentable here

The sampling instant is the most contested square millimetre in a high-speed converter, and the patents show it: bootstrapped switches that hold V_{GS} constant so the on-resistance does not depend on the signal; clock paths that deliberately do *not* share a buffer with anything else; sampling schemes that cancel charge injection by symmetry rather than by trimming; and, one level up, time-interleaved architectures whose entire difficulty is that each slice samples at a slightly wrong instant. **Every one of these is an argument about the aperture, not about the amplifier.**

T6. Data Converters — Where Each Architecture Spends the Same Limits

No architecture beats kT/C , jitter or mismatch. They differ in which one they pay first, and that is the entire basis on which you choose.

Before you read this

voltage · current · charge · probability · standard deviation · logarithms · SNR · SNDR · ENOB · kT/C noise · aperture jitter · sampling theorem · oversampling ratio · noise shaping · charge injection · static noise margin

New in this chapter

flash ADC · SAR ADC · pipeline ADC · time interleaving · capacitive DAC · current-steering DAC · segmentation · INL · DNL · missing code · monotonicity · redundancy · digital calibration · glitch energy · matching

Converter architecture looks like a catalogue and is really a single question: given a resolution and a sample rate, which fundamental limit do you hit first, and which architecture lets you pay that one in the currency you have? Every architecture faces the same kT/C , the same jitter, the same mismatch. What differs is how many times per conversion each limit is paid.

T6.1 The map, and what sets each boundary

Table 24. The four architectures, and the limit that stops each

Architecture	Comparisons per sample	Practical range	What stops it
Flash	$2^N - 1$ in parallel	4–8 b, up to tens of GS/s	Area and input capacitance double per bit; 2^N comparator offsets must each be $< \frac{1}{2}$ LSB
SAR	N in series (one per cycle)	8–14 b, up to ~100 MS/s per slice	The DAC settling chain: N settlings per sample, each to the full accuracy
Pipeline	~1.5 b per stage, concurrent	10–16 b, 50 MS/s–1 GS/s	Inter-stage gain accuracy and the op-amp that provides it
$\Delta\Sigma$	1 (or few) at $OSR \times$ the rate	16–24 b, kHz–MHz bandwidth	Loop stability and the OSR the process can clock
Time-inter-leaved	M slices of any of the above	Extends speed, not resolution	Mismatch between slices — gain, offset and sampling-instant skew appear as spurs at f_s/M

The last row is the one that surprises people. Interleaving M slices multiplies throughput by M and leaves resolution worse, because three new error terms appear that do not exist in a single converter. Timing skew between slices is the hardest: it is a jitter problem (T5) that is **deterministic**, so it produces spurs rather than a noise floor, and a spur at a predictable frequency is exactly what a customer's spectrum plot will show.

T6.2 The SAR's capacitor array: where matching beats noise

A SAR converter is a binary search implemented with a capacitive DAC. Its accuracy therefore rests on capacitor ratios, and the design question is simply how big the unit capacitor must be. Two independent requirements set it, and which one wins is not obvious until you compute both.

Derivation — Two requirements on the same capacitor, and where they cross

	This follows	Because
1	Requirement 1 — noise : the array is the sampling capacitor, so kT/C_{total} must stay below the quantisation noise (T4).	For 1 V full scale this gives $C \geq 12kT \cdot 2^{2N}/V_{\text{FS}}^2$: 0.83 pF at 12 bits.
2	Requirement 2 — matching : DNL at the mid-scale transition is dominated by the mismatch of the largest capacitor against the sum of the rest.	$3\sigma \text{ DNL} < \frac{1}{2} \text{ LSB}$ needs $\sigma_u/C_u < 0.26\%$ at 12 bits — and by Pelgrom (T12) that relative sigma is fixed by <i>area</i> .
3	A capacitor that matches to that level is about 1 fF in a modern MOM-capacitor process, which makes the 12-bit array 4.1 pF — 4.91× larger than the noise requirement.	At 12 bits the array is set by matching, not by noise. Sizing it from kT/C alone gives a converter with an excellent noise floor and visible DNL steps.
4	At 14 bits the two requirements are 16.4 pF and 13.3 pF — they have converged.	Above ~14 bits noise takes over and the unit capacitor can be whatever matches; below it, matching rules. Knowing which side you are on tells you which knob does anything.

Table 25. Which limit sizes the SAR array ($V_{\text{FS}} = 1 \text{ V}$, unit cap 1 fF)

Bits	Matching needs $\sigma_u/C_u <$	Array from matching	Array from kT/C	Binds
10	0.52 %	1.02 pF	0.05 pF	matching
12	0.26 %	4.1 pF	0.83 pF	matching
14	0.13 %	16.4 pF	13.3 pF	matching

Redundancy — buying back settling time with one extra bit

A plain binary search has no way to recover from a wrong decision: if the DAC has not settled when the comparator fires, the bit is wrong and every subsequent bit refines the wrong interval. Redundancy changes the radix so that successive ranges **overlap** — each step can correct an earlier error of up to the overlap.

The cost is one or two extra cycles; the benefit is that early cycles no longer need to settle to the full N-bit accuracy, only to within the overlap. Since settling time is logarithmic in accuracy (T2), **relaxing the early cycles is most of the conversion time.**

Where it is used	Every modern high-speed SAR, and the inter-stage stages of pipelines (where it is called 1.5-bit-per-stage).
When it binds	Whenever the conversion must be fast. It is nearly free in area and it is the difference between a 10 MS/s and a 100 MS/s SAR at the same resolution.
How to apply it	Choose a sub-binary radix (e.g. 1.86 instead of 2), size the capacitors accordingly, and reconstruct the output digitally with the correction weights. The weights can also be calibrated , which turns capacitor mismatch into a measured constant rather than an error.

What the industry code looks like

```
// Non-binary SAR reconstruction. The weights are NOT powers of two,
// and they are per-chip once calibration has run.
localparam int W [0:N] = '{2048, 1088, 576, 320, 176, 96, 56, 32, 18, 10, 6, 4, 2, 1};
always_comb begin
    code = 0;
    for (int i = 0; i <= N; i++) if (bit_decision[i]) code += W[i];
end
// The sum of weights EXCEEDS full scale on purpose -- that surplus is the redundancy.
// A design review question worth asking: what happens if the weights are wrong by 1%?
// (Answer: nothing, until the error exceeds the overlap -- which is the point.)
```

This is where it goes wrong. Redundancy protects against **incomplete settling and comparator noise**, not against a DAC that is monotonically wrong. If a capacitor is mis-sized beyond the overlap, redundancy hides the symptom in simple tests and leaves a code-dependent error that only a full INL sweep reveals.

T6.3 Static linearity: INL, DNL, and the two ways a converter lies

DNL — The deviation of each code's width from 1 LSB. $DNL = -1$ LSB means the code has **zero width** — **a missing code**, and the converter is no longer monotonic there.

INL — The cumulative deviation of each transition point from the ideal straight line — the running sum of DNL. It is what distortion measurements see; DNL is what a control loop feels.

Table 26. Which one to care about, and when

Application	The one that matters	Why
Audio / RF spectrum	INL	Curvature is distortion; the customer's FFT shows harmonics
Closed-loop control (e.g. a DAC in a servo)	DNL and monotonicity	A non-monotonic code inverts the loop's sign locally and can oscillate
Sensor threshold detection	DNL	A missing code is a value the system can never report
Digital calibration of another block	INL	The correction is only as good as the reference curve

What actually happened

A converter can pass an SNDR test and be badly non-monotonic. A sine-wave histogram test averages over all codes; a single missing code moves SNDR by a fraction of a dB and is invisible. **Static linearity needs its own test** — a slow ramp or a code-density sweep — and an IP deliverable that reports only SNDR has not characterised the part.

T6.4 The DAC side: segmentation and glitch energy

segmentation (세그먼테이션) — thermometer where it matters, binary where it does not

A fully binary DAC has the worst possible glitch at mid-scale: every switch changes at once, and their timing skew produces a transient whose area (**glitch energy**, in $V \cdot s$) does not average out — it appears as distortion at the update rate. A fully thermometer-coded DAC has no such transition but needs $2^N - 1$ elements and the decoder to drive them.

Real DACs segment: the top M bits thermometer, the rest binary. M is chosen so that the largest simultaneous switching event is small enough, and the practical answer is usually 4–6 — the decoder cost grows as 2^M while the glitch improvement saturates.

Where it is used	Current-steering DACs for transmitters and waveform generators; the reference DACs inside pipeline ADCs; any DAC whose output is observed in the frequency domain.
When it binds	Whenever SFDR is specified. A binary DAC's mid-scale glitch usually sets SFDR before any static error does.
How to apply it	Segment the MSBs, match the unit current sources by layout (common centroid, dummies), and randomise or rotate the selection of unit elements (DEM) so residual mismatch becomes noise instead of a spur.

What the industry code looks like

```
// Segmented decode: 6 thermometer MSBs + 8 binary LSBs.
// The point of the thermometer part is that consecutive codes change ONE element.
logic [62:0] therm;
always_comb begin
    therm = '0;
```

```

    for (int i = 0; i < 63; i++) therm[i] = (code[13:8] > i);
end
// Dynamic element matching: rotate the starting index so that element mismatch
// is spread across codes and appears as a raised noise floor, not as a spur.
always_ff @(posedge clk) rot <= rot + code[13:8];
assign sel = {therm, therm} >> rot;    // barrel rotate

```

This is where it goes wrong. DEM converts a spur into noise — it does not remove the error. If the spectrum requirement is a noise floor, that is a win; if it is total noise power, it is neutral. Reporting 'SFDR improved by 15 dB' after enabling DEM without also reporting the noise floor is the kind of half-truth a customer finds in their own lab.

T6.5 What to do with this on Monday

Table 27. Where this chapter's concepts are actually used

Concept	Block it appears in	What it decides
flash ADC · SAR ADC · pipeline ADC · time interleaving	Architecture selection, first week	Which limit you will be paying
capacitive DAC · segmentation	Sizing the array before any transistor	Unit capacitor, total area, and the reference driver's current
redundancy · digital calibration	Speed and yield	How much settling accuracy the early cycles can be spared
INL · DNL · missing code	Characterisation and the data-sheet	Whether the part is usable in a control loop, not just in a spectrum
glitch energy	Any DAC observed in the frequency domain	SFDR, and how many thermometer bits to pay for

The number to carry from this chapter: at 12 bits a SAR's capacitor array is **4.91× larger than noise alone would require**, because matching sets it — and at 14 bits that factor is 1.23, where noise has taken over. Design the 12-bit part from the matching requirement and the 14-bit part from kT/C , and you are sizing the same array by two different physics because the crossover sits between them.

The next chapters leave the converter and take up the circuit that decides *when* all of this happens — the phase-locked loop — where the same trade between noise, bandwidth and power appears in a feedback system rather than in a capacitor.

Where the inventions are — what is patentable here

Converter patents cluster in three places and the pattern is worth seeing: **switching schemes** that lower the energy of a SAR's capacitive DAC (monotonic, split-cap, V_{cm} -based — all the same array, switched differently); **calibration** that measures a physical error once and subtracts it in the digital domain forever; and **interleaving corrections** for gain, offset and timing skew between slices. All three share a shape: *accept the analog error and remove it somewhere cheaper*. That is the most reliably patentable idea in mixed-signal design, and it is available in almost every block.

T7. The Phase-Locked Loop — a Feedback System Whose Output Is Time

Every other block treats the clock as given. This one makes it, and the trade it faces has no good side: the bandwidth that suppresses one noise source lets in the other.

Before you read this

voltage · current · frequency · phase · integration · differentiation · logarithms · probability · standard deviation · transconductance · output resistance · RC delay · thermal noise · flicker noise · aperture jitter · SNR · noise bandwidth

New in this chapter

phase detector · charge pump · loop filter · VCO · divider · loop gain · damping factor · natural frequency · loop bandwidth · phase noise · integrated jitter · reference spur · fractional-N · delta-sigma modulation · spread spectrum · lock detect · supply pushing

A PLL is the only block in this book whose output quantity is *time*. Everything else in the chapters so far has treated the clock as an input with some jitter attached; here the jitter is manufactured, and the design question is how to manufacture as little of it as possible from two sources that respond to the loop in opposite ways.

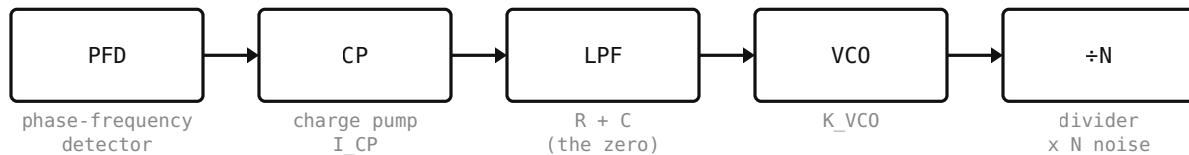


Figure 12. The charge-pump PLL. The divider closes the loop, and it is also what multiplies every noise source inside the loop by N .

T7.1 What the loop actually controls

Derivation — From the phase detector to a second-order system

	This follows	Because
1	The phase-frequency detector produces a signal proportional to the phase <i>error</i> between the reference and the divided output.	Phase, not frequency — frequency is the derivative, which is why a PLL locks frequency as a side effect of locking phase and why the loop has an inherent integrator.
2	The charge pump converts that error into a current of $\pm I_{CP}$, applied for the duration of the error pulse.	The average current is $I_{CP} \cdot \Delta\phi / 2\pi$ — a linear phase-to-current gain of $I_{CP} / 2\pi$ A/rad.
3	The filter integrates that current on C , and the VCO integrates voltage into phase with gain K_{VCO} .	Two integrators in the loop. That is why a plain capacitor makes the loop unconditionally unstable and why a zero (the series R) is not optional.
4	Open-loop gain: $G(s) = (I_{CP}/2\pi) \cdot (1/sC)(1+sRC) \cdot (K_{VCO}/s) \cdot (1/N)$.	Two poles at the origin and a zero at $1/RC$. The zero supplies the phase margin.
5	Closed-loop this is a standard second-order system with $\omega_n = \sqrt{(I_{CP}K_{VCO})/(2\pi NC)}$ and $\zeta = (R/2)\sqrt{(I_{CP}K_{VCO}C)/(2\pi N)}$.	Everything else in this chapter is an argument about where to place those two numbers.
6	For the worked design ($f_0 = 2.5$ GHz, $f_{ref} = 25$ MHz, $N = 100$, $I_{CP} = 1$ μ A, $K_{VCO} = 5$ MHz/V), targeting $f_L = 1$ MHz with $\zeta = 0.9$: $C = 41$ pF, $R = 12.6$ k Ω .	Settling to 1 % takes about $4/(\zeta\omega_n) = 1.27$ μ s — which is the number a system architect actually asks for.

The divider is where the unpleasantness enters. Every noise source *inside* the loop — reference, PFD, charge pump, divider — is multiplied by N^2 in power at the output, which is 40 dB for $N = 100$. This is why a PLL that multiplies by 100 has a far worse in-band phase noise floor than the crystal it started from, and why designs that need very low jitter use the highest reference frequency they can (a smaller N), not the most convenient one.

T7.2 The trade with no good side: loop bandwidth

Which noise the loop suppresses, and which it lets through
The loop is a **low-pass** for everything inside it (reference, PFD, charge pump, divider) and a **high-pass** for the VCO. Inside the loop bandwidth the feedback forces the output to track the reference, so the VCO's own noise is corrected; outside it, the loop is too slow to correct anything and the VCO runs free.

So widening the loop suppresses more VCO noise and admits more reference-path noise; narrowing it does the reverse. **There is always an optimum, and it is the only design variable in the loop that has one.**

Where it is used	Any clock generator, every SerDes transmit and receive PLL, the clock for a data converter, and the on-chip clock of an SoC that must meet a jitter budget.
When it binds	Fixed early: the loop bandwidth constrains the VCO specification and the reference choice, and changing it late changes both.
How to apply it	Integrate the shaped densities and sweep. The optimum is usually where the in-band floor and the VCO's free-running noise cross — which can be read straight off a phase-noise plot without any integration.

What the industry code looks like

```
# Phase noise measurement, and the integration that turns it into jitter.
# The instrument gives L(f) in dBc/Hz; the number the customer wants is rms jitter.
#  sigma_t = sqrt( 2 * integral( 10^(L(f)/10) df ) ) / (2*pi*f0)
#  THE INTEGRATION LIMITS ARE PART OF THE SPECIFICATION.
#  PCIe: 10 kHz - 20 MHz (with the protocol's own filter applied)
#  JESD204B: 100 Hz - 20 MHz
#  Ethernet: often 12 kHz - 20 MHz
#  A jitter number quoted without its limits is not comparable to any other one.
```

This is where it goes wrong. Integrating from 'DC' is meaningless — the integral of the VCO's $1/f^3$ region diverges, and every standard therefore specifies a lower limit. Two vendors quoting 'sub-picosecond jitter' over different limits are not making comparable claims, and the narrower limit usually belongs to the worse part.

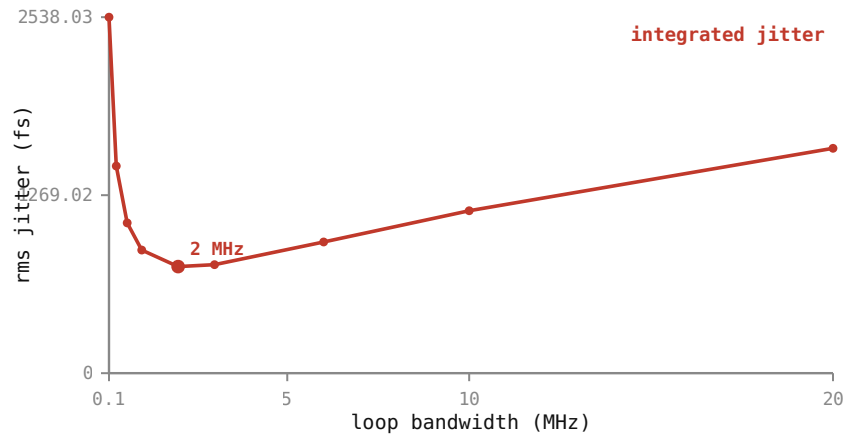


Figure 13. Integrated jitter against loop bandwidth for the worked design (in-band floor ≈ -110 dBc/Hz, VCO ≈ -100 dBc/Hz at 1 MHz). The optimum is 2 MHz at 76 fs rms — and it is **shallow**: 1 MHz and 3 MHz are within 16 % of it. Choose the side that eases whichever block is harder.

Table 28. Integrated jitter against loop bandwidth (computed, not recalled)

Loop BW	rms phase	rms jitter @ 2.5 GHz
0.1 MHz	39,867 μ rad	2,538 fs
0.3 MHz	23,197 μ rad	1,477 fs
0.6 MHz	16,829 μ rad	1,071 fs
1 MHz	13,789 μ rad	877.8 fs
2 MHz	11,936 μ rad	759.9 fs
3 MHz	12,152 μ rad	773.6 fs
6 MHz	14,692 μ rad	935.3 fs
10 MHz	18,191 μ rad	1,158 fs
20 MHz	25,183 μ rad	1,603 fs

The shallowness is the practical result. A factor of three in loop bandwidth costs a few per cent of jitter, so the bandwidth should be chosen for the **other** constraints it controls — settling time, reference spur suppression, tolerance to VCO pulling — rather than squeezed against a flat optimum. Engineers who spend a week finding the exact minimum have optimised the one axis that does not move.

T7.3 Spurs: the errors that are not noise

reference spur (기준스퍼) — A discrete tone at an offset equal to the reference frequency (and its harmonics), caused by any periodic disturbance of the control voltage at the comparison rate — charge-pump up/down mismatch, leakage on the filter capacitor, or coupling from the reference into the VCO.

Table 29. Where a spur comes from, and what actually removes it

Cause	Mechanism	Fix that works	Fix that does not
CP current mismatch	Up and down currents differ, so the loop must sit at a non-zero phase error and the pump fires every cycle	Replica bias that serves I_{up} to I_{dn}	Lowering I_{CP} — it scales the spur and the loop gain together
Filter leakage	Capacitor and switch leakage drain the control node between updates	Thicker-oxide devices on the filter node; larger C	More loop gain — the disturbance is at the input

Cause	Mechanism	Fix that works	Fix that does not
Supply / sub- strate coupling	Reference edges couple into the VCO directly	Physical separation, a regulator for the VCO, differential tuning	Anything inside the loop — the loop never sees this path
Fractional-N quantisation	The divider alternates between integers, so the phase error is periodic by construction	$\Delta\Sigma$ modulation to shape it, plus enough loop filtering	A slower reference

fractional-N (분수분주) and the noise it creates on purpose

An integer-N loop can only produce multiples of the reference. Dividing by an average of $N.f$ — switching between N and $N+1$ — produces any frequency, at the cost of a periodic phase error whose spectrum is concentrated at the fraction's repeat rate. A $\Delta\Sigma$ modulator **randomises and shapes** that sequence so the error becomes high-frequency noise, which the loop filter then attenuates.

The trade is explicit: fractional-N buys frequency resolution and pays for it in noise *outside* the loop bandwidth, which is exactly the region a wide loop cannot filter. **This is why fractional-N designs tend to use narrower loops than integer-N ones**, which in turn makes the VCO's own noise matter more — the chapter's central trade, reappearing.

Where it is used	Any system needing a frequency that is not a multiple of the available crystal: radio LOs, SerDes with several line rates from one reference, audio clocks, spread-spectrum clocking.
When it binds	When the required output frequency grid is finer than the reference. Prefer integer-N whenever the system architecture allows choosing the crystal — it is always quieter.
How to apply it	Choose the modulator order (2nd–3rd is usual), check for idle tones at rational fractions, and verify that the shaped noise is far enough outside the loop bandwidth that the filter's roll-off has caught it.

What the industry code looks like

```
// Behavioural PLL for system simulation -- real-valued, event driven.
// It models what the DIGITAL side needs (frequency, lock time, jitter injection)
// and nothing else. Replacing this with a transistor model in an SoC simulation is
// how a regression that used to take 10 minutes starts taking a week.
module pll_behav #(parameter real FREF=25.0e6, parameter int N=100,
                    parameter real LOCK_US=2.0, parameter real JIT_RMS=0.8e-12)
    (input logic ref_clk, rst_n, output logic out_clk, output logic locked);
    real period = 1.0/(FREF*N);
    initial begin out_clk = 0; locked = 0; end
    always #(1s*period/2) out_clk = ~out_clk; // ideal edges
    initial begin @(posedge rst_n); #(LOCK_US*1us); locked = 1; end
endmodule

// SDC: the output is a GENERATED clock -- declaring it as a second create_clock
// makes STA treat the two domains as unrelated and silently stops checking paths.
// create_generated_clock -name pll_out -source [get_ports ref_clk] \
//     -multiply_by 100 [get_pins u_pll/out_clk]
```

This is where it goes wrong. Declaring the PLL output with `create_clock` instead of `create_generated_clock` is a common and expensive SDC error: the tool then has no phase relationship between reference and output, treats every crossing between them as asynchronous, and **stops reporting the paths you care about** while the report still looks clean.

T7.4 What actually breaks

Table 30. Failures that do not show up in the loop equations

Failure	Symptom	Root cause	What finds it
Supply pushing	Output frequency moves with V_{DD} ; jitter correlates with digital activity	$K_{VCO, supply}$ is not zero — the VCO is a voltage-controlled oscillator in more ways than intended	Sweep the supply in simulation and measure df/dV ; budget it against the PDN's ripple (T13)
VCO pulling	A spur appears at the offset of a nearby strong signal	Injection locking through substrate or supply from another oscillator or a high-power output	Two-tone bench measurement; floor-plan review
False lock	Locks to a harmonic or to the wrong edge; works after reset sometimes	A phase detector without frequency capability, or a divider that starts in an illegal state	Start-up sweeps across corners and initial conditions — not one nominal run
Lock detect lies	'Locked' asserts before the loop has settled	The detector compares phase within a window that is wider than the specification needs	Check the detector's window against the actual settling time; make the system wait for both

What actually happened

Supply pushing is the one that ambushes digital-heavy SoCs. A VCO with a supply sensitivity of 1 %/V sitting on a rail with 30 mV of activity-dependent ripple sees its frequency modulated at whatever rate the digital load switches — which is not a constant, and is correlated with exactly the workload the customer runs. It appears in the lab as 'jitter that depends on the software'. The fix is a dedicated regulator for the VCO, and it must be designed in from the start because it changes the floorplan and the supply count.

T7.5 What to do with this on Monday

Table 31. Where this chapter's concepts are actually used

Concept	Block it appears in	What it decides
loop gain · damping factor · loop bandwidth	Loop filter design	C and R, and therefore settling time and area
phase noise · integrated jitter	The jitter budget, before the VCO is specified	Which of VCO noise or in-band floor you must spend on
reference spur	Spectrum compliance and receiver desense	Charge-pump topology and the filter node's leakage budget
fractional-N · delta-sigma modulation	Frequency planning	Whether the crystal can be chosen to avoid fractional-N entirely
supply pushing	Floorplan and PDN	Whether the VCO needs its own regulator — a decision that is hard to reverse

The numbers to carry: for this worked design the loop filter is $C = 41$ pF and $R = 12.6$ k Ω , settling in 1.27 μ s; the jitter optimum sits at 2 MHz of loop bandwidth and is **shallow enough that the bandwidth should be chosen for settling and spur requirements instead**. And the divider multiplies every in-band noise source by 40 dB — which is the strongest argument for the highest reference frequency the system can provide.

The next chapter puts this clock to work: a serial link, where the same jitter must be budgeted against a channel that has already closed most of the eye.

Where the inventions are — what is patentable here

PLL patents are almost never about the loop equation — that is 1930s control theory. They are about **what the loop is allowed to see**: fractional dividers whose quantisation noise is shaped away from the band that matters; charge pumps whose up/down mismatch is cancelled by replication rather than trimmed; digital loops that replace the analog filter with an accu-

mulator and therefore scale with the process; and spread-spectrum schemes that modulate the divider to move radiated energy off a regulated frequency. All of these are arrangements around an unchanged loop.

T8. The Serial Link — Measured, Including Where It Does Not Work

A link is the one block where the channel is someone else's problem and entirely your responsibility.

Before you read this

voltage · frequency · phase · probability · standard deviation · logarithms · integration · SNR · aperture jitter · phase noise · integrated jitter · loop bandwidth · aliasing · ENOB · thermal noise

New in this chapter

intersymbol interference · pulse response · cursor · pre-cursor · post-cursor · peak distortion · eye diagram · CTLE · FFE · DFE · error propagation · bit error rate · rule of three · clock and data recovery · jitter tolerance · PAM4

Everything in a serial link is a consequence of one fact: the channel is a low-pass filter with far less bandwidth than the symbol rate, so each symbol arrives smeared across its neighbours. The receiver's job is to undo that smearing well enough that a slicer can decide, and the measurements below — run with this repository's own channel and link model — show both how much equalisation buys and, more usefully, **where it buys nothing at all**.

T8.1 The pulse response, and the numbers that come out of it

cursor (커서) — The samples of the channel's pulse response taken at symbol intervals. The **main cursor** is the one the slicer uses; everything before it is a **pre-cursor** and everything after a **post-cursor**. The sum of the magnitudes of all non-main cursors is the worst-case ISI.

Table 32. Measured pulse response of this model's channel (sps = 8)

Loss	Main	Pre-cursor 1	Post-cursor 1	$\Sigma ISI $	Worst-case eye
10 dB	3.388	0.348	1.329	4.234	-1.692
20 dB	1.693	0.547	1.035	5.444	-7.502
30 dB	0.918	0.487	0.744	5.681	-9.525

Read the last column carefully. At **every** loss — including 10 dB, where the link runs essentially error-free — the worst-case eye is **negative**. That is not a contradiction: worst-case (peak-distortion) analysis assumes the data pattern that maximises every interferer simultaneously, and that pattern is astronomically rare. **Peak distortion is a bound, not a prediction**. A design signed off on it alone is over-designed; a design that ignores it has no bound at all. Both numbers belong in the deliverable.

Derivation — Why loss shows up as post-cursors, and why that matters

	This follows	Because
1	A lossy channel attenuates high frequencies more than low ones.	Skin effect ($\propto \sqrt{f}$) and dielectric loss ($\propto f$). Both are physics of the medium, not of the driver.
2	A sharp edge is therefore spread in time: the pulse response develops a long tail.	Measured at 20 dB: the first post-cursor is 1.035 against a main cursor of 1.693 — 61.1 % of the signal arrives one symbol late .
3	The tail is <i>causal and past</i> — it depends on symbols already decided.	That is exactly what makes a decision-feedback equaliser possible: the receiver already knows those bits, so it can subtract their contribution without amplifying noise.
4	Pre-cursors are not, so they can only be handled by a filter that looks <i>ahead</i> — an FFE, which is linear and therefore amplifies noise along with signal.	Measured pre-cursor at 20 dB is 0.547, a third of the post-cursor. This asymmetry is why receivers use both kinds of equaliser rather than one.

T8.2 What each equaliser actually buys — measured

Table 33. BER against channel loss and equalisation (SNR = 26 dB, PRBS, seeds 1·2·3 combined, 420,000 bits per cell)

Loss	No EQ	CTLE 6 dB	+ FFE 3	+ DFE 4
10 dB	2.38×10^{-6}	$< 7.14 \times 10^{-6}$ (0 errors, rule of three)	$< 7.14 \times 10^{-6}$ (0 errors, rule of three)	$< 7.14 \times 10^{-6}$ (0 errors, rule of three)
20 dB	0.14	0.01	0.01	0
30 dB	0.27	0.23	0.25	0.25

What actually happened

The 30 dB row is not a result — it is a broken operating point. Every configuration sits at a BER of roughly 0.2, which is what a slicer does when the eye is fully closed: it guesses. Comparing equalisers there is comparing coin flips, and the apparent finding that FFE is worse than CTLE alone (0.25 against 0.23) is noise, not physics.

This repository has a rule for exactly this, learned the hard way: **check the operating point is healthy before reporting anything measured on it.** A BER near 0.5, an equaliser that did not converge, a wrong sampling phase — all of them produce numbers that look like data.

Table 34. Finding a healthy operating point: SNR sweep at 20 dB loss with CTLE + FFE 5 + DFE 6 (630,000 bits per row)

SNR	Errors	BER
26 dB	2226	0
30 dB	9	1.43×10^{-5}
34 dB	0	$< 4.76 \times 10^{-6}$ (0 errors, rule of three)
38 dB	0	$< 4.76 \times 10^{-6}$ (0 errors, rule of three)

Table 35. The same comparison at a healthy operating point (20 dB loss, SNR = 34 dB)

Configuration	Errors	BER
none	81496	0.13
CTLE	40	6.35×10^{-5}
CTLE+FFE5	0	$< 4.76 \times 10^{-6}$ (0 errors, rule of three)
CTLE+FFE5+DFE6	0	$< 4.76 \times 10^{-6}$ (0 errors, rule of three)

rule of three (3의법칙) — what zero errors is allowed to mean

Two rows above report **zero errors**. Zero errors is not a BER of zero; it is an upper bound. With no errors in n bits, the 95 % confidence upper bound on the error rate is $3/n$ — the **rule of three**. For the 630,000-bit runs that is 4.76×10^{-6} .

So the measurement says CTLE + FFE and CTLE + FFE + DFE are both below 4.76×10^{-6} , and it says **nothing about which is better**. Reporting that DFE 'made no difference' would be a claim the data does not support; the honest statement is that this run could not distinguish them, and that distinguishing them needs roughly 100× more bits or a worse channel.

Where it is used	Every BER measurement, every compliance report, and every datasheet line that contains the letters BER.
When it binds	Whenever the error count is small — which, for any link worth shipping, is always.
How to apply it	Report errors and bits, not just the ratio. If errors = 0, quote the $3/n$ bound and the confidence level. To claim $1e-12$ you must observe at least $3e12$ bits, which at 25 Gb/s is two minutes — and at $1e-15$, a day.

What the industry code looks like

```
# The line that belongs in every link report.
# errors = 0 in 6.30e5 bits -> BER < 4.8e-6 (95% confidence, rule of three)
# NOT "BER = 0".
def ber_bound(errors, bits, conf=0.95):
    if errors == 0:
        import math
        return -math.log(1 - conf) / bits          # 3.0/bits at 95%
    return errors / bits                          # point estimate; add a CI for small n
# And the check that catches the broken operating point before any of this matters:
assert 0.0 <= ber < 0.2, f"BER {ber} -- the eye is closed; this run measured nothing"
```

This is where it goes wrong. A BER of exactly 0.5 usually means a polarity or alignment bug, not a bad channel — the receiver is comparing against an inverted or misaligned reference. A BER near 0.25 with PAM4 means the same thing one level up. **Check for these before blaming the equaliser:** they are far more common than a genuinely closed eye.

T8.3 The three equalisers, and the one that can make things worse

Table 36. What each equaliser does to signal and to noise

	CTLE	FFE	DFE
Where	Analog, before sampling	Analog or digital, linear FIR	After the slicer, feedback
Handles	Broad high-frequency loss	Pre- and post-cursors	Post-cursors only
Noise	Amplifies it with the signal	Amplifies it — gain at high frequency is noise gain	Does not — it subtracts known symbols
Cost	Small; a few devices and a peaking network	Taps × multipliers; the tap count grows with the tail	A multiply-add inside one symbol period — the timing wall of T2
Fails when	The loss is not a smooth slope (reflections)	Noise is already the limit	A wrong decision propagates (error burst)

error propagation (오류전파), and why DFE is still worth it

A DFE subtracts the contribution of symbols it has already decided. If a decision is wrong, it subtracts the wrong thing, which makes the next decision more likely to be wrong — errors arrive in bursts rather than singly.

The effect is real and it is smaller than intuition suggests: for a one-tap DFE the burst length is short because the error has to survive the next correct decision. What matters downstream is not the BER but the **burst structure**, because the FEC that follows was designed for a burst length. A link that meets its BER and violates the assumed burst distribution will fail after decoding while every pre-FEC measurement looks fine.

Where it is used	Every receiver above ~10 dB of loss, and every conversation with the FEC designer.
When it binds	Whenever the post-cursor tail is significant relative to the main cursor — at 20 dB in this model the first post-cursor alone is 61.1 % of the main.
How to apply it	Measure the burst-length distribution, not just BER, and hand it to whoever sizes the interleaver. Unrolling (speculation) removes the feedback multiply from the critical path — the T2 argument — at the cost of 2^k parallel slicers for k unrolled taps.

What the industry code looks like

```
// Unrolled (speculative) first DFE tap: both outcomes are precomputed,
// and the previous decision only selects. The multiply leaves the loop.
always_comb begin
    d_if_prev0 = (sample > +c1) ;      // precomputed thresholds
    d_if_prev1 = (sample > -c1) ;
end
always_ff @(posedge clk) prev <= prev ? d_if_prev1 : d_if_prev0;  // 2:1 mux only
// The loop now contains a mux and a flop instead of a multiply-add.
// Cost: one extra slicer per unrolled tap, and 2^k for k taps.
```

This is where it goes wrong. Unrolling is only a win if the loop was the critical path. Measure first: in this repository's own measurement of a DFE at a realistic operating point the feedback taps were $|c| \leq 0.041$ — **the loop was idle**, and the real critical path was the slicer. Optimising an idle loop is the most expensive kind of correct work.

T8.4 Recovering the clock, and the jitter that decides whether you can

CDR (클럭데이터복원) — the PLL of T7 with the data as its reference

There is no separate clock wire. The receiver extracts timing from the data's own transitions using a phase detector (Alexander/bang-bang, or a linear Mueller-Müller) driving the same second-order loop as T7. Everything from that chapter applies, with two changes: the 'reference' arrives only when the data transitions, and the loop must **track** the transmitter's spread-spectrum modulation while **rejecting** high-frequency jitter.

That is the jitter tolerance mask: below the CDR bandwidth the receiver follows the jitter (so large amplitudes are tolerated); above it, the receiver cannot follow and the jitter eats the eye directly.

Where it is used	Every receiver in every serial standard — PCIe, USB, Ethernet, MIPI, SATA, HDMI.
When it binds	CDR bandwidth is fixed by the standard's jitter-tolerance mask and by the spread-spectrum profile it must track; it is not a free parameter.
How to apply it	Take the standard's mask, place the CDR bandwidth so the tolerance curve sits above it with margin, and check that the transmit PLL's jitter (T7) plus the residual falls inside the budget.

What the industry code looks like

```
# The budget is a SUM, and each term has an owner.
# TX PLL jitter (T7)          0.30 ps rms    <- clock design
# TX driver / ISI (DJ)       0.15 UI pp     <- output stage and package
# Channel ISI residual       0.20 UI pp     <- equalisation, this chapter
# RX CDR tracking error      0.05 UI pp     <- loop bandwidth
# Sampler aperture (T5)      0.10 ps rms    <- sampling network
# Random terms add in QUADRATURE, deterministic ones LINEARLY -- mixing the two
# rules is the most common budgeting error, and it always errs optimistic.
```

This is where it goes wrong. Spread-spectrum clocking is a **requirement to track**, not a disturbance to reject: a CDR whose bandwidth is too low to follow a 33 kHz SSC profile loses lock on a compliant transmitter. A receiver that works on the bench with a clean generator and fails against real hosts is usually this.

T8.5 What to do with this on Monday

Table 37. Where this chapter's concepts are actually used

Concept	Block it appears in	What it decides
pulse response · cursor · peak distortion	Before choosing any equaliser	How many taps of what kind — and the bound that goes in the datasheet
CTLE · FFE · DFE	Receiver architecture	Noise amplification versus error propagation, and the timing wall
rule of three	Every BER number you report	Whether the measurement supports the claim at all
error propagation	The interface with FEC	Interleaver depth — the number the FEC designer needs from you
clock and data recovery · jitter tolerance	Compliance	CDR bandwidth, which the standard has already chosen for you

Two measured results to carry. At 20 dB of loss and SNR 34 dB, CTLE alone takes the link from 0.13 to 6.35×10^{-5} — a factor of about 2,037 — and adding FFE pushes it below what 630,000 bits can resolve. And at 30 dB of loss with the same SNR, **no equaliser does anything at all**, because the operating point itself is broken. Reporting the second table without that sentence would be the exact failure this book's rules exist to prevent.

The next chapter goes underneath the model used here and asks where the channel's loss actually comes from — and what a package, a via and a connector do to the numbers above.

Where the inventions are — what is patentable here

Link patents concentrate where the loop is: speculative (unrolled) DFE that removes the feedback multiply from the critical path; adaptation schemes that converge without a training sequence; CDR phase detectors that tolerate more ISI; and, lately, ADC-based receivers whose equalisation is entirely digital. **This repository's own research sits in that first family**, and the argument there is the one from T2: the invention is *where the delay was moved*, not how fast the transistor is.

T9. The Channel — Where the Loss Comes From, Term by Term

Loss you can equalise. Reflections you cannot. Knowing which one you have is the difference between a receiver design and a board redesign.

Before you read this

voltage · frequency · phase · logarithms · integration · intersymbol interference · pulse response · cursor · CTLE · FFE · DFE · eye diagram

New in this chapter

characteristic impedance · transmission line · insertion loss · skin effect · dielectric loss · reflection coefficient · return loss · notch · via stub · back-drilling · NEXT · FEXT · S-parameters · channel specification · COM

The previous chapter treated the channel as a loss number. That number is a sum of physically distinct terms which scale differently with frequency, come from different parts of the system, and — crucially — respond differently to equalisation. A receiver designer who knows only 'the channel is 20 dB' cannot tell whether the design is difficult or impossible.

T9.1 The four terms, and how to tell them apart from one measurement

insertion loss (삽입손실) as a fitted sum

The IEEE 802.3 channel specifications fit measured insertion loss with $IL(f) = a_0 + a_1\sqrt{f} + a_2f + a_4f^2$, and each term has a physical owner:

- a_0 (**flat**) — connectors, bond wires, anything whose loss does not care about frequency.
- $a_1\sqrt{f}$ — **skin effect**. Current crowds into a surface layer whose depth goes as $1/\sqrt{f}$, so the effective resistance goes as $\sqrt{f}$. This is the copper.
- a_2f — **dielectric loss**. The board material's loss tangent; energy goes into heating the substrate. This is the laminate choice.
- a_4f^2 — **roughness and radiation**. Surface treatment of the copper, and whatever the trace radiates.

Fitting a measurement to this form tells you **which purchase order fixes it**: better copper, better laminate, fewer connectors, or none of the above.

Where it is used	Reading any channel specification, budgeting a link before a board exists, and arguing with the system architect about the laminate.
When it binds	At the very start. The fit decides whether the receiver needs 3 taps or 15, and that decision sizes the whole receiver.
How to apply it	Take the measured S21 in dB, fit the four coefficients over the band of interest, and look at which term dominates at Nyquist. Below is the same decomposition measured on this repository's channel model.

What the industry code looks like

```
# Fitting a real S21 to the standard form -- four numbers that a board
# engineer can act on. (numpy, on a Touchstone file read with scikit-rf.)
import numpy as np, skrf
ntwk = skrf.Network('channel.s4p')
f = ntwk.f / f_nyquist # normalise to Nyquist
IL = 20*np.log10(np.abs(ntwk.s[:, 1, 0])) # dB, and it is NEGATIVE
A = np.vstack([np.ones_like(f), np.sqrt(f), f, f*f]).T
```



```
a, *_ = np.linalg.lstsq(A, IL, rcond=None)
print(f"flat {a[0]:.2f} dB | skin {a[1]:.2f} | dielectric {a[2]:.2f} | rough {a[3]:.2f}")
# If |a[2]| dominates at Nyquist, the fix is laminate, not copper.
```

This is where it goes wrong. A fit over the wrong band tells you the wrong thing. Fit over the band the **signal** occupies (DC to roughly $1.5\times$ Nyquist), not over whatever range the VNA happened to sweep — the f^2 term in particular is nearly unconstrained below Nyquist and will absorb whatever the fit needs.

Table 38. Measured insertion loss of this model's channel, by term (f in units of $1/UI$; Nyquist = 0.5)

Channel	0.125	0.25	0.5 (Nyquist)	1.0
conductor only (12 dB)	-6 dB	-8.5 dB	-12 dB	-17 dB
dielectric only (8 dB)	-2 dB	-4 dB	-8 dB	-16 dB
both	-8 dB	-12.5 dB	-20 dB	-33 dB

Derivation — Reading the physics straight out of that table

	This follows	Because
1	The skin-effect row goes $-6.0 \rightarrow -8.5 \rightarrow -12.0 \rightarrow -17.0$ dB as frequency doubles.	Each step multiplies by $1.417 \approx \sqrt{2}$ — which is what $a_1\sqrt{f}$ predicts. The model implements what the standard's form says, and the measurement confirms it.
2	The dielectric row goes $-2.0 \rightarrow -4.0 \rightarrow -8.0 \rightarrow -16.0$ dB.	Exactly $2\times$ per doubling — the a_2f term, confirmed the same way.
3	At low frequency the conductor term dominates (-6.0 against -2.0); by $1.0/UI$ they are comparable (-17.0 against -16.0).	The dominant loss mechanism changes across the band , so 'the channel is copper-limited' is only ever true at a stated frequency.
4	The combined row is the sum of the two in dB, to within the read-out precision.	$-6.0 + -2.0 = -8.0$ measured; $-12.0 + -8.0 = -20.0$ measured. The terms are independent , which is why fitting them separately is legitimate.

T9.2 Reflections: the part equalisation cannot fix

transmission line, Z_0 (전송선 · 특성임피던스) — Above the frequency at which a trace is a meaningful fraction of a wavelength, it stops being a wire and becomes a **transmission line**: a structure that carries a travelling wave whose voltage-to-current ratio is fixed by geometry and dielectric, $Z_0 = \sqrt{L/C}$ per unit length. A wave meeting a different Z_0 is partly reflected — which is the subject of the rest of this section, and the reason 'impedance' is a specification on a board and not just a number in Ohm's law.

reflection coefficient (반사계수), ρ — At an impedance discontinuity, $\rho = (Z_2 - Z_1)/(Z_2 + Z_1)$ of the incident wave is sent back. A 10 % impedance error gives $\rho \approx 0.05$; a via stub, a connector footprint or a package ball can give far more.

Derivation — Where a reflection puts its notch, and how deep

	This follows	Because
1	A reflection returns after a round-trip delay τ and adds to the direct path.	One echo, one delay — the simplest case, and the one that already explains most measured channels.
2	The transfer function becomes $H(f) \cdot (1 - \rho e^{-j2\pi f\tau})$ for an inverting reflection.	Direct plus delayed copy: interference, not attenuation.
3	It cancels where the delayed copy is in anti-phase, i.e. at $f = 1/(2\tau)$, $3/(2\tau)$, ...	The notch position tells you the distance — and in a real board that is how a stub is identified.
4	Its depth is $20 \cdot \log_{10}(1 - \rho)$.	Depth measures the magnitude of the discontinuity.

Table 39. Measured against the closed form — an independent cross-check

p	Delay (UI)	Notch position (measured)	Position (theory)	Depth (measured)	Depth (theory)
0.2	4	0.125 /UI	0.125 /UI	-1.94 dB	-1.938 dB
0.35	11	0.045 /UI	0.045 /UI	-3.75 dB	-3.742 dB
0.35	2	0.25 /UI	0.25 /UI	-3.74 dB	-3.742 dB

What actually happened

The first attempt at that measurement reported the notch at 0.894/UI for a reflection whose theory says 0.125/UI — a factor of seven wrong, from a plot that looked entirely reasonable. The cause: the code took the **minimum of the insertion loss**, and at high frequency the smooth loss is far deeper than any notch, so the minimum was simply the top of the band. **The trivial explanation had not been excluded.** Dividing by the reflection-free channel first — comparing like with like — put every number within 0.2 % of theory.

Why a notch is worse than loss of the same depth

A linear equaliser inverts the channel. Inverting a smooth roll-off costs noise amplification proportional to the gain applied — expensive but bounded. Inverting a **notch** requires enormous gain at one frequency, and at that frequency the received signal is not merely small, it has been cancelled: there is almost no signal to amplify, only noise. **The information at that frequency is gone.**

This is why the same 20 dB of flat loss and 20 dB of notch are entirely different problems, and why board reviews chase impedance discontinuities rather than trace length.

Where it is used	Board and package review, connector selection, and the decision of whether a channel is workable at all before a receiver is designed.
When it binds	Before committing to a data rate. A notch inside the signal band is a board fix, not a receiver fix.
How to apply it	Look at return loss (S11) , not only insertion loss. A channel with good IL and poor RL has its energy going somewhere, and a DFE can cancel a <i>post-cursor</i> echo but only if it lands within the tap span — an echo 40 UI away needs 40 taps.

What the industry code looks like

```
# Where the echo lands decides whether ANY receiver can fix it.
# round-trip 40 mm on FR4, eps_eff ~ 4.0 -> t = 2*0.040/(3e8/2) = 533 ps
# at 25 Gbaud (40 ps/UI) -> 13 UI away
# A 13-tap DFE can reach it; a 4-tap one cannot, and no amount of CTLE helps.
# THIS calculation belongs in the datasheet's integration section, because the
# customer's board is what decides it.
```

This is where it goes wrong. A DFE cancels a reflection only if the reflection is **past** the main cursor and within the tap span. Pre-cursor reflections (from a discontinuity near the transmitter) are not reachable by a DFE at all, and an FFE that tries amplifies the noise at exactly the frequency the notch removed.

T9.3 Crosstalk, and which kind actually hurts**Table 40. Near-end and far-end crosstalk**

	NEXT (near end)	FEXT (far end)
Where it couples	Aggressor's transmitter into your receiver, at the same end	Aggressor's transmitter into your receiver, at the far end

	NEXT (near end)	FEXT (far end)
Attenuated by the channel?	No — it never traverses the channel	Yes — it travels the same lossy path as the signal
Relative to the signal	Full-strength aggressor against your channel-attenuated signal	Both attenuated, so the ratio is roughly preserved
Grows with	Coupling length and edge rate	Coupling length, and it grows with frequency
Fixed by	Separation, shielding, and not routing TX beside RX	Spacing, dielectric choice, and cancellation in the receiver
Which dominates	Long, lossy channels — the signal is small and the aggressor is not	Short channels and dense parallel buses

The asymmetry in that table is the practical point. On a 30 dB channel your signal arrives 30 dB down while a NEXT aggressor arrives at full strength — so an aggressor coupled at -40 dB is only 10 dB below your signal. **NEXT is a layout-review item, not a receiver item**: no equaliser can subtract a signal it has never seen. FEXT, by contrast, has traversed the same channel and is in some architectures *cancellable*, because the aggressor's data is available on the same die.

T9.4 The via, the package, and what the standards do about it

via stub (비아 스텝) and back-drilling

A via that carries a signal from layer 2 to layer 3 in a twelve-layer board leaves the rest of the barrel — layers 4 to 12 — connected and unused. That stub is an open transmission line, and an open stub of length $\lambda/4$ presents a **short** at its resonant frequency: the signal is shorted to nothing at $f = c/(4 \cdot L \cdot \sqrt{\epsilon_r})$.

The fix is mechanical: **back-drill** the unused barrel away, leaving a stub short enough that the resonance sits above the band. It costs a manufacturing step, and the decision belongs to whoever writes the channel specification — which is why an IP datasheet should state the channel it assumes rather than a bare 'supports 25 Gb/s'.

Where it is used	Any board with more than a few layers running above ~10 Gb/s; the package substrate; the connector footprint.
When it binds	Board stack-up review — long before silicon exists, and the one review where an IP vendor's input is genuinely valuable.
How to apply it	Compute the resonance from stub length, compare against 1.5× Nyquist, and if it lands in band, back-drill or change layers. State the assumed channel — including whether back-drilling is assumed — in the integration guide.

What the industry code looks like

```
# Stub resonance, and the line that belongs in INTEGRATION.md.
# stub 1.5 mm, eps_r 4.0 -> f = c/(4*L*sqrt(eps_r))
#                               = 3e8/(4*1.5e-3*2) = 25 GHz
# At 25 Gbaud (Nyquist 12.5 GHz) that is 2x Nyquist -- survivable.
# At 50 Gbaud (Nyquist 25 GHz) it sits ON Nyquist -- fatal, back-drill required.
#
# INTEGRATION.md:
# Channel assumed: <= 20 dB insertion loss at Nyquist, return loss <= -12 dB,
# no via stub resonance below 1.5x Nyquist (back-drilling assumed above 25 Gb/s).
```

This is where it goes wrong. The resonance depends on $\sqrt{\epsilon_r}$, and the effective ϵ_r of a via barrel is not the laminate's datasheet number — it is closer to the bulk value than the stripline's effective value. Using the wrong one moves the predicted resonance by tens of per cent, which is the difference between 'above the band' and 'in it'.

Table 41. How a channel is actually specified, and what each part binds

Specification	What it is	What it binds
S-parameters (.s4p)	Measured 4-port network, differential	Everything — the other rows are summaries of it
Insertion loss at Nyquist	S21 in dB	Equaliser strength; the first number anyone quotes
Return loss (S11)	Reflected energy	Whether the loss is smooth or has notches
ICR / ICN	Integrated crosstalk ratio / noise	Layout separation and connector choice
COM (channel operating margin)	A single dB figure from a standardised reference receiver applied to the measured S-parameters	Pass/fail for Ethernet compliance — and it is the number the customer will run , not your simulation

COM deserves a sentence of its own. It takes the measured channel, applies a **specified** reference transmitter and receiver (fixed equaliser architecture, fixed noise, fixed jitter), and returns one number in dB. That makes channels comparable, which is its purpose — but it also means a receiver that is better than the reference gets no credit, and one that is differently structured may be penalised. **Know your COM before promising a channel reach**, because the customer will compute it and will not care that your receiver is cleverer.

T9.5 What to do with this on Monday

Table 42. Where this chapter's concepts are actually used

Concept	Block it appears in	What it decides
insertion loss · skin effect · dielectric loss	Before sizing the receiver	Tap count, and which purchase order would reduce the problem
reflection coefficient · notch	Board and package review	Whether the channel is workable at all — equalisation will not fix a notch
NEXT · FEXT	Layout review	Separation rules, and whether cancellation is even possible
via stub · back-drilling	Stack-up, before fabrication	A manufacturing step, and a line in the integration guide
S-parameters · COM	The customer conversation	The number they will actually compute — make sure you computed it first

The measured results to carry: in this model the skin-effect term rises as $\sqrt{f}$ (6 dB $\rightarrow$ 8.5 dB $\rightarrow$ 12 dB $\rightarrow$ 17 dB across four doublings) while the dielectric term rises as f (2 dB $\rightarrow$ 4 dB $\rightarrow$ 8 dB $\rightarrow$ 16 dB) — so which one dominates depends on where you look. And a reflection of $\rho = 0.2$ at 4 UI puts a -1.94 dB notch at exactly 0.12/UI, matching the closed form to within 0.2 %: **reflections are predictable, and that is precisely why they should be designed out rather than equalised.**

The next chapter returns to the inside of the chip and to a circuit family that trades bandwidth for accuracy in a way no continuous-time circuit can: switched capacitors.

Where the inventions are — what is patentable here

Channel patents belong to the board and connector houses, not to the IP house — but the **receiver** patents that matter are all responses to channel pathology: equalisers that adapt to a notch instead of fighting it, schemes that detect and cancel a specific reflection, and crosstalk cancellation that uses the aggressor's own data as a reference. Read this chapter as a catalogue of the problems your receiver will be asked to solve, and notice which of them equalisation *cannot* solve — those are where the inventions are.

T10. Switched Capacitors — Accuracy Bought with Time

A resistor whose value is a capacitor ratio and a clock. That one substitution is why precision analog survived the move to digital processes.

Before you read this

voltage · current · charge · resistance · frequency · integration · logarithms · kT/C noise · charge injection · bootstrapped switch · sampling theorem · RC delay · logical effort · matching · INL · DNL

New in this chapter

equivalent resistance · two-phase clocks · non-overlap · bottom-plate sampling · switched-capacitor integrator · feedback factor · settling time · finite-gain error · correlated double sampling · auto-zeroing · charge conservation · parasitic-insensitive

A precision analog circuit needs an accurate coefficient, and in an integrated process there is exactly one accurate thing: the **ratio** of two identically drawn structures. Absolute resistance varies $\pm 20\%$ over process; absolute capacitance likewise; but the ratio of two capacitors drawn side by side matches to a fraction of a per cent. Switched-capacitor technique is the discipline of expressing everything you need in terms of that ratio — and paying for it with sampling, and therefore with time.

T10.1 A resistor made of a capacitor and a clock

Derivation — From charge transfer to an equivalent resistance

	This follows	Because
1	Charge a capacitor C to V_1 , then dump it onto a node at V_2 . The charge moved is $\Delta Q = C(V_1 - V_2)$.	This is the whole circuit: two switches and one capacitor, driven by non-overlapping clocks.
2	Repeat at f_s . The average current is $I = f_s \cdot C \cdot (V_1 - V_2)$.	Charge per cycle times cycles per second.
3	That is Ohm's law with $R_{eq} = 1/(f_s C)$.	For $C = 1$ pF and $f_s = 10$ MHz, $R_{eq} = 100$ k Ω — a resistor that would be enormous in silicon, built from a capacitor that is not.
4	A time constant made this way is $R_{eq} C_2 = C_2/(f_s C_1)$ — a capacitor ratio divided by a clock frequency .	Absolute values cancel. The accuracy of the circuit is now the accuracy of a ratio (fractions of a per cent) and of a crystal (parts per million), instead of the accuracy of an absolute RC (tens of per cent).

Table 43. Equivalent resistance of a switched capacitor

C	f _s	R _{eq}	Note
1 pF	10 MHz	100 k Ω	A poly resistor this size is hundreds of μm^2
1 pF	100 MHz	10 k Ω	
0.5 pF	500 MHz	4 k Ω	At this rate the settling requirement below is the binding constraint

two-phase non-overlapping clocks (두 상 겹침 없는 클럭)

The two switches must **never be on together**: if they are, the source and the destination are shorted and the transferred charge is wrong by an amount that depends on the overlap — which varies with process and temperature. The clocks are therefore generated with a deliberate dead time, typically from a cross-coupled NAND pair whose delay chains set the gap.

Bottom-plate sampling is the refinement that matters: open the switch connected to the *fixed* plate slightly first. That switch sees a constant voltage, so the charge it injects is constant and becomes an offset rather than signal-dependent distortion (T5's charge-injection problem, solved by sequencing instead of by circuitry).

Where it is used	Every switched-capacitor circuit: SC filters, SAR and pipeline ADC stages, sample-and-holds, reference buffers.
When it binds	Always. The dead time is not an optimisation — a design with overlapping phases fails at some corner, and it fails as a slow accuracy drift rather than as an obvious break.
How to apply it	Generate the phases from one clock in a single cell, simulate the overlap at the fast corner (where it is smallest), and route the two phases together so they share delay variation.

What the industry code looks like

```
// Two-phase non-overlapping clock. The feedback is what guarantees
// the gap: neither output can rise until the other has fallen.
module clk_2ph (input clk, output logic p1, p1d, p2, p2d);
    wire n1 = ~(clk & p2);          // cross-coupled NAND pair
    wire n2 = ~((~clk) & p1);
    assign p1 = delay_chain(n1);     // the chain sets the dead time
    assign p2 = delay_chain(n2);
    assign p1d = delay_small(p1);    // 'delayed' phases for BOTTOM-PLATE sampling:
    assign p2d = delay_small(p2);    // the fixed-plate switch opens FIRST
endmodule
// Verify at the FAST corner -- that is where the dead time is smallest and
// where overlap, if it exists at all, appears.
```

This is where it goes wrong. Simulating the phase generator alone proves nothing: the overlap that matters is at the **switches**, after routing, with their real loads. A layout that routes one phase across the chip and the other locally has already broken the guarantee that the schematic provides.

T10.2 What the amplifier must do in half a clock period

Derivation — Two requirements on one op-amp, and why they fight

	This follows	Because
1	In the transfer phase the amplifier must settle the output to within $\frac{1}{2}$ LSB.	Anything less and the charge did not fully move — an error that is <i>gain-like</i> and therefore shows up as INL, not as noise.
2	Settling is exponential, so reaching N-bit accuracy takes $\ln(2^N)$ time constants.	8.32 time constants for 12 bits, 11.1 for 16 — the requirement grows only logarithmically with resolution , which is the good news.
3	The available time is half a clock period, so the closed-loop bandwidth must be at least $\ln(2^N)/(\pi \cdot T_s)$ — and the open-loop unity-gain frequency is that divided by the feedback factor β .	β is the fraction of the output fed back, typically 1/2 to 1/4 in a gain stage; a small β multiplies the requirement.
4	Separately, static accuracy needs loop gain: the residual error is $1/(A\beta)$, so A must exceed $2^{N+1}/\beta$.	78.3 dB for 12 bits at $\beta = 1$, 102 dB for 16 — and this grows exponentially with resolution . This is the bad news, and it is why high-resolution SC design is hard in a process with low intrinsic gain.
5	Bandwidth wants a short channel and lots of current; gain wants a long channel and cascodes, which cost headroom.	The two requirements pull in opposite directions in every process node, and the gap has widened with scaling — which is why calibration and error-cancellation now appear in designs that would once have used a better amplifier.

Table 44. What the amplifier must deliver, per resolution

Bits	Time constants to settle	Minimum DC gain ($\beta = 1$)	Minimum DC gain ($\beta = 1/4$)
10	6.93	66.23 dB	78.27 dB
12	8.32	78.27 dB	90.31 dB
14	9.7	90.31 dB	102.4 dB
16	11.1	102.4 dB	114.4 dB

Compare the two columns' growth. Going from 10 to 16 bits multiplies the settling requirement by 1.6 and the gain requirement by 64. **Speed scales gently with resolution; static accuracy does not.** That asymmetry is the reason the high-resolution end of the market moved to architectures that *measure* the amplifier's shortfall and correct it digitally, rather than architectures that demand a better amplifier.

correlated double sampling (상관 이중 표본화)

Sample the amplifier's error — its offset and its low-frequency (flicker) noise — in one phase, then subtract that sample during the signal phase. Because the error is correlated between the two closely spaced samples, the subtraction removes most of it.

What it costs: the uncorrelated (thermal) noise of the two samples adds in power, so **broadband noise gets worse by $\sqrt{2}$** while offset and flicker get much better. It also folds noise from above the sampling rate into the band (T5), so the amplifier's bandwidth must be limited deliberately.

Where it is used	Precision SC gain stages, image-sensor readout (where it is universal), any circuit whose input is near DC and whose amplifier has a flicker problem (T4).
When it binds	When offset and $1/f$ dominate — which is to say, when the signal band reaches below the flicker corner.
How to apply it	Add one phase and one capacitor. Then check the noise budget again: the $\sqrt{2}$ penalty on thermal noise may cancel the benefit if the design was already thermal-limited.

What the industry code looks like

```
* The measurement that shows whether CDS helped -- run BOTH.
.noise v(out) VIN dec 20 1 100MEG      $ with CDS disabled
* ... then with CDS enabled, and compare the INTEGRATED noise over the signal band.
* Reporting only the 1/f improvement while the broadband floor rose by 3 dB is
* the kind of half-truth that gets found in the customer's lab.
```

This is where it goes wrong. CDS removes what is **correlated between the two samples**. Noise at frequencies comparable to the sampling rate is not correlated and is not removed — it is aliased in. A design that samples the error long before it uses it has all of the cost and little of the benefit.

T10.3 Noise: you pay kT/C more than once

Table 45. kT/C in a two-phase circuit

C	kT/C (one sample)	Two phases ($\sqrt{2}$)	Bits at 1 V FS
0.5 pF	91 μ V	129 μ V	11.1
1 pF	64.4 μ V	91 μ V	11.6
4 pF	32.2 μ V	45.5 μ V	12.6

A switched-capacitor stage samples at least twice per cycle — once on the input capacitor and once on the feedback capacitor — so the noise power adds. The $\sqrt{2}$ is the **minimum**; a pipeline stage with several capacitors and a reference path can easily reach $2\times$. Taking the single-sample kT/C from T4 and sizing to it is a factor-of-two error in noise power, which is half a bit — and half a bit is exactly the kind of shortfall that gets discovered after tape-out.

T10.4 What to do with this on Monday

Table 46. Where this chapter's concepts are actually used

Concept	Block it appears in	What it decides
equivalent resistance	Filter and bias design in a digital process	Whether a resistor is needed at all
two-phase clocks · bottom-plate sampling	Every SC circuit, at schematic time	Whether charge injection is an offset or a distortion
settling time · finite-gain error	Op-amp specification	Bandwidth and DC gain — and whether the process can supply the gain
correlated double sampling · auto-zeroing	Near-DC precision paths	Whether the flicker problem is solved in the circuit or in the architecture
kT/C noise	Capacitor sizing	The factor of $\sqrt{2}$ that is missing from the textbook formula

The numbers to carry: reaching 16 bits needs 11.1 settling time constants — only $1.33\times$ more than 12 bits — but 102.4 dB of DC gain against 78.27 dB, which is $16\times$ more. **Resolution is cheap in time and expensive in gain**, and every modern architecture in this space is an answer to that one sentence.

The next chapter takes the quantity all of these circuits assume they have — a stable bias current that does not move with supply or temperature — and shows what it costs to make one.

Where the inventions are — what is patentable here

The switched-capacitor field is old and its basic circuits are long out of patent, but two families keep producing claims: **error-cancellation schemes** (correlated double sampling, auto-zeroing, chopper-stabilised amplifiers, and every variant that measures an error in one phase and subtracts it in the next) and **amplifier substitutes** — ring amplifiers, dynamic amplifiers, comparator-based switched capacitor circuits — whose whole point is that a modern process cannot build the high-gain op-amp the textbook assumes. Both families answer the same question: *what do you do when the op-amp is not good enough?*

T11. Bias and References — Making a Number That Does Not Move

Everything else in this book assumes a stable current or voltage exists. Making one costs more thought than using one.

Before you read this

voltage · current · resistance · temperature · exponential functions · logarithms · differentiation · threshold voltage · transconductance · output resistance · matching · flicker noise · thermal noise · equivalent resistance

New in this chapter

PTAT · CTAT · bandgap reference · curvature correction · temperature coefficient · start-up circuit · degenerate operating point · PSRR · load regulation · constant- g_m bias · reference current · trimming

A chip has no accurate absolute quantities. Resistors vary $\pm 20\%$, capacitors $\pm 10\%$, threshold voltages by tens of millivolts, and everything moves with temperature. Yet a data converter needs a reference stable to a fraction of an LSB, and an amplifier needs a bias current that does not double over the automotive temperature range. This chapter is about where that stability comes from — and the answer is that it is **constructed by cancelling two opposite temperature dependencies against each other**, which is why it is never free and never perfect.

T11.1 Two things a process gives you for free, both temperature-dependent

CTAT (complementary to absolute temperature) — A quantity that **falls** with temperature. The base-emitter voltage of a bipolar transistor at fixed current is the canonical one: $V_{BE} \approx 0.7\text{ V}$ at room temperature with a slope of about $-2\text{ mV}/^\circ\text{C}$, and it is *not* linear — the curvature it leaves behind is the floor on every plain bandgap.

PTAT (proportional to absolute temperature) — A quantity that **rises** proportionally with absolute temperature. The difference between two V_{BE} s at a current-density ratio N is exactly PTAT: $\Delta V_{BE} = (kT/q) \cdot \ln N$. It is **linear in T by construction**, and it depends only on a ratio and on physical constants — not on process.

Derivation — Why adding them in the right proportion gives $\sim 1.2\text{ V}$

	This follows	Because
1	$V_{BE}(T)$ falls at about $-2\text{ mV}/^\circ\text{C}$; $\Delta V_{BE} = (kT/q) \cdot \ln N$ rises at $+0.086 \cdot \ln N\text{ mV}/^\circ\text{C}$.	For $N = 8$, that is $+0.179\text{ mV}/^\circ\text{C}$ — far too small on its own, which is why it must be amplified.
2	Amplify the PTAT term by $R1/R3$ and add: $V_{ref} = V_{BE} + (R1/R3) \cdot \Delta V_{BE}$.	The gain is a resistor ratio — the one accurate thing a process offers (T10), which is why this topology survived.
3	Choose the ratio so the slopes cancel at the temperature of interest.	One equation, one unknown. The cancellation is exact only at one temperature because V_{BE} is curved and the PTAT term is straight.
4	The value at which the slopes cancel is the silicon bandgap extrapolated to 0 K — about 1.2 V. You do not choose it; it chooses itself.	Measured below: the best temperature coefficient lands at $V_{ref} = 1.1652\text{ V}$. Nobody designed that number — it is where the cancellation happens to occur.

T11.2 The measurement: a sharp optimum, and what sets its floor

Table 47. Measured temperature coefficient against PTAT gain (ngspice, $-40\dots 125\text{ }^\circ\text{C}$, $N = 8$, box method)

R3	V_ref	TC	
6 k Ω	1.5678 V	817.8 ppm/ $^\circ\text{C}$	
8 k Ω	1.3243 V	385.6 ppm/ $^\circ\text{C}$	

R3	V_ref	TC	
9 k Ω	1.2425 V	202.5 ppm/ $^{\circ}$ C	
9.8 k Ω	1.1889 V	68.8 ppm/ $^{\circ}$ C	
10 k Ω	1.1768 V	37 ppm/ $^{\circ}$ C	
10.2 k Ω	1.1652 V	12.7 ppm/ $^{\circ}$ C	optimum
10.3 k Ω	1.1595 V	15.5 ppm/ $^{\circ}$ C	
11 k Ω	1.1228 V	113.7 ppm/ $^{\circ}$ C	
12 k Ω	1.0776 V	251.4 ppm/ $^{\circ}$ C	
14 k Ω	1.0061 V	494.5 ppm/ $^{\circ}$ C	

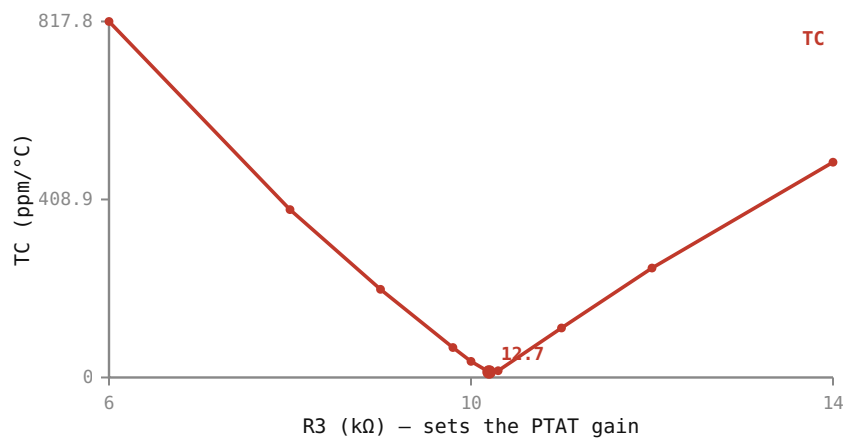


Figure 14. The optimum is **sharp**, unlike the PLL's loop-bandwidth optimum in T7: a 2 % error in the ratio triples the temperature coefficient. That is the difference between an optimum set by a *cancellation* and one set by a *sum of two opposing costs* — and it is why references are trimmed and loop filters are not.

Worked example 7. How accurately must that ratio be set?

Given. The measured optimum is $R3 = 10.2 \text{ k}\Omega$ giving $12.7 \text{ ppm}/^{\circ}\text{C}$. Neighbouring points: $10 \text{ k}\Omega \rightarrow 37 \text{ ppm}/^{\circ}\text{C}$ and $10.3 \text{ k}\Omega \rightarrow 15.5 \text{ ppm}/^{\circ}\text{C}$.

Method. Compute the fractional ratio error at each point and compare with the TC degradation.

Answer. A 2 % error costs $2.91\times$ the temperature coefficient. **Sub-per-cent ratio accuracy is required**, which is a matching problem (T12) and usually a trim.

This is where it goes wrong. Assuming the optimum found in simulation is the optimum in silicon. The cancellation depends on V_{BE} 's actual slope, which is a process parameter with its own spread — so the optimum **moves per lot**. This is precisely why production references are trimmed at test, and why a design that cannot be trimmed must be designed for the *worst-case* optimum offset, not the nominal one.

Table 48. The same knob from the other side: area ratio N at fixed $R3 = 10 \text{ k}\Omega$

N	V_ref	TC
4	0.9773 V	602.5 ppm/ $^{\circ}$ C
8	1.1768 V	37 ppm/ $^{\circ}$ C
16	1.3732 V	484.7 ppm/ $^{\circ}$ C
32	1.5678 V	817.8 ppm/ $^{\circ}$ C

Both tables tune the same physical quantity — the PTAT gain — and both show the same sharp optimum. The practical difference is cost: N enters as $\ln N$, so doubling the area ratio buys only 0.69 of a unit of gain, while the resistor ratio is linear and free. **That is why real designs use a modest N (8 is almost universal) and set the fine adjustment with resistors** — the area would otherwise be enormous for a ratio that a resistor provides for nothing.

curvature (곡률) — the floor the plain topology cannot cross

The measured best is 12.7 ppm/°C, and no choice of ratio does better. The reason is structural: $V_{BE}(T)$ contains a $T \cdot \ln T$ term while the PTAT correction is exactly linear in T , so their difference cannot be zero everywhere. What remains is a bow — the reference is equal at two temperatures and off in between.

Curvature-corrected references add a third term with the right curvature (from a resistor's own temperature coefficient, from a PTAT² current, or from the difference of two V_{BE} s at different current *temperature dependencies*) and reach single-digit ppm/°C. Each of those schemes costs area and a new sensitivity, which is why the plain topology is still shipped wherever tens of ppm are enough.

Where it is used	Any reference feeding a converter of 12 bits or more, and any system specified over the automotive or industrial temperature range.
When it binds	Compare the reference's drift with the converter's LSB over the operating range before choosing. At 12 bits and 1 V full scale an LSB is 244 μ V; a 12.7 ppm/°C reference drifts 2,515 μ V over 165 °C — about 10 LSB , which is why converters are specified at a temperature and references are trimmed.
How to apply it	Measure TC as a box (max–min over the range, not a local slope), report the range it was measured over, and state whether it is trimmed. Those three facts make two references comparable; without them, nothing does.

What the industry code looks like

```
* Box TC, which is what a datasheet means, and the local slope,
* which is what a naive .measure gives. They differ by 2-5x.
.dc TEMP -40 125 5
.measure dc vmax MAX v(out)
.measure dc vmin MIN v(out)
.measure dc vavg AVG v(out)
.measure dc tc_box param='(vmax-vmin)/vavg/165*1e6' $ ppm/degC -- the honest one
* Reporting a slope taken at 27 C as 'the TC' is the standard way to make a
* reference look 3x better than it is.
```

This is where it goes wrong. A temperature coefficient quoted without its range is not a specification. The box method over –40...125 °C and a local slope at 27 °C can differ by a factor of five on the **same** circuit, and the second number is the one that appears in optimistic datasheets.

T11.3 The failure mode that has nothing to do with temperature

start-up (시동 회로) and the degenerate operating point

A bandgap's loop has **two** stable solutions: the intended one, and the one where every current is zero. Zero is perfectly self-consistent — no current means no V_{BE} , which means no current — and nothing in the loop distinguishes it from the useful solution.

A start-up circuit injects a small current when the reference is near zero and then **gets out of the way**. Both halves matter: a start-up that keeps conducting adds its own temperature dependence to the output, and a start-up that is too weak leaves a fraction of parts dead on the slow corner.

Where it is used	Every bandgap, every constant- g_m bias cell, every circuit whose bias is derived from itself.
------------------	--

When it binds	It is not optional and it is not a nominal-corner check: the zero state is reached by a slow supply ramp, by a brown-out, or by any sequence the designer did not think of.
How to apply it	Simulate the supply ramp , not the operating point, at every corner and at several ramp rates including a very slow one. Then force the output to zero with an initial condition and verify that it recovers.

What the industry code looks like

```
* The two simulations that must both pass. The .op alone proves nothing --
* it simply finds A solution, and a degenerate circuit HAS a solution.
.tran 1u 500u                                $ 1) slow supply ramp
Vcc vcc 0 PWL(0 0 400u 1.8)                  $ 400 us ramp -- deliberately slow
.measure tran vref_final FIND v(ref) AT=480u

.ic v(ref)=0                                  $ 2) forced into the dead state
.tran 1u 100u
.measure tran vref_rec FIND v(ref) AT=90u
* Both must land at the design value. A circuit that passes .op and fails these
* is a part that works on the bench and fails in the field at cold start.
```

This is where it goes wrong. The most common start-up bug is not the absence of the circuit but its **strength at the fast corner**: a start-up device sized for the slow corner may still be injecting current at the fast one, shifting the reference by a few millivolts in a way that tracks process. Check that the start-up current is essentially zero once the loop is up — at every corner.

T11.4 A reference has three specifications, not one

Table 49. What a reference must guarantee

Specification	What it is	Fails as	Set by
Temperature coefficient	Drift over the operating range	Gain error that changes with ambient	PTAT/CTAT balance, curvature, trim
PSRR	Rejection of supply noise, at frequency	The converter's noise floor rises when the digital side is busy	The amplifier's supply rejection and the layout
Load regulation	Output shift versus load current	The reference moves when the converter samples — a code-dependent error	Output impedance and the buffer, not the core
Noise	Integrated output noise in band	Directly adds to the converter's noise floor (T4)	Resistor thermal noise and the amplifier's 1/f

What actually happened

Load regulation is the specification that surprises digital integrators. A SAR converter's reference is not a DC load: it draws a charge pulse at every bit trial, and the reference must settle between trials to the full accuracy of the conversion (T10's settling argument, applied to a load that changes every cycle). A reference that meets its DC specification perfectly can still produce a converter with **code-dependent INL**, because the charge demanded depends on the code being tested. **The fix is a local decoupling capacitor sized against the charge per trial** — and that capacitor is often larger than the converter's own array.

constant- g_m bias (일정 g_m 바이어스)

Most analog specifications — bandwidth, noise, gain — depend on g_m , not on current. A bias that holds current constant therefore holds the *wrong* thing: $g_m = \sqrt{(2\mu C_{ox}(W/L)I)}$ still moves with mobility, which moves with temperature and process.

The constant- g_m cell sets $g_m = 1/(R \cdot (1 - 1/\sqrt{K}))$ by forcing the source-degeneration resistor to define it — so g_m now tracks a **resistor** rather than a mobility. It does not remove the dependence; it *moves it* to a component whose variation you can choose by picking a resistor type.

Where it is used	The bias of amplifiers, comparators, output drivers — anything whose bandwidth must be stable across corners.
When it binds	When bandwidth or noise must hold across process and temperature. It is the default for signal-path bias in mixed-signal IP.
How to apply it	Pick the resistor type deliberately (poly resistors and diffusion resistors have very different temperature coefficients), and remember this cell also has a degenerate zero state — it needs its own start-up.

What the industry code looks like

```
* What the bias must actually hold, checked across corners.
* Constant CURRENT is the wrong target; constant gm is the right one.
.dc TEMP -40 125 5
.measure dc gm_min MIN @m.xamp.m1[gm]
.measure dc gm_max MAX @m.xamp.m1[gm]
.measure dc gm_spread param='(gm_max-gm_min)/gm_min*100' $ percent
* A constant-current bias typically gives 30-40% here; constant-gm, under 10%.
```

This is where it goes wrong. Constant- g_m makes g_m track $1/R$, so the circuit's bandwidth now depends on the resistor's temperature coefficient — which for a diffusion resistor can be thousands of ppm/°C. Choosing the resistor is part of the design, not a layout detail.

T11.5 What to do with this on Monday

Table 50. Where this chapter's concepts are actually used

Concept	Block it appears in	What it decides
PTAT · CTAT · bandgap reference	Any converter or precision block	The reference topology and whether it must be trimmed
temperature coefficient · curvature correction	System specification over temperature	Whether tens of ppm are enough or curvature correction is needed
start-up circuit · degenerate operating point	Every self-biased cell	Whether the part comes up at cold start — at every corner
PSRR · load regulation	Integration with the digital side	The decoupling the reference needs, and who pays for it
constant- g_m bias	Signal-path bias	Whether bandwidth holds across corners, and which resistor type

The measured result to carry: the plain bandgap's best temperature coefficient in this simulation is 12.7 ppm/°C at $V_{ref} = 1.1652$ V, and the optimum is **sharp** — a 2 % ratio error costs 2.91× the drift. Everything else in this chapter follows from that sharpness: it is why references are trimmed, why matching (the next chapter) is the limiting discipline, and why the value nobody chose — 1.2 V — is printed on every datasheet in the field.

The next chapter takes the assumption this one has been leaning on — that two identically drawn devices are identical — and replaces it with a distribution.

Where the inventions are — what is patentable here

References are an old field with a live patent stream, and it clusters in three places: **curvature correction** (every scheme that cancels the V_{BE} non-linearity the plain bandgap leaves behind), **low-voltage operation** (current-mode and resistive-divider bandgaps that work below 1.2 V, which the classic topology cannot), and **trim and calibration** (one-point trim schemes that infer the curvature from a single measurement). The first and third are the same idea seen twice: *accept a physical error and remove it somewhere cheaper* — the pattern from T6.

T12. Matching — Two Identical Devices Are Not Identical

The only accurate thing on a chip is a ratio, and even that is a distribution. Everything precise is built on knowing its width.

Before you read this

voltage · current · probability · mean · variance · standard deviation · normal distribution · logarithms · square roots · threshold voltage · transconductance · matching · $V_{\min}$ · INL · DNL · PTAT · temperature coefficient

New in this chapter

Pelgrom's law · random mismatch · systematic mismatch · common centroid · dummy devices · matching area · yield sigma · importance sampling · offset · trim · layout-dependent effects

Two transistors drawn identically, adjacent, in the same orientation, still differ. Dopant atoms are discrete and land at random; the gate edge is not perfectly straight; the oxide thickness varies atom by atom. The resulting parameter difference is a random variable, and precision analog design is the discipline of **knowing its standard deviation and budgeting for its tail**.

T12.1 Pelgrom's law, and why it is a square root

Pelgrom's law (펠그롬 법칙)

For two adjacent identically drawn devices, the standard deviation of a parameter difference falls as the square root of area: $\sigma(\Delta V_{th}) = A_{VT}/\sqrt{(W \cdot L)}$, with A_{VT} in $\text{mV} \cdot \mu\text{m}$ — a per-process constant that the foundry measures and publishes (a few $\text{mV} \cdot \mu\text{m}$, improving slowly with each node).

The square root is not empirical curve-fitting: the mismatch comes from counting statistics on independent microscopic events (dopant placement, edge roughness), and the relative fluctuation of a count of N independent events falls as $1/\sqrt{N}$.

Area is the count.

Where it is used	Every current mirror, differential pair, comparator, capacitor array, and SRAM cell in the design. Also in the yield model — this is the distribution that $V_{\min}$ (T4) is a tail of.
When it binds	Before drawing anything precise: the required σ fixes the minimum area, and the minimum area often fixes the whole block's size and its parasitic capacitance, and therefore its speed.
How to apply it	Work backwards from the specification. If an offset of 5 mV at 3σ is allowed, then $\sigma(\Delta V_{th}) \leq 1.67 \text{ mV}$, so $W \cdot L \geq (A_{VT}/1.67 \text{ mV})^2$. With $A_{VT} = 5 \text{ mV} \cdot \mu\text{m}$ that is $8.96 \mu\text{m}^2$ — per device .

What the industry code looks like

```
* Mismatch in a PDK is a MODEL PARAMETER, not something you add by hand.
* The foundry supplies statistical models; you enable them and run Monte Carlo:
.lib 'models/pdk.lib' mc          $ NOT the 'tt' corner -- the statistical deck
.mc 200 op                        $ 200 samples
.measure op ios param='abs(i(vd2)-i(vd1))'
* AND THEN: the corners and the mismatch are DIFFERENT analyses.
*   corner = global shift (all devices move together)
*   mismatch = local spread (devices move independently)
* Running 'tt + mismatch' and calling it worst case misses the combination that
* actually fails: slow corner AND an unlucky pair.
```

This is where it goes wrong. A_{VT} applies to the **difference between two adjacent devices**. Using it for the absolute value of one device, or for devices on opposite sides of the die, is wrong in both directions: the absolute spread is larger (it includes global process variation) and the device-to-device correlation across a die is a different model entirely.

Table 51. Measured: current-mirror ratio versus device area (ngspice Monte Carlo, 80 samples, $A_{VT} = 5 \text{ mV} \cdot \mu\text{m}$, $I_{\text{ref}} = 10 \mu\text{A}$)

$W \times L$	Area	$\sigma(\Delta V_{th})$	Mean ratio	$\sigma(\text{ratio})$
1x0.2	$0.2 \mu\text{m}^2$	11.18 mV	0.9779	14.2 %
2x0.4	$0.8 \mu\text{m}^2$	5.59 mV	0.9876	7.17 %
4x0.8	$3.2 \mu\text{m}^2$	2.8 mV	0.9934	3.61 %
8x1.6	$12.8 \mu\text{m}^2$	1.4 mV	0.9966	1.81 %

Derivation — Reading Pelgrom straight out of that table

	This follows	Because
1	Area $0.2 \mu\text{m}^2 \rightarrow \sigma(\text{ratio}) = 14.16 \%$; area $0.8 \mu\text{m}^2 \rightarrow 7.17 \%$.	Area $\times 4$, $\sigma \div 1.97$ — the square-root law, measured, not assumed.
2	The next two steps repeat it: $\div 1.99$ and $\div 1.99$.	Three independent confirmations of the same exponent. If a measurement like this does not give $\sqrt{\cdot}$, the model or the harness is wrong — and that is the fastest check available on a mismatch setup.
3	The mean ratio is not 1: it rises from 0.9779 toward 0.9966 as the devices grow.	A symmetric V_{th} error maps to an <i>asymmetric</i> current error — the square-law is convex, so the average of the ratio is pulled below 1. Mismatch biases the mean, not only the spread , and the effect vanishes only as the spread does.

T12.2 The other knob: overdrive

Table 52. Measured: the same device, different bias current ($W/L = 2/0.4$, 80 samples)

I_{ref}	$\sigma(\text{ratio})$	Relative to $10 \mu\text{A}$
$1 \mu\text{A}$	22.1 %	$3.08\times$
$10 \mu\text{A}$	7.17 %	$1\times$
$100 \mu\text{A}$	2.29 %	$0.32\times$

Derivation — Why more current matches better, and what it costs

	This follows	Because
1	A V_{th} error δ produces a current error through the transconductance: $\delta I = g_m \cdot \delta$.	The device does not know why its gate voltage is effectively different.
2	Relative current mismatch is therefore $\sigma(I)/I = (g_m/I_D) \cdot \sigma(\Delta V_{th})$.	g_m/I_D is the whole story (T1). High g_m per amp — the efficient, low-power operating point — is also the <i>worst-matching</i> one.
3	In strong inversion $g_m/I_D = 2/V_{ov}$, so $\sigma(I)/I = 2\sigma(\Delta V_{th})/V_{ov}$.	Measured: 22.1 % at $1 \mu\text{A}$ against 2.29 % at $100 \mu\text{A}$ — $9.65\times$ better for $100\times$ the current, which is the $\sqrt{\cdot}$ again ($V_{ov} \propto \sqrt{I}$).
4	In weak inversion g_m/I_D saturates at $\sim 1/(nV_T) \approx 25\text{--}30 \text{ V}^{-1}$ and matching is at its worst.	This is why a subthreshold current mirror is a bad current mirror , and why ultra-low-power analog pays for its efficiency in area.

Those two tables are the whole design space. Accuracy can be bought with **area** ($\sigma \propto 1/\sqrt{WL}$) or with **overdrive** ($\sigma \propto 1/V_{ov}$), and both have a square-root character, so both are expensive. Everything else in precision analog — chopping, auto-zero, DEM, digital calibration — exists to avoid paying either.

T12.3 The mismatch that is not random

systematic mismatch (계통 불일치) — A difference that is the same on every die: a gradient in oxide thickness across the wafer, a temperature gradient from a nearby power device, stress from the package or from a neighbouring well, or simply different surroundings on the two sides of a pair. **It does not average out over samples**, so no amount of Monte Carlo finds it.

Table 53. Two kinds of mismatch, and what each responds to

	Random	Systematic
Source	Discrete dopants, edge roughness — counting statistics	Gradients, stress, different neighbours, thermal
Scales as	$1/\sqrt{W \cdot L}$	Does not scale with area — it can get worse , since a larger device spans more of the gradient
Found by	Monte Carlo	Layout review, and measurement on silicon
Removed by	Area, overdrive, calibration	Layout: common centroid, dummies, identical orientation and surroundings
Typical size	7.17 % for a $0.8 \mu\text{m}^2$ mirror (measured above)	0.1–1 % from a gradient across a few hundred μm

common-centroid layout (공통 중심 배치) and dummies

Split each device of a matched pair into pieces and interleave them so both devices share the same centre of mass: A B B A, or a checkerboard for a two-dimensional gradient. A **linear** gradient then affects both halves equally and cancels to first order.

Dummy devices at the edges solve a different problem: the outermost real device has a different neighbour than the inner ones, so etch and stress differ. Dummies give every real device identical surroundings, at the cost of area that does nothing electrically.

Where it is used	Every matched pair that matters: differential pairs, current-mirror arrays, capacitor arrays in converters (T6), bandgap BJT arrays (T11).
When it binds	At layout, and it must be specified by the designer — a layout engineer cannot infer which devices must match from a netlist.
How to apply it	Mark matched groups in the schematic, require common-centroid with dummies, keep orientation identical (never mirror a device to save area), route the two halves symmetrically, and keep the pair away from the package's stress maxima — typically the die corners.

What the industry code looks like

```
// The instruction that must reach the layout engineer. A netlist
// does not carry it, so it goes in the constraint file AND the schematic.
// match_group : M1 M2 common_centroid = ABBA, dummies = 2 each side
// orientation : identical (NO mirroring -- STI stress is not symmetric)
// routing : symmetric; equal parasitic on both gates
// keep_out : > 50 um from the die seal ring and from any power device
// Verified by an LVS-with-matching check, not by inspection:
// assert centroid(M1) == centroid(M2) within 0.5 um
```

This is where it goes wrong. Common centroid cancels a **linear** gradient. A quadratic one — which is what a nearby heat source produces — is not cancelled, and interleaving can even make it worse by spreading the device over more of the curvature. For thermal gradients the answer is distance and orientation relative to the isotherms, not interleaving.

What actually happened

Mismatch is also the reason SRAM $V_{\min}$ (T4) is a yield question rather than a design-point question. An array of 10 million cells needs the **worst** cell to work, which is roughly a -5.5σ event. Ordinary Monte Carlo with a thousand samples cannot see that tail at all — it would take on the order of 10^8 samples. Production flows therefore use **importance sampling** or analytic tail methods, and a report that says 'Monte Carlo passed, 1000 samples' for a memory has not addressed the question it was asked.

T12.4 What to do with this on Monday

Table 54. Where this chapter's concepts are actually used

Concept	Block it appears in	What it decides
Pelgrom's law · matching area	Sizing any matched device	The minimum area — which then sets parasitics and speed
transconductance · random mismatch	Choosing the bias point	Whether the efficient operating point is affordable
systematic mismatch · common centroid · dummy devices	Layout constraints, written by the designer	What the layout engineer must be told, because the netlist does not say it
yield sigma · importance sampling	Memory and any large array	How many σ the design must survive, and which method can see that far
trim · digital calibration	When area becomes unaffordable	Whether to pay in silicon or in test time

The measured results to carry: current-mirror mismatch falls as the square root of area — 14.16 % → 7.17 % → 3.61 % → 1.81 % for four doublings of area — and as the square root of current: 22.1 % at 1 μA against 2.29 % at 100 μA . **Both knobs cost a square**, and that single fact is why calibration exists.

The next chapter leaves the devices and takes up the network that feeds them all — the power delivery network — where the same 'it is a distribution' argument reappears as a question about impedance and time.

Where the inventions are — what is patentable here

Matching itself cannot be patented — it is statistics — but every scheme that **buys accuracy without area** can be: dynamic element matching, chopper and auto-zero offset cancellation, digital foreground and background calibration, and layout techniques that cancel gradients. Notice the shape again: since σ falls only as $1/\sqrt{\text{area}}$, buying 4× accuracy with area costs **16×** the silicon — so almost every commercially interesting idea in precision analog is an alternative to paying that.

T13. Power Integrity — the Network Everything Else Sits On

The supply is not a node; it is an impedance with a frequency response, and your IP's worst jitter usually comes from its peak.

Before you read this

voltage · current · resistance · frequency · logarithms · characteristic impedance · RC delay · Elmore delay · phase noise · supply pushing · PSRR · aperture jitter

New in this chapter

target impedance · PDN · ESR · ESL · anti-resonance · damping · decoupling capacitor · di/dt · IR drop · simultaneous switching noise · ripple budget

Every block in this book assumed a supply. This chapter is about what that supply actually is: a network of capacitors and inductances whose impedance varies by two orders of magnitude across frequency, with **peaks at frequencies nobody designed**. The current your IP draws flows through that impedance and becomes a voltage on your own supply — and the failures it causes appear as jitter (T7), reference drift (T11), or timing failures that correlate with software.

T13.1 The specification is an impedance, not a voltage

target impedance (목표 임피던스)

$Z_{\text{target}} = V_{\text{DD}} \cdot (\text{allowed ripple}) / I_{\text{max}}$. For 1 V, 5 % ripple and 10 A of transient current that is **5 mΩ** — and it must hold *at every frequency the load actually contains*, not on average.

This turns a vague requirement ('clean supply') into a curve that can be measured, simulated and signed off, which is the whole reason the concept exists.

Where it is used	Board design, package design, and the on-die grid — and it is the number your IP's integration guide should state as a requirement.
When it binds	Before the package is chosen. The package's inductance sets the mid-frequency impedance and cannot be fixed later by board capacitors.
How to apply it	Take the block's maximum transient current and its spectral content, set the allowed ripple from the most sensitive circuit (usually a PLL or a reference, not the logic), and check the impedance curve against it at the peaks .

What the industry code looks like

```
# What an IP deliverable should say, so the integrator can meet it.
# INTEGRATION.md:
#   Supply: 1.0 V +/- 5%, Z_pdn <= 5 mOhm from DC to 200 MHz at the block's pins.
#   Peak transient: 10 A over 2 ns at the start of a burst (di/dt = 5 A/ns).
#   On-die decoupling included in the macro: 20 nF.
#   The PLL supply (VDDA) requires a separate regulator; sharing it with the
#   digital rail has been measured to add 0.4 ps rms jitter at 1 GHz.
# Without these four lines the integrator guesses, and the guess is usually wrong
# in the direction that makes your IP look bad.
```

This is where it goes wrong. A flat target is a simplification. The real requirement is the load's current spectrum times the impedance, integrated where the victim circuit is sensitive — a 20 mΩ peak at 1.4 MHz matters enormously to a PLL whose loop bandwidth is 1 MHz (T7) and not at all to a logic block.

Table 55. Measured impedance of this chapter's PDN (target 5 mΩ)

Frequency	Z	Meets target?
0.01 MHz	15.5 mΩ	no
0.1 MHz	1.345 mΩ	yes
1 MHz	5.673 mΩ	no
10 MHz	5.232 mΩ	no
100 MHz	2.111 mΩ	yes
1, MHz	23.07 mΩ	no
5, MHz	115.1 mΩ	no

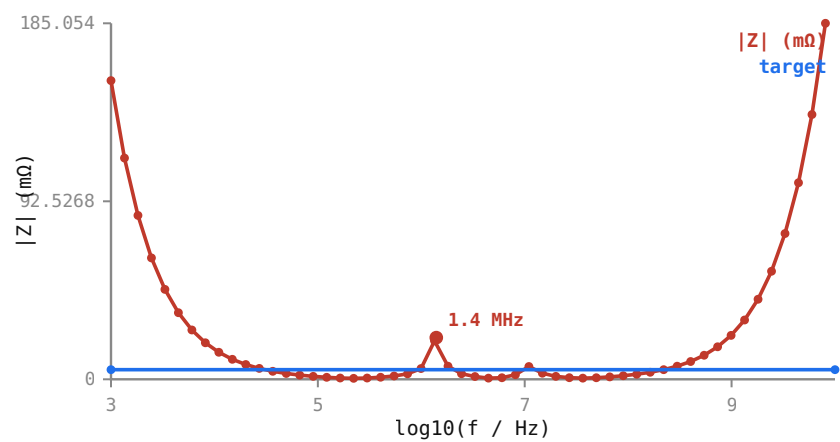


Figure 15. The impedance profile. It meets the target over most of the range and **violates it at the anti-resonance** — 21.66 mΩ at 1.4 MHz, 4.33× the target. The peak, not the average, is the specification.

T13.2 Anti-resonance: why adding a capacitor can make it worse

Derivation — Where the peak comes from

	This follows	Because
1	A real capacitor is series R–L–C: capacitive below its self-resonance, inductive above it .	ESL comes from the package and the mounting; it is why a 100 μF bulk capacitor does nothing at 100 MHz.
2	Put two capacitors in parallel. Between their self-resonances, one is already inductive and the other is still capacitive.	An inductor in parallel with a capacitor is a parallel resonant tank , and a parallel tank has <i>high</i> impedance at resonance.
3	So the combination has a peak where the individual parts each had a dip.	Measured on this PDN: peaks at 1.4 MHz (21.66 mΩ) and 11.1 MHz (6.683 mΩ).
4	The peak's height is set by the damping — the ESR in the loop.	Which produces the result below, and it is the one that surprises people.

Table 56. Increasing ESR to lower the peak (the ceramic branch)

Ceramic ESR	Peak frequency	Peak Z	
5 mΩ	1.4 MHz	26.77 mΩ	
10 mΩ	1.4 MHz	21.66 mΩ	
30 mΩ	1.4 MHz	12.64 mΩ	

Ceramic ESR	Peak frequency	Peak Z	
100 mΩ	1.62 MHz	7.018 mΩ	minimum
300 mΩ	3.01 MHz	12.96 mΩ	

Read that table twice. Going from 5 mΩ to 100 mΩ of ESR — making each capacitor **worse** by the usual figure of merit — lowers the anti-resonance peak from 26.77 mΩ to 7.018 mΩ, a factor of 3.81. Past the optimum it rises again, because the branch stops conducting at all. **'Lowest ESR' is not the specification; the right ESR is.** This is why controlled-ESR capacitors exist and why a well-meaning substitution to a 'better' part can break a board that was working.

T13.3 The three droops, and which one is yours

Table 57. A current step produces three separate droops

Droop	Time scale	Set by	Fixed by
First	sub-ns to a few ns	On-die grid resistance and on-die decap	More on-die decap, a denser grid — inside the die, nothing else helps
Second	tens of ns	Package inductance against package and on-die capacitance	Package choice, more package capacitance, more bumps
Third	μs	Board capacitance and the regulator's loop bandwidth	Bulk capacitance, regulator response

di/dt, and why the derivative is the enemy

The resistive drop $I \cdot R$ is easy to compute and rarely the problem. The inductive drop is $L \cdot di/dt$, and di/dt in a modern digital block is enormous: a clock-gated domain waking up can move amperes in nanoseconds.

Worse, di/dt is **periodic and correlated with the workload**. A processor running a loop that alternates between a heavy and a light instruction produces a current square wave at whatever rate the loop runs — which can land exactly on an anti-resonance. The classic symptom: a machine that fails one specific benchmark and passes everything else.

Where it is used	Any design with clock gating, power gating, or a bursty accelerator — which is every SoC.
When it binds	At architecture time. Staggering the wake-up of a wide datapath is a micro-architectural decision, not a physical one.
How to apply it	Stagger enables over several cycles, ramp clock-gate release, and never release a whole power domain on one edge. On the analysis side, drive the PDN model with the real current profile, not a step.

What the industry code looks like

```
// The cheapest di/dt fix in RTL: stagger the ungating.
// Releasing 64 lanes on one edge is a current step; releasing them over 8 cycles
// is the same charge with 1/8 the di/dt, and usually nobody notices the latency.
always_ff @(posedge clk) begin
    if (wake) stagger <= {stagger[6:0], 1'b1};    // 8-deep shift
    else      stagger <= '0;
end
generate for (genvar i = 0; i < 64; i++)
    assign lane_en[i] = stagger[i/8];
endgenerate
// Measured effect belongs in the integration guide: peak di/dt with and without.
```

This is where it goes wrong. Simulating the PDN with a current **step** gives the worst case and is useful, but it hides resonance: a step contains all frequencies at declining amplitude, while the real failure is a periodic load that concentrates energy at one frequency. Drive the model with the real workload's current spectrum as well.

What actually happened

Supply noise reaches your IP through two doors, and only one of them is the supply. The first is direct: V_{DD} moves, and a VCO's frequency moves with it (T7's pushing). The second is the **substrate**: current returning through the die's bulk modulates body voltages, which modulates thresholds (T1's body effect). The second path does not appear in any PDN impedance plot, and it is why physical separation and guard rings matter even when the supply is separately regulated.

T13.4 What to do with this on Monday

Table 58. Where this chapter's concepts are actually used

Concept	Block it appears in	What it decides
target impedance	The integration guide you ship	The requirement the integrator must meet — state it or inherit their guess
anti-resonance · damping	Board and package capacitor selection	Which capacitors, and that 'lowest ESR' is the wrong criterion
di/dt	RTL and micro-architecture	Whether wake-up is staggered — a free fix at design time, expensive later
decoupling capacitor	Floorplan	How much on-die capacitance fits, and the leakage it costs
simultaneous switching noise	I/O design	How many simultaneous outputs are allowed to switch

The computed results to carry: this PDN meets its 5 mΩ target at most frequencies and misses it by 4.33× at a 1.4 MHz anti-resonance that no component has by itself. And raising the ceramic branch's ESR from 5 mΩ to 100 mΩ **lowers** that peak by 3.81× — the one place in this book where making a component worse makes the system better.

The next chapter takes the longest time scale of all: what happens to these circuits after they have been running for ten years.

Where the inventions are — what is patentable here

Power delivery is a board and package discipline, but the IP-side claims are real and they all have the same shape: **make the block insensitive instead of making the supply clean**. Regulators local to a sensitive block, differential and supply-referenced signalling, bias schemes whose output does not track V_{DD} , and clock distribution that is immune to supply modulation. Every one of them is cheaper than lowering a PDN's impedance by the same factor.

T14. Reliability — What the Circuit Looks Like After Ten Years

Every parameter in this book drifts. A design signed off at time zero has been signed off for the wrong day.

Before you read this

voltage · current · temperature · exponential functions · logarithms · probability · normal distribution · threshold voltage · transconductance · slack · settling time · Pelgrom's law · offset · $V_{\min}$ · IR drop

New in this chapter

BTI · HCI · TDDB · electromigration · Arrhenius · acceleration factor · activation energy · Black's equation · Weibull · burn-in · ageing-aware design · lifetime target

A transistor that meets its specification on the day it is made will not meet it in ten years. Thresholds shift, oxides break down, and metal atoms move. None of this is a defect: it is the ordinary ageing of the materials, it is characterised and modelled by the foundry, and it is **your job to budget for it** — because a timing signoff at time zero is a signoff for a chip that has not yet been shipped.

T14.1 The four mechanisms, and what each one eats

Table 59. The mechanisms you must budget for

Mechanism	What physically happens	What it degrades	Worst when
BTI (bias temperature instability)	Charge trapping at the oxide interface under gate bias	$ V_{th} $ rises → speed falls, offsets drift	High temperature, high V_{GS} , DC bias (no recovery)
HCI (hot carrier injection)	Energetic carriers damage the drain-end oxide	Drive current falls, mismatch grows	High V_{DS} <i>while switching</i> — fast edges, high activity
TDDB (time-dependent dielectric breakdown)	The gate oxide eventually conducts, permanently	The device — it is a hard failure , not a drift	High field, thin oxide, large total gate area
Electromigration	Momentum transfer from electrons moves metal atoms	Wires open or short — also a hard failure	High current density, DC (unidirectional) current, high temperature

Two of these are **drifts** and two are **failures**, and they need different arithmetic. A drift is budgeted as a margin: you sign off timing with a degraded library and accept the area cost. A failure is budgeted as a *probability*: you compute the fraction of parts that fail before the lifetime target and compare it against the allowed FIT rate. Mixing the two — for instance treating TDDB as 'a bit more V_{th} drift' — produces a design that meets its timing and fails its qualification.

T14.2 Acceleration: how a 1000-hour test predicts ten years

Arrhenius acceleration (아레니우스 가속)

Wear-out rates follow $\exp(-E_a/kT)$, so raising the temperature multiplies the rate by a computable factor: $AF = \exp[(E_a/k)(1/T_{\text{use}} - 1/T_{\text{stress}})]$.

With $E_a = 0.9$ eV, stressing at 125 °C instead of 85 °C is **18.7× faster** — which is how a 1000-hour oven test stands in for years of field use. The same arithmetic run backwards is the familiar rule of thumb: every 10 °C of cooling roughly 2.08× the life.

Where it is used	Qualification planning, and the argument with the thermal engineer about junction temperature.
When it binds	Whenever a lifetime is claimed. A lifetime without a temperature is not a specification.

How to apply it

Pick E_a from the foundry's model for the mechanism in question — **not a generic 0.7 eV** — and state it alongside the result, because the answer depends on it strongly.

What the industry code looks like

```
# The same 1000-hour test, read with three different activation energies.
# This is why Ea must be quoted with any lifetime claim.
Ea = {0.7: 9.8, 0.9: 18.7, 1.1: 35.9}    # AF from 85 C to 125 C
# 1000 h at 125 C therefore stands for 9800 h / 18700 h / 35900 h at 85 C
#   -> 1.1 years / 2.1 years / 4.1 years.
# The SAME DATA supports "passes 1 year" or "passes 4 years" depending on a
# number the report often does not state.
```

This is where it goes wrong. Activation energy is per **mechanism**, and using one number for a mixed failure population is the standard way to get a lifetime wrong by 3.68×. If a qualification report does not say which mechanism and which E_a , it has not made a claim you can check.

Table 60. Acceleration factor from 85 °C use to 125 °C stress

E_a	AF (125 °C vs 85 °C)	10 °C cooling buys	1000 h stress stands for
0.7 eV	9.76×	1.77×	1.11 yr
0.9 eV	18.7×	2.08×	2.14 yr
1.1 eV	35.9×	2.44×	4.1 yr

Worked example 8. A 1000-hour BTI stress: what does it say about ten years?

Given. BTI's threshold shift grows as a power law in time, $\Delta V_{th} \propto t^n$, with n typically 0.15–0.25. A part is stressed for 1000 hours and shows some shift.

Method. Ten years is 87,600 hours, or 87.6× the stress time. The shift grows by that ratio raised to n .

Answer. $n = 0.15 \rightarrow 1.96\times$; $n = 0.2 \rightarrow 2.45\times$; $n = 0.25 \rightarrow 3.06\times$. **The same measurement predicts a shift that differs by 1.56× depending on an exponent nobody measured in this test.**

Table 61. Voltage acceleration, for the same reason

γ	100 mV overvoltage ages it
8 /V	2.23×
12 /V	3.32×
16 /V	4.95×

This is where it goes wrong. Extrapolating a power law from one time point. With a single stress duration, n is an assumption, not a measurement — and the answer is roughly linear in it. A credible BTI characterisation stresses at **several durations** and fits n ; anything else is a number with an unstated multiplier.

Voltage acceleration is the reason a design must never be characterised only at nominal. At $\gamma = 12$ /V — a typical value — running 100 mV above nominal ages the device 3.32× faster, so a rail that overshoots during a DVFS transition can consume years of budget in seconds. **The reliability question about DVFS is not the average voltage; it is the transients.**

T14.3 Electromigration, and the rule that surprises digital designers

Black's equation (블랙 식) and why AC is easier than DC

$MTF = A \cdot J^{-n} \cdot \exp(E_a/kT)$, with $n \approx 2$ for the usual regime. Lifetime falls as the **square** of current density and rises exponentially as temperature falls: at $E_a = 0.9$ eV, cooling a wire from 110 °C to 85 °C buys 6.7× its life.

The part that surprises digital designers: **bidirectional (AC) current is far less damaging than unidirectional (DC)**, because atoms pushed one way are partly pushed back. EM rules therefore give a much higher limit for signal nets than for power nets — and the nets that fail are almost always power, clock drivers, or any signal with a duty cycle far from 50 %.

Where it is used	Power grid sizing, clock-tree driver outputs, and the output stage of any driver with a static current.
When it binds	Checked automatically by the EM/IR tools at signoff — but designed for long before, since the grid's width is a floorplan decision.
How to apply it	Keep current density below the foundry's limit at the maximum junction temperature, and remember the limit depends on the duty cycle and direction. Vias are usually the binding constraint, not wires: one via carries far less than the wire it connects.

What the industry code looks like

```
# What the EM report actually flags, in order of frequency:
# 1. VIAS on power straps -- a single via where four were needed
# 2. Clock driver outputs -- high activity, high capacitance, one direction of
#    net current per edge
# 3. Long thin nets from a pad to a block at the far side of the die
# The fix for (1) and (2) is always the same: more parallel vias, wider metal,
# and it is FREE if done at floorplan time and expensive after routing.
```

This is where it goes wrong. EM limits are specified at a temperature. A design checked at 85 °C and run at 110 °C has 6.7× less margin than the report says — and the junction temperature of a block just woken from sleep is not the average temperature anybody simulated.

T14.4 Designing for the tenth year, not the first

Table 62. How each mechanism enters the design flow

Mechanism	Enters as	Who owns it
BTI / HCI	A degraded timing library — signoff runs twice, at time zero and at end of life	Timing signoff; the margin is the design's
TDDDB	A limit on gate area at a given field, and a FIT budget	Foundry rules; the designer chooses the oxide type
Electromigration	Current-density checks on every net	Physical design, but the grid was decided at floorplan
Ageing mismatch	$\sigma(\Delta V_{th})$ grows over life, so offsets and V_{min} get worse	Analog designer — and it is often forgotten

What actually happened

The last row is the one that catches analog designs. BTI is not only a mean shift; it has a **spread**, and that spread adds to the mismatch of T12. A comparator whose offset was trimmed at test drifts back, because the two halves age differently — and they age differently precisely because they carry different duty cycles. **A trimmed offset is trimmed for the day it was trimmed.** Designs that must hold an offset for ten years either re-trim in the field (auto-zero, T10) or are built so that both halves age identically by construction.

ageing-aware design (노화 인지 설계)

Three things make a design survive its tenth year, and none of them is margin alone:

- **Balance the stress.** If a differential pair's two sides see different duty cycles, they will diverge. Swapping roles periodically (chopping, T10) makes them age together.
- **Prefer ratios of aged devices.** A reference built from two devices that age identically drifts far less than one built from an absolute.
- **Allow recovery.** BTI recovers partially when the bias is removed; a circuit that is DC-biased for ten years gets the worst case, and one that is duty-cycled does not.

Where it is used	Any IP with a ten-year or automotive lifetime target, and any analog block whose accuracy is trimmed.
When it binds	At topology selection. None of these can be retrofitted.
How to apply it	Run the ageing simulation the PDK provides (a degraded model deck), with the real duty cycles — not with everything at DC — and check the offset and V_{min} at end of life, not just the timing.

What the industry code looks like

```
* Ageing simulation: the same netlist, twice, with the PDK's aged models.
.lib 'models/pdk.lib' tt
.option reliability_analysis           $ vendor-specific; every PDK has one
.param age_years = 10
.param duty_cycle = 0.5               $ THIS is the parameter people get wrong --
                                      $ a DC-biased device is duty_cycle = 1.0

.op
.measure op vth_shift param='v(m1.vth_aged) - v(m1.vth_fresh)'
```

* Sign off BOTH decks. A design that only passes fresh is a design that only
* passes on the day it is made.

This is where it goes wrong. Ageing models are statistical and conservative, and running them at the worst corner *and* worst duty cycle *and* worst temperature simultaneously produces a design that cannot be built. The skill is choosing which combinations are physically possible — a DC-biased device at maximum temperature is, but maximum activity at maximum temperature and minimum voltage usually is not.

T14.5 What to do with this on Monday

Table 63. Where this chapter's concepts are actually used

Concept	Block it appears in	What it decides
BTI · HCI	Timing signoff, run twice	The end-of-life margin, and therefore the area
Arrhenius · acceleration factor	Qualification planning and datasheet claims	What a 1000-hour test is allowed to say
Black's equation · electromigration	Power grid and driver sizing at floor-plan	Wire widths and via counts — free early, expensive late
ageing-aware design	Topology selection for anything trimmed	Whether the trim survives to year ten
lifetime target	The datasheet	The lifetime, with its temperature and duty cycle — or it says nothing

The numbers to carry: at $E_a = 0.9$ eV a 125 °C stress runs 18.7× faster than 85 °C use, so 1000 hours stands for 2.14 yr — and with a different E_a the same data says 1.11 yr or 4.1 yr. A lifetime claim without its E_a , its temperature and its duty cycle is not a claim. And 100 mV of overvoltage ages a device 3.32× faster, which is why the transients of a DVFS scheme matter more than its average.

The next chapter is where all of this becomes geometry: the layout, where a schematic that is correct in every respect can still fail.

Where the inventions are — what is patentable here

Reliability physics belongs to the foundry; the IP-side claims are about **surviving it**: circuits that sense their own degradation and compensate (adaptive body bias, adaptive voltage scaling driven by on-die ageing monitors), topologies whose critical parameter is a ratio of two equally aged devices rather than an absolute, and duty-cycling schemes that let devices recover. The last one is worth noticing — **BTI partially recovers when the stress is removed**, which turns a physics constant into a design variable.

T15. Layout — Where a Correct Schematic Can Still Fail

The schematic says what is connected. The layout decides what it does.

Before you read this

resistance · voltage · current · frequency · RC delay · Elmore delay · wire delay · repeater · matching · systematic mismatch · common centroid · dummy devices · threshold voltage · electromigration · IR drop · target impedance · layout-dependent effects

New in this chapter

parasitics · well proximity effect · STI stress · density rules · antenna rules · extraction · LVS · DRC · floorplan · pin placement · abstract view · GDS · hard macro · soft macro

Everything this book has computed assumed devices connected by ideal wires. Layout removes that assumption in four ways at once: wires have resistance and capacitance (T2), devices change their behaviour depending on what is drawn next to them, the supply grid becomes the impedance of T13, and the geometry itself must satisfy hundreds of manufacturing rules that have nothing to do with the circuit. A schematic that is correct in every respect can still fail all four.

T15.1 The device is not what the schematic says

LDE, layout-dependent effects (레이아웃 의존 효과) — Systematic changes in a device's parameters caused by its surroundings rather than by its own dimensions. The same W/L, drawn differently, gives a **different threshold, different mobility and different matching** — and the difference is repeatable, so it is not mismatch (T12); it is a shift you can predict and must extract.

Table 64. The three that matter most, and what each one does

Effect	Cause	Typical size	How to avoid it
Well proximity (WPE)	Ions scatter off the well mask edge during implant, raising V_{th} near it	Tens of mV within $\sim 1\ \mu\text{m}$ of the well edge	Keep matched devices well away from the well boundary — and equally far
STI stress (LOD)	The shallow-trench oxide compresses the channel; mobility changes with the distance from gate to trench	Several per cent of drive current	Identical diffusion extensions on every device of a matched set — which is what dummy devices give you
Poly spacing / OD spacing	Lithography and etch see a different environment at the edge of an array	A few per cent	Dummy gates at the ends of every row

All three have the same practical consequence and it is worth stating plainly: **a device at the edge of an array is not the same device as one in the middle**. That is why dummies exist, why a matched pair must have identical surroundings rather than merely identical dimensions, and why 'the layout engineer can figure it out' is not a design methodology — the constraint is electrical and only the designer knows which devices carry it.

extraction (추출) — the only netlist that means anything

Post-layout extraction produces a netlist with every parasitic resistance and capacitance the layout created. It is the netlist that corresponds to silicon; the schematic netlist is a statement of intent.

The gap between them is not small. In an analog block the extracted capacitance on a high-impedance node routinely doubles the load the designer assumed, and the routing resistance in a current mirror's source connection can add a systematic offset larger than the mismatch budget of T12. **A design is not verified until it has been simulated post-extraction**, at every corner that mattered pre-layout.

Where it is used	Every block, before tape-out. For analog it is the main verification step; for digital it is inside the STA flow (T2's SPEF).
-------------------------	---

When it binds	As soon as a first layout exists — not at the end. An extraction that reveals a 2× capacitance on a critical node is a re-design, and it is cheaper to learn that early.
How to apply it	Extract with coupling capacitance included (not just ground capacitance), and re-run the corner set. Compare the pre- and post- numbers explicitly — the ratio is the thing to look at, and a surprise in it usually points at one net.

What the industry code looks like

```
# The comparison that belongs in every block's review, as a table:
#   node           C_schematic  C_extracted  ratio
#   vout_amp       42 fF        96 fF        2.3x  <- investigate THIS
#   vbias          15 fF        31 fF        2.1x
#   vin            80 fF        94 fF        1.2x
# A 2x on a high-impedance node is a bandwidth halving. Finding it here costs a
# day; finding it in silicon costs a spin.
```

This is where it goes wrong. Extraction tools have **modes** — ground-only, coupled, with or "without inductance — and a block extracted in the cheap mode can pass everything and fail in silicon through crosstalk (T9's lesson, one level down). Record which mode was used alongside the result.

T15.2 Rules that are not about your circuit

Table 65. Manufacturing rules a designer must plan for, not merely obey

Rule	Why it exists	What it costs you if ignored
Density (metal and OD fill)	CMP polishes an uneven surface unevenly	Fill is inserted automatically — into your matched analog region , changing the capacitance you carefully matched. Reserve fill-exclusion regions deliberately
Antenna	A long metal stub connected to a gate collects charge during plasma etch and punches the oxide	The tool inserts a diode or a metal jog — usually fine, but on a high-impedance analog node the diode's leakage is not
Minimum area / enclosure / spacing	Lithography and etch limits	DRC failures, which are mechanical to fix — the cost is schedule, not function
Latch-up rules (guard rings, well ties)	The parasitic thyristor formed by every CMOS structure	A chip that destroys itself when an I/O is driven beyond the rail — and it passes every functional test first

What actually happened

Density fill is the one that surprises analog designers. The tool adds floating metal shapes to meet a density rule, and those shapes sit next to the matched capacitors you laid out by hand — **adding capacitance that is neither matched nor extracted in early runs**. The professional habit is to place your own fill inside the analog region, in a pattern you control and that the extractor sees, rather than letting the tool do it after you stopped looking.

T15.3 What an IP actually hands over

Table 66. Two ways to deliver a block, and what each commits you to

	Soft macro (RTL)	Hard macro (GDS)
What ships	RTL, constraints, testbench, documentation	GDS, LEF abstract, Liberty timing, and the same documentation

	Soft macro (RTL)	Hard macro (GDS)
Process	Any — the integrator synthesises it	One. A new node is a new implementation
Integrator's freedom	Full: they choose the library and the floorplan	None inside the macro; they place a rectangle
What you must guarantee	That it synthesises and meets timing at a stated frequency, in their library	Timing, power, EM, antenna, density and pin access — everything
Where the risk sits	With the integrator, who can get it wrong	With you, and it is fixed at tape-out
Typical for	Controllers, protocol logic, anything digital	PHYs, SerDes, PLLs, memories, anything with analog

abstract / LEF (추상화 뷰) and the pin-access problem

An integrator does not place your GDS; the tool places your **abstract** — a simplified view giving the outline, the pin shapes and the blockages. If the abstract says a pin is on metal 3 in the middle of the macro's edge, the router must be able to reach it with the layers it is allowed to use.

Pin access is the most common integration failure for hard macros, and it is entirely avoidable: pins on a coarse grid, on the layers the integrator's flow routes with, with enough space around them, and never all crowded on one side. It costs nothing at design time and it is unfixable afterwards.

Where it is used	Every hard macro, and it is decided when the floorplan is decided.
When it binds	Before the internal layout is finished — the pin plan constrains the internal routing, not the other way round.
How to apply it	Ask the integrator what their routing layers and track pitch are, put pins on that pitch, and hand them a test placement that routes. Then ship the LEF and a small example integration that demonstrates it.

What the industry code looks like

```
# The deliverable list for a hard macro. Missing any one of these turns
# into a support call, which is the thing a one-person house cannot afford (Z12).
# design.gds          the layout
# design.lef          abstract: outline, pins, blockages    <- pin access lives here
# design.lib          timing/power at each corner           <- one per corner
# design.cdl          netlist for LVS
# design_antenna.rpt  antenna and density status of the macro
# design.sdc          the constraints the integrator must apply
# INTEGRATION.md      supply, decoupling (T13), keep-out, temperature, lifetime
# example/            a placement that routes, so they can diff against it
```

This is where it goes wrong. A macro that meets timing standalone can fail in the integrator's context because its clock arrives through **their** tree: the .lib must state the input clock's assumed transition, and the SDC must state the insertion delay you assumed. Shipping an ideal-clock .lib is the same error as quoting ideal-clock $F_{\max}$ in T2.

T15.4 What to do with this on Monday

Table 67. Where this chapter's concepts are actually used

Concept	Block it appears in	What it decides
layout-dependent effects · well proximity effect · STI stress	Any matched or precision device	Keep-out distances and dummy patterns, written as constraints

Concept	Block it appears in	What it decides
extraction	Before declaring a block finished	Whether the simulated design is the one you drew
density rules · antenna rules	Analog regions, early	Whether the tool edits your matched structures after you stop looking
abstract view · pin placement	Hard-macro floorplan	Whether the integrator can route to you at all
hard macro · soft macro	The business decision	Which process risk you own, and what you can charge for

Layout is where an IP business decides what it is selling. A soft macro moves the physical risk to the customer and can be sold into any process; a hard macro keeps the risk and commands a higher price because the customer is buying **certainty**. The engineering in this chapter is what makes that certainty deliverable — and the documentation list above is what makes it saleable.

The next chapter is about the failure that happens before the chip is ever powered: a few hundred volts arriving on a pin from a human finger. After it, the volume turns from the device to the flow that turns a netlist into geometry.

Where the inventions are — what is patentable here

Layout technique is mostly trade practice rather than patent, but two adjacent areas are not: **automated layout generation** for analog (generators that emit a matched, DRC-clean array from a schematic and a set of constraints) and **layout-aware synthesis**, where the placement feeds back into sizing. Both attack the same weakness — that the constraints which matter most (match these, keep this away from that, this net carries DC) live in the designer's head and are not in the netlist.

T16. ESD — the Failure That Happens Before the Chip Is Powered

Every pin must survive a few amps from a finger, and the circuit that survives it is in the signal path of the circuit that must not be slowed down by it.

Before you read this

voltage · current · charge · resistance · frequency · integration · RC delay · gate capacitance · threshold voltage · reflection coefficient · parasitics · layout-dependent effects · TDDB · electromigration · characteristic impedance

New in this chapter

electrostatic discharge · human body model · machine model · charged device model · clamp · snapback · design window · holding voltage · trigger voltage · current path · pad capacitance · second breakdown · latch-up

A person walking across a carpet charges to a few kilovolts. When they touch a pin, that charge flows into the chip in a hundred nanoseconds, and every internal gate oxide is a few nanometres thick. Protection is therefore not optional and it is not a detail: **it is a circuit in series with your signal**, and its capacitance is subtracted directly from your bandwidth.

T16.1 Three models, three entirely different events

HBM, MM, CDM (인체·기계·충전소자 모델)

HBM models a charged person: 1 pF charged to 2 kV, discharged through 1.5 kΩ. That gives a peak current of 1.33 A with a time constant of 15 ns and an energy of 2 μJ — **computed, and worth doing once by hand** because the numbers are less dramatic than people expect.

CDM models the *chip itself* charged and then discharged through one pin. It is far faster (rise time well under a nanosecond) and the peak current is several amps, but the total energy is small. Published CDM waveforms for 500 V give peaks of roughly 5–10 A — a figure from the standards and practice, **not computed here**, because the peak is set by the package's inductance rather than by a resistor.

MM (machine model) sits between them and is largely retired.

Where it is used	Qualification of every pin of every product. The targets are usually 2 kV HBM and 500 V CDM, and customers sometimes ask for more.
When it binds	At the very start of I/O design — the protection's capacitance is a term in the bandwidth budget and cannot be added later.
How to apply it	Design for HBM at the pad with a clamp and diodes, and for CDM with secondary protection close to the gate it protects, plus a small series resistance. The two need different structures because they have different time scales.

What the industry code looks like

- * TLP (transmission-line pulse) is how protection is characterised:
- * rectangular 100 ns pulses of increasing current, measuring V at each step.
- * The plot that comes out IS the design window check.
- * I_{t2} : the current at which the structure fails (second breakdown)
- * V_h : holding voltage after snapback -- must be ABOVE the supply
- * V_{t1} : trigger voltage -- must be BELOW the oxide limit
- * A 2 kV HBM target needs I_{t2} ≥ 1.33 A with margin -- the number computed above.

This is where it goes wrong. HBM and CDM fail **different** structures. A design that passes 4 kV HBM can fail 250 V CDM, because CDM's current appears inside the die, rises in picoseconds, and finds the nearest thin oxide before the pad clamp has turned on. Reporting only HBM is reporting the easy half.

Table 68. The three events, side by side

	HBM	MM	CDM
What is charged	A person	A tool or fixture	The chip itself
Typical target	2 kV	200 V (largely retired)	500 V
Peak current	1.33 A (computed)	several A	5–10 A (from practice)
Rise time	~10 ns	~ns	< 250 ps
Duration	~15 ns	~ns	~1 ns
Energy	2 μ J	~ μ J	~1 μ J
Fails	Pad structures, clamps	Similar to HBM	Internal thin oxides — far from the pad

T16.2 The design window, and why it is a window

Derivation — Four inequalities that must hold at once

	This follows	Because
1	The clamp must not conduct in normal operation: $V_{\text{trigger}} > V_{\text{DD,max}}$ including overshoot.	A clamp that triggers on a supply transient is a short circuit across the power rail — the chip destroys itself while working.
2	It must turn on before anything else breaks: $V_{\text{trigger}} < V_{\text{oxide breakdown}}$	The whole purpose. This and the previous line define the window, and in an advanced node with a thin oxide and a supply close to it, the window is only a volt or two wide .
3	After snapback it must hold above the supply: $V_{\text{hold}} > V_{\text{DD}}$	Otherwise, once triggered by an event, the clamp stays on when normal power returns — latch-up , and the part is destroyed on the bench after the ESD tester has already passed it.
4	It must carry the current without failing: $I_{t2} >$ the target's peak, i.e. above 1.33 A for 2 kV HBM, with margin.	Second breakdown is thermal and it is permanent.

The second and third inequalities move in opposite directions as the process scales: the oxide gets thinner (lowering the ceiling) while the supply falls more slowly (raising the floor). **The design window narrows with every node**, which is why ESD design has become a specialisation and why the protection for a new node is rarely a straight port of the old one.

the current path (전류 경로) — the part people forget

Protection is not a component you attach to a pin; it is a **path between every pair of pins**. A pulse applied to an input with respect to some other I/O must find a low-impedance route: typically up through a diode to the local supply, along the supply bus, through a rail clamp to ground, and back through another diode.

Three things break that path and all three are layout, not schematic: **bus resistance** (the supply bus between the pin and the clamp — at several amps, a few hundred milliohms is volts), **missing back-to-back diodes** between separate supply domains, and **a clamp that is too far away**. The schematic shows the protection; only the extracted layout shows the path.

Where it is used	Every I/O, and especially every pin that sits in its own supply domain — a PHY's analog supply, a separately regulated PLL rail (T13).
When it binds	At floorplan, because the answer is 'a clamp near every pad' and that costs area at the edge of the die.
How to apply it	Enumerate the pin pairs (I/O to I/O, I/O to any supply, supply to supply) and make sure each has a designed path. Then verify it on the extracted layout with an ESD-path checking tool — the voltage across the whole path at the target current is the number that matters, not the clamp's own voltage.

What the industry code looks like

```
# What an ESD path check reports, and how to read it.
#   pad_rx -> VSS   : clamp V 4.1 V + bus IR 1.3 V = 5.4 V @ 1.33 A   <- FAIL (oxide 5.0)
#   pad_rx -> VDDA  : diode 1.1 V + bus 0.4 V      = 1.5 V           <- ok
#   VDDA   -> VDD   : NO PATH                                     <- FAIL
# Two failures, and NEITHER is the clamp's fault:
#   the first is a BUS WIDTH problem; the second is a MISSING back-to-back
#   diode pair between two supply domains. Both are floorplan decisions.
```

This is where it goes wrong. Separate supply domains are the classic hole. Two rails that are never connected on the die still get connected on the tester, and if no designed path exists between them the current finds one through whatever signal crosses the boundary — usually a level shifter's thin-oxide gate.

T16.3 What it costs the signal

Table 69. Pad capacitance against bandwidth ($f_{-3dB} = 1/(2\pi RC)$; 25 Ω is a doubly-terminated 50 Ω line)

Pad + protection C	f_{-3dB} at 25 Ω	f_{-3dB} at 50 Ω	Usable for
100 fF	63.66 GHz	31.83 GHz	112G PAM4 class
200 fF	31.83 GHz	15.92 GHz	25–56 G
300 fF	21.22 GHz	10.61 GHz	25–56 G
500 fF	12.73 GHz	6.366 GHz	10 G and below
1, fF	6.366 GHz	3.183 GHz	low speed / control

Derivation — Why this table is the real ESD constraint for a SerDes

	This follows	Because
1	A diode big enough to carry amps has a junction area, and a junction has capacitance.	Protection capacitance and I_{t2} are roughly proportional — you cannot have one without the other, which is the whole problem.
2	That capacitance sits at the pad, in parallel with the line.	At 300 fF and 25 Ω the pole is 21.22 GHz — below Nyquist for a 56 GBd link.
3	So protection directly removes channel margin — the same margin T8's equalisers were fighting for.	ESD and link budget are the same negotiation , and the ESD engineer is usually not in the room when the data rate is promised.
4	The escapes are architectural: distribute the protection along an inductive network (T-coil) so the capacitance is absorbed into a transmission line, or use smaller primary protection plus fast secondary protection inside.	Both are why this is a patent-dense area. Neither is free — a T-coil costs area and its own parasitics.

What actually happened

ESD is where a one-person IP house should be explicit about its limits. Designing and **qualifying** protection needs TLP measurement equipment and silicon to characterise, neither of which exists in a solo digital flow. The honest deliverable is therefore a soft macro that stops at the pad, an integration guide that states the assumed protection and its capacitance budget, and the sentence that this repository's rules demand: **what was not verified, stated as not verified**. A datasheet that implies ESD qualification that never happened is the kind of claim that ends a small vendor.

T16.4 What to do with this on Monday

Table 70. Where this chapter's concepts are actually used

Concept	Block it appears in	What it decides
human body model · charged device model	The qualification plan and the data-sheet	The targets, and that CDM is reported as well as HBM
design window · holding voltage · trigger voltage	Clamp selection for the node	Whether the protection can exist at this supply and oxide at all
current path	Floorplan and supply-domain planning	Clamp placement, bus width, and the diodes between domains
pad capacitance	The link budget, with the SerDes architect present	Data rate — the same margin the equalisers are fighting for
latch-up	Guard rings and well ties everywhere	Whether the part survives its own I/O being driven out of range

The computed numbers to carry: a 2 kV HBM event is 1.33 A for about 15 ns, carrying 2 μJ — small enough to be survivable and large enough to destroy a thin oxide. And protection of 300 fF at a pad puts a pole at 21.22 GHz, which is the sentence that connects this chapter to T8: **ESD is not a separate discipline from the link budget; it is a line in it.**

That closes the device half of this volume. It began with one device in four operating regions and ends at the pad, having passed through delay, metastability, noise, sampling, conversion, clocking, links, channels, switched capacitors, references, matching, power, ageing and geometry. The common thread was a single discipline: **every number was computed or measured, the trivial explanations were killed before the number was reported, and what could not be established was said to be unestablished.**

What follows is the other half: the flow that turns a verified netlist into manufactured, tested silicon. It is a different kind of engineering — the quantities are runtimes, track counts, corner lists, test seconds and defective parts per million rather than volts and picoseconds — but the discipline is the same one, and the six chapters apply it to the six stages: the flow and what a loop costs (T17), the floorplan and its power grid (T18), placement, clocking and routing (T19), signoff (T20), the verification environment that had to come first (T21), and the tester that every shipped part passes through (T22).

Where the inventions are — what is patentable here

ESD is one of the most actively patented areas in I/O design, and the reason is the constraint itself: the protection sits **in the signal path**, so every claim is a way to get more protection per femtofarad. Distributed protection along a T-coil or an inductor-peaked network; active clamps that conduct only during a real event; protection that reuses the driver's own devices; and CDM-specific secondary protection placed at the gate rather than at the pad. If you ship a high-speed interface, this is the chapter where your differentiation lives.

T17. The Implementation Flow — What Each Stage Decides

Every stage removes a degree of freedom. The cost of a mistake is the cost of putting that freedom back.

Before you read this

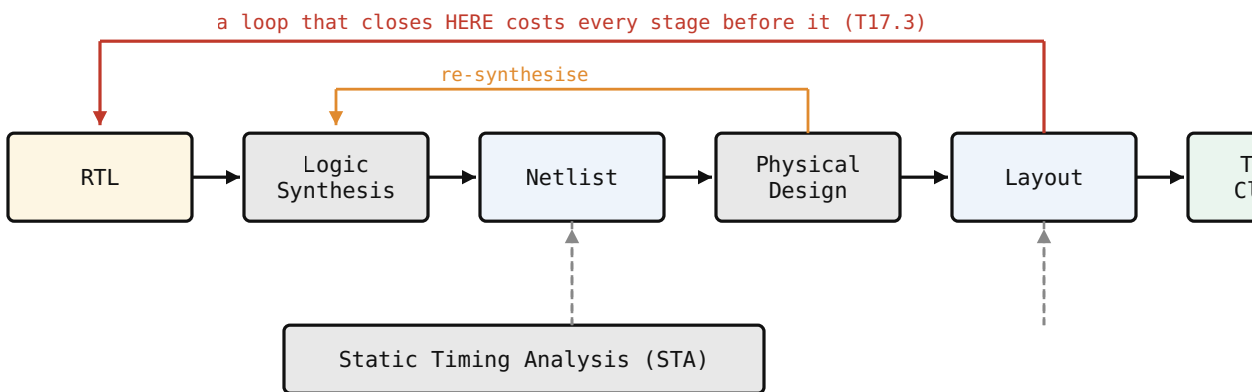
RC delay · wire delay · slew · slack · arrival time · required time · on-chip variation · floorplan · abstract view · extraction · GDS · DRC · LVS · reset synchroniser · recovery time

New in this chapter

implementation flow · logic synthesis · netlist · standard-cell library · timing library (.lib) · LEF (abstract view) · DEF (placed and routed design) · SDC (design constraints) · SPEF (extracted parasitics) · SDF (back-annotated delays) · static timing analysis · timing closure · ECO (engineering change order) · wire-load model · physically aware synthesis · signoff

A design does not become a chip in one step. It passes through a chain of tools, each of which takes a description that is free in some respect and returns one that is not: synthesis fixes which gates exist, placement fixes where they are, CTS fixes when each one is clocked, routing fixes which metal carries which net. **Each stage removes a degree of freedom, and no later stage can put it back cheaply.** That single sentence explains the shape of the whole flow — why the effort is front-loaded, why the file formats look the way they do, and why the quantity a project manager actually tracks is not progress but *iteration count*.

T17.1 The chain, and what crosses each boundary



Each stage removes a degree of freedom. The information each stage needs about its successor does not e; so every stage estimates the next one, and the estimate's drift is what decides how many times you go r

Figure 16. The same chain drawn as the flow actually runs it. The grey arrows are STA, which sees the design twice — once on the netlist, where the wires are estimated, and once on the layout, where they are real. The two coloured loops are what this chapter costs out: the orange one re-runs synthesis, the red one re-runs everything.

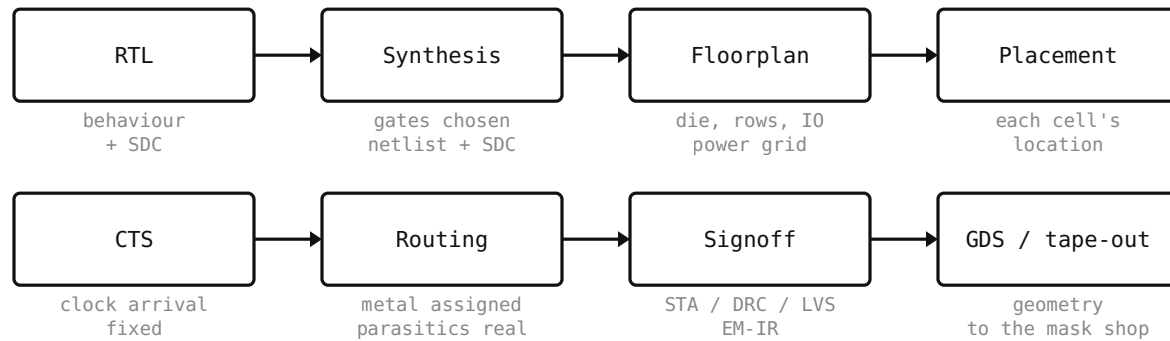


Figure 17. The digital implementation flow. Every arrow is a file, and the file formats in T17.2 are what make the arrow possible at all.

implementation flow (구현 흐름) — The ordered chain of tools that turns an RTL description plus a set of constraints into manufacturable geometry. Ordered, because each stage's output is the next stage's input — and because **the information each stage needs about the stages after it does not exist yet**. That last clause is the source of every difficulty in this chapter. The flow is a chain, but the **information** runs the other way. Synthesis must choose gate sizes to meet a delay it cannot know, because the delay depends on wire lengths that placement has not chosen yet. Placement must choose locations to meet a delay it cannot know, because routing has not happened. Every stage is therefore **estimating its own successor**, and the quality of that estimate — not the quality of the optimisation — decides how many times you go round.

Table 71. What crosses each boundary, and what it is for

File	Written by	Carries	The failure when it is wrong
.v / .sv (netlist)	Synthesis	Which cells exist and how they connect	Nothing later can add a register that RTL did not describe
.lib (Liberty)	The foundry / library vendor	Delay, transition, power and constraint tables for every cell, one file per corner	Timing signed off at a corner the silicon never sees
.lef	Library vendor + macro owner	Abstract geometry: cell outline, pin shapes, blockages, routing layers	The router cannot reach a pin (T15.3) — found after routing, fixed in the floorplan
.def	Place & route	The actual placement and routing: every instance's location, every net's metal	This is the design; a DEF that loses a power connection is a silent short-to-nothing
.sdc	The designer	Clocks, IO timing, false and multicycle paths, derates	The only file in this list a tool cannot generate — and the one that decides what 'meets timing' means
.spef	Extraction	Per-net R and C from the real geometry	Pre-route estimates replace it, and the estimate is optimistic
.sdf	STA	Back-annotated delays for gate-level simulation	Simulation passes on delays that signoff never agreed to
.gds / .oas	The flow's last step	Every polygon on every mask layer	This is what is manufactured

What actually happened

A block met timing at every corner and failed in silicon at cold. The SDC had a `set_false_path` on a reset that was genuinely asynchronous on assertion — and synchronous on **de-assertion** (T3, X42). The false path removed the one check that would have caught the recovery violation. Nothing in the flow was wrong; the file that says what the flow should check was wrong, and it is the one file nobody generates automatically.

T17.2 The estimate that every stage makes about its successor

wire-load model (배선 부하 모델) and why it was abandoned

Classical synthesis estimated a net's capacitance from its **fanout** using a table supplied with the library: a 4-fanout net in a block of this size has, statistically, this much wire on it. It is a defensible estimate when wire delay is a small part of the path.

It stopped being defensible for the reason computed in X14: wire delay does not scale with the transistors. Once the wire is a large fraction of the path, a *statistical* wire length is useless, because the paths that fail are exactly the ones whose wires are **not** typical. The estimate is right on average and wrong where it matters.

The replacement is **physical-aware synthesis**: synthesis runs a coarse placement internally, so its wire lengths come from positions rather than from a table. It does not make the estimate exact — it makes it wrong in the same direction as the next stage, which is what a usable estimate must be.

Where it is used	Every synthesis run of a block large enough that the longest wires cross a meaningful fraction of the die.
When it binds	Always, now. The wire-load flow survives only in very small blocks and in teaching material.
How to apply it	Give synthesis the floorplan (die outline, macro positions, IO locations) before it optimises, not after. Then compare its predicted slack with post-route slack on the same paths — that difference is the number that tells you whether your flow converges .

What the industry code looks like

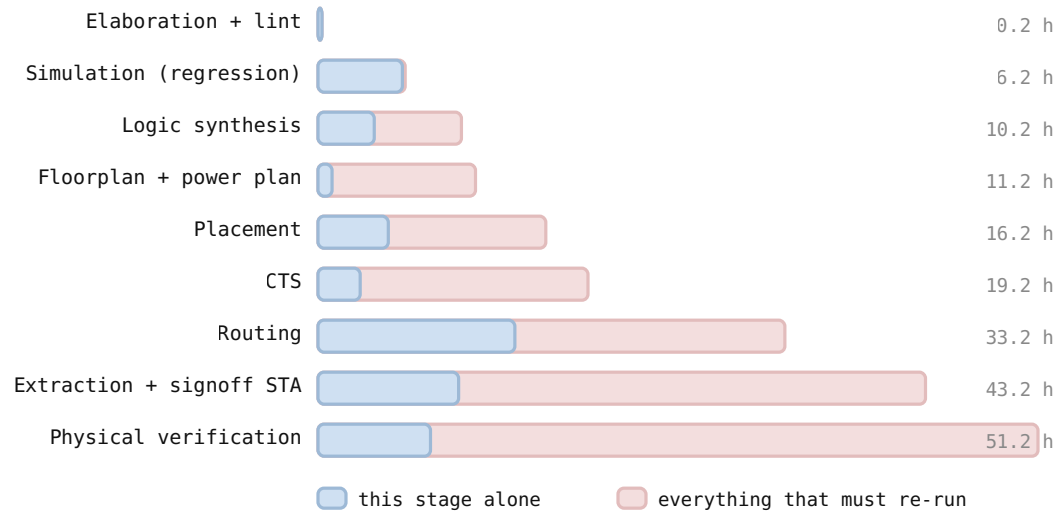
```
# The correlation check that belongs in every block's log. Not
# "did it meet timing" -- how far the estimate moved between stages.
#
#   path                synth slack   place slack   route slack   drift
#   u_alu/.../sum[31]      +120 ps      +85 ps        -40 ps        -160 ps
#   u_ctl/.../state[2]     +95 ps      +90 ps        +70 ps        -25 ps
#
# A drift column that is large and negative means the flow is not converging:
# you are not closing timing, you are discovering it.
```

This is where it goes wrong. A synthesis run that reports large positive slack on a physically aware flow is not good news by itself — check that the floorplan it used is the floorplan place-and-route will use. An optimistic estimate made against a stale floorplan is worse than no estimate, because it is believed.

T17.3 What a loop costs, measured

The flow's economics are not about tool quality. They are about where in the chain a mistake is found. A problem discovered at stage k requires re-running stages 1 through k , because each stage consumed the output of the one before it. The table below uses **assumed** runtimes for a one-million-instance block on a single server — assumed, and marked as such, because the real numbers depend on the tool, the machine and the design. What does not depend on those is the **shape**: the cost of a loop grows superlinearly with how late it closes, because the expensive stages are at the end.

What a loop costs, by where it closes



A problem found in the last stage costs 256x what the same problem costs at lint.
That factor, not a preference for careful work, is the argument for moving every check earlier.

Figure 18. The blue bar is what a stage costs on its own; the pink bar behind it is everything that must be re-run to reach that stage again. The two diverge because the expensive stages are at the end.

Table 72. Runtime per stage (assumed), and the cost of a loop that closes there

Stage	Runs for	A loop closing here costs	Relative to a lint loop
Elaboration + lint	0.2 h	0.2 h	1 ×
Simulation (regression)	6 h	6.2 h	31 ×
Logic synthesis	4 h	10.2 h	51 ×
Floorplan + power plan	1 h	11.2 h	56 ×
Placement	5 h	16.2 h	81 ×
CTS	3 h	19.2 h	96 ×
Routing	14 h	33.2 h	166 ×
Extraction + signoff STA	10 h	43.2 h	216 ×
Physical verification	8 h	51.2 h	256 ×

Read the last row: a problem found in physical verification costs 51.2 h of compute to re-reach that point, against 0.2 h for the same problem found by lint — a factor of 256 ×. And that is only the machine time. The engineer's time, the calendar days, and the fact that the fix now has to be made without disturbing everything that already closed, are all on top of it.

This is the entire argument for front-loading. It is not a cultural preference for careful RTL; it is arithmetic. The stages are ordered cheap-to- expensive, so **any check you can move earlier is multiplied by the ratio above**. That is why lint, CDC analysis, formal equivalence and a synthesis run with a real floorplan are run constantly, and why 'we will fix it in the backend' is a decision to pay several hundred times more for the same fix.

T17.4 Why timing closure sometimes does not close

The loop cost above assumes you go round once. The harder question is how many times you go round — and there the flow has a property that surprises people the first time they meet it: **closure can diverge**. Fixing one violation changes the design, and a changed design can create violations that were not there before. Upsizing a cell to fix a setup path loads its driver; inserting a buffer takes placement space and pushes a neighbour; a hold fix adds delay a setup path was relying on not having.

Derivation — Expected number of fixes when each fix can create another

	This follows	Because
1	Start with one violation. Fixing it is one unit of work.	That is the work we are counting — one ECO applied and re-analysed.
2	Each fix creates a new violation with probability p , independently.	This is the modelling assumption, and it is the one to argue with. It says a fix's side effects do not depend on how many fixes came before.
3	The process is then a branching process with mean offspring p .	One parent, p children on average — the standard Galton–Watson form.
4	Generation n has expected size p^n .	Expectation is multiplicative across independent generations.
5	Total expected work is $1 + p + p^2 + \dots = 1/(1-p)$ for $p < 1$.	Geometric series. Its sum is the total number of fixes, not the number of rounds of the tool.
6	For $p \geq 1$ the series diverges: closure does not converge at all.	Each fix creates at least one more on average, so the queue never empties. The design does not need a better engineer; it needs a change of floorplan, constraints or target frequency.

Table 73. Expected total fixes, two independent ways

p (a fix creates another)	Series $1+p+p^2+\dots$	Simulated branching process	Disagreement
0.1	1.111	1.111	0 %
0.3	1.429	1.441	0.9 %
0.5	2	2.012	0.6 %
0.7	3.333	3.361	0.8 %
0.9	10	9.976	0.2 %
0.9	20	20.06	0.3 %

The two columns are computed by **different methods** — a summed series and a simulated queue that never uses the series — and they agree to within 0.9 % over the range shown. That agreement is what licenses the claim; a single calculation would not, because the trivial explanation for a clean-looking curve is that it is the same arithmetic printed twice.

Worked example 9. Is this block going to close?

Given. Round 1 of ECO fixed 400 violations and the next STA run reported 260 new ones that did not exist before. Round 2 fixed those 260 and produced 180 new.

Method. Estimate p from the observed ratios: $260/400 = 0.65$ and $180/260 \approx 0.69$. Take $p \approx 0.67$. Expected total remaining work from the 180 now open is $180/(1-0.67)$.

Answer. About 545 more fixes — roughly three more rounds of the same size. It converges, slowly, **if p stays at 0.67**.

The independent check that matters here is not mathematical. Before believing p , confirm the new violations are genuinely new paths and not the same paths re-reported after a constraint change — a tool that re-reports is the trivial explanation for an apparent p , and it is measured by diffing the endpoint lists, not by looking at the counts.

This is where it goes wrong. p is not a constant. It rises as utilisation rises and as the remaining violations get harder, so a p measured on the easy fixes underestimates the tail. Two consecutive rounds with a rising ratio is the signal to stop doing

ECOs and change something structural — the arithmetic above says that once p reaches 1 no amount of ECO effort terminates.

T17.5 What to do with this on Monday

Table 74. Where this chapter's concepts are actually used

Concept	Block it appears in	What it decides
implementation flow · signoff	Planning any block	Which checks you can afford to run, and how often
wire-load model · physically aware synthesis	Setting up synthesis	Whether your early slack numbers mean anything
LEF (abstract view) · DEF (placed and routed design) · SDC (design constraints) · SPEF (extracted parasitics) · SDF (back-annotated delays)	Every handoff	What you owe the next stage and what you are owed
timing closure · ECO (engineering change order)	When the backend stops converging	Whether to keep fixing or to change the floorplan or the frequency

The next three chapters walk the chain in order and compute what each stage actually decides: the floorplan and the power grid that everything else sits on (T18), placement, clock tree and routing (T19), and the signoff checks that decide whether the geometry may be manufactured (T20).

Where the inventions are — what is patentable here

The flow itself is not patentable, but two things inside it are and both are actively litigated: **timing prediction before the geometry exists** (estimating post-route delay from an RTL or netlist feature set, so the loop closes in minutes rather than days) and **ECO placement** — inserting fixes into a routed design without disturbing what already met timing. Both attack the same economics this chapter computes: the flow's cost is dominated by how many times you go round it.

T18. Floorplan and Power Plan — The Geometry Everything Sits On

Placement, clock and routing all optimise inside a box somebody drew. This chapter computes the box.

Before you read this

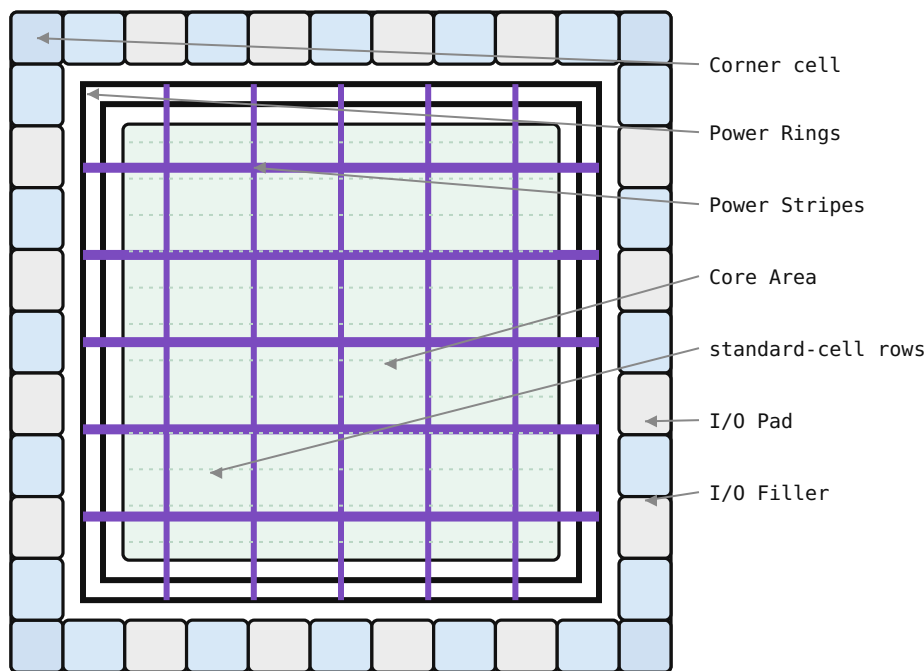
floorplan · implementation flow · LEF (abstract view) · DEF (placed and routed design) · signoff · IR drop · target impedance · PDN · decoupling capacitor · simultaneous switching noise · electromigration · Black's equation · hard macro · pin placement · pad capacitance · clamp

New in this chapter

die · core area · utilisation · aspect ratio · standard-cell row · SITE · power grid · power ring · power stripe · special route · I/O pad ring · corner cell · I/O filler · filler cell · tap cell · keep-out · pad-limited die · core-limited die

Every later stage optimises *inside* a boundary that this stage draws. Placement cannot use area the floorplan did not allocate; the clock tree cannot be shorter than the distance between the flops the floorplan separated; routing cannot use tracks the power grid consumed. **The floorplan is the only stage whose mistakes cannot be fixed downstream** — which is why it is drawn first and why almost nothing about it is automatic.

T18.1 From a gate count to a rectangle



Special route builds rings and stripes BEFORE any signal is routed — the tracks they take are gone, and the router fits into what is left.

Figure 19. The floorplan, from the outside in: the pad ring with its corner cells and fillers, two power rings, the stripes that cross the core, and the rows the cells will sit on. Every element in this drawing is sized by a calculation in this chapter.

SITE (자리) and the 표준셀 행 (standard-cell row) — A digital library does not place cells anywhere. It defines a **site** — the smallest placement unit — and cells are integer multiples of it. Rows are one site high and run the width of the core; a

cell snaps to a site boundary in a row. Everything about digital area arithmetic follows from this: the layout is a **tiling**, not a free placement.

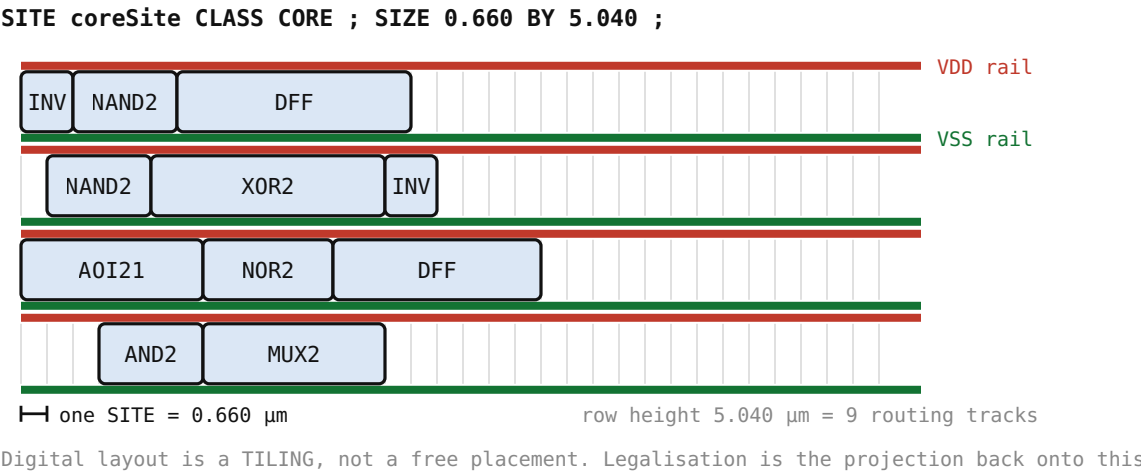


Figure 20. Rows, sites and the two supply rails that run the length of every row. A cell occupies a whole number of sites and snaps to a site boundary; that is what makes digital area arithmetic a tiling problem.

Table 75. A real SITE definition, and what each number means

LEF	Value here	What it decides
SITE coreSite	the name	Which rows this cell may sit in; a library can define more than one (e.g. a double-height row)
CLASS CORE	core	Core cells, as opposed to PAD sites in the I/O ring
SIZE 0.660 BY 5.040	0.66 μm $\times$ 5.04 μm	Row height (fixed by the library, and the reason cell area is quantised) and the placement grid in x

Derivation — Core area from an instance count

	This follows	Because
1	One site is 0.66 μm $\times$ 5.04 μm = 3.326 μm^2 .	Read straight off the LEF above.
2	Take an average cell of 4 sites, so 13.31 μm^2 per instance.	Assumed. The real figure comes from the synthesis area report and differs per design — a datapath-heavy block is wider, a control block narrower.
3	100,000 instances therefore occupy 1.331 mm^2 of cell area.	Multiplication. This is cell area only — no routing space, no power grid, no gaps.
4	At 70 % utilisation the core is 1.901 mm^2 .	Utilisation is cell area $\div$ core area. The rest is deliberately empty so the router has somewhere to go and the ECO stage has somewhere to put a buffer.
5	A square core is 1,379 μm on a side, which is 273.6 rows.	Square minimises the longest wire for a given area, so it is the default starting point — not a requirement.

utilisation (점유율) — the number that trades area against everything else

Utilisation is the fraction of the core occupied by cells. It is the floorplan's single most consequential knob, and it is **not** a quality metric: high utilisation is not good engineering, it is a bet.

Raising it shrinks the die and therefore the cost per part. It also removes the space that routing, clock buffers, hold-fix buffers and post-route ECOs all need. A block that is routable and closes timing at 70 % can become unroutable at 80 % — not gradually, but as a cliff, because congestion is a threshold phenomenon (X14).

The honest way to choose it is to **measure**: run the flow at several values and look at where routing time and DRC-violation count start rising superlinearly. That knee is the design's limit, and it moves with the design.

Where it is used	Chosen at the floorplan, before any placement exists.
When it binds	Revisited whenever the design grows — an RTL change of 10 % in gate count changes utilisation, and nobody re-runs the floorplan for it.
How to apply it	Start low (60–70 %) for a first pass so that the flow converges at all, then raise it once timing closes, watching congestion maps rather than the utilisation number itself.

What the industry code looks like

```
# The floorplan sweep that should exist in every block's history.
# util   route time   DRC after route   worst slack   die
# 0.60    9 h         0                 +60 ps       1.90 mm^2
# 0.70   14 h         0                 +40 ps       1.63 mm^2
# 0.75   22 h         41                -15 ps       1.52 mm^2   <- knee
# 0.80   49 h        890                -120 ps       1.43 mm^2
# The knee, not the smallest die, is the answer.
```

This is where it goes wrong. Utilisation computed by the tool usually excludes macro area and blockages. Two flows quoting '70 %' can mean different things; compare **cell area** ÷ (**core area** – **macro area**) explicitly before believing a comparison between blocks.

T18.2 The ring of pads, and which dimension is really limiting

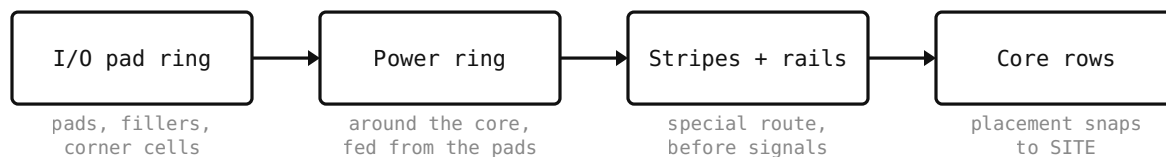


Figure 21. The floorplan from the outside in. Each layer feeds the next: the pads feed the ring, the ring feeds the stripes, the stripes feed the row rails, the rails feed the cells.

Table 76. The cells in the I/O ring that are not I/O

Cell	What it is for	What happens without it
Corner cell	Carries the pad ring's power and signal buses around the 90° turn	The ring's supply buses are four disconnected segments; the pads on one side have no return path
I/O filler	Fills the gaps between pads so the ring's buses are continuous and the implant and well layers are legal	Broken supply bus and a DRC forest at every gap
Filler cell (core)	Same idea inside the core: continuity of the n-well and the power rails between placed cells	Floating well regions and rail discontinuities — LVS and DRC failures, found late
Tap cell	Ties the well and substrate to supply at a bounded spacing	Latch-up (T16). The spacing is a foundry rule, not a preference

pad-limited (패드 제한 다이) vs 코어 제한 다이 (core-limited)

Two independent things set the die size: the area the logic needs, and the perimeter the pads need. A die is **core-limited** if the logic sets the size and the pads fit comfortably around it; it is **pad-limited** if the pad count forces a perimeter larger than the logic requires, leaving the core partly empty.

The distinction decides what is worth optimising. In a core-limited die, every gate you remove is money. In a pad-limited die, removing gates is **free of charge and free of benefit** — the die does not shrink — and the only levers are pad pitch, staggered or area-array pads, and reducing pin count.

Where it is used	The very first floorplan estimate, before any RTL is final.
When it binds	As soon as the pin list exists. It is one multiplication and it changes what the team should spend its time on for the next six months.
How to apply it	Compute the perimeter the pads need (pad count $\times$ pitch, allowing for the four corner cells) and the side of the square core the logic needs. Whichever is larger sets the die.

What the industry code looks like

```
# One line of arithmetic that reorders a project's priorities:
#   pads = 120, pad pitch = 50 um -> perimeter = 6000 um -> side = 1500 um
#   logic needs a core side of                                     1278 um
#   => PAD-LIMITED. Shrinking the netlist buys nothing. Reduce pins,
#       or stagger the pads, or accept the empty core and fill it with decap.
```

This is where it goes wrong. An empty core in a pad-limited die is not waste if you use it: it is the cheapest decoupling capacitance you will ever get (T13), and it costs only the fill cells.

Worked example 10. Is this die pad-limited?

Given. 120 pads at 50 μm pitch, and the logic of T18.1 needing a core side of 1,379 μm .

Method. Perimeter needed by the pads is $120 \times 50 \mu\text{m} = 6,000 \mu\text{m}$, i.e. a side of 1,5 μm . Compare with the core side.

Answer. Pad-limited — the pads need 1,5 μm a side against the logic's 1,379 μm , so the pads set the die and there is 0.35 mm^2 of core the logic does not use.

The same comparison decides whether to use an **area-array** (flip-chip) pad arrangement, which removes the perimeter constraint entirely at the cost of a more expensive package and a different ESD strategy (T16).

This is where it goes wrong. The pad pitch is not free to choose: it is set by the package's bond-wire or bump capability, and a pitch that the package cannot assemble is not a floorplan, it is a wish. Ask the package house for the number before the arithmetic, not after.

T18.3 Sizing the power grid — and what actually forces it to be dense

The core draws 194.4 mA at 1.8 V for the 35 mW assumed above. That current has to arrive through metal, and the metal has to satisfy two separate constraints that have nothing to do with each other: the **voltage** must not sag more than the budget (T13), and the **current density** must not exceed the electromigration limit that T14's Black equation sets for the required life-time. Sizing the grid means satisfying both, and knowing which of the two is binding.

Derivation — IR drop along a stripe fed from both ends

	This follows	Because
1	A stripe of length L and width w has resistance $R = R_s L/w$.	Sheet resistance times the number of squares. R_s is a process constant per metal layer.
2	The load is not at the end; it is distributed along the stripe.	The cells that draw current are spread over the rows the stripe crosses.

	This follows	Because
3	Feed it from both ends and let the current per unit length be $i = I/L$.	Both ends is what a power ring gives you — this is why the ring exists.
4	Current at distance x from one end is $I/2 - i \cdot x$.	Half the total enters each end; it is consumed as it travels.
5	Drop to the centre is $\int_0^{L/2} (I/2 - i \cdot x)(R_s/w) dx = R \cdot I/8$.	Elementary integral; the L^2 terms collect into $R = R_s L/w$.
6	So the worst drop on the stripe is $R \cdot I/8$, at the centre.	Compare with $R \cdot I$ for an end-fed point load: distributing the load and feeding both ends is a factor of 8. That factor is the whole reason a grid is a grid.

Table 77. Stripe width required, for a 3 % (54 mV) IR budget and an EM limit of 2 mA/μm

Stripes	Width for IR	Width for EM	Which binds	Total metal width	of the core width
2	12.4 μm	24.3 μm	EM	48.61 μm	3.53 %
4	6.21 μm	12.2 μm	EM	48.61 μm	3.53 %
8	3.1 μm	6.08 μm	EM	48.61 μm	3.53 %
16	1.55 μm	3.04 μm	EM	48.61 μm	3.53 %
32	0.78 μm	1.52 μm	EM	48.61 μm	3.53 %

Look at the last two columns: **the total metal width does not change with the number of stripes**. Both constraints scale as $1/N$ per stripe, so N stripes of width w/N cost exactly what one stripe of width w costs. The naive conclusion — "the stripe count is free, pick anything" — is wrong, and finding out *why* it is wrong is the point of the rest of this section. The uniform-load model above is missing the path from the stripe to the cell.

Where the supply voltage goes

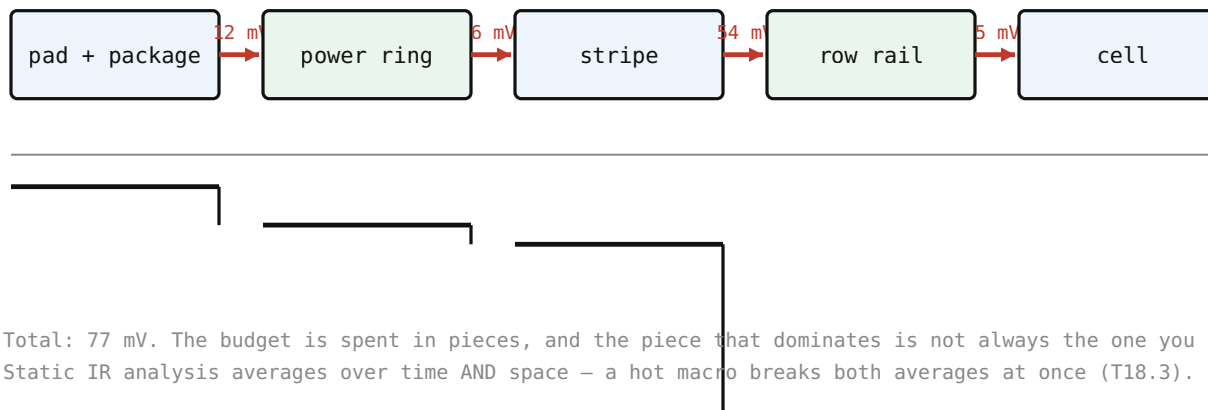


Figure 22. The supply budget is spent in pieces, and the arithmetic of this section is about which piece dominates. The staircase below the boxes is the same numbers drawn as a voltage against distance.

Derivation — What the stripe count actually buys

	This follows	Because
1	A cell does not connect to a stripe; it connects to the rail in its row.	Rails run horizontally in every row at the library's row pitch; stripes run vertically over them and drop down through vias.
2	So the current travels along the rail from the nearest stripe, over a distance up to half the stripe pitch $p = L/N$.	The rail is fed from the stripes on both sides of it, which is the same both-ends geometry as before, at a smaller scale.

	This follows	Because
3	The rail segment between two stripes carries the current of the area it serves: $I \cdot (\text{row height} \times p) / L^2$.	Uniform current density over the core, times the area of that segment.
4	Its drop is $(R_{s,M1} \cdot p / w_{\text{rail}}) \cdot I_{\text{seg}} / 8$, which is $\propto p^2$.	One factor of p from the resistance, one from the current the longer segment collects. This is where N appears.
5	At $N = 4$ that is 5.1 mV; at $N = 1$, 81.7 mV.	Computed from the numbers at the top of this chapter. Still small — which is an honest result and not the one most people expect.

Table 78. The drop, decomposed — uniform load

Stripes	On the stripe	On the row rail	Total	Budget
1	54 mV	81.7 mV	135.7 mV	54 mV
2	54 mV	20.4 mV	74.42 mV	54 mV
4	54 mV	5.1 mV	59.1 mV	54 mV
8	54 mV	1.28 mV	55.28 mV	54 mV
16	54 mV	0.32 mV	54.32 mV	54 mV

The independent control. The stripe drop above comes from the integral in the derivation. To check the algebra rather than trust it, the same stripe is also solved as a 400-segment resistive ladder with both ends grounded and the current drawn uniformly — Kirchhoff's laws and a linear solve, using none of the formula. It gives 54 mV against the closed form's 54 mV, a disagreement of 1.0×10^{-11} %. That check validates the *arithmetic*. It does not validate the *model*: both assume a uniform load and an ideal ring, and the next paragraph is about what happens when that assumption fails.

What actually happened

A block signed off with 54 mV of static IR drop failed at speed. The static analysis was correct and irrelevant: the current was not uniform. A hard macro in one corner drew a third of the total from a tenth of the area, and the switching was correlated — the whole datapath clocked at once. Static IR analysis averages over time and space, and **both averages were wrong in the same direction**.

Table 79. Where a dense grid earns its tracks: 30 % of the current in 10 % of the area

Stripes	Pitch	Rail drop, uniform	Rail drop, hot spot	Ratio
2	689.3 μm	20.4 mV	24.5 mV	1.2 $\times$
4	344.7 μm	5.1 mV	15.3 mV	3 $\times$
8	172.3 μm	1.28 mV	3.83 mV	3 $\times$
16	86.17 μm	0.32 mV	0.96 mV	3 $\times$
32	43.08 μm	0.08 mV	0.24 mV	3 $\times$

Kill the trivial reading first. The ratio column is 3 $\times$ and stays there, and it is not a discovery: it is exactly the local power-density ratio, 0.30/0.10. Nothing subtle happened. What is useful is the consequence, because the rail drop goes as p^2 : to hold the same drop under a 3 $\times$ density you must reduce the pitch by $\sqrt{3} = 1.732$, i.e. put about 73.2 % more stripes over the hot region than over the rest. That is a number you can act on, and it is the quantitative form of the rule practitioners state without the derivation: **grid density is set by the worst local power density, not by the average**. The scheduling corollary follows — you cannot size the grid before you know where the power is, so the floorplan and the power plan are revised together once a real power map exists, not drawn once at the start.

special route (스페셜 라우트) — why power is routed before signals

The power grid is not routed by the signal router. It is built first, by a separate step, and the signal router then treats it as an obstacle. The reason is that power routing has properties signal routing does not: the shapes are wide, regular, on fixed layers, connected to everything, and their width is decided by analysis rather than by connectivity.

The order also encodes the priority. Tracks the grid takes are gone; the router must fit the design into what is left. **Doing it the other way round would mean sizing the power grid to whatever space the signals happened to leave**, which is exactly backwards — the grid's width comes from IR and EM limits that do not negotiate.

Where it is used	Immediately after the floorplan, before placement is legalised and long before signal routing.
When it binds	Re-run whenever the power map changes materially — a new macro, a frequency change, a different operating voltage.
How to apply it	Rings around the core and around each macro; stripes at the pitch the hot-spot analysis demands; then the rail connections, then the via arrays at every crossing. Check EM at the vias , not only in the metal — a via array is usually the current bottleneck.

What the industry code looks like

```
# The order, and why each step is where it is:
# 1 add_io_ring          pads, corner cells, fillers      <- perimeter fixed
# 2 add_power_ring       core ring, macro rings           <- both-end feed exists
# 3 add_stripes          pitch from the hot-spot map      <- tracks consumed here
# 4 connect_rails        stripes down to every row rail
# 5 add_via_arrays       every crossing; check EM per via
# 6 ...only now: place, cts, route
```

This is where it goes wrong. Via electromigration is usually the binding constraint and is the one most often missed, because metal width is visible in the layout and via count is not. A stripe sized correctly for EM, dropping to a rail through two vias, fails at the vias.

T18.4 What to do with this on Monday

Table 80. Where this chapter's concepts are actually used

Concept	Block it appears in	What it decides
die · core area · utilisation · aspect ratio	First floorplan, before any RTL freeze	Whether the part is affordable, and which dimension to optimise
pad-limited die · core-limited die · I/O pad ring · corner cell · I/O filler	As soon as the pin list exists	Whether shrinking the logic buys anything at all
power ring · power stripe · power grid · special route	Immediately after the floorplan	IR drop, EM lifetime, and how many routing tracks are left
filler cell · tap cell · keep-out	Before physical verification	Whether the geometry is legal at all (T20)

The numbers to carry out of this chapter are structural rather than specific. Total grid metal is set by the total current and does not depend on how you divide it; grid **pitch** is set by the worst local power density and is invisible to a uniform analysis — in the example above the hot spot multiplies the rail drop by $3 \times$ at the same stripe count. And the factor of 8 in the both-ends distributed-load result is the quantitative reason the power distribution is a ring and a grid rather than a tree.

With a box, a grid and a set of rows, the tools can finally start moving things. The next chapter is what they do with them: placement, the clock tree, and routing.

Where the inventions are — what is patentable here

Power-grid synthesis — deciding stripe pitch and width automatically from a power map rather than from a designer's rule of thumb — is an active area, as is **floorplanning by machine learning**, where a model proposes macro placements. Both are attempts to automate the one stage of the flow that is still mostly judgement, and both are judged by exactly the numbers this chapter computes: IR drop, congestion and area.

T19. Placement, Clock Tree and Routing

Three stages, three decisions: where each cell is, when it is clocked, and which metal carries which net.

Before you read this

implementation flow · netlist · standard-cell row · SITE · utilisation · core area · power grid · power stripe · static timing analysis · slack · on-chip variation · wire delay · RC delay · slew · Elmore delay · repeater · logical effort · SDC (design constraints) · DEF (placed and routed design) · ECO (engineering change order) · timing closure · MTBF · synchroniser depth · antenna rules · filler cell · decoupling capacitor

New in this chapter

placement · legalisation · HPWL (wirelength) · timing-driven placement · congestion · routing track · gcell · Rent's rule · clock-tree synthesis · insertion delay · clock skew · useful skew · clock mesh · H-tree · clock gating · integrated clock-gating cell · global routing · detail routing · search and repair · antenna violation

The floorplan of T18 produced a box, a grid of power and a set of rows. What is inside the box is still a netlist — a list of cells and connections, with no positions. The three stages of this chapter give it geometry, and each of them fixes something the next one cannot change: placement fixes distance, clock-tree synthesis fixes arrival time, routing fixes the actual parasitics. After routing, the design has no remaining degrees of freedom except the small, local, expensive ones called ECOs.

T19.1 Placement: an optimisation with a discrete constraint

legalisation (합법화) — The step that moves cells from wherever the optimiser wanted them to the nearest **legal** position: on a row, snapped to a site boundary (T18), not overlapping another cell, respecting every placement rule the library carries. Global placement solves a continuous problem; legalisation projects the answer back onto the discrete set of positions that actually exist.

That two-step structure — solve a relaxed continuous problem, then project — is why placement quality reports show a jump at legalisation. The continuous solution is a lower bound nobody can achieve. **If the jump is large, the cause is almost always density**: the optimiser piled cells where there was no room, and legalisation had to move them a long way. That is a floorplan or utilisation problem reported by the placer, not a placer problem.

Table 81. What each placement objective actually minimises

Objective	The quantity	When it is the right one
Wirelength (HPWL)	Sum over nets of the half-perimeter of the net's bounding box	The default. Cheap to evaluate and differentiable enough to optimise; a good proxy for both delay and congestion when neither is critical
Timing-driven	Weighted wirelength, where a net's weight rises with how critical its path is	Once timing is close. It trades total wirelength for the few paths that decide $F_{\max}$
Congestion-driven	Predicted routing demand per region, spread out	When routing fails rather than timing. It deliberately <i>increases</i> wirelength to make the design routable at all

What actually happened

A block was re-placed with aggressive timing weights after a marginal STA run. Worst slack improved by 40 ps and the design became unroutable: the placer had pulled the critical datapath into a tight cluster, and the region's routing demand went past supply. **The three objectives above are in tension, and optimising one while not measuring the others is how a converging flow stops converging.**

T19.2 Congestion: supply, demand, and why the average lies

track (트랙) · **gcell** · 혼잡도 (**congestion**) — A **track** is one routable line on one metal layer, at the layer's pitch. The router divides the core into **gcells** (global routing cells, a few tens of tracks across) and, for each gcell edge, compares the number of

wires that must cross it with the number of tracks available. **Congestion** is that ratio. Above 1, some net cannot be routed there and must detour — or, if everything nearby is also above 1, cannot be routed at all.

Derivation — Track supply for the T18 block — pure geometry

	This follows	Because
1	The core is 1,379 μm on a side (T18).	Carried over so the two chapters describe the same chip.
2	At a routing pitch of 0.56 μm , one layer offers 2,462 tracks across the core.	Core width divided by pitch. Assumed pitch , consistent with the 5.04 μm row height being nine tracks.
3	Each track runs the full 1,379 μm .	Length available per track, ignoring blockages — an upper bound, and it is worth remembering that it is one.
4	With 4 usable signal layers, supply is 13.58 m of routing.	Layers the power grid did not take. That subtraction is where T18 and this chapter meet: stripes consume tracks.
5	Demand is 100,000 nets $\times$ 30 μm average = 3 m.	The average net length is the weak link in this estimate and it is assumed. It is also the only quantity here that a real flow measures rather than predicts.
6	Global ratio: 0.22.	Comfortably below 1 — and the next paragraph explains why that tells you almost nothing.

Table 82. How much the conclusion depends on the one assumed number

Assumed average net length	Demand	Demand $\div$ supply	Reading
20 μm	2 m	0.15	routable on average
30 μm	3 m	0.22	routable on average
45 μm	4.5 m	0.33	routable on average
60 μm	6 m	0.44	routable on average
80 μm	8 m	0.59	routable on average

Two honest statements about the table. First, the estimate is **dominated by an assumption**: a factor of four in the average net length moves the answer from comfortable to impossible, and no amount of care in the geometry changes that. Second, and more important: **the global ratio is the wrong quantity**. Routing fails locally. A design at a global 0.22 can have gcells above 1.0 in a region a hundred microns across, and those gcells are what decides whether the router finishes. This is the same structural error as T18's uniform power map — an average that is right everywhere and useful nowhere.

Rent's rule (렌트 법칙) — why routing demand grows faster than area

Empirically, if you draw a boundary around N randomly chosen connected cells in a real design, the number of connections crossing that boundary is $T = k \cdot N^p$, with p typically between 0.5 and 0.75 for logic. The exponent is a measure of how much a design communicates with itself.

The consequence for routing is direct. A region of area A holds $N \propto A$ cells and must carry $T \propto N^p$ connections across its boundary, while the boundary itself offers track capacity $\propto \sqrt{A} \propto N^{0.5}$. For any $p > 0.5$ — that is, for essentially all real logic — demand outgrows supply as the block gets bigger. **This is why routing does not get easier when you add metal layers to a growing design, and why hierarchy exists.**

It is an empirical law, not a theorem, and it should be used as one: measure p on your own netlists by counting crossings for randomly chosen partitions, and be suspicious of a quoted value.

Where it is used	Estimating whether a block should be partitioned, and predicting congestion before a placement exists.
When it binds	At the architecture stage, when block sizes are still negotiable — which is the only time the answer can be acted on cheaply (T17).

How to apply it

Partition the netlist at several sizes, count boundary nets, fit $\log T$ against $\log N$. A p above about 0.7 says the block will be congestion-limited and should be split or given more metal.

What the industry code looks like

```
# Measuring p on your own netlist instead of quoting a textbook value:
#   N      T (crossings)    log-log fit -> p
#   100      68
#   1000     271
#   10000    1090          p ~ 0.60, k ~ 4.3
# Then: a 100k-instance block implies ~ 4.3 * 100000^0.60 crossings at the
# top level -- compare with the tracks its perimeter actually offers.
```

This is where it goes wrong. Rent's rule describes *average* connectivity. A design with a broadcast bus or a large crossbar violates it locally in exactly the place that will fail, so a good global p is not a routability guarantee — it is a reason to look at the structures the law does not describe.

T19.3 Clock-tree synthesis: why insertion delay costs skew

insertion delay (삽입지연) · 클럭스큐 (**clock skew**) — **Insertion delay** (or latency) is the time from the clock's entry point to a flop's clock pin. **Skew** is the difference in insertion delay between two flops. Setup and hold analysis cares about skew; latency, on its own, cancels out of the launch-capture comparison — which is why the textbook answer is 'latency does not matter'. The textbook answer is wrong in silicon, for the reason computed next.

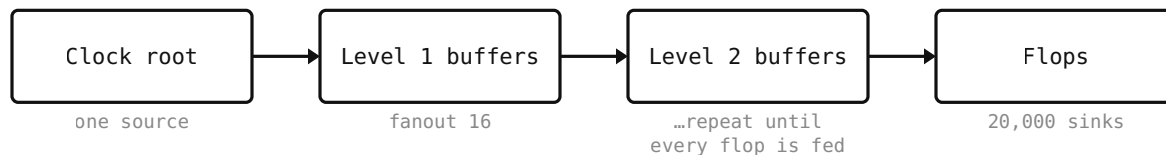


Figure 23. A buffered clock tree. With a fanout of 16 and 20,000 sinks it is 4 levels deep and holds about 1,335 buffers — every one of which is a source of the variation computed below.

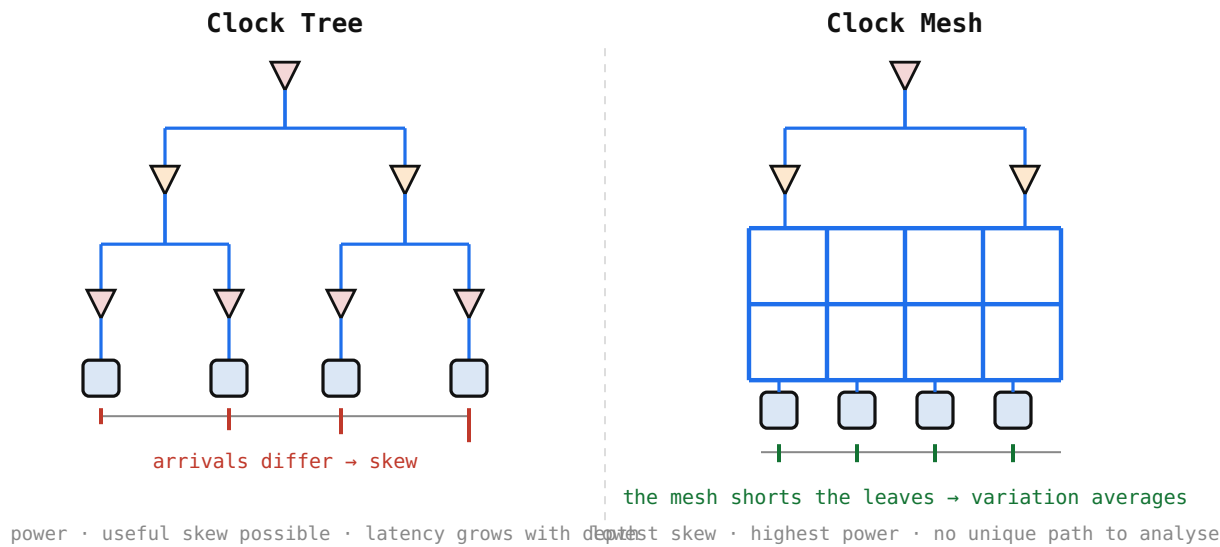


Figure 24. The two structures, and the property that separates them. In the tree each leaf has its own path, so local variation accumulates along it; in the mesh the leaves are shorted together, so it averages instead.

Derivation — How on-chip variation turns latency into skew

	This follows	Because
1	Two flops share the clock path up to the point where the tree branches.	By construction — the tree is a tree. Call the shared fraction the common path ; here it is 98 % of the latency (assumed).
2	OCV analysis derates the launch path late and the capture path early.	It must assume the worst combination of process and environment variation between two points on the die, because STA has no way to know which combination silicon will present.
3	With ± 7 % derates, the late path is $\times 1.07$ and the early path $\times 0.93$.	Assumed derate values; real ones come from the foundry and depend on distance and on the number of stages in the path.
4	Effective skew = $1.00 \times 1.07 - 0.98 \times 0.93 = 15.86$ % of the insertion delay.	The common part does not cancel, because the two derates applied to it are different. Only common-path pessimism removal gives that part back, and only for the path that is genuinely shared.
5	So a tree whose real skew is 2 % of latency is analysed as though it were 15.9 %.	This is the answer to 'why does latency matter' . Latency is multiplied by the derate difference and reappears as skew — so a deep tree costs frequency even when it is perfectly balanced.

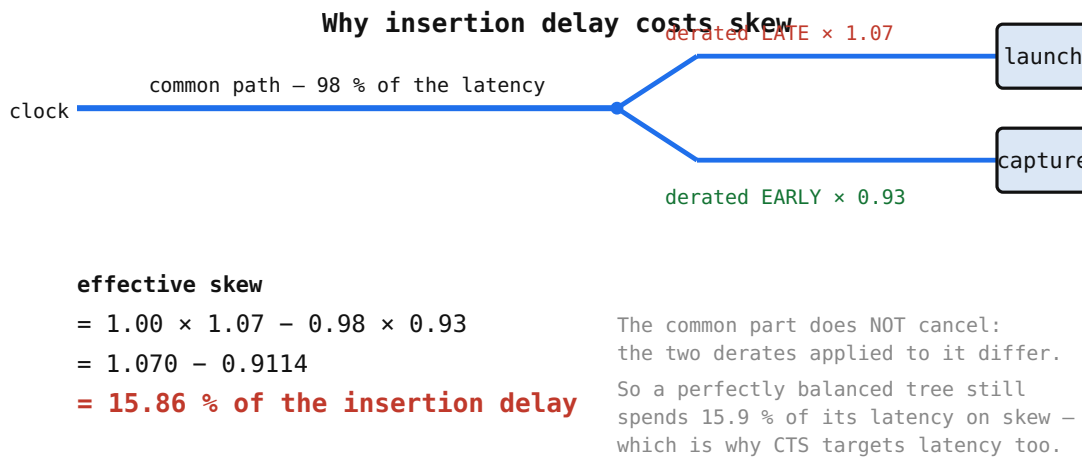


Figure 25. The arithmetic in one picture. The common path is derated late on one side and early on the other, so it does not cancel — and what does not cancel is a fraction of the insertion delay.

Table 83. What insertion delay costs, at 5 MHz (2, ps period)

Insertion delay	Real skew (2 %)	Skew after OCV derate	of the clock period
200 ps	4 ps	31.72 ps	1.59 %
400 ps	8 ps	63.44 ps	3.17 %
600 ps	12 ps	95.16 ps	4.76 %
900 ps	18 ps	142.7 ps	7.14 %
1,2 ps	24 ps	190.3 ps	9.52 %

Read the last column. At 1,2 ps of insertion delay, OCV alone consumes 9.52 % of the period before a single gate has switched. **The engineering target in CTS is therefore not skew alone but latency and skew together**, and the most effective fix is usually structural: fewer levels, a shorter path from the root, or a different distribution topology.

Table 84. Three clock distribution structures, and what each one buys

Structure	Skew	Power and area	When to use it
Buffered tree (CTS default)	Good, and controllable per-sink; latency grows with depth	Lowest — only the buffers the fanout requires	Nearly always. It is the only one that supports useful skew and gating
H-tree	Excellent by construction — every leaf is the same distance from the root	Higher; it ignores where the sinks actually are	Regular arrays where the sinks really are on a grid: an ADC's comparators, a mesh of identical tiles
Clock mesh	Lowest. The mesh shorts the leaves together, so local variation averages instead of accumulating	Highest — a large, always-switching capacitance, and difficult to analyse because the mesh has no unique path	High-frequency cores where skew is the binding constraint and the power is worth paying

useful skew (유용 스큐) — spending the clock deliberately

Skew is not only an error. If a path from A to B is tight, delivering B's clock *later* gives that path more time — and takes the same amount from B's own outgoing path. CTS tools do this deliberately: they solve for a set of arrival times that maximises the worst slack across the design rather than equalising arrivals.

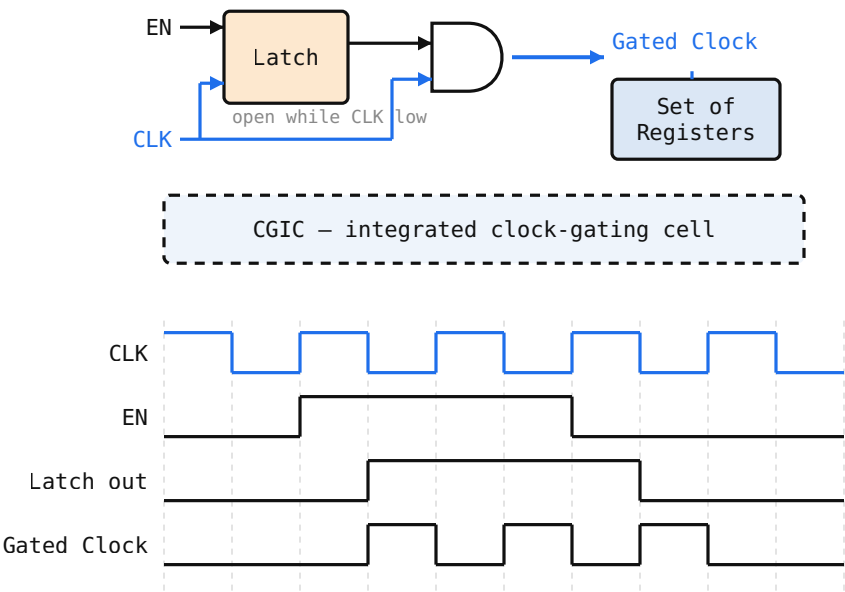
It is genuinely free frequency, and it has a genuine cost: **the design now depends on the clock tree being what the analysis assumed**. A tree balanced to zero skew degrades gracefully when a buffer is slower than modelled; a tree carrying 80 ps of useful skew on a critical path degrades into a hold violation, which is not fixable by slowing the clock.

Where it is used	Inside CTS and again in post-CTS optimisation.
When it binds	When a handful of paths block frequency and everything else has margin — the classic case for it.
How to apply it	Bound it. Allow useful skew up to a fraction of the period, keep hold margin explicitly larger on the paths that use it, and re-check at every corner, because the skew and the path delay do not track each other across corners.

What the industry code looks like

```
# The report to read after CTS, per critical endpoint:
# endpoint      latency  useful skew  setup slack  hold slack
# u_alu/acc[7]   412 ps   +80 ps      +12 ps      +9 ps  <- thin
# u_ctl/st[1]    398 ps    0 ps       +95 ps      +60 ps
# An endpoint living on useful skew with 9 ps of hold margin is a silicon
# failure waiting for a slow corner. Bound the skew, not just the slack.
```

This is where it goes wrong. Useful skew and clock gating interact badly if the gating cell sits on a path carrying skew: enabling and disabling changes the loading on the branch and therefore the skew it delivers. Keep gating cells above the point where useful skew is applied.



Without the latch, EN changing while CLK is high produces a GLITCH: a partial pulse, long enough for a flop to see and short enough to violate its minimum pulse width. The latch confines EN changes to the low phase.

Figure 26. The integrated clock-gating cell and why the latch is not optional. The waveform is the whole argument: the latch is transparent only while the clock is low, so the enable is stable for the entire high phase and the gated clock is always a full pulse or no pulse.

clock gating (클럭 게이팅) and the latch that makes it legal

A clock network switches every cycle whether or not the flops it feeds have anything to do. Gating it off is the single largest dynamic-power lever in a digital design. The circuit that does it — the **integrated clock-gating cell** (CGIC) — is a latch followed by an AND, and the latch is not optional.

Why the latch. ANDing the enable directly with the clock produces a glitch whenever the enable changes while the clock is high: a partial pulse that is long enough to be seen by a flop and short enough to violate its minimum pulse width. The latch is transparent while the clock is *low*, so the enable can only change during the low phase and is held stable for the whole high phase. The gated clock is then always a full pulse or no pulse.

The clock network of the block assumed here is 20,000 flop clock pins plus about 1,335 buffers — roughly 84,005 fF of capacitance switching at 5 MHz, which is 136.1 mW. Gating it off 60 % of the time saves 81.65 mW.

Check that against the block, not against itself. T18 assumed the whole block draws 350 mW. The clock network computed here is 38.9 % of that — high, and it should be: this is an ungated tree feeding 20,000 flops, which is precisely the design that motivates gating. Gating 60 % of it returns 23.3 % of the whole block's power. Had the fraction come out at 5 % or at 90 %, the right response would be to distrust the capacitance assumptions rather than to report the saving.

Where it is used	Inserted by synthesis wherever the RTL has an enable, and again by hand where the RTL does not express one.
When it binds	Everywhere. The question is not whether to gate but at what granularity — one CGIC per 8 flops is typical; one per flop costs more than it saves.
How to apply it	Write the RTL so the enable is visible (<code>if (en) q <= d;</code> rather than a recirculating mux), then check the report for the percentage of flops actually gated. Ungated flops in a low-power design are usually an RTL style problem, not a tool problem.

What the industry code looks like

```
// The RTL that lets the tool insert a CGIC:
always_ff @(posedge clk) if (en) q <= d;           // gateable

// The RTL that prevents it -- the clock still toggles every cycle:
always_ff @(posedge clk) q <= en ? d : q;          // recirculating mux

// Both simulate identically. Only the first one saves power, and the
// difference does not appear anywhere in functional verification.
```

This is where it goes wrong. A gated clock is a new clock as far as CDC and STA are concerned. It needs its own constraint, and the enable itself must be timed — an enable that arrives late enough to violate the latch's setup produces exactly the glitch the latch was there to prevent.

T19.4 Routing, in two passes and a repair loop

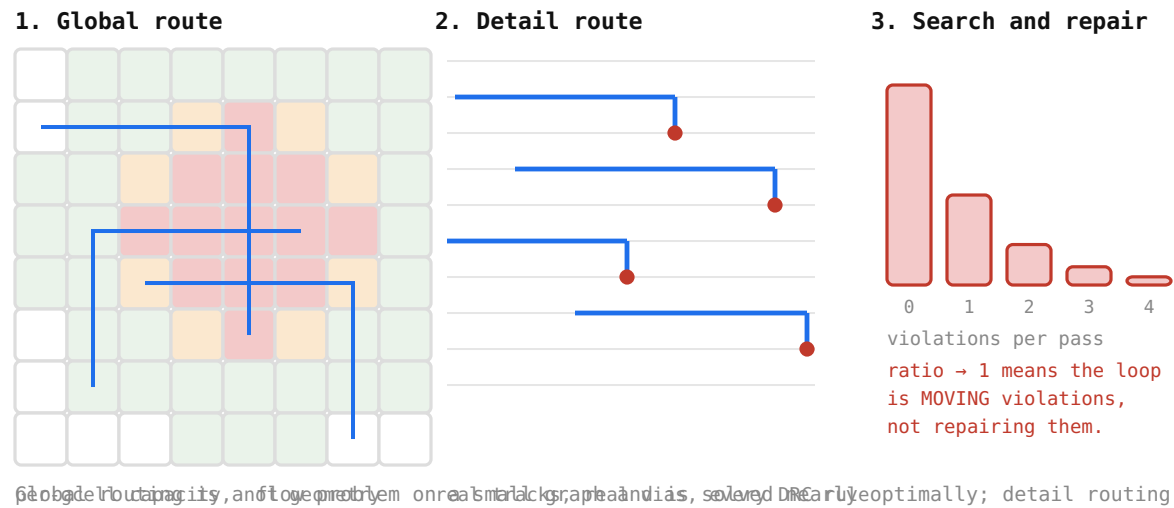


Figure 27. Three different problems solved three different ways. The red cells in the first panel are gcells whose demand exceeds supply; the red squares in the second are DRC violations that detail routing could not avoid; the third panel is what the repair loop does with them.

Table 85. Why routing is not one algorithm

Pass	Works on	Decides	What it deliberately ignores
Global route	The gcell grid	Which gcells each net passes through, and on which layers	Exact wire positions, most design rules — it is a capacity problem, not a geometry problem
Detail route	Real tracks and vias inside each gcell	The actual metal: every segment, every via, every jog	Nothing — this pass must produce DRC-clean geometry
Search and repair	The remaining violations	Rip up the offending wires and route them again, with the violation as a penalty	Optimality. It is a local repair loop, and it is iterative because each repair can create new violations

The split exists because the two problems have different shapes. Global routing is a flow problem on a graph of a few thousand nodes and is solved nearly optimally. Detail routing is a geometry problem with hundreds of rules, in a space far too large to search — so it is solved greedily, net by net, and the greediness is exactly what search-and-repair afterwards cleans up.

Table 86. A converging repair loop: each pass removes 55 % of what is left

Iteration	Violations open	Removed this pass
0	4,200	0
1	1,890	0
2	850	0
3	383	0
4	172	0
5	78	0
6	35	0
7	16	0

Worked example 11. Is this repair loop going to finish?

Given. After detail routing a block reports 4,200 DRC violations. Successive search-and-repair passes report 4,200 → 2,310 → 1,290 → 980 → 910 → 890.

Method. The first two ratios are about 0.55 and 0.56 — a clean geometric decay. The last three are 0.76, 0.93, 0.98: the decay has stopped. Fit the tail rather than the head, because the tail is what decides termination.

Answer. It will not finish. A ratio approaching 1 means the remaining violations are not being repaired but moved — the same congested region handed back and forth. The residue of roughly 890 is structural: not enough track supply where those nets have to go.

The remedy is never another repair pass. It is upstream: lower " utilisation in that region, spread the placement, add a routing layer, or " change the pin access. That is T17's loop-cost arithmetic arriving in " practice — the fix is cheap where the cause is and expensive where the " symptom is.

This is where it goes wrong. The trivial explanation must be killed before concluding anything structural: confirm the violations are the **same** ones, in the same place, and not a fresh set each pass. Diff the violation coordinates between passes. If they move, the loop is thrashing on a global shortage; if they sit still, it is one local cause — often a macro pin that cannot be reached (T15.3).

antenna violation (안테나 위반) — a rule that is about manufacturing, not about the circuit

During etch, a long metal segment connected to a gate but not yet connected to any diffusion acts as a charge collector; the accumulated charge discharges through the thin oxide and damages it. The rule bounds the ratio of connected metal area to gate area **at every intermediate stage of manufacturing**, not in the finished chip.

That last clause is what makes it confusing: a layout can be antenna-clean when complete and violate the rule at metal 2, because metal 3 — which would have connected the segment to a diode — does not exist yet at that point in the process.

The two fixes are to break the long segment by jumping to a higher layer (so the collector is never large on any one layer) or to add a small diode that provides a discharge path. Routers insert both automatically; the reason to understand it is that the fixes cost area and delay on nets that are usually already critical.

Where it is used	After detail routing, as part of physical verification (T20).
When it binds	Every time. It is not optional and not negotiable with the foundry.
How to apply it	Let the router fix it, then look at where it had to: a cluster of antenna fixes marks a net that is long, on one layer, and reaching a gate — often a clock or a reset, which are precisely the nets where the added diode capacitance is least welcome.

This is where it goes wrong. An antenna diode on a clock leaf adds capacitance to a branch CTS already balanced, and it is added **after** CTS. Re-check skew after antenna fixing rather than assuming the tree survived it.

T19.5 What to do with this on Monday

Table 87. Where this chapter's concepts are actually used

Concept	Block it appears in	What it decides
placement · legalisation · HPWL (wirelength) · timing-driven placement	Every placement run	Whether the jump at legalisation is telling you the floorplan is too dense
congestion · routing track · gcell · Rent's rule	Before routing, and before partitioning	Whether this block can be routed at all at this size
clock-tree synthesis · insertion delay · clock skew · useful skew · clock mesh · H-tree	After placement	How much of the period the clock network consumes before any logic runs

Concept	Block it appears in	What it decides
clock gating · integrated clock-gating cell	RTL, and then the gating report	The largest single dynamic-power lever, and whether your RTL style allows it
global routing · detail routing · search and repair · antenna violation	After CTS	Whether the repair loop converges, and what to change upstream if it does not

The number to carry is the OCV multiplication: 15.86 % of insertion delay reappears as analysed skew under ± 7 % derates with a 98 % common path. It is the reason a clock tree is engineered for latency as well as balance, the reason clock meshes exist despite their power, and the quantitative content of the advice to keep the clock tree shallow.

The geometry now exists and is, as far as the router is concerned, finished. Whether it may be manufactured is a separate question, answered by a separate set of tools, and that is the next chapter.

Where the inventions are — what is patentable here

Two areas here are patent-dense and worth knowing about before you reinvent them. **Clock-tree structures** — mesh, multi-source CTS, and the hybrid schemes that use a mesh at the top and trees below — are heavily filed, because skew is worth frequency and frequency is worth money. So is **detailed routing with machine-learned congestion prediction**, which tries to move the congestion information earlier, for exactly the loop-cost reason computed in T17.

T20. Signoff — The Checks That Decide Whether It May Be Built

Every signoff check answers one narrow question, and the failures that reach silicon live in the gaps between them.

Before you read this

signoff · implementation flow · DRC · LVS · antenna violation · density rules · extraction · SPEF (extracted parasitics) · SDF (back-annotated delays) · SDC (design constraints) · static timing analysis · on-chip variation · slack · ECO (engineering change order) · GDS · IR drop · electromigration · Black's equation · detail routing · clock skew · threshold voltage · leakage current · latch-up

New in this chapter

physical verification · ERC (electrical rule check) · metal fill · CMP (chemical-mechanical polishing) · corner · multi-mode multi-corner analysis · temperature inversion · static IR analysis · dynamic IR analysis · switching vector · functional ECO · metal-only ECO · spare cell · tape-out · mask

A routed design is a set of polygons and a claim: that these polygons, made in this process, will behave the way the RTL said. Signoff is the set of independent checks that test that claim before anyone spends mask money on it. Each check is a different tool with a different model of the design, and it is worth being precise about which question each one answers — because **the failures that reach silicon are almost never inside a check; they are in the space between two.**

T20.1 What each check actually asserts

Table 88. The signoff set, stated as claims

Check	The claim it establishes	What it cannot see
DRC	The geometry obeys every manufacturing rule the foundry publishes for this process and this metal stack	Whether the geometry implements the design. A DRC-clean empty die passes
LVS	The devices and connections extracted from the layout match the netlist, device for device and net for net	Whether the <i>netlist</i> is right. LVS compares the layout to the schematic; if the schematic is wrong, LVS confirms the error
ERC	No floating gates, no unconnected wells, no supply shorted through a device that was not meant to conduct	Anything dynamic — it is a topology check
Antenna (T19)	No metal segment collects enough charge during etch to damage a gate oxide	Nothing about the finished circuit
Density / fill	Every layer's local metal density is inside the window CMP requires	The capacitance the fill adds — that is extraction's job, and it must be re-run after fill
EM / IR	No wire exceeds its current-density limit for the required lifetime (T14), and no node sags below the budget	Whether the switching activity used to drive it resembles the real workload — see T20.3
STA	Every timing path meets setup and hold, at every corner in the list, in every mode in the list	Paths the SDC excluded, and modes nobody listed

Each check is sound, narrow, and blind to the next one

DRC	geometry obeys the rules	does not know what it implements
LVS	layout matches the netlist	does not know the netlist is right
STA	every listed path meets timing	paths the SDC excluded
EM / IR	no wire over its limit	vectors that were not run
Antenna	no oxide damage during etch	nothing about the finished circuit
Density	CMP window is met	the capacitance fill just added

The left column is what signoff establishes. The right column is where silicon failures live – not inside a check, but in the space between two of them. That space is the engineering judgement

Figure 28. The same table as a picture, because the shape is the message: every check establishes something narrow, and the red column on the right is where the failures that reach silicon actually live.

Read the third column as a list of where to look after a silicon failure. Each check is sound; each is narrow; and the narrowness is deliberate, because a check that tried to establish everything could not be run in finite time. **The engineering judgement in signoff is not in running the checks — it is in knowing what the union of them leaves out.**

metal fill (금속 채움) and the extraction it invalidates

Chemical-mechanical polishing removes material at a rate that depends on the local pattern density. A region with little metal polishes faster and ends up thinner, which changes both resistance and the thickness of the dielectric above it. The foundry therefore requires every layer's density to sit inside a window, and the flow satisfies that by adding **fill** — floating or grounded metal shapes in the empty areas.

Fill is added after routing, which means it is added **after** the extraction that timing signoff used. Those shapes are metal next to signal wires; they add capacitance. Grounded fill adds it predictably; floating fill adds a smaller, harder-to-model amount.

So the order is: route, fill, **re-extract**, re-run STA. A flow that signs off timing on the pre-fill extraction has signed off a design that will not be manufactured.

Where it is used	Between detail routing and final timing signoff.
When it binds	Always. The only variation is whether the fill is inserted by the P&R tool or by the physical-verification tool, and the two do not produce identical results.
How to apply it	Insert fill, re-extract with fill present, re-run the corner set, and diff the slack against the pre-fill run. A small, uniform degradation is expected; a large change on a few nets means fill landed next to a long, critical, sparsely-surrounded wire.

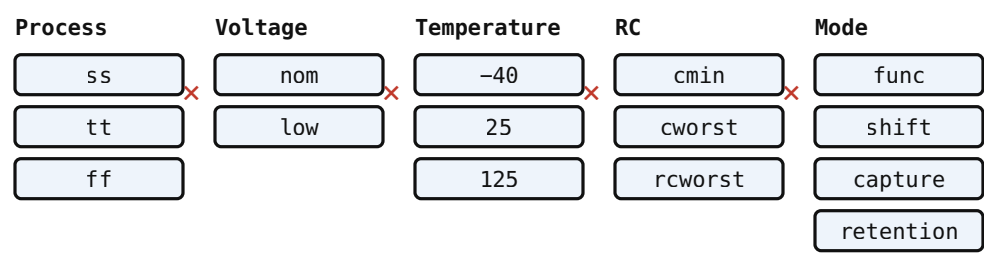
What the industry code looks like

```
# The diff that must exist before tape-out:
#   path                slack pre-fill    slack post-fill    delta
#   u_dsp/.../mac_out[17]      +45 ps          +31 ps          -14 ps
#   u_if/.../clk_gate_en      +88 ps          +12 ps          -76 ps  <- look
# A -76 ps outlier is a wire that was alone in an empty region and is now
```

surrounded by fill. It is real, it is predictable, and it is missed by any
flow that signs off before filling.

This is where it goes wrong. Fill also changes the inductance and the return path of high-speed signals, which matters for the links of T8 far more than for logic. For those, ask for fill exclusion zones — and remember that an exclusion zone is a density violation unless the surrounding area compensates.

T20.2 Corners multiply, and most of them are dominated



3 × 2 × 3 × 3 × 4 = 216 corner-mode combinations
× 2 h of STA each = 18 days of compute
and a single ECO throws all of it away. That is T17.3's loop cost in its most expensive form.

Figure 29. Why signoff runs on a farm. The axes are independent of each other, so they multiply rather than add — and the corner list is therefore negotiated rather than assumed.

Derivation — Where the signoff runtime comes from

	This follows	Because
1	Process: 3 · Voltage: 2 · Temperature: 3 · RC (interconnect): 3 · Mode: 4.	Each axis is independent of the others — that is why they multiply rather than add. The counts are assumed and typical.
2	That is 3 × 2 × 3 × 3 × 4 = 216 corner-mode combinations.	Multiplication. Nothing subtle is happening; that is the point.
3	At 2 h per STA run, a full set is 432 h of compute.	18 days on one machine — which is why signoff runs on a farm and why the corner list is negotiated rather than assumed.
4	A single ECO invalidates all of it.	This is T17's loop-cost arithmetic in its most expensive form: the last stage in the chain is the largest one.

Table 89. The axes, and why each one is there

Axis	Values	Why it cannot be dropped
Process	3	slow / typical / fast — and slow-n/fast-p corners on top of that in a mixed-signal part
Voltage	2	nominal and the low end of the regulator's tolerance
Temperature	3	−40 °C, 25 °C, 125 °C
RC (interconnect)	3	cmin / cworst / rcworst — extraction corners are independent of the device corners
Mode	4	functional, scan-shift, scan-capture, low-power retention

MMMC — analysing modes and corners together instead of one at a time

The 216 combinations above are not 216 separate analyses in a modern flow. Multi-mode multi-corner analysis loads the several mode constraints and the several corner libraries into one session and propagates once, sharing the work that is common between them.

The saving is real but it is not the main reason to use it. The main reason is that **optimisation must see all the corners at once**. Fixing a setup violation at the slow corner by adding drive strength can create a hold violation at the fast corner; a tool that sees one corner at a time will oscillate between them, and that oscillation is exactly the non-convergence of T17 with $p \geq 1$.

Where it is used	Post-CTS optimisation onwards, and all of signoff.
When it binds	From the first optimisation that can make a corner-dependent trade — which in practice means as soon as hold fixing starts.
How to apply it	Define the analysis views explicitly (a view is one mode plus one delay corner plus one RC corner), mark which are for setup and which for hold, and check that the list covers the product's real operating envelope rather than the list inherited from the previous project.

What the industry code looks like

```
# An analysis-view list is a specification, not a configuration:
# view          mode          delay corner  rc corner  checks
# func_ss_cw    functional    ss_0.9v_125c  cworst    setup
# func_ff_cb    functional    ff_1.1v_m40c  cbest     hold
# shift_ss_cw   scan_shift    ss_0.9v_125c  cworst    setup
# shift_ff_cb   scan_shift    ff_1.1v_m40c  cbest     hold
# The question to ask of this table is not "is it right" but
# "which operating condition of the product is NOT in it".
```

This is where it goes wrong. Scan-shift mode is the one most often forgotten and the one most often broken, because its clock structure is different from the functional one and its hold requirements are severe. A part that fails only on the tester is usually a shift-mode timing problem, not a defect.

The corner list is long partly because the dominance people expect does not hold. The intuition is that the slow corner bounds setup and the fast corner bounds hold, so two corners would do. That intuition comes from a regime where delay increases monotonically with temperature — and modern low-voltage operation is not in that regime.

Derivation — Temperature inversion, from the device model of T1

	This follows	Because
1	Delay $\propto C \cdot V / I_d$, with $I_d \propto \mu(T) \cdot (V - V_{th}(T))^\alpha$.	The α -power form from T1. $\alpha \approx 1.3$ for a short-channel device.
2	Mobility falls with temperature: $\mu \propto T^{-1.5}$. That makes the part slower when hot.	Phonon scattering. This is the effect everyone knows.
3	Threshold also falls with temperature: $dV_{th}/dT \approx -0.8$ mV/K. That makes the part faster when hot.	The Fermi level moves with temperature. This is the effect that gets forgotten.
4	Which one wins depends on the overdrive $V - V_{th}$, because the threshold shift is a large fraction of a small overdrive and a small fraction of a large one.	The two effects enter multiplicatively, so the comparison is between their logarithmic derivatives.
5	$d(\ln \text{delay})/dT = 1.5/T - \alpha k/(V - V_{th})$. Setting it to zero gives $V - V_{th} = \alpha k T / 1.5$.	Differentiate and solve. The result depends on nothing but the two temperature coefficients and α .

	This follows	Because
6	At 300 K that is an overdrive of 0.208 V, i.e. $V \approx 0.658$ V.	Below that supply the part is slowest when cold , and the corner list must contain both temperature extremes for setup.

Table 90. Delay against temperature, normalised to 25 °C

Supply	−40 °C	25 °C	125 °C	Which is the slow corner
1.8 V	0.728 ×	1 ×	1.432 ×	normal
1.2 V	0.759 ×	1 ×	1.352 ×	normal
0.9 V	0.812 ×	1 ×	1.247 ×	normal
0.75 V	0.887 ×	1 ×	1.133 ×	normal
0.65 V	1.026 ×	1 ×	0.994 ×	inverted
0.6 V	1.211 ×	1 ×	0.881 ×	inverted

The table stops at 0.6 V for a reason worth stating: at −40 °C this model's threshold is 0.503 V, so below about 0.5 V of supply the device has no overdrive at all when cold and the delay is not merely long, it is undefined. That is not an artefact — it is the near-threshold regime, and it is why a low-voltage retention mode has a minimum supply specified at the **cold** corner.

The independent control. The crossover supply is computed twice: once from the closed form $V = V_{th0} + \alpha T/1.5$, giving 0.658 V, and once by bisecting on the sign of a *numerically differentiated* delay, using none of that algebra, giving 0.658 V — a disagreement of 2.53×10^{-8} mV. And the trivial explanation is killed by the table rather than argued away: if the effect were an artefact of the normalisation, the 'inverted' verdict would not move with supply, and it does.

What actually happened

A design signed off at 125 °C and typical voltage failed at −40 °C in a cold-start test. It was a low-voltage retention mode: at that supply the part was slowest cold, and the corner list — inherited from a 1.8 V project where the intuition held — contained only the hot corner for setup. **Nothing in the flow was broken. The list of what to check was written by someone applying a rule that had stopped being true.**

T20.3 EM and IR signoff need something the design does not contain

static IR (정적 IR) · 동적 IR (dynamic IR) · 스위칭 벡터 — **Static** IR analysis distributes an average current over the grid and solves the resistive network — the calculation of T18. **Dynamic** IR analysis solves the grid with the supply's inductance and decoupling included (T13), driven by the current waveform that a particular pattern of switching produces. That pattern is the **switching vector**, and it is not part of the design: it has to be supplied.

This is the one signoff check whose answer depends on an input that is neither in the netlist nor in the constraints. Choose a quiet vector and the analysis passes; choose the worst one and it may not — and nothing in the flow tells you which you chose. **The honest procedure is to state, in the signoff report, which vectors were used and why they are believed to bound the real workload**, and to treat a dynamic-IR pass without that statement as an unverified claim rather than a result.

Table 91. Three ways to get a vector, and what each is worth

Source	How representative	What it misses
Functional simulation of a real workload	Best available — it is the actual switching	Only covers the workload you simulated, and long simulations produce enormous activity files
Vectorless (tool-propagated probabilities from a few toggle rates)	Adequate for a first pass and for finding gross grid errors	Correlation. It assumes blocks switch independently, and the failures come from everything switching at once

Source	How representative	What it misses
Scan-capture pattern	The worst case for many designs and rarely analysed	Nothing — it is often the real answer, because scan capture switches far more of the die simultaneously than any functional mode does

Worked example 12. Which mode sets the peak current?

Given. The block of T18/T19 draws 350 mW functionally. In scan-shift mode every flop toggles on every cycle, and the shift clock runs at a quarter of the functional frequency.

Method. Functional switching activity for logic is typically 10–20 % of nodes per cycle; shift activity approaches 50 % of flops per cycle. Take the clock network as fixed (it toggles either way) and compare the flop-output switching: roughly 0.15 → 0.5 is a factor of about 3.3 in that component, against a factor of 0.25 from the slower clock.

Answer. Shift mode draws roughly **0.8×** the functional switching power in that component — comparable, not obviously smaller. The peak is set by whichever mode runs its clock fastest at high activity, and on many parts that is scan *capture*, not shift.

This is also why test engineers reduce shift frequency and use low-power ATPG that limits simultaneous toggling — not for test quality, but because the part cannot supply the current. That constraint belongs in the floorplan discussion of T18, and it usually arrives after it.

This is where it goes wrong. The numbers above are ratios of assumptions, not measurements, and should not be quoted as results. The conclusion that survives is structural: **test modes are not low-power modes**, and a grid signed off only against functional vectors has not been signed off against the mode that will run first, on the tester, on every part shipped.

T20.4 Changing a signed-off design

Table 92. Two kinds of ECO, and what each one costs

Kind	What changes	Which signoff must be redone
Metal-only ECO (uses spare cells)	Routing only. The function comes from cells already placed and already on the mask, re-wired on the metal layers	Everything downstream of routing: extraction, STA, EM/IR, DRC, LVS. But only the changed masks are re-made — which is the entire point
Functional ECO (new cells)	Placement and routing. New instances go into gaps, displacing what was there	All of the above, and every mask is new

spare cells (스페어 셀) — paying now for the option to change later

Spare cells are unused gates — typically a mix of NANDs, NORs, inverters and flops — scattered through the placement and tied off. They cost area and leakage and do nothing. What they buy is the ability to implement a logic change by **rewiring metal only**, because the transistors the change needs are already on the diffusion masks.

The economics are a straightforward option price. A full mask set at an advanced node costs millions; a metal-only revision costs a fraction of it and takes weeks rather than months. Spending 1–2 % of the area on spares is cheap insurance if the probability of needing a change is anything but negligible — and on a first silicon it never is.

Where it is used	Inserted at placement, distributed rather than clustered.
When it binds	Every design that will have a first silicon. Omit them only on a well-understood derivative of a proven part.
How to apply it	Scatter them so that any region has some within reach — a spare cell 500 μm from the fix is not usable, because the wire that reaches it breaks the timing the fix was made to satisfy. Include flops: the changes you cannot predict are usually sequential.

What the industry code looks like

```
# A spare-cell plan is a distribution, not a count:
#   region      spares: inv  nand2  nor2  dff      nearest-spare distance
#   core NW          24    16    16    8          < 60 um
#   core NE          24    16    16    8          < 60 um
#   near macro A      6     4     4     2          < 90 um  <- thin
# 'We have 400 spare cells' says nothing. 'No point in the core is more
# than 90 um from a spare flop' is the statement that makes an ECO possible.
```

This is where it goes wrong. Tie the spare inputs off properly and check that ERC sees them as tied. An untied spare gate is a floating input — the failure mode is excess current through a half-on inverter, and it passes DRC, LVS and STA because none of them is looking for it.

tape-out (테이프아웃) — The point at which the GDS is released to the mask shop. It is a commercial event rather than a technical one: after it, changing the design costs a mask set. Everything this chapter describes exists to make the claim that the release is safe — and the honest form of that claim always includes the list of what was **not** checked.

T20.5 What to do with this on Monday

Table 93. Where this chapter's concepts are actually used

Concept	Block it appears in	What it decides
physical verification · ERC (electrical rule check) · metal fill · CMP (chemical-mechanical polishing)	Before every tape-out	Whether the geometry is manufacturable, and whether timing was signed off on the geometry that will actually be made
corner · multi-mode multi-corner analysis · temperature inversion	Writing the analysis-view list	Whether the corner list covers the product, or the previous project
static IR analysis · dynamic IR analysis · switching vector	Power-integrity signoff	Whether the vectors bound the real workload — including test modes
functional ECO · metal-only ECO · spare cell	Placement, months before you need them	Whether a post-silicon fix costs a metal revision or a full mask set
tape-out · mask	The release decision	What you are asserting, and what you are not

The numbers to carry: 216 corner-mode combinations at 2 h each is 18 days of compute that one ECO throws away — the strongest quantitative argument in this book for fixing things early. And the crossover overdrive of 0.208 V is why "slow is hot" stopped being a safe rule: below about 0.66 V of supply, in this model, the slow corner is the cold one.

The part can now be built. Whether an individual die works is a different question again, answered not by a tool but by a machine with the die in it — and that is the last chapter of this arc.

Where the inventions are — what is patentable here

Signoff itself is the least patent-encumbered part of the flow — the rules belong to the foundry and the checks are commodity. What is filed heavily is **ECO automation**: choosing which spare cells to use and how to reroute with the minimum disturbance, so that a fix does not invalidate the signoff that has already been paid for. That is a direct attack on the cost computed in T17.

T21. The Verification Environment — What It Proves

Coverage says you tried it. Only the checker says it worked. Confusing the two is the most expensive mistake in verification.

Before you read this

implementation flow · netlist · CDC verification · metastability · synchroniser depth · asynchronous FIFO · handshake · gray code · signoff · ECO (engineering change order) · static timing analysis · timing closure

New in this chapter

verification environment · testbench · agent · sequence · sequencer · driver · monitor · scoreboard · reference model · virtual interface · functional coverage · covergroup · coverpoint · cross coverage · constrained-random stimulus · assertion · coverage closure · directed test · regression

Everything after this chapter's stage in the flow assumes the RTL is right. Synthesis will faithfully implement a bug; place-and-route will close timing on it; signoff will certify that it is manufacturable. **Verification is the only stage that can answer whether the design does what it was supposed to do**, and it is the stage that consumes the most engineering effort on almost every project. This chapter is about the structure that effort takes, and about the one distinction that decides whether the effort was worth anything.

T21.1 The structure, and why it is split the way it is

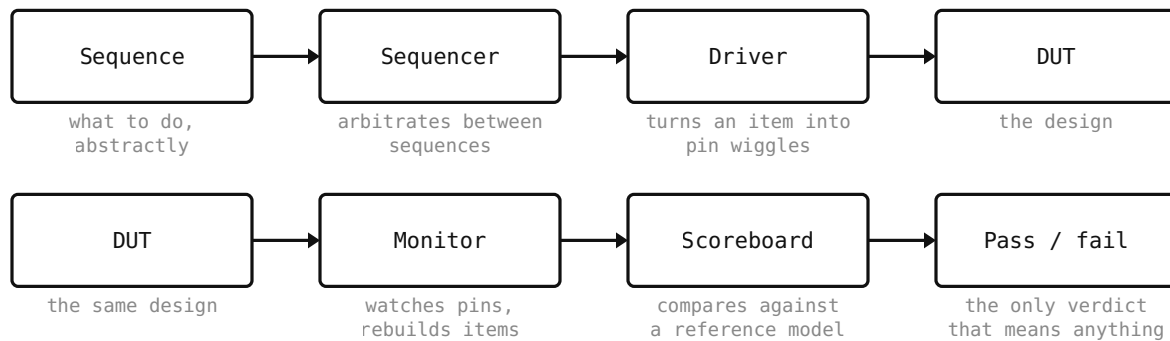
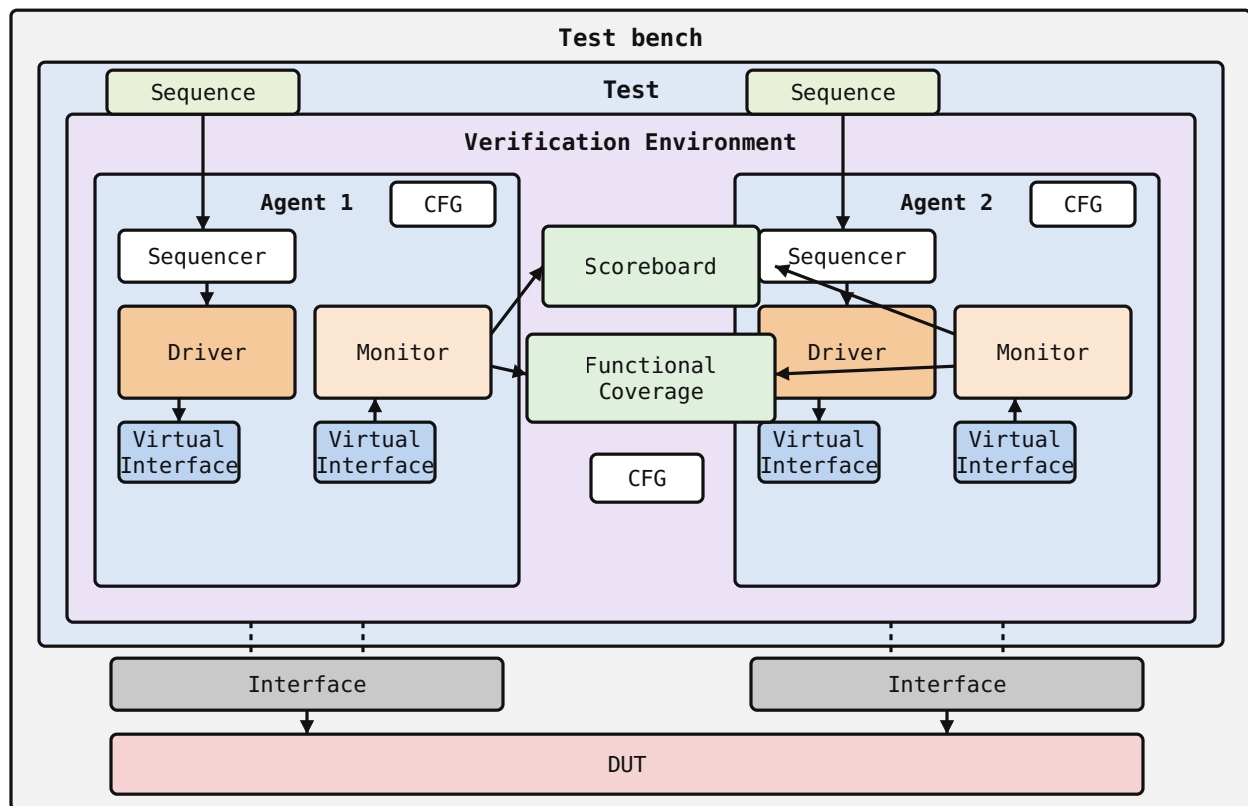


Figure 30. The two halves of an agent. The upper path drives; the lower path observes and judges. They are deliberately not connected to each other — the monitor reconstructs what happened from the pins, so that a driver bug cannot hide itself by telling the scoreboard what it intended.



The upper path drives; the lower path observes. They are deliberately NOT connected to each other — a monitor that trusts the driver's intent cannot detect a driver that drove the wrong thing.

Coverage says you TRIED it. Only the scoreboard says it WORKED.

Figure 31. The environment in full. Read it as two halves that meet only at the DUT: sequences drive down the left of each agent, monitors observe up the right, and the scoreboard in the middle is the only thing that ever says the design is wrong.

Table 94. Each component, and the one question it answers

Component	Answers	Why it is separate from its neighbour
Sequence	What transactions should happen?	It is written per test and knows nothing about pins — so the same sequence runs on a different bus protocol
Sequencer	In what order, when several sequences compete?	Arbitration is a policy, and policies change per test while drivers do not
Driver	How is one transaction expressed on the wires?	It is the only component that knows the protocol's timing. Everything above it is protocol-agnostic
Monitor	What actually happened on the wires?	It must be independent of the driver. A monitor that trusts the driver's intent cannot detect a driver that drove the wrong thing
Scoreboard	Was it correct?	It holds the reference model. It is the only component whose failure means the DUT is wrong
Coverage collector	What did we try?	It must not judge. Mixing coverage and checking produces a testbench that reports 100 % of nothing
Config object	Which of the above exist, and how are they set up?	So that one environment serves a block, a subsystem and the chip, with the agents that do not apply turned off rather than deleted

virtual interface (가상 인터페이스) — the one construct with no software analogue

The testbench's driver and monitor are **class objects**: created at run time, garbage-collected, passed around by handle. The pins they must wiggle belong to a **module instance**, which is created at elaboration and exists for the whole simulation. A class cannot hold a reference to a static elaborated object directly, because at the time the class is written the instance does not exist.

A virtual interface is the bridge: a handle, held by the class, pointing at an interface instance that *was* elaborated. The testbench's configuration step sets it, and after that the driver can drive real pins from dynamic code.

The most common failure in a new environment is a null virtual interface — the driver was built before anything assigned it. It is not a subtle bug, but it is unfamiliar, because no ordinary software language has this split between two object lifetimes.

Where it is used	Every agent that touches the DUT.
When it binds	Set during the build/connect phase, before any sequence runs.
How to apply it	Assign it from the top-level module into the configuration database, and have every driver and monitor fetch it during build — then assert that it is non-null rather than letting the first pin access fail with a null-handle error a thousand cycles later.

What the industry code looks like

```
// Top level, where both worlds meet:
my_if dut_if (.clk(clk));           // elaborated: exists for the run
my_dut u_dut (.p(dut_if));

initial begin
    // Hand the elaborated thing to the dynamic world, by name:
    uvm_config_db#(virtual my_if)::set(null, "", "vif", dut_if);
    run_test();
end

// In the driver's build phase -- and check it, do not assume it:
if (!uvm_config_db#(virtual my_if)::get(this, "", "vif", vif))
    `uvm_fatal("NOVIF", "virtual interface was never set")
```

This is where it goes wrong. Clocking blocks matter here: driving a DUT input from the testbench without one races with the DUT's own evaluation at the clock edge, and the race is resolved differently by different simulators. A testbench that passes on one simulator and fails on another is almost always a missing clocking block, not a tool bug.

T21.2 Coverage is a measure of stimulus, not of correctness

functional coverage (기능 커버리지) · 커버포인트 · 크로스 — A **coverpoint** samples a value and sorts it into **bins**; a **covergroup** collects coverpoints and is sampled at a defined event; a **cross** is the set of bin combinations of two or more coverpoints. Coverage is the fraction of bins that have been hit at least once. Note what that definition does **not** contain: any reference to whether the design behaved correctly when the bin was hit.

100 % functional coverage with a scoreboard that never fires is 100 % evidence of nothing. Coverage answers 'did the stimulus reach this situation'. Correctness is answered only by the scoreboard and the assertions. The two are reported side by side, they look similar, and they measure entirely different things — and a green coverage report is the most persuasive way to ship a bug, because it is a number, it is high, and nobody asks what it counted.

What actually happened

A block shipped with every covergroup at 100 %. The scoreboard's compare method had an early return added during de-

bugging and never removed: it checked the first beat of each burst and silently passed the rest. Every bin was hit. Nothing was compared. The escape was found by a customer, and the testbench that missed it had a perfect coverage report throughout. **The check that would have caught it is not more coverage — it is a mutation test: change one bit of the DUT's output and confirm the scoreboard goes red.**

mutation testing (자해 검사) for a testbench

A testbench is a program that is supposed to fail when the design is wrong. Nothing in a normal regression ever exercises that property — every run is against a design that is (by then) right, so the failure path is never taken.

The fix is to break the design on purpose: invert a bit in a response, drop a transaction, delay a handshake by one cycle, corrupt a byte in the middle of a burst. Then run the regression and confirm it goes red, and confirm **which** check caught it. A mutation nothing catches is a hole in the checking, located precisely.

This is the same discipline that this book applies to its own numbers: a measurement that cannot come out wrong has not measured anything.

Where it is used	Every environment, at least once per major block, and in CI if you can afford the runtime.
When it binds	Before believing a coverage report. A mutation campaign is the only evidence that the coverage number is attached to a checker at all.
How to apply it	Keep a list of mutations as a file, each with the check that is expected to catch it. Run them; a mutation caught by <i>no</i> check, or caught only by a timeout, is a finding.

What the industry code looks like

```
# mutations.txt -- and the expected catcher, which is the real content
#  id  mutation                                expected catcher
#  m1  invert resp[0]                          scoreboard data compare
#  m2  drop every 8th write                     scoreboard transaction count
#  m3  delay ready by 1 cycle                   protocol assertion a_ready_stable
#  m4  return wrong burst length                scoreboard length compare
#  m5  corrupt byte 3 of burst                  scoreboard data compare
#
# Result column after the run. Any row whose catcher is "none" or
# "test timeout" is a hole -- a timeout means nothing noticed, it just
# stopped.
```

This is where it goes wrong. A mutation caught by every check is not a good result either — it means the mutation was too coarse to locate anything. The useful mutations are the ones caught by exactly one check, because those map a check to the failure it is responsible for.

T21.3 Why crosses explode: the arithmetic of random closure

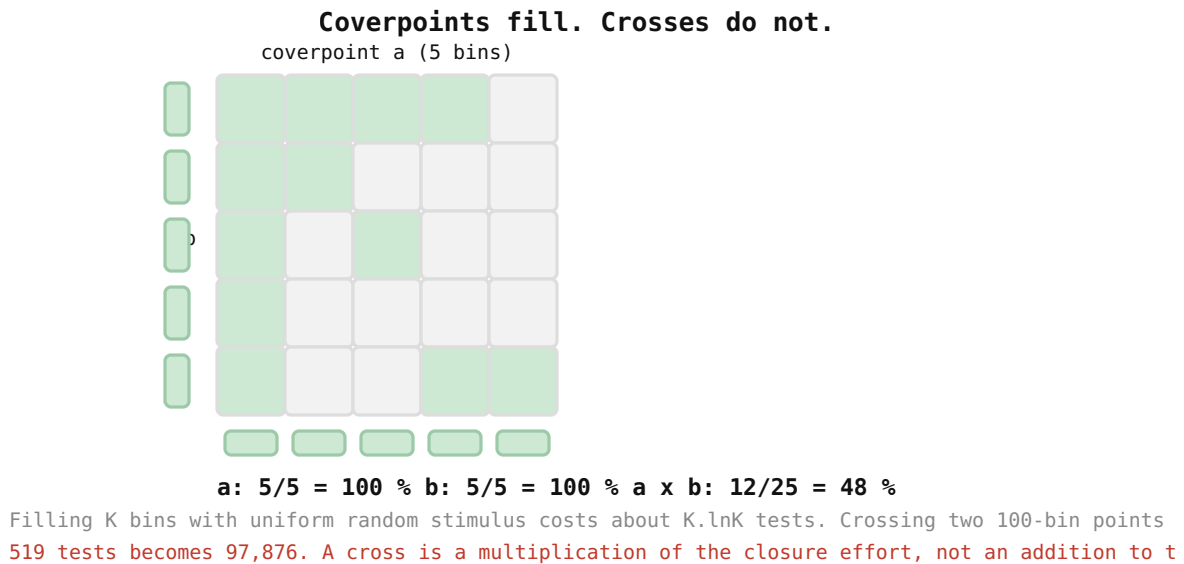


Figure 32. Each axis is fully covered and the cross is not. The bars along the edges are what a coverpoint report shows; the grid is what closure actually requires.

Derivation — How long random stimulus takes to fill K bins

	This follows	Because
1	Assume each test lands in one of K bins, uniformly and independently.	The assumption to argue with. Real constrained-random stimulus is neither uniform nor independent — but this case is the optimistic one, so what it computes is a lower bound on the effort.
2	With j bins already hit, the chance a test hits a new one is $(K-j)/K$.	Counting. The remaining bins are $K-j$ of the K.
3	So the expected number of tests to go from j to j+1 is $K/(K-j)$.	A geometric waiting time has mean $1/p$.
4	Summing over $j = 0 \dots K-1$ gives $K \cdot (1 + 1/2 + \dots + 1/K) = K \cdot H_K$.	Expectation is additive even though the waits are not independent of each other's outcomes.
5	$H_K \approx \ln K + 0.577$, so the cost is about $K \cdot \ln K$.	Slightly worse than linear in the number of bins — which is fine. The problem is not this factor; it is that K itself is a product when you cross.

Table 95. Expected tests to fill every bin, two independent ways

Bins K	$K \cdot H_K$	Simulated draw-until-full	Disagreement
10	29.29	29.564	0.94 %
100	518.74	519.18	0.08 %
1,000	7,485.5	7,356	1.73 %
10,000	97,876.1	not simulated	the point: it is already $\sim 10^5$ tests

The two columns are computed by different means — a summed series and an actual repeated drawing that uses none of it — and they agree to within a few per cent, with the residual being the Monte-Carlo sampling error at 300 trials for the largest case. That is what licenses the last row, which is the one that matters: **crossing two 100-bin coverpoints makes K = 10,000**

and turns a 518.7-test job into a 97,876.1-test one. A cross is not a small addition to a coverage model; it is a multiplication of the closure effort.

Table 96. And that was the optimistic case — one rare bin dominates everything

Probability of the rarest bin	Expected tests to hit it once	What it means in practice
0.01	100	comfortable
0.001	1,000	an overnight regression
1.000×10^{-4}	10,	an overnight regression
1.000×10^{-5}	100,	an overnight regression
1.000×10^{-6}	1.0000×10^6	will never happen — write a directed test

The uniform calculation understates the problem badly, because random closure is governed by the **rarest** bin, not the average. The last rows are the quantitative reason directed tests still exist in a constrained-random flow: a corner with probability 10^{-6} under the constraints will not be reached by adding machines. It is reached by changing the constraints for one test, or by writing that test by hand — and either way, by someone deciding it matters, which is a design decision and not a verification one.

Worked example 13. Should we add this cross?

Given. A proposal to cross 'transfer size' (8 bins) with 'address alignment' (4 bins) with 'outstanding transactions' (16 bins).

Method. $K = 8 \times 4 \times 16 = 512$. Under uniform random stimulus that is about 3,490.1 tests to fill. But check the legality first: a 1-byte transfer cannot be 8-byte-misaligned, so a good fraction of the 512 bins are unreachable by construction.

Answer. Add it, with the illegal combinations declared `illegal_bins` rather than left empty. The reachable count is what closure is measured against; leaving 512 as the denominator guarantees the report never reaches 100 % and trains everyone to ignore it.

Declaring illegal bins has a second benefit that is worth more than the first: an `illegal_bins` that *fires* is an immediate error. It turns a statement about what cannot happen into a check that it did not.

This is where it goes wrong. The trap is the opposite of the obvious one. The danger is not that the cross is too expensive — it is that an un-pruned cross produces a coverage number that can never reach its target, and a target that is known to be unreachable stops being a target. **An honest 100 % over reachable bins is worth more than an honest 87 % over a denominator nobody trusts.**

T21.4 Knowing when to stop

Coverage closure is a necessary condition for stopping, not a sufficient one. The question a project actually has to answer is different: *how many bugs are still in there?* That cannot be measured directly, but the rate at which bugs are being found can be, and a decaying rate carries information.

Derivation — Estimating the bugs you have not found yet

	This follows	Because
1	Suppose the number of new bugs found per week falls by a constant factor q .	An empirical observation in many projects, and the assumption this estimate rests on. It is not a law.
2	Then the bugs still to come are $n \cdot q + n \cdot q^2 + \dots = n \cdot q / (1 - q)$, where n is this week's count.	Geometric series, the same one as T17's ECO convergence — and it fails the same way when q approaches 1.
3	With $n = 4$ found this week and $q = 0.7$, that is 9.33 more.	Substitution. The number is small enough to be encouraging and uncertain enough to be dangerous, which is why the next step is the important one.

	This follows	Because
4	Now kill the trivial explanation. The rate can fall because bugs are running out, or because effort fell, or because the tests stopped changing.	Three causes, one observable. Distinguishing them is not optional — two of the three mean the estimate is meaningless.
5	Measure effort and new-stimulus rate alongside the bug count.	Constant simulation hours and rising coverage with a falling bug rate is evidence. A falling bug rate with a falling regression count is a measurement of the team's schedule , not of the design.

Table 97. Four bugs this week: what the decay rate implies

q (week-on-week ratio)	Expected bugs remaining	Reading
0.5	4	converging
0.7	9.333	converging
0.85	22.67	slow
0.95	76	no information — do not ship on this

The last row is the honest one. At $q = 0.95$ the estimate is 76 more bugs, and the model's error bars are far wider than that number — the geometric assumption is doing all the work. **An estimate whose value is dominated by an unverified assumption should be reported as an assumption, not as an estimate**, which is the same rule this book applies to every number in it.

T21.5 What to do with this on Monday

Table 98. Where this chapter's concepts are actually used

Concept	Block it appears in	What it decides
verification environment · testbench · agent · sequence · sequencer · driver · monitor	Building an environment	Whether it can be reused at the next level up
scoreboard · reference model · assertion	Every environment, first	The only components that can say the design is wrong
functional coverage · covergroup · coverpoint · cross coverage	Planning the coverage model	What the stimulus reached — and nothing more
constrained-random stimulus · directed test · coverage closure · regression	Deciding where to spend simulation time	Which bins random will never reach, so you write them by hand
virtual interface	Connecting the environment to the DUT	The one construct that has no analogue in ordinary software

The numbers to carry: filling K bins with uniform random stimulus costs about $K \cdot \ln K$ tests, so a cross of two 100-bin points costs 97,876.1 rather than 518.7 — crosses multiply the closure effort, and pruning illegal combinations is therefore engineering rather than tidiness. And a bin with probability 10^{-6} takes 1.00000×10^6 tests to reach once, which is the arithmetic that keeps directed tests alive.

Verification establishes that the design is right. It says nothing about whether a particular piece of silicon was manufactured correctly — that question is answered by a machine with the die in it, and answering it cheaply is the subject of the last chapter.

Where the inventions are — what is patentable here

Verification methodology is published rather than patented — the methodology libraries are open source, which is the point of them. What is filed is around **coverage-directed test generation**: using the coverage result to steer the constraint solver towards the bins that are still empty, which is a direct attack on the coupon-collector arithmetic computed in this chapter.

T22. From DFT to the Tester — Time, Cost and What Escapes

Test does not prove a die is good. It bounds how often a bad one ships, and the bound is computable.

Before you read this

implementation flow · verification environment · scoreboard · signoff · tape-out · static timing analysis · clock gating · dynamic IR analysis · switching vector · yield sigma · pad capacitance · multi-mode multi-corner analysis · corner · burn-in · Weibull

New in this chapter

scan · scan chain · ATPG · stuck-at fault · transition (at-speed) fault · fault coverage · scan compression · decompressor · compactor · logic BIST · memory BIST · ATE (automatic test equipment) · pin electronics · pattern memory · test program · test time · multi-site test · die yield · defect level · DPPM (defective parts per million) · wafer sort · known-good die

A verified design (T21) manufactured through a signed-off flow (T20) still produces dies that do not work, because manufacturing is a physical process with defects. Test is the operation that separates them, and it is the only stage of the whole flow whose cost is paid **per part** rather than once. That single fact determines everything about how it is engineered.

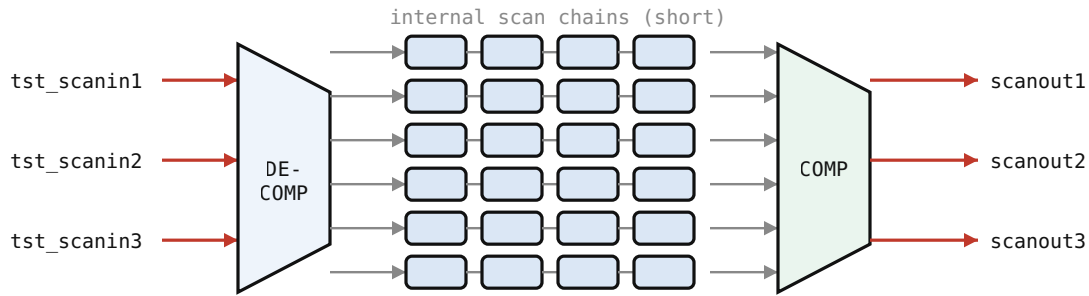
T22.1 Scan, and the price of observability

scan (스캔) · 스캔 체인 (scan chain) — Every flop is replaced by one with a multiplexer on its input, selecting between the functional D and a scan input. In test mode the flops are wired into shift registers — **chains**. This makes every flop both controllable (shift a value in) and observable (shift the result out), which turns the intractable problem of testing sequential logic into the tractable one of testing combinational logic between two register boundaries.

Table 99. What scan costs, and what it buys

Cost	Size	What is bought
Area	The mux in every flop: roughly 5–10 % of the design	Every flop becomes a test point
Timing	The mux is in the D path — a real delay on every register input	Nothing, directly. This is pure cost, and it is why the scan mux is one of the most carefully optimised cells in any library
Power	Shift toggles far more nodes than functional operation (T20.3)	Nothing. It constrains the shift frequency, which costs test time
Routing	The chains are long nets crossing the die	Nothing, and chain ordering after placement is how the cost is contained

Scan compression: a few tester channels, many internal chains



Shift length = flops / (chains × compression). The tester sees 3 channels; the die has 6 chains.
 Test time AND pattern memory both fall in the same proportion – and memory is often the binding o
 lab: `python3 lab/run.py --stage dft` (measures both on the real netlist)

Figure 33. Compression in one picture: a decompressor fans a few tester channels out to many short internal chains, and a compressor squeezes the responses back. Both the test time and the pattern volume fall by the same factor.

Table 100. Fault models, and the defects each one is a proxy for

Model	What it assumes	What it misses
Stuck-at	A node is permanently 0 or 1	Anything timing-dependent. It is a topological model and catches gross defects well
Transition / at-speed	A node changes too slowly to make the next capture	Requires two patterns at functional speed — expensive, and constrained by the power the part can draw while doing it
Bridging	Two nodes shorted	Which two — the realistic list comes from layout extraction, not from the netlist
IDDQ	Quiescent current is elevated when a defect conducts	Nothing subtle, but it stopped working at leaky nodes: the leakage of a good die became comparable to the defect signal

T22.2 Test time is the design decision

Derivation — How long one die takes on the tester

	This follows	Because
1	The block of T19 has 20,000 flops.	Carried across so this chapter is about the same chip as the last four.
2	With one chain, applying a pattern means shifting 20,000 cycles.	The chain is a shift register of that length; every pattern must be pushed all the way in and the result all the way out.
3	ATPG produces about 5,000 patterns for a block of this size.	Assumed , and it varies more with logic structure than with size.
4	At a 50 MHz shift clock that is 2 s per die.	Multiplication. The shift clock is slow because of the power problem of T20.3, not because the flops cannot run faster.
5	Split into N chains and the shift length divides by N.	Chains shift in parallel; the tester drives N inputs and watches N outputs. The limit is tester channels , not the design.
6	With 100 chains: 0.02 s. With 100 chains and 50× compression: 8.000×10^{-4} s.	Compression puts a decompressor on-chip that fans a few tester channels out to many internal chains, so the tester-visible chain count rises without using more channels.

Table 101. Test time, pattern volume and cost per die (tester at 2 M\$, 5 years, 70 % utilisation)

Design	Chains	Compression	Shift length	Time per die	Pattern memory	Tester cost per die
block (20k flops)	1	1×	20,	2 s	25 MB	0.036 \$
block (20k flops)	10	1×	2,000	0.2 s	25 MB	0.004 \$
block (20k flops)	100	1×	200	0.02 s	25 MB	3.696×10 ⁻⁴ \$
SoC (2M flops)	1	1×	2.0000×10 ⁶	1,6 s	20, MB	28.99 \$
SoC (2M flops)	100	1×	20,	16 s	20, MB	0.29 \$
SoC (2M flops)	100	50×	400	0.323 s	400 MB	0.006 \$
SoC (2M flops)	100	200×	100	0.083 s	100 MB	0.002 \$

The block does not need compression; the SoC cannot do without it. That contrast is the point of putting both in one table, and it is a useful corrective: at 20,000 flops, splitting into 100 chains already brings the test to 0.02 s and compression on top of that buys a number too small to care about. At 2,000,000 flops the same single-chain test is 1,6 s — 26.7 minutes per die, which is not a test plan, it is a reason the product does not exist — and 100 chains with 50× compression brings it to 0.323 s, or 0.006 \$ of tester time per part.

The **pattern memory** column deserves separate attention, because it is often the binding constraint rather than time. A tester has a finite pattern memory, and a pattern set that does not fit must be split into several loads, each costing handler and reload time on top of test time. A design that fits in memory at 50× compression and does not at 10× has a test cost discontinuity that no amount of shift-frequency tuning will smooth out.

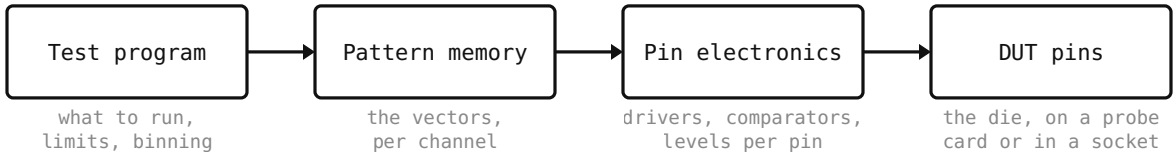
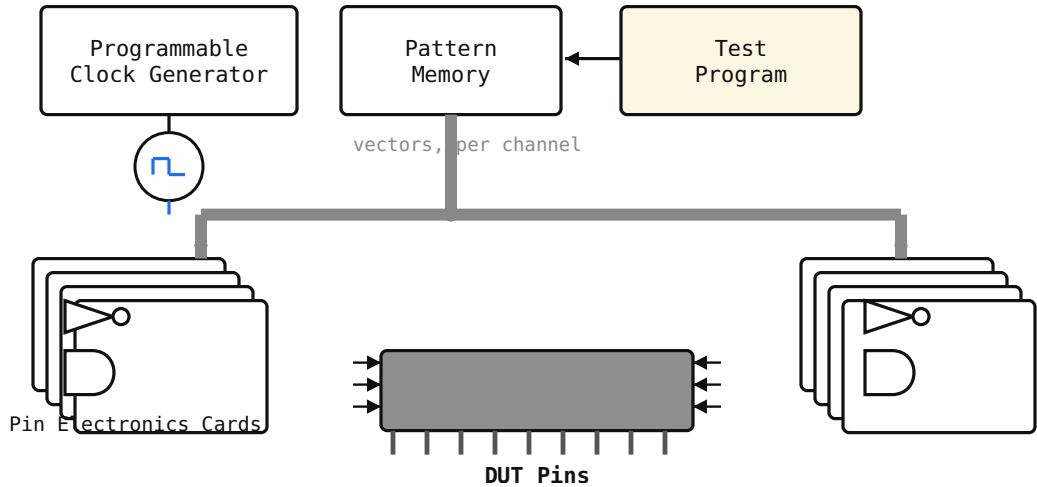


Figure 34. What an ATE is, functionally: a very precise, very expensive pattern player with a programmable timing generator and one driver/comparator pair per pin. Everything in the previous table is a statement about one of these boxes.



Cost per die = (test time) × (tester \$/second ÷ sites). Everything DFT does is an attack on or

Figure 35. The machine the last column of that table is renting. The pattern memory holds the vectors, the pin electronics drive and compare one pin each, and the test program decides what runs and what counts as a pass.

multi-site test (병렬 시험) — the other way to divide the cost

Compression reduces the time one die takes. Multi-site test reduces the cost of that time by testing several dies at once on the same tester, using separate channels for each. The cost per die divides by the site count almost exactly, because the tester's cost per second is fixed and the handler's overhead is shared.

The limit is channels and power. A 4-site setup needs four times the channels, and the pin count per die therefore decides how many sites fit — which is a DFT decision made years earlier, when someone chose how many scan channels to bring out.

For the block above: 1.450×10^{-5} \$ at one site becomes 3.624×10^{-6} \$ at four, for the same test content and the same quality.

Where it is used	Production test planning, and — decisively — in the pin budget at floorplan time.
When it binds	Considered as soon as the package pin list is discussed. Adding scan channels later is a mask change.
How to apply it	Count: (tester channels) ÷ (channels per die) = sites. Then check the power: the tester's DUT power supply must source the shift current of all sites at once, and that is where 4-site setups actually fail.

This is where it goes wrong. Multi-site amplifies a marginal test. If a site has slightly different contact resistance or a longer path from the driver, one site will fail parts the others pass — and the yield report will show it as a process problem. **Always check yield per site before investigating a yield excursion.**

T22.3 BIST: moving the tester onto the die

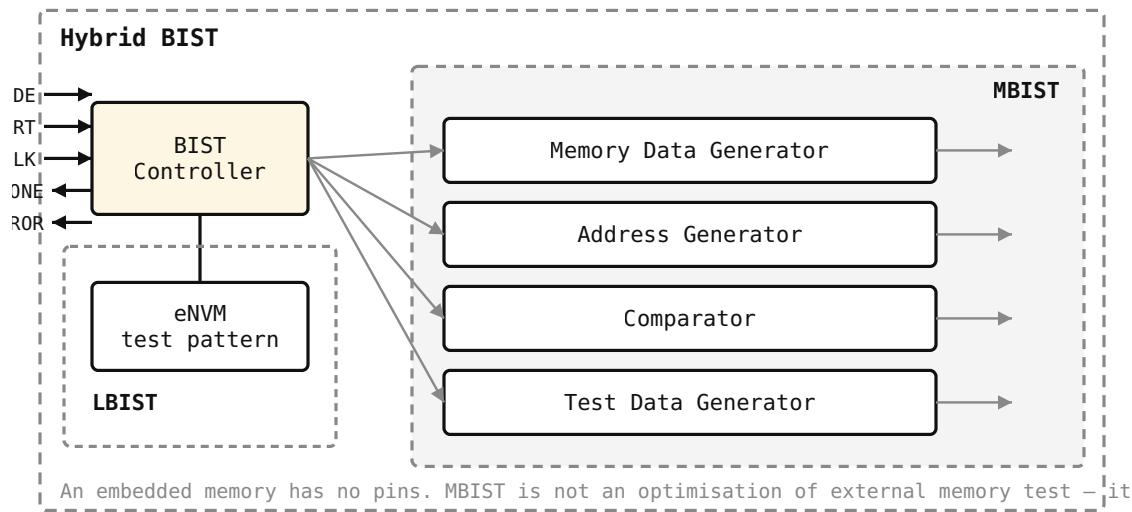


Figure 36. BIST moves the pattern source onto the die. The memory side replaces data with an algorithm; the logic side replaces it with an LFSR and a signature. The controller decides which of the two is running.

Table 102. Where BIST wins and where it does not

Kind	What it tests	Why it is better than external patterns there
MBIST (memory)	Embedded RAMs, with algorithmic patterns (March tests)	Memories are regular, so the pattern is an <i>algorithm</i> , not data — a small state machine replaces megabytes of vectors. And embedded memory pins are not accessible from outside at all
LBIST (logic)	Random logic, with an on-chip LFSR feeding the chains and a MISR compacting the results	It runs at functional speed without a fast tester, and it can be re-run in the field. Its coverage is lower than ATPG's for the same time, because the patterns are random rather than targeted

Kind	What it tests	Why it is better than external patterns there
Hybrid	LBIST for the bulk, top-up ATPG patterns for what random misses	The combination reaches ATPG-like coverage with a fraction of the pattern volume — the two are complementary rather than alternatives

The interesting property of MBIST is not that it saves test time. It is that **an embedded memory has no pins**: there is no way to apply a pattern to it from outside except through the surrounding logic, which is slow, low-coverage and enormously expensive in patterns. MBIST is not an optimisation of external memory test; it is the only practical way to do it at all. And once the hardware is there it can also repair — blowing fuses to swap in redundant rows — which turns a failing die into a good one and is worth more than the test time it saves.

T22.4 What test is actually worth: yield, coverage and escapes

Derivation — Die yield, and why test coverage has a price

	This follows	Because
1	The core of T18 is 1.901 mm ² ; the SoC used above is 100 mm ² .	Both carried over. The block's figure ignores the pad ring, so it is an upper bound; the SoC figure is the one the arithmetic below uses, because yield is an area story and a small block has nothing interesting to say about it.
2	With a defect density of 0.1 /cm ² , $Y = \exp(-A \cdot D)$ is 99.81 % for the block and 90.48 % for the SoC.	Assumed defect density, and the Poisson model itself is the optimistic one — real defects cluster, which raises yield above this for the same average density.
3	Of the dies that fail, test catches a fraction T — the fault coverage.	Coverage is measured by ATPG against a fault list, which is itself a model (T22.1). It is not the fraction of <i>defects</i> caught, and treating it as such is the standard error here.
4	A die ships bad if it has at least one defect and every defect is missed.	The event that matters. It is the intersection of two things, which is why the answer is not simply 1–T.
5	Williams and Brown: the defect level among shipped parts is DL = 1 – Y^(1–T) .	The classic result. Note its shape: at T = 1 it is zero, and at T = 0 it is 1 – Y, the raw fail rate — both limits are right, which is the first check any formula should pass.

Table 103. Shipped defect level at the SoC's Y = 90.48 %, two independent ways

Fault coverage T	Williams–Brown	Simulated wafer of 200,000 dies	Disagreement
90 %	9,950.2 DPPM	10,107 DPPM	1.58 %
95 %	4,987.5 DPPM	5,087.5 DPPM	2.01 %
99 %	999.5 DPPM	1,143 DPPM	14.4 %
99.9 %	99.995 DPPM	66.332 DPPM	33.7 %

Read the disagreement column honestly. The simulation builds dies one at a time, scatters Poisson defects into them, lets each defect be caught with probability T, and counts how many defective dies got through — using none of the formula. The agreement is good at high defect levels and gets worse at low ones, and that is *expected*: at the last row only a few dozen escapes occur in 200,000 trials, so the Monte-Carlo sampling error is tens of per cent while the closed form has none. A disagreement that grows as the event gets rarer is the signature of sampling noise, not of a wrong formula — and distinguishing those two is exactly the check that makes an independent control worth running.

Worked example 14. Is 99 % coverage good enough?

Given. The SoC above, Y = 90.48 %, shipping into an automotive customer who specifies under 100 DPPM.

Method. At T = 99 % the defect level is 999.5 DPPM. At T = 99.9 % it is 100 DPPM. Solve for the coverage that reaches 100 DPPM: $T = 1 - \ln(1 - DL) / \ln Y$.

Answer. T must be at least 99.9 %. Every tenth of a per cent of coverage above 99 % costs patterns, and patterns cost test time and pattern memory — the table in T22.2.

And the model's own limits belong in the answer: fault coverage is coverage of a *fault model*, not of defects. A defect class the model does not describe — a resistive via that only fails hot, a marginal timing path — contributes to DPPM without appearing anywhere in T. That is what burn-in (T14) and at-speed patterns are for, and why an automotive flow uses both rather than pushing stuck-at coverage to 99.99 %.

This is where it goes wrong. Coverage is not the only lever, and it is the expensive one. Raising yield moves the same number: at Y = 95 % rather than 90.5 %, the same T = 99 % gives 512.8 DPPM instead of 999.5 DPPM. A defect-density improvement from the fab is worth more than the last per cent of fault coverage, and it does not cost test time.

What actually happened

A part met its DPPM target at wafer sort and failed it at the customer. The escapes were all marginal at-speed failures on one clock domain: the at-speed patterns existed, but shift power forced the capture clock down, and the 'at-speed' test was running 30 % below functional frequency. **The coverage number was real and the test it described was not the test that ran.** Test-time and power constraints change what the patterns mean, and the coverage report does not know that.

T22.5 What to do with this on Monday

Table 104. Where this chapter's concepts are actually used

Concept	Block it appears in	What it decides
scan · scan chain · ATPG · stuck-at fault · transition (at-speed) fault · fault coverage	DFT planning, before RTL freeze	Whether the part is testable at all, and at what coverage
scan compression · decompressor · compactor · test time · pattern memory	Choosing chain count and compression ratio	The per-part test cost, which is paid on every part ever shipped
ATE (automatic test equipment) · pin electronics · test program · multi-site test	Production planning, and in the pin budget years earlier	How many dies share one tester second
logic BIST · memory BIST	Any embedded memory; any part that self-tests in the field	The only practical way to test a block with no pins
die yield · defect level · DPPM (defective parts per million) · wafer sort · known-good die	Agreeing a quality target with a customer	What coverage that target actually requires — and whether yield is the cheaper lever

The numbers to carry: on the 2 M-flop SoC, one chain is 26.67 minutes per die and 100 chains with 50× compression is 0.323 s — a factor of 4,951 × in a cost that is paid on every part ever shipped. And at 90.48 % yield, 99 % fault coverage ships 999.5 DPPM — which is the sentence that connects a DFT decision made before RTL freeze to a number in a customer's quality specification.

That closes the implementation arc. It began with a netlist and a set of constraints (T17), drew a box and a power grid (T18), placed, clocked and routed what was inside it (T19), established that the geometry may be manufactured (T20), established that the design was right in the first place (T21), and ended with the only question that is asked of every individual part ever shipped: does this one work? Every stage was governed by a computed quantity rather than a convention, and in every stage the quantity that mattered was the one that could not be seen from inside that stage alone.

Where the inventions are — what is patentable here

Test compression is one of the most heavily patented areas in EDA — the decompressor architectures (ring generators, broadcast networks, sequential linear expanders) are the product, and the ratios they achieve are the competitive claim. So is **low-power ATPG**, which constrains patterns to limit simultaneous switching for the reason computed in T20.3. Both exist to move the numbers this chapter calculates.

T23. The Flow, Run End to End — One Block, Nine Stages, Real Numbers

Every number in this chapter was produced by running the flow in this repository. None of them is quoted.

Before you read this

implementation flow · logic synthesis · netlist · static timing analysis · slack · standard-cell row · SITE · utilisation · core area · power stripe · placement · legalisation · HPWL (wirelength) · clock-tree synthesis · insertion delay · clock skew · global routing · detail routing · search and repair · congestion · signoff · physical verification · corner · scan · scan chain · ATPG · stuck-at fault · fault coverage · defect level · DPPM (defective parts per million) · test time · scan compression · verification environment · scoreboard · functional coverage · cross coverage · constrained-random stimulus · directed test · electromigration · IR drop

New in this chapter

the lab flow · regression baseline

The five chapters before this one computed the flow's quantities from models. This one runs it. The design is a small MAC with a control FSM, a two-flop synchroniser and a gated accumulator; it is synthesised by **yosys** against a cell library generated from the RC model of T2, and the remaining eight stages are implemented in this repository because the machine has no commercial place-and-route tool. That constraint turned out to be an advantage: **every intermediate quantity is visible and re-measurable**, which is exactly what a commercial tool does not give you.

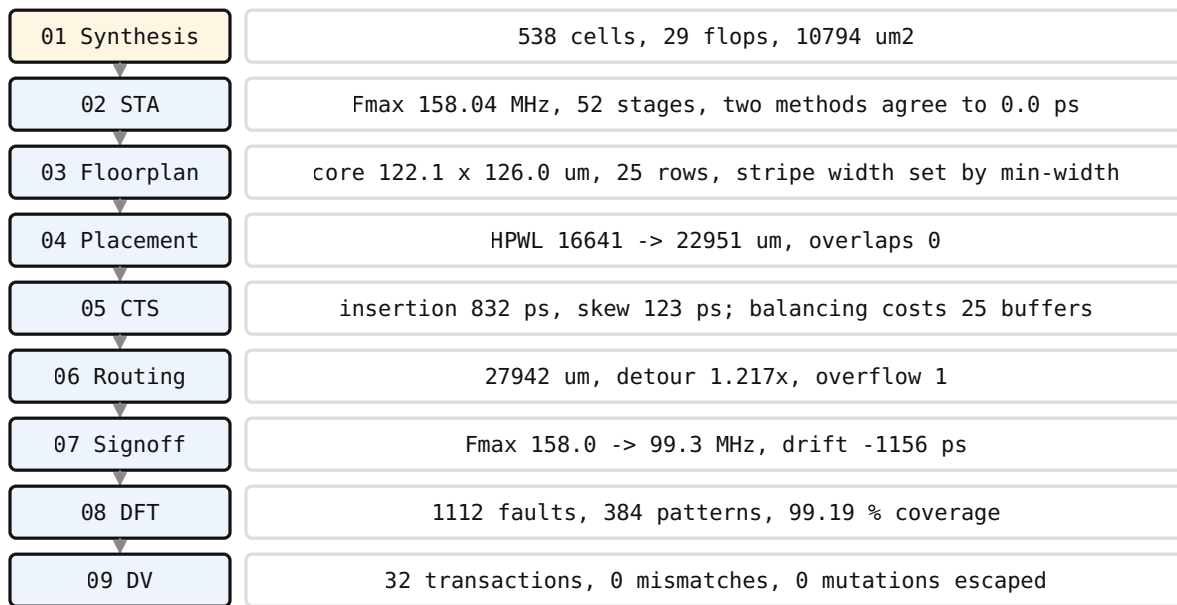
Reproduce everything in this chapter with one command: `python3 lab/run.py`. It takes about eight seconds. Individual stages: `python3 lab/run.py --stage sta` (or `fp place cts route signoff dft dv`). The numbers below are read from `lab/기준.json`, which that command writes, and `tests/test_lab.py` re-runs the flow and checks they still agree — so **if this chapter goes stale, a test goes red**.

T23.1 The nine stages, and what each one measured

Table 105. One run, end to end

Stage	What it did	The number that matters
01 Synthesis (yosys)	RTL → 538 cells, 29 flops	10,794.2 μm^2 of cell area — the input to every area decision downstream
02 STA (pre-route)	52-stage critical path	$F_{\text{max}} = 158.04$ MHz, and the two independent methods agree to 0 ps
03 Floorplan	25 rows × 185 sites at 70 % utilisation	core 122.1 μm × 126 μm ; the stripe width is set by min-width
04 Placement	force-directed, then two-sweep legalisation	HPWL 16,640.8 μm → 22,951 μm , overlaps 0
05 CTS	2-level tree, 5 buffers	insertion 832.14 ps, skew 122.7 ps
06 Routing	global on a 21×21 gcell grid, then rip-up and reroute	27,942 μm of wire — detour factor 1.217 × over HPWL
07 Signoff	re-extract, re-run STA with real RC	F_{max} 158.04 MHz → 99.31 MHz
08 DFT	1112 stuck-at faults, parallel-pattern fault simulation	99.19 % coverage with 384 random patterns
09 DV	32 transactions through a UVM-shaped environment	0 mismatches; 5 mutations, 0 escaped

python3 lab/run.py – nine stages, about eight seconds



Yellow is the one stage that calls an outside tool (yosys). The other eight are in this repository, which is why every intermediate quantity above can be re-measured – and is, by tests/test_lab.py.

Figure 37. The run this chapter reports, stage by stage. Reproduce it with one command; every box is a number a test re-measures.

T23.2 The four results worth arguing with

T23.2 The four results worth arguing with

Derivation — 1. Pre-route timing is not optimistic by a little

	This follows	Because
1	Synthesis and pre-route STA report 158.04 MHz with 3,672.4 ps of slack.	No wire RC at all — the optimistic bound of T17.2.
2	After routing, the same endpoints report 99.31 MHz and -69.14 ps.	Same netlist, same library, same constraints. The only new information is where the wires actually went.
3	That is 37.2 % of $F_{\max}$, and the average endpoint moved -1,155.5 ps.	Average , not worst. Every endpoint got worse, because every net acquired resistance and capacitance it did not have.
4	The design now fails : 1 endpoint(s) violate at the target period.	Which is the honest outcome of running a flow without a physically-aware synthesis step — and the quantitative argument for having one.

Kill the trivial explanation. A 37.2 % drop could just mean the wire model is too pessimistic. Two things say otherwise. The detour factor is $1.217 \times$ over HPWL, which is in the range real routers produce; and the wire delay comes from the same R and C per micron used to size the power grid, where the resulting IR drop came out sane. Neither is proof. What would be proof is a second extraction method, and this lab does not have one — **so the number is reported with that limitation attached.**

Derivation — 2. Skew is bought, not given

	This follows	Because
1	The tree as built by geometry alone has 122.7 ps of skew on 832.14 ps of insertion delay — 14.74 %.	Clustering by position minimises wire, not arrival time. Nothing in the construction balances anything.
2	Balancing it to about 2 % costs 25 extra buffers.	Delay is added to the early branches until they match the late one. That is what a CTS tool does after it builds the tree.
3	Those buffers raise the clock network's power by 21.09 %.	They switch every cycle, like everything else on the clock. The trade is skew against power and it is quantitative.
4	Result: 19.6 ps, i.e. 2.35 % of insertion delay.	Which is the figure the industry quotes, and now it has a price tag attached.

Table 106. 3. The repair loop does not converge — measured

Pass	Violations open	Ratio to the previous pass
0	2	—
1	4	2
2	15	3.75
3	16	1.067
4	13	0.812
5	7	0.538
6	10	1.429
7	7	0.7
8	9	1.286

The sequence is [2, 4, 15, 16, 13, 7, 10, 7, 9] and the ratios rise above 1 repeatedly, so the loop is **moving** violations rather than repairing them — the exact signature T19.4 describes. Two consequences were built into the lab because of it. The router keeps the best solution it has seen rather than the last one (best 2, last 9), which is what a real router does and what turns "more iterations make it worse" from a conclusion into an artefact. And the correct response to a non-converging loop is not another pass: it is to go back up the flow — lower the utilisation, spread the placement, add a layer.

Table 107. 4. The mutation campaign found a hole in the checking, not in the stimulus

Mutation	What it breaks	Caught by
m1_cnt	RUN one cycle shorter — one accumulate is lost	acc compare
m2_load	LOAD does not load the product	acc compare
m3_pulse	level instead of a pulse — start re-triggers	done count
m4_done	done is never asserted	no done
m5_prod	sum instead of product	acc compare

What actually happened

The m3_pulse mutation — replacing the one-cycle pulse with a level — was **not caught** by the first regression, and it was not caught after adding a directed test that holds start high for twenty cycles either. The reason is instructive: with a level, the FSM re-triggers as soon as it returns to idle, and the *first* completion's accumulator value is still correct. A value comparison cannot see it. **The hole was in the checking, not in the stimulus**, and it closed only when the scoreboard began asserting that each transaction produces exactly one done. That single protocol check is what takes the campaign from one escape to 0.

T23.3 What this lab does not do

Table 108. Honest limits — read these before quoting any number above

Not done	Why it matters	What would fix it
No commercial P&R	The placer, router and CTS here are teaching implementations. Their absolute quality is not comparable to a production tool	OpenROAD would run this same netlist; the flow is deliberately structured so it could be swapped in
One extraction model	Wire R and C come from a single per-micron constant. The 37.2 % $F_{\max}$ drop therefore has no independent control	A second extraction (field solver, or a different model) and a comparison of the two
No DRC, no LVS	Routing produces track counts, not geometry, so nothing here can assert manufacturability (T20.1)	A real layout database and a rule deck — neither of which exists on this machine
Stuck-at only	The 99.19 % coverage is stuck-at coverage. Transition and bridging faults are not simulated	At-speed patterns and a layout-extracted bridging list (T22.1)
Random ATPG only	9 faults remain and random patterns will not reach them — this is T21.3's rare-bin problem in its test form	Deterministic ATPG (a D-algorithm or a SAT call) for the residue
Power is assumed, not measured	The floorplan's current comes from an assumed W/mm ² , so the IR and EM conclusions inherit that assumption	Toggle rates from the DV run, fed back into the power estimate — the two stages are in the same repository and are not yet connected

That last row is worth dwelling on, because it is the kind of gap that survives for years. The verification run knows exactly how often every node toggles. The floorplan needs exactly that number. They are in the same program, in the same language, eight seconds apart — and they are not connected, so the power number is an assumption while the information that would replace it is sitting in the next directory. **Most of what looks like missing analysis in a real flow is this: two stages that have what the other needs and no path between them.**

T23.4 What to do with this on Monday

Table 109. Where this chapter's concepts are actually used

Concept	Block it appears in	What it decides
the lab flow	Learning the flow, or testing a change to it	Nine stages you can read, break and re-measure in eight seconds
regression baseline	Every change to lab/ or to this chapter	Whether the numbers in the book still match a real run

Run it, then break something on purpose: raise the utilisation in `lab/se/floorplan.py` until legalisation fails; weaken the spreading force until legalisation destroys the placement; delete the done-count check in `lab/se/dv.py` and watch a mutation escape. **Each of those is a lesson from an earlier chapter arriving as a failure you caused**, which is the only way this material stops being a list of facts. The one command is `python3 lab/run.py`; the one file that holds the flow honest is `tests/test_lab.py`.

Where the inventions are — what is patentable here

Nothing here is patentable and that is the point: the flow is commodity, the value is in knowing what each number means and which of them is lying to you.